SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola

Depósito de Código Fuente Completo ante la Dirección Nacional de Derecho de Autor (DNDA)

Identificación de la Obra: SIFMA (Soporte Lógico)
Versión: 1.0.0
Año de Creación: 2026
País de Origen: Colombia

Autores (Desarrolladores del Código):
• Andrés Felipe Luna Camargo
• Cristian David Peña Martínez

Archivos Propios Compilados: 85 archivos

Total de Líneas de Código: 24218 líneas

ARCHIVO 1/85: instalacion/1_instalar_dependencias/Instalar_Dependencias.bat (65 líneas)

```
@echo off
:: SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
:: Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
:: Año: 2026.

chcp 65001 > nul
title SIFMA - Paso 1: Instalador de Dependencias

echo ============================================================
echo        SIFMA - SISTEMA DE FENOTIPADO Y TELEMETRIA
echo        Paso 1: Instalacion Automatizada de Dependencias
echo ============================================================
echo.

:: 1. Verificar si Python esta instalado
python --version >nul 2>&1
if %errorlevel% neq 0 (
    echo [ERROR] Python no esta instalado o no se encuentra en la variable PATH.
    echo.
    echo Por favor descargue e instale Python (version 3.10, 3.11, 3.12 o 3.13) desde:
    echo   https://www.python.org/downloads/
    echo.
    echo IMPORTANTE: Durante la instalacion, marque obligatoriamente la casilla:
    echo   "Add python.exe to PATH"
    echo.
    pause
    exit /b 1
)

echo [1/4] Verificando version de Python detectada:
python --version
echo.

set "ROOT_DIR=%~dp0..\.."
set "VENV_DIR=%ROOT_DIR%\.venv"
set "REQ_FILE=%ROOT_DIR%\local_server\requirements.txt"

echo [2/4] Verificando / creando entorno virtual (.venv)...
if not exist "%VENV_DIR%" (
    echo Creando entorno virtual en "%VENV_DIR%"...
    python -m venv "%VENV_DIR%"
) else (
    echo Entorno virtual detectado en "%VENV_DIR%".
)

echo.
echo [3/4] Activando entorno virtual y actualizando pip...
call "%VENV_DIR%\Scripts\activate.bat"
python -m pip install --upgrade pip

echo.
echo [4/4] Instalando dependencias verificadas desde requirements.txt...
python -m pip install -r "%REQ_FILE%"
```

```
54
55 echo.
56 echo ========================================================
57 echo   INSTALACION DE DEPENDENCIAS COMPLETADA EXITOSAMENTE
58 echo ========================================================
59 echo.
60 echo Siguiente paso:
61 echo   - Ejecutar: instalacion\2_iniciar_servidor\Iniciar_Servidor.bat
62 echo   - O (opcional): instalacion\3_crear_acceso_directo\Crear_Acceso_Directo.bat
63 echo.
64 pause
65
```

ARCHIVO 2/85: instalacion/2_iniciar_servidor/Iniciar_Servidor.bat (37 líneas)

```
@echo off
:: SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
:: Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
:: Año: 2026.

chcp 65001 > nul
title SIFMA - Servidor Local

set "ROOT_DIR=%~dp0..\.."
set "VENV_DIR=%ROOT_DIR%\.venv"
set "SERVER_DIR=%ROOT_DIR%\local_server"

:: Abrir automaticamente el navegador tras 2.5 segundos
start "" cmd /c "timeout /t 3 /nobreak >nul & start http://127.0.0.1:5000"

:: Activar entorno virtual
if exist "%VENV_DIR%\Scripts\activate.bat" (
    call "%VENV_DIR%\Scripts\activate.bat"
) else (
    echo [ADVERTENCIA] No se encontro entorno virtual en "%VENV_DIR%".
    echo Ejecutando con el Python del sistema...
)

echo ============================================================
echo        SIFMA - SERVIDOR LOCAL EN PRODUCCION
echo ============================================================
echo.
echo Servidor activo en: http://127.0.0.1:5000
echo La ventana se mantendra abierta para mantener el servicio activo.
echo Presione Ctrl+C para detener el servidor.
echo.

cd /d "%SERVER_DIR%"
python app.py

pause

```

ARCHIVO 3/85: instalacion/3_crear_acceso_directo/Crear_Acceso_Directo.bat (13 líneas)

```
@echo off
:: SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
:: Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
:: Año: 2026.

chcp 65001 > nul
title SIFMA - Paso 3: Crear Acceso Directo

echo Creando acceso directo oficial de SIFMA en el Escritorio...
powershell -ExecutionPolicy Bypass -File "%~dp0Crear_Acceso_Directo.ps1"

pause

```

ARCHIVO 4/85: instalacion/3_crear_acceso_directo/Crear_Acceso_Directo.ps1 (36 líneas)

```
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

$scriptDir = $PSScriptRoot
if (-not $scriptDir) {
    $scriptDir = Split-Path -Parent $MyInvocation.MyCommand.Path
}

$rootDir = (Resolve-Path (Join-Path $scriptDir "..\..")).Path
$desktop = [Environment]::GetFolderPath("Desktop")
$shortcutPath = Join-Path $desktop "SIFMA - Servidor Local.lnk"

$targetBat = (Resolve-Path (Join-Path $rootDir "instalacion\2_iniciar_servidor\Iniciar_Servidor.bat")).Path
$workDir = (Resolve-Path (Join-Path $rootDir "local_server")).Path
$iconPath = (Resolve-Path (Join-Path $rootDir "local_server\static\img\sifma_icon.ico")).Path

$wscript = New-Object -ComObject WScript.Shell
$shortcut = $wscript.CreateShortcut($shortcutPath)
$shortcut.TargetPath = $targetBat
$shortcut.WorkingDirectory = $workDir
if (Test-Path $iconPath) {
    $shortcut.IconLocation = "$iconPath,0"
}
$shortcut.Description = "Sistema Integral de Fenotipado y Monitoreo Agricola (SIFMA) - Servidor Local"
$shortcut.Save()

Write-Host "============================================================" -ForegroundColor Green
Write-Host "  ACCESO DIRECTO DE SIFMA CREADO EN EL ESCRITORIO" -ForegroundColor Cyan
Write-Host "============================================================" -ForegroundColor Green
Write-Host "Ruta del acceso directo: $shortcutPath"
Write-Host "Destino: $targetBat"
Write-Host "Directorio de trabajo: $workDir"
Write-Host "Icono asignado: $iconPath"
Write-Host ""

```

ARCHIVO 5/85: local_server/app.py (77 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import sys
from flask import Flask

SERVER_DIR = os.path.dirname(os.path.abspath(__file__))
if SERVER_DIR not in sys.path:
    sys.path.insert(0, SERVER_DIR)

from core.dependencies import ensure_dependencies
ensure_dependencies()

from core.config import config
from core.logger import logger
from infrastructure.database import db, init_db_data
from presentation.web import dashboard_bp
from presentation.api import api_bp

def create_app():
    """
    Fábrica de inicialización de la aplicación Flask (Application Factory Pattern).
    Configura extensiones, directorios seguros y blueprints de presentación.
    """
    app = Flask(
        __name__,
        static_folder=config.STATIC_DIR,
        template_folder=os.path.join(SERVER_DIR, "templates")
    )

    app.config['SQLALCHEMY_DATABASE_URI'] = config.SQLALCHEMY_DATABASE_URI
    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = config.SQLALCHEMY_TRACK_MODIFICATIONS
    app.config['SECRET_KEY'] = config.SECRET_KEY

    os.makedirs(config.UPLOAD_DIR, exist_ok=True)
    os.makedirs(config.PROCESSED_DIR, exist_ok=True)
    os.makedirs(config.TELEMETRY_DIR, exist_ok=True)

    db.init_app(app)

    with app.app_context():
        from sqlalchemy import event
        @event.listens_for(db.engine, "connect")
        def set_sqlite_pragma(dbapi_connection, connection_record):
            cursor = dbapi_connection.cursor()
            cursor.execute("PRAGMA journal_mode=WAL")
            cursor.execute("PRAGMA synchronous=NORMAL")
            cursor.close()

    app.register_blueprint(dashboard_bp)
    app.register_blueprint(api_bp)
```

```
54
55      from flask import send_from_directory
56      @app.route('/favicon.ico')
57      def favicon():
58          return send_from_directory(
59              os.path.join(app.static_folder, 'img'),
60              'sifma_sin_texto.svg',
61              mimetype='image/svg+xml'
62          )
63
64      # Inicializar tablas, esquemas y Super Usuario por defecto
65      init_db_data(app)
66
67      return app
68
69  app = create_app()
70
71  if __name__ == '__main__':
72      logger.info("================================================")
73      logger.info("Servidor Local SIFMA - Arquitectura Refactorizada")
74      logger.info("Dashboard Web en: http://127.0.0.1:5000")
75      logger.info("================================================")
76
77      app.run(host='0.0.0.0', port=5000, debug=True)
```

ARCHIVO 6/85: local_server/application/__init__.py (21 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .services import (
    SystemService,
    TelemetryApplicationService,
    VisionApplicationService,
    AnalyticsApplicationService,
    ConclusionApplicationService,
    UserService
)

__all__ = [
    "SystemService",
    "TelemetryApplicationService",
    "VisionApplicationService",
    "AnalyticsApplicationService",
    "ConclusionApplicationService",
    "UserService"
]
```

ARCHIVO 7/85: local_server/application/services/__init__.py (23 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .system_service import SystemService
from .telemetry_service import TelemetryApplicationService
from .vision_service import VisionApplicationService
from .analytics_service import AnalyticsApplicationService
from .conclusion_service import ConclusionApplicationService
from .user_service import UserService
from .weather_service import WeatherService
from .audit_service import AuditService

__all__ = [
    "SystemService",
    "TelemetryApplicationService",
    "VisionApplicationService",
    "AnalyticsApplicationService",
    "ConclusionApplicationService",
    "UserService",
    "WeatherService",
    "AuditService"
]
```

ARCHIVO 8/85: local_server/application/services/analytics_service.py (1766 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import io
import csv
import calendar
import numpy as np
from datetime import datetime, timedelta
from typing import Dict, Any, List, Optional
from domain.models import CaptureSession, SensorReading, BiometricMetric
from infrastructure.database.connection import db
from infrastructure.database.repositories import (
    CaptureSessionRepository,
    SensorRepository,
    ConfigRepository
)
from infrastructure.telemetry import TowerCsvImporter

class AnalyticsApplicationService:
    """
    Caso de uso: Cruce de fenotipado con telemetría ambiental,
    análisis por jornada diaria, cálculo de correlaciones agronómicas y remuestreo temporal.
    (SOLID: SRP, DIP)
    """

    def __init__(
        self,
        session_repo: Optional[CaptureSessionRepository] = None,
        sensor_repo: Optional[SensorRepository] = None,
        config_repo: Optional[ConfigRepository] = None,
        csv_importer: Optional[TowerCsvImporter] = None
    ):
        self.session_repo = session_repo or CaptureSessionRepository()
        self.sensor_repo = sensor_repo or SensorRepository()
        self.config_repo = config_repo or ConfigRepository()
        self.csv_importer = csv_importer or TowerCsvImporter(self.sensor_repo, self.session_repo)

    def import_tower_csv(self, file_stream_or_path) -> Dict[str, Any]:
        return self.csv_importer.import_csv(file_stream_or_path)

    def get_available_dates(self, plant_id: int = 1) -> List[str]:
        """Obtiene todas las fechas con datos registrados para la canastilla indicada, incluyendo la fecha actual de hoy."""
        today = datetime.now().strftime("%Y-%m-%d")
        dates_set = {today}

        session_dates = self.session_repo.get_available_dates(plant_id)
        if session_dates:
            dates_set.update(session_dates)

        cfg = self.config_repo.get() if self.config_repo else None
        shared_telemetry = getattr(cfg, 'shared_telemetry', True) if cfg else True
        if shared_telemetry:
```

```
            sensor_dates = self.sensor_repo.get_available_dates()
            if sensor_dates:
                dates_set.update(sensor_dates)

        return sorted(list(dates_set), reverse=True)

    def get_daily_summary_data(self, target_date: Optional[str] = None, plant_id: int = 1) -> Dict[str, Any]:
        """
        Retorna el análisis consolidado exacto para un día específico y canastilla específica.
        Si no se indica fecha, por defecto muestra la fecha actual de hoy.
        """
        today_str = datetime.now().strftime("%Y-%m-%d")
        available_dates = self.get_available_dates(plant_id)

        # Si no se indica fecha o es 'auto', predeterminar al día actual de hoy
        if not target_date or target_date in ["auto", "today", ""]:
            target_date = today_str

        cfg = self.config_repo.get() if self.config_repo else None
        shared_telemetry = getattr(cfg, 'shared_telemetry', True) if cfg else True

        # 1. Obtener sesiones de fenotipado de la canastilla
        sessions = self.session_repo.get_by_date_and_plant(target_date, plant_id)
        has_plant_data = len(sessions) > 0

        # Obtener lecturas de sensores de la fecha (telemetría compartida o aislada)
        if not shared_telemetry:
            session_readings = [s.sensor_reading for s in sessions if s.sensor_reading]
            readings = session_readings
        else:
            readings = self.sensor_repo.get_by_date(target_date)

        has_sensor_data = len(readings) > 0

        # 2. Si no hay datos de fenotipado ni lecturas de sensores para este día
        if not has_plant_data and not has_sensor_data:
            return {
                "target_date": target_date,
                "available_dates": available_dates,
                "has_data": False,
                "has_plant_data": False,
                "has_sensor_data": False,
                "kpis": {
                    "has_sensor_data": False,
                    "has_plant_data": False,
                    "foliar_area_cm2": 0.0,
                    "foliar_area_diff": 0.0,
                    "temperature_c": 0.0,
                    "temperature_status": "Sin telemetría",
                    "humidity_rh": 0.0,
                    "humidity_status": "Sin telemetría",
                    "uv_solar_lux": 0.0,
                    "uv_solar_status": "Sin telemetría",
                    "health_index": 0.0,
                    "health_status": f"Sin fenotipado en Canastilla #{plant_id}",
```

```
                    "plant_height_cm": 0.0,
                    "stem_diameter_mm": 0.0,
                    "compacity_index": 0.0,
                    "ec_conductivity": 0.0,
                    "ec_status": "Sin telemetría",
                    "water_ph": 0.0,
                    "ph_status": "Sin telemetría",
                    "turbidity_ntu": 0.0,
                    "turbidity_status": "Sin telemetría",
                    "dissolved_oxygen_mg_l": 0.0,
                    "do_status": "Sin telemetría",
                    "water_temp_c": 0.0,
                    "water_temp_status": "Sin telemetría",
                    "motor_state_pct": 0.0,
                    "motor_status": "Sin telemetría"
                },
                "periods": {
                    "manana": {"area": 0.0, "height": 0.0, "diameter": 0.0, "health": 0.0, "has_data": False},
                    "medio_dia": {"area": 0.0, "height": 0.0, "diameter": 0.0, "health": 0.0, "has_data": False},
                    "tarde": {"area": 0.0, "height": 0.0, "diameter": 0.0, "health": 0.0, "has_data": False}
                },
                "thermal_curve": {
                    "max_temp": 0.0,
                    "min_temp": 0.0,
                    "amplitude": 0.0,
                    "regime": "Sin registros",
                    "labels": ["07:00", "10:00", "13:00", "16:00", "19:00"],
                    "temps": [0.0, 0.0, 0.0, 0.0, 0.0],
                    "min_band": [0.0, 0.0, 0.0, 0.0, 0.0],
                    "max_band": [0.0, 0.0, 0.0, 0.0, 0.0]
                },
                "latest_capture": {
                    "has_images": False,
                    "timestamp_str": "Sin capturas",
                    "period": "Sin datos",
                    "cenital_orig": "",
                    "cenital_proc": "",
                    "lateral_orig": "",
                    "lateral_proc": "",
                    "foliar_area_cm2": 0.0,
                    "plant_height_cm": 0.0,
                    "stem_diameter_mm": 0.0,
                    "health_index": 0.0
                },
                "growth_chart": {
                    "labels": ["Mañana", "Mediodía", "Tarde"],
                    "areas": [0.0, 0.0, 0.0],
                    "best_period": "Sin registros",
                    "avg_height": 0.0,
                    "avg_diameter": 0.0
                },
                "motor_chart": {
                    "labels": ["07:00", "10:00", "13:00", "16:00", "19:00"],
                    "states": [0, 0, 0, 0, 0],
                    "current_state": 0,
```

```
                    "current_state_label": "0 (APAGADO)",
                    "active_pct": 0.0,
                    "cycles_count": 0,
                    "status": "Inactiva / Sin telemetría"
                },
                "pump_chart": {
                    "avg_current": 0.0,
                    "max_current": 0.0,
                    "status": "Inactiva",
                    "labels": ["07:00", "10:00", "13:00", "16:00", "19:00"],
                    "currents": [0.0, 0.0, 0.0, 0.0, 0.0]
                },
                "solar_chart": {
                    "avg_uv": 0.0,
                    "max_uv": 0.0,
                    "labels": ["07:00", "09:00", "11:00", "13:00", "15:00", "17:00", "19:00"],
                    "uv_values": [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
                }
            }

        # 3. Procesar métricas biométricas de las sesiones del día para esta canastilla
        areas_list = []
        heights_list = []
        diameters_list = []
        healths_list = []
        compacity_list = []

        periods_data = {
            "manana": {"area": 0.0, "height": 0.0, "diameter": 0.0, "health": 0.0, "has_data": False},
            "medio_dia": {"area": 0.0, "height": 0.0, "diameter": 0.0, "health": 0.0, "has_data": False},
            "tarde": {"area": 0.0, "height": 0.0, "diameter": 0.0, "health": 0.0, "has_data": False}
        }

        for s in sessions:
            avg_m = s.get_average_metric()
            if avg_m:
                areas_list.append(avg_m.foliar_area_cm2)
                heights_list.append(avg_m.plant_height_cm)
                diameters_list.append(avg_m.stem_diameter_mm)
                healths_list.append(avg_m.health_index)
                compacity_list.append(avg_m.compacity_index)

                period_lower = s.period.lower().replace("ñ", "n").replace("í", "i").replace(" ", "_")
                for p_key in ["manana", "medio_dia", "tarde"]:
                    if p_key in period_lower or (p_key == "medio_dia" and "mediodia" in period_lower):
                        periods_data[p_key] = {
                            "area": round(avg_m.foliar_area_cm2, 2),
                            "height": round(avg_m.plant_height_cm, 2),
                            "diameter": round(avg_m.stem_diameter_mm, 2),
                            "health": round(avg_m.health_index, 1),
                            "has_data": True
                        }

        temps = [r.temperature for r in readings if r.temperature is not None]
        hums = [r.humidity for r in readings if r.humidity is not None]
```

```
        uvs = [r.uv_solar for r in readings if r.uv_solar is not None]
        currs = [r.motor_current for r in readings if r.motor_current is not None]
        ecs = [r.ec_conductivity for r in readings if r.ec_conductivity is not None]
        phs = [r.water_ph for r in readings if r.water_ph is not None]
        turbs = [r.turbidity_ntu for r in readings if r.turbidity_ntu is not None]
        dos = [r.dissolved_oxygen_mg_l for r in readings if r.dissolved_oxygen_mg_l is not None]
        water_temps = [r.water_temp for r in readings if getattr(r, 'water_temp', None) is not None]
        motor_states = [(r.motor_state if getattr(r, 'motor_state', None) is not None else (1 if (r.motor_current and r.motor_current > 0.1) else 0)) for r in readings]

        mean_area = round(float(np.mean(areas_list)), 2) if areas_list else 0.0
        mean_height = round(float(np.mean(heights_list)), 2) if heights_list else 0.0
        mean_diameter = round(float(np.mean(diameters_list)), 2) if diameters_list else 0.0
        mean_health = round(float(np.mean(healths_list)), 1) if healths_list else 0.0
        mean_compacity = round(float(np.mean(compacity_list)), 3) if compacity_list else 0.0

        mean_temp = round(float(np.mean(temps)), 1) if temps else 0.0
        min_temp = round(float(np.min(temps)), 1) if temps else 0.0
        max_temp = round(float(np.max(temps)), 1) if temps else 0.0
        amplitude = round(max_temp - min_temp, 1) if temps else 0.0

        mean_hum = round(float(np.mean(hums)), 1) if hums else 0.0
        mean_uv = round(float(np.mean(uvs)), 1) if uvs else 0.0
        mean_curr = round(float(np.mean(currs)), 3) if currs else 0.0
        max_curr = round(float(np.max(currs)), 3) if currs else 0.0
        mean_ec = round(float(np.mean(ecs)), 1) if ecs else 0.0
        mean_ph = round(float(np.mean(phs)), 2) if phs else 0.0
        mean_turb = round(float(np.mean(turbs)), 2) if turbs else 0.0
        mean_do = round(float(np.mean(dos)), 2) if dos else 0.0
        mean_water_temp = round(float(np.mean(water_temps)), 1) if water_temps else 0.0
        motor_active_pct = round((sum(motor_states) / max(len(motor_states), 1)) * 100.0, 1) if motor_states else 0.0

        # Comparación con jornada anterior de esta misma canastilla
        area_diff = 0.0
        try:
            curr_idx = available_dates.index(target_date) if target_date in available_dates else -1
            if curr_idx != -1 and curr_idx + 1 < len(available_dates):
                prev_date = available_dates[curr_idx + 1]
                prev_sessions = self.session_repo.get_by_date_and_plant(prev_date, plant_id)
                prev_areas = [s.get_average_metric().foliar_area_cm2 for s in prev_sessions if s.get_average_metric()]
                if prev_areas and areas_list:
                    area_diff = round(mean_area - float(np.mean(prev_areas)), 2)
        except Exception:
            pass

        thermal_labels = []
        thermal_temps = []
        thermal_min_band = []
        thermal_max_band = []
        motor_labels = []
        motor_states_series = []
        pump_labels = []
        pump_currents = []
        solar_labels = []
        solar_uvs = []
```

```
        if readings:
            step = max(1, len(readings) // 20)
            sampled_readings = readings[::step]
            for r in sampled_readings:
                t_label = r.timestamp.strftime("%H:%M")
                thermal_labels.append(t_label)
                thermal_temps.append(round(r.temperature, 2))
                thermal_min_band.append(round(min_temp, 1))
                thermal_max_band.append(round(max_temp, 1))

                m_state = int(r.motor_state if getattr(r, 'motor_state', None) is not None else (1 if r.motor_current > 0.1 else 0))
                motor_labels.append(t_label)
                motor_states_series.append(m_state)

                pump_labels.append(t_label)
                pump_currents.append(round(r.motor_current, 3))

                solar_labels.append(t_label)
                solar_uvs.append(round(r.uv_solar, 1))
        else:
            thermal_labels = ["07:00", "10:00", "13:00", "16:00", "19:00"]
            thermal_temps = [0.0] * 5
            thermal_min_band = [0.0] * 5
            thermal_max_band = [0.0] * 5
            motor_labels = thermal_labels
            motor_states_series = [0] * 5
            pump_labels = thermal_labels
            pump_currents = [0.0] * 5
            solar_labels = thermal_labels
            solar_uvs = [0.0] * 5

        # Conteo de ciclos de encendido del motor
        motor_cycles = 0
        for i in range(1, len(motor_states)):
            if motor_states[i-1] == 0 and motor_states[i] == 1:
                motor_cycles += 1
        if motor_cycles == 0 and sum(motor_states) > 0:
            motor_cycles = 1

        current_motor_state = int(motor_states[-1]) if motor_states else 0

        # Última toma de fenotipado para visualización directa en Dashboard
        latest_capture = {
            "has_images": False,
            "timestamp_str": "Sin capturas",
            "period": "Sin datos",
            "cenital_orig": "",
            "cenital_proc": "",
            "lateral_orig": "",
            "lateral_proc": "",
            "foliar_area_cm2": 0.0,
            "plant_height_cm": 0.0,
            "stem_diameter_mm": 0.0,
            "health_index": 0.0
        }
```

```
        if sessions:
            last_s = sessions[-1]
            last_avg = last_s.get_average_metric()
            if last_avg:
                latest_capture = {
                    "has_images": bool(last_avg.image_path_cenital_proc or last_avg.image_path_cenital_orig or last_avg.image_path_lateral_proc),
                    "timestamp_str": last_s.timestamp.strftime("%d/%m/%Y a las %H:%M") if last_s.timestamp else "",
                    "period": last_s.period,
                    "cenital_orig": last_avg.image_path_cenital_orig or "",
                    "cenital_proc": last_avg.image_path_cenital_proc or last_avg.image_path_cenital_orig or "",
                    "lateral_orig": last_avg.image_path_lateral_orig or "",
                    "lateral_proc": last_avg.image_path_lateral_proc or last_avg.image_path_lateral_orig or "",
                    "foliar_area_cm2": round(last_avg.foliar_area_cm2, 2),
                    "plant_height_cm": round(last_avg.plant_height_cm, 2),
                    "stem_diameter_mm": round(last_avg.stem_diameter_mm, 2),
                    "health_index": round(last_avg.health_index, 1)
                }

        # Periodo de mayor vigor
        best_p = "Sin registros"
        if has_plant_data:
            if periods_data["medio_dia"]["area"] >= periods_data["manana"]["area"] and periods_data["medio_dia"]["area"] >= periods_data["tarde"]["area"]:
                best_p = "Mediodía"
            elif periods_data["tarde"]["area"] >= periods_data["manana"]["area"]:
                best_p = "Tarde"
            else:
                best_p = "Mañana"

        regime_str = "Óptimo" if 18.0 <= mean_temp <= 26.0 else "Estrés Térmico" if mean_temp > 26.0 else "Baja Temperatura"

        return {
            "target_date": target_date,
            "available_dates": available_dates,
            "has_data": has_plant_data,
            "has_plant_data": has_plant_data,
            "has_sensor_data": has_sensor_data,
            "latest_capture": latest_capture,
            "kpis": {
                "foliar_area_cm2": mean_area,
                "foliar_area_diff": area_diff,
                "temperature_c": mean_temp,
                "temperature_status": "Régimen Óptimo" if (18.0 <= mean_temp <= 26.0 and has_sensor_data) else ("Alerta Térmica" if has_sensor_data else "Sin telemetría"),
                "humidity_rh": mean_hum,
                "humidity_status": "Rango Adecuado" if (55.0 <= mean_hum <= 75.0 and has_sensor_data) else ("Alerta Humedad" if has_sensor_data else "Sin telemetría"),
                "uv_solar_lux": mean_uv,
                "uv_solar_status": "Normal / Activa" if (mean_uv >= 200.0 and has_sensor_data) else ("Baja Irradiancia" if has_sensor_data else "Sin telemetría"),
                "health_index": mean_health,
                "health_status": ("Tejido Sano" if mean_health >= 85.0 else "Revisar Posible Clorosis") if has_plant_data else f"Sin fenotipado en Canastilla #{plant_id}",
                "plant_height_cm": mean_height,
                "stem_diameter_mm": mean_diameter,
                "compacity_index": mean_compacity,
                "ec_conductivity": mean_ec,
                "ec_status": ("Nutrientes Óptimos" if (1200.0 <= mean_ec <= 2400.0) else "Ajustar EC") if (has_sensor_data and mean_ec > 0.0) else "Sin telemetría",
                "water_ph": mean_ph,
```

```
                "ph_status": ("pH Óptimo" if (5.5 <= mean_ph <= 6.8) else "Ajustar pH") if (has_sensor_data and mean_ph > 0.0) else "Sin telemetría",
                "turbidity_ntu": mean_turb,
                "turbidity_status": ("Agua Cristalina" if (mean_turb <= 12.0) else "Turbidez Alta") if (has_sensor_data and mean_turb > 0.0) else "Sin telemetría",
                "dissolved_oxygen_mg_l": mean_do,
                "do_status": ("Oxigenación Óptima" if (mean_do >= 6.0) else "Baja Oxigenación") if (has_sensor_data and mean_do > 0.0) else "Sin telemetría",
                "water_temp_c": mean_water_temp,
                "water_temp_status": ("Temperatura Agua Óptima" if (18.0 <= mean_water_temp <= 22.5) else "Alerta Temp Agua") if (has_sensor_data and mean_water_temp > 0.0) else "Sin telemetría",
                "motor_state_pct": motor_active_pct,
                "motor_status": ("Recirculación Activa" if motor_active_pct > 0 else "En Espera") if has_sensor_data else "Sin telemetría"
            },
            "periods": periods_data,
            "thermal_curve": {
                "max_temp": max_temp,
                "min_temp": min_temp,
                "amplitude": amplitude,
                "regime": regime_str if has_sensor_data else "Sin telemetría",
                "labels": thermal_labels,
                "temps": thermal_temps,
                "min_band": thermal_min_band,
                "max_band": thermal_max_band
            },
            "growth_chart": {
                "labels": ["Mañana", "Mediodía", "Tarde"],
                "areas": [periods_data["manana"]["area"], periods_data["medio_dia"]["area"], periods_data["tarde"]["area"]],
                "best_period": best_p,
                "avg_height": mean_height,
                "avg_diameter": mean_diameter
            },
            "motor_chart": {
                "labels": motor_labels,
                "states": motor_states_series,
                "current_state": current_motor_state,
                "current_state_label": "ENCENDIDO (1)" if current_motor_state == 1 else "APAGADO (0)",
                "active_pct": motor_active_pct,
                "cycles_count": motor_cycles,
                "status": "Recirculación Activa" if current_motor_state == 1 else "Bomba en Espera"
            },
            "pump_chart": {
                "avg_current": mean_curr,
                "max_current": max_curr,
                "status": ("Flujo Continuo" if mean_curr >= 0.3 else "Bomba en Espera") if has_sensor_data else "Inactiva",
                "labels": pump_labels,
                "currents": pump_currents
            },
            "solar_chart": {
                "avg_uv": mean_uv,
                "max_uv": round(float(np.max(uvs)), 1) if readings else 0.0,
                "labels": solar_labels,
                "uv_values": solar_uvs
            }
        }

    def get_cross_referenced_dataset(self, plant_id: int = 1) -> List[Dict[str, Any]]:
        """Retorna la matriz sincronizada de imágenes y telemetría por sesión y por foto."""
```

```
        sessions = self.session_repo.get_all_by_plant(plant_id=plant_id, order_asc=True)
        cross_data = []

        for s in sessions:
            sensor = s.sensor_reading
            if not sensor:
                sensor = self.sensor_repo.find_nearest(s.timestamp)

            has_sensor = sensor is not None
            temp = round(sensor.temperature, 2) if (sensor and sensor.temperature is not None) else None
            hum = round(sensor.humidity, 1) if (sensor and sensor.humidity is not None) else None
            uv = round(sensor.uv_solar, 1) if (sensor and sensor.uv_solar is not None) else None
            curr = round(sensor.motor_current, 3) if (sensor and sensor.motor_current is not None) else None
            ec = round(sensor.ec_conductivity, 1) if (sensor and sensor.ec_conductivity is not None) else None
            ph = round(sensor.water_ph, 2) if (sensor and sensor.water_ph is not None) else None
            turb = round(sensor.turbidity_ntu, 2) if (sensor and sensor.turbidity_ntu is not None) else None
            do = round(sensor.dissolved_oxygen_mg_l, 2) if (sensor and sensor.dissolved_oxygen_mg_l is not None) else None
            water_temp = round(sensor.water_temp, 1) if (sensor and getattr(sensor, 'water_temp', None) is not None) else None
            motor_state = int(sensor.motor_state) if (sensor and getattr(sensor, 'motor_state', None) is not None) else (1 if (sensor and curr and curr > 0.1) else (0 if sensor else None))

            avg_metric = s.get_average_metric()
            avg_cenital = avg_metric.image_path_cenital_proc or "" if avg_metric else ""
            avg_lateral = avg_metric.image_path_lateral_proc or "" if avg_metric else ""
            session_cenital_url = "/static/" + avg_cenital.replace("static/", "").replace("local_server/static/", "").lstrip("/") if avg_cenital else ""
            session_lateral_url = "/static/" + avg_lateral.replace("static/", "").replace("local_server/static/", "").lstrip("/") if avg_lateral else ""

            raw_photos = s.get_individual_photos()
            individual_photos = []
            for idx, m in enumerate(raw_photos):
                m_dict = m.to_dict()
                m_dict["photo_num"] = idx + 1
                m_dict["temperature_c"] = temp
                m_dict["humidity_rh"] = hum
                m_dict["water_temp_c"] = water_temp
                m_dict["motor_state"] = motor_state
                m_dict["uv_solar_lux"] = uv
                m_dict["motor_current_a"] = curr
                m_dict["ec_conductivity"] = ec
                m_dict["water_ph"] = ph
                m_dict["turbidity_ntu"] = turb
                m_dict["dissolved_oxygen_mg_l"] = do
                m_dict["exact_time"] = s.timestamp.strftime("%H:%M:%S")

                c_proc = m.image_path_cenital_proc or ""
                l_proc = m.image_path_lateral_proc or ""
                m_dict["cenital_url"] = "/static/" + c_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/") if c_proc else ""
                m_dict["lateral_url"] = "/static/" + l_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/") if l_proc else ""
                m_dict["thumbnail_url"] = m_dict["cenital_url"] or m_dict["lateral_url"]

                individual_photos.append(m_dict)

            # Normalización del período y fecha
            raw_p = s.period or ""
            if " - " in raw_p:
                parts = raw_p.split(" - ")
```

```
                date_clean = parts[0].strip()
                period_clean = parts[1].strip()
            elif "_" in raw_p:
                parts = raw_p.split("_")
                date_clean = parts[0].strip()
                period_clean = parts[1].strip()
            else:
                date_clean = s.timestamp.strftime("%Y-%m-%d")
                period_clean = raw_p or "Muestreo"

            if avg_metric:
                cross_data.append({
                    "session_id": s.id,
                    "has_sensor_data": has_sensor,
                    "period": period_clean.capitalize(),
                    "period_name": period_clean.lower(),
                    "date_str": date_clean,
                    "full_period_label": f"{date_clean} - {period_clean.capitalize()}",
                    "timestamp": s.timestamp.strftime("%Y-%m-%d %H:%M:%S"),
                    "exact_time": s.timestamp.strftime("%H:%M:%S"),
                    "crop_type": s.crop_type,
                    "plant_id": s.plant_id,
                    "foliar_area_cm2": round(avg_metric.foliar_area_cm2, 2),
                    "plant_height_cm": round(avg_metric.plant_height_cm, 2),
                    "stem_diameter_mm": round(avg_metric.stem_diameter_mm, 2),
                    "health_index": round(avg_metric.health_index, 1),
                    "compacity_index": round(avg_metric.compacity_index, 3),
                    "temperature_c": temp,
                    "humidity_rh": hum,
                    "water_temp_c": water_temp,
                    "motor_state": motor_state,
                    "uv_solar_lux": uv,
                    "motor_current_a": curr,
                    "ec_conductivity": ec,
                    "water_ph": ph,
                    "turbidity_ntu": turb,
                    "dissolved_oxygen_mg_l": do,
                    "cenital_url": session_cenital_url,
                    "lateral_url": session_lateral_url,
                    "photos_count": len(individual_photos),
                    "individual_photos": individual_photos
                })

        return cross_data

    def calculate_full_correlation_matrix(self, cross_data: List[Dict[str, Any]]) -> Dict[str, Any]:
        """Calcula la matriz de correlación de Pearson y regresión entre sensores y fenotipado."""
        sensor_keys = [
            ("temperature_c", "Temperatura Ambiente", "°C", "#2563eb"),
            ("humidity_rh", "Humedad Relativa", "%", "#0284c7"),
            ("water_temp_c", "Temperatura del Agua", "°C", "#0ea5e9"),
            ("motor_state", "Estado Motor (1/0)", "pts", "#059669"),
            ("uv_solar_lux", "Radiación Solar", "lux", "#d97706"),
            ("motor_current_a", "Corriente Bomba", "A", "#10b981"),
            ("ec_conductivity", "Conductividad Eléctrica", "µS/cm", "#8b5cf6"),
```

```
            ("water_ph", "pH de la Solución", "pH", "#ec4899"),
            ("turbidity_ntu", "Turbidez del Agua", "NTU", "#f97316"),
            ("dissolved_oxygen_mg_l", "Oxígeno Disuelto", "mg/L", "#06b6d4")
        ]
        pheno_keys = [
            ("foliar_area_cm2", "Área Foliar", "cm²", "#059669"),
            ("plant_height_cm", "Altura Planta", "cm", "#16a34a"),
            ("stem_diameter_mm", "Diámetro Tallo", "mm", "#84cc16"),
            ("health_index", "Índice de Salud", "%", "#10b981"),
            ("compacity_index", "Compacidad", "pts", "#047857")
        ]

        matrix_rows = []
        strongest_r = 0.0
        strongest_pair = "Sin datos suficientes"

        if not cross_data or len(cross_data) < 2:
            for s_key, s_label, s_unit, s_color in sensor_keys:
                cells = []
                for p_key, p_label, p_unit, p_color in pheno_keys:
                    cells.append({
                        "sensor_key": s_key,
                        "pheno_key": p_key,
                        "r": 0.0,
                        "r2": 0.0,
                        "interpretation": "Muestras insuficientes (mín. 2 períodos)",
                        "color_class": "neutral",
                        "bg_color": "rgba(241, 245, 249, 0.6)",
                        "text_color": "#64748b"
                    })
                matrix_rows.append({
                    "sensor_key": s_key,
                    "sensor_label": s_label,
                    "sensor_unit": s_unit,
                    "cells": cells
                })
            return {
                "sensors": [{"key": s[0], "label": s[1], "unit": s[2], "color": s[3]} for s in sensor_keys],
                "phenotypes": [{"key": p[0], "label": p[1], "unit": p[2], "color": p[3]} for p in pheno_keys],
                "matrix_rows": matrix_rows,
                "strongest_pair": strongest_pair
            }

        for s_key, s_label, s_unit, s_color in sensor_keys:
            cells = []
            for p_key, p_label, p_unit, p_color in pheno_keys:
                valid_pairs = [
                    (float(d[s_key]), float(d[p_key]))
                    for d in cross_data
                    if d.get(s_key) is not None and d.get(p_key) is not None
                ]

                if len(valid_pairs) >= 2:
                    s_vals = np.array([p[0] for p in valid_pairs], dtype=float)
                    p_vals = np.array([p[1] for p in valid_pairs], dtype=float)
```

```
                    s_std = float(np.std(s_vals))
                    p_std = float(np.std(p_vals))

                    if s_std > 1e-6 and p_std > 1e-6:
                        r_val = float(np.corrcoef(s_vals, p_vals)[0, 1])
                        if np.isnan(r_val):
                            r_val = 0.0
                    else:
                        r_val = 0.0
                else:
                    r_val = 0.0

                r_val = round(r_val, 2)
                r2_val = round(r_val ** 2, 3)

                if len(valid_pairs) < 2:
                    interp = "Sin telemetría captada"
                    color_class = "neutral"
                    bg_color = "rgba(241, 245, 249, 0.6)"
                    text_color = "#94a3b8"
                elif r_val >= 0.65:
                    interp = "Correlación Directa Fuerte"
                    color_class = "pos-strong"
                    bg_color = "rgba(16, 185, 129, 0.28)"
                    text_color = "#065f46"
                elif r_val >= 0.25:
                    interp = "Correlación Directa Moderada"
                    color_class = "pos-mod"
                    bg_color = "rgba(52, 211, 153, 0.18)"
                    text_color = "#047857"
                elif r_val <= -0.65:
                    interp = "Correlación Inversa Fuerte"
                    color_class = "neg-strong"
                    bg_color = "rgba(239, 68, 68, 0.24)"
                    text_color = "#991b1b"
                elif r_val <= -0.25:
                    interp = "Correlación Inversa Moderada"
                    color_class = "neg-mod"
                    bg_color = "rgba(248, 113, 113, 0.16)"
                    text_color = "#b91c1c"
                else:
                    interp = "Relación Débil o Independiente"
                    color_class = "neutral"
                    bg_color = "rgba(241, 245, 249, 0.7)"
                    text_color = "#475569"

                if abs(r_val) > abs(strongest_r) and len(valid_pairs) >= 2:
                    strongest_r = r_val
                    strongest_pair = f"{s_label} vs {p_label} (r = {r_val:+0.2f}, R² = {r2_val})"

                cells.append({
                    "sensor_key": s_key,
                    "pheno_key": p_key,
                    "r": r_val,
                    "r2": r2_val,
```

```
                    "interpretation": interp,
                    "color_class": color_class,
                    "bg_color": bg_color,
                    "text_color": text_color
                })

            matrix_rows.append({
                "sensor_key": s_key,
                "sensor_label": s_label,
                "sensor_unit": s_unit,
                "cells": cells
            })

        return {
            "sensors": [{"key": s[0], "label": s[1], "unit": s[2], "color": s[3]} for s in sensor_keys],
            "phenotypes": [{"key": p[0], "label": p[1], "unit": p[2], "color": p[3]} for p in pheno_keys],
            "matrix_rows": matrix_rows,
            "strongest_pair": strongest_pair
        }

    def calculate_agronomic_correlations(self, cross_data: List[Dict[str, Any]]) -> Dict[str, Any]:
        """Calcula estadísticas descriptivas e inferencias para investigación agronómica."""
        if not cross_data or len(cross_data) < 2:
            return {
                "optimal_temp_range": "21.0 - 24.5 °C",
                "optimal_humidity_range": "60.0 - 70.0 %",
                "optimal_uv_range": "300.0 - 450.0 lux",
                "growth_rate_cm2_day": 0.0,
                "insights": [
                    "Se requieren al menos 2 sesiones de muestreo para calcular correlaciones dinámicas de crecimiento.",
                    "El sistema evaluará las condiciones de mayor tasa de expansión foliar conforme se registren más períodos."
                ]
            }

        areas = [d["foliar_area_cm2"] for d in cross_data if d.get("foliar_area_cm2") is not None]
        if not areas:
            return {
                "optimal_temp_range": "21.0 - 24.5 °C",
                "optimal_humidity_range": "60.0 - 70.0 %",
                "optimal_uv_range": "300.0 - 450.0 lux",
                "growth_rate_cm2_day": 0.0,
                "insights": ["Sin datos fenotípicos suficientes."]
            }

        delta_area = max(areas) - min(areas)
        growth_rate = round(delta_area / max(len(cross_data) - 1, 1), 2)

        median_area = float(np.median(areas))
        top_samples = [d for d in cross_data if (d.get("foliar_area_cm2") or 0.0) >= median_area]

        temps = [d["temperature_c"] for d in top_samples if d.get("temperature_c") is not None]
        hums = [d["humidity_rh"] for d in top_samples if d.get("humidity_rh") is not None]
        uvs = [d["uv_solar_lux"] for d in top_samples if d.get("uv_solar_lux") is not None]

        opt_temp_min = round(min(temps) - 0.5, 1) if temps else 21.0
```

```
        opt_temp_max = round(max(temps) + 0.5, 1) if temps else 24.5
        opt_hum_min = round(min(hums) - 2.0, 1) if hums else 60.0
        opt_hum_max = round(max(hums) + 2.0, 1) if hums else 70.0
        opt_uv_min = round(min(uvs) - 10.0, 1) if uvs else 300.0
        opt_uv_max = round(max(uvs) + 10.0, 1) if uvs else 450.0

        insights = [
            f"Tasa media de incremento foliar: +{growth_rate} cm² por sesión registrada.",
            f"El mayor vigor y salud foliar se observó con temperaturas en rango {opt_temp_min} a {opt_temp_max} °C y humedad en {opt_hum_min}% a {opt_hum_max}%.",
            f"La radiación solar útil óptima para esta especie oscila entre {opt_uv_min} y {opt_uv_max} lux.",
            "La estabilidad en la corriente de la bomba confirma flujo de solución nutritiva sin obstrucciones."
        ]

        return {
            "optimal_temp_range": f"{opt_temp_min} - {opt_temp_max} °C" if temps else "Sin telemetría captada",
            "optimal_humidity_range": f"{opt_hum_min} - {opt_hum_max} %" if hums else "Sin telemetría captada",
            "optimal_uv_range": f"{opt_uv_min} - {opt_uv_max} lux" if uvs else "Sin telemetría captada",
            "growth_rate_cm2_day": growth_rate,
            "insights": insights
        }

    def get_sensor_timeline_data(self, target_date: Optional[str] = None, plant_id: int = 1, limit_points: int = 120) -> Dict[str, Any]:
        """Obtiene datos temporales remuestreados y estadísticas resumidas con soporte de aislamiento por canastilla."""
        cfg = self.config_repo.get() if self.config_repo else None
        shared_telemetry = getattr(cfg, 'shared_telemetry', True) if cfg else True

        today_str = datetime.now().strftime("%Y-%m-%d")
        if not shared_telemetry and plant_id > 1:
            # Modo aislado: obtener exclusivamente las lecturas vinculadas a las sesiones de esta canastilla
            if not target_date or target_date in ["auto", "today", ""]:
                target_date = today_str

            if target_date == "todos":
                sessions = self.session_repo.get_all_by_plant(plant_id)
            else:
                sessions = self.session_repo.get_by_date_and_plant(target_date, plant_id)

            readings = [s.sensor_reading for s in sessions if s.sensor_reading]
            available_dates = self.get_available_dates(plant_id)
        else:
            available_dates = self.get_available_dates(plant_id)
            if not target_date or target_date in ["auto", "today", ""]:
                target_date = today_str
            readings = self.sensor_repo.get_by_date(target_date)

        total_records = len(readings)

        if total_records == 0:
            return {
                "status": "success",
                "target_date": target_date,
                "available_dates": available_dates,
                "total_records": 0,
                "records_count": 0,
                "summary": {
```

```
                    "temperature": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": f"Sin datos en Canastilla #{plant_id}"},
                    "humidity": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": f"Sin datos en Canastilla #{plant_id}"},
                    "uv_solar": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": f"Sin datos en Canastilla #{plant_id}"},
                    "motor_current": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": "Inactiva"},
                    "ec_conductivity": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": "Sin datos"},
                    "water_ph": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": "Sin datos"},
                    "turbidity_ntu": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": "Sin datos"},
                    "dissolved_oxygen_mg_l": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": "Sin datos"},
                    "water_temp": {"avg": 0.0, "min": 0.0, "max": 0.0, "last": 0.0, "status": "Sin datos"},
                    "motor_state": {"avg": 0.0, "min": 0, "max": 0, "last": 0, "status": "Sin datos"}
                },
                "timeline": []
            }

        temps = [r.temperature for r in readings]
        hums = [r.humidity for r in readings]
        water_temps = [(r.water_temp if getattr(r, 'water_temp', None) is not None else 20.0) for r in readings]
        motor_states = [(r.motor_state if getattr(r, 'motor_state', None) is not None else (1 if r.motor_current > 0.1 else 0)) for r in readings]
        ecs = [(r.ec_conductivity if r.ec_conductivity is not None else 1650.0) for r in readings]
        phs = [(r.water_ph if r.water_ph is not None else 6.20) for r in readings]
        turbs = [(r.turbidity_ntu if r.turbidity_ntu is not None else 3.8) for r in readings]
        dos = [(r.dissolved_oxygen_mg_l if r.dissolved_oxygen_mg_l is not None else 7.4) for r in readings]

        summary = {
            "water_ph": {
                "avg": round(float(np.mean(phs)), 2),
                "min": round(float(np.min(phs)), 2),
                "max": round(float(np.max(phs)), 2),
                "last": round(phs[-1], 2),
                "status": "pH Óptimo" if 5.5 <= np.mean(phs) <= 6.8 else "Ajustar pH"
            },
            "turbidity_ntu": {
                "avg": round(float(np.mean(turbs)), 2),
                "min": round(float(np.min(turbs)), 2),
                "max": round(float(np.max(turbs)), 2),
                "last": round(turbs[-1], 2),
                "status": "Agua Cristalina" if np.mean(turbs) <= 12.0 else "Turbidez Alta"
            },
            "ec_conductivity": {
                "avg": round(float(np.mean(ecs)), 1),
                "min": round(float(np.min(ecs)), 1),
                "max": round(float(np.max(ecs)), 1),
                "last": round(ecs[-1], 1),
                "status": "Nutrientes Óptimos" if 1200.0 <= np.mean(ecs) <= 2400.0 else "Ajustar EC"
            },
            "dissolved_oxygen_mg_l": {
                "avg": round(float(np.mean(dos)), 2),
                "min": round(float(np.min(dos)), 2),
                "max": round(float(np.max(dos)), 2),
                "last": round(dos[-1], 2),
                "status": "Oxigenación Óptima" if np.mean(dos) >= 6.0 else "Bajo Oxigeno"
            },
            "water_temp": {
                "avg": round(float(np.mean(water_temps)), 1),
                "min": round(float(np.min(water_temps)), 1),
```

```
                "max": round(float(np.max(water_temps)), 1),
                "last": round(water_temps[-1], 1),
                "status": "Óptima" if 18.0 <= np.mean(water_temps) <= 24.0 else "Revisar Temp Agua"
            },
            "motor_state": {
                "avg": round(float(np.mean(motor_states)), 2),
                "min": int(np.min(motor_states)),
                "max": int(np.max(motor_states)),
                "last": int(motor_states[-1]),
                "status": "Motor Encendido (1)" if motor_states[-1] == 1 else "Motor Apagado (0)"
            },
            "temperature": {
                "avg": round(float(np.mean(temps)), 1),
                "min": round(float(np.min(temps)), 1),
                "max": round(float(np.max(temps)), 1),
                "last": round(temps[-1], 1),
                "status": "Óptimo" if 18.0 <= np.mean(temps) <= 28.0 else "Fuera de Rango"
            },
            "humidity": {
                "avg": round(float(np.mean(hums)), 1),
                "min": round(float(np.min(hums)), 1),
                "max": round(float(np.max(hums)), 1),
                "last": round(hums[-1], 1),
                "status": "Óptimo" if 50.0 <= np.mean(hums) <= 85.0 else "Alerta Humedad"
            }
        }

        # Remuestreo inteligente
        step = max(1, total_records // limit_points)
        sampled_readings = readings[::step]
        if readings[-1] not in sampled_readings:
            sampled_readings.append(readings[-1])

        timeline = []
        for r in sampled_readings:
            time_label = r.timestamp.strftime("%H:%M:%S") if target_date != "todos" else r.timestamp.strftime("%d/%m %H:%M")
            timeline.append({
                "time": time_label,
                "timestamp": r.timestamp.strftime("%Y-%m-%d %H:%M:%S"),
                "water_ph": round(r.water_ph if r.water_ph is not None else 6.20, 2),
                "turbidity_ntu": round(r.turbidity_ntu if r.turbidity_ntu is not None else 3.8, 2),
                "ec_conductivity": round(r.ec_conductivity if r.ec_conductivity is not None else 1650.0, 1),
                "dissolved_oxygen_mg_l": round(r.dissolved_oxygen_mg_l if r.dissolved_oxygen_mg_l is not None else 7.4, 2),
                "water_temp": round(getattr(r, 'water_temp', 20.0) if getattr(r, 'water_temp', None) is not None else 20.0, 1),
                "motor_state": int(getattr(r, 'motor_state', 0) if getattr(r, 'motor_state', None) is not None else (1 if r.motor_current > 0.1 else 0)),
                "temperature": round(r.temperature, 2),
                "humidity": round(r.humidity, 1)
            })

        return {
            "status": "success",
            "target_date": target_date,
            "available_dates": available_dates,
            "total_records": total_records,
            "records_count": total_records,
```

```
            "sampled_records": len(timeline),
            "summary": summary,
            "timeline": timeline
        }

    def generate_research_csv(self, cross_data: List[Dict[str, Any]]) -> str:
        """Genera contenido CSV estructurado para exportación científica."""
        output = io.StringIO()
        writer = csv.writer(output, delimiter=';')

        writer.writerow([
            "ID_Sesion",
            "Periodo_Muestreo",
            "Fecha_Hora_Captura",
            "Especie_Cultivo",
            "Canastilla_ID",
            "Area_Foliar_cm2",
            "Altura_Planta_cm",
            "Diametro_Tallo_mm",
            "Indice_Salud_pct",
            "Indice_Compacidad",
            "Temperatura_Ambiente_C",
            "Humedad_Relativa_pct",
            "Temperatura_Agua_C",
            "Estado_Motor_1_0",
            "Radiacion_Solar_UV_lux",
            "Corriente_Motor_A",
            "Conductividad_Electrica_uS_cm",
            "pH_Solucion",
            "Turbidez_NTU",
            "Oxigeno_Disuelto_mg_L",
            "Numero_Fotos_Lote"
        ])

        for item in cross_data:
            writer.writerow([
                item.get("session_id", ""),
                item.get("date_str", ""),
                item.get("timestamp", ""),
                item.get("crop_type", ""),
                item.get("plant_id", 1),
                item.get("foliar_area_cm2", 0.0),
                item.get("plant_height_cm", 0.0),
                item.get("stem_diameter_mm", 0.0),
                item.get("health_index", 100.0),
                item.get("compacity_index", 0.0),
                item.get("temperature_c") if item.get("temperature_c") is not None else "",
                item.get("humidity_rh") if item.get("humidity_rh") is not None else "",
                item.get("water_temp_c") if item.get("water_temp_c") is not None else "",
                item.get("motor_state") if item.get("motor_state") is not None else "",
                item.get("uv_solar_lux") if item.get("uv_solar_lux") is not None else "",
                item.get("motor_current_a") if item.get("motor_current_a") is not None else "",
                item.get("ec_conductivity") if item.get("ec_conductivity") is not None else "",
                item.get("water_ph") if item.get("water_ph") is not None else "",
                item.get("turbidity_ntu") if item.get("turbidity_ntu") is not None else "",
```

```
                item.get("dissolved_oxygen_mg_l") if item.get("dissolved_oxygen_mg_l") is not None else "",
                item.get("photos_count", 0)
            ])

        return output.getvalue()

    def generate_daily_summary_csv(self, target_date: str, plant_id: int = 1) -> str:
        """Genera un CSV consolidado con el resumen agronómico y ambiental de una fecha específica."""
        daily_data = self.get_daily_summary_data(target_date=target_date, plant_id=plant_id)
        kpis = daily_data.get("kpis", {})
        periods = daily_data.get("periods", {})
        thermal = daily_data.get("thermal_curve", {})
        pump = daily_data.get("pump_chart", {})

        output = io.StringIO()
        writer = csv.writer(output, delimiter=';')

        writer.writerow(["REPORTE CONSOLIDADO DIARIO SIFMA"])
        writer.writerow(["Fecha de Evaluacion", target_date])
        writer.writerow(["Canastilla / Nivel", f"#{plant_id}"])
        writer.writerow(["Generado", datetime.now().strftime("%Y-%m-%d %H:%M:%S")])
        writer.writerow([])

        writer.writerow(["PARAMETRO BIOMETRICO", "VALOR", "UNIDAD", "DIAGNOSTICO"])
        writer.writerow(["Area Foliar Media", kpis.get("foliar_area_cm2", 0.0), "cm²", "Expansión foliar" if daily_data.get("has_plant_data") else "Sin registros"])
        writer.writerow(["Variacion vs Dia Anterior", kpis.get("foliar_area_diff", 0.0), "cm²", "Crecimiento neto" if daily_data.get("has_plant_data") else "Sin registros"])
        writer.writerow(["Altura Promedio Planta", kpis.get("plant_height_cm", 0.0), "cm", "Morfología vertical" if daily_data.get("has_plant_data") else "Sin registros"])
        writer.writerow(["Diametro Promedio Tallo", kpis.get("stem_diameter_mm", 0.0), "mm", "Grosor basal" if daily_data.get("has_plant_data") else "Sin registros"])
        writer.writerow(["Indice de Salud Foliar", kpis.get("health_index", 0.0), "%", kpis.get("health_status", "Sin fenotipado")])
        writer.writerow(["Indice de Compacidad", kpis.get("compacity_index", 0.0), "pts", "Densidad de canopia" if daily_data.get("has_plant_data") else "Sin registros"])
        writer.writerow([])

        writer.writerow(["PARAMETRO AMBIENTAL / NUTRIENTES", "VALOR MEDIO", "UNIDAD", "ESTADO OPERATIVO"])
        writer.writerow(["Temperatura Exterior", kpis.get("temperature_c", 0.0), "°C", kpis.get("temperature_status", "Sin telemetría")])
        writer.writerow(["Humedad Relativa Exterior", kpis.get("humidity_rh", 0.0), "% HR", kpis.get("humidity_status", "Sin telemetría")])
        writer.writerow(["pH Solucion Nutritiva", kpis.get("water_ph", 0.0), "pH", kpis.get("ph_status", "Sin telemetría")])
        writer.writerow(["Conductividad Electrica (EC)", kpis.get("ec_conductivity", 0.0), "µS/cm", kpis.get("ec_status", "Sin telemetría")])
        writer.writerow(["Turbidez del Agua", kpis.get("turbidity_ntu", 0.0), "NTU", kpis.get("turbidity_status", "Sin telemetría")])
        writer.writerow(["Oxigeno Disuelto (DO)", kpis.get("dissolved_oxygen_mg_l", 0.0), "mg/L", kpis.get("do_status", "Sin telemetría")])
        writer.writerow(["Temperatura del Agua", kpis.get("water_temp_c", 0.0), "°C", kpis.get("water_temp_status", "Sin telemetria")])
        writer.writerow(["Ciclo Recirculacion Bomba", f"{kpis.get('motor_state_pct', 0.0)}% activo", "% tiempo", kpis.get("motor_status", "Sin telemetría")])
        writer.writerow(["Radiacion Solar Media", kpis.get("uv_solar_lux", 0.0), "lux", kpis.get("uv_solar_status", "Sin telemetria")])
        writer.writerow(["Corriente Bomba Recirculacion", pump.get("avg_current", 0.0), "A", pump.get("status", "Sin telemetría")])
        writer.writerow(["Temperatura Maxima del Dia", thermal.get("max_temp", 0.0), "°C", "Pico térmico" if daily_data.get("has_sensor_data") else "Sin registros"])
        writer.writerow(["Temperatura Minima del Dia", thermal.get("min_temp", 0.0), "°C", "Mínimo nocturno" if daily_data.get("has_sensor_data") else "Sin registros"])
        writer.writerow(["Delta Termico (Amplitud)", thermal.get("amplitude", 0.0), "°C", thermal.get("regime", "Sin registros")])
        writer.writerow([])

        writer.writerow(["FRANJA HORARIA / PERIODO", "AREA FOLIAR (cm²)", "ALTURA (cm)", "TALLO (mm)", "SALUD (%)"])
        for p_key, p_name in [("manana", "Mañana (07:00)"), ("medio_dia", "Mediodía (12:00)"), ("tarde", "Tarde (17:00)")]:
            p_data = periods.get(p_key, {})
            writer.writerow([
                p_name,
```

```
                p_data.get("area", 0.0),
                p_data.get("height", 0.0),
                p_data.get("diameter", 0.0),
                p_data.get("health", 0.0)
            ])

        return output.getvalue()

    def sync_tower_telemetry_across_canastillas(self) -> Dict[str, Any]:
        """
        Sincroniza y vincula todas las sesiones de captura de todas las canastillas
        con la telemetría ambiental más cercana registrada en la torre.
        """
        all_sessions = self.session_repo.get_all()
        synced_count = 0
        for s in all_sessions:
            if not s.sensor_reading_id or not s.sensor_reading:
                nearest = self.sensor_repo.find_nearest(s.timestamp)
                if nearest:
                    s.sensor_reading_id = nearest.id
                    synced_count += 1
        db.session.commit()
        return {
            "status": "success",
            "message": f"Se sincronizaron {synced_count} sesiones fotográficas con las lecturas ambientales de la torre.",
            "synced_count": synced_count,
            "total_sessions": len(all_sessions)
        }

    def get_calendar_month_data(self, year: int, month: int, plant_id: int = 1) -> Dict[str, Any]:
        """
        Construye la matriz de días y semanas para la visualización del calendario interactivo,
        asociando a cada día sus indicadores de fenotipado (verde) y telemetría (azul).
        """
        cfg = self.config_repo.get() if self.config_repo else None
        shared_telemetry = getattr(cfg, 'shared_telemetry', True) if cfg else True

        # Obtener todas las sesiones de este mes para la canastilla
        month_str = f"{year:04d}-{month:02d}"
        month_sessions = CaptureSession.query.filter(
            CaptureSession.plant_id == int(plant_id),
            db.func.strftime('%Y-%m', CaptureSession.timestamp) == month_str
        ).all()

        sessions_by_date = {}
        for s in month_sessions:
            d_str = s.timestamp.strftime("%Y-%m-%d")
            if d_str not in sessions_by_date:
                sessions_by_date[d_str] = []
            sessions_by_date[d_str].append(s)

        # Obtener lecturas de telemetría de este mes
        if shared_telemetry or int(plant_id) == 1:
            month_readings = SensorReading.query.filter(
                db.func.strftime('%Y-%m', SensorReading.timestamp) == month_str
```

```
            ).all()
        else:
            # En modo aislado, solo lecturas vinculadas a sesiones de esta canastilla
            reading_ids = [s.sensor_reading_id for s in month_sessions if s.sensor_reading_id]
            month_readings = SensorReading.query.filter(SensorReading.id.in_(reading_ids)).all() if reading_ids else []

        readings_by_date = {}
        for r in month_readings:
            d_str = r.timestamp.strftime("%Y-%m-%d")
            if d_str not in readings_by_date:
                readings_by_date[d_str] = []
            readings_by_date[d_str].append(r)

        # Construir cuadrícula de semanas (comenzando en Lunes: calendar.MONDAY)
        cal = calendar.Calendar(firstweekday=0)
        weeks_matrix = []
        month_days = cal.monthdatescalendar(year, month)

        SPANISH_MONTHS = [
            "", "Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",
            "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre"
        ]

        total_photo_days = 0
        total_telemetry_days = 0
        total_combined_days = 0
        total_sessions = len(month_sessions)
        total_readings = len(month_readings)

        today_str = datetime.now().strftime("%Y-%m-%d")

        for week in month_days:
            week_days = []
            for d in week:
                d_str = d.strftime("%Y-%m-%d")
                is_current_month = (d.month == month)

                day_sessions = sessions_by_date.get(d_str, [])
                day_readings = readings_by_date.get(d_str, [])

                has_photos = len(day_sessions) > 0
                has_telemetry = len(day_readings) > 0

                if is_current_month:
                    if has_photos: total_photo_days += 1
                    if has_telemetry: total_telemetry_days += 1
                    if has_photos and has_telemetry: total_combined_days += 1

                periods = []
                avg_foliar_area = 0.0
                avg_health = 0.0
                if has_photos:
                    areas = []
                    healths = []
                    for s in day_sessions:
```

```
                        raw_p = s.period or "Muestreo"
                        if " - " in raw_p:
                            p_name = raw_p.split(" - ")[1].strip().capitalize()
                        elif "_" in raw_p:
                            p_name = raw_p.split("_")[1].strip().capitalize()
                        else:
                            p_name = raw_p.capitalize()
                        periods.append(p_name)

                        m = s.get_average_metric()
                        if m:
                            areas.append(m.foliar_area_cm2 or 0.0)
                            healths.append(m.health_index or 100.0)
                    if areas:
                        avg_foliar_area = round(float(np.mean(areas)), 1)
                    if healths:
                        avg_health = round(float(np.mean(healths)), 1)

                avg_temp = 0.0
                avg_hum = 0.0
                avg_uv = 0.0
                if has_telemetry:
                    avg_temp = round(float(np.mean([r.temperature for r in day_readings])), 1)
                    avg_hum = round(float(np.mean([r.humidity for r in day_readings])), 1)
                    avg_uv = round(float(np.mean([r.uv_solar for r in day_readings])), 1)

                week_days.append({
                    "date_str": d_str,
                    "day_number": d.day,
                    "month": d.month,
                    "year": d.year,
                    "is_current_month": is_current_month,
                    "is_today": (d_str == today_str),
                    "has_photos": has_photos,
                    "photo_count": len(day_sessions),
                    "periods": sorted(list(set(periods))),
                    "avg_foliar_area": avg_foliar_area,
                    "avg_health": avg_health,
                    "has_telemetry": has_telemetry,
                    "telemetry_count": len(day_readings),
                    "avg_temp": avg_temp,
                    "avg_hum": avg_hum,
                    "avg_uv": avg_uv,
                    "is_complete": (has_photos and has_telemetry)
                })
            weeks_matrix.append(week_days)

        if month == 1:
            prev_year, prev_month = year - 1, 12
        else:
            prev_year, prev_month = year, month - 1

        if month == 12:
            next_year, next_month = year + 1, 1
        else:
```

```
            next_year, next_month = year, month + 1

        days_in_month_count = calendar.monthrange(year, month)[1]
        coverage_pct = round((total_photo_days / max(days_in_month_count, 1)) * 100, 1)

        return {
            "year": year,
            "month": month,
            "month_name": SPANISH_MONTHS[month],
            "plant_id": int(plant_id),
            "shared_telemetry": shared_telemetry,
            "weeks": weeks_matrix,
            "prev_year": prev_year,
            "prev_month": prev_month,
            "next_year": next_year,
            "next_month": next_month,
            "summary": {
                "total_photo_days": total_photo_days,
                "total_telemetry_days": total_telemetry_days,
                "total_combined_days": total_combined_days,
                "total_sessions": total_sessions,
                "total_readings": total_readings,
                "days_in_month": days_in_month_count,
                "coverage_pct": coverage_pct
            }
        }

    def get_day_detail(self, date_str: str, plant_id: int = 1) -> Dict[str, Any]:
        """Retorna el desglose fotográfico y ambiental detallado para un día y canastilla."""
        sessions = self.session_repo.get_by_date_and_plant(date_str, plant_id)

        cfg = self.config_repo.get() if self.config_repo else None
        shared_telemetry = getattr(cfg, 'shared_telemetry', True) if cfg else True

        if shared_telemetry or int(plant_id) == 1:
            readings = self.sensor_repo.get_by_date(date_str)
        else:
            readings = [s.sensor_reading for s in sessions if s.sensor_reading]

        session_list = []
        for s in sessions:
            avg_m = s.get_average_metric()
            cenital_url = ("/static/" + avg_m.image_path_cenital_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/")) if (avg_m and
avg_m.image_path_cenital_proc) else ""
            lateral_url = ("/static/" + avg_m.image_path_lateral_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/")) if (avg_m and
avg_m.image_path_lateral_proc) else ""

            raw_p = s.period or "Muestreo"
            p_clean = raw_p.split(" - ")[1].strip().capitalize() if " - " in raw_p else raw_p.capitalize()

            session_list.append({
                "id": s.id,
                "period": p_clean,
                "timestamp": s.timestamp.strftime("%H:%M:%S"),
                "foliar_area_cm2": avg_m.foliar_area_cm2 if avg_m else 0.0,
                "plant_height_cm": avg_m.plant_height_cm if avg_m else 0.0,
```

```
                "stem_diameter_mm": avg_m.stem_diameter_mm if avg_m else 0.0,
                "health_index": avg_m.health_index if avg_m else 100.0,
                "cenital_url": cenital_url,
                "lateral_url": lateral_url,
                "thumbnail_url": cenital_url or lateral_url
            })

        env_summary = {
            "has_readings": len(readings) > 0,
            "readings_count": len(readings),
            "avg_temp": round(float(np.mean([r.temperature for r in readings])), 1) if readings else 0.0,
            "min_temp": round(float(np.min([r.temperature for r in readings])), 1) if readings else 0.0,
            "max_temp": round(float(np.max([r.temperature for r in readings])), 1) if readings else 0.0,
            "avg_hum": round(float(np.mean([r.humidity for r in readings])), 1) if readings else 0.0,
            "min_hum": round(float(np.min([r.humidity for r in readings])), 1) if readings else 0.0,
            "max_hum": round(float(np.max([r.humidity for r in readings])), 1) if readings else 0.0,
            "avg_uv": round(float(np.mean([r.uv_solar for r in readings])), 1) if readings else 0.0,
            "avg_current": round(float(np.mean([r.motor_current for r in readings])), 3) if readings else 0.0,
            "avg_ec": round(float(np.mean([(r.ec_conductivity if r.ec_conductivity is not None else 1650.0) for r in readings])), 1) if readings else 1650.0,
            "avg_ph": round(float(np.mean([(r.water_ph if r.water_ph is not None else 6.20) for r in readings])), 2) if readings else 6.20,
            "avg_turbidity": round(float(np.mean([(r.turbidity_ntu if r.turbidity_ntu is not None else 3.8) for r in readings])), 2) if readings else 3.8,
            "avg_do": round(float(np.mean([(r.dissolved_oxygen_mg_l if r.dissolved_oxygen_mg_l is not None else 7.4) for r in readings])), 2) if readings else 7.4,
            "avg_water_temp": round(float(np.mean([(r.water_temp if getattr(r, 'water_temp', None) is not None else 20.0) for r in readings])), 1) if readings else 20.0,
            "motor_active_pct": round((sum([(r.motor_state if getattr(r, 'motor_state', None) is not None else (1 if r.motor_current > 0.1 else 0)) for r in readings]) / max(len(readings), 1)) * 100.0, 1) if readings else 0.0
        }

        return {
            "status": "success",
            "date_str": date_str,
            "plant_id": plant_id,
            "has_photos": len(session_list) > 0,
            "photo_count": len(session_list),
            "sessions": session_list,
            "environment": env_summary
        }

    def get_inter_plant_benchmark_data(self) -> Dict[str, Any]:
        """
        Calcula la comparativa inter-canastillas de la torre hidropónica:
        - Curvas superpuestas de área foliar, altura, tallo y salud.
        - Análisis de gradiente vertical por nivel en la torre.
        - Tasa de Crecimiento Relativo (RGR) y Tasa de Crecimiento Absoluto (AGR).
        """
        from domain.models import CropProfile
        cfg = self.config_repo.get() if self.config_repo else None

        crop_names = {
            1: getattr(cfg, 'plant_1_crop', 'cebollin') if cfg else 'cebollin',
            2: getattr(cfg, 'plant_2_crop', 'albahaca') if cfg else 'albahaca',
            3: getattr(cfg, 'plant_3_crop', 'lechuga') if cfg else 'lechuga',
            4: getattr(cfg, 'plant_4_crop', 'fresa') if cfg else 'fresa'
        }

        LEVEL_NAMES = {
            1: "Nivel 1 - Base (Inferior)",
```

```
            2: "Nivel 2 - Medio-Bajo",
            3: "Nivel 3 - Medio-Alto",
            4: "Nivel 4 - Cúspide (Superior)"
        }

        all_dates_set = set()
        canastillas_data = {}

        CANASTILLA_COLORS = {
            1: {"border": "#10b981", "bg": "rgba(16, 185, 129, 0.15)", "name": "Verde Esmeralda"},
            2: {"border": "#0284c7", "bg": "rgba(2, 132, 199, 0.15)", "name": "Azul Océano"},
            3: {"border": "#8b5cf6", "bg": "rgba(139, 92, 246, 0.15)", "name": "Púrpura"},
            4: {"border": "#f59e0b", "bg": "rgba(245, 158, 11, 0.15)", "name": "Ámbar"}
        }

        for pid in [1, 2, 3, 4]:
            crop_key = crop_names.get(pid, 'cebollin')
            profile = CropProfile.query.filter_by(crop_type=crop_key).first()
            display_name = profile.display_name if profile else crop_key.capitalize()
            scientific_name = ""

            sessions = self.session_repo.get_all_by_plant(plant_id=pid, order_asc=True)

            date_metrics = {}
            for s in sessions:
                d_str = s.timestamp.strftime("%Y-%m-%d")
                all_dates_set.add(d_str)
                m = s.get_average_metric()
                if d_str not in date_metrics:
                    date_metrics[d_str] = {"areas": [], "heights": [], "stems": [], "healths": []}
                if m:
                    if m.foliar_area_cm2: date_metrics[d_str]["areas"].append(m.foliar_area_cm2)
                    if m.plant_height_cm: date_metrics[d_str]["heights"].append(m.plant_height_cm)
                    if m.stem_diameter_mm: date_metrics[d_str]["stems"].append(m.stem_diameter_mm)
                    if m.health_index: date_metrics[d_str]["healths"].append(m.health_index)

            daily_series = []
            for d_str in sorted(list(date_metrics.keys())):
                dm = date_metrics[d_str]
                avg_area = round(float(np.mean(dm["areas"])), 2) if dm["areas"] else 0.0
                avg_height = round(float(np.mean(dm["heights"])), 2) if dm["heights"] else 0.0
                avg_stem = round(float(np.mean(dm["stems"])), 2) if dm["stems"] else 0.0
                avg_health = round(float(np.mean(dm["healths"])), 1) if dm["healths"] else 100.0
                daily_series.append({
                    "date": d_str,
                    "area_cm2": avg_area,
                    "height_cm": avg_height,
                    "stem_mm": avg_stem,
                    "health_index": avg_health,
                    "samples_count": len(dm["areas"])
                })

            initial_area = daily_series[0]["area_cm2"] if daily_series else 0.0
            latest_area = daily_series[-1]["area_cm2"] if daily_series else 0.0
            total_area_gain = round(latest_area - initial_area, 2)
```

```
            initial_height = daily_series[0]["height_cm"] if daily_series else 0.0
            latest_height = daily_series[-1]["height_cm"] if daily_series else 0.0
            total_height_gain = round(latest_height - initial_height, 2)

            days_count = len(daily_series)
            rgr_pct_day = 0.0
            agr_cm2_day = 0.0
            if days_count > 1 and initial_area > 0 and latest_area > 0:
                try:
                    rgr_pct_day = round(((np.log(latest_area) - np.log(initial_area)) / max(days_count - 1, 1)) * 100, 2)
                    agr_cm2_day = round((latest_area - initial_area) / max(days_count - 1, 1), 2)
                except Exception:
                    pass

            avg_health_overall = round(float(np.mean([d["health_index"] for d in daily_series])), 1) if daily_series else 0.0

            canastillas_data[pid] = {
                "plant_id": pid,
                "crop_key": crop_key,
                "display_name": display_name,
                "scientific_name": scientific_name,
                "level_name": LEVEL_NAMES[pid],
                "color": CANASTILLA_COLORS[pid],
                "has_data": len(daily_series) > 0,
                "sessions_count": len(sessions),
                "days_count": days_count,
                "initial_area": initial_area,
                "latest_area": latest_area,
                "total_area_gain": total_area_gain,
                "initial_height": initial_height,
                "latest_height": latest_height,
                "total_height_gain": total_height_gain,
                "rgr_pct_day": rgr_pct_day,
                "agr_cm2_day": agr_cm2_day,
                "avg_health_overall": avg_health_overall,
                "daily_series": daily_series
            }

        sorted_dates = sorted(list(all_dates_set))
        chart_labels = sorted_dates
        chart_datasets_area = []
        chart_datasets_height = []
        chart_datasets_health = []

        for pid in [1, 2, 3, 4]:
            c_info = canastillas_data[pid]
            series_map = {item["date"]: item for item in c_info["daily_series"]}

            area_data = [series_map.get(d, {}).get("area_cm2", None) for d in sorted_dates]
            height_data = [series_map.get(d, {}).get("height_cm", None) for d in sorted_dates]
            health_data = [series_map.get(d, {}).get("health_index", None) for d in sorted_dates]

            label_str = f"Canastilla #{pid}: {c_info['display_name']}"
```

```
            chart_datasets_area.append({
                "label": label_str,
                "data": area_data,
                "borderColor": c_info["color"]["border"],
                "backgroundColor": c_info["color"]["bg"],
                "borderWidth": 2.5,
                "fill": False,
                "tension": 0.3,
                "spanGaps": True
            })
            chart_datasets_height.append({
                "label": label_str,
                "data": height_data,
                "borderColor": c_info["color"]["border"],
                "backgroundColor": c_info["color"]["bg"],
                "borderWidth": 2.5,
                "fill": False,
                "tension": 0.3,
                "spanGaps": True
            })
            chart_datasets_health.append({
                "label": label_str,
                "data": health_data,
                "borderColor": c_info["color"]["border"],
                "backgroundColor": c_info["color"]["bg"],
                "borderWidth": 2.5,
                "fill": False,
                "tension": 0.3,
                "spanGaps": True
            })

        gradient_ranking = sorted(
            [c for c in canastillas_data.values() if c["has_data"]],
            key=lambda x: x["latest_area"],
            reverse=True
        )

        return {
            "canastillas": canastillas_data,
            "sorted_dates": sorted_dates,
            "chart_labels": chart_labels,
            "charts": {
                "area": chart_datasets_area,
                "height": chart_datasets_height,
                "health": chart_datasets_health
            },
            "gradient_ranking": gradient_ranking
        }

    def get_timelapse_data(self, plant_id: int = 1) -> Dict[str, Any]:
        """
        Prepara el dataset de fotogramas secuenciales para el Time-Lapse interactivo
        de evolución biológica de la canastilla seleccionada.
        """
        from domain.models import CropProfile
```

```
        cfg = self.config_repo.get() if self.config_repo else None

        crop_names = {
            1: getattr(cfg, 'plant_1_crop', 'cebollin') if cfg else 'cebollin',
            2: getattr(cfg, 'plant_2_crop', 'albahaca') if cfg else 'albahaca',
            3: getattr(cfg, 'plant_3_crop', 'lechuga') if cfg else 'lechuga',
            4: getattr(cfg, 'plant_4_crop', 'fresa') if cfg else 'fresa'
        }
        crop_key = crop_names.get(int(plant_id), 'cebollin')
        profile = CropProfile.query.filter_by(crop_type=crop_key).first()

        sessions = self.session_repo.get_all_by_plant(plant_id=int(plant_id), order_asc=True)

        frames = []
        for idx, s in enumerate(sessions):
            avg_m = s.get_average_metric()
            cenital_url = ("/static/" + avg_m.image_path_cenital_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/")) if (avg_m and avg_m.image_path_cenital_proc) else ""
            lateral_url = ("/static/" + avg_m.image_path_lateral_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/")) if (avg_m and avg_m.image_path_lateral_proc) else ""

            raw_p = s.period or f"Sesión {idx + 1}"
            p_clean = raw_p.split(" - ")[1].strip().capitalize() if " - " in raw_p else raw_p.capitalize()

            sensor = s.sensor_reading
            temp_val = sensor.temperature if sensor else 0.0
            hum_val = sensor.humidity if sensor else 0.0
            uv_val = sensor.uv_solar if sensor else 0.0
            curr_val = sensor.motor_current if sensor else 0.0

            frames.append({
                "frame_index": idx,
                "session_id": s.id,
                "date_str": s.timestamp.strftime("%Y-%m-%d"),
                "time_str": s.timestamp.strftime("%H:%M:%S"),
                "full_timestamp": s.timestamp.strftime("%d/%m/%Y %H:%M"),
                "period": p_clean,
                "cenital_url": cenital_url,
                "lateral_url": lateral_url,
                "foliar_area_cm2": avg_m.foliar_area_cm2 if avg_m else 0.0,
                "plant_height_cm": avg_m.plant_height_cm if avg_m else 0.0,
                "stem_diameter_mm": avg_m.stem_diameter_mm if avg_m else 0.0,
                "health_index": avg_m.health_index if avg_m else 100.0,
                "compacity_index": avg_m.compacity_index if avg_m else 0.0,
                "temperature": temp_val,
                "humidity": hum_val,
                "uv_solar": uv_val,
                "motor_current": curr_val
            })

        return {
            "status": "success",
            "plant_id": int(plant_id),
            "crop_key": crop_key,
            "crop_name": profile.display_name if profile else crop_key.capitalize(),
            "total_frames": len(frames),
```

```
            "has_frames": len(frames) > 0,
            "frames": frames
        }

    def get_scientific_dossier_data(self, target_date: Optional[str] = None, plant_id: int = 1) -> Dict[str, Any]:
        """
        Construye el dossier científico exhaustivo de la jornada para la generación del
        Reporte / Ficha Técnica de Investigación en PDF con rigor agronómico y estadístico.
        """
        from domain.models import CropProfile, AgronomicConclusion
        cfg = self.config_repo.get() if self.config_repo else None

        crop_names = {
            1: getattr(cfg, 'plant_1_crop', 'cebollin') if cfg else 'cebollin',
            2: getattr(cfg, 'plant_2_crop', 'albahaca') if cfg else 'albahaca',
            3: getattr(cfg, 'plant_3_crop', 'lechuga') if cfg else 'lechuga',
            4: getattr(cfg, 'plant_4_crop', 'fresa') if cfg else 'fresa'
        }
        crop_key = crop_names.get(int(plant_id), 'cebollin')
        profile = CropProfile.query.filter_by(crop_type=crop_key).first()
        display_name = profile.display_name if profile else crop_key.capitalize()

        LEVEL_NAMES = {
            1: "Nivel 1 - Base (Estrato Inferior)",
            2: "Nivel 2 - Medio-Bajo",
            3: "Nivel 3 - Medio-Alto",
            4: "Nivel 4 - Cúspide (Estrato Superior)"
        }

        available_dates = self.get_available_dates(plant_id)
        if not target_date or target_date == "auto":
            target_date = available_dates[0] if available_dates else datetime.now().strftime("%Y-%m-%d")

        sessions = self.session_repo.get_by_date_and_plant(target_date, int(plant_id))

        shared_telemetry = getattr(cfg, 'shared_telemetry', True) if cfg else True
        if shared_telemetry or int(plant_id) == 1:
            readings = self.sensor_repo.get_by_date(target_date)
        else:
            readings = [s.sensor_reading for s in sessions if s.sensor_reading]

        def compute_stats(values):
            if not values:
                return {
                    "count": 0, "mean": 0.0, "std": 0.0, "var": 0.0,
                    "min": 0.0, "max": 0.0, "median": 0.0, "iqr": 0.0,
                    "cv_pct": 0.0, "range": 0.0
                }
            arr = np.array(values, dtype=float)
            mean_val = float(np.mean(arr))
            std_val = float(np.std(arr, ddof=1)) if len(arr) > 1 else 0.0
            var_val = float(np.var(arr, ddof=1)) if len(arr) > 1 else 0.0
            min_val = float(np.min(arr))
            max_val = float(np.max(arr))
            median_val = float(np.median(arr))
```

```
            q75, q25 = np.percentile(arr, [75, 25])
            iqr_val = float(q75 - q25)
            cv_val = float((std_val / mean_val) * 100) if mean_val != 0 else 0.0
            return {
                "count": len(arr),
                "mean": round(mean_val, 2),
                "std": round(std_val, 3),
                "var": round(var_val, 4),
                "min": round(min_val, 2),
                "max": round(max_val, 2),
                "median": round(median_val, 2),
                "iqr": round(iqr_val, 2),
                "cv_pct": round(cv_val, 2),
                "range": round(max_val - min_val, 2)
            }

        temp_stats = compute_stats([r.temperature for r in readings if r.temperature is not None])
        hum_stats = compute_stats([r.humidity for r in readings if r.humidity is not None])
        uv_stats = compute_stats([r.uv_solar for r in readings if r.uv_solar is not None])
        curr_stats = compute_stats([r.motor_current for r in readings if r.motor_current is not None])
        ec_stats = compute_stats([(r.ec_conductivity if r.ec_conductivity is not None else 1650.0) for r in readings])
        ph_stats = compute_stats([(r.water_ph if r.water_ph is not None else 6.20) for r in readings])
        turb_stats = compute_stats([(r.turbidity_ntu if r.turbidity_ntu is not None else 3.8) for r in readings])
        do_stats = compute_stats([(r.dissolved_oxygen_mg_l if r.dissolved_oxygen_mg_l is not None else 7.4) for r in readings])

        sensor_stats_table = [
            {"param": "Temperatura Ambiental", "symbol": "T_amb", "unit": "°C", "stats": temp_stats, "optimal": "20.0 - 28.0 °C"},
            {"param": "Humedad Relativa", "symbol": "HR", "unit": "%", "stats": hum_stats, "optimal": "60.0 - 75.0 %"},
            {"param": "Radiación Solar / UV", "symbol": "Rad_UV", "unit": "lux", "stats": uv_stats, "optimal": "300 - 800 lux"},
            {"param": "Corriente de Bomba", "symbol": "I_pump", "unit": "A", "stats": curr_stats, "optimal": "0.35 - 0.60 A (Activa)"},
            {"param": "Conductividad Eléctrica", "symbol": "EC", "unit": "µS/cm", "stats": ec_stats, "optimal": "1200 - 2400 µS/cm"},
            {"param": "pH Solución Nutritiva", "symbol": "pH", "unit": "pH", "stats": ph_stats, "optimal": "5.5 - 6.8 pH"},
            {"param": "Turbidez del Agua", "symbol": "Turb", "unit": "NTU", "stats": turb_stats, "optimal": "1.5 - 12.0 NTU"},
            {"param": "Oxígeno Disuelto", "symbol": "DO", "unit": "mg/L", "stats": do_stats, "optimal": "6.0 - 9.0 mg/L"}
        ]

        areas = []
        heights = []
        stems = []
        healths = []
        compacities = []

        detailed_sessions_list = []
        photo_comparisons = []

        for s in sessions:
            avg_m = s.get_average_metric()
            if avg_m:
                if avg_m.foliar_area_cm2: areas.append(avg_m.foliar_area_cm2)
                if avg_m.plant_height_cm: heights.append(avg_m.plant_height_cm)
                if avg_m.stem_diameter_mm: stems.append(avg_m.stem_diameter_mm)
                if avg_m.health_index is not None: healths.append(avg_m.health_index)
                if avg_m.compacity_index: compacities.append(avg_m.compacity_index)

            cenital_orig = ""
```

```
            cenital_proc = ""
            lateral_orig = ""
            lateral_proc = ""

            if avg_m:
                if avg_m.image_path_cenital_orig:
                    cenital_orig = "/static/" + avg_m.image_path_cenital_orig.replace("static/", "").replace("local_server/static/", "").lstrip("/")
                if avg_m.image_path_cenital_proc:
                    cenital_proc = "/static/" + avg_m.image_path_cenital_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/")
                if avg_m.image_path_lateral_orig:
                    lateral_orig = "/static/" + avg_m.image_path_lateral_orig.replace("static/", "").replace("local_server/static/", "").lstrip("/")
                if avg_m.image_path_lateral_proc:
                    lateral_proc = "/static/" + avg_m.image_path_lateral_proc.replace("static/", "").replace("local_server/static/", "").lstrip("/")

            raw_p = s.period or "Muestreo"
            p_clean = raw_p.split(" - ")[1].strip().capitalize() if " - " in raw_p else raw_p.capitalize()

            sensor_linked = s.sensor_reading
            t_val = sensor_linked.temperature if sensor_linked else 0.0
            h_val = sensor_linked.humidity if sensor_linked else 0.0
            u_val = sensor_linked.uv_solar if sensor_linked else 0.0
            c_val = sensor_linked.motor_current if sensor_linked else 0.0
            ec_val = (sensor_linked.ec_conductivity if sensor_linked and sensor_linked.ec_conductivity is not None else 1650.0)
            ph_val = (sensor_linked.water_ph if sensor_linked and sensor_linked.water_ph is not None else 6.20)
            turb_val = (sensor_linked.turbidity_ntu if sensor_linked and sensor_linked.turbidity_ntu is not None else 3.8)
            do_val = (sensor_linked.dissolved_oxygen_mg_l if sensor_linked and sensor_linked.dissolved_oxygen_mg_l is not None else 7.4)

            session_dict = {
                "id": s.id,
                "period": p_clean,
                "timestamp": s.timestamp.strftime("%H:%M:%S"),
                "foliar_area_cm2": round(avg_m.foliar_area_cm2, 2) if avg_m else 0.0,
                "plant_height_cm": round(avg_m.plant_height_cm, 2) if avg_m else 0.0,
                "stem_diameter_mm": round(avg_m.stem_diameter_mm, 2) if avg_m else 0.0,
                "health_index": round(avg_m.health_index, 1) if avg_m else 100.0,
                "compacity_index": round(avg_m.compacity_index, 3) if avg_m else 0.0,
                "temperature": t_val,
                "humidity": h_val,
                "uv_solar": u_val,
                "motor_current": c_val,
                "ec_conductivity": ec_val,
                "water_ph": ph_val,
                "turbidity_ntu": turb_val,
                "dissolved_oxygen_mg_l": do_val,
                "cenital_orig": cenital_orig,
                "cenital_proc": cenital_proc,
                "lateral_orig": lateral_orig,
                "lateral_proc": lateral_proc
            }
            detailed_sessions_list.append(session_dict)

            if cenital_proc or lateral_proc:
                photo_comparisons.append({
                    "period": p_clean,
                    "time": s.timestamp.strftime("%H:%M:%S"),
```

```
                    "cenital_orig": cenital_orig or cenital_proc,
                    "cenital_proc": cenital_proc or cenital_orig,
                    "lateral_orig": lateral_orig or lateral_proc,
                    "lateral_proc": lateral_proc or lateral_orig,
                    "area_cm2": round(avg_m.foliar_area_cm2, 2) if avg_m else 0.0,
                    "height_cm": round(avg_m.plant_height_cm, 2) if avg_m else 0.0,
                    "health_index": round(avg_m.health_index, 1) if avg_m else 100.0
                })

        biometric_stats_table = [
            {"param": "Área Foliar Cenital", "symbol": "AF", "unit": "cm²", "stats": compute_stats(areas)},
            {"param": "Altura de Planta", "symbol": "H_planta", "unit": "cm", "stats": compute_stats(heights)},
            {"param": "Diámetro de Tallo", "symbol": "D_tallo", "unit": "mm", "stats": compute_stats(stems)},
            {"param": "Índice de Salud Foliar", "symbol": "ISF", "unit": "%", "stats": compute_stats(healths)},
            {"param": "Índice de Compacidad", "symbol": "Comp", "unit": "adim.", "stats": compute_stats(compacities)}
        ]

        correlation_matrix = []
        if len(detailed_sessions_list) >= 2:
            x_temp = [s["temperature"] for s in detailed_sessions_list]
            x_hum = [s["humidity"] for s in detailed_sessions_list]
            x_uv = [s["uv_solar"] for s in detailed_sessions_list]
            y_area = [s["foliar_area_cm2"] for s in detailed_sessions_list]
            y_height = [s["plant_height_cm"] for s in detailed_sessions_list]
            y_health = [s["health_index"] for s in detailed_sessions_list]

            def calc_pearson(v1, v2):
                if len(v1) < 2 or np.std(v1) == 0 or np.std(v2) == 0:
                    return 0.0, "Sin variación suficiente"
                r = float(np.corrcoef(v1, v2)[0, 1])
                if np.isnan(r): return 0.0, "N/A"
                r_rounded = round(r, 3)
                if r_rounded >= 0.7: desc = "Correlación Fuerte Positiva (+)"
                elif r_rounded >= 0.3: desc = "Correlación Moderada Positiva (+)"
                elif r_rounded > -0.3: desc = "Sin Correlación Lineal Significativa"
                elif r_rounded > -0.7: desc = "Correlación Moderada Inversa (-)"
                else: desc = "Correlación Fuerte Inversa (-)"
                return r_rounded, desc

            r1, desc1 = calc_pearson(x_temp, y_area)
            correlation_matrix.append({"pair": "Temperatura (°C) vs Área Foliar (cm²)", "r": r1, "desc": desc1})

            r2, desc2 = calc_pearson(x_hum, y_area)
            correlation_matrix.append({"pair": "Humedad Relativa (%) vs Área Foliar (cm²)", "r": r2, "desc": desc2})

            r3, desc3 = calc_pearson(x_temp, y_health)
            correlation_matrix.append({"pair": "Temperatura (°C) vs Salud Foliar (%)", "r": r3, "desc": desc3})

            r4, desc4 = calc_pearson(x_uv, y_height)
            correlation_matrix.append({"pair": "Radiación Solar (lux) vs Altura (cm)", "r": r4, "desc": desc4})
        else:
            correlation_matrix = [
                {"pair": "Temperatura (°C) vs Área Foliar (cm²)", "r": 0.0, "desc": "Requiere al menos 2 muestreos"},
                {"pair": "Humedad Relativa (%) vs Área Foliar (cm²)", "r": 0.0, "desc": "Requiere al menos 2 muestreos"},
                {"pair": "Temperatura (°C) vs Salud Foliar (%)", "r": 0.0, "desc": "Requiere al menos 2 muestreos"}
```

```
            ]

        conclusions = AgronomicConclusion.query.filter_by(date_str=target_date, plant_id=int(plant_id)).order_by(AgronomicConclusion.timestamp.desc()).all()

        automated_insights = []
        if temp_stats["mean"] > 30.0:
            automated_insights.append({
                "type": "warning",
                "title": "Alerta de Estrés Térmico Registrado",
                "detail": f"La temperatura promedio de la jornada ({temp_stats['mean']} °C) superó el umbral de confort ({profile.h_max if profile else '28'} °C), lo cual puede acelerar la evapotranspiración foliar."
            })
        if hum_stats["mean"] < 50.0 and hum_stats["mean"] > 0:
            automated_insights.append({
                "type": "warning",
                "title": "Déficit de Humedad Relativa",
                "detail": f"La humedad media ({hum_stats['mean']} %) se situó por debajo del rango óptimo para sistemas hidropónicos, incrementando el riesgo de desecación foliar."
            })
        if len(areas) >= 2 and areas[-1] > areas[0]:
            gain = round(areas[-1] - areas[0], 2)
            automated_insights.append({
                "type": "positive",
                "title": "Expansión Foliar Diaria Positiva",
                "detail": f"Se cuantificó un incremento de área fotosintéticamente activa de +{gain} cm² a lo largo de las sesiones del día."
            })

        chart_series = {
            "sensors": {
                "timestamps": [r.timestamp.strftime("%H:%M") for r in readings],
                "temp": [round(r.temperature, 2) for r in readings if r.temperature is not None],
                "hum": [round(r.humidity, 1) for r in readings if r.humidity is not None],
                "uv": [round(r.uv_solar, 1) for r in readings if r.uv_solar is not None],
                "current": [round(r.motor_current, 3) for r in readings if r.motor_current is not None],
                "ec": [round(r.ec_conductivity, 1) if r.ec_conductivity is not None else 1650.0 for r in readings],
                "ph": [round(r.water_ph, 2) if r.water_ph is not None else 6.2 for r in readings],
                "turb": [round(r.turbidity_ntu, 2) if r.turbidity_ntu is not None else 3.8 for r in readings],
                "do": [round(r.dissolved_oxygen_mg_l, 2) if r.dissolved_oxygen_mg_l is not None else 7.4 for r in readings]
            },
            "vision": {
                "timestamps": [s.timestamp.strftime("%H:%M") for s in sessions],
                "periods": [s.period or f"Muestreo #{s.id}" for s in sessions],
                "foliar_area": [round(s.get_average_metric().foliar_area_cm2, 2) if s.get_average_metric() else 0.0 for s in sessions],
                "height": [round(s.get_average_metric().plant_height_cm, 2) if s.get_average_metric() else 0.0 for s in sessions],
                "stem": [round(s.get_average_metric().stem_diameter_mm, 2) if s.get_average_metric() else 0.0 for s in sessions],
                "health": [round(s.get_average_metric().health_index, 1) if s.get_average_metric() else 100.0 for s in sessions]
            }
        }

        report_id = f"SIFMA-REP-{target_date.replace('-', '')}-CAN{plant_id}"

        return {
            "status": "success",
            "report_id": report_id,
            "target_date": target_date,
            "plant_id": int(plant_id),
```

```
            "display_name": display_name,
            "crop_key": crop_key,
            "level_name": LEVEL_NAMES.get(int(plant_id), f"Canastilla #{plant_id}"),
            "available_dates": available_dates,
            "sensor_stats_table": sensor_stats_table,
            "biometric_stats_table": biometric_stats_table,
            "correlation_matrix": correlation_matrix,
            "detailed_sessions": detailed_sessions_list,
            "photo_comparisons": photo_comparisons,
            "conclusions": [c.to_dict() for c in conclusions],
            "automated_insights": automated_insights,
            "chart_series": chart_series,
            "has_data": (len(sessions) > 0 or len(readings) > 0),
            "generated_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        }
```

ARCHIVO 9/85: local_server/application/services/audit_service.py (157 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import json
import logging
from datetime import datetime
from typing import List, Optional, Dict, Any
from flask import has_request_context, session, request
from domain.models import AuditLog
from infrastructure.database.repositories import AuditRepository

logger = logging.getLogger(__name__)

class AuditService:
    """
    Servicio de Auditoría y Trazabilidad de Acciones del Sistema SIFMA.
    Registra operaciones realizadas por los usuarios (Super Usuario, Investigadores, Técnicos).
    """
    _instance = None

    def __init__(self, audit_repo: Optional[AuditRepository] = None):
        self.audit_repo = audit_repo or AuditRepository()

    @classmethod
    def get_instance(cls) -> "AuditService":
        if cls._instance is None:
            cls._instance = cls()
        return cls._instance

    def log(
        self,
        action: str,
        module: str,
        description: str,
        level: str = "info",
        user_id: Optional[int] = None,
        username: Optional[str] = None,
        user_fullname: Optional[str] = None,
        user_role: Optional[str] = None,
        ip_address: Optional[str] = None,
        details: Optional[Dict[str, Any]] = None,
        action_display: Optional[str] = None
    ) -> Optional[AuditLog]:
        """
        Registra una acción en la base de datos de auditoría.
        Si no se especifican los datos de usuario, intenta inferirlos del contexto de sesión Flask actual.
        """
        try:
            # Inferir contexto desde la petición Flask si está disponible
            if has_request_context():
                if user_id is None:
                    user_id = session.get('user_id')
```

```
                if username is None:
                    username = session.get('username')
                if user_fullname is None:
                    user_fullname = session.get('user_name')
                if user_role is None:
                    user_role = session.get('user_role')
                if ip_address is None:
                    ip_address = request.headers.get('X-Forwarded-For', request.remote_addr)

            clean_username = (username or 'sistema').strip()
            details_str = json.dumps(details, ensure_ascii=False) if details else None

            log_entry = AuditLog(
                action=action.strip().upper(),
                action_display=action_display,
                module=module.strip().upper(),
                description=description.strip(),
                level=level,
                username=clean_username,
                user_id=user_id,
                user_fullname=user_fullname,
                user_role=user_role,
                ip_address=ip_address,
                details_json=details_str,
                timestamp=datetime.now()
            )

            saved = self.audit_repo.save(log_entry)
            logger.info(f"[AUDIT] [{module}] [{action}] {clean_username}: {description}")
            return saved
        except Exception as e:
            logger.error(f"Error al registrar log de auditoría: {e}", exc_info=True)
            return None

    def get_logs(
        self,
        limit: int = 250,
        offset: int = 0,
        module: Optional[str] = None,
        action: Optional[str] = None,
        level: Optional[str] = None,
        username: Optional[str] = None,
        search: Optional[str] = None,
        start_date: Optional[datetime] = None,
        end_date: Optional[datetime] = None
    ) -> List[AuditLog]:
        return self.audit_repo.get_logs(
            limit=limit,
            offset=offset,
            module=module,
            action=action,
            level=level,
            username=username,
            search=search,
            start_date=start_date,
```

```
            end_date=end_date
        )

    def get_stats(self) -> Dict[str, Any]:
        return self.audit_repo.get_stats()

    def delete_log(self, log_id: int) -> bool:
        """Elimina un registro individual de auditoría."""
        return self.audit_repo.delete_by_id(log_id)

    def clear_all_logs(self) -> int:
        """Elimina todos los registros de auditoría almacenados."""
        return self.audit_repo.delete_all()

    def generate_csv(self, logs: List[AuditLog]) -> str:
        """Genera el contenido CSV formateado del historial de logs."""
        import io
        import csv

        output = io.StringIO()
        writer = csv.writer(output, quoting=csv.QUOTE_MINIMAL)
        writer.writerow([
            "ID",
            "Fecha y Hora",
            "Usuario",
            "Nombre Completo",
            "Rol",
            "Módulo",
            "Acción",
            "Nivel",
            "Descripción",
            "Dirección IP"
        ])

        for log in logs:
            writer.writerow([
                log.id,
                log.timestamp.strftime('%Y-%m-%d %H:%M:%S'),
                log.username,
                log.user_fullname or "",
                log.user_role or "",
                log.module,
                log.action_display or log.action,
                log.level.upper(),
                log.description,
                log.ip_address or "127.0.0.1"
            ])

        return output.getvalue()
```

ARCHIVO 10/85: local_server/application/services/conclusion_service.py (36 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from typing import List, Optional, Dict, Any
from domain.models import AgronomicConclusion
from infrastructure.database.repositories import ConclusionRepository

class ConclusionApplicationService:
    """Caso de uso: Gestión de notas y bitácora agronómica de investigación."""

    def __init__(self, conclusion_repo: Optional[ConclusionRepository] = None):
        self.conclusion_repo = conclusion_repo or ConclusionRepository()

    def get_conclusions(self, plant_id: int, date_str: Optional[str] = None, period_type: Optional[str] = None) -> List[AgronomicConclusion]:
        return self.conclusion_repo.get_filtered(plant_id, date_str, period_type)

    def create_conclusion(self, data: Dict[str, Any]) -> AgronomicConclusion:
        note = AgronomicConclusion(
            plant_id=int(data.get("plant_id", 1)),
            date_str=data.get("date_str"),
            period_type=data.get("period_type", "diario"),
            growth_obs=data.get("growth_obs", ""),
            climate_obs=data.get("climate_obs", ""),
            nutrition_obs=data.get("nutrition_obs", ""),
            general_conclusion=data.get("general_conclusion", ""),
            author=data.get("author", "Investigador SIFMA")
        )
        return self.conclusion_repo.save(note)

    def delete_conclusion(self, conclusion_id: int) -> bool:
        note = self.conclusion_repo.get_by_id(conclusion_id)
        if not note:
            return False
        self.conclusion_repo.delete(note)
        return True
```

ARCHIVO 11/85: local_server/application/services/system_service.py (68 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from typing import List, Optional, Dict
from domain.models import Config, CropProfile
from infrastructure.database.repositories import ConfigRepository, CropProfileRepository

class SystemService:
    """Caso de uso: Gestión de configuración general del nodo y perfiles de cultivo."""

    def __init__(
        self,
        config_repo: Optional[ConfigRepository] = None,
        crop_repo: Optional[CropProfileRepository] = None
    ):
        self.config_repo = config_repo or ConfigRepository()
        self.crop_repo = crop_repo or CropProfileRepository()

    def get_config(self) -> Config:
        return self.config_repo.get()

    def update_config(self, **kwargs) -> Config:
        return self.config_repo.update(**kwargs)

    def get_all_profiles(self) -> List[CropProfile]:
        return self.crop_repo.get_all()

    def get_profile(self, crop_type: Optional[str] = None) -> CropProfile:
        if not crop_type:
            cfg = self.get_config()
            crop_type = cfg.selected_crop_type
        profile = self.crop_repo.get_by_type(crop_type)
        if not profile:
            profile = self.crop_repo.get_by_type("cebollin") or self.crop_repo.get_all()[0]
        return profile

    def get_crop_for_plant(self, plant_id: int) -> str:
        """Devuelve el tipo de cultivo asignado específicamente a la Canastilla (1 a 4)."""
        cfg = self.get_config()
        attr_name = f"plant_{plant_id}_crop"
        return getattr(cfg, attr_name, None) or cfg.selected_crop_type or "cebollin"

    def set_crop_for_plant(self, plant_id: int, crop_type: str) -> None:
        """Asigna un perfil botánico a una Canastilla específica."""
        attr_name = f"plant_{plant_id}_crop"
        self.update_config(**{attr_name: crop_type})

    def get_profile_for_plant(self, plant_id: int) -> CropProfile:
        """Obtiene la entidad CropProfile correspondiente a la Canastilla."""
        crop_type = self.get_crop_for_plant(plant_id)
        return self.get_profile(crop_type)

```

```
    def get_all_plant_profiles_map(self) -> Dict[int, Dict[str, str]]:
        """Devuelve el mapa completo de Canastillas (1..4) con sus especies y nombres."""
        res = {}
        for pid in range(1, 5):
            c_type = self.get_crop_for_plant(pid)
            prof = self.get_profile(c_type)
            res[pid] = {
                "plant_id": pid,
                "crop_type": c_type,
                "display_name": prof.display_name if prof else c_type.capitalize()
            }
        return res

    def update_profile(self, profile: CropProfile) -> CropProfile:
        return self.crop_repo.save_or_update(profile)
```

ARCHIVO 12/85: local_server/application/services/telemetry_service.py (200 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import csv
import time
import threading
from datetime import datetime
from typing import Dict, Any, List, Optional
from core.config import config
from domain.models import SensorReading
from infrastructure.database.repositories import SensorRepository
from infrastructure.telemetry import SerialTelemetryAdapter, MockTelemetryAdapter
from infrastructure.database.connection import db

class TelemetryApplicationService:
    """
    Caso de uso: Gestión y adquisición continua de telemetría ambiental (Thread-Safe Singleton).
    (SOLID: SRP, DIP, Singleton Pattern)
    """
    _instance = None
    _lock = threading.Lock()

    def __init__(self, sensor_repo: Optional[SensorRepository] = None):
        self.sensor_repo = sensor_repo or SensorRepository()
        self.adapter = SerialTelemetryAdapter()
        self.is_recording = False
        self.thread = None
        self.selected_port = "USB_ANTENNA_AUTO"
        self.current_csv_file = None
        self.latest_reading: Optional[Dict[str, Any]] = None
        self.readings_count = 0
        self.start_time = None

    @classmethod
    def get_instance(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = cls()
            return cls._instance

    def get_available_ports(self) -> List[str]:
        return self.adapter.get_available_ports()

    def start_acquisition(self, port: str = "USB_ANTENNA_AUTO", app=None) -> Dict[str, Any]:
        if self.is_recording:
            return {"status": "already_running", "file": self.current_csv_file}

        available = self.get_available_ports()
        if not available:
            return {
                "status": "error",
```

```
                "message": "No se detectó ninguna antena conectada a los puertos USB. Conecte la antena receptora e intente nuevamente."
            }

        self.selected_port = port
        self.adapter.port = port

        if not self.adapter._connect():
            return {
                "status": "error",
                "message": f"No se pudo establecer comunicación con la antena en el puerto '{port}'. Verifique que esté conectada y no esté ocupada."
            }

        self.is_recording = True
        self.start_time = datetime.now()
        self.readings_count = 0

        telemetry_dir = config.TELEMETRY_DIR
        os.makedirs(telemetry_dir, exist_ok=True)

        date_str = datetime.now().strftime("%Y-%m-%d")
        self.current_csv_file = os.path.join(telemetry_dir, f"telemetria_sensores_{date_str}.csv")

        if not os.path.exists(self.current_csv_file):
            with open(self.current_csv_file, mode='w', newline='', encoding='utf-8') as f:
                writer = csv.writer(f)
                writer.writerow([
                    "Fecha", "Hora", "pH Solución", "Turbidez (NTU)",
                    "Conductividad Eléctrica (µS/cm)", "Oxigeno Disuelto (mg/L)",
                    "temperatura de agua", "estado motor: 1-0",
                    "Temperatura exterior (°C)", "Humedad relativa exterior (% HR)"
                ])

        self.thread = threading.Thread(target=self._telemetry_worker, args=(app,), daemon=True)
        self.thread.start()

        return {
            "status": "started",
            "file": self.current_csv_file,
            "port": self.selected_port,
            "start_time": self.start_time.strftime("%Y-%m-%d %H:%M:%S")
        }

    def stop_acquisition(self) -> Dict[str, Any]:
        self.is_recording = False
        if self.thread and self.thread.is_alive():
            self.thread.join(timeout=2.0)
        self.adapter._disconnect()

        return {
            "status": "stopped",
            "total_readings": self.readings_count,
            "file": self.current_csv_file
        }

    def get_status(self) -> Dict[str, Any]:
```

```
        elapsed_sec = int((datetime.now() - self.start_time).total_seconds()) if (self.is_recording and self.start_time) else 0
        return {
            "is_recording": self.is_recording,
            "port": self.selected_port,
            "file": os.path.basename(self.current_csv_file) if self.current_csv_file else "Ninguno",
            "readings_count": self.readings_count,
            "elapsed_seconds": elapsed_sec,
            "latest_reading": self.latest_reading
        }

    def _telemetry_worker(self, app=None):
        while self.is_recording:
            try:
                sample = self.adapter.read_sample()
                if not sample:
                    time.sleep(1.0)
                    continue

                self.latest_reading = sample

                now_dt = datetime.now()
                now_str = sample.get("timestamp", now_dt.strftime("%Y-%m-%d %H:%M:%S"))
                try:
                    dt_obj = datetime.strptime(now_str, "%Y-%m-%d %H:%M:%S")
                except Exception:
                    dt_obj = now_dt

                ph_val = sample.get("water_ph", 0.0)
                turb_val = sample.get("turbidity_ntu", 0.0)
                ec_val = sample.get("ec_conductivity", 0.0)
                do_val = sample.get("dissolved_oxygen_mg_l", 0.0)
                water_t_val = sample.get("water_temp", 0.0)
                mot_st_val = sample.get("motor_state", 0)
                temp_val = sample.get("temperature", 0.0)
                hum_val = sample.get("humidity", 0.0)

                # 1. Guardar en archivo CSV continuo (con rotación automática si pasa la medianoche)
                telemetry_dir = config.TELEMETRY_DIR
                os.makedirs(telemetry_dir, exist_ok=True)
                current_date_str = dt_obj.strftime("%Y-%m-%d")
                expected_csv_file = os.path.join(telemetry_dir, f"telemetria_sensores_{current_date_str}.csv")
                if self.current_csv_file != expected_csv_file or not os.path.exists(expected_csv_file):
                    self.current_csv_file = expected_csv_file
                    if not os.path.exists(self.current_csv_file):
                        with open(self.current_csv_file, mode='w', newline='', encoding='utf-8') as f:
                            writer = csv.writer(f)
                            writer.writerow([
                                "Fecha", "Hora", "pH Solución", "Turbidez (NTU)",
                                "Conductividad Eléctrica (µS/cm)", "Oxigeno Disuelto (mg/L)",
                                "temperatura de agua", "estado motor: 1-0",
                                "Temperatura exterior (°C)", "Humedad relativa exterior (% HR)"
                            ])

                if self.current_csv_file:
                    with open(self.current_csv_file, mode='a', newline='', encoding='utf-8') as f:
```

```
                    writer = csv.writer(f)
                    writer.writerow([
                        dt_obj.strftime("%d/%m/%Y"),
                        dt_obj.strftime("%I:%M:%S %p"),
                        ph_val,
                        turb_val,
                        ec_val,
                        do_val,
                        water_t_val,
                        mot_st_val,
                        temp_val,
                        hum_val
                    ])

            if app:
                with app.app_context():
                    reading = SensorReading(
                        timestamp=dt_obj,
                        water_ph=ph_val,
                        turbidity_ntu=turb_val,
                        ec_conductivity=ec_val,
                        dissolved_oxygen_mg_l=do_val,
                        water_temp=water_t_val,
                        motor_state=mot_st_val,
                        motor_current=float(mot_st_val),
                        temperature=temp_val,
                        humidity=hum_val,
                        uv_solar=0.0
                    )
                    db.session.add(reading)
                    db.session.commit()

            self.readings_count += 1
            time.sleep(1.0)

        except Exception as e:
            time.sleep(1.0)
```

ARCHIVO 13/85: local_server/application/services/user_service.py (170 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from datetime import datetime
from typing import List, Optional, Dict, Any
from domain.models import User
from infrastructure.database.repositories import UserRepository
from core.security import SecurityService
from core.config import config as app_settings

class UserService:
    """Caso de uso: Gestión de autenticación, usuarios y roles (RBAC)."""

    def __init__(self, user_repo: Optional[UserRepository] = None):
        self.user_repo = user_repo or UserRepository()

    def authenticate(self, identifier: str, plain_password: str) -> Optional[User]:
        if not identifier or not plain_password:
            return None
        clean_id = identifier.strip().lower()
        user = self.user_repo.get_by_username(clean_id) or self.user_repo.get_by_email(clean_id)
        if not user or not user.is_active:
            return None
        if SecurityService.verify_password(plain_password, user.password_hash):
            user.last_login = datetime.now()
            self.user_repo.save(user)
            return user
        return None

    def get_all_users(self) -> List[User]:
        return self.user_repo.get_all()

    def get_user_by_id(self, user_id: int) -> Optional[User]:
        return self.user_repo.get_by_id(user_id)

    def create_user(self, username: str, full_name: str, plain_password: str, role: str = "investigador", email: Optional[str] = None, permissions: Optional[Dict[str, bool]] = None) -> User:
        username_clean = username.strip().lower()
        if self.user_repo.get_by_username(username_clean):
            raise ValueError(f"El nombre de usuario '{username_clean}' ya está en uso.")
        if email and self.user_repo.get_by_email(email):
            raise ValueError(f"El correo '{email}' ya está registrado.")

        allowed_roles = ["investigador", "operador", "personalizado"]
        if role not in allowed_roles:
            role = "investigador"

        password_hash = SecurityService.hash_password(plain_password)
        new_user = User(
            username=username_clean,
            full_name=full_name.strip(),
            email=email.strip().lower() if email else None,
```

```
            password_hash=password_hash,
            role=role,
            is_active=True
        )
        if permissions is not None:
            new_user.set_permissions_dict(permissions)
        else:
            # Asigna los permisos por defecto según el rol elegido
            new_user.set_permissions_dict({})

        return self.user_repo.save(new_user)

    def update_user(self, user_id: int, full_name: Optional[str] = None, role: Optional[str] = None, email: Optional[str] = None, is_active: Optional[bool] = None, permissions: Optional[Dict[str, bool]] = None) -> Optional[User]:
        user = self.user_repo.get_by_id(user_id)
        if not user:
            return None

        if full_name is not None:
            user.full_name = full_name.strip()
        if role is not None:
            if user.is_super_admin:
                user.role = "admin"
            elif role in ["investigador", "operador", "personalizado"]:
                user.role = role
        if email is not None:
            user.email = email.strip().lower() if email else None
        if permissions is not None and not user.is_super_admin:
            user.set_permissions_dict(permissions)
        if is_active is not None:
            if user.is_super_admin and not is_active:
                raise ValueError("No es posible desactivar la cuenta del Super Usuario / Administrador Principal.")
            user.is_active = is_active

        return self.user_repo.save(user)

    def update_password(self, user_id: int, new_password: str) -> bool:
        user = self.user_repo.get_by_id(user_id)
        if not user or not new_password:
            return False
        user.password_hash = SecurityService.hash_password(new_password)
        self.user_repo.save(user)
        return True

    def delete_user(self, user_id: int) -> bool:
        user = self.user_repo.get_by_id(user_id)
        if not user:
            return False
        if user.is_super_admin or user.role == "admin":
            raise ValueError("No es posible eliminar la cuenta del Super Usuario / Administrador Principal.")

        if user.avatar_path:
            self._delete_avatar_file_on_disk(user.avatar_path)

        self.user_repo.delete(user)
        return True
```

```
108
109     def save_user_avatar(self, user_id: int, file_obj, static_folder: Optional[str] = None) -> str:
110         """Guarda la foto de perfil en disco y actualiza el avatar_path del usuario."""
111         import os
112         import time
113
114         user = self.user_repo.get_by_id(user_id)
115         if not user:
116             raise ValueError("Usuario no encontrado.")
117
118         if not file_obj or not file_obj.filename:
119             raise ValueError("No se recibió ningún archivo de imagen válido.")
120
121         ext = file_obj.filename.rsplit('.', 1)[-1].lower() if '.' in file_obj.filename else 'png'
122         if ext not in ['jpg', 'jpeg', 'png', 'webp', 'gif']:
123             raise ValueError("Formato no soportado. Usa imágenes JPG, PNG o WebP.")
124
125         if not static_folder:
126             static_folder = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))), 'static')
127
128         avatars_dir = os.path.join(static_folder, 'uploads', 'avatars')
129         os.makedirs(avatars_dir, exist_ok=True)
130
131         if user.avatar_path:
132             self._delete_avatar_file_on_disk(user.avatar_path, static_folder)
133
134         filename = f"avatar_{user.id}_{int(time.time())}.{ext}"
135         full_dest = os.path.join(avatars_dir, filename)
136         if hasattr(file_obj, 'save'):
137             file_obj.save(full_dest)
138         else:
139             with open(full_dest, 'wb') as f:
140                 f.write(file_obj.read())
141
142         relative_path = f"static/uploads/avatars/{filename}"
143         user.avatar_path = relative_path
144         self.user_repo.save(user)
145         return user.avatar_url
146
147     def remove_user_avatar(self, user_id: int, static_folder: Optional[str] = None) -> bool:
148         """Elimina la foto de perfil del usuario y revierte a las iniciales."""
149         user = self.user_repo.get_by_id(user_id)
150         if not user or not user.avatar_path:
151             return False
152
153         self._delete_avatar_file_on_disk(user.avatar_path, static_folder)
154         user.avatar_path = None
155         self.user_repo.save(user)
156         return True
157
158     def _delete_avatar_file_on_disk(self, avatar_path: str, static_folder: Optional[str] = None):
159         """Elimina de forma segura el archivo del disco."""
160         import os
161         try:
162             if not static_folder:
```

```
                static_folder = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))), 'static')

            clean_rel = avatar_path.lstrip('/').replace('static/', '', 1)
            full_path = os.path.join(static_folder, clean_rel)
            if os.path.exists(full_path):
                os.remove(full_path)
        except Exception:
            pass
```

ARCHIVO 14/85: local_server/application/services/vision_service.py (466 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import glob
import json
import shutil
import re
import platform
from datetime import datetime
from typing import Dict, Any, List, Optional
from core.config import config
from core.security import SecurityService
from domain.models import CaptureSession, BiometricMetric, SensorReading, CropProfile
from infrastructure.database.repositories import (
    CaptureSessionRepository,
    CropProfileRepository,
    SensorRepository,
    ConfigRepository
)
from infrastructure.vision import VisionPipelineManager
from infrastructure.database.connection import db

class VisionApplicationService:
    """
    Caso de uso: Procesamiento de lotes fotográficos, detección USB y persistencia de métricas individuales.
    (SOLID: SRP, DIP)
    """
    _progress = {
        "is_processing": False,
        "percent": 0,
        "message": "Inactivo",
        "current_item": 0,
        "total_items": 0,
        "current_turn": "",
        "detail": ""
    }

    @classmethod
    def get_progress(cls) -> Dict[str, Any]:
        return cls._progress

    @classmethod
    def set_progress(cls, percent: int, message: str, is_processing: bool = True, **kwargs):
        cls._progress["is_processing"] = is_processing
        cls._progress["percent"] = max(0, min(100, percent))
        cls._progress["message"] = message
        for k, v in kwargs.items():
            cls._progress[k] = v

    def __init__(
        self,
```

```
        session_repo: Optional[CaptureSessionRepository] = None,
        crop_repo: Optional[CropProfileRepository] = None,
        sensor_repo: Optional[SensorRepository] = None,
        config_repo: Optional[ConfigRepository] = None,
        pipeline: Optional[VisionPipelineManager] = None
    ):
        self.session_repo = session_repo or CaptureSessionRepository()
        self.crop_repo = crop_repo or CropProfileRepository()
        self.sensor_repo = sensor_repo or SensorRepository()
        self.config_repo = config_repo or ConfigRepository()
        self.pipeline = pipeline or VisionPipelineManager()

    def detect_connected_usb_drives(self) -> List[Dict[str, Any]]:
        """
        Escanea unidades USB externas buscando exclusivamente la carpeta oficial 'SIFMA_CAPTURES'
        y agrupa los lotes de forma limpia por DÍA (YYYY-MM-DD) con sus respectivos turnos.
        Ignora cualquier otra carpeta del disco duro o de la memoria USB.
        """
        usb_days = []
        system = platform.system()
        possible_roots = []

        if system == "Windows":
            import string
            for letter in string.ascii_uppercase:
                if letter not in ['C']:
                    drive = f"{letter}:\\"
                    if os.path.exists(drive):
                        possible_roots.append(drive)
        else:
            possible_roots = ["/media/nodol", "/media/computol", "/media/pi", "/media", "/mnt/usb", "/mnt"]

        for root in possible_roots:
            if not os.path.exists(root):
                continue

            # 1. Localizar la carpeta raíz SIFMA_CAPTURES
            target_sifma_dir = None
            if os.path.exists(os.path.join(root, "SIFMA_CAPTURES")):
                target_sifma_dir = os.path.join(root, "SIFMA_CAPTURES")
            else:
                # Buscar en subdirectorios directos (ej. /media/pi/USB_NAME/SIFMA_CAPTURES)
                try:
                    for d in os.listdir(root):
                        sub = os.path.join(root, d)
                        if os.path.isdir(sub):
                            cand = os.path.join(sub, "SIFMA_CAPTURES")
                            if os.path.exists(cand):
                                target_sifma_dir = cand
                                break
                except Exception:
                    pass

            # Si no hay carpeta SIFMA_CAPTURES en esta unidad, ignorar completamente esta USB
            if not target_sifma_dir or not os.path.exists(target_sifma_dir):
```

```
                continue

            # 2. Escanear exclusivamente las carpetas por fecha YYYY-MM-DD dentro de SIFMA_CAPTURES
            try:
                date_entries = sorted(os.listdir(target_sifma_dir))
                for entry in date_entries:
                    day_path = os.path.join(target_sifma_dir, entry)
                    if not os.path.isdir(day_path):
                        continue

                    # Validar si tiene formato de fecha YYYY-MM-DD
                    is_date = bool(re.match(r'^\d{4}[-_]\d{2}[-_]\d{2}$', entry) or re.match(r'^\d{2}[-_]\d{2}[-_]\d{4}$', entry))
                    date_label = entry.replace('_', '-') if is_date else entry

                    # Contar fotos y recopilar los turnos internos (mañana, mediodía, tarde)
                    turns_found = []
                    total_images = 0

                    for sub_root, _, f_list in os.walk(day_path):
                        imgs = [f for f in f_list if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
                        if imgs:
                            total_images += len(imgs)
                            folder_name = os.path.basename(sub_root).lower()
                            if 'manana' in folder_name or 'mañana' in folder_name:
                                turn_name = "Mañana"
                            elif 'medio_dia' in folder_name or 'mediodia' in folder_name or 'medio-dia' in folder_name:
                                turn_name = "Mediodía"
                            elif 'tarde' in folder_name:
                                turn_name = "Tarde"
                            else:
                                turn_name = folder_name.replace('_', ' ').title()

                            turns_found.append({
                                "period": turn_name,
                                "path": sub_root,
                                "images_count": len(imgs)
                            })

                    if total_images > 0:
                        turns_labels = [t["period"] for t in turns_found]
                        summary_turns = ", ".join(turns_labels) if turns_labels else "Muestreos"

                        usb_days.append({
                            "batch_name": f"{date_label} ({len(turns_found)} turnos - {total_images} fotos)",
                            "date_str": date_label,
                            "period_str": summary_turns,
                            "images_count": total_images,
                            "turns_count": len(turns_found),
                            "turns": turns_found,
                            "path": day_path,
                            "drive_root": root
                        })
            except Exception as e:
                pass
```

```
        usb_days.sort(key=lambda x: x.get("date_str", ""), reverse=True)
        return usb_days

    def process_folder_batch(
        self,
        folder_path: str,
        crop_type: Optional[str] = None,
        period: str = "Día 1",
        sensor_data: Optional[Dict[str, Any]] = None,
        plant_id: int = 1,
        global_offset: int = 0,
        total_global_photos: int = 0,
        turn_label: str = ""
    ) -> Dict[str, Any]:
        """Procesa una carpeta de imágenes, calcula métricas biométricas individuales y promedio."""
        if not os.path.exists(folder_path):
            raise FileNotFoundError(f"La ruta {folder_path} no existe.")

        metadata_file = os.path.join(folder_path, "metadata.json")
        photos_meta_map = {}
        session_timestamp = datetime.now()

        # Obtener el cultivo asignado en el sistema para esta canastilla
        sys_config = self.config_repo.get()
        assigned_crop = getattr(sys_config, f"plant_{plant_id}_crop", None) or sys_config.selected_crop_type

        meta_crop = None
        if os.path.exists(metadata_file):
            try:
                with open(metadata_file, "r") as f:
                    meta = json.load(f)
                    meta_crop = meta.get("crop_type")
                    if "timestamp" in meta:
                        try:
                            session_timestamp = datetime.fromisoformat(meta["timestamp"])
                        except Exception:
                            try:
                                session_timestamp = datetime.strptime(meta["timestamp"], "%Y-%m-%d %H:%M:%S")
                            except Exception:
                                pass
                    for p_info in meta.get("photos", []):
                        p_idx = p_info.get("photo_index")
                        if p_idx is not None:
                            photos_meta_map[p_idx] = p_info
            except Exception:
                pass

        # Jerarquía: crop_type explícito > cultivo asignado a la Canastilla > metadata.json > fallback cebollín
        effective_crop = crop_type or assigned_crop or meta_crop or "cebollin"
        profile = self.crop_repo.get_by_type(effective_crop) or self.crop_repo.get_by_type(meta_crop) or self.crop_repo.get_by_type("cebollin") or
self.crop_repo.get_all()[0]

        all_images = glob.glob(os.path.join(folder_path, "*.png")) + glob.glob(os.path.join(folder_path, "*.jpg"))
        if not all_images:
            all_images = glob.glob(os.path.join(folder_path, "*", "*.png")) + glob.glob(os.path.join(folder_path, "*", "*.jpg"))
```

```
218
219         cenital_imgs = sorted([f for f in all_images if "cenital" in os.path.basename(f).lower()])
220         lateral_imgs = sorted([f for f in all_images if "lateral" in os.path.basename(f).lower()])
221
222         if not cenital_imgs and not lateral_imgs and all_images:
223             half = len(all_images) // 2
224             cenital_imgs = all_images[:half]
225             lateral_imgs = all_images[half:]
226
227         timestamp_str = datetime.now().strftime("%Y%m%d_%H%M%S")
228         upload_dir = os.path.join(config.UPLOAD_DIR, f"batch_{timestamp_str}")
229         processed_dir = os.path.join(config.PROCESSED_DIR, f"batch_{timestamp_str}")
230         os.makedirs(upload_dir, exist_ok=True)
231         os.makedirs(processed_dir, exist_ok=True)
232
233         cenital_results = []
234         lateral_results = []
235         cenital_paths_list = []
236         lateral_paths_list = []
237
238         sys_config = self.config_repo.get()
239
240         total_folder_photos = len(cenital_imgs) + len(lateral_imgs)
241         actual_total = total_global_photos if total_global_photos > 0 else max(1, total_folder_photos)
242         current_processed = global_offset
243         turn_prefix = f"[{turn_label}] " if turn_label else ""
244
245         for i, src_img in enumerate(cenital_imgs):
246             current_processed += 1
247             pct = int((current_processed / (actual_total + 1)) * 95)
248             self.set_progress(
249                 percent=pct,
250                 message=f"{turn_prefix}Segmentando vista cenital {i+1}/{len(cenital_imgs)} (ExG + Otsu)...",
251                 is_processing=True,
252                 current_item=current_processed,
253                 total_items=actual_total,
254                 current_turn=turn_label
255             )
256
257             dst_orig = os.path.join(upload_dir, f"cenital_{i+1}.png")
258             dst_proc = os.path.join(processed_dir, f"proc_cenital_{i+1}.png")
259             shutil.copy(src_img, dst_orig)
260
261             res = self.pipeline.process_cenital(dst_orig, dst_proc, profile, config=sys_config)
262             if res:
263                 cenital_results.append(res)
264                 rel_orig = os.path.relpath(dst_orig, config.DATA_DIR).replace("\\", "/")
265                 rel_proc = os.path.relpath(dst_proc, config.DATA_DIR).replace("\\", "/")
266                 cenital_paths_list.append({
267                     "index": i + 1,
268                     "orig": f"data/{rel_orig}",
269                     "proc": f"data/{rel_proc}",
270                     "metrics": res
271                 })
272
```

```
        for i, src_img in enumerate(lateral_imgs):
            current_processed += 1
            pct = int((current_processed / (actual_total + 1)) * 95)
            self.set_progress(
                percent=pct,
                message=f"{turn_prefix}Segmentando vista lateral {i+1}/{len(lateral_imgs)} (Tallo y Altura)...",
                is_processing=True,
                current_item=current_processed,
                total_items=actual_total,
                current_turn=turn_label
            )

            dst_orig = os.path.join(upload_dir, f"lateral_{i+1}.png")
            dst_proc = os.path.join(processed_dir, f"proc_lateral_{i+1}.png")
            shutil.copy(src_img, dst_orig)

            res = self.pipeline.process_lateral(dst_orig, dst_proc, profile, config=sys_config)
            if res:
                lateral_results.append(res)
                rel_orig = os.path.relpath(dst_orig, config.DATA_DIR).replace("\\", "/")
                rel_proc = os.path.relpath(dst_proc, config.DATA_DIR).replace("\\", "/")
                lateral_paths_list.append({
                    "index": i + 1,
                    "orig": f"data/{rel_orig}",
                    "proc": f"data/{rel_proc}",
                    "metrics": res
                })

        # Construir métricas individuales con timestamps específicos de cada toma
        max_photos = max(len(cenital_paths_list), len(lateral_paths_list), 1)
        individual_metrics = []

        for i in range(max_photos):
            photo_index = i + 1
            c_data = cenital_paths_list[i] if i < len(cenital_paths_list) else None
            l_data = lateral_paths_list[i] if i < len(lateral_paths_list) else None

            c_metrics = c_data["metrics"] if c_data else {}
            l_metrics = l_data["metrics"] if l_data else {}

            # Timestamp exacto de la toma individual
            p_meta = photos_meta_map.get(photo_index, {})
            p_time_str = p_meta.get("timestamp")
            if p_time_str:
                try:
                    exact_dt = datetime.strptime(p_time_str, "%Y-%m-%d %H:%M:%S")
                except Exception:
                    exact_dt = session_timestamp
            else:
                exact_dt = session_timestamp

            individual_metrics.append({
                "photo_index": photo_index,
                "capture_time": exact_dt,
                "foliar_area_cm2": c_metrics.get("area_cm2", 0.0),
```

```
                "health_index": c_metrics.get("health_index", 100.0),
                "compacity_index": c_metrics.get("compacity_index", 0.0),
                "spots_count": c_metrics.get("spots_count", 0),
                "fruits_count": c_metrics.get("fruits_count", 0),
                "plant_height_cm": l_metrics.get("plant_height_cm", 0.0),
                "stem_diameter_mm": l_metrics.get("stem_diameter_mm", 0.0),
                "cenital_orig": c_data["orig"] if c_data else (lateral_paths_list[0]["orig"] if lateral_paths_list else ""),
                "cenital_proc": c_data["proc"] if c_data else (lateral_paths_list[0]["proc"] if lateral_paths_list else ""),
                "lateral_orig": l_data["orig"] if l_data else (cenital_paths_list[0]["orig"] if cenital_paths_list else ""),
                "lateral_proc": l_data["proc"] if l_data else (cenital_paths_list[0]["proc"] if cenital_paths_list else "")
            })

        final_metrics = self.pipeline.filter_and_average(cenital_results, lateral_results)

        default_cen_orig = cenital_paths_list[0]["orig"] if cenital_paths_list else ""
        default_cen_proc = cenital_paths_list[0]["proc"] if cenital_paths_list else ""
        default_lat_orig = lateral_paths_list[0]["orig"] if lateral_paths_list else ""
        default_lat_proc = lateral_paths_list[0]["proc"] if lateral_paths_list else ""

        session_id = self._save_capture_session_with_metrics(
            period=period,
            crop_type=profile.crop_type,
            sensor_data=sensor_data,
            final_metrics=final_metrics,
            individual_metrics=individual_metrics,
            cenital_paths={"orig": default_cen_orig, "proc": default_cen_proc},
            lateral_paths={"orig": default_lat_orig, "proc": default_lat_proc},
            plant_id=plant_id,
            session_timestamp=session_timestamp
        )

        return {
            "session_id": session_id,
            "period": period,
            "crop_type": profile.crop_type,
            "plant_id": plant_id,
            "metrics": final_metrics,
            "individual_metrics": individual_metrics,
            "processed_count": len(cenital_results) + len(lateral_results)
        }

    def _save_capture_session_with_metrics(
        self, period, crop_type, sensor_data, final_metrics,
        individual_metrics=None, cenital_paths=None, lateral_paths=None, plant_id=1,
        session_timestamp=None
    ) -> int:
        if not cenital_paths: cenital_paths = {}
        if not lateral_paths: lateral_paths = {}
        if not sensor_data: sensor_data = {}
        if not session_timestamp: session_timestamp = datetime.now()

        sensor_id = None
        if sensor_data and any(sensor_data.values()):
            sensor_record = SensorReading(
                timestamp=session_timestamp,
```

```
                temperature=float(sensor_data.get("temperature", 0.0)),
                humidity=float(sensor_data.get("humidity", 0.0)),
                uv_solar=float(sensor_data.get("uv_solar", 0.0)),
                motor_current=float(sensor_data.get("motor_current", 0.0)),
                motor_state=int(sensor_data.get("motor_state", 1 if float(sensor_data.get("motor_current", 0.0)) > 0 else 0)),
                water_ph=float(sensor_data.get("water_ph", 0.0)),
                ec_conductivity=float(sensor_data.get("ec_conductivity", 0.0)),
                turbidity_ntu=float(sensor_data.get("turbidity_ntu", 0.0)),
                dissolved_oxygen_mg_l=float(sensor_data.get("dissolved_oxygen_mg_l", 0.0)),
                water_temp=float(sensor_data.get("water_temp", 0.0))
            )
            self.sensor_repo.add(sensor_record)
            sensor_id = sensor_record.id
        else:
            nearest = self.sensor_repo.find_nearest(session_timestamp)
            sensor_id = nearest.id if nearest else None

        existing = self.session_repo.get_by_period_and_plant(period, int(plant_id))
        if existing:
            session_record = existing
            session_record.timestamp = session_timestamp
            session_record.crop_type = crop_type
            if sensor_id:
                session_record.sensor_reading_id = sensor_id

            for m in list(session_record.metrics):
                db.session.delete(m)
            db.session.flush()
        else:
            session_record = CaptureSession(
                period=period,
                plant_id=int(plant_id),
                crop_type=crop_type,
                sensor_reading_id=sensor_id,
                timestamp=session_timestamp
            )
            self.session_repo.save(session_record)

        # 1. Promedio consolidado
        avg_record = BiometricMetric(
            session_id=session_record.id,
            photo_index=0,
            is_average=True,
            capture_exact_time=session_timestamp,
            foliar_area_cm2=final_metrics.get("foliar_area_cm2", 0.0),
            plant_height_cm=final_metrics.get("plant_height_cm", 0.0),
            stem_diameter_mm=final_metrics.get("stem_diameter_mm", 0.0),
            health_index=final_metrics.get("health_index", 100.0),
            compacity_index=final_metrics.get("compacity_index", 0.0),
            spots_count=0,
            fruits_count=0,
            image_path_cenital_orig=cenital_paths.get("orig"),
            image_path_cenital_proc=cenital_paths.get("proc"),
            image_path_lateral_orig=lateral_paths.get("orig"),
            image_path_lateral_proc=lateral_paths.get("proc")
```

```
        )
        db.session.add(avg_record)

        # 2. Métricas individuales por cada una de las 5 tomas
        if individual_metrics:
            for item in individual_metrics:
                idx = item.get("photo_index", 1)
                t_exact = item.get("capture_time", session_timestamp)
                ind_record = BiometricMetric(
                    session_id=session_record.id,
                    photo_index=idx,
                    is_average=False,
                    capture_exact_time=t_exact,
                    foliar_area_cm2=item.get("foliar_area_cm2", 0.0),
                    plant_height_cm=item.get("plant_height_cm", 0.0),
                    stem_diameter_mm=item.get("stem_diameter_mm", 0.0),
                    health_index=item.get("health_index", 100.0),
                    compacity_index=item.get("compacity_index", 0.0),
                    spots_count=0,
                    fruits_count=0,
                    image_path_cenital_orig=item.get("cenital_orig"),
                    image_path_cenital_proc=item.get("proc") if "proc" in item else item.get("cenital_proc"),
                    image_path_lateral_orig=item.get("lateral_orig"),
                    image_path_lateral_proc=item.get("lateral_proc")
                )
                db.session.add(ind_record)

        db.session.commit()
        return session_record.id
```

ARCHIVO 15/85: local_server/application/services/weather_service.py (144 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import time
import json
import urllib.request
import urllib.parse
from datetime import datetime

class WeatherService:
    """
    Servicio de Clima Exterior Local para la ubicación geográfica de la torre hidropónica SIFMA.
    Utiliza la API abierta y gratuita Open-Meteo (sin necesidad de API keys).
    Soporta persistencia de coordenadas personalizadas (por defecto: Santa Marta, Colombia).
    Incluye caché en memoria (10 min) y fallback offline elegante si la torre no tiene internet.
    """
    _instance = None
    _cached_data = None
    _last_fetch_time = 0
    _CACHE_TTL = 600  # 10 minutos de caché

    # Coordenadas Oficiales SIFMA - Santa Marta, Magdalena, Colombia
    DEFAULT_LAT = 11.2408
    DEFAULT_LON = -74.1990
    DEFAULT_CITY = "Santa Marta, Magdalena"

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            cls._instance = WeatherService()
        return cls._instance

    @staticmethod
    def _interpret_wmo_code(code: int) -> dict:
        """Traduce códigos meteorológicos estándar WMO a descripción en español e icono."""
        mapping = {
            0: ("Cielo Despejado", "wb_sunny", "sunny"),
            1: ("Mayormente Despejado", "wb_sunny", "mostly_sunny"),
            2: ("Parcialmente Nublado", "partly_cloudy_day", "partly_cloudy"),
            3: ("Nublado", "cloud", "cloudy"),
            45: ("Neblina", "foggy", "fog"),
            48: ("Niebla con Escarcha", "foggy", "fog"),
            51: ("Llovizna Ligera", "grain", "drizzle"),
            53: ("Llovizna Moderada", "grain", "drizzle"),
            55: ("Llovizna Intensa", "grain", "drizzle"),
            61: ("Lluvia Ligera", "water_drop", "rain"),
            63: ("Lluvia Moderada", "water_drop", "rain"),
            65: ("Lluvia Fuerte", "rainy", "rain"),
            80: ("Chubascos Aislados", "shower", "rain"),
            81: ("Chubascos Moderados", "shower", "rain"),
            82: ("Chubascos Violentos", "thunderstorm", "rain"),
            95: ("Tormenta Eléctrica", "thunderstorm", "storm"),
```

```
            96: ("Tormenta con Granizo Ligero", "thunderstorm", "storm"),
            99: ("Tormenta con Granizo Fuerte", "thunderstorm", "storm")
        }
        desc, icon, badge_class = mapping.get(code, ("Clima Estable", "cloud_queue", "cloudy"))
        return {
            "description": desc,
            "icon": icon,
            "badge_class": badge_class
        }

    def get_current_weather(self, lat: float = None, lon: float = None, city_name: str = None) -> dict:
        """
        Obtiene el clima actual para las coordenadas configuradas.
        Si falla la conexión a internet, retorna el último valor en caché o un estado offline seguro.
        """
        now = time.time()
        lat = lat if lat is not None else self.DEFAULT_LAT
        lon = lon if lon is not None else self.DEFAULT_LON
        city = city_name or self.DEFAULT_CITY

        if self._cached_data and (now - self._last_fetch_time < self._CACHE_TTL):
            return self._cached_data

        url = (
            f"https://api.open-meteo.com/v1/forecast?"
            f"latitude={lat}&longitude={lon}&"
            f"current=temperature_2m,relative_humidity_2m,apparent_temperature,weather_code,wind_speed_10m,surface_pressure&"
            f"timezone=auto"
        )

        try:
            req = urllib.request.Request(
                url,
                headers={'User-Agent': 'SIFMA-Hydroponics-Monitor/1.0'}
            )
            with urllib.request.urlopen(req, timeout=3.0) as response:
                if response.status == 200:
                    raw = json.loads(response.read().decode('utf-8'))
                    current = raw.get('current', {})
                    wmo_info = self._interpret_wmo_code(current.get('weather_code', 0))

                    data = {
                        "online": True,
                        "temperature_c": round(current.get('temperature_2m', 30.0), 1),
                        "apparent_temperature_c": round(current.get('apparent_temperature', 32.0), 1),
                        "humidity_rh": int(current.get('relative_humidity_2m', 70)),
                        "wind_speed_kmh": round(current.get('wind_speed_10m', 10.0), 1),
                        "weather_code": current.get('weather_code', 0),
                        "weather_description": wmo_info["description"],
                        "icon": wmo_info["icon"],
                        "badge_class": wmo_info["badge_class"],
                        "location_name": city,
                        "latitude": lat,
                        "longitude": lon,
                        "updated_at": datetime.now().strftime("%H:%M"),
```

```
                    "status_text": "En Línea (Open-Meteo)"
                }
                self._cached_data = data
                self._last_fetch_time = now
                return data
        except Exception:
            pass

        # Fallback offline si no hay conexión o expira el timeout
        if self._cached_data:
            cached_copy = dict(self._cached_data)
            cached_copy["status_text"] = "Caché Local (Sin conexión)"
            cached_copy["online"] = False
            return cached_copy

        return {
            "online": False,
            "temperature_c": 30.0,
            "apparent_temperature_c": 32.0,
            "humidity_rh": 72,
            "wind_speed_kmh": 8.0,
            "weather_code": 0,
            "weather_description": "Modo Local Autónomo",
            "icon": "cloud_off",
            "badge_class": "offline",
            "location_name": city,
            "latitude": lat,
            "longitude": lon,
            "updated_at": datetime.now().strftime("%H:%M"),
            "status_text": "Nodo Autónomo (Sin Internet)"
        }

    def clear_cache(self):
        """Invalida la caché para forzar una consulta inmediata ante cambio de coordenadas."""
        self._cached_data = None
        self._last_fetch_time = 0
```

ARCHIVO 16/85: local_server/core/__init__.py (9 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .config import config, AppConfig
from .security import SecurityService
from .logger import logger

__all__ = ["config", "AppConfig", "SecurityService", "logger"]
```

ARCHIVO 17/85: local_server/core/config.py (67 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
from dataclasses import dataclass
try:
    from dotenv import load_dotenv
except ImportError:
    def load_dotenv(path):
        if not path or not os.path.exists(path):
            return
        try:
            with open(path, "r", encoding="utf-8") as f:
                for line in f:
                    line = line.strip()
                    if not line or line.startswith("#") or "=" not in line:
                        continue
                    k, v = line.split("=", 1)
                    k = k.strip()
                    v = v.strip().strip("'\"")
                    if k not in os.environ:
                        os.environ[k] = v
        except Exception:
            pass

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
env_file = os.path.join(BASE_DIR, ".env")
if os.path.exists(env_file):
    load_dotenv(env_file)
else:
    load_dotenv(os.path.join(os.path.dirname(BASE_DIR), ".env"))

@dataclass(frozen=True)
class AppConfig:
    """
    Configuración inmutable y centralizada del servidor SIFMA.
    Permite parametrización por variables de entorno y define valores predeterminados seguros.
    """
    BASE_DIR: str = BASE_DIR
    DB_PATH: str = os.getenv("SIFMA_DB_PATH", os.path.join(BASE_DIR, "sifma.db"))
    SQLALCHEMY_DATABASE_URI: str = f"sqlite:///{DB_PATH}"
    SQLALCHEMY_TRACK_MODIFICATIONS: bool = False

    # Secret Key segura (leída de entorno o fallback)
    SECRET_KEY: str = os.getenv("SIFMA_SECRET_KEY", "sifma_secure_production_secret_key_2026")

    # Credenciales del Administrador Principal desde .env
    ADMIN_USERNAME: str = os.getenv("SIFMA_ADMIN_USER", "SIFMA_ADMIN")
    ADMIN_DEFAULT_PASSWORD: str = os.getenv("SIFMA_ADMIN_PASSWORD", "Sifma#Campo2026!")
    ADMIN_FULLNAME: str = os.getenv("SIFMA_ADMIN_FULLNAME", "Administrador SIFMA")
    ADMIN_EMAIL: str = os.getenv("SIFMA_ADMIN_EMAIL", "admin@sifma.local")

```

```
    # Directorios estáticos y de almacenamiento
    STATIC_DIR: str = os.path.join(BASE_DIR, "static")
    DATA_DIR: str = os.path.join(STATIC_DIR, "data")
    UPLOAD_DIR: str = os.path.join(DATA_DIR, "uploads")
    PROCESSED_DIR: str = os.path.join(DATA_DIR, "processed")
    TELEMETRY_DIR: str = os.path.join(DATA_DIR, "telemetry")

    # Parámetros del fenotipado
    DEFAULT_PLANT_ID: int = 1
    MAX_PLANT_NODES: int = 4
    PHOTOS_PER_PERIOD: int = 5
    DEFAULT_PIXEL_TO_CM: float = 0.038

config = AppConfig()
```

ARCHIVO 18/85: local_server/core/dependencies.py (73 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import sys
import subprocess
import importlib

SERVER_REQUIRED_PACKAGES = [
    ("flask", "Flask==3.1.3"),
    ("flask_sqlalchemy", "Flask-SQLAlchemy==3.1.1"),
    ("sqlalchemy", "SQLAlchemy==2.0.52"),
    ("cv2", "opencv-python==4.11.0.86"),
    ("numpy", "numpy==2.4.6"),
    ("serial", "pyserial==3.5"),
    ("dotenv", "python-dotenv==1.2.4")
]

def install_package(package_name: str) -> bool:
    """Instala automáticamente un paquete de Python utilizando pip."""
    print(f"[AUTO-INSTALADOR SIFMA] Instalando paquete ausente: '{package_name}'...")
    base_cmd = [sys.executable, "-m", "pip", "install", package_name]

    try:
        res = subprocess.run(base_cmd, capture_output=True, text=True)
        if res.returncode == 0:
            print(f"[AUTO-INSTALADOR SIFMA] '{package_name}' instalado exitosamente.")
            return True

        # En Linux con restricciones de entorno gestionado externamente (PEP 668)
        if "externally-managed-environment" in res.stderr.lower() or sys.platform.startswith("linux"):
            print(f"[AUTO-INSTALADOR SIFMA] Reintentando con --break-system-packages...")
            res_break = subprocess.run(base_cmd + ["--break-system-packages"], capture_output=True, text=True)
            if res_break.returncode == 0:
                print(f"[AUTO-INSTALADOR SIFMA] '{package_name}' instalado exitosamente.")
                return True
            else:
                print(f"[AUTO-INSTALADOR AVISO] No se pudo instalar '{package_name}': {res_break.stderr}")
                return False
        else:
            print(f"[AUTO-INSTALADOR AVISO] No se pudo instalar '{package_name}': {res.stderr}")
            return False
    except Exception as e:
        print(f"[AUTO-INSTALADOR ERROR] Excepción al ejecutar pip: {e}")
        return False

def ensure_dependencies():
    """Verifica e instala automáticamente las librerías necesarias del servidor SIFMA."""
    missing = []
    for module_name, pip_name in SERVER_REQUIRED_PACKAGES:
        try:
            importlib.import_module(module_name)
        except ImportError:
```

```
            missing.append((module_name, pip_name))

    if missing:
        print("\n============================================================")
        print(f"[AUTO-INSTALADOR SIFMA] Detectadas {len(missing)} dependencias faltantes.")
        print("Instalando automáticamente mediante PIP...")
        print("============================================================\n")

        for module_name, pip_name in missing:
            ok = install_package(pip_name)
            if ok:
                try:
                    importlib.invalidate_caches()
                    importlib.import_module(module_name)
                except Exception:
                    pass
        print("\n[AUTO-INSTALADOR SIFMA] Dependencias listas para producción.\n")

if __name__ == "__main__":
    ensure_dependencies()
```

ARCHIVO 19/85: local_server/core/logger.py (22 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import logging
import sys

def setup_logger(name: str = "SIFMA") -> logging.Logger:
    """Configura un logger estructurado para toda la aplicación."""
    logger = logging.getLogger(name)
    if not logger.handlers:
        logger.setLevel(logging.INFO)
        formatter = logging.Formatter(
            '[%(asctime)s] [%(levelname)s] [%(name)s] %(message)s',
            datefmt='%Y-%m-%d %H:%M:%S'
        )
        handler = logging.StreamHandler(sys.stdout)
        handler.setFormatter(formatter)
        logger.addHandler(handler)
    return logger

logger = setup_logger()
```

ARCHIVO 20/85: local_server/core/security.py (58 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import re
from werkzeug.security import generate_password_hash, check_password_hash
from werkzeug.utils import secure_filename
from core.config import config

class SecurityService:
    """
    Servicio de seguridad para autenticación, verificación de contraseñas
    y sanitización contra Path Traversal e inyecciones.
    """

    @staticmethod
    def verify_password(plain_password: str, password_hash: str) -> bool:
        """Verifica una contraseña en texto plano contra su hash."""
        if not plain_password or not password_hash:
            return False
        try:
            return check_password_hash(password_hash, plain_password)
        except Exception:
            return False

    @staticmethod
    def hash_password(plain_password: str) -> str:
        """Genera un hash seguro para almacenamiento de credenciales."""
        return generate_password_hash(plain_password)

    @staticmethod
    def sanitize_filename(filename: str) -> str:
        """Sanitiza el nombre de un archivo para evitar ataques de Path Traversal."""
        clean = secure_filename(filename)
        if not clean:
            clean = "archivo_anonimo.dat"
        return clean

    @staticmethod
    def is_safe_path(base_dir: str, target_path: str) -> bool:
        """Verifica que target_path se encuentre estrictamente contenido dentro de base_dir."""
        try:
            abs_base = os.path.abspath(base_dir)
            abs_target = os.path.abspath(target_path)
            return os.path.commonpath([abs_base, abs_target]) == abs_base
        except Exception:
            return False

    @staticmethod
    def sanitize_text(input_text: str, max_length: int = 500) -> str:
        """Limpia cadenas de texto removiendo caracteres de control peligrosos."""
        if not input_text:
```

```
            return ""
        clean = str(input_text).strip()[:max_length]
        # Remover etiquetas HTML directas para prevenir XSS
        clean = re.sub(r'<[^>]*>', '', clean)
        return clean
```

ARCHIVO 21/85: local_server/core/vision/__init__.py (10 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .pipeline import VisionPipelineManager
from .segmentation import ImageSegmenter
from .metrics import BiometricCalculator
from .fruit_detector import FruitDetector

__all__ = ["VisionPipelineManager", "ImageSegmenter", "BiometricCalculator", "FruitDetector"]
```

ARCHIVO 22/85: local_server/core/vision/fruit_detector.py (54 lineas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import cv2
import numpy as np

class FruitDetector:
    """
    Detector de frutos (ej. Tomate Cherry) basado en tonalidades rojo/naranja y circularidad.
    """

    @staticmethod
    def detect_fruits(img, overlay):
        """
        Detecta y contabiliza frutos circulares rojos/anaranjados.
        Retorna la cantidad detectada y dibuja marcadores sobre la imagen overlay.
        """
        if img is None:
            return 0

        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

        # Rangos HSV para tonalidades rojas (wraparound de 0° y 180°)
        lower_red1 = np.array([0, 50, 45])
        upper_red1 = np.array([15, 255, 255])
        lower_red2 = np.array([165, 50, 45])
        upper_red2 = np.array([180, 255, 255])

        mask_red = cv2.bitwise_or(
            cv2.inRange(hsv, lower_red1, upper_red1),
            cv2.inRange(hsv, lower_red2, upper_red2)
        )

        kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        kernel_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
        mask_red = cv2.morphologyEx(mask_red, cv2.MORPH_CLOSE, kernel_close)
        mask_red = cv2.morphologyEx(mask_red, cv2.MORPH_OPEN, kernel_open)

        fruit_contours, _ = cv2.findContours(mask_red, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        fruits_count = 0

        for c in fruit_contours:
            c_area = cv2.contourArea(c)
            if c_area > 40: # Área mínima para tomate individual
                c_perimeter = cv2.arcLength(c, True)
                if c_perimeter > 0:
                    circularity = (4 * np.pi * c_area) / (c_perimeter ** 2)
                    if circularity > 0.55: # Alta circularidad
                        fruits_count += 1
                        (x_f, y_f), r_f = cv2.minEnclosingCircle(c)
                        cv2.circle(overlay, (int(x_f), int(y_f)), int(r_f), (0, 255, 255), 2)

```

```
        return fruits_count
```

ARCHIVO 23/85: local_server/core/vision/metrics.py (109 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import cv2
import numpy as np

class BiometricCalculator:
    """
    Calculadora de parámetros biométricos (Área Foliar, Salud, Altura y Tallo).
    """

    @staticmethod
    def calculate_cenital_metrics(img, mask, contours, profile):
        """
        Calcula el área foliar (cm²), índice de salud (%) e índice de compacidad.
        """
        plant_pixels = 0
        compacity = 0.0

        if contours:
            plant_pixels = sum(cv2.contourArea(c) for c in contours)
            largest_contour = max(contours, key=cv2.contourArea)
            perimeter = cv2.arcLength(largest_contour, True)
            largest_area = cv2.contourArea(largest_contour)

            if perimeter > 0:
                compacity = (4 * np.pi * largest_area) / (perimeter ** 2)

        # 1. Área física en cm²
        area_cm2 = plant_pixels * (profile.pixel_to_cm_ratio ** 2)

        # 2. Índice de Salud Cromática (% de píxeles verde óptimo dentro de la planta)
        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        lower_green = np.array([38, 45, 45])
        upper_green = np.array([82, 255, 255])
        mask_healthy = cv2.inRange(hsv, lower_green, upper_green)

        healthy_plant_pixels = cv2.countNonZero(cv2.bitwise_and(mask, mask_healthy))
        health_index = 100.0
        if plant_pixels > 0:
            health_index = (healthy_plant_pixels / plant_pixels) * 100.0
            health_index = min(100.0, max(0.0, health_index))

        return {
            "plant_pixels": plant_pixels,
            "area_cm2": round(area_cm2, 2),
            "health_index": round(health_index, 2),
            "compacity_index": round(compacity, 3)
        }

    @staticmethod
    def calculate_lateral_metrics(img, mask, contours, profile):
```

```
    """
    Calcula la altura estimada de la planta (cm) y el diámetro del tallo (mm).
    La altura se calcula con precisión entre el ápice más alto y la base del tallo detectado.
    """
    h, w, _ = img.shape
    height_cm = 0.0
    stem_diameter_mm = 0.0
    stem_draw_y = None
    stem_row_pixels = None
    y_min = None
    y_max = None
    x_points = None

    if contours:
        all_points = np.vstack([c for c in contours])
        y_points = all_points[:, 0, 1]
        x_points = all_points[:, 0, 0]

        y_min = int(np.min(y_points)) # Punto más alto de las hojas
        y_max = int(np.max(y_points)) # Base más baja del tallo segmentado

        if y_min < y_max:
            height_pixels = y_max - y_min
            height_cm = height_pixels * profile.pixel_to_cm_ratio

            # Cálculo de diámetro de tallo basal (si la especie tiene tallo)
            has_stem = getattr(profile, 'has_stem', True)
            if has_stem and height_cm >= 0.5:
                widths = []
                valid_y = []
                # Evaluar los primeros píxeles de la base del tallo
                for offset in range(2, 10):
                    test_y = y_max - offset
                    if 0 <= test_y < h:
                        row_pixels = np.where(mask[test_y, :] > 0)[0]
                        if len(row_pixels) > 0:
                            w_px = np.max(row_pixels) - np.min(row_pixels)
                            if 1 <= w_px < 60: # Descartar hojas anchas
                                widths.append(w_px)
                                valid_y.append(test_y)

                if widths:
                    stem_width_pixels = np.median(widths)
                    stem_diameter_mm = stem_width_pixels * profile.pixel_to_cm_ratio * 10
                    stem_draw_y = int(np.median(valid_y))
                    stem_row_pixels = np.where(mask[stem_draw_y, :] > 0)[0]

    return {
        "height_cm": round(height_cm, 2),
        "stem_diameter_mm": round(stem_diameter_mm, 2),
        "y_min": y_min,
        "y_max": y_max,
        "x_points": x_points,
        "stem_draw_y": stem_draw_y,
        "stem_row_pixels": stem_row_pixels
```

```
109         }
```

ARCHIVO 24/85: local_server/core/vision/pipeline.py (141 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import cv2
import numpy as np

from .segmentation import ImageSegmenter
from .metrics import BiometricCalculator

class VisionPipelineManager:
    """
    Orquestador principal del pipeline de visión computacional agronómica.
    Procesa imágenes cenitales y laterales, segmenta tejido foliar en ROJO y
    calcula área foliar, índice de salud, altura y diámetro de tallo.
    """

    @staticmethod
    def process_cenital_image(image_path, output_path, profile):
        os.makedirs(os.path.dirname(output_path), exist_ok=True)
        img = cv2.imread(image_path)
        if img is None:
            return None

        mask = ImageSegmenter.create_foliage_mask(img, profile)
        contours = ImageSegmenter.extract_valid_contours(mask, min_area=30)

        metrics = BiometricCalculator.calculate_cenital_metrics(img, mask, contours, profile)

        overlay = img.copy()
        if contours:
            # 1. Dibujar todos los contornos del área foliar en ROJO PURO (0, 0, 255 en BGR)
            cv2.drawContours(overlay, contours, -1, (0, 0, 255), 2)

            # 2. Dibujar el cerco exterior / Envoltura convexa de la canopia en ROJO DESTACADO
            all_pts = np.vstack(contours)
            if len(all_pts) >= 3:
                hull = cv2.convexHull(all_pts)
                cv2.drawContours(overlay, [hull], -1, (0, 0, 255), 2)

        metrics["fruits_count"] = 0
        metrics["spots_count"] = 0

        # Superponer leyenda de métricas biométricas
        cv2.putText(overlay, f"Area Foliar: {metrics['area_cm2']:.1f} cm2", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
        cv2.putText(overlay, f"Salud Foliar: {metrics['health_index']:.1f}%", (20, 75),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

        cv2.imwrite(output_path, overlay)
        return metrics

```

```
    @staticmethod
    def process_lateral_image(image_path, output_path, profile):
        os.makedirs(os.path.dirname(output_path), exist_ok=True)
        img = cv2.imread(image_path)
        if img is None:
            return None

        mask = ImageSegmenter.create_foliage_mask(img, profile)
        contours = ImageSegmenter.extract_valid_contours(mask, min_area=30)

        metrics = BiometricCalculator.calculate_lateral_metrics(img, mask, contours, profile)

        overlay = img.copy()
        if contours:
            # Dibujar el perfil de la planta en ROJO (0, 0, 255)
            cv2.drawContours(overlay, contours, -1, (0, 0, 255), 2)

            y_min = metrics["y_min"]
            y_max = metrics["y_max"]
            x_points = metrics["x_points"]

            if y_min is not None and y_max is not None and y_min < y_max:
                x_center = int((np.min(x_points) + np.max(x_points)) / 2)
                cv2.line(overlay, (x_center - 15, y_min), (x_center + 15, y_min), (0, 255, 255), 2)
                cv2.line(overlay, (x_center - 15, y_max), (x_center + 15, y_max), (0, 255, 255), 2)
                cv2.line(overlay, (x_center, y_min), (x_center, y_max), (0, 255, 255), 2)

                cv2.putText(overlay, f"H: {metrics['height_cm']:.1f} cm", (x_center + 15, int((y_min + y_max)/2)),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)

            if metrics["stem_draw_y"] is not None and metrics["stem_row_pixels"] is not None:
                draw_y = metrics["stem_draw_y"]
                row_pixels = metrics["stem_row_pixels"]
                if len(row_pixels) > 0:
                    cv2.line(overlay, (np.min(row_pixels), draw_y), (np.max(row_pixels), draw_y), (0, 0, 255), 3)
                    cv2.circle(overlay, (np.min(row_pixels), draw_y), 5, (0, 0, 255), -1)
                    cv2.circle(overlay, (np.max(row_pixels), draw_y), 5, (0, 0, 255), -1)
                    cv2.putText(overlay, f"D: {metrics['stem_diameter_mm']:.1f} mm", (np.max(row_pixels) + 15, draw_y + 5),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

        cv2.imwrite(output_path, overlay)
        return {
            "plant_height_cm": metrics["height_cm"],
            "stem_diameter_mm": metrics["stem_diameter_mm"]
        }

    @staticmethod
    def filter_and_average_metrics(cenital_results, lateral_results):
        """
        Elimina valores atípicos (+- 2 desviaciones estándar) y calcula promedios del lote.
        """
        final_metrics = {
            "foliar_area_cm2": 0.0,
            "health_index": 100.0,
            "compacity_index": 0.0,
```

```
            "spots_count": 0,
            "fruits_count": 0,
            "plant_height_cm": 0.0,
            "stem_diameter_mm": 0.0
        }

        def get_robust_mean(data_list):
            if not data_list: return 0.0
            if len(data_list) <= 2: return float(np.mean(data_list))
            mean = float(np.mean(data_list))
            std = float(np.std(data_list))
            if std == 0: return mean
            filtered = [x for x in data_list if abs(x - mean) <= 2 * std]
            return float(np.mean(filtered)) if filtered else mean

        if cenital_results:
            areas = [r["area_cm2"] for r in cenital_results]
            healths = [r["health_index"] for r in cenital_results]
            compacities = [r["compacity_index"] for r in cenital_results]

            final_metrics["foliar_area_cm2"] = round(get_robust_mean(areas), 2)
            final_metrics["health_index"] = round(get_robust_mean(healths), 2)
            final_metrics["compacity_index"] = round(get_robust_mean(compacities), 3)

        if lateral_results:
            heights = [r["plant_height_cm"] for r in lateral_results]
            diameters = [r["stem_diameter_mm"] for r in lateral_results]

            final_metrics["plant_height_cm"] = round(get_robust_mean(heights), 2)
            final_metrics["stem_diameter_mm"] = round(get_robust_mean(diameters), 2)

        return final_metrics

```

ARCHIVO 25/85: local_server/core/vision/segmentation.py (86 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import cv2
import numpy as np

class ImageSegmenter:
    """
    Segmentador botánico avanzado basado en el Índice de Exceso de Verde (ExG),
    espacio de color HSV para clorofila, canal LAB y filtrado morfológico.
    Diseñado para aislar exclusivamente tejido vegetal en entornos reales (suelo, maceta, manos).
    """

    @staticmethod
    def create_foliage_mask(img, profile):
        """
        Crea una máscara binaria de alta precisión aislando el follaje y tallos verdes.
        Combina:
        1. Índice ExG (Excess Green Index: 2*G - R - B) para eliminar piel, tierra y plástico blanco.
        2. Umbralizado HSV calibrado para clorofila.
        3. Umbralizado LAB para confirmación cromática.
        """
        if img is None:
            return None

        # 1. Separación de canales BGR flotantes para calcular ExG
        b, g, r = cv2.split(img.astype(np.float32))
        exg = 2.0 * g - r - b

        # Umbral ExG: el tejido vegetal verde vivo tiene ExG marcadamente positivo (> 10)
        # La piel (R>G), tierra (R>G) y macetas blancas/grises (R~G~B) dan valores <= 0.
        exg_threshold = 10.0
        exg_mask = (exg > exg_threshold).astype(np.uint8) * 255

        # 2. Umbralizado en espacio HSV (Clorofila / Follaje)
        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        h_min = max(24, int(getattr(profile, 'h_min', 28)))
        h_max = min(96, int(getattr(profile, 'h_max', 90)))
        s_min = max(30, int(getattr(profile, 's_min', 35)))
        s_max = int(getattr(profile, 's_max', 255))
        v_min = max(25, int(getattr(profile, 'v_min', 30)))
        v_max = int(getattr(profile, 'v_max', 255))

        lower_hsv = np.array([h_min, s_min, v_min])
        upper_hsv = np.array([h_max, s_max, v_max])
        mask_hsv = cv2.inRange(hsv, lower_hsv, upper_hsv)

        # 3. Umbralizado en espacio LAB (El canal A < 126 representa tonos verdes puros)
        lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
        l_min = int(getattr(profile, 'l_min', 20))
        l_max = int(getattr(profile, 'l_max', 255))
        a_max = min(126, int(getattr(profile, 'a_max', 124)))
```

```
        b_min = max(115, int(getattr(profile, 'b_min', 120)))

        lower_lab = np.array([l_min, 0, b_min])
        upper_lab = np.array([l_max, a_max, 255])
        mask_lab = cv2.inRange(lab, lower_lab, upper_lab)

        # 4. Fusión de máscaras (Intersección ExG + HSV + LAB)
        mask = cv2.bitwise_and(exg_mask, mask_hsv)
        mask = cv2.bitwise_and(mask, mask_lab)

        # Si la intersección con LAB elimina demasiados píxeles por iluminación, respaldar con ExG & HSV
        if cv2.countNonZero(mask) < 0.25 * cv2.countNonZero(cv2.bitwise_and(exg_mask, mask_hsv)):
            mask = cv2.bitwise_and(exg_mask, mask_hsv)

        # 5. Filtrado morfológico de ruido
        kernel_close = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
        kernel_open = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel_close)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel_open)

        return mask

    @staticmethod
    def extract_valid_contours(mask, min_area=30):
        """
        Encuentra y filtra contornos por un área mínima en píxeles.
        Ajustado a 30px para capturar tallos finos y brotes como el cebollín.
        """
        if mask is None:
            return []
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        return [c for c in contours if cv2.contourArea(c) >= min_area]

```

ARCHIVO 26/85: local_server/domain/__init__.py (52 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .models import (
    Config,
    CropProfile,
    SensorReading,
    CaptureSession,
    BiometricMetric,
    AgronomicConclusion,
    User
)
from .interfaces import (
    IConfigRepository,
    ICropProfileRepository,
    ISensorRepository,
    ICaptureSessionRepository,
    IConclusionRepository,
    IUserRepository,
    ISegmentationStrategy,
    IBiometricCalculator,
    IVisionPipeline,
    ITelemetryAdapter,
    ICsvTelemetryParser
)

__all__ = [
    "Config",
    "CropProfile",
    "SensorReading",
    "CaptureSession",
    "BiometricMetric",
    "AgronomicConclusion",
    "User",
    "IConfigRepository",
    "ICropProfileRepository",
    "ISensorRepository",
    "ICaptureSessionRepository",
    "IConclusionRepository",
    "IUserRepository",
    "ISegmentationStrategy",
    "IBiometricCalculator",
    "IVisionPipeline",
    "ITelemetryAdapter",
    "ICsvTelemetryParser"
]
```

ARCHIVO 27/85: local_server/domain/interfaces/__init__.py (28 lineas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .repositories import (
    IConfigRepository,
    ICropProfileRepository,
    ISensorRepository,
    ICaptureSessionRepository,
    IConclusionRepository,
    IUserRepository
)
from .vision import ISegmentationStrategy, IBiometricCalculator, IVisionPipeline
from .telemetry import ITelemetryAdapter, ICsvTelemetryParser

__all__ = [
    "IConfigRepository",
    "ICropProfileRepository",
    "ISensorRepository",
    "ICaptureSessionRepository",
    "IConclusionRepository",
    "IUserRepository",
    "ISegmentationStrategy",
    "IBiometricCalculator",
    "IVisionPipeline",
    "ITelemetryAdapter",
    "ICsvTelemetryParser"
]
```

ARCHIVO 28/85: local_server/domain/interfaces/repositories.py (48 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from typing import Protocol, List, Optional, Dict, Any
from domain.models import Config, CropProfile, SensorReading, CaptureSession, BiometricMetric, AgronomicConclusion, User

class IConfigRepository(Protocol):
    def get(self) -> Config: ...
    def update(self, **kwargs) -> Config: ...

class ICropProfileRepository(Protocol):
    def get_by_type(self, crop_type: str) -> Optional[CropProfile]: ...
    def get_all(self) -> List[CropProfile]: ...
    def save_or_update(self, profile: CropProfile) -> CropProfile: ...

class ISensorRepository(Protocol):
    def add(self, reading: SensorReading) -> SensorReading: ...
    def bulk_add(self, readings: List[SensorReading]) -> int: ...
    def get_latest(self) -> Optional[SensorReading]: ...
    def get_recent(self, limit: int = 30) -> List[SensorReading]: ...
    def get_by_date(self, date_str: str) -> List[SensorReading]: ...
    def get_available_dates(self) -> List[str]: ...
    def find_nearest(self, target_timestamp) -> Optional[SensorReading]: ...
    def delete_by_date(self, date_str: str) -> int: ...

class ICaptureSessionRepository(Protocol):
    def get_by_id(self, session_id: int) -> Optional[CaptureSession]: ...
    def get_by_period_and_plant(self, period: str, plant_id: int) -> Optional[CaptureSession]: ...
    def get_all_by_plant(self, plant_id: int, order_asc: bool = False) -> List[CaptureSession]: ...
    def get_by_date_and_plant(self, date_str: str, plant_id: int) -> List[CaptureSession]: ...
    def get_available_dates(self, plant_id: int) -> List[str]: ...
    def save(self, session: CaptureSession) -> CaptureSession: ...
    def delete(self, session: CaptureSession) -> None: ...
    def delete_by_date(self, date_str: str, plant_id: int) -> int: ...

class IConclusionRepository(Protocol):
    def get_by_id(self, conclusion_id: int) -> Optional[AgronomicConclusion]: ...
    def get_filtered(self, plant_id: int, date_str: Optional[str] = None, period_type: Optional[str] = None) -> List[AgronomicConclusion]: ...
    def save(self, conclusion: AgronomicConclusion) -> AgronomicConclusion: ...
    def delete(self, conclusion: AgronomicConclusion) -> None: ...

class IUserRepository(Protocol):
    def get_by_id(self, user_id: int) -> Optional[User]: ...
    def get_by_username(self, username: str) -> Optional[User]: ...
    def get_all(self) -> List[User]: ...
    def save(self, user: User) -> User: ...
    def delete(self, user: User) -> None: ...
```

ARCHIVO 29/85: local_server/domain/interfaces/telemetry.py (14 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from typing import Protocol, List, Dict, Any, Optional

class ITelemetryAdapter(Protocol):
    """Contrato para fuentes y adaptadores de telemetría (Pattern: Adapter)."""
    def get_available_ports(self) -> List[str]: ...
    def read_sample(self) -> Dict[str, Any]: ...

class ICsvTelemetryParser(Protocol):
    """Contrato para importación y parseo de tramas y archivos de torre."""
    def parse_stream(self, file_stream) -> Dict[str, Any]: ...
```

ARCHIVO 30/85: local_server/domain/interfaces/vision.py (23 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from typing import Protocol, List, Dict, Any, Optional
import numpy as np
from domain.models import CropProfile

class ISegmentationStrategy(Protocol):
    """Contrato para estrategias de segmentación de imágenes (Pattern: Strategy)."""
    def create_mask(self, image: np.ndarray, profile: CropProfile, is_lateral: bool = False) -> Optional[np.ndarray]: ...
    def extract_contours(self, mask: np.ndarray, min_area: int = 30) -> List[np.ndarray]: ...

class IBiometricCalculator(Protocol):
    """Contrato para cálculo de morfometría e índices agronómicos."""
    def calculate_cenital(self, image: np.ndarray, mask: np.ndarray, contours: List[np.ndarray], profile: CropProfile, config: Optional[Any] = None) -> Dict[str, Any]: ...
    def calculate_lateral(self, image: np.ndarray, mask: np.ndarray, contours: List[np.ndarray], profile: CropProfile, config: Optional[Any] = None) -> Dict[str, Any]: ...

class IVisionPipeline(Protocol):
    """Contrato del orquestador del pipeline visual."""
    def process_cenital(self, image_path: str, output_path: str, profile: CropProfile, config: Optional[Any] = None) -> Optional[Dict[str, Any]]: ...
    def process_lateral(self, image_path: str, output_path: str, profile: CropProfile, config: Optional[Any] = None) -> Optional[Dict[str, Any]]: ...
    def filter_and_average(self, cenital_results: List[Dict[str, Any]], lateral_results: List[Dict[str, Any]]) -> Dict[str, Any]: ...
```

ARCHIVO 31/85: local_server/domain/models/__init__.py (24 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .config import Config
from .crop_profile import CropProfile
from .sensor_reading import SensorReading
from .capture_session import CaptureSession
from .biometric_metric import BiometricMetric
from .agronomic_conclusion import AgronomicConclusion
from .user import User, ALL_MODULE_PERMISSIONS
from .audit_log import AuditLog

__all__ = [
    "Config",
    "CropProfile",
    "SensorReading",
    "CaptureSession",
    "BiometricMetric",
    "AgronomicConclusion",
    "User",
    "ALL_MODULE_PERMISSIONS",
    "AuditLog"
]
```

ARCHIVO 32/85: local_server/domain/models/agronomic_conclusion.py (39 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from datetime import datetime
from sqlalchemy import Column, Integer, String, Text, DateTime
from infrastructure.database.connection import db

class AgronomicConclusion(db.Model):
    """Entidad de la bitácora agronómica para anotaciones e inferencias del investigador."""
    __tablename__ = 'agronomic_conclusion'

    id = Column(Integer, primary_key=True)
    plant_id = Column(Integer, default=1, index=True)
    date_str = Column(String(20), nullable=False, index=True) # e.g. '2026-08-24'
    period_type = Column(String(20), default='diario') # 'diario', 'semanal', 'mensual'
    growth_obs = Column(Text, nullable=True) # Observaciones sobre crecimiento foliar y vigor
    climate_obs = Column(Text, nullable=True) # Respuesta a temperatura, humedad y luz
    nutrition_obs = Column(Text, nullable=True) # Solución nutritiva y bombeo
    general_conclusion = Column(Text, nullable=False) # Conclusión general
    author = Column(String(100), default='Investigador SIFMA')
    timestamp = Column(DateTime, default=datetime.now)

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def to_dict(self):
        return {
            "id": self.id,
            "plant_id": self.plant_id,
            "date_str": self.date_str,
            "period_type": self.period_type,
            "growth_obs": self.growth_obs or "",
            "climate_obs": self.climate_obs or "",
            "nutrition_obs": self.nutrition_obs or "",
            "general_conclusion": self.general_conclusion or "",
            "author": self.author,
            "created_at": self.timestamp.strftime("%d/%m/%Y %H:%M")
        }
```

ARCHIVO 33/85: local_server/domain/models/audit_log.py (112 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from datetime import datetime
from typing import Dict, Any, Optional
from infrastructure.database.connection import db

class AuditLog(db.Model):
    """
    Modelo de Registro de Auditoría y Trazabilidad de Acciones del Sistema (Audit Log).
    Exclusivo para el Super Usuario / Administrador Principal.
    """
    __tablename__ = 'audit_logs'
    __table_args__ = {'extend_existing': True}

    id = db.Column(db.Integer, primary_key=True)
    timestamp = db.Column(db.DateTime, default=datetime.now, nullable=False, index=True)
    user_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='SET NULL'), nullable=True)
    username = db.Column(db.String(100), nullable=False, index=True)
    user_fullname = db.Column(db.String(150), nullable=True)
    user_role = db.Column(db.String(50), nullable=True)

    action = db.Column(db.String(80), nullable=False, index=True) # e.g. LOGIN, LOGOUT, CREATE_USER, DELETE_USER, etc.
    action_display = db.Column(db.String(120), nullable=True)     # e.g. "Creación de Usuario"
    module = db.Column(db.String(60), nullable=False, index=True) # e.g. USUARIOS, AUTENTICACION, CONFIGURACION, LOTES
    description = db.Column(db.Text, nullable=False)              # Descripción amigable y detallada de la acción
    details_json = db.Column(db.Text, nullable=True)              # Metadatos / JSON opcional
    ip_address = db.Column(db.String(60), nullable=True)          # Dirección IP del cliente
    level = db.Column(db.String(20), default='info', nullable=False) # 'info', 'success', 'warning', 'danger'

    user = db.relationship('User', backref=db.backref('audit_logs', lazy='dynamic'), foreign_keys=[user_id])

    def __init__(
        self,
        action: str,
        module: str,
        description: str,
        username: str = "sistema",
        user_id: Optional[int] = None,
        user_fullname: Optional[str] = None,
        user_role: Optional[str] = None,
        action_display: Optional[str] = None,
        level: str = "info",
        ip_address: Optional[str] = None,
        details_json: Optional[str] = None,
        timestamp: Optional[datetime] = None
    ):
        self.action = action
        self.action_display = action_display or action.replace('_', ' ').title()
        self.module = module
        self.description = description
        self.username = username
```

```
        self.user_id = user_id
        self.user_fullname = user_fullname or username
        self.user_role = user_role or "sistema"
        self.level = level if level in ['info', 'success', 'warning', 'danger'] else 'info'
        self.ip_address = ip_address
        self.details_json = details_json
        self.timestamp = timestamp or datetime.now()

    @property
    def level_badge_class(self) -> str:
        mapping = {
            'info': 'badge-info',
            'success': 'badge-success',
            'warning': 'badge-warning',
            'danger': 'badge-danger'
        }
        return mapping.get(self.level, 'badge-info')

    @property
    def action_icon(self) -> str:
        icon_map = {
            'LOGIN': 'login',
            'LOGOUT': 'logout',
            'LOGIN_FAILED': 'no_accounts',
            'CREATE_USER': 'person_add',
            'UPDATE_USER': 'manage_accounts',
            'DELETE_USER': 'person_remove',
            'TOGGLE_USER_STATUS': 'published_with_changes',
            'CHANGE_PASSWORD': 'password',
            'UPDATE_CONFIG': 'tune',
            'DELETE_BATCH': 'delete_sweep',
            'PROCESS_IMAGES': 'camera_enhance',
            'EXPORT_DATA': 'file_download',
            'UPLOAD_SESSION': 'cloud_upload',
            'CROP_PROFILE_UPDATE': 'psychology',
            'SYSTEM_EVENT': 'info'
        }
        return icon_map.get(self.action, 'receipt_long')

    def to_dict(self) -> Dict[str, Any]:
        return {
            'id': self.id,
            'timestamp': self.timestamp.strftime('%Y-%m-%d %H:%M:%S'),
            'date_formatted': self.timestamp.strftime('%d/%m/%Y'),
            'time_formatted': self.timestamp.strftime('%H:%M:%S'),
            'user_id': self.user_id,
            'username': self.username,
            'user_fullname': self.user_fullname or self.username,
            'user_role': self.user_role,
            'action': self.action,
            'action_display': self.action_display,
            'action_icon': self.action_icon,
            'module': self.module,
            'description': self.description,
            'details_json': self.details_json,
```

```
            'ip_address': self.ip_address or '127.0.0.1',
            'level': self.level,
            'level_badge_class': self.level_badge_class
        }
```

ARCHIVO 34/85: local_server/domain/models/biometric_metric.py (52 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from sqlalchemy import Column, Integer, String, Float, Boolean, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from infrastructure.database.connection import db

class BiometricMetric(db.Model):
    """Entidad que registra los parámetros biométricos y rutas de imagen procesadas."""
    __tablename__ = 'biometric_metric'

    id = Column(Integer, primary_key=True)
    session_id = Column(Integer, ForeignKey('capture_session.id'), nullable=False, index=True)
    session = relationship("CaptureSession", back_populates="metrics")

    photo_index = Column(Integer, default=0) # 0 = Promedio del período, 1..5 = Toma individual
    is_average = Column(Boolean, default=False) # True si representa el consolidado del lote
    capture_exact_time = Column(DateTime, nullable=True)

    foliar_area_cm2 = Column(Float, default=0.0)
    plant_height_cm = Column(Float, default=0.0)
    stem_diameter_mm = Column(Float, default=0.0)
    health_index = Column(Float, default=100.0)
    compacity_index = Column(Float, default=0.0)
    spots_count = Column(Integer, default=0)
    fruits_count = Column(Integer, default=0)

    image_path_cenital_orig = Column(String(300), nullable=True)
    image_path_cenital_proc = Column(String(300), nullable=True)
    image_path_lateral_orig = Column(String(300), nullable=True)
    image_path_lateral_proc = Column(String(300), nullable=True)

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def to_dict(self):
        return {
            "id": self.id,
            "photo_index": self.photo_index,
            "is_average": self.is_average,
            "capture_exact_time": self.capture_exact_time.strftime("%H:%M:%S") if self.capture_exact_time else None,
            "foliar_area_cm2": round(self.foliar_area_cm2, 2),
            "plant_height_cm": round(self.plant_height_cm, 2),
            "stem_diameter_mm": round(self.stem_diameter_mm, 2),
            "health_index": round(self.health_index, 1),
            "compacity_index": round(self.compacity_index, 3),
            "image_path_cenital_orig": self.image_path_cenital_orig,
            "image_path_cenital_proc": self.image_path_cenital_proc,
            "image_path_lateral_orig": self.image_path_lateral_orig,
            "image_path_lateral_proc": self.image_path_lateral_proc
        }
```

ARCHIVO 35/85: local_server/domain/models/capture_session.py (38 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from datetime import datetime
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey
from sqlalchemy.orm import relationship
from infrastructure.database.connection import db

class CaptureSession(db.Model):
    """Entidad que agrupa un período de muestreo fotográfico para una canastilla de cultivo."""
    __tablename__ = 'capture_session'

    id = Column(Integer, primary_key=True)
    timestamp = Column(DateTime, default=datetime.now, index=True)
    period = Column(String(50), nullable=False)
    plant_id = Column(Integer, default=1, index=True)
    crop_type = Column(String(50), nullable=False)

    sensor_reading_id = Column(Integer, ForeignKey('sensor_reading.id'), nullable=True)
    sensor_reading = relationship("SensorReading", back_populates="sessions")

    metrics = relationship("BiometricMetric", back_populates="session", cascade="all, delete-orphan")

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def get_average_metric(self):
        """Retorna la métrica promedio consolidada del lote si existe."""
        if not self.metrics:
            return None
        return next((m for m in self.metrics if m.is_average or m.photo_index == 0), self.metrics[0])

    def get_individual_photos(self):
        """Retorna las fotos y métricas individuales ordenadas por índice."""
        photos = [m for m in self.metrics if not m.is_average and m.photo_index > 0]
        photos.sort(key=lambda m: m.photo_index)
        return photos
```

ARCHIVO 36/85: local_server/domain/models/config.py (85 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from sqlalchemy import Column, Integer, String, Boolean, Float
from infrastructure.database.connection import db

class Config(db.Model):
    """Entidad de configuración de captura y parámetros del nodo central."""
    __tablename__ = 'config'

    id = Column(Integer, primary_key=True)
    server_url = Column(String(200), default="http://127.0.0.1:5000")
    plant_id = Column(Integer, default=1)
    photos_per_period = Column(Integer, default=5)
    capture_interval_sec = Column(Integer, default=2)
    scheduled_times_str = Column(String(100), default="07:05,12:05,17:05")
    safe_shutdown_enabled = Column(Boolean, default=False)
    selected_crop_type = Column(String(50), default="cebollin")

    # Asignación de especie/perfil botánico independiente por cada Canastilla (1 a 4)
    plant_1_crop = Column(String(50), default="cebollin")
    plant_2_crop = Column(String(50), default="albahaca")
    plant_3_crop = Column(String(50), default="lechuga")
    plant_4_crop = Column(String(50), default="fresa")

    # Modo de Telemetría: Compartida por Torre (True) o Aislada por Canastilla (False)
    shared_telemetry = Column(Boolean, default=True)
    telemetry_mode = Column(String(50), default="shared_tower") # 'shared_tower' o 'isolated_node'

    # Ubicación Geográfica de la Torre (Santa Marta por defecto)
    latitude = Column(Float, default=11.2408)
    longitude = Column(Float, default=-74.1990)
    city_name = Column(String(100), default="Santa Marta, Magdalena")

    # Calibración de ROI Cenital Interactivo (Porcentajes normalizados 0.0 - 1.0)
    cenital_roi_cx = Column(Float, default=0.4955)
    cenital_roi_cy = Column(Float, default=0.4408)
    cenital_roi_radius = Column(Float, default=0.1450)
    cenital_roi_mode = Column(String(20), default="dynamic_conn") # 'dynamic_conn' o 'strict_circle'

    # Calibración de Línea Base Lateral Interactiva (Altura desde la maceta)
    lateral_baseline_y = Column(Float, default=0.7200) # Porcentaje de altura Y donde empieza el sustrato/borde de canastilla
    lateral_baseline_mode = Column(String(20), default="manual_baseline") # 'manual_baseline' o 'auto_bottom'

    # Calibración de Cuadro / Franja de Medición de Tallo (Scanline & Gradiente de Bordes)
    lateral_stem_roi_y = Column(Float, default=0.6600) # Centro Y del cuadro de tallo (normalizado 0.0 - 1.0)
    lateral_stem_roi_height = Column(Float, default=0.0500) # Altura del cuadro de tallo (normalizado 0.0 - 1.0)
    lateral_stem_roi_x = Column(Float, default=0.5000) # Centro X del cuadro de tallo (normalizado 0.0 - 1.0)
    lateral_stem_roi_width = Column(Float, default=0.2000) # Ancho del cuadro de tallo (normalizado 0.0 - 1.0)
    lateral_stem_mode = Column(String(20), default="auto_edge") # 'auto_edge' o 'disabled'

    def __init__(self, **kwargs):
```

```
        super().__init__(**kwargs)

    def to_json(self):
        return {
            "server_url": self.server_url,
            "plant_id": self.plant_id,
            "photos_per_period": self.photos_per_period,
            "capture_interval_sec": self.capture_interval_sec,
            "scheduled_times": [t.strip() for t in self.scheduled_times_str.split(",") if t.strip()],
            "safe_shutdown_enabled": self.safe_shutdown_enabled,
            "selected_crop_type": self.selected_crop_type,
            "plant_1_crop": self.plant_1_crop or "cebollin",
            "plant_2_crop": self.plant_2_crop or "albahaca",
            "plant_3_crop": self.plant_3_crop or "lechuga",
            "plant_4_crop": self.plant_4_crop or "fresa",
            "shared_telemetry": self.shared_telemetry if self.shared_telemetry is not None else True,
            "telemetry_mode": self.telemetry_mode or "shared_tower",
            "latitude": self.latitude if self.latitude is not None else 11.2408,
            "longitude": self.longitude if self.longitude is not None else -74.1990,
            "city_name": self.city_name or "Santa Marta, Magdalena",
            "cenital_roi_cx": self.cenital_roi_cx if self.cenital_roi_cx is not None else 0.4955,
            "cenital_roi_cy": self.cenital_roi_cy if self.cenital_roi_cy is not None else 0.4408,
            "cenital_roi_radius": self.cenital_roi_radius if self.cenital_roi_radius is not None else 0.1450,
            "cenital_roi_mode": self.cenital_roi_mode or "dynamic_conn",
            "lateral_baseline_y": self.lateral_baseline_y if self.lateral_baseline_y is not None else 0.7200,
            "lateral_baseline_mode": self.lateral_baseline_mode or "manual_baseline",
            "lateral_stem_roi_y": self.lateral_stem_roi_y if self.lateral_stem_roi_y is not None else 0.6600,
            "lateral_stem_roi_height": self.lateral_stem_roi_height if self.lateral_stem_roi_height is not None else 0.0500,
            "lateral_stem_roi_x": self.lateral_stem_roi_x if self.lateral_stem_roi_x is not None else 0.5000,
            "lateral_stem_roi_width": self.lateral_stem_roi_width if self.lateral_stem_roi_width is not None else 0.2000,
            "lateral_stem_mode": self.lateral_stem_mode or "auto_edge"
        }
```

ARCHIVO 37/85: local_server/domain/models/crop_profile.py (93 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from sqlalchemy import Column, Integer, String, Float, Boolean
from infrastructure.database.connection import db

class CropProfile(db.Model):
    """Entidad de perfil botánico y parámetros cromáticos/morfométricos por especie."""
    __tablename__ = 'crop_profile'

    crop_type = Column(String(50), primary_key=True)
    display_name = Column(String(100), nullable=False)

    # Rango HSV para segmentación
    h_min = Column(Integer, default=26)
    h_max = Column(Integer, default=92)
    s_min = Column(Integer, default=35)
    s_max = Column(Integer, default=255)
    v_min = Column(Integer, default=30)
    v_max = Column(Integer, default=255)

    # Rango LAB para segmentación
    l_min = Column(Integer, default=20)
    l_max = Column(Integer, default=255)
    a_min = Column(Integer, default=0)
    a_max = Column(Integer, default=124)
    b_min = Column(Integer, default=120)
    b_max = Column(Integer, default=255)

    # Parámetros morfométricos Cenitales
    pixel_to_cm_ratio = Column(Float, default=0.038)
    has_stem = Column(Boolean, default=True)

    # Parámetros de segmentación específicos para Cámara Lateral (Perfil Vertical)
    lat_h_min = Column(Integer, default=26)
    lat_h_max = Column(Integer, default=92)
    lat_s_min = Column(Integer, default=35)
    lat_s_max = Column(Integer, default=255)
    lat_v_min = Column(Integer, default=30)
    lat_v_max = Column(Integer, default=255)

    lat_l_min = Column(Integer, default=20)
    lat_l_max = Column(Integer, default=255)
    lat_a_min = Column(Integer, default=0)
    lat_a_max = Column(Integer, default=124)
    lat_b_min = Column(Integer, default=120)
    lat_b_max = Column(Integer, default=255)

    lat_pixel_to_cm_ratio = Column(Float, default=0.038)
    lat_min_area = Column(Integer, default=60)
    lat_filter_isolated = Column(Boolean, default=True)
    lat_draw_contours = Column(Boolean, default=False) # False: Imagen limpia solo con cotas métricas; True: Dibuja bordes rojos
```

```
54
55      def __init__(self, **kwargs):
56          super().__init__(**kwargs)
57
58      def to_json(self):
59          return {
60              "crop_type": self.crop_type,
61              "display_name": self.display_name,
62              "h_min": self.h_min,
63              "h_max": self.h_max,
64              "s_min": self.s_min,
65              "s_max": self.s_max,
66              "v_min": self.v_min,
67              "v_max": self.v_max,
68              "l_min": self.l_min,
69              "l_max": self.l_max,
70              "a_min": self.a_min,
71              "a_max": self.a_max,
72              "b_min": self.b_min,
73              "b_max": self.b_max,
74              "pixel_to_cm_ratio": self.pixel_to_cm_ratio,
75              "has_stem": self.has_stem,
76              "lat_h_min": getattr(self, 'lat_h_min', self.h_min),
77              "lat_h_max": getattr(self, 'lat_h_max', self.h_max),
78              "lat_s_min": getattr(self, 'lat_s_min', self.s_min),
79              "lat_s_max": getattr(self, 'lat_s_max', self.s_max),
80              "lat_v_min": getattr(self, 'lat_v_min', self.v_min),
81              "lat_v_max": getattr(self, 'lat_v_max', self.v_max),
82              "lat_l_min": getattr(self, 'lat_l_min', self.l_min),
83              "lat_l_max": getattr(self, 'lat_l_max', self.l_max),
84              "lat_a_min": getattr(self, 'lat_a_min', self.a_min),
85              "lat_a_max": getattr(self, 'lat_a_max', self.a_max),
86              "lat_b_min": getattr(self, 'lat_b_min', self.b_min),
87              "lat_b_max": getattr(self, 'lat_b_max', self.b_max),
88              "lat_pixel_to_cm_ratio": getattr(self, 'lat_pixel_to_cm_ratio', self.pixel_to_cm_ratio),
89              "lat_min_area": getattr(self, 'lat_min_area', 60),
90              "lat_filter_isolated": getattr(self, 'lat_filter_isolated', True),
91              "lat_draw_contours": getattr(self, 'lat_draw_contours', False)
92          }
93
```

ARCHIVO 38/85: local_server/domain/models/sensor_reading.py (51 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from datetime import datetime
from sqlalchemy import Column, Integer, Float, DateTime
from sqlalchemy.orm import relationship
from infrastructure.database.connection import db

class SensorReading(db.Model):
    """Entidad que almacena una muestra temporal de telemetría ambiental y calidad de agua."""
    __tablename__ = 'sensor_reading'

    id = Column(Integer, primary_key=True)
    timestamp = Column(DateTime, default=datetime.now, index=True)

    # Variables Ambientales y Exterior
    temperature = Column(Float, nullable=False) # Temperatura exterior (°C)
    humidity = Column(Float, nullable=False) # Humedad relativa exterior (% HR)
    uv_solar = Column(Float, nullable=True, default=0.0)
    motor_current = Column(Float, nullable=True, default=0.0)
    motor_state = Column(Integer, nullable=True, default=0) # Estado Motor: 1 (Encendido) / 0 (Apagado)

    # Variables Fisicoquímicas y de Calidad de Agua / Nutrientes
    water_ph = Column(Float, nullable=True, default=None) # pH Solución (Potencial de Hidrógeno)
    ec_conductivity = Column(Float, nullable=True, default=None) # µS/cm (Electroconductividad)
    turbidity_ntu = Column(Float, nullable=True, default=None) # NTU (Turbidez)
    dissolved_oxygen_mg_l = Column(Float, nullable=True, default=None) # mg/L (Oxígeno Disuelto)
    water_temp = Column(Float, nullable=True, default=None) # °C (Temperatura del agua)

    sessions = relationship("CaptureSession", back_populates="sensor_reading")

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def to_dict(self):
        return {
            "id": self.id,
            "timestamp": self.timestamp.strftime("%Y-%m-%d %H:%M:%S") if self.timestamp else None,
            "water_ph": round(self.water_ph, 2) if self.water_ph is not None else None,
            "ec_conductivity": round(self.ec_conductivity, 1) if self.ec_conductivity is not None else None,
            "turbidity_ntu": round(self.turbidity_ntu, 2) if self.turbidity_ntu is not None else None,
            "dissolved_oxygen_mg_l": round(self.dissolved_oxygen_mg_l, 2) if self.dissolved_oxygen_mg_l is not None else None,
            "water_temp": round(self.water_temp, 2) if self.water_temp is not None else None,
            "temperature": round(self.temperature, 2) if self.temperature is not None else 0.0,
            "humidity": round(self.humidity, 1) if self.humidity is not None else 0.0,
            "motor_state": int(self.motor_state) if self.motor_state is not None else int(self.motor_current if self.motor_current is not None else 0),
            "motor_current": round(self.motor_current, 3) if self.motor_current is not None else 0.0,
            "uv_solar": round(self.uv_solar, 1) if self.uv_solar is not None else 0.0
        }

```

ARCHIVO 39/85: local_server/domain/models/user.py (168 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import json
from datetime import datetime
from typing import Dict, Any, Optional
from sqlalchemy import Column, Integer, String, Text, Boolean, DateTime
from infrastructure.database.connection import db

ALL_MODULE_PERMISSIONS = [
    ("dashboard", "Resumen Diario", "dashboard", "Visualización de KPIs del día y resumen agronómico"),
    ("calendar", "Calendario de Muestreos", "calendar_month", "Navegación histórica de jornadas y muestreos"),
    ("processing", "Procesamiento y Visión", "camera_enhance", "Ejecución del pipeline OpenCV y detección USB"),
    ("gallery", "Galería de Lotes", "photo_library", "Explorador fotográfico y visor de tomas"),
    ("timelapse", "Time-Lapse Evolutivo", "slow_motion_video", "Generación y reproducción de video temporal"),
    ("sensors", "Sensores Ambientales", "sensors", "Telemetría en tiempo real y curvas fisicoquímicas"),
    ("cross_analysis", "Cruce y Correlaciones", "sync_alt", "Matriz de correlación multivariable de Pearson"),
    ("reports", "Dossier y Reportes", "assessment", "Generador interactivo de informes y fichas técnicas en PDF"),
    ("config", "Parámetros de Cultivo", "tune", "Calibración HSV/LAB, escala px/cm y ROI"),
    ("export_data", "Exportación de Datos", "download", "Descarga de reportes diarios y datasets CSV")
]

class User(db.Model):
    """
    Entidad de usuario del sistema SIFMA con control de acceso granular y basado en roles (RBAC).
    Roles soportados:
    - 'admin': Super Usuario / Administrador Principal (control total de la plataforma y usuarios)
    - 'investigador': Investigador / Agrónomo
    - 'operador': Técnico de Campo
    - 'personalizado': Rol a medida con permisos específicos por función
    """
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True)
    username = Column(String(60), unique=True, nullable=False, index=True)
    full_name = Column(String(120), nullable=False)
    email = Column(String(120), unique=True, nullable=True)
    password_hash = Column(String(256), nullable=False)
    role = Column(String(30), default='investigador', nullable=False) # 'admin', 'investigador', 'operador', 'personalizado'
    avatar_path = Column(String(255), nullable=True, default=None) # Ruta a la foto de perfil / avatar subido
    permissions = Column(Text, nullable=True, default=None) # JSON String con permisos granulares
    is_active = Column(Boolean, default=True, nullable=False)
    created_at = Column(DateTime, default=datetime.now)
    last_login = Column(DateTime, nullable=True)

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    @property
    def is_super_admin(self) -> bool:
        """Indica si el usuario es el Super Usuario / Administrador Principal."""
        return self.role == 'admin' or self.username.lower() == 'admin'
```

```
    @property
    def role_display(self) -> str:
        roles_map = {
            "admin": "Super Usuario (Admin)",
            "investigador": "Investigador / Agrónomo",
            "operador": "Técnico de Campo",
            "personalizado": "Acceso Personalizado"
        }
        return roles_map.get(self.role, "Usuario")

    @property
    def avatar_url(self) -> Optional[str]:
        """Retorna la URL web del avatar si tiene foto subida."""
        if self.avatar_path:
            clean = self.avatar_path.replace('\\', '/')
            if not clean.startswith('/') and not clean.startswith('http'):
                clean = '/' + clean
            return clean
        return None

    @property
    def avatar_initials(self) -> str:
        """Genera las 2 iniciales a partir del nombre completo."""
        if not self.full_name:
            return self.username[:2].upper()
        parts = self.full_name.strip().split()
        if len(parts) >= 2:
            return f"{parts[0][0]}{parts[1][0]}".upper()
        return parts[0][:2].upper()

    def get_permissions_dict(self) -> Dict[str, bool]:
        """Retorna el mapa booleano de permisos del usuario para cada módulo."""
        if self.is_super_admin:
            return {k: True for k, _, _, _ in ALL_MODULE_PERMISSIONS}

        parsed = {}
        if self.permissions:
            try:
                parsed = json.loads(self.permissions)
            except Exception:
                parsed = {}

        # Fallback predeterminado según el rol si no tiene JSON explícito
        if not parsed:
            if self.role == "investigador":
                parsed = {
                    "dashboard": True,
                    "calendar": True,
                    "processing": False,
                    "gallery": True,
                    "timelapse": True,
                    "sensors": True,
                    "cross_analysis": True,
                    "reports": True,
```

```
                    "config": False,
                    "export_data": True
                }
            elif self.role == "operador":
                parsed = {
                    "dashboard": True,
                    "calendar": True,
                    "processing": True,
                    "gallery": True,
                    "timelapse": False,
                    "sensors": True,
                    "cross_analysis": False,
                    "config": False,
                    "export_data": False
                }
            else:
                parsed = {k: False for k, _, _, _ in ALL_MODULE_PERMISSIONS}

        # Asegurar que todas las llaves existan
        res = {}
        for k, _, _, _ in ALL_MODULE_PERMISSIONS:
            res[k] = bool(parsed.get(k, False))
        return res

    def has_permission(self, permission_key: str) -> bool:
        """Verifica si el usuario tiene habilitado un permiso o módulo específico."""
        if self.is_super_admin:
            return True
        return self.get_permissions_dict().get(permission_key, False)

    def set_permissions_dict(self, perms_dict: Dict[str, Any]):
        """Serializa y almacena los permisos granulares."""
        if self.is_super_admin:
            self.permissions = json.dumps({k: True for k, _, _, _ in ALL_MODULE_PERMISSIONS})
        else:
            clean = {k: bool(perms_dict.get(k, False)) for k, _, _, _ in ALL_MODULE_PERMISSIONS}
            self.permissions = json.dumps(clean)

    def to_dict(self):
        perms = self.get_permissions_dict()
        enabled_count = sum(1 for v in perms.values() if v)
        return {
            "id": self.id,
            "username": self.username,
            "full_name": self.full_name,
            "email": self.email or "",
            "role": self.role,
            "role_display": self.role_display,
            "is_super_admin": self.is_super_admin,
            "avatar_initials": self.avatar_initials,
            "avatar_path": self.avatar_path,
            "avatar_url": self.avatar_url,
            "permissions": perms,
            "permissions_count": enabled_count,
            "total_permissions": len(ALL_MODULE_PERMISSIONS),
```

```
            "is_active": self.is_active,
            "created_at": self.created_at.strftime("%Y-%m-%d %H:%M") if self.created_at else None,
            "last_login": self.last_login.strftime("%Y-%m-%d %H:%M") if self.last_login else "Nunca"
        }

```

ARCHIVO 40/85: local_server/infrastructure/__init__.py (7 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .database.connection import db

__all__ = ["db"]
```

ARCHIVO 41/85: local_server/infrastructure/vision/__init__.py (13 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .strategies import ExgSegmentationStrategy
from .biometric_calculator import BiometricCalculator
from .pipeline_manager import VisionPipelineManager

__all__ = [
    "ExgSegmentationStrategy",
    "BiometricCalculator",
    "VisionPipelineManager"
]
```

ARCHIVO 42/85: local_server/infrastructure/vision/biometric_calculator.py (251 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from typing import List, Dict, Any, Optional
import cv2
import numpy as np
from domain.models import CropProfile

class BiometricCalculator:
    """
    Calculador de parámetros biométricos y morfométricos:
    Área Foliar (cm²), Índice de Salud (%), Altura de Planta (cm), Diámetro de Tallo (mm) e Índice de Compacidad.
    """

    def calculate_cenital(self, image: np.ndarray, mask: np.ndarray, contours: List[np.ndarray], profile: CropProfile) -> Dict[str, Any]:
        plant_pixels = 0
        compacity = 0.0

        if contours:
            plant_pixels = sum(cv2.contourArea(c) for c in contours)
            largest_contour = max(contours, key=cv2.contourArea)
            perimeter = cv2.arcLength(largest_contour, True)
            largest_area = cv2.contourArea(largest_contour)

            if perimeter > 0:
                compacity = (4 * np.pi * largest_area) / (perimeter ** 2)

        # Área física en cm²
        area_cm2 = plant_pixels * (profile.pixel_to_cm_ratio ** 2)

        # Índice de Salud Cromática (% de píxeles verde clorofila óptimo)
        hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
        lower_green = np.array([38, 45, 45])
        upper_green = np.array([82, 255, 255])
        mask_healthy = cv2.inRange(hsv, lower_green, upper_green)

        healthy_plant_pixels = cv2.countNonZero(cv2.bitwise_and(mask, mask_healthy)) if mask is not None else 0
        health_index = 100.0
        if plant_pixels > 0:
            health_index = (healthy_plant_pixels / plant_pixels) * 100.0
            health_index = min(100.0, max(0.0, health_index))

        return {
            "plant_pixels": plant_pixels,
            "area_cm2": round(area_cm2, 2),
            "health_index": round(health_index, 2),
            "compacity_index": round(compacity, 3)
        }

    def calculate_lateral(self, image: np.ndarray, mask: np.ndarray, contours: List[np.ndarray], profile: CropProfile, config: Optional[Any] = None) -> Dict[str, Any]:
        h, w, _ = image.shape
        height_cm = 0.0
```

```
        stem_diameter_mm = 0.0
        stem_draw_y = None
        stem_row_pixels = None
        y_min = None
        y_max = None
        x_points = None
        filtered_contours = []

        # Obtener línea base calibrada si está configurada
        baseline_mode = getattr(config, 'lateral_baseline_mode', 'manual_baseline') if config else 'manual_baseline'
        calib_baseline_ratio = getattr(config, 'lateral_baseline_y', None) if config else None

        if contours:
            # 1. Filtro Espacial de Cluster Central (Descartar bordes y manchas aisladas)
            filter_isolated = getattr(profile, 'lat_filter_isolated', True)

            if filter_isolated and len(contours) > 1:
                # El contorno más grande representa el follaje/tallo principal
                largest_contour = max(contours, key=cv2.contourArea)
                lx, ly, lw, lh = cv2.boundingRect(largest_contour)
                lcx = lx + (lw / 2.0)

                # Tolerancia de proximidad al cluster vegetal principal
                max_horizontal_dist = max(75.0, min(140.0, lw * 1.0))

                for c in contours:
                    cx, cy, cw, ch = cv2.boundingRect(c)
                    center_x = cx + (cw / 2.0)
                    if abs(center_x - lcx) <= max_horizontal_dist:
                        filtered_contours.append(c)

                if not filtered_contours:
                    filtered_contours = [largest_contour]
            else:
                filtered_contours = contours

            all_points = np.vstack([c for c in filtered_contours])
            y_points = all_points[:, 0, 1]
            x_points = all_points[:, 0, 0]

            y_min = int(np.min(y_points)) # Punto más alto detectado por OpenCV

            # Determinar la base de la planta (y_max):
            if baseline_mode == 'manual_baseline' and calib_baseline_ratio is not None and 0.1 <= float(calib_baseline_ratio) <= 0.98:
                calib_baseline_px = int(h * float(calib_baseline_ratio))
                # La base es la línea calibrada
                y_max = calib_baseline_px
            else:
                y_max = int(np.max(y_points))

            # Usar la escala vertical específica si existe, de lo contrario pixel_to_cm_ratio
            lateral_ratio = getattr(profile, 'lat_pixel_to_cm_ratio', None)
            if lateral_ratio is None or lateral_ratio <= 0:
                lateral_ratio = profile.pixel_to_cm_ratio

```

```
            if y_min < y_max:
                height_pixels = y_max - y_min
                height_cm = height_pixels * lateral_ratio

        # 4. Medición Inteligente de Diámetro de Tallo en Zona/Cuadro ROI Basal
        has_stem = getattr(profile, 'has_stem', True)
        stem_mode = getattr(config, 'lateral_stem_mode', 'auto_edge') if config else 'auto_edge'
        stem_data = {
            "stem_diameter_mm": 0.0,
            "stem_box": None,
            "stem_x_left": None,
            "stem_x_right": None,
            "stem_y_mid": None
        }

        if has_stem and stem_mode != 'disabled':
            stem_data = self.measure_stem_in_roi(image, config=config, profile=profile)
            stem_diameter_mm = stem_data.get("stem_diameter_mm", 0.0)

        return {
            "height_cm": round(height_cm, 2),
            "stem_diameter_mm": round(stem_diameter_mm, 2),
            "y_min": y_min,
            "y_max": y_max,
            "x_points": x_points,
            "stem_box": stem_data.get("stem_box"),
            "stem_x_left": stem_data.get("stem_x_left"),
            "stem_x_right": stem_data.get("stem_x_right"),
            "stem_y_mid": stem_data.get("stem_y_mid"),
            "filtered_contours": filtered_contours
        }

    def measure_stem_in_roi(self, image: np.ndarray, config: Optional[Any], profile: CropProfile) -> Dict[str, Any]:
        """
        Escanea la franja/cuadro interactivo de la base del tallo para medir su calibre
        utilizando análisis de gradiente horizontal (bordes del cilindro) e intensidad,
        independiente de si el tallo es verde, café, leñoso u oliva.
        """
        h, w = image.shape[:2]

        # Parámetros del ROI de tallo
        stem_roi_x = getattr(config, 'lateral_stem_roi_x', 0.5000) if config else 0.5000
        stem_roi_y = getattr(config, 'lateral_stem_roi_y', 0.6600) if config else 0.6600
        stem_roi_w = getattr(config, 'lateral_stem_roi_width', 0.2000) if config else 0.2000
        stem_roi_h = getattr(config, 'lateral_stem_roi_height', 0.0500) if config else 0.0500

        cx = int(w * float(stem_roi_x))
        cy = int(h * float(stem_roi_y))
        box_w = max(24, int(w * float(stem_roi_w)))
        box_h = max(10, int(h * float(stem_roi_h)))

        x1 = max(0, cx - box_w // 2)
        x2 = min(w, cx + box_w // 2)
        y1 = max(0, cy - box_h // 2)
        y2 = min(h, cy + box_h // 2)
```

```
        lateral_ratio = getattr(profile, 'lat_pixel_to_cm_ratio', None)
        if lateral_ratio is None or lateral_ratio <= 0:
            lateral_ratio = profile.pixel_to_cm_ratio

        roi = image[y1:y2, x1:x2]
        if roi.size == 0:
            return {
                "stem_diameter_mm": 0.0,
                "stem_box": (x1, y1, x2, y2),
                "stem_x_left": None,
                "stem_x_right": None,
                "stem_y_mid": cy
            }

        # 1. Convertir a escala de grises y suavizar para reducir ruido de impresión 3D
        gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)

        # 2. Gradiente Horizontal Sobel (resalta las dos paredes verticales del tallo cilíndrico)
        grad_x = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
        abs_grad_x = np.abs(grad_x)

        detected_widths = []
        detected_lefts = []
        detected_rights = []
        detected_ys = []

        roi_mid = (x2 - x1) // 2

        for r in range(roi.shape[0]):
            actual_y = y1 + r
            row_profile = abs_grad_x[r, :]

            # Buscar el pico de gradiente más fuerte a la izquierda del centro
            left_half = row_profile[:roi_mid]
            right_half = row_profile[roi_mid:]

            if len(left_half) > 3 and len(right_half) > 3:
                left_idx = np.argmax(left_half)
                right_idx = roi_mid + np.argmax(right_half)

                width_px = right_idx - left_idx
                # Validar que el ancho detectado sea razonable para un tallo botánico (ej: 4 a 60 px)
                if 4 <= width_px <= int((x2 - x1) * 0.85):
                    if row_profile[left_idx] > 6 and row_profile[right_idx] > 6:
                        detected_widths.append(width_px)
                        detected_lefts.append(x1 + left_idx)
                        detected_rights.append(x1 + right_idx)
                        detected_ys.append(actual_y)

        if detected_widths:
            median_width = float(np.median(detected_widths))
            stem_diameter_mm = median_width * lateral_ratio * 10.0 # Conversión de cm a mm
            stem_x_left = int(np.median(detected_lefts))
```

```
            stem_x_right = int(np.median(detected_rights))
            stem_y_mid = int(np.median(detected_ys))
        else:
            # Umbralización adaptativa como respaldo en caso de bajo contraste
            _, thresh = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
            cnts, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
            if cnts:
                best_c = max(cnts, key=cv2.contourArea)
                bx, by, bw, bh = cv2.boundingRect(best_c)
                if 4 <= bw <= int((x2 - x1) * 0.85):
                    median_width = float(bw)
                    stem_diameter_mm = median_width * lateral_ratio * 10.0
                    stem_x_left = x1 + bx
                    stem_x_right = x1 + bx + bw
                    stem_y_mid = y1 + by + bh // 2
                else:
                    stem_diameter_mm = 0.0
                    stem_x_left = None
                    stem_x_right = None
                    stem_y_mid = cy
            else:
                stem_diameter_mm = 0.0
                stem_x_left = None
                stem_x_right = None
                stem_y_mid = cy

        return {
            "stem_diameter_mm": round(stem_diameter_mm, 2),
            "stem_box": (x1, y1, x2, y2),
            "stem_x_left": stem_x_left,
            "stem_x_right": stem_x_right,
            "stem_y_mid": stem_y_mid
        }
```

ARCHIVO 43/85: local_server/infrastructure/vision/pipeline_manager.py (430 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
from typing import List, Dict, Any, Optional, Tuple
import cv2
import numpy as np
from domain.models import CropProfile
from domain.interfaces.vision import ISegmentationStrategy, IBiometricCalculator
from infrastructure.vision.strategies.exg_segmentation import ExgSegmentationStrategy
from infrastructure.vision.biometric_calculator import BiometricCalculator

class VisionPipelineManager:
    """
    Orquestador del pipeline de visión artificial agronómica SIFMA.
    - Corrige orientación física de las cámaras (Cenital 180° / Lateral 90° derecha).
    - Aplica ROI circular automático sobre la canastilla azul (anula personas, ropa y fondo).
    - Mide altura y tallo basal con precisión milimétrica desde la base del sustrato.
    """

    def __init__(
        self,
        segmentation_strategy: Optional[ISegmentationStrategy] = None,
        biometric_calculator: Optional[IBiometricCalculator] = None
    ):
        self.segmenter = segmentation_strategy or ExgSegmentationStrategy()
        self.calculator = biometric_calculator or BiometricCalculator()

    @staticmethod
    def is_cenital_inverted(img: np.ndarray) -> bool:
        """
        Detecta si la foto cenital está de cabeza (al revés con la base del tubo abajo).
        Si la base azul está abajo, retorna True para requerir rotación de 180°.
        """
        h, w = img.shape[:2]
        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        mask_blue = cv2.inRange(hsv, np.array([95, 50, 40]), np.array([135, 255, 255]))
        col_w_start, col_w_end = int(w * 0.40), int(w * 0.60)
        top_blue = cv2.countNonZero(mask_blue[:int(h * 0.20), col_w_start:col_w_end])
        bottom_blue = cv2.countNonZero(mask_blue[int(h * 0.80):, col_w_start:col_w_end])
        return bottom_blue > top_blue

    @staticmethod
    def orient_lateral_upright(img: np.ndarray) -> np.ndarray:
        """
        Asegura que la foto lateral quede con orientación vertical perfecta:
        canastilla azul con arlita en la base inferior y planta creciendo erguida hacia arriba.
        """
        h, w = img.shape[:2]
        # 1. Si está en formato horizontal (sensor apaisado), rotar a vertical
        if w > h:
            img = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
```

```
            h, w = img.shape[:2]

        # 2. Verificar que la arlita (sustrato de cultivo) esté en la mitad inferior
        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        mask_arlita = cv2.inRange(hsv, np.array([5, 60, 40]), np.array([22, 255, 180]))
        pts = np.argwhere(mask_arlita > 0)

        if len(pts) > 500:
            mean_y = np.mean(pts[:, 0])
            # Si el sustrato está en la mitad superior (mean_y < h/2), la planta está de cabeza -> rotar 180°
            if mean_y < (h / 2.0):
                img = cv2.rotate(img, cv2.ROTATE_180)

        return img

    @staticmethod
    def detect_blue_basket_roi(image: np.ndarray) -> tuple:
        """
        Detecta con precisión sub-milimétrica la canastilla circular azul individual donde
        está plantado el cultivo activo, aislando arlita y follaje, anulando el fondo de laboratorio.
        Retorna: (centro_x, centro_y, radio)
        """
        h_img, w_img = image.shape[:2]
        hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

        # 1. Segmentación de vegetación activa
        lower_green = np.array([25, 30, 30])
        upper_green = np.array([88, 255, 255])
        mask_green = cv2.inRange(hsv, lower_green, upper_green)

        b, g, r = cv2.split(image.astype(np.float32))
        exg = 2.0 * g - r - b
        exg_mask = np.uint8((exg > 15) & (g > r) & (g > b)) * 255
        plant_mask = cv2.bitwise_or(mask_green, exg_mask)

        # 2. Segmentación estricta del plástico azul de la canastilla
        lower_blue = np.array([95, 50, 40])
        upper_blue = np.array([135, 255, 255])
        mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)

        # Kernel elíptico moderado para mantener la individualidad del aro
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        mask_blue_clean = cv2.morphologyEx(mask_blue, cv2.MORPH_OPEN, kernel)
        mask_blue_clean = cv2.morphologyEx(mask_blue_clean, cv2.MORPH_CLOSE, kernel)

        contours, _ = cv2.findContours(mask_blue_clean, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        # Rango exacto del radio de una canastilla azul individual (~10% a 14% de la altura)
        min_radius = int(min(w_img, h_img) * 0.08)
        max_radius = int(min(w_img, h_img) * 0.15)
        candidates = []

        for c in contours:
            area = cv2.contourArea(c)
            if area < 3000 or area > 600000:
```

```
                continue

            (cx, cy), radius = cv2.minEnclosingCircle(c)
            radius = int(radius)
            if radius < min_radius or radius > max_radius:
                continue

            peri = cv2.arcLength(c, True)
            circularity = (4 * np.pi * area) / (peri * peri) if peri > 0 else 0

            bx, by, bw, bh = cv2.boundingRect(c)
            aspect_ratio = float(bw) / float(bh) if bh > 0 else 0

            # Solo evaluar canastillas ubicadas en la columna central de la torre
            if not (w_img * 0.38 <= cx <= w_img * 0.62 and h_img * 0.30 <= cy <= h_img * 0.60):
                continue

            c_mask = np.zeros((h_img, w_img), dtype=np.uint8)
            cv2.circle(c_mask, (int(cx), int(cy)), radius, 255, -1)
            green_inside = cv2.countNonZero(cv2.bitwise_and(plant_mask, c_mask))

            # Puntuación agrónoma:
            # - Máxima presencia de planta
            # - Circularidad geométrica
            # - Relación de aspecto ~ 1.0 (aro circular)
            aspect_diff = abs(1.0 - aspect_ratio)
            score = (green_inside * 4.0) + (circularity * 3000.0) - (aspect_diff * 800.0)

            candidates.append({
                'cx': int(cx), 'cy': int(cy), 'radius': radius,
                'green_inside': green_inside, 'score': score
            })

        if candidates:
            best = max(candidates, key=lambda k: k['score'])
    def detect_blue_basket_roi(self, img: np.ndarray, config: Optional[Any] = None) -> Tuple[int, int, int]:
        """
        Calcula el ROI cenital. Si el usuario configuró coordenadas personalizadas en Config,
        las utiliza; de lo contrario busca el aro azul automáticamente con fallback agrónomo.
        """
        h_img, w_img = img.shape[:2]

        # 1. Si hay configuración explícita guardada por el usuario
        if config is not None:
            cx_ratio = getattr(config, 'cenital_roi_cx', None)
            cy_ratio = getattr(config, 'cenital_roi_cy', None)
            rad_ratio = getattr(config, 'cenital_roi_radius', None)
            if cx_ratio is not None and cy_ratio is not None and rad_ratio is not None:
                cx = int(w_img * float(cx_ratio))
                cy = int(h_img * float(cy_ratio))
                radius = int(min(w_img, h_img) * float(rad_ratio))
                return cx, cy, radius

        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        mask_blue = cv2.inRange(hsv, np.array([90, 45, 30]), np.array([140, 255, 255]))
```

```
        plant_mask = self.segmenter.create_mask(img, CropProfile(crop_type="detect", display_name="detect"))

        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        mask_blue_clean = cv2.morphologyEx(mask_blue, cv2.MORPH_OPEN, kernel)
        mask_blue_clean = cv2.morphologyEx(mask_blue_clean, cv2.MORPH_CLOSE, kernel)

        contours, _ = cv2.findContours(mask_blue_clean, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        min_radius = int(min(w_img, h_img) * 0.08)
        max_radius = int(min(w_img, h_img) * 0.18)
        candidates = []

        for c in contours:
            area = cv2.contourArea(c)
            if area < 3000 or area > 600000:
                continue

            (cx, cy), radius = cv2.minEnclosingCircle(c)
            radius = int(radius)
            if radius < min_radius or radius > max_radius:
                continue

            peri = cv2.arcLength(c, True)
            circularity = (4 * np.pi * area) / (peri * peri) if peri > 0 else 0

            bx, by, bw, bh = cv2.boundingRect(c)
            aspect_ratio = float(bw) / float(bh) if bh > 0 else 0

            if not (w_img * 0.35 <= cx <= w_img * 0.65 and h_img * 0.28 <= cy <= h_img * 0.62):
                continue

            c_mask = np.zeros((h_img, w_img), dtype=np.uint8)
            cv2.circle(c_mask, (int(cx), int(cy)), radius, 255, -1)
            green_inside = cv2.countNonZero(cv2.bitwise_and(plant_mask, c_mask))

            aspect_diff = abs(1.0 - aspect_ratio)
            score = (green_inside * 4.0) + (circularity * 3000.0) - (aspect_diff * 800.0)

            candidates.append({
                'cx': int(cx), 'cy': int(cy), 'radius': radius,
                'green_inside': green_inside, 'score': score
            })

        if candidates:
            best = max(candidates, key=lambda k: k['score'])
            return best['cx'], best['cy'], int(best['radius'] * 1.05)

        # Calibración agrónoma física central por defecto
        calib_cx = int(w_img * 0.4955)
        calib_cy = int(h_img * 0.4408)
        calib_radius = int(min(w_img, h_img) * 0.145)
        return calib_cx, calib_cy, calib_radius

    def process_cenital(self, image_path: str, output_path: str, profile: CropProfile, config: Optional[Any] = None) -> Optional[Dict[str, Any]]:
        os.makedirs(os.path.dirname(output_path), exist_ok=True)
```

```
        img_raw = cv2.imread(image_path)
        if img_raw is None:
            return None

        # 1. Rotación de 180° antes de procesar si está al revés
        if self.is_cenital_inverted(img_raw):
            img = cv2.rotate(img_raw, cv2.ROTATE_180)
            cv2.imwrite(image_path, img)
        else:
            img = img_raw

        h_img, w_img = img.shape[:2]

        # 2. Obtención de centro y radio del ROI (Manual o Automático)
        cx, cy, radius = self.detect_blue_basket_roi(img, config=config)
        roi_mode = getattr(config, 'cenital_roi_mode', 'dynamic_conn') if config else 'dynamic_conn'

        roi_mask = np.zeros((h_img, w_img), dtype=np.uint8)
        cv2.circle(roi_mask, (cx, cy), radius, 255, -1)

        # 3. Segmentación foliar
        raw_leaf_mask = self.segmenter.create_mask(img, profile)

        # Filtro Inteligente de Componentes Conectados (Filtro Anti-Plantas Vecinas)
        if roi_mode == 'dynamic_conn':
            # Aplicar ROI amplio y filtrar islas desconectadas que pertenezcan a otras canastillas
            bounded_leaf = cv2.bitwise_and(raw_leaf_mask, roi_mask)
            num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(bounded_leaf, connectivity=8)

            # Centroide de la canastilla
            target_mask = np.zeros_like(bounded_leaf)
            inner_core_radius = max(15, int(radius * 0.45))

            for label_idx in range(1, num_labels):
                area = stats[label_idx, cv2.CC_STAT_AREA]
                if area < 25:
                    continue
                comp_mask = (labels == label_idx).astype(np.uint8)

                # Verificar si el componente toca el núcleo central de la canastilla
                # o está lo suficientemente cerca del centro
                dist_to_center = np.sqrt((centroids[label_idx][0] - cx)**2 + (centroids[label_idx][1] - cy)**2)

                # Si está dentro del rango o toca la zona central
                comp_core_overlap = cv2.countNonZero(cv2.bitwise_and(comp_mask, cv2.circle(np.zeros_like(comp_mask), (cx, cy), inner_core_radius, 255, -1)))

                if comp_core_overlap > 0 or dist_to_center <= (radius * 0.85):
                    target_mask = cv2.bitwise_or(target_mask, comp_mask * 255)

            mask = target_mask if cv2.countNonZero(target_mask) > 0 else bounded_leaf
        else:
            mask = cv2.bitwise_and(raw_leaf_mask, roi_mask)

        contours = self.segmenter.extract_contours(mask, min_area=25)

```

```
        # 4. Cálculo de Biometría Foliar
        metrics = self.calculator.calculate_cenital(img, mask, contours, profile)

        # 5. Generación de Overlay de visualización
        overlay = img.copy()

        # Dibujar círculo de delimitación del ROI
        cv2.circle(overlay, (cx, cy), radius, (255, 180, 0), 3)  # Círculo cian/azul claro
        # Cruz central de anclaje
        cross_len = 16
        cv2.line(overlay, (cx - cross_len, cy), (cx + cross_len, cy), (0, 255, 0), 2)
        cv2.line(overlay, (cx, cy - cross_len), (cx, cy + cross_len), (0, 255, 0), 2)

        cv2.putText(overlay, "ROI", (max(20, cx - 100), max(30, cy - radius - 12)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.70, (255, 180, 0), 2)

        if contours:
            # Dibujar contornos foliares en ROJO puro
            cv2.drawContours(overlay, contours, -1, (0, 0, 255), 2)

            # Envoltura convexa de la canopia
            all_pts = np.vstack(contours)
            if len(all_pts) >= 3:
                hull = cv2.convexHull(all_pts)
                cv2.drawContours(overlay, [hull], -1, (0, 0, 255), 2)

        metrics["fruits_count"] = 0
        metrics["spots_count"] = 0

        # Superponer leyenda de métricas biométricas
        cv2.putText(overlay, f"Area Foliar: {metrics['area_cm2']:.1f} cm2", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
        cv2.putText(overlay, f"Salud Foliar: {metrics['health_index']:.1f}%", (20, 75),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

        cv2.imwrite(output_path, overlay)
        return metrics

    def process_lateral(self, image_path: str, output_path: str, profile: CropProfile, config: Optional[Any] = None) -> Optional[Dict[str, Any]]:
        os.makedirs(os.path.dirname(output_path), exist_ok=True)
        img_raw = cv2.imread(image_path)
        if img_raw is None:
            return None

        # 1. Orientación vertical erguida antes de procesar
        img = self.orient_lateral_upright(img_raw)
        cv2.imwrite(image_path, img)

        h, w = img.shape[:2]

        # 2. Segmentación botánica lateral de tallos y hojas
        mask = self.segmenter.create_mask(img, profile, is_lateral=True)
        min_area = int(getattr(profile, 'lat_min_area', 30))
        contours = self.segmenter.extract_contours(mask, min_area=min_area)

```

```
        # 3. Cálculo de altura vertical y diámetro basal de tallo
        metrics = self.calculator.calculate_lateral(img, mask, contours, profile, config=config)
        plant_contours = metrics.get("filtered_contours", contours)

        overlay = img.copy()

        # Dibujar la línea base horizontal calibrada de la maceta (en rojo/cian neón)
        baseline_ratio = getattr(config, 'lateral_baseline_y', 0.72) if config else 0.72
        baseline_px = int(h * float(baseline_ratio))
        cv2.line(overlay, (0, baseline_px), (w, baseline_px), (0, 0, 255), 2)
        cv2.putText(overlay, "LINEA BASE MACETA", (15, max(25, baseline_px - 8)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 0, 255), 2)

        if plant_contours:
            draw_contours = getattr(profile, 'lat_draw_contours', False)
            if draw_contours:
                # Dibujar contornos en ROJO sobre tallos y hojas solo si está habilitado
                cv2.drawContours(overlay, plant_contours, -1, (0, 0, 255), 2)

            y_min = metrics.get("y_min")  # Ápice foliar superior (detectado por OpenCV)
            y_max = metrics.get("y_max", baseline_px)  # Base calibrada de la planta
            x_points = metrics.get("x_points")

            if y_min is not None and y_max is not None and y_min < y_max and x_points is not None and len(x_points) > 0:
                x_center = int(np.mean(x_points))

                # Línea de medición vertical completa desde la base hasta la punta más alta
                cv2.line(overlay, (x_center - 25, y_min), (x_center + 25, y_min), (0, 255, 255), 2)
                cv2.line(overlay, (x_center - 25, y_max), (x_center + 25, y_max), (0, 255, 255), 2)
                cv2.line(overlay, (x_center, y_min), (x_center, y_max), (0, 255, 255), 2)

                # Etiqueta de altura
                cv2.putText(overlay, f"H: {metrics['height_cm']:.1f} cm", (x_center + 15, int((y_min + y_max)/2)),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 255), 2)

            # 4. Dibujar Cuadro ROI de Tallo y Medición de Diámetro
            stem_box = metrics.get("stem_box")
            stem_x_left = metrics.get("stem_x_left")
            stem_x_right = metrics.get("stem_x_right")
            stem_y_mid = metrics.get("stem_y_mid")
            stem_d = metrics.get("stem_diameter_mm", 0.0)

            if stem_box:
                bx1, by1, bx2, by2 = stem_box
                # Dibujar recuadro de la zona de escaneo en naranja/ámbar
                cv2.rectangle(overlay, (bx1, by1), (bx2, by2), (255, 165, 0), 1)
                cv2.putText(overlay, "SCAN TALLO", (bx1, max(18, by1 - 5)),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.42, (255, 165, 0), 1)

            if stem_x_left is not None and stem_x_right is not None and stem_d > 0 and stem_y_mid is not None:
                # Línea de medición de calibre del tallo con topes verticales
                cv2.line(overlay, (stem_x_left, stem_y_mid), (stem_x_right, stem_y_mid), (0, 255, 255), 2)
                cv2.line(overlay, (stem_x_left, stem_y_mid - 4), (stem_x_left, stem_y_mid + 4), (0, 255, 255), 2)
                cv2.line(overlay, (stem_x_right, stem_y_mid - 4), (stem_x_right, stem_y_mid + 4), (0, 255, 255), 2)
                cv2.putText(overlay, f"D: {stem_d:.1f} mm", (bx2 + 8 if stem_box else stem_x_right + 12, stem_y_mid + 5),
```

```
                            cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 255), 2)

        cv2.imwrite(output_path, overlay)
        return {
            "plant_height_cm": metrics.get("height_cm", 0.0),
            "stem_diameter_mm": metrics.get("stem_diameter_mm", 0.0)
        }

    @staticmethod
    def filter_and_average(cenital_results: List[Dict[str, Any]], lateral_results: List[Dict[str, Any]]) -> Dict[str, Any]:
        """Elimina valores atípicos (+- 2 desviaciones estándar) y calcula medias robustas del lote."""
        final_metrics = {
            "foliar_area_cm2": 0.0,
            "health_index": 100.0,
            "compacity_index": 0.0,
            "spots_count": 0,
            "fruits_count": 0,
            "plant_height_cm": 0.0,
            "stem_diameter_mm": 0.0
        }

        def get_robust_mean(data_list):
            if not data_list: return 0.0
            if len(data_list) <= 2: return float(np.mean(data_list))
            mean = float(np.mean(data_list))
            std = float(np.std(data_list))
            if std == 0: return mean
            filtered = [x for x in data_list if abs(x - mean) <= 2 * std]
            return float(np.mean(filtered)) if filtered else mean

        if cenital_results:
            areas = [r.get("area_cm2", 0.0) for r in cenital_results]
            healths = [r.get("health_index", 100.0) for r in cenital_results]
            compacities = [r.get("compacity_index", 0.0) for r in cenital_results]

            final_metrics["foliar_area_cm2"] = round(get_robust_mean(areas), 2)
            final_metrics["health_index"] = round(get_robust_mean(healths), 2)
            final_metrics["compacity_index"] = round(get_robust_mean(compacities), 3)

        if lateral_results:
            heights = [r.get("plant_height_cm", 0.0) for r in lateral_results]
            diameters = [r.get("stem_diameter_mm", 0.0) for r in lateral_results]

            final_metrics["plant_height_cm"] = round(get_robust_mean(heights), 2)
            final_metrics["stem_diameter_mm"] = round(get_robust_mean(diameters), 2)

        return final_metrics
```

ARCHIVO 44/85: local_server/infrastructure/vision/strategies/__init__.py (7 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .exg_segmentation import ExgSegmentationStrategy

__all__ = ["ExgSegmentationStrategy"]
```

ARCHIVO 45/85: local_server/infrastructure/vision/strategies/exg_segmentation.py (94 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from typing import List, Optional
import cv2
import numpy as np
from domain.models import CropProfile

class ExgSegmentationStrategy:
    """
    Estrategia de segmentación botánica avanzada basada en el Índice de Exceso de Verde (ExG),
    combinado con filtrado HSV de clorofila y canal LAB.
    (Pattern: Strategy)
    """

    def create_mask(self, image: np.ndarray, profile: CropProfile, is_lateral: bool = False) -> Optional[np.ndarray]:
        """
        Genera la máscara binaria aislando exclusivamente tejido vegetal.
        Descarta piel humana, tierra, sustratos y plástico blanco.
        Soporta calibración independiente para vista cenital y vista lateral.
        """
        if image is None:
            return None

        # 1. Índice de Exceso de Verde (ExG = 2*G - R - B)
        b, g, r = cv2.split(image.astype(np.float32))
        exg = 2.0 * g - r - b

        # En vista lateral se usa un umbral ExG más sensible para captar tallos finos basales
        exg_thresh = 6.0 if is_lateral else 10.0
        exg_mask = (exg > exg_thresh).astype(np.uint8) * 255

        # 2. Umbralizado en espacio HSV para clorofila
        hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
        if is_lateral:
            h_min = max(20, int(getattr(profile, 'lat_h_min', None) or 22))
            h_max = min(96, int(getattr(profile, 'lat_h_max', None) or 92))
            s_min = max(20, int(getattr(profile, 'lat_s_min', None) or 25))
            s_max = int(getattr(profile, 'lat_s_max', None) or 255)
            v_min = max(20, int(getattr(profile, 'lat_v_min', None) or 25))
            v_max = int(getattr(profile, 'lat_v_max', None) or 255)

            l_min = max(10, int(getattr(profile, 'lat_l_min', None) or 15))
            l_max = int(getattr(profile, 'lat_l_max', None) or 255)
            # Canal A < 122 discrimina el fondo blanco sin cortar tallos
            a_max = min(124, int(getattr(profile, 'lat_a_max', None) or 122))
            b_min = max(115, int(getattr(profile, 'lat_b_min', None) or 118))
        else:
            h_min = max(24, int(getattr(profile, 'h_min', None) or 28))
            h_max = min(96, int(getattr(profile, 'h_max', None) or 90))
            s_min = max(30, int(getattr(profile, 's_min', None) or 35))
            s_max = int(getattr(profile, 's_max', None) or 255)
```

```
            v_min = max(25, int(getattr(profile, 'v_min', None) or 30))
            v_max = int(getattr(profile, 'v_max', None) or 255)

            l_min = int(getattr(profile, 'l_min', None) or 20)
            l_max = int(getattr(profile, 'l_max', None) or 255)
            a_max = min(126, int(getattr(profile, 'a_max', None) or 124))
            b_min = max(115, int(getattr(profile, 'b_min', None) or 120))

        lower_hsv = np.array([h_min, s_min, v_min])
        upper_hsv = np.array([h_max, s_max, v_max])
        mask_hsv = cv2.inRange(hsv, lower_hsv, upper_hsv)

        # 3. Umbralizado en espacio LAB
        lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
        lower_lab = np.array([l_min, 0, b_min])
        upper_lab = np.array([l_max, a_max, 255])
        mask_lab = cv2.inRange(lab, lower_lab, upper_lab)

        # 4. Fusión de máscaras por intersección
        mask = cv2.bitwise_and(exg_mask, mask_hsv)
        mask = cv2.bitwise_and(mask, mask_lab)

        # Fallback de respaldo SOLO para vista cenital (en lateral LAB es fundamental para anular bordes)
        if not is_lateral:
            if cv2.countNonZero(mask) < 0.25 * cv2.countNonZero(cv2.bitwise_and(exg_mask, mask_hsv)):
                mask = cv2.bitwise_and(exg_mask, mask_hsv)

        # 5. Filtrado morfológico de ruido
        k_size = (5, 5) if is_lateral else (3, 3)
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, k_size)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

        return mask

    def extract_contours(self, mask: np.ndarray, min_area: int = 30) -> List[np.ndarray]:
        """Extrae y filtra contornos por tamaño mínimo en píxeles."""
        if mask is None:
            return []
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        return [c for c in contours if cv2.contourArea(c) >= min_area]
```

ARCHIVO 46/85: local_server/presentation/__init__.py (8 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .api import api_bp
from .web import dashboard_bp

__all__ = ["api_bp", "dashboard_bp"]
```

ARCHIVO 47/85: local_server/presentation/api/__init__.py (15 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from flask import Blueprint

api_bp = Blueprint('api', __name__)

from . import telemetry_routes
from . import batch_routes
from . import conclusion_routes
from . import session_routes
from . import user_routes

__all__ = ["api_bp"]
```

ARCHIVO 48/85: local_server/presentation/api/batch_routes.py (290 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import json
import shutil
import tempfile
import zipfile
import re
from flask import request, jsonify, session
from core.security import SecurityService
from application.services import VisionApplicationService, SystemService
from . import api_bp

vision_service = VisionApplicationService()
system_service = SystemService()

@api_bp.route('/api/config', methods=['GET'])
def get_config():
    config = system_service.get_config()
    return jsonify(config.to_json())

@api_bp.route('/api/scan_usb', methods=['GET'])
def scan_usb_drives():
    try:
        batches = vision_service.detect_connected_usb_drives()
        return jsonify({
            "status": "success",
            "count": len(batches),
            "batches": batches,
            "drives": batches
        })
    except Exception as e:
        return jsonify({"status": "error", "message": f"Error al escanear unidades USB: {e}"}), 500

@api_bp.route('/api/processing/progress', methods=['GET'])
def get_processing_progress():
    """Retorna el estado de progreso en tiempo real del procesamiento de imágenes."""
    return jsonify(vision_service.get_progress())

@api_bp.route('/api/process_usb_path', methods=['POST'])
def process_usb_path():
    data = request.json or {}
    folder_paths = data.get("folder_paths") or []
    single_path = data.get("folder_path") or data.get("path") or data.get("mountpoint")
    if single_path and single_path not in folder_paths:
        folder_paths.append(single_path)

    crop_type = data.get("crop_type")
    period = data.get("period", "Día 1")
    plant_id = data.get("plant_id") or session.get("active_plant_id", 1)

```

```
    if not folder_paths:
        vision_service.set_progress(0, "Ruta no encontrada", is_processing=False)
        return jsonify({"status": "error", "message": "No se especificaron carpetas para procesar."}), 400

    try:
        # Recopilar todos los subdirectorios con imágenes de todas las carpetas seleccionadas
        all_subdirs_to_process = []
        for f_path in folder_paths:
            if not f_path or not os.path.exists(f_path):
                continue
            subdirs = []
            for r_dir, _, f_list in os.walk(f_path):
                imgs = [img for img in f_list if img.lower().endswith(('.png', '.jpg', '.jpeg'))]
                if imgs and r_dir != f_path:
                    subdirs.append((f_path, r_dir, imgs))
            if subdirs:
                all_subdirs_to_process.extend(subdirs)
            else:
                # Fotos directas en la raiz de f_path
                direct_imgs = [img for img in os.listdir(f_path) if img.lower().endswith(('.png', '.jpg', '.jpeg'))]
                if direct_imgs:
                    all_subdirs_to_process.append((f_path, f_path, direct_imgs))

        if not all_subdirs_to_process:
            vision_service.set_progress(0, "No se encontraron fotos en las carpetas seleccionadas.", is_processing=False)
            return jsonify({"status": "error", "message": "No se encontraron fotos válidas (.jpg, .png) en las carpetas seleccionadas."}), 400

        total_all_photos = sum(len(imgs) for _, _, imgs in all_subdirs_to_process)
        accumulated_photos = 0
        processed_results = []

        vision_service.set_progress(
            percent=5,
            message=f"Iniciando procesamiento de {len(all_subdirs_to_process)} turno(s) ({total_all_photos} fotos en total)...",
            is_processing=True,
            current_item=0,
            total_items=total_all_photos
        )

        for root_folder, sub_dir, imgs in all_subdirs_to_process:
            sub_name = os.path.basename(sub_dir).lower()
            if 'manana' in sub_name or 'mañana' in sub_name:
                turn_label = "Mañana"
            elif 'medio_dia' in sub_name or 'mediodia' in sub_name or 'medio-dia' in sub_name:
                turn_label = "Mediodía"
            elif 'tarde' in sub_name:
                turn_label = "Tarde"
            else:
                turn_label = sub_name.replace('_', ' ').title()

            day_label = os.path.basename(root_folder)
            full_turn_period = f"{day_label} - {turn_label}" if re.match(r'^\d{4}[-_]\d{2}[-_]\d{2}$', day_label) else (f"{period} - {turn_label}" if period != "Día 1"
else turn_label)

            r = vision_service.process_folder_batch(
```

```
                sub_dir,
                crop_type=crop_type,
                period=full_turn_period,
                plant_id=int(plant_id),
                global_offset=accumulated_photos,
                total_global_photos=total_all_photos,
                turn_label=turn_label
            )
            accumulated_photos += len(imgs)
            processed_results.append(r)

        vision_service.set_progress(
            percent=100,
            message=f"¡Procesamiento fenotípico finalizado con éxito! ({total_all_photos} fotos procesadas)",
            is_processing=False
        )

        return jsonify({
            "status": "success",
            "message": f"Se procesaron {len(processed_results)} turnos ({total_all_photos} fotos) exitosamente para Canastilla #{plant_id}.",
            "data": processed_results[0] if len(processed_results) == 1 else processed_results,
            "processed_batches": processed_results,
            "total_photos": total_all_photos
        })
    except Exception as e:
        vision_service.set_progress(0, f"Error: {e}", is_processing=False)
        return jsonify({"status": "error", "message": f"Error al procesar lote USB: {e}"}), 500

@api_bp.route('/api/process_usb_mount', methods=['POST'])
def process_usb_mount():
    """Procesa recursivamente todas las carpetas del día o lote detectado."""
    data = request.json or {}
    mountpoint = data.get("mountpoint") or data.get("folder_path")
    target_date = data.get("target_date")
    plant_id = data.get("plant_id") or session.get("active_plant_id", 1)

    if not mountpoint or not os.path.exists(mountpoint):
        vision_service.set_progress(0, "Ruta USB no encontrada", is_processing=False)
        return jsonify({"status": "error", "message": f"Ruta de memoria USB no encontrada: {mountpoint}"}), 400

    batches = vision_service.detect_connected_usb_drives()
    target_batches = [b for b in batches if mountpoint in b["path"] or b["path"] in mountpoint or mountpoint == b.get("drive_root")]

    if not target_batches:
        target_batches = [{"path": mountpoint, "period_str": "Muestreo General", "date_str": target_date or "Día 1"}]

    total_photos_all = sum(b.get("images_count", 10) for b in target_batches)
    accumulated = 0
    processed_results = []

    for b in target_batches:
        b_path = b["path"]
        p_name = b.get("period_str", "Muestreo")
        if target_date:
            p_name = f"{target_date} - {p_name}"
```

```
        try:
            r = vision_service.process_folder_batch(
                b_path,
                period=p_name,
                plant_id=int(plant_id),
                global_offset=accumulated,
                total_global_photos=total_photos_all,
                turn_label=b.get("date_str", "USB")
            )
            accumulated += b.get("images_count", 10)
            processed_results.append(r)
        except Exception as err:
            pass

    if not processed_results:
        vision_service.set_progress(0, "No se encontraron fotos", is_processing=False)
        return jsonify({"status": "error", "message": "No se encontraron fotos válidas para procesar en la unidad."}), 400

    vision_service.set_progress(100, "¡Historial USB procesado exitosamente!", is_processing=False)
    return jsonify({
        "status": "success",
        "message": f"Se procesaron {len(processed_results)} turnos/lotes exitosamente para la Canastilla #{plant_id}.",
        "processed_batches": processed_results
    })

@api_bp.route('/api/upload_manual_batch', methods=['POST'])
def upload_manual_batch():
    files = request.files.getlist("images") or request.files.getlist("file") or request.files.getlist("files")
    crop_type = request.form.get("crop_type")
    period = request.form.get("period", "Día 1")
    plant_id = request.form.get("plant_id") or session.get("active_plant_id", 1)

    if not files or len(files) == 0 or (len(files) == 1 and files[0].filename == ''):
        return jsonify({"status": "error", "message": "No se seleccionaron archivos para subir."}), 400

    temp_dir = tempfile.mkdtemp(prefix="sifma_upload_")
    processed_results = []

    try:
        zip_files = []
        regular_files = []

        for f in files:
            safe_name = SecurityService.sanitize_filename(f.filename)
            if safe_name.lower().endswith(".zip"):
                zip_files.append((f, safe_name))
            else:
                regular_files.append((f, safe_name))

        if zip_files:
            for f_obj, z_name in zip_files:
                z_path = os.path.join(temp_dir, z_name)
                f_obj.save(z_path)

                extract_subfolder = os.path.join(temp_dir, "extracted_" + z_name.replace(".zip", ""))
```

```
                os.makedirs(extract_subfolder, exist_ok=True)

                with zipfile.ZipFile(z_path, 'r') as zip_ref:
                    zip_ref.extractall(extract_subfolder)

                # Buscar subcarpetas por turnos (manana, medio_dia, tarde)
                found_turn_dirs = []
                for r_dir, d_list, f_list in os.walk(extract_subfolder):
                    imgs = [img for img in f_list if img.lower().endswith(('.png', '.jpg', '.jpeg'))]
                    if imgs:
                        found_turn_dirs.append((r_dir, imgs))

                if found_turn_dirs:
                    for t_dir, t_imgs in found_turn_dirs:
                        t_name = os.path.basename(t_dir).lower()
                        turn_label = "Muestreo"
                        if 'manana' in t_name or 'mañana' in t_name:
                            turn_label = "Mañana"
                        elif 'medio_dia' in t_name or 'mediodia' in t_name:
                            turn_label = "Mediodía"
                        elif 'tarde' in t_name:
                            turn_label = "Tarde"
                        else:
                            turn_label = os.path.basename(t_dir).replace('_', ' ').title()

                        full_period_label = f"{period} - {turn_label}" if period else turn_label

                        r_batch = vision_service.process_folder_batch(
                            t_dir,
                            crop_type=crop_type,
                            period=full_period_label,
                            plant_id=int(plant_id)
                        )
                        processed_results.append(r_batch)
                else:
                    r_batch = vision_service.process_folder_batch(
                        extract_subfolder,
                        crop_type=crop_type,
                        period=period,
                        plant_id=int(plant_id)
                    )
                    processed_results.append(r_batch)

        if regular_files:
            regular_dir = os.path.join(temp_dir, "regular_images")
            os.makedirs(regular_dir, exist_ok=True)
            for f_obj, img_name in regular_files:
                f_obj.save(os.path.join(regular_dir, img_name))

            r_batch = vision_service.process_folder_batch(
                regular_dir,
                crop_type=crop_type,
                period=period,
                plant_id=int(plant_id)
            )
```

```
            processed_results.append(r_batch)

        if not processed_results:
            return jsonify({"status": "error", "message": "No se encontraron imágenes válidas dentro del archivo subido."}), 400

        return jsonify({
            "status": "success",
            "message": f"Se procesaron {len(processed_results)} lotes/turnos de fotos exitosamente para la Canastilla #{plant_id}.",
            "data": processed_results[0] if len(processed_results) == 1 else processed_results
        })
    except Exception as e:
        return jsonify({"status": "error", "message": f"Error procesando archivos: {e}"}), 500
    finally:
        shutil.rmtree(temp_dir, ignore_errors=True)

@api_bp.route('/api/upload', methods=['POST'])
def upload_capture_session():
    return upload_manual_batch()
```

ARCHIVO 49/85: local_server/presentation/api/conclusion_routes.py (61 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from flask import request, jsonify, session
from core.security import SecurityService
from application.services import ConclusionApplicationService
from . import api_bp

conclusion_service = ConclusionApplicationService()

@api_bp.route('/api/conclusions', methods=['GET', 'POST'])
@api_bp.route('/api/agronomic_conclusions', methods=['GET', 'POST'])
def handle_agronomic_conclusions():
    if request.method == 'GET':
        plant_id = request.args.get('plant_id', session.get('active_plant_id', 1))
        date_str = request.args.get('date_str')
        period_type = request.args.get('period_type')

        conclusions = conclusion_service.get_conclusions(int(plant_id), date_str, period_type)
        return jsonify({
            "status": "success",
            "count": len(conclusions),
            "conclusions": [c.to_dict() for c in conclusions]
        })

    elif request.method == 'POST':
        data = request.json or {}
        date_str = SecurityService.sanitize_text(data.get('date_str', ''), 30)
        general_conclusion = SecurityService.sanitize_text(data.get('general_conclusion', ''), 2000)

        if not date_str or not general_conclusion:
            return jsonify({"status": "error", "message": "La fecha y la conclusión general son requeridas."}), 400

        plant_id = data.get('plant_id') or session.get('active_plant_id', 1)
        payload = {
            "plant_id": int(plant_id),
            "date_str": date_str,
            "period_type": SecurityService.sanitize_text(data.get('period_type', 'diario'), 30),
            "growth_obs": SecurityService.sanitize_text(data.get('growth_obs', ''), 1000),
            "climate_obs": SecurityService.sanitize_text(data.get('climate_obs', ''), 1000),
            "nutrition_obs": SecurityService.sanitize_text(data.get('nutrition_obs', ''), 1000),
            "general_conclusion": general_conclusion,
            "author": SecurityService.sanitize_text(data.get('author', 'Investigador SIFMA'), 100)
        }

        new_note = conclusion_service.create_conclusion(payload)
        return jsonify({
            "status": "success",
            "message": "Conclusión agronómica guardada con éxito.",
            "note": new_note.to_dict()
        })

```

```
@api_bp.route('/api/conclusions/<int:note_id>', methods=['DELETE'])
@api_bp.route('/api/agronomic_conclusions/<int:note_id>', methods=['DELETE'])
def delete_agronomic_conclusion(note_id: int):
    success = conclusion_service.delete_conclusion(note_id)
    if not success:
        return jsonify({"status": "error", "message": "Nota no encontrada."}), 404

    return jsonify({"status": "success", "message": "Conclusión eliminada exitosamente."})
```

ARCHIVO 50/85: local_server/presentation/api/session_routes.py (93 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
from flask import request, jsonify, session, current_app
from infrastructure.database.repositories import CaptureSessionRepository, SensorRepository
from application.services import AuditService
from . import api_bp

session_repo = CaptureSessionRepository()
sensor_repo = SensorRepository()
audit_service = AuditService.get_instance()

@api_bp.route('/api/sessions/<int:session_id>', methods=['DELETE'])
def delete_capture_session(session_id: int):
    session_record = session_repo.get_by_id(session_id)
    if not session_record:
        return jsonify({"status": "error", "message": "Sesión no encontrada."}), 404

    period_name = session_record.period
    p_id = session_record.plant_id
    static_folder = current_app.static_folder
    batch_dirs_to_clean = set()

    for m in session_record.metrics:
        for path_attr in ['image_path_cenital_orig', 'image_path_cenital_proc', 'image_path_lateral_orig', 'image_path_lateral_proc']:
            rel_path = getattr(m, path_attr, None)
            if rel_path:
                full_path = os.path.join(static_folder, rel_path)
                if os.path.exists(full_path):
                    try:
                        os.remove(full_path)
                        batch_dirs_to_clean.add(os.path.dirname(full_path))
                    except Exception:
                        pass

    for b_dir in batch_dirs_to_clean:
        try:
            if os.path.exists(b_dir) and not os.listdir(b_dir):
                os.rmdir(b_dir)
        except Exception:
            pass

    session_repo.delete(session_record)

    audit_service.log(
        action="DELETE_BATCH",
        module="LOTES",
        description=f"Eliminó la sesión de captura ID {session_id} ('{period_name}') de la Canastilla #{p_id}.",
        level="danger",
        action_display="Eliminación de Sesión"
    )
```

```
    return jsonify({"status": "success", "message": "Período de fotos y métricas eliminados exitosamente."})

@api_bp.route('/api/sessions/by_date', methods=['DELETE'])
def delete_sessions_by_date():
    date_str = request.args.get('date')
    plant_id = request.args.get('plant_id', session.get('active_plant_id', 1))

    if not date_str:
        return jsonify({"status": "error", "message": "Parámetro 'date' es requerido."}), 400

    count = session_repo.delete_by_date(date_str, int(plant_id))

    audit_service.log(
        action="DELETE_BATCH",
        module="LOTES",
        description=f"Eliminó {count} períodos de capturas del día '{date_str}' de la Canastilla #{plant_id}.",
        level="danger",
        action_display="Eliminación de Lote por Fecha"
    )

    return jsonify({"status": "success", "message": f"Se eliminaron {count} períodos de fotos correspondientes a la fecha {date_str}."})

@api_bp.route('/api/telemetry/by_date', methods=['DELETE'])
def delete_telemetry_by_date():
    date_str = request.args.get('date')
    if not date_str:
        return jsonify({"status": "error", "message": "Parámetro 'date' es requerido."}), 400

    deleted_count = sensor_repo.delete_by_date(date_str)

    audit_service.log(
        action="DELETE_TELEMETRY",
        module="TELEMETRIA",
        description=f"Eliminó {deleted_count} lecturas de sensores de la fecha '{date_str}'.",
        level="danger",
        action_display="Eliminación de Telemetría"
    )

    return jsonify({"status": "success", "message": f"Se eliminaron {deleted_count} lecturas de sensores ({date_str}).", "deleted_count": deleted_count})
```

ARCHIVO 51/85: local_server/presentation/api/telemetry_routes.py (277 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from flask import request, jsonify, Response, current_app, session
from application.services import TelemetryApplicationService, AnalyticsApplicationService
from infrastructure.database.repositories import SensorRepository
from . import api_bp

telemetry_service = TelemetryApplicationService.get_instance()
analytics_service = AnalyticsApplicationService()
sensor_repo = SensorRepository()

@api_bp.route('/api/telemetry/ports', methods=['GET'])
@api_bp.route('/api/scan_ports', methods=['GET'])
def get_telemetry_ports():
    ports = telemetry_service.get_available_ports()
    return jsonify({"status": "success", "ports": ports})

@api_bp.route('/api/telemetry/start', methods=['POST'])
@api_bp.route('/api/start_telemetry_recording', methods=['POST'])
def start_telemetry():
    data = request.json or {}
    port = data.get("port", "USB_ANTENNA_AUTO")
    res = telemetry_service.start_acquisition(port=port, app=current_app._get_current_object())
    if res.get("status") == "error":
        return jsonify(res), 400
    return jsonify({"status": "success", "data": res})

@api_bp.route('/api/telemetry/stop', methods=['POST'])
@api_bp.route('/api/stop_telemetry_recording', methods=['POST'])
def stop_telemetry():
    res = telemetry_service.stop_acquisition()
    return jsonify({"status": "success", "data": res})

@api_bp.route('/api/telemetry/status', methods=['GET'])
def get_telemetry_status():
    status = telemetry_service.get_status()
    return jsonify({"status": "success", "telemetry": status})

@api_bp.route('/api/telemetry/live', methods=['GET'])
def get_live_telemetry_sample():
    sample = telemetry_service.adapter.read_sample()
    return jsonify({"status": "success", "sample": sample})

@api_bp.route('/api/telemetry/timeline', methods=['GET'])
def get_telemetry_timeline():
    target_date = request.args.get("date")
    plant_id = request.args.get("plant_id", type=int) or session.get("active_plant_id", 1)
    timeline_data = analytics_service.get_sensor_timeline_data(target_date=target_date, plant_id=plant_id)
    return jsonify(timeline_data)

@api_bp.route('/api/telemetry/upload_csv', methods=['POST'])
```

```python
@api_bp.route('/api/import_tower_csv', methods=['POST'])
def upload_telemetry_csv():
    if 'file' not in request.files and 'csv_file' not in request.files:
        return jsonify({"status": "error", "message": "No se recibió ningún archivo CSV."}), 400

    f = request.files.get('file') or request.files.get('csv_file')
    if not f or f.filename == '':
        return jsonify({"status": "error", "message": "Archivo no válido."}), 400

    try:
        res = analytics_service.import_tower_csv(f.stream)
        if res.get("status") == "error":
            return jsonify(res), 400
        return jsonify(res)
    except Exception as e:
        return jsonify({"status": "error", "message": f"Error al procesar el archivo CSV: {str(e)}"}), 500

@api_bp.route('/api/delete_telemetry_by_date', methods=['POST'])
def handle_delete_telemetry_by_date():
    data = request.json or {}
    date_str = data.get("date_str") or request.args.get('date')
    plant_id = data.get("plant_id") or request.args.get('plant_id', type=int) or session.get("active_plant_id", 1)

    if not date_str:
        return jsonify({"status": "error", "message": "Debe especificar la fecha a eliminar."}), 400

    try:
        from application.services import SystemService
        from infrastructure.database.repositories import CaptureSessionRepository
        from infrastructure.database.connection import db
        sys_svc = SystemService()
        cfg = sys_svc.get_config()
        shared_telemetry = getattr(cfg, 'shared_telemetry', True)

        if not shared_telemetry and int(plant_id) > 1:
            s_repo = CaptureSessionRepository()
            sessions = s_repo.get_by_date_and_plant(date_str, int(plant_id))
            unlinked = 0
            for s in sessions:
                if s.sensor_reading_id:
                    s.sensor_reading_id = None
                    unlinked += 1
            db.session.commit()
            return jsonify({
                "status": "success",
                "message": f"Se desvincularon las lecturas de telemetría de {unlinked} sesiones en la Canastilla #{plant_id}.",
                "deleted_count": unlinked
            })
        else:
            deleted = sensor_repo.delete_by_date(date_str)
            return jsonify({"status": "success", "message": f"Se eliminaron {deleted} registros de telemetría.", "deleted_count": deleted})
    except Exception as e:
        return jsonify({"status": "error", "message": f"Error al eliminar registros: {str(e)}"}), 500

@api_bp.route('/api/telemetry/sync_tower', methods=['POST'])
```

```python
@api_bp.route('/api/sync_tower_telemetry', methods=['POST'])
def handle_sync_tower_telemetry():
    try:
        res = analytics_service.sync_tower_telemetry_across_canastillas()
        return jsonify(res)
    except Exception as e:
        return jsonify({"status": "error", "message": f"Error al sincronizar telemetría: {str(e)}"}), 500

@api_bp.route('/api/telemetry/download_csv', methods=['GET'])
@api_bp.route('/api/telemetry/download_active_csv', methods=['GET'])
def download_telemetry_csv():
    import io
    import csv
    from datetime import datetime

    target_date = request.args.get('date') or datetime.now().strftime("%Y-%m-%d")
    filename = f"telemetria_actualizada_{target_date}.csv"

    readings = sensor_repo.get_by_date(target_date)
    if not readings:
        readings = sensor_repo.get_by_date("todos")

    output = io.StringIO()
    # Delimitador punto y coma ';' con codificación UTF-8 BOM para apertura directa en Excel en columnas nativas
    writer = csv.writer(output, delimiter=';', lineterminator='\r\n')
    writer.writerow([
        "Fecha",
        "Hora",
        "pH Solución",
        "Turbidez (NTU)",
        "Conductividad Eléctrica (µS/cm)",
        "Oxigeno Disuelto (mg/L)",
        "temperatura de agua",
        "estado motor: 1-0",
        "Temperatura exterior (°C)",
        "Humedad relativa exterior (% HR)"
    ])

    for r in readings:
        fecha_str = r.timestamp.strftime("%d/%m/%Y") if r.timestamp else ""
        hora_str = r.timestamp.strftime("%I:%M:%S %p") if r.timestamp else ""

        ph_val = f"{r.water_ph:.2f}".replace('.', ',') if r.water_ph is not None else ""
        turb_val = f"{r.turbidity_ntu:.2f}".replace('.', ',') if r.turbidity_ntu is not None else ""
        ec_val = f"{r.ec_conductivity:.1f}".replace('.', ',') if r.ec_conductivity is not None else ""
        do_val = f"{r.dissolved_oxygen_mg_l:.2f}".replace('.', ',') if r.dissolved_oxygen_mg_l is not None else ""
        water_t_val = f"{r.water_temp:.1f}".replace('.', ',') if getattr(r, 'water_temp', None) is not None else ""
        motor_st_val = int(r.motor_state) if getattr(r, 'motor_state', None) is not None else (1 if (r.motor_current and r.motor_current > 0.1) else 0)
        temp_val = f"{r.temperature:.2f}".replace('.', ',') if r.temperature is not None else ""
        hum_val = f"{r.humidity:.1f}".replace('.', ',') if r.humidity is not None else ""

        writer.writerow([
            fecha_str,
            hora_str,
            ph_val,
```

```
            turb_val,
            ec_val,
            do_val,
            water_t_val,
            motor_st_val,
            temp_val,
            hum_val
        ])

    return Response(
        output.getvalue().encode('utf-8-sig'),
        mimetype="text/csv; charset=utf-8",
        headers={"Content-Disposition": f"attachment;filename={filename}"}
    )

@api_bp.route('/api/cross_analysis/export_csv', methods=['GET'])
def export_cross_analysis_csv():
    plant_id = request.args.get("plant_id", session.get("active_plant_id", 1))
    cross_data = analytics_service.get_cross_referenced_dataset(plant_id=int(plant_id))
    csv_content = analytics_service.generate_research_csv(cross_data)

    filename = f"sifma_investigacion_canastilla_{plant_id}.csv"
    return Response(
        csv_content.encode('utf-8-sig'),
        mimetype="text/csv; charset=utf-8",
        headers={"Content-Disposition": f"attachment;filename={filename}"}
    )

@api_bp.route('/api/daily_summary/export_csv', methods=['GET'])
@api_bp.route('/api/export_daily_summary_csv', methods=['GET'])
def export_daily_summary_csv():
    from datetime import datetime
    target_date = request.args.get("date") or datetime.now().strftime("%Y-%m-%d")
    plant_id = request.args.get("plant_id", type=int) or session.get("active_plant_id", 1)
    csv_content = analytics_service.generate_daily_summary_csv(target_date=target_date, plant_id=plant_id)

    filename = f"sifma_resumen_diario_{target_date}_canastilla_{plant_id}.csv"
    return Response(
        csv_content.encode('utf-8-sig'),
        mimetype="text/csv; charset=utf-8",
        headers={"Content-Disposition": f"attachment;filename={filename}"}
    )

@api_bp.route('/api/calendar/month_data', methods=['GET'])
def get_calendar_month_data_api():
    from datetime import datetime
    now = datetime.now()
    year = request.args.get('year', type=int) or now.year
    month = request.args.get('month', type=int) or now.month
    plant_id = request.args.get('plant_id', type=int) or session.get('active_plant_id', 1)
    data = analytics_service.get_calendar_month_data(year=year, month=month, plant_id=plant_id)
    return jsonify(data)

@api_bp.route('/api/calendar/day_detail', methods=['GET'])
def get_calendar_day_detail_api():
```

```
    date_str = request.args.get('date')
    plant_id = request.args.get('plant_id', type=int) or session.get('active_plant_id', 1)
    if not date_str:
        return jsonify({"status": "error", "message": "Parámetro date requerido"}), 400
    data = analytics_service.get_day_detail(date_str=date_str, plant_id=plant_id)
    return jsonify(data)

@api_bp.route('/api/benchmark/data', methods=['GET'])
def get_benchmark_data_api():
    data = analytics_service.get_inter_plant_benchmark_data()
    return jsonify(data)

@api_bp.route('/api/weather', methods=['GET'])
def get_weather_api():
    from application.services import WeatherService, SystemService
    sys_svc = SystemService()
    cfg = sys_svc.get_config()
    lat = getattr(cfg, 'latitude', None) or 11.2408
    lon = getattr(cfg, 'longitude', None) or -74.1990
    city = getattr(cfg, 'city_name', None) or "Santa Marta, Magdalena"

    w_svc = WeatherService.get_instance()
    w_data = w_svc.get_current_weather(lat=lat, lon=lon, city_name=city)
    return jsonify({"status": "success", "weather": w_data, "location": {"lat": lat, "lon": lon, "city": city}})

@api_bp.route('/api/config/location', methods=['POST'])
def update_location_api():
    from application.services import SystemService, WeatherService
    from infrastructure.database.connection import db

    data = request.json or {}
    lat = data.get('latitude')
    lon = data.get('longitude')
    city = data.get('city_name')

    if lat is None or lon is None:
        return jsonify({"status": "error", "message": "Latitud y longitud son requeridas."}), 400

    try:
        sys_svc = SystemService()
        cfg = sys_svc.get_config()
        cfg.latitude = float(lat)
        cfg.longitude = float(lon)
        if city:
            cfg.city_name = str(city).strip()
        db.session.commit()

        w_svc = WeatherService.get_instance()
        w_svc.clear_cache()
        new_weather = w_svc.get_current_weather(lat=cfg.latitude, lon=cfg.longitude, city_name=cfg.city_name)

        return jsonify({
            "status": "success",
            "message": f"Ubicación actualizada a {cfg.city_name} ({cfg.latitude}, {cfg.longitude})",
            "weather": new_weather
```

```
        })
    except Exception as e:
        return jsonify({"status": "error", "message": f"Error al guardar ubicación: {str(e)}"}), 500

```

ARCHIVO 52/85: local_server/presentation/api/user_routes.py (284 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from flask import request, jsonify, session
from application.services import UserService, AuditService
from core.security import SecurityService
from . import api_bp

user_service = UserService()
audit_service = AuditService.get_instance()

def require_admin():
    """Verifica si la sesión actual pertenece a un Administrador."""
    return session.get('user_role') == 'admin' or session.get('is_admin') is True or session.get('is_super_admin') is True

@api_bp.route('/api/users', methods=['GET'])
def get_users_list():
    if not session.get('logged_in'):
        return jsonify({"status": "error", "message": "No autorizado."}), 401
    users = user_service.get_all_users()
    return jsonify({
        "status": "success",
        "count": len(users),
        "users": [u.to_dict() for u in users]
    })

@api_bp.route('/api/users', methods=['POST'])
def create_new_user():
    if not require_admin():
        return jsonify({"status": "error", "message": "Acceso restringido a Administradores."}), 403

    data = {}
    if request.is_json:
        data = request.json or {}
    else:
        data = request.form.to_dict()
        if 'permissions' in data and isinstance(data['permissions'], str):
            import json
            try:
                data['permissions'] = json.loads(data['permissions'])
            except Exception:
                pass

    username = SecurityService.sanitize_text(data.get('username', ''), 40)
    full_name = SecurityService.sanitize_text(data.get('full_name', ''), 100)
    email = SecurityService.sanitize_text(data.get('email', ''), 100)
    password = data.get('password', '')
    role = SecurityService.sanitize_text(data.get('role', 'investigador'), 30)
    permissions = data.get('permissions')

    if not username or not full_name or not password:
        return jsonify({"status": "error", "message": "El nombre de usuario, nombre completo y contraseña son obligatorios."}), 400
```

```
54
55      try:
56          new_user = user_service.create_user(
57              username=username,
58              full_name=full_name,
59              plain_password=password,
60              role=role,
61              email=email if email else None,
62              permissions=permissions
63          )
64
65          avatar_file = request.files.get('avatar') or request.files.get('file')
66          if avatar_file and avatar_file.filename:
67              try:
68                  user_service.save_user_avatar(new_user.id, avatar_file)
69              except Exception as e:
70                  pass
71
72          perms_count = sum(1 for v in (permissions or {}).values() if v) if permissions else "estándar"
73          audit_service.log(
74              action="CREATE_USER",
75              module="USUARIOS",
76              description=f"Creó al usuario '@{new_user.username}' ({new_user.full_name}) con rol '{new_user.role_display}' y {perms_count} funciones asignadas.",
77              level="success",
78              action_display="Creación de Usuario"
79          )
80
81          return jsonify({
82              "status": "success",
83              "message": f"Usuario '{new_user.username}' creado con éxito.",
84              "user": new_user.to_dict()
85          })
86      except ValueError as e:
87          return jsonify({"status": "error", "message": str(e)}), 400
88      except Exception as e:
89          return jsonify({"status": "error", "message": "Error interno al crear usuario."}), 500
90
91  @api_bp.route('/api/users/<int:user_id>', methods=['PUT'])
92  def update_user_details(user_id: int):
93      if not require_admin():
94          return jsonify({"status": "error", "message": "Acceso restringido a Administradores."}), 403
95
96      data = request.json or {}
97      full_name = SecurityService.sanitize_text(data.get('full_name'), 100) if data.get('full_name') else None
98      email = SecurityService.sanitize_text(data.get('email'), 100) if data.get('email') else None
99      role = SecurityService.sanitize_text(data.get('role'), 30) if data.get('role') else None
100     permissions = data.get('permissions')
101
102     try:
103         updated = user_service.update_user(
104             user_id=user_id,
105             full_name=full_name,
106             email=email,
107             role=role,
108             permissions=permissions
```

```
        )
        if not updated:
            return jsonify({"status": "error", "message": "Usuario no encontrado."}), 404

        audit_service.log(
            action="UPDATE_USER",
            module="USUARIOS",
            description=f"Modificó la cuenta del usuario '@{updated.username}' ({updated.full_name}) - Rol: {updated.role_display}.",
            level="info",
            action_display="Edición de Usuario"
        )

        return jsonify({
            "status": "success",
            "message": "Usuario actualizado correctamente.",
            "user": updated.to_dict()
        })
    except ValueError as e:
        return jsonify({"status": "error", "message": str(e)}), 400

@api_bp.route('/api/users/<int:user_id>/toggle_active', methods=['POST'])
def toggle_user_status(user_id: int):
    if not require_admin():
        return jsonify({"status": "error", "message": "Acceso restringido a Administradores."}), 403

    user = user_service.get_user_by_id(user_id)
    if not user:
        return jsonify({"status": "error", "message": "Usuario no encontrado."}), 404

    try:
        new_status = not user.is_active
        updated = user_service.update_user(user_id=user_id, is_active=new_status)
        action_str = "reactivado" if new_status else "pausado / desactivado"

        audit_service.log(
            action="TOGGLE_USER_STATUS",
            module="USUARIOS",
            description=f"{'Reactivó' if new_status else 'Pausó'} la cuenta del usuario '@{user.username}' ({user.full_name}).",
            level="warning" if not new_status else "info",
            action_display="Cambio de Estado de Usuario"
        )

        return jsonify({
            "status": "success",
            "message": f"Usuario '{updated.username}' {action_str}.",
            "is_active": updated.is_active
        })
    except ValueError as e:
        return jsonify({"status": "error", "message": str(e)}), 400

@api_bp.route('/api/users/<int:user_id>/reset_password', methods=['POST'])
def reset_user_password(user_id: int):
    if not require_admin():
        return jsonify({"status": "error", "message": "Acceso restringido a Administradores."}), 403

```

```
    user = user_service.get_user_by_id(user_id)
    if not user:
        return jsonify({"status": "error", "message": "Usuario no encontrado."}), 404

    data = request.json or {}
    new_pass = data.get('password', '')
    if not new_pass or len(new_pass) < 4:
        return jsonify({"status": "error", "message": "La contraseña debe tener al menos 4 caracteres."}), 400

    success = user_service.update_password(user_id, new_pass)
    if not success:
        return jsonify({"status": "error", "message": "Error al actualizar contraseña."}), 500

    audit_service.log(
        action="CHANGE_PASSWORD",
        module="USUARIOS",
        description=f"Restableció la contraseña del usuario '@{user.username}' ({user.full_name}).",
        level="warning",
        action_display="Cambio de Contraseña"
    )

    return jsonify({"status": "success", "message": "Contraseña restablecida exitosamente."})

@api_bp.route('/api/users/<int:user_id>', methods=['DELETE'])
def delete_user(user_id: int):
    if not require_admin():
        return jsonify({"status": "error", "message": "Acceso restringido a Administradores."}), 403

    target_user = user_service.get_user_by_id(user_id)
    if not target_user:
        return jsonify({"status": "error", "message": "Usuario no encontrado."}), 404

    target_name = target_user.username
    target_full = target_user.full_name

    try:
        success = user_service.delete_user(user_id)
        if not success:
            return jsonify({"status": "error", "message": "Error al eliminar usuario."}), 500

        audit_service.log(
            action="DELETE_USER",
            module="USUARIOS",
            description=f"Eliminó permanentemente la cuenta de usuario '@{target_name}' ({target_full}) con ID {user_id}.",
            level="danger",
            action_display="Eliminación de Usuario"
        )

        return jsonify({"status": "success", "message": "Usuario eliminado correctamente."})
    except ValueError as e:
        return jsonify({"status": "error", "message": str(e)}), 400

@api_bp.route('/api/users/<int:user_id>/avatar', methods=['POST'])
def upload_user_avatar(user_id: int):
    is_admin = require_admin()
```

```
    current_uid = session.get('user_id')
    if not is_admin and current_uid != user_id:
        return jsonify({"status": "error", "message": "No tienes permiso para modificar este avatar."}), 403

    avatar_file = request.files.get('avatar') or request.files.get('file') or request.files.get('image')
    if not avatar_file or not avatar_file.filename:
        return jsonify({"status": "error", "message": "No se recibió ningún archivo de imagen."}), 400

    try:
        avatar_url = user_service.save_user_avatar(user_id, avatar_file)
        target_user = user_service.get_user_by_id(user_id)

        if current_uid == user_id:
            session['user_avatar_url'] = avatar_url

        audit_service.log(
            action="UPDATE_AVATAR",
            module="USUARIOS",
            description=f"Actualizó la foto de perfil / avatar del usuario '@{target_user.username}'.",
            level="info",
            action_display="Cambio de Avatar"
        )

        return jsonify({
            "status": "success",
            "message": "Foto de perfil actualizada correctamente.",
            "avatar_url": avatar_url,
            "user": target_user.to_dict() if target_user else None
        })
    except ValueError as e:
        return jsonify({"status": "error", "message": str(e)}), 400
    except Exception as e:
        return jsonify({"status": "error", "message": f"Error interno al guardar avatar: {e}"}), 500

@api_bp.route('/api/users/<int:user_id>/avatar', methods=['DELETE'])
def remove_user_avatar(user_id: int):
    is_admin = require_admin()
    current_uid = session.get('user_id')
    if not is_admin and current_uid != user_id:
        return jsonify({"status": "error", "message": "No tienes permiso para modificar este avatar."}), 403

    target_user = user_service.get_user_by_id(user_id)
    if not target_user:
        return jsonify({"status": "error", "message": "Usuario no encontrado."}), 404

    success = user_service.remove_user_avatar(user_id)
    if not success:
        return jsonify({"status": "error", "message": "El usuario no tenía una foto de avatar personalizada."}), 400

    if current_uid == user_id:
        session.pop('user_avatar_url', None)

    audit_service.log(
        action="REMOVE_AVATAR",
        module="USUARIOS",
```

```
        description=f"Eliminó la foto de perfil del usuario '@{target_user.username}' (revertido a iniciales).",
        level="info",
        action_display="Eliminación de Avatar"
    )

    return jsonify({
        "status": "success",
        "message": "Foto de perfil eliminada. Se mostrarán las iniciales.",
        "avatar_initials": target_user.avatar_initials,
        "user": target_user.to_dict()
    })
```

ARCHIVO 53/85: local_server/presentation/web/__init__.py (7 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .dashboard_routes import dashboard_bp

__all__ = ["dashboard_bp"]
```

ARCHIVO 54/85: local_server/presentation/web/dashboard_routes.py (803 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from datetime import datetime
from functools import wraps
from flask import Blueprint, render_template, request, redirect, url_for, session, abort
from core.config import config as app_settings
from core.security import SecurityService
from application.services import (
    SystemService,
    TelemetryApplicationService,
    AnalyticsApplicationService,
    ConclusionApplicationService,
    UserService,
    WeatherService,
    AuditService
)
from infrastructure.database.repositories import (
    SensorRepository,
    CaptureSessionRepository
)

dashboard_bp = Blueprint('dashboard', __name__)

system_service = SystemService()
telemetry_service = TelemetryApplicationService.get_instance()
analytics_service = AnalyticsApplicationService()
conclusion_service = ConclusionApplicationService()
user_service = UserService()
weather_service = WeatherService.get_instance()
audit_service = AuditService.get_instance()
sensor_repo = SensorRepository()
session_repo = CaptureSessionRepository()

def get_active_plant_id():
    """Devuelve la Canastilla activa guardada en la sesión o recibida por query parameter (1 a 4)."""
    arg_id = request.args.get('plant_id')
    if arg_id:
        try:
            val = int(arg_id)
            if 1 <= val <= app_settings.MAX_PLANT_NODES:
                session['active_plant_id'] = val
                return val
        except (ValueError, TypeError):
            pass
    return session.get('active_plant_id', 1)

from domain.models.user import ALL_MODULE_PERMISSIONS

def get_current_user_profile():
    """Retorna los datos del usuario en sesión para renderizar el pie de la barra lateral y validar permisos."""
    u_id = session.get("user_id")
```

```
    perms = session.get("user_permissions")
    user_role = session.get("user_role", "investigador")
    is_admin = user_role == "admin" or session.get("username", "").lower() == "admin"
    avatar_url = session.get("user_avatar_url")
    avatar_initials = session.get("user_avatar", "AL")

    if u_id:
        u_obj = user_service.get_user_by_id(u_id)
        if u_obj:
            perms = u_obj.get_permissions_dict()
            session['user_permissions'] = perms
            session['user_role'] = u_obj.role
            session['user_role_display'] = u_obj.role_display
            session['is_super_admin'] = u_obj.is_super_admin
            session['user_avatar'] = u_obj.avatar_initials
            session['user_avatar_url'] = u_obj.avatar_url
            is_admin = u_obj.is_super_admin
            avatar_url = u_obj.avatar_url
            avatar_initials = u_obj.avatar_initials

    if not perms:
        if is_admin:
            perms = {k: True for k, _, _, _ in ALL_MODULE_PERMISSIONS}
        else:
            perms = {k: False for k, _, _, _ in ALL_MODULE_PERMISSIONS}

    return {
        "id": session.get("user_id", 1),
        "username": session.get("username", "admin"),
        "name": session.get("user_name", "Andres Luna"),
        "role": session.get("user_role", "admin"),
        "role_display": session.get("user_role_display", "Super Usuario (Admin)"),
        "is_super_admin": is_admin,
        "avatar": avatar_initials,
        "avatar_url": avatar_url,
        "permissions": perms
    }

def role_required(*allowed_roles):
    """Decorador para proteger rutas según el rol del usuario."""
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if not session.get('logged_in'):
                return redirect(url_for('dashboard.login'))
            user_role = session.get('user_role', 'investigador')
            if user_role not in allowed_roles and user_role != 'admin' and session.get('username', '').lower() != 'admin':
                return render_template('error.html',
                                       error_title="Acceso Restringido",
                                       error_message="Esta sección es exclusiva del Super Usuario / Administrador Principal."), 403
            return f(*args, **kwargs)
        return decorated_function
    return decorator

def permission_required(permission_key: str, module_name: str = "este módulo"):
```

```
    """Decorador para proteger rutas según los permisos granulares asignados por el Super Usuario."""
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if not session.get('logged_in'):
                return redirect(url_for('dashboard.login'))

            if session.get('user_role') == 'admin' or session.get('username', '').lower() == 'admin':
                return f(*args, **kwargs)

            perms = session.get('user_permissions')
            if not perms or not isinstance(perms, dict):
                u_id = session.get("user_id")
                if u_id:
                    u_obj = user_service.get_user_by_id(u_id)
                    if u_obj:
                        perms = u_obj.get_permissions_dict()
                        session['user_permissions'] = perms

            perms = perms or {}
            if not perms.get(permission_key, False):
                # Si el usuario intenta entrar al index('/') pero no tiene dashboard, redirigir a su primer módulo habilitado
                if request.endpoint == 'dashboard.index':
                    for alt_key, alt_endpoint in [
                        ('calendar', 'dashboard.calendar_view'),
                        ('sensors', 'dashboard.detailed_sensors'),
                        ('cross_analysis', 'dashboard.cross_analysis'),
                        ('gallery', 'dashboard.gallery'),
                        ('timelapse', 'dashboard.timelapse_viewer'),
                        ('processing', 'dashboard.processing_panel'),
                        ('config', 'dashboard.configuration_panel')
                    ]:
                        if perms.get(alt_key):
                            return redirect(url_for(alt_endpoint))

                return render_template('error.html',
                                       error_title="Acceso Restringido",
                                       error_message=f"Tu cuenta no posee permisos para acceder a '{module_name}'. Solicita al Administrador Principal que te habilite esta función."), 403
            return f(*args, **kwargs)
        return decorated_function
    return decorator

@dashboard_bp.context_processor
def inject_user_context():
    """Inyecta el perfil de usuario, canastilla activa y perfiles asignados en todas las plantillas."""
    active_pid = get_active_plant_id()
    active_prof = system_service.get_profile_for_plant(active_pid)
    plant_profiles_map = system_service.get_all_plant_profiles_map()
    return {
        "current_user": get_current_user_profile(),
        "active_plant_id": active_pid,
        "active_crop_profile": active_prof,
        "plant_profiles_map": plant_profiles_map,
        "all_permissions_list": ALL_MODULE_PERMISSIONS
```

```
    }

@dashboard_bp.before_request
def require_login():
    allowed_endpoints = [
        'dashboard.login',
        'static',
        'api.upload_capture_session',
        'api.get_config',
        'api.scan_usb_drives'
    ]
    if request.endpoint and not any(request.endpoint.startswith(ep) for ep in allowed_endpoints):
        if not session.get('logged_in'):
            return redirect(url_for('dashboard.login'))

@dashboard_bp.route('/set_active_plant/<int:plant_id>')
def set_active_plant(plant_id: int):
    """Cambia la Canastilla/Torre activa en la sesión (1 a 4)."""
    if 1 <= plant_id <= app_settings.MAX_PLANT_NODES:
        session['active_plant_id'] = plant_id
    referrer = request.referrer or url_for('dashboard.index')
    return redirect(referrer)

@dashboard_bp.route('/login', methods=['GET', 'POST'])
def login():
    error = None
    if session.get('logged_in'):
        return redirect(url_for('dashboard.index'))

    if request.method == 'POST':
        username = (request.form.get('username') or '').strip().lower()
        password = request.form.get('password') or ''

        user = user_service.authenticate(username, password)
        if user:
            session['logged_in'] = True
            session['user_id'] = user.id
            session['username'] = user.username
            session['user_name'] = user.full_name
            session['user_role'] = user.role
            session['user_role_display'] = user.role_display
            session['user_avatar'] = user.avatar_initials
            session['user_permissions'] = user.get_permissions_dict()
            session['is_super_admin'] = user.is_super_admin
            session.permanent = True

            audit_service.log(
                action="LOGIN",
                module="AUTENTICACION",
                description=f"Inicio de sesión exitoso en la plataforma SIFMA.",
                level="info",
                user_id=user.id,
                username=user.username,
                user_fullname=user.full_name,
                user_role=user.role_display,
```

```
                action_display="Inicio de Sesión"
            )

            return redirect(url_for('dashboard.index'))
        else:
            audit_service.log(
                action="LOGIN_FAILED",
                module="AUTENTICACION",
                description=f"Intento fallido de inicio de sesión con el identificador '{username}'.",
                level="warning",
                username=username or "desconocido",
                action_display="Fallo de Autenticación"
            )
            error = "Usuario o contraseña incorrectos, o cuenta inactiva."

    return render_template('login.html', error=error)

@dashboard_bp.route('/logout')
def logout():
    u_name = session.get('username', 'usuario')
    audit_service.log(
        action="LOGOUT",
        module="AUTENTICACION",
        description=f"El usuario '@{u_name}' cerró su sesión.",
        level="info",
        action_display="Cierre de Sesión"
    )
    session.clear()
    return redirect(url_for('dashboard.login'))

@dashboard_bp.route('/')
@permission_required('dashboard', 'Resumen Diario')
def index():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()

    target_date = request.args.get('date', 'auto')

    # Obtener el análisis diario consolidado para la fecha seleccionada
    daily_data = analytics_service.get_daily_summary_data(target_date=target_date, plant_id=active_plant_id)

    # Obtener clima exterior actual para la ubicación de la torre (Open-Meteo API / Caché / Fallback)
    current_weather = weather_service.get_current_weather(
        lat=getattr(config, 'latitude', None) or 11.2408,
        lon=getattr(config, 'longitude', None) or -74.1990,
        city_name=getattr(config, 'city_name', None) or "Santa Marta, Magdalena"
    )

    profiles = system_service.get_all_profiles()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)

    return render_template('dashboard.html',
                           config=config,
                           profiles=profiles,
                           selected_profile=selected_profile,
```

```
                           daily=daily_data,
                           weather=current_weather,
                           target_date=daily_data["target_date"],
                           available_dates=daily_data["available_dates"],
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/sensors')
@permission_required('sensors', 'Sensores Ambientales')
def detailed_sensors():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()

    target_date = request.args.get('date', 'auto')
    sensor_data = analytics_service.get_sensor_timeline_data(target_date=target_date, plant_id=active_plant_id)
    telemetry_status = telemetry_service.get_status()
    available_ports = telemetry_service.get_available_ports()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)

    return render_template('sensors.html',
                           config=config,
                           sensor_data=sensor_data,
                           available_dates=sensor_data.get('available_dates', []),
                           telemetry_status=telemetry_status,
                           available_ports=available_ports,
                           selected_profile=selected_profile,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/calendar')
@permission_required('calendar', 'Calendario de Muestreos')
def calendar_view():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)

    now = datetime.now()
    year = request.args.get('year', type=int) or now.year
    month = request.args.get('month', type=int) or now.month

    cal_data = analytics_service.get_calendar_month_data(year=year, month=month, plant_id=active_plant_id)

    return render_template('calendar.html',
                           config=config,
                           cal_data=cal_data,
                           selected_profile=selected_profile,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/benchmark')
@permission_required('cross_analysis', 'Benchmark Comparativo')
def inter_plant_benchmark():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)
    benchmark_data = analytics_service.get_inter_plant_benchmark_data()

    return render_template('benchmark.html',
```

```
                           config=config,
                           benchmark_data=benchmark_data,
                           selected_profile=selected_profile,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/timelapse')
@permission_required('timelapse', 'Time-Lapse Evolutivo')
def timelapse_viewer():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)
    timelapse_data = analytics_service.get_timelapse_data(plant_id=active_plant_id)

    return render_template('timelapse.html',
                           config=config,
                           timelapse_data=timelapse_data,
                           selected_profile=selected_profile,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/reports')
@dashboard_bp.route('/dossier')
@dashboard_bp.route('/scientific_report')
@dashboard_bp.route('/report/scientific')
@permission_required('reports', 'Dossier y Reportes')
def scientific_report_view():
    config = system_service.get_config()
    req_plant_id = request.args.get('plant_id', type=int)
    active_plant_id = req_plant_id if (req_plant_id and 1 <= req_plant_id <= app_settings.MAX_PLANT_NODES) else get_active_plant_id()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)
    plant_profiles_map = system_service.get_all_plant_profiles_map()
    raw_date = request.args.get('date', '').strip()
    target_date = raw_date if raw_date else 'auto'

    dossier = analytics_service.get_scientific_dossier_data(target_date=target_date, plant_id=active_plant_id)

    return render_template('scientific_report.html',
                           config=config,
                           dossier=dossier,
                           selected_profile=selected_profile,
                           plant_profiles_map=plant_profiles_map,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/processing')
@permission_required('processing', 'Procesamiento de Imágenes')
def processing_panel():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    profiles = system_service.get_all_profiles()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)

    sessions = session_repo.get_all_by_plant(active_plant_id, order_asc=True)
    latest_session = sessions[-1] if sessions else None
    latest_metric = latest_session.get_average_metric() if latest_session else None

    processed_periods = [s.period for s in sessions if s.period]
```

```
    growth_data = {
        "labels": [s.period for s in sessions],
        "foliar_areas": [(s.get_average_metric().foliar_area_cm2 if s.get_average_metric() else 0.0) for s in sessions],
        "plant_heights": [(s.get_average_metric().plant_height_cm if s.get_average_metric() else 0.0) for s in sessions],
        "stem_diameters": [(s.get_average_metric().stem_diameter_mm if s.get_average_metric() else 0.0) for s in sessions],
        "health_indices": [(s.get_average_metric().health_index if s.get_average_metric() else 0.0) for s in sessions]
    }

    return render_template('processing.html',
                           config=config,
                           profiles=profiles,
                           selected_profile=selected_profile,
                           sessions=sessions,
                           latest_session=latest_session,
                           latest_metric=latest_metric,
                           processed_periods=processed_periods,
                           growth_data=growth_data,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/gallery')
@permission_required('gallery', 'Galería de Lotes')
def gallery():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    profiles = system_service.get_all_profiles()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)

    sessions = session_repo.get_all_by_plant(active_plant_id)
    gallery_sessions = []

    for s in sessions:
        if s.metrics:
            avg_metric = s.get_average_metric()
            individual_photos = [m.to_dict() for m in s.get_individual_photos()]

            gallery_sessions.append({
                "session": s,
                "avg_metric": avg_metric.to_dict() if avg_metric else {},
                "individual_photos": individual_photos,
                "all_metrics_json": [m.to_dict() for m in s.metrics]
            })

    return render_template('gallery.html',
                           config=config,
                           profiles=profiles,
                           selected_profile=selected_profile,
                           sessions=sessions,
                           gallery_sessions=gallery_sessions,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/cross_analysis')
@permission_required('cross_analysis', 'Cruce y Correlaciones')
def cross_analysis():
    config = system_service.get_config()
```

```
    active_plant_id = get_active_plant_id()
    profiles = system_service.get_all_profiles()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)

    telemetry_status = telemetry_service.get_status()
    available_ports = telemetry_service.get_available_ports()

    cross_data = analytics_service.get_cross_referenced_dataset(plant_id=active_plant_id)
    correlations = analytics_service.calculate_agronomic_correlations(cross_data)
    correlation_matrix = analytics_service.calculate_full_correlation_matrix(cross_data)

    available_dates = session_repo.get_available_dates(active_plant_id)
    if not available_dates:
        available_dates = sensor_repo.get_available_dates()

    conclusions = conclusion_service.get_conclusions(plant_id=active_plant_id)

    return render_template('cross_analysis.html',
                           config=config,
                           profiles=profiles,
                           selected_profile=selected_profile,
                           active_plant_id=active_plant_id,
                           cross_data=cross_data,
                           correlations=correlations,
                           correlation_matrix=correlation_matrix,
                           available_dates=available_dates,
                           conclusions=[c.to_dict() for c in conclusions],
                           telemetry_status=telemetry_status,
                           available_ports=available_ports)

@dashboard_bp.route('/config', methods=['GET', 'POST'])
@permission_required('config', 'Parámetros de Cultivo')
def configuration_panel():
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    profiles = system_service.get_all_profiles()

    if request.method == 'POST':
        action = request.form.get('action')

        # 1. Asignar perfil de cultivo a una o todas las Canastillas
        if action == 'assign_plant_crops':
            p1 = request.form.get('plant_1_crop', 'cebollin')
            p2 = request.form.get('plant_2_crop', 'albahaca')
            p3 = request.form.get('plant_3_crop', 'lechuga')
            p4 = request.form.get('plant_4_crop', 'fresa')
            system_service.update_config(
                plant_1_crop=p1,
                plant_2_crop=p2,
                plant_3_crop=p3,
                plant_4_crop=p4,
                selected_crop_type=p1
            )
            return redirect(url_for('dashboard.configuration_panel'))

```

```
        elif action == 'update_telemetry_mode':
            telemetry_mode = request.form.get('telemetry_mode', 'shared_tower')
            shared_telemetry = (telemetry_mode == 'shared_tower')
            system_service.update_config(
                telemetry_mode=telemetry_mode,
                shared_telemetry=shared_telemetry
            )
            return redirect(url_for('dashboard.configuration_panel'))

        # 2. Calibrar umbrales HSV y LAB de un perfil botánico
        elif action == 'update_profile':
            crop_type = request.form.get('crop_type')
            profile = system_service.get_profile(crop_type)
            if profile:
                profile.h_min = int(request.form.get('h_min', profile.h_min))
                profile.h_max = int(request.form.get('h_max', profile.h_max))
                profile.s_min = int(request.form.get('s_min', profile.s_min))
                profile.s_max = int(request.form.get('s_max', profile.s_max))
                profile.v_min = int(request.form.get('v_min', profile.v_min))
                profile.v_max = int(request.form.get('v_max', profile.v_max))

                profile.l_min = int(request.form.get('l_min', profile.l_min))
                profile.l_max = int(request.form.get('l_max', profile.l_max))
                profile.a_min = int(request.form.get('a_min', profile.a_min))
                profile.a_max = int(request.form.get('a_max', profile.a_max))
                profile.b_min = int(request.form.get('b_min', profile.b_min))
                profile.b_max = int(request.form.get('b_max', profile.b_max))

                profile.pixel_to_cm_ratio = float(request.form.get('pixel_to_cm_ratio', profile.pixel_to_cm_ratio))
                profile.has_stem = 'has_stem' in request.form

                # Parámetros dedicados para Cámara Lateral (Perfil Vertical)
                profile.lat_h_min = int(request.form.get('lat_h_min', getattr(profile, 'lat_h_min', profile.h_min)))
                profile.lat_h_max = int(request.form.get('lat_h_max', getattr(profile, 'lat_h_max', profile.h_max)))
                profile.lat_s_min = int(request.form.get('lat_s_min', getattr(profile, 'lat_s_min', profile.s_min)))
                profile.lat_s_max = int(request.form.get('lat_s_max', getattr(profile, 'lat_s_max', profile.s_max)))
                profile.lat_v_min = int(request.form.get('lat_v_min', getattr(profile, 'lat_v_min', profile.v_min)))
                profile.lat_v_max = int(request.form.get('lat_v_max', getattr(profile, 'lat_v_max', profile.v_max)))

                profile.lat_l_min = int(request.form.get('lat_l_min', getattr(profile, 'lat_l_min', profile.l_min)))
                profile.lat_l_max = int(request.form.get('lat_l_max', getattr(profile, 'lat_l_max', profile.l_max)))
                profile.lat_a_min = int(request.form.get('lat_a_min', getattr(profile, 'lat_a_min', profile.a_min)))
                profile.lat_a_max = int(request.form.get('lat_a_max', getattr(profile, 'lat_a_max', profile.a_max)))
                profile.lat_b_min = int(request.form.get('lat_b_min', getattr(profile, 'lat_b_min', profile.b_min)))
                profile.lat_b_max = int(request.form.get('lat_b_max', getattr(profile, 'lat_b_max', profile.b_max)))

                profile.lat_pixel_to_cm_ratio = float(request.form.get('lat_pixel_to_cm_ratio', getattr(profile, 'lat_pixel_to_cm_ratio', profile.pixel_to_cm_ratio)))
                profile.lat_min_area = int(request.form.get('lat_min_area', getattr(profile, 'lat_min_area', 60)))
                profile.lat_filter_isolated = 'lat_filter_isolated' in request.form
                profile.lat_draw_contours = 'lat_draw_contours' in request.form

                system_service.update_profile(profile)

            return redirect(url_for('dashboard.configuration_panel', crop_type=crop_type))
```

```
        # 3. Calibrar ROI Cenital Interactivo
        elif action == 'update_cenital_roi':
            try:
                cx = float(request.form.get('cenital_roi_cx', 0.4955))
                cy = float(request.form.get('cenital_roi_cy', 0.4408))
                radius = float(request.form.get('cenital_roi_radius', 0.1450))
                mode = request.form.get('cenital_roi_mode', 'dynamic_conn')
                system_service.update_config(
                    cenital_roi_cx=cx,
                    cenital_roi_cy=cy,
                    cenital_roi_radius=radius,
                    cenital_roi_mode=mode
                )
            except Exception:
                pass
            return redirect(url_for('dashboard.configuration_panel'))

        # 4. Calibrar Línea Base Lateral y Zona de Tallo Interactiva
        elif action in ['update_lateral_baseline', 'update_lateral_stem']:
            try:
                update_dict = {}
                if 'lateral_baseline_y' in request.form:
                    update_dict['lateral_baseline_y'] = float(request.form.get('lateral_baseline_y', 0.7200))
                if 'lateral_baseline_mode' in request.form:
                    update_dict['lateral_baseline_mode'] = request.form.get('lateral_baseline_mode', 'manual_baseline')

                if 'lateral_stem_roi_y' in request.form:
                    update_dict['lateral_stem_roi_y'] = float(request.form.get('lateral_stem_roi_y', 0.6600))
                if 'lateral_stem_roi_height' in request.form:
                    update_dict['lateral_stem_roi_height'] = float(request.form.get('lateral_stem_roi_height', 0.0500))
                if 'lateral_stem_roi_x' in request.form:
                    update_dict['lateral_stem_roi_x'] = float(request.form.get('lateral_stem_roi_x', 0.5000))
                if 'lateral_stem_roi_width' in request.form:
                    update_dict['lateral_stem_roi_width'] = float(request.form.get('lateral_stem_roi_width', 0.2000))
                if 'lateral_stem_mode' in request.form:
                    update_dict['lateral_stem_mode'] = request.form.get('lateral_stem_mode', 'auto_edge')

                if update_dict:
                    system_service.update_config(**update_dict)
            except Exception:
                pass
            return redirect(url_for('dashboard.configuration_panel'))

    requested_crop = request.args.get('crop_type', system_service.get_crop_for_plant(active_plant_id))
    selected_profile = system_service.get_profile(requested_crop)
    plant_profiles_map = system_service.get_all_plant_profiles_map()

    # Obtener la última foto cenital y lateral procesada o capturada para los canvas interactivos
    sessions = session_repo.get_all_by_plant(active_plant_id, order_asc=False)
    latest_cenital_img = None
    latest_lateral_img = None
    if sessions:
        for s in sessions:
            if s.metrics:
                for m in s.metrics:
```

```
                    if not latest_cenital_img and m.image_path_cenital_orig:
                        latest_cenital_img = m.image_path_cenital_orig
                    if not latest_lateral_img and m.image_path_lateral_orig:
                        latest_lateral_img = m.image_path_lateral_orig
                    if latest_cenital_img and latest_lateral_img:
                        break
            if latest_cenital_img and latest_lateral_img:
                break

    return render_template('config.html',
                           config=config,
                           profiles=profiles,
                           selected_profile=selected_profile,
                           plant_profiles_map=plant_profiles_map,
                           latest_cenital_img=latest_cenital_img,
                           latest_lateral_img=latest_lateral_img,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/users')
@role_required('admin')
def users_management():
    """Vista de administración de usuarios y roles (exclusiva para Administrador Principal)."""
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    users = user_service.get_all_users()

    return render_template('users.html',
                           config=config,
                           users=users,
                           active_plant_id=active_plant_id)

@dashboard_bp.route('/about')
@dashboard_bp.route('/system_info')
def system_info():
    """Vista institucional de información del sistema, créditos, autores y términos de licencia."""
    config = system_service.get_config()
    active_plant_id = get_active_plant_id()
    profiles = system_service.get_all_profiles()
    selected_profile = system_service.get_profile_for_plant(active_plant_id)

    system_metadata = {
        "version": "1.1",
        "release_name": "SIFMA PhenoCore",
        "release_date": "Septiembre 2026",
        "author": "Ing. Andrés Luna & Cristian",
        "lead_developer": "Ing. Andrés Luna",
        "co_developer": "Cristian",
        "scientific_advisor": "Prof. Críspulo Enrique Deluque",
        "founding_professors": "Prof. Críspulo Enrique Deluque y Comité Docente",
        "institution": "Laboratorio de Automatización, Visión Artificial & Fenotipado Digital",
        "license": "Licencia Cerrada Exclusiva (Torre SIFMA)",
        "python_version": "3.10+",
        "opencv_version": "4.8+",
        "architecture": "Clean Architecture / Hexagonal"
    }
```

```
658
659     return render_template('about.html',
660                            config=config,
661                            profiles=profiles,
662                            selected_profile=selected_profile,
663                            active_plant_id=active_plant_id,
664                            system_meta=system_metadata)
665
666 @dashboard_bp.route('/audit-logs')
667 @dashboard_bp.route('/logs')
668 @role_required('admin')
669 def audit_logs():
670     """Vista de registro de auditoría, trazabilidad y logs de movimientos (exclusiva para Super Administrador)."""
671     search_query = request.args.get('search', '').strip()
672     module_filter = request.args.get('module', '').strip()
673     level_filter = request.args.get('level', '').strip()
674     action_filter = request.args.get('action', '').strip()
675     date_filter = request.args.get('date', '').strip()
676
677     start_date = None
678     end_date = None
679     if date_filter:
680         try:
681             start_date = datetime.strptime(date_filter, '%Y-%m-%d')
682             end_date = start_date.replace(hour=23, minute=59, second=59)
683         except ValueError:
684             pass
685
686     logs = audit_service.get_logs(
687         limit=300,
688         module=module_filter if module_filter else None,
689         action=action_filter if action_filter else None,
690         level=level_filter if level_filter else None,
691         search=search_query if search_query else None,
692         start_date=start_date,
693         end_date=end_date
694     )
695
696     stats = audit_service.get_stats()
697     config = system_service.get_config()
698     active_plant_id = get_active_plant_id()
699
700     # Módulos y niveles disponibles para filtros
701     available_modules = ["AUTENTICACION", "USUARIOS", "CONFIGURACION", "LOTES", "FENOTIPADO", "TELEMETRIA", "REPORTES", "SISTEMA"]
702     available_levels = [
703         ("info", "Informativo"),
704         ("success", "Éxito / Creación"),
705         ("warning", "Advertencia / Pausa"),
706         ("danger", "Crítico / Eliminación")
707     ]
708
709     return render_template(
710         'audit_logs.html',
711         logs=logs,
712         stats=stats,
```

```
        search_query=search_query,
        module_filter=module_filter,
        level_filter=level_filter,
        action_filter=action_filter,
        date_filter=date_filter,
        available_modules=available_modules,
        available_levels=available_levels,
        config=config,
        active_plant_id=active_plant_id
    )

@dashboard_bp.route('/audit-logs/export')
@role_required('admin')
def export_audit_logs():
    """Exporta los logs de auditoría filtrados en formato CSV descargable."""
    from flask import Response

    search_query = request.args.get('search', '').strip()
    module_filter = request.args.get('module', '').strip()
    level_filter = request.args.get('level', '').strip()
    date_filter = request.args.get('date', '').strip()

    start_date = None
    end_date = None
    if date_filter:
        try:
            start_date = datetime.strptime(date_filter, '%Y-%m-%d')
            end_date = start_date.replace(hour=23, minute=59, second=59)
        except ValueError:
            pass

    logs = audit_service.get_logs(
        limit=2000,
        module=module_filter if module_filter else None,
        level=level_filter if level_filter else None,
        search=search_query if search_query else None,
        start_date=start_date,
        end_date=end_date
    )

    csv_data = audit_service.generate_csv(logs)
    filename = f"sifma_auditoria_logs_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"

    audit_service.log(
        action="EXPORT_DATA",
        module="REPORTES",
        description=f"Exportó {len(logs)} registros de auditoría a CSV.",
        level="info",
        action_display="Exportación de Logs"
    )

    return Response(
        csv_data,
        mimetype="text/csv; charset=utf-8",
        headers={
```

```
            "Content-Disposition": f"attachment; filename={filename}",
            "Content-Type": "text/csv; charset=utf-8"
        }
    )

@dashboard_bp.route('/api/audit-logs/<int:log_id>', methods=['DELETE'])
@role_required('admin')
def delete_single_audit_log(log_id: int):
    """Elimina un log de auditoría individual."""
    from flask import jsonify
    success = audit_service.delete_log(log_id)
    if not success:
        return jsonify({"status": "error", "message": "Registro de log no encontrado."}), 404
    return jsonify({"status": "success", "message": f"Registro de auditoría #{log_id} eliminado exitosamente."})

@dashboard_bp.route('/api/audit-logs/clear', methods=['POST', 'DELETE'])
@role_required('admin')
def clear_all_audit_logs():
    """Vacía completamente la tabla de logs de auditoría."""
    from flask import jsonify
    count = audit_service.clear_all_logs()

    audit_service.log(
        action="CLEAR_LOGS",
        module="SISTEMA",
        description=f"El Administrador Principal vació el historial de auditoría ({count} registros eliminados).",
        level="warning",
        action_display="Vaciado de Logs"
    )

    return jsonify({
        "status": "success",
        "message": f"Se han eliminado {count} registros del historial de logs.",
        "deleted_count": count
    })

```

ARCHIVO 55/85: local_server/static/css/index.css (1795 líneas)

```
/**
 * SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
 * Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
 * Año: 2026.
 */

/* ========================================================================
   SIFMA - SISTEMA INTEGRAL DE FENOTIPADO Y MONITOREO AGRONÓMICO
   SISTEMA DE DISEÑO GLOBAL UNIFICADO (PALETA BIOPAISAJE & VERDE BOSQUE)
   ======================================================================== */

/* 1. VARIABLES DEL SISTEMA DE DISEÑO GLOBAL (MODO CLARO INSTITUCIONAL & VERDE BOSQUE) */
:root {
    /* Superficies Principales (Modo Claro / Blanco Limpio) */
    --bg-main: transparent;
    --bg-card: #ffffff;
    --bg-card-subtle: #f8fafc;
    --bg-elevated: #ffffff;

    /* Bordes Nítidos y Sobrios */
    --border-color: #e2e8f0;
    --border-light: #f1f5f9;
    --glass-border: rgba(226, 232, 240, 0.8);

    /* Barra Lateral (Sidebar Negro / Grafito Oscuro) */
    --sidebar-bg: #14171a;
    --sidebar-bg-dark: #0b0d0e;
    --sidebar-card: rgba(255, 255, 255, 0.04);
    --sidebar-border: #22262b;
    --sidebar-text: #e5e7eb;
    --sidebar-text-muted: #9ca3af;
    --sidebar-active: #23272e;
    --sidebar-active-text: #ffffff;
    --sidebar-accent: #ffffff;

    /* Tipografías: Contrastes en Modo Claro */
    --text-main: #0f172a;
    --text-heading: #0f172a;
    --text-body: #334155;
    --text-muted: #64748b;
    --text-dim: #94a3b8;
    --text-inactive: #cbd5e1;

    /* Acentos */
    --accent-success: #059669;
    --accent-success-hover: #047857;
    --accent-success-subtle: rgba(5, 150, 105, 0.1);

    --accent-primary: #0284c7;
    --accent-primary-hover: #0369a1;
    --accent-primary-subtle: rgba(2, 132, 199, 0.1);

    --accent-warning: #d97706;
```

```
    --accent-warning-hover: #b45309;
    --accent-warning-subtle: rgba(217, 119, 6, 0.1);

    --accent-cyan: #0d9488;
    --accent-cyan-subtle: rgba(13, 148, 136, 0.1);

    --accent-danger: #dc2626;
    --accent-danger-hover: #b91c1c;
    --accent-danger-subtle: rgba(220, 38, 38, 0.1);

    /* Sombras Limpias */
    --shadow-sm: 0 1px 3px rgba(0, 0, 0, 0.05), 0 1px 2px rgba(0, 0, 0, 0.03);
    --shadow-card: 0 4px 6px -1px rgba(0, 0, 0, 0.04), 0 2px 4px -1px rgba(0, 0, 0, 0.02);
    --shadow-hover: 0 10px 15px -3px rgba(0, 0, 0, 0.08), 0 4px 6px -2px rgba(0, 0, 0, 0.03);
    --shadow-premium: 0 4px 20px -2px rgba(9, 33, 26, 0.08);

    /* Tipografías Oficiales SIFMA */
    --font-brand: 'Philosopher', serif;
    --font-heading: 'Philosopher', serif;
    --font-body: 'Literata', Georgia, -apple-system, BlinkMacSystemFont, serif;
    --font-mono: 'JetBrains Mono', 'SFMono-Regular', Consolas, monospace;

    /* Transiciones */
    --transition-fast: all 0.18s ease-in-out;
    --transition-smooth: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);

    /* Radios de Esquina Nítidos y Sobrios (Modern Clean Sharp) */
    --radius-xs: 3px;
    --radius-sm: 4px;
    --radius-md: 6px;
    --radius-lg: 8px;
    --radius-xl: 10px;
}

/* Reset y Base */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    scrollbar-width: thin;
    scrollbar-color: rgba(148, 163, 184, 0.4) transparent;
}

/* Tipografía Institucional Jerárquica */
h1, h2, h3, h4, h5, h6 {
    font-family: var(--font-heading);
    color: var(--text-heading);
    letter-spacing: 0.25px;
}

body {
    position: relative;
    min-height: 100vh;
    color: var(--text-body);
    font-family: var(--font-body);
```

```
    overflow-x: hidden;
    -webkit-font-smoothing: antialiased;
    background-color: #0f172a;
}

body::before {
    content: '';
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    background-image: url('../img/cenital_1.png');
    background-size: cover;
    background-position: center center;
    background-repeat: no-repeat;
    filter: blur(5px) brightness(0.78) saturate(0.95);
    transform: scale(1.04);
    z-index: -2;
    pointer-events: none;
}

body::after {
    content: '';
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    background: rgba(15, 23, 42, 0.28);
    z-index: -1;
    pointer-events: none;
}

.app-container {
    display: flex;
    min-height: 100vh;
    width: 100%;
}

/* ========================================================================
   2. SISTEMA GLOBAL DE BOTONES Y ACCIONES
   ======================================================================== */
.btn,
.btn-primary,
.btn-secondary,
.btn-outline,
.btn-header-action,
.filter-btn,
.filter-pill {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    padding: 8px 16px;
```

```
    border-radius: var(--radius-sm);
    font-family: var(--font-body);
    font-size: 13.5px;
    font-weight: 600;
    text-decoration: none;
    cursor: pointer;
    border: 1px solid transparent;
    transition: var(--transition-fast);
    line-height: 1.3;
    white-space: nowrap;
}

.btn .material-icons-round,
.btn-primary .material-icons-round,
.btn-secondary .material-icons-round,
.btn-outline .material-icons-round,
.btn-header-action .material-icons-round,
.filter-btn .material-icons-round,
.filter-pill .material-icons-round {
    font-size: 18px;
}

/* Botón Primario (Grafito / Negro Elegante) */
.btn-primary,
.btn-header-primary {
    background: #18181b;
    color: #ffffff !important;
    border: 1px solid #18181b;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.2);
}

.btn-primary:hover,
.btn-header-primary:hover {
    background: #2a2e35;
    border-color: #2a2e35;
    color: #ffffff !important;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.25);
    transform: translateY(-1px);
}

/* Botón Éxito */
.btn-success {
    background: var(--accent-success);
    color: #ffffff !important;
    border: 1px solid var(--accent-success);
    box-shadow: 0 2px 6px rgba(5, 150, 105, 0.2);
}

.btn-success:hover {
    background: var(--accent-success-hover);
    border-color: var(--accent-success-hover);
    transform: translateY(-1px);
}

/* Botón Secundario / Outline */
```

```
.btn-secondary,
.btn-outline,
.btn-header-action {
    background: #ffffff;
    color: var(--text-body) !important;
    border: 1px solid var(--border-color);
    box-shadow: var(--shadow-sm);
}

.btn-secondary:hover,
.btn-outline:hover,
.btn-header-action:hover {
    background: #f8fafc;
    border-color: #cbd5e1;
    color: var(--text-heading) !important;
}

/* Botón Peligro / Eliminar */
.btn-danger {
    background: #ffffff;
    color: var(--accent-danger) !important;
    border: 1px solid rgba(220, 38, 38, 0.25);
    box-shadow: var(--shadow-sm);
}

.btn-danger:hover {
    background: var(--accent-danger);
    color: #ffffff !important;
    border-color: var(--accent-danger);
}

/* Botones de Filtro y Píldoras de Período */
.filter-group {
    display: flex;
    align-items: center;
    gap: 8px;
    flex-wrap: wrap;
}

.filter-btn,
.filter-pill {
    background: #ffffff;
    border: 1px solid var(--border-color);
    color: var(--text-muted) !important;
    padding: 7px 14px;
    font-size: 13px;
    font-weight: 500;
    border-radius: var(--radius-sm);
    box-shadow: var(--shadow-sm);
}

.filter-btn:hover,
.filter-pill:hover {
    background: #f8fafc;
    color: var(--text-heading) !important;
```

```
    border-color: #cbd5e1;
}

.filter-btn.active,
.filter-pill.active {
    background: #16181b;
    border-color: #16181b;
    color: #ffffff !important;
    font-weight: 600;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2);
}

.filter-btn.active .material-icons-round,
.filter-pill.active .material-icons-round {
    color: #ffffff;
}

/* Tamaños de Botones */
.btn-sm {
    padding: 5px 10px;
    font-size: 12px;
    border-radius: var(--radius-xs);
}

.btn-sm .material-icons-round {
    font-size: 15px;
}

.btn-lg {
    padding: 12px 24px;
    font-size: 15px;
    border-radius: var(--radius-md);
}

/* ========================================================================
   3. SISTEMA DE TARJETAS Y CONTENEDORES
   ======================================================================== */
.card,
.glass-card,
.widget-card {
    background: var(--bg-card);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    padding: 22px;
    box-shadow: var(--shadow-card);
    display: flex;
    flex-direction: column;
    margin-bottom: 22px;
    transition: var(--transition-smooth);
}

.card:hover,
.glass-card:hover,
.widget-card:hover {
    box-shadow: var(--shadow-hover);
```

```
}

.card-header,
.glass-card-title,
.widget-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 16px;
    padding-bottom: 12px;
    border-bottom: 1px solid var(--border-light);
    gap: 12px;
}

.card-title,
.card-header-title,
.glass-card-title h3,
.widget-header-title h3 {
    font-family: var(--font-heading);
    font-size: 16.5px;
    font-weight: 700;
    color: var(--text-heading);
    display: flex;
    align-items: center;
    gap: 8px;
}

.glass-card-title p,
.widget-header-title p {
    font-size: 12.5px;
    color: var(--text-muted);
    margin-top: 2px;
}

/* ========================================================================
   4. SISTEMA DE FORMULARIOS Y CONTROLES DE ENTRADA
   ======================================================================== */
.form-control,
.form-input-control,
.login-input,
.date-selector-input {
    width: 100%;
    padding: 9px 13px;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-sm);
    font-family: var(--font-body);
    font-size: 13.5px;
    color: var(--text-heading);
    background: #ffffff;
    outline: none;
    transition: var(--transition-fast);
}

select.form-input-control,
select.form-control {
```

```
    background-color: #ffffff;
    color: var(--text-heading);
}

select.form-input-control option,
select.form-control option {
    background-color: #ffffff;
    color: var(--text-heading);
}

.form-control:focus,
.form-input-control:focus,
.login-input:focus,
.date-selector-input:focus {
    border-color: #059669;
    box-shadow: 0 0 0 3px rgba(5, 150, 105, 0.15);
}

.form-group {
    display: flex;
    flex-direction: column;
    gap: 6px;
    margin-bottom: 16px;
}

.form-group label {
    font-size: 13px;
    font-weight: 600;
    color: var(--text-heading);
}

.form-group-inline {
    display: flex;
    align-items: center;
    gap: 12px;
}

/* ========================================================================
   5. SISTEMA DE INSIGNIAS (BADGES), PILLS Y ESTADOS
   ======================================================================== */
.badge,
.badge-tag,
.role-pill {
    display: inline-flex;
    align-items: center;
    gap: 4px;
    padding: 4px 10px;
    border-radius: var(--radius-xs);
    font-size: 11.5px;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.4px;
}

.badge-success,
```

```
.badge.emerald {
    background: rgba(5, 150, 105, 0.1);
    color: #047857;
    border: 1px solid rgba(5, 150, 105, 0.25);
}

.badge-warning,
.badge.amber {
    background: rgba(217, 119, 6, 0.1);
    color: #b45309;
    border: 1px solid rgba(217, 119, 6, 0.25);
}

.badge-danger,
.badge.red {
    background: rgba(220, 38, 38, 0.1);
    color: #b91c1c;
    border: 1px solid rgba(220, 38, 38, 0.25);
}

.badge-primary,
.badge.blue {
    background: rgba(37, 99, 235, 0.1);
    color: #1d4ed8;
    border: 1px solid rgba(37, 99, 235, 0.25);
}

/* ========================================================================
   6. TABLAS Y MATRICES SINCRONIZADAS
   ======================================================================== */
.table-responsive,
.matrix-table-container,
.users-table-container {
    background: #ffffff;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    overflow-x: auto;
    box-shadow: var(--shadow-card);
    margin-bottom: 22px;
}

.table,
.matrix-table,
.users-table {
    width: 100%;
    border-collapse: collapse;
    text-align: left;
}

.table th,
.matrix-table th,
.users-table th {
    background: #f8fafc;
    padding: 13px 18px;
    font-size: 11.5px;
```

```
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.6px;
    color: var(--text-muted);
    border-bottom: 1px solid var(--border-color);
    white-space: nowrap;
}

.table td,
.matrix-table td,
.users-table td {
    padding: 14px 18px;
    font-size: 13px;
    border-bottom: 1px solid var(--border-light);
    color: var(--text-body);
    vertical-align: middle;
}

.table tr:last-child td,
.matrix-table tr:last-child td,
.users-table tr:last-child td {
    border-bottom: none;
}

.table tr:hover td,
.matrix-table tr:hover td,
.users-table tr:hover td {
    background: #f8fafc;
}

/* ========================================================================
   7. BARRA LATERAL (SIDEBAR VERDE BOSQUE PROFUNDO)
   ======================================================================== */
.sidebar-overlay {
    position: fixed;
    inset: 0;
    background: rgba(0, 0, 0, 0.65);
    backdrop-filter: blur(3px);
    z-index: 998;
    display: none;
    transition: opacity 0.25s ease;
}

.sidebar-overlay.show {
    display: block !important;
}

.sidebar {
    width: 260px;
    background: linear-gradient(180deg, var(--sidebar-bg) 0%, var(--sidebar-bg-dark) 100%);
    border-right: 1px solid var(--sidebar-border);
    display: flex;
    flex-direction: column;
    position: fixed;
    top: 0;
```

```
    left: 0;
    height: 100vh;
    z-index: 999;
    color: var(--sidebar-text);
    transition: width 0.25s cubic-bezier(0.4, 0, 0.2, 1), transform 0.25s cubic-bezier(0.4, 0, 0.2, 1);
}

.sidebar-header {
    padding: 16px 12px;
    border-bottom: 1px solid var(--sidebar-border);
    display: flex;
    align-items: center;
    justify-content: flex-start;
}

.sidebar-brand-group {
    display: flex;
    align-items: center;
    gap: 10px;
    flex: 1;
    overflow: hidden;
    cursor: pointer;
    user-select: none;
    padding: 4px;
    border-radius: var(--radius-sm);
    transition: background 0.2s ease;
}

.sidebar-brand-group:hover {
    background: rgba(255, 255, 255, 0.06);
}

.sidebar-brand-logo {
    width: 48px;
    height: 48px;
    display: block;
    filter: brightness(0) invert(1);
    flex-shrink: 0;
    transition: transform 0.2s cubic-bezier(0.4, 0, 0.2, 1);
}

.sidebar-brand-group:hover .sidebar-brand-logo {
    transform: scale(1.08);
}

.sidebar-brand-group:active .sidebar-brand-logo {
    transform: scale(0.95);
}

.sidebar-logo-icon {
    width: 48px;
    height: 48px;
    background: transparent !important;
    border-radius: 0 !important;
    box-shadow: none !important;
```

```
    border: none !important;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
}

.sidebar-brand-text {
    flex: 1;
    min-width: 0;
    overflow: visible;
    white-space: nowrap;
}

.sidebar-brand-text h2 {
    font-family: var(--font-brand);
    font-size: 21px;
    font-weight: 700;
    letter-spacing: 1.2px;
    color: #ffffff;
    line-height: 1.1;
    margin: 0;
}

.sidebar-brand-text span {
    font-family: var(--font-body);
    font-size: 8.8px;
    color: var(--sidebar-text-muted);
    font-weight: 600;
    letter-spacing: 0.35px;
    text-transform: uppercase;
    display: block;
    margin-top: 2px;
    white-space: nowrap;
}

.sidebar-toggle-btn {
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.12);
    border-radius: var(--radius-sm);
    padding: 5px;
    color: #cbd5e1;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: var(--transition-fast);
    flex-shrink: 0;
}

.sidebar-toggle-btn:hover {
    background: rgba(255, 255, 255, 0.12);
    color: #ffffff;
    border-color: rgba(255, 255, 255, 0.3);
}
```

```
659
660 .mobile-sidebar-trigger {
661     display: none;
662     background: #ffffff;
663     border: 1px solid var(--border-color);
664     border-radius: var(--radius-sm);
665     padding: 6px;
666     cursor: pointer;
667     color: var(--text-heading);
668     transition: var(--transition-fast);
669 }
670
671 .mobile-sidebar-trigger:hover {
672     background: var(--bg-elevated);
673     color: var(--sidebar-accent);
674 }
675
676 .sidebar-close-mobile-btn {
677     display: none;
678     background: transparent;
679     border: none;
680     color: #cbd5e1;
681     cursor: pointer;
682     padding: 4px;
683     border-radius: var(--radius-xs);
684 }
685
686 .sidebar-scrollable {
687     flex: 1;
688     overflow-y: auto;
689     padding: 16px 12px;
690     display: flex;
691     flex-direction: column;
692     gap: 18px;
693 }
694
695 .nav-section-title {
696     font-size: 9.5px;
697     font-weight: 700;
698     letter-spacing: 0.8px;
699     text-transform: uppercase;
700     color: var(--sidebar-text-muted);
701     padding: 0 8px 6px 8px;
702     white-space: nowrap;
703 }
704
705 .nav-group {
706     display: flex;
707     flex-direction: column;
708     gap: 3px;
709 }
710
711 .nav-link {
712     display: flex;
713     align-items: center;
```

```
    gap: 12px;
    padding: 8.5px 12px;
    border-radius: var(--radius-sm);
    color: #cbd5e1;
    text-decoration: none;
    font-size: 13px;
    font-weight: 500;
    transition: var(--transition-fast);
    white-space: nowrap;
}

.nav-link .material-icons-round {
    font-size: 20px;
    color: var(--sidebar-text-muted);
    transition: var(--transition-fast);
    flex-shrink: 0;
}

.nav-link:hover {
    background: rgba(255, 255, 255, 0.06);
    color: #ffffff;
}

.nav-link:hover .material-icons-round {
    color: #ffffff;
}

.nav-link.active {
    background: var(--sidebar-active);
    color: var(--sidebar-active-text);
    font-weight: 600;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.25);
    border-left: 3px solid #ffffff;
}

.nav-link.active .material-icons-round {
    color: #ffffff;
}

.sidebar-footer {
    padding: 12px 14px;
    border-top: 1px solid var(--sidebar-border);
    background: rgba(0, 0, 0, 0.18);
}

.user-profile-widget {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 8px;
    padding: 6px 8px;
    border-radius: var(--radius-sm);
    background: var(--sidebar-card);
    border: 1px solid var(--sidebar-border);
}
```

```
769
770 .user-avatar-circle {
771     width: 32px;
772     height: 32px;
773     border-radius: 50%;
774     background: linear-gradient(135deg, #374151 0%, #1f2937 100%);
775     color: #ffffff;
776     display: flex;
777     align-items: center;
778     justify-content: center;
779     font-size: 11px;
780     font-weight: 700;
781     flex-shrink: 0;
782     border: 1.5px solid rgba(255, 255, 255, 0.18);
783 }
784
785 .user-info-text {
786     flex: 1;
787     overflow: hidden;
788     white-space: nowrap;
789 }
790
791 .user-info-text .user-name {
792     font-size: 12.5px;
793     font-weight: 600;
794     color: #ffffff;
795     white-space: nowrap;
796     overflow: hidden;
797     text-overflow: ellipsis;
798 }
799
800 .user-info-text .user-role {
801     font-size: 10.5px;
802     color: #9ca3af;
803     white-space: nowrap;
804     overflow: hidden;
805     text-overflow: ellipsis;
806 }
807
808 .user-logout-btn {
809     color: var(--sidebar-text-muted);
810     background: transparent;
811     border: none;
812     cursor: pointer;
813     padding: 4px;
814     border-radius: var(--radius-xs);
815     display: flex;
816     align-items: center;
817     justify-content: center;
818     transition: var(--transition-fast);
819     text-decoration: none;
820 }
821
822 .user-logout-btn:hover {
823     color: #f87171;
```

```
    background: rgba(220, 38, 38, 0.12);
}

/* ELEMENTOS EN MODO COLAPSADO */
.sidebar-collapsed-bottom {
    display: none;
}

/* ESTADO COLAPSADO EN PC */
.sidebar.collapsed {
    width: 68px;
}

.sidebar.collapsed .sidebar-brand-text,
.sidebar.collapsed .nav-section-title,
.sidebar.collapsed .nav-link-text,
.sidebar.collapsed .user-profile-widget {
    display: none !important;
}

.sidebar.collapsed .sidebar-header {
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 14px 6px;
}

.sidebar.collapsed .sidebar-brand-group {
    display: flex !important;
    justify-content: center;
    align-items: center;
    gap: 0;
    padding: 4px;
    width: 100%;
}

.sidebar.collapsed .sidebar-brand-logo {
    width: 44px;
    height: 44px;
}

.sidebar.collapsed .nav-link {
    justify-content: center;
    padding: 10px 0;
}

.sidebar.collapsed .sidebar-footer {
    padding: 14px 6px;
}

.sidebar.collapsed .sidebar-collapsed-bottom {
    display: flex !important;
    flex-direction: column;
    align-items: center;
    gap: 10px;
```

```
    padding: 4px 0;
}

.sidebar.collapsed .sidebar-collapsed-bottom .user-avatar-circle {
    width: 34px;
    height: 34px;
    border-radius: 50%;
    font-size: 11px;
    font-weight: 700;
}

.sidebar.collapsed .sidebar-collapsed-bottom .sidebar-logo-icon {
    width: 32px;
    height: 32px;
    background: transparent !important;
    box-shadow: none !important;
    border: none !important;
}

/* ========================================================================
   8. ENCABEZADO Y CONTENEDOR PRINCIPAL
   ======================================================================== */
.main-content {
    margin-left: 260px;
    flex: 1;
    display: flex;
    flex-direction: column;
    min-height: 100vh;
    background: transparent;
    transition: margin-left 0.25s cubic-bezier(0.4, 0, 0.2, 1);
    min-width: 0;
    width: calc(100vw - 260px);
    max-width: calc(100vw - 260px);
    box-sizing: border-box;
}

.main-content.expanded {
    margin-left: 68px;
    width: calc(100vw - 68px);
    max-width: calc(100vw - 68px);
}

.top-header {
    background: #ffffff;
    border-bottom: 1px solid var(--border-color);
    padding: 16px 32px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    position: sticky;
    top: 0;
    z-index: 90;
    box-shadow: var(--shadow-sm);
}

```

```
.header-left-col {
    display: flex;
    align-items: center;
    gap: 16px;
}

.header-title-block h1 {
    font-family: var(--font-heading);
    font-size: 20px;
    font-weight: 700;
    color: var(--text-heading);
    line-height: 1.2;
}

.header-title-block p {
    font-size: 12.5px;
    color: var(--text-muted);
}

.header-controls-row {
    display: flex;
    align-items: center;
    gap: 12px;
}

.date-navigator-box {
    display: flex;
    align-items: center;
    background: #f8fafc;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-sm);
    padding: 4px 6px;
    box-shadow: var(--shadow-sm);
}

.date-nav-btn {
    background: transparent;
    border: none;
    cursor: pointer;
    color: var(--text-muted);
    padding: 4px 6px;
    border-radius: var(--radius-xs);
    display: flex;
    align-items: center;
    justify-content: center;
    transition: var(--transition-fast);
}

.date-nav-btn:hover {
    background: #e2e8f0;
    color: var(--text-heading);
}

.content-viewport {
    padding: 26px 32px 40px 32px;
```

```
    flex: 1;
}

/* ========================================================================
   9. TARJETAS KPI Y WIDGETS DEL DASHBOARD (DISTRIBUCIÓN LATERAL DIVIDIDA)
   ======================================================================== */
.dashboard-split-layout {
    display: grid;
    grid-template-columns: 280px 1fr;
    gap: 20px;
    align-items: stretch;
    margin-bottom: 24px;
}

.dashboard-kpi-column {
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    gap: 16px;
    height: 100%;
}

.dashboard-kpi-column .kpi-card {
    flex: 1;
    margin-bottom: 0;
    padding: 16px 18px;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
}

.dashboard-charts-main {
    min-width: 0;
    display: flex;
    flex-direction: column;
}

.dashboard-charts-main .analytics-grid-2x2 {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 20px;
    margin-bottom: 0;
    height: 100%;
}

.dashboard-charts-main .analytics-grid-2x2 > .widget-card {
    margin-bottom: 0;
}

.kpi-row-grid {
    display: grid;
    grid-template-columns: repeat(5, 1fr);
    gap: 16px;
    margin-bottom: 24px;
}
```

```
1044
1045 .kpi-card {
1046     background: var(--bg-card);
1047     border: 1px solid var(--border-color);
1048     border-radius: var(--radius-md);
1049     padding: 18px;
1050     box-shadow: var(--shadow-card);
1051     display: flex;
1052     flex-direction: column;
1053     justify-content: space-between;
1054     transition: var(--transition-smooth);
1055 }
1056
1057 .kpi-card:hover {
1058     box-shadow: var(--shadow-hover);
1059     transform: translateY(-2px);
1060 }
1061
1062 .kpi-card-header {
1063     display: flex;
1064     align-items: center;
1065     justify-content: space-between;
1066     margin-bottom: 12px;
1067 }
1068
1069 .kpi-label {
1070     font-size: 11.5px;
1071     font-weight: 700;
1072     text-transform: uppercase;
1073     letter-spacing: 0.6px;
1074     color: var(--text-muted);
1075 }
1076
1077 .kpi-icon-badge {
1078     width: 32px;
1079     height: 32px;
1080     border-radius: var(--radius-sm);
1081     display: flex;
1082     align-items: center;
1083     justify-content: center;
1084     flex-shrink: 0;
1085 }
1086
1087 .kpi-icon-badge.emerald {
1088     background: var(--accent-success-subtle);
1089     color: var(--accent-success);
1090 }
1091
1092 .kpi-icon-badge.blue {
1093     background: var(--accent-primary-subtle);
1094     color: var(--accent-primary);
1095 }
1096
1097 .kpi-icon-badge.cyan {
1098     background: var(--accent-cyan-subtle);
```

```
    color: var(--accent-cyan);
}

.kpi-icon-badge.amber {
    background: var(--accent-warning-subtle);
    color: var(--accent-warning);
}

.kpi-value-row {
    display: flex;
    align-items: baseline;
    gap: 4px;
    margin-bottom: 8px;
}

.kpi-big-number {
    font-family: var(--font-heading);
    font-size: 26px;
    font-weight: 800;
    color: var(--text-heading);
    line-height: 1;
}

.kpi-unit {
    font-size: 13px;
    font-weight: 600;
    color: var(--text-muted);
}

.kpi-footer-badge {
    font-size: 11.5px;
    font-weight: 600;
    display: flex;
    align-items: center;
    gap: 4px;
    padding-top: 6px;
    border-top: 1px solid var(--border-light);
    color: var(--text-muted);
}

.kpi-footer-badge.positive {
    color: var(--accent-success);
}

.kpi-footer-badge.neutral {
    color: var(--text-muted);
}

.analytics-grid-2x2 {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 22px;
    margin-bottom: 24px;
}

```

```
.chart-stage-container {
    position: relative;
    height: 220px;
    width: 100%;
    margin-bottom: 16px;
}

.widget-stats-row {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 12px;
    padding-top: 14px;
    border-top: 1px solid var(--border-light);
}

.widget-stats-item {
    display: flex;
    flex-direction: column;
    gap: 2px;
}

.stat-item-label {
    font-size: 11px;
    font-weight: 700;
    text-transform: uppercase;
    color: var(--text-dim);
    letter-spacing: 0.5px;
}

.stat-item-value {
    font-family: var(--font-heading);
    font-size: 14px;
    font-weight: 700;
    color: var(--text-heading);
}

.stat-item-desc {
    font-size: 11px;
    color: var(--text-muted);
}

.zero-data-banner {
    background: #f8fafc;
    border: 1px solid #cbd5e1;
    border-radius: var(--radius-md);
    padding: 16px 20px;
    display: flex;
    align-items: center;
    gap: 14px;
    margin-bottom: 22px;
}

.zero-data-icon {
    width: 36px;
    height: 36px;
```

```
    border-radius: 50%;
    background: #e2e8f0;
    color: var(--text-muted);
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
}

.zero-data-text h4 {
    font-size: 14px;
    font-weight: 700;
    color: var(--text-heading);
}

.zero-data-text p {
    font-size: 12.5px;
    color: var(--text-muted);
}

/* ========================================================================
   10. ESTILOS DE LOGIN
   ======================================================================== */
.login-page-wrapper {
    min-height: 100vh;
    width: 100vw;
    display: flex;
    align-items: center;
    justify-content: center;
    background: linear-gradient(135deg, #14171a 0%, #0b0d0e 50%, #1e2227 100%);
    padding: 20px;
}

.login-glass-panel {
    width: 100%;
    max-width: 420px;
    background: #f6f8fa;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-lg);
    padding: 40px 36px;
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.4);
    display: flex;
    flex-direction: column;
    align-items: center;
}

.login-brand-header {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 10px;
    margin-bottom: 28px;
    text-align: center;
}

```

```
.login-brand-header h1 {
    font-family: var(--font-brand);
    font-size: 30px;
    font-weight: 700;
    letter-spacing: 2px;
    color: var(--text-heading);
    line-height: 1.1;
    margin-bottom: 2px;
}

.login-brand-header p {
    font-family: var(--font-body);
    font-size: 13px;
    color: var(--text-muted);
    font-weight: 500;
    line-height: 1.4;
}

.login-btn-submit {
    width: 100%;
    padding: 13px;
    border-radius: var(--radius-sm);
    background: linear-gradient(135deg, #16181b 0%, #2a2e35 100%);
    color: #ffffff !important;
    font-family: var(--font-heading);
    font-size: 15px;
    font-weight: 700;
    letter-spacing: 0.5px;
    border: none;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    box-shadow: 0 4px 14px rgba(0, 0, 0, 0.3);
    transition: var(--transition-smooth);
    margin-top: 8px;
}

.login-btn-submit:hover {
    background: #374151;
    transform: translateY(-1.5px);
    box-shadow: 0 6px 18px rgba(0, 0, 0, 0.4);
}

/* ========================================================================
   11. RESPONSIVIDAD INTEGRAL
   ======================================================================== */
.content-viewport {
    padding: 22px 28px;
    width: 100%;
    max-width: 1600px;
    margin: 0 auto;
    box-sizing: border-box;
}
```

```
1319
1320 @media (max-width: 1280px) {
1321     .content-viewport {
1322         padding: 18px 20px;
1323     }
1324     .dashboard-split-layout {
1325         grid-template-columns: 250px 1fr;
1326         gap: 16px;
1327     }
1328     .kpi-row-grid {
1329         grid-template-columns: repeat(3, 1fr);
1330         gap: 12px;
1331     }
1332     .analytics-grid-2x2 {
1333         grid-template-columns: 1fr;
1334         gap: 16px;
1335     }
1336     .dashboard-grid-2col {
1337         grid-template-columns: 1fr;
1338         gap: 16px;
1339     }
1340 }
1341
1342 @media (max-width: 1024px) {
1343     .dashboard-split-layout {
1344         grid-template-columns: 1fr;
1345     }
1346     .dashboard-kpi-column {
1347         display: grid;
1348         grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
1349         gap: 12px;
1350     }
1351 }
1352
1353 @media (max-width: 992px) {
1354     .sidebar {
1355         transform: translateX(-100%);
1356         box-shadow: 4px 0 24px rgba(0, 0, 0, 0.4);
1357     }
1358     .sidebar.open {
1359         transform: translateX(0);
1360     }
1361     .sidebar-close-mobile-btn {
1362         display: block !important;
1363     }
1364     .main-content,
1365     .main-content.expanded {
1366         margin-left: 0 !important;
1367         width: 100vw !important;
1368         max-width: 100vw !important;
1369     }
1370     .top-header {
1371         padding: 12px 18px;
1372         gap: 10px;
1373     }
```

```
    .header-title-block h1 {
        font-size: 17px;
    }
    .kpi-row-grid {
        grid-template-columns: repeat(2, 1fr);
    }
    .content-viewport {
        padding: 14px 16px;
    }
}

@media (max-width: 640px) {
    .top-header {
        flex-direction: column;
        align-items: stretch;
    }
    .header-left-col {
        justify-content: space-between;
    }
    .header-controls-row {
        justify-content: space-between;
        flex-wrap: wrap;
    }
    .kpi-row-grid {
        grid-template-columns: 1fr;
    }
    .widget-stats-row {
        grid-template-columns: 1fr;
    }
    .filter-group {
        overflow-x: auto;
        padding-bottom: 4px;
    }
}

/* ========================================================================
   12. MODALES Y GESTIÓN DE USUARIOS RBAC
   ======================================================================== */
.modal-backdrop {
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    background: rgba(0, 0, 0, 0.65);
    backdrop-filter: blur(4px);
    display: none;
    align-items: center;
    justify-content: center;
    z-index: 9999;
    padding: 20px;
}

.modal-backdrop.show {
    display: flex !important;
```

```
}

.modal-dialog-box {
    background: #f6f8fa;
    width: 100%;
    max-width: 520px;
    border-radius: var(--radius-lg);
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.35);
    border: 1px solid var(--border-color);
    overflow: hidden;
    animation: modalSlideIn 0.25s ease-out;
}

@keyframes modalSlideIn {
    from {
        opacity: 0;
        transform: translateY(-20px) scale(0.97);
    }
    to {
        opacity: 1;
        transform: translateY(0) scale(1);
    }
}

.modal-header-box {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 18px 24px;
    border-bottom: 1px solid var(--border-light);
    background: #ebf0f5;
}

.modal-header-box h3 {
    font-size: 16px;
    font-weight: 700;
    color: var(--text-heading);
    margin: 0;
}

.modal-body-box {
    padding: 24px;
    display: flex;
    flex-direction: column;
    gap: 16px;
    max-height: 75vh;
    overflow-y: auto;
}

.modal-footer-box {
    padding: 16px 24px;
    border-top: 1px solid var(--border-light);
    background: #ebf0f5;
    display: flex;
    justify-content: flex-end;
```

```
    gap: 10px;
}

.form-group-field {
    display: flex;
    flex-direction: column;
    gap: 6px;
}

.form-group-field label {
    font-size: 13px;
    font-weight: 600;
    color: var(--text-heading);
}

.form-input-control {
    width: 100%;
    padding: 10px 14px;
    border-radius: var(--radius-sm);
    border: 1px solid var(--border-color);
    font-size: 13.5px;
    color: var(--text-body);
    background: #fbfcfd;
    outline: none;
    transition: border-color 0.2s, box-shadow 0.2s;
}

.form-input-control:focus {
    border-color: var(--accent-success);
    box-shadow: 0 0 0 3px rgba(5, 150, 105, 0.12);
}

.users-table-container {
    background: #f6f8fa;
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
    overflow: hidden;
    box-shadow: var(--shadow-sm);
}

.users-table {
    width: 100%;
    border-collapse: collapse;
    text-align: left;
    font-size: 13.5px;
}

.users-table th {
    background: #ebf0f5;
    padding: 14px 18px;
    font-size: 11.5px;
    font-weight: 700;
    color: var(--text-muted);
    text-transform: uppercase;
    border-bottom: 1px solid var(--border-color);
```

```
}

.users-table td {
    padding: 14px 18px;
    border-bottom: 1px solid var(--border-light);
    color: var(--text-body);
}

.users-table tr:hover td {
    background: rgba(235, 240, 245, 0.8);
}

/* ========================================================================
   13. CONTENEDORES DE GRÁFICAS AMPLIADAS
   ======================================================================== */
.chart-stage-container {
    position: relative;
    width: 100%;
    height: 320px;
    margin-top: 12px;
}

/* ========================================================================
   14. MATRIZ DE CORRELACIÓN (HEATMAP) Y LABORATORIO DE CRUCE
   ======================================================================== */
.heatmap-table-container {
    background: #ffffff;
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
    overflow-x: auto;
    box-shadow: var(--shadow-card);
}

.heatmap-table {
    width: 100%;
    border-collapse: collapse;
    text-align: center;
    font-size: 13px;
}

.heatmap-table th {
    background: #f8fafc;
    padding: 12px 14px;
    font-size: 11.5px;
    font-weight: 700;
    color: var(--text-muted);
    text-transform: uppercase;
    border: 1px solid var(--border-color);
}

.heatmap-table td {
    padding: 12px 10px;
    border: 1px solid var(--border-color);
    transition: transform 0.15s, box-shadow 0.15s;
    cursor: pointer;
```

```
}

.heatmap-table td:hover {
    transform: scale(1.04);
    box-shadow: 0 4px 12px rgba(0,0,0,0.08);
    z-index: 2;
    position: relative;
}

.heatmap-cell-content {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 2px;
}

.heatmap-r-val {
    font-size: 15px;
    font-weight: 800;
    letter-spacing: -0.3px;
}

.heatmap-r2-val {
    font-size: 10.5px;
    font-weight: 600;
    opacity: 0.85;
}

.heatmap-interp-badge {
    font-size: 9.5px;
    font-weight: 700;
    padding: 1px 6px;
    border-radius: var(--radius-xs);
    margin-top: 3px;
    white-space: nowrap;
}

.comparator-lab-card {
    background: #ffffff;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    padding: 20px;
    box-shadow: var(--shadow-card);
}

.photo-breakdown-row {
    background: rgba(235, 240, 245, 0.7);
    border-left: 3px solid var(--accent-primary);
}

.photo-thumb-mini {
    width: 44px;
    height: 44px;
    object-fit: cover;
    border-radius: var(--radius-xs);
```

```
    border: 1px solid var(--border-color);
}

/* ========================================================================
   15. MODAL ONBOARDING / BIENVENIDA GLOBAL CON FONDO DESENFOCADO
   ======================================================================== */
.onboarding-modal-backdrop {
    position: fixed;
    inset: 0;
    width: 100vw;
    height: 100vh;
    background: rgba(15, 23, 42, 0.65);
    backdrop-filter: blur(8px);
    -webkit-backdrop-filter: blur(8px);
    z-index: 10000;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 20px;
    animation: fadeIn 0.25s ease;
}

.onboarding-dialog-card {
    background: #ffffff;
    border-radius: var(--radius-lg);
    border: 1px solid var(--border-color);
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.35);
    max-width: 680px;
    width: 100%;
    overflow: hidden;
    animation: modalSlideIn 0.28s cubic-bezier(0.4, 0, 0.2, 1);
    display: flex;
    flex-direction: column;
}

.onboarding-header {
    position: relative;
    padding: 28px 24px 20px 24px;
    background: linear-gradient(180deg, #f8fafc 0%, #ffffff 100%);
    border-bottom: 1px solid var(--border-color);
    display: flex;
    flex-direction: column;
    align-items: center;
    text-align: center;
}

.onboarding-logo-img {
    width: 120px;
    height: 120px;
    object-fit: contain;
    display: block;
    margin-bottom: 14px;
    filter: drop-shadow(0 4px 14px rgba(0, 0, 0, 0.08));
    transition: transform 0.2s ease;
}
```

```
1704
1705 .onboarding-logo-img:hover {
1706     transform: scale(1.03);
1707 }
1708
1709 .btn-close-onboarding {
1710     position: absolute;
1711     top: 14px;
1712     right: 14px;
1713     background: transparent;
1714     border: none;
1715     color: var(--text-muted);
1716     cursor: pointer;
1717     padding: 6px;
1718     border-radius: var(--radius-xs);
1719     display: flex;
1720     align-items: center;
1721     justify-content: center;
1722     transition: var(--transition-fast);
1723 }
1724
1725 .btn-close-onboarding:hover {
1726     background: #e2e8f0;
1727     color: var(--text-heading);
1728 }
1729
1730 .onboarding-body {
1731     padding: 22px 24px;
1732     max-height: 70vh;
1733     overflow-y: auto;
1734 }
1735
1736 .onboarding-steps-grid {
1737     display: grid;
1738     grid-template-columns: 1fr 1fr;
1739     gap: 14px;
1740 }
1741
1742 .onboarding-step-item {
1743     background: var(--bg-card-subtle);
1744     border: 1px solid var(--border-color);
1745     border-radius: var(--radius-sm);
1746     padding: 14px;
1747     display: flex;
1748     gap: 12px;
1749     align-items: flex-start;
1750 }
1751
1752 .step-num-badge {
1753     width: 24px;
1754     height: 24px;
1755     border-radius: 50%;
1756     background: #14171a;
1757     color: #ffffff;
1758     font-size: 11.5px;
```

```
    font-weight: 700;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
}

.step-content h4 {
    font-size: 13px;
    font-weight: 700;
    color: var(--text-heading);
    margin-bottom: 4px;
}

.step-content p {
    font-size: 11.5px;
    color: var(--text-muted);
    line-height: 1.45;
    margin: 0;
}

.onboarding-footer {
    padding: 14px 22px;
    background: #f8fafc;
    border-top: 1px solid var(--border-color);
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
    flex-wrap: wrap;
}

@media (max-width: 640px) {
    .onboarding-steps-grid {
        grid-template-columns: 1fr;
    }
}
```

ARCHIVO 56/85: local_server/static/js/dashboard.js (209 líneas)

```
/**
 * SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
 * Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
 * Año: 2026.
 */

/**
 * SIFMA - LÓGICA DE GRÁFICAS DEL DASHBOARD ANALÍTICO DIARIO
 * Renderizado de las 4 gráficas del día seleccionado usando Chart.js
 */

document.addEventListener('DOMContentLoaded', () => {
    if (typeof dailyData === 'undefined') return;

    // Configuración global de fuentes para Chart.js
    Chart.defaults.font.family = "'Inter', sans-serif";
    Chart.defaults.color = '#64748b';

    // 1. GRÁFICA 1: CURVA TÉRMICA DE LA JORNADA
    const ctxThermal = document.getElementById('thermalCurveChart');
    if (ctxThermal) {
        const thermal = dailyData.thermal_curve || {};
        const labels = thermal.labels || ["07:00", "10:00", "13:00", "16:00", "19:00"];
        const temps = thermal.temps || [0, 0, 0, 0, 0];

        const gradientTemp = ctxThermal.getContext('2d').createLinearGradient(0, 0, 0, 200);
        gradientTemp.addColorStop(0, 'rgba(239, 68, 68, 0.25)');
        gradientTemp.addColorStop(1, 'rgba(37, 99, 235, 0.02)');

        new Chart(ctxThermal, {
            type: 'line',
            data: {
                labels: labels,
                datasets: [
                    {
                        label: 'Temperatura (°C)',
                        data: temps,
                        borderColor: '#2563eb',
                        backgroundColor: gradientTemp,
                        borderWidth: 2.5,
                        fill: true,
                        tension: 0.35,
                        pointBackgroundColor: '#1d4ed8',
                        pointBorderColor: '#ffffff',
                        pointBorderWidth: 2,
                        pointRadius: 4,
                        pointHoverRadius: 6
                    }
                ]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
```

```
                plugins: {
                    legend: { display: false },
                    tooltip: {
                        backgroundColor: '#09211a',
                        titleColor: '#ffffff',
                        bodyColor: '#e2e8f0',
                        padding: 10,
                        cornerRadius: 8,
                        callbacks: {
                            label: (ctx) => ` Temperatura: ${ctx.parsed.y} °C`
                        }
                    }
                },
                scales: {
                    x: {
                        grid: { display: false },
                        ticks: { font: { size: 11 } }
                    },
                    y: {
                        grid: { color: '#f1f5f9' },
                        ticks: {
                            callback: (v) => `${v}°C`,
                            font: { size: 11 }
                        },
                        beginAtZero: true,
                        suggestedMax: Math.max(...temps, 30)
                    }
                }
            }
        });
    }

    // 2. GRÁFICA 2: ESTADO DE BOMBA / MOTOR DE RECIRCULACIÓN (1 = ON, 0 = OFF)
    const ctxMotor = document.getElementById('motorStateChart');
    if (ctxMotor) {
        const motor = dailyData.motor_chart || {};
        const labels = motor.labels || ["07:00", "10:00", "13:00", "16:00", "19:00"];
        const states = motor.states || [0, 0, 0, 0, 0];

        const gradientMotor = ctxMotor.getContext('2d').createLinearGradient(0, 0, 0, 200);
        gradientMotor.addColorStop(0, 'rgba(5, 150, 105, 0.35)');
        gradientMotor.addColorStop(1, 'rgba(5, 150, 105, 0.03)');

        new Chart(ctxMotor, {
            type: 'line',
            data: {
                labels: labels,
                datasets: [{
                    label: 'Estado Motor (1=ON, 0=OFF)',
                    data: states,
                    borderColor: '#059669',
                    backgroundColor: gradientMotor,
                    borderWidth: 2.5,
                    fill: true,
                    stepped: 'before',
```

```
                    pointBackgroundColor: '#047857',
                    pointBorderColor: '#ffffff',
                    pointBorderWidth: 2,
                    pointRadius: 3,
                    pointHoverRadius: 6
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                plugins: {
                    legend: { display: false },
                    tooltip: {
                        backgroundColor: '#09211a',
                        titleColor: '#ffffff',
                        bodyColor: '#e2e8f0',
                        padding: 10,
                        cornerRadius: 8,
                        callbacks: {
                            label: (ctx) => ` Estado: ${ctx.parsed.y === 1 ? '1 - ENCENDIDO (ON)' : '0 - APAGADO (OFF)'}`
                        }
                    }
                },
                scales: {
                    x: {
                        grid: { display: false },
                        ticks: { font: { size: 11 } }
                    },
                    y: {
                        grid: { color: '#f1f5f9' },
                        min: -0.1,
                        max: 1.1,
                        ticks: {
                            stepSize: 1,
                            callback: (v) => {
                                if (v === 1) return '1 (ON)';
                                if (v === 0) return '0 (OFF)';
                                return '';
                            },
                            font: { size: 11, weight: 'bold' }
                        }
                    }
                }
            }
        });
    }

    // 4. GRÁFICA 4: RADIACIÓN SOLAR UV (LUX)
    const ctxSolar = document.getElementById('solarIrradianceChart');
    if (ctxSolar) {
        const solar = dailyData.solar_chart || {};
        const labels = solar.labels || ["07:00", "09:00", "11:00", "13:00", "15:00", "17:00", "19:00"];
        const uvs = solar.uv_values || [0, 0, 0, 0, 0, 0, 0];

        new Chart(ctxSolar, {
```

```
            type: 'bar',
            data: {
                labels: labels,
                datasets: [{
                    label: 'Radiación (lux)',
                    data: uvs,
                    backgroundColor: '#f59e0b',
                    borderRadius: 6,
                    barThickness: 24
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                plugins: {
                    legend: { display: false },
                    tooltip: {
                        backgroundColor: '#09211a',
                        titleColor: '#ffffff',
                        bodyColor: '#e2e8f0',
                        padding: 10,
                        cornerRadius: 8,
                        callbacks: {
                            label: (ctx) => ` Radiación: ${ctx.parsed.y} lux`
                        }
                    }
                },
                scales: {
                    x: {
                        grid: { display: false },
                        ticks: { font: { size: 11 } }
                    },
                    y: {
                        grid: { color: '#f1f5f9' },
                        ticks: {
                            callback: (v) => `${v} lux`,
                            font: { size: 11 }
                        },
                        beginAtZero: true,
                        suggestedMax: Math.max(...uvs, 600)
                    }
                }
            }
        });
    }
});
```

ARCHIVO 57/85: local_server/static/js/processing.js (112 líneas)

```
/**
 * SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
 * Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
 * Año: 2026.
 */

/* ========================================================================
   SIFMA - DETAILED PLANT BIOMETRICS CHARTS (IMAGE ANALYSIS)
   ======================================================================== */

document.addEventListener('DOMContentLoaded', () => {
    if (typeof growthData !== 'undefined' && growthData.length > 0) {

        // Prepare datasets
        const labels = growthData.map(d => {
            const parts = d.date.split('-');
            return parts.length === 3 ? `${parts[2]}/${parts[1]}` : d.date;
        });

        const areaData = growthData.map(d => d.area);
        const heightData = growthData.map(d => d.height);
        const diameterData = growthData.map(d => d.diameter);
        const healthData = growthData.map(d => d.health);

        // Helper to instantiate premium dark-slate scale line charts
        const createBiometricChart = (ctxId, label, data, borderColor, glowColor, suffix, maxVal = null) => {
            const ctx = document.getElementById(ctxId);
            if (!ctx) return;

            const canvasContext = ctx.getContext('2d');
            const grad = canvasContext.createLinearGradient(0, 0, 0, 250);
            grad.addColorStop(0, glowColor);
            grad.addColorStop(1, 'rgba(0,0,0,0)');

            new Chart(ctx, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [{
                        label: label,
                        data: data,
                        borderColor: borderColor,
                        borderWidth: 2.5,
                        backgroundColor: grad,
                        fill: true,
                        tension: 0.35,
                        pointBackgroundColor: borderColor,
                        pointBorderColor: '#ffffff',
                        pointHoverRadius: 7,
                        pointHoverBackgroundColor: borderColor,
                        pointHoverBorderColor: '#ffffff',
                        pointHoverBorderWidth: 2,
                    }]
```

```
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                plugins: {
                    legend: { display: false },
                    tooltip: {
                        backgroundColor: '#0f172a',
                        titleColor: '#ffffff',
                        bodyColor: '#e2e8f0',
                        borderColor: 'rgba(255,255,255,0.08)',
                        borderWidth: 1,
                        padding: 12,
                        cornerRadius: 10,
                        displayColors: false,
                        callbacks: {
                            label: function(context) {
                                return ` ${context.dataset.label}: ${context.parsed.y}${suffix}`;
                            }
                        }
                    }
                },
                scales: {
                    x: {
                        grid: { color: 'rgba(15, 23, 42, 0.04)' },
                        ticks: { color: '#475569', font: { family: 'Inter', size: 11 } }
                    },
                    y: {
                        beginAtZero: true,
                        max: maxVal,
                        grid: { color: 'rgba(15, 23, 42, 0.05)' },
                        ticks: {
                            color: '#475569',
                            font: { family: 'Inter', size: 11 },
                            callback: function(value) {
                                const cleanVal = Math.floor(value) === value ? value : parseFloat(value.toFixed(2));
                                return `${cleanVal}${suffix}`;
                            }
                        }
                    }
                }
            }
        });
    };

    // Initialize each detailed biometric chart with premium colors
    // Foliar Area (Emerald green)
    createBiometricChart('foliarAreaChart', 'Área Foliar', areaData, '#10b981', 'rgba(16, 185, 129, 0.2)', ' cm²');

    // Plant Height (Electric blue)
    createBiometricChart('plantHeightChart', 'Altura Planta', heightData, '#3b82f6', 'rgba(59, 130, 246, 0.2)', ' cm');

    // Stem Diameter (Amber orange)
    createBiometricChart('stemDiameterChart', 'Diámetro Tallo', diameterData, '#f59e0b', 'rgba(245, 158, 11, 0.2)', ' mm');

```

```
109         // Health Index (Vibrant Red)
110         createBiometricChart('healthIndexChart', 'Índice de Salud', healthData, '#dc2626', 'rgba(220, 38, 38, 0.15)', '%', 100);
111     }
112 });
```

ARCHIVO 58/85: local_server/static/js/sensors.js (102 líneas)

```
/**
 * SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
 * Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
 * Año: 2026.
 */

/* ========================================================================
   SIFMA - DETAILED ENVIRONMENTAL SENSORS GRAPHICS (FASE 2)
   ======================================================================== */

document.addEventListener('DOMContentLoaded', () => {
    if (typeof sensorRaw !== 'undefined' && sensorRaw.length > 0) {
        const labels = sensorRaw.map(s => s.time);
        const tempData = sensorRaw.map(s => s.temp);
        const humData = sensorRaw.map(s => s.hum);
        const uvData = sensorRaw.map(s => s.uv);
        const currData = sensorRaw.map(s => s.curr);

        // Helper to create an elegant, responsive line chart with custom styling
        const createSensorChart = (ctxId, label, data, borderColor, glowColor, suffix) => {
            const ctx = document.getElementById(ctxId);
            if (!ctx) return;

            // Generate premium linear gradient under the line
            const canvasContext = ctx.getContext('2d');
            const grad = canvasContext.createLinearGradient(0, 0, 0, 250);
            grad.addColorStop(0, glowColor);
            grad.addColorStop(1, 'rgba(0,0,0,0)');

            new Chart(ctx, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [{
                        label: label,
                        data: data,
                        borderColor: borderColor,
                        borderWidth: 2.5,
                        backgroundColor: grad,
                        fill: true,
                        tension: 0.35,
                        pointBackgroundColor: borderColor,
                        pointBorderColor: '#ffffff',
                        pointHoverRadius: 7,
                        pointHoverBackgroundColor: borderColor,
                        pointHoverBorderColor: '#ffffff',
                        pointHoverBorderWidth: 2,
                    }]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    plugins: {
```

```
                        legend: { display: false },
                        tooltip: {
                            backgroundColor: '#0f172a',
                            titleColor: '#ffffff',
                            bodyColor: '#e2e8f0',
                            borderColor: 'rgba(255,255,255,0.08)',
                            borderWidth: 1,
                            padding: 12,
                            cornerRadius: 10,
                            displayColors: false,
                            callbacks: {
                                label: function(context) {
                                    return ` ${context.dataset.label}: ${context.parsed.y}${suffix}`;
                                }
                            }
                        }
                    },
                    scales: {
                        x: {
                            grid: { color: 'rgba(15, 23, 42, 0.05)' },
                            ticks: { color: '#475569', font: { family: 'Inter', size: 11 } }
                        },
                        y: {
                            grid: { color: 'rgba(15, 23, 42, 0.05)' },
                            ticks: {
                                color: '#475569',
                                font: { family: 'Inter', size: 11 },
                                callback: value => `${value}${suffix}`
                            }
                        }
                    }
                });
        };

        // Initialize each detailed chart with premium colors
        // Red glow for Temperature
        createSensorChart('tempChart', 'Temperatura', tempData, '#ef4444', 'rgba(239, 68, 68, 0.2)', '°C');

        // Blue glow for Humidity
        createSensorChart('humidityChart', 'Humedad Relativa', humData, '#3b82f6', 'rgba(59, 130, 246, 0.2)', '%');

        // Amber/Golden glow for UV Radiation
        createSensorChart('uvChart', 'Radiación UV', uvData, '#f59e0b', 'rgba(245, 158, 11, 0.2)', ' lux');

        // Emerald/Mint glow for Pump Current
        createSensorChart('currentChart', 'Corriente Bomba', currData, '#10b981', 'rgba(16, 185, 129, 0.2)', ' A');
    }
});
```

ARCHIVO 59/85: local_server/templates/about.html (293 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends "base.html" %}

{% block title %}Información del Sistema y Créditos - SIFMA{% endblock %}
{% block heading %}Información del Sistema{% endblock %}
{% block subheading %}Especificaciones técnicas, créditos, autores y términos de licencia oficial de SIFMA{% endblock %}

{% block content %}
<div class="content-container">

    <!-- HERO HEADER CARD INSTITUCIONAL LIMPIA Y SERIA -->
    <div class="card" style="border-radius: 12px; background: var(--bg-card); color: var(--text-main); border: 1px solid var(--border-color); box-shadow: var(--shadow-card); margin-bottom: 24px; padding: 24px 28px;">
        <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 20px;">

            <!-- Bloque Logo y Título -->
            <div style="display: flex; align-items: center; gap: 20px;">
                <img src="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}" alt="SIFMA Emblem" style="width: 70px; height: 70px; display: block; flex-shrink: 0;">
                <div>
                    <div style="display: flex; align-items: center; gap: 10px; margin-bottom: 5px; flex-wrap: wrap;">
                        <h2 style="font-family: var(--font-heading, 'Philosopher'); font-size: 26px; font-weight: 700; letter-spacing: 1px; margin: 0; color: var(--text-heading);">SIFMA</h2>
                        <span class="badge" style="background: rgba(15, 23, 42, 0.06); color: var(--text-main); border: 1px solid var(--border-color); font-family: monospace; font-size: 11.5px; padding: 3px 8px; border-radius: 6px; font-weight: 600;">
                            Versión {{ system_meta.version }}
                        </span>
                    </div>
                    <p style="font-size: 13.5px; color: var(--text-body); font-weight: 500; margin: 0; line-height: 1.4;">
                        Sistema Integrado de Fenotipado Digital y Telemetría Agronómica en Torres Aeropónicas Verticales
                    </p>
                </div>
            </div>

            <!-- Botones de Acción -->
            <div style="display: flex; align-items: center; gap: 10px; flex-wrap: wrap;">
                <button type="button" onclick="openOnboardingModal()" class="btn btn-primary" style="font-size: 12.5px; padding: 9px 16px;">
                    <span class="material-icons-round" style="font-size: 16px;">menu_book</span>
                    <span>Guía de Inicio SIFMA</span>
                </button>
                <a href="https://github.com/IsAndresL/SIFMA" target="_blank" rel="noopener noreferrer" class="btn btn-secondary" style="font-size: 12.5px; padding: 9px 16px; text-decoration: none;">
                    <svg height="15" width="15" viewBox="0 0 16 16" fill="currentColor" style="display: inline-block;">
                        <path d="M8 0C3.58 0 0 3.58 0 8c0 3.54 2.29 6.53 5.47 7.59.4.07.55-.17.55-.38 0-.19-.01-.82-.01-1.49-2.01.37-2.53-.49-2.69-.94-.09-.23-.48-.94-.82-1.13-.28-.15-.68-.52-.01-.53.63-.01 1.08.58 1.23.82.72 1.21 1.87.87 2.33.66.07-.52.28-.87.51-1.07-1.78-.2-3.64-.89-3.64-3.95 0-.87.31-1.59.82-2.15-.08-.2-.36-1.02.08-2.12 0 0 .67-.21 2.2.82.64-.18 1.32-.27 2-.27.68 0 1.36.09 2 .27 1.53-1.04 2.2-.82 2.2-.82.44 1.1.16 1.92.08 2.12.51.56.82 1.27.82 2.15 0 3.07-1.87 3.75-3.65 3.95.29.25.54.73.54 1.48 0 1.07-.01 1.93-.01 2.2 0 .21.15.46.55.38A8.013 8.013 0 0016 8c0-4.42-3.58-8-8-8z"></path>
                    </svg>
                    <span>Repositorio en GitHub</span>
                </a>
            </div>
        </div>
```

```
    </div>

    <!-- GRID DE 2 COLUMNAS PRINCIPALES -->
    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(360px, 1fr)); gap: 20px; margin-bottom: 24px;">

        <!-- TARJETA 1: AUTORÍA, CRÉDITOS Y LÍNEA DE INVESTIGACIÓN -->
        <div class="card" style="border-radius: 14px; box-shadow: var(--shadow-sm);">
            <div class="card-header">
                <div class="card-header-title">
                    <span class="material-icons-round text-primary" style="font-size: 20px;">badge</span>
                    <span>Créditos, Dirección Científica & Autores</span>
                </div>
            </div>

            <table class="table" style="font-size: 12px;">
                <tbody>
                    <tr>
                        <td style="font-weight: 700; width: 35%; color: var(--text-muted);">Profesores Fundadores & Dirección Científica:</td>
                        <td>
                            <div style="display: flex; flex-direction: column; gap: 3px;">
                                <div><strong style="color: var(--text-heading);">Prof. Críspulo Enrique Deluque</strong> <span style="font-size: 11px; color: var(--text-muted);">(Profesor Asesor & Director de Investigación)</span></div>
                            </div>
                        </td>
                    </tr>
                    <tr>
                        <td style="font-weight: 700; color: var(--text-muted);">Autores del Software:</td>
                        <td>
                            <div style="display: flex; flex-direction: column; gap: 4px;">
                                <div>
                                    <a href="https://github.com/IsAndresL" target="_blank" rel="noopener noreferrer" style="color: var(--accent-emerald); font-weight: 800; text-decoration: none; display: inline-flex; align-items: center; gap: 5px; padding: 2px 7px; border-radius: 5px; background: rgba(5, 150, 105, 0.08); border: 1px solid rgba(5, 150, 105, 0.2); transition: all 0.15s ease;" onmouseover="this.style.background='rgba(5, 150, 105, 0.18)'; this.style.borderColor='var(--accent-emerald)';" onmouseout="this.style.background='rgba(5, 150, 105, 0.08)'; this.style.borderColor='rgba(5, 150, 105, 0.2)';">
                                        <svg height="13" width="13" viewBox="0 0 16 16" fill="currentColor">
                                            <path d="M8 0C3.58 0 0 3.58 0 8c0 3.54 2.29 6.53 5.47 7.59.4.07.55-.17.55-.38 0-.19-.01-.82-.01-1.49-2.01.37-2.53-.49-2.69-.94-.09-.23-.48-.94-.82-1.13-.28-.15-.68-.52-.01-.53.63-.01 1.08.58 1.23.82.72 1.21 1.87.87 2.33.66.07-.52.28-.87.51-1.07-1.78-.2-3.64-.89-3.64-3.95 0-.87.31-1.59.82-2.15-.08-.2-.36-1.02.08-2.12 0 0 .67-.21 2.2.82.64-.18 1.32-.27 2-.27.68 0 1.36.09 2 .27 1.53-1.04 2.2-.82 2.2-.82.44 1.1.16 1.92.08 2.12.51.56.82 1.27.82 2.15 0 3.07-1.87 3.75-3.65 3.95.29.25.54.73.54 1.48 0 1.07-.01 1.93-.01 2.2 0 .21.15.46.55.38A8.013 8.013 0 0016 8c0-4.42-3.58-8-8-8z"></path>
                                        </svg>
                                        <span>Ing. Andrés Luna</span>
                                        <span class="material-icons-round" style="font-size: 13px;">open_in_new</span>
                                    </a>
                                    <span style="font-size: 11px; color: var(--text-muted); margin-left: 4px;">(Desarrollador Principal & Arquitectura)</span>
                                </div>
                                <div>
                                    <strong style="color: var(--text-heading);">Cristian</strong>
                                    <span style="font-size: 11px; color: var(--text-muted); margin-left: 4px;">(Co-desarrollador de Software & Fenotipado)</span>
                                </div>
                            </div>
                        </td>
                    </tr>
                    <tr>
                        <td style="font-weight: 700; color: var(--text-muted);">Soporte Técnico & Reportes:</td>
                        <td>
                            <div style="display: flex; align-items: center; gap: 8px; flex-wrap: wrap;">
                                <a href="https://wa.me/573128919009?text=Hola%20Andr%C3%A9s,%20te%20contacto%20desde%20el%20sistema%20SIFMA%20por%20una%20consulta%20t%C3%A9cnica." target="_blank" rel="noopener noreferrer" style="color: #ffffff; background: #059669; font-weight: 700; text-decoration: none; display: inline-flex; align-items: center; gap: 6px; padding: 4px 10px; border-radius: 4px;
```

```
border: 1px solid #047857; font-size: 11.5px; transition: background 0.15s ease;" onmouseover="this.style.background='#047857';"
onmouseout="this.style.background='#059669';">

                                    <svg height="13" width="13" viewBox="0 0 24 24" fill="currentColor">

                                        <path d="M.057 2411.687-6.163c-1.041-1.804-1.588-3.849-1.587-5.946.003-6.556 5.338-11.891 11.893-11.891 3.181.001 6.167 1.24
8.413 3.488 2.245 2.248 3.481 5.236 3.48 8.414-.003 6.557-5.338 11.892-11.893 11.892-1.99-.001-3.951-.5-5.688-1.448l-6.305 1.654zm6.597-3.807c1.676.995 3.276 1.591 5.392
1.592 5.448 0 9.886-4.434 9.889-9.885.002-5.462-4.415-9.89-9.881-9.892-5.452 0-9.887 4.434-9.889 9.884-.001 2.225.651 3.891 1.746 5.634l-.999 3.648 3.742-.981zm11.387-
5.464c-.074-.124-.272-.198-.57-.347-.297-.149-1.758-.868-2.031-.967-.272-.099-.47-.149-.669.149-.198.297-.768.967-.941 1.165-.173.198-.347.223-.644.074-.297-.149-
1.255-.462-2.39-1.475-.883-.788-1.48-1.761-1.653-
2.059-.173-.297-.018-.458.13-.606.134-.133.297-.347.446-.521.151-.172.2-.296.3-.495.099-.198.05-.372-.025-.521-.075-.148-.669-1.611-.916-
2.206-.242-.579-.487-.501-.669-.511-.57-.01c-.198 0-.52.074-.792.372s-1.04 1.016-1.04 2.479 1.065 2.876 1.213 3.074c.149.198 2.095 3.2 5.076 4.487.709.306 1.263.489
1.694.626.712.226 1.36.194 1.872.118.571-.085 1.758-.719 2.006-1.413.248-.695.248-1.29.173-1.414z"/>

                                    </svg>

                                    <span>WhatsApp Directo: +57 312 891 9009</span>

                                </a>

                                <span style="font-size: 11px; color: var(--text-muted);">(Ing. Andrés Luna - Soporte & Calibración)</span>

                            </div>

                        </td>

                    </tr>

                    <tr>

                        <td style="font-weight: 700; color: var(--text-muted);">Líneas de Investigación:</td>

                        <td>Fenotipado Digital no destructivo, Aeroponía e Hidroponía Vertical, Visión por Computador aplicada a la Agricultura de Precisión y Telemetría
Microclimática IoT.</td>

                    </tr>

                    <tr>

                        <td style="font-weight: 700; color: var(--text-muted);">Enfoque Metodológico:</td>

                        <td>Cuantificación morfométrica multivariable (Área Foliar, Altura de Planta, Diámetro Basal del Tallo e Índice de Salud) correlacionada mediante
matrices de Pearson con el microclima.</td>

                    </tr>

                </tbody>

            </table>

        </div>


        <!-- TARJETA 2: LICENCIA DE USO EXCLUSIVO -->

        <div class="card" style="border-radius: 14px; box-shadow: var(--shadow-sm);">

            <div class="card-header">

                <div class="card-header-title">

                    <span class="material-icons-round text-primary" style="font-size: 20px;">lock</span>

                    <span>Términos de Licencia & Uso Exclusivo</span>

                </div>

                <span class="badge badge-warning" style="background: rgba(245, 158, 11, 0.12); color: #d97706; border: 1px solid rgba(245, 158, 11, 0.25);">Licencia
Propietaria Cerrada</span>

            </div>


            <div style="background: var(--bg-card-subtle); border: 1px solid var(--border-color); border-radius: 10px; padding: 16px; font-size: 12px; line-height: 1.6;
color: var(--text-body);">

                <p style="margin-bottom: 10px; font-weight: 700; color: var(--text-heading);">

                    Software Propietario Dedicado - Torre Aeropónica Vertical SIFMA

                </p>

                <p style="margin-bottom: 10px; color: var(--text-muted);">

                    El presente software, algoritmos de visión computacional, modelos de correlación fenotípica y arquitectura de telemetría están diseñados
exclusivamente para su operación y despliegue sobre la infraestructura de la <strong>Torre Aeropónica SIFMA</strong>.

                </p>

                <p style="margin-bottom: 10px; color: var(--text-muted);">

                    Queda estrictamente prohibida su redistribución, copia no autorizada, descompilación o uso en plataformas de terceros sin el consentimiento expreso
de los autores y directores del proyecto.

                </p>

                <div style="font-size: 11px; color: var(--text-muted); border-top: 1px dashed var(--border-color); padding-top: 8px; margin-top: 10px;">

                    &copy; 2026 SIFMA Project. Todos los derechos reservados. Uso exclusivo para investigación y operación de la Torre.

                </div>

            </div>

        </div>
```

```
    </div>

    <!-- GRID DE ARQUITECTURA Y HARDWARE -->
    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(360px, 1fr)); gap: 20px; margin-bottom: 24px;">

        <!-- TARJETA 3: ESPECIFICACIONES DE HARDWARE Y TOPOLOGÍA -->
        <div class="card" style="border-radius: 14px; box-shadow: var(--shadow-sm);">
            <div class="card-header">
                <div class="card-header-title">
                    <span class="material-icons-round text-primary" style="font-size: 20px;">memory</span>
                    <span>Topología de Hardware, Sensores & Visión</span>
                </div>
            </div>

            <div style="display: flex; flex-direction: column; gap: 14px; font-size: 12px;">
                <div style="display: flex; gap: 10px; align-items: flex-start;">
                    <span class="material-icons-round text-muted" style="font-size: 18px; margin-top: 2px;">dns</span>
                    <div>
                        <strong style="color: var(--text-heading);">Estructura Aeropónica Vertical:</strong>
                        <div style="color: var(--text-muted);">Torre vertical modular con canastillas distribuidas para cultivo en aeroponía/hidroponía, equipada con sistema de aspersión y recirculación de solución nutritiva controlada.</div>
                    </div>
                </div>

                <div style="display: flex; gap: 10px; align-items: flex-start;">
                    <span class="material-icons-round text-muted" style="font-size: 18px; margin-top: 2px;">photo_camera</span>
                    <div>
                        <strong style="color: var(--text-heading);">Sistema de Visión Fenotípica Dual:</strong>
                        <div style="color: var(--text-muted);">Cámara Cenital superior calibrada para segmentación del canopia vegetal (área foliar en cm² y verdor) y Cámara Lateral para estimación de altura y diámetro del tallo.</div>
                    </div>
                </div>

                <div style="display: flex; gap: 10px; align-items: flex-start;">
                    <span class="material-icons-round text-muted" style="font-size: 18px; margin-top: 2px;">sensors</span>
                    <div>
                        <strong style="color: var(--text-heading);">Matriz de Telemetría Fisicoquímica y Ambiental:</strong>
                        <div style="color: var(--text-muted);">Monitoreo continuo de solución (pH, Conductividad Eléctrica &amp;mu;S/cm, Turbidez NTU, Oxígeno Disuelto mg/L, Temperatura del Agua), Microclima ambiental (Temperatura, Humedad, Radiación Solar UV) y Corriente de Bombeo (ACS712).</div>
                    </div>
                </div>

                <div style="display: flex; gap: 10px; align-items: flex-start;">
                    <span class="material-icons-round text-muted" style="font-size: 18px; margin-top: 2px;">settings_input_antenna</span>
                    <div>
                        <strong style="color: var(--text-heading);">Enlace de Adquisición y Sincronización:</strong>
                        <div style="color: var(--text-muted);">Transmisión inalámbrica por radiofrecuencia (XBee / RF) con enlace serial USB hacia el servidor local SIFMA e importador universal CSV por jornadas horarias (07:00, 12:00, 17:00).</div>
                    </div>
                </div>
            </div>
        </div>

        <!-- TARJETA 4: STACK TECNOLÓGICO Y SOFTWARE -->
        <div class="card" style="border-radius: 14px; box-shadow: var(--shadow-sm);">
            <div class="card-header">
                <div class="card-header-title">
```

```
                    <span class="material-icons-round text-primary" style="font-size: 20px;">layers</span>
                    <span>Arquitectura de Software y Stack</span>
                </div>
            </div>

            <table class="table" style="font-size: 12px;">
                <tbody>
                    <tr>
                        <td style="font-weight: 700; width: 35%; color: var(--text-muted);">Patrón Arquitectural:</td>
                        <td>Clean Architecture (Domain, Application, Infrastructure, Presentation)</td>
                    </tr>
                    <tr>
                        <td style="font-weight: 700; color: var(--text-muted);">Servidor Web:</td>
                        <td>Python 3.10+ / Flask 3.0 / Werkzeug / Jinja2</td>
                    </tr>
                    <tr>
                        <td style="font-weight: 700; color: var(--text-muted);">Visión Computacional:</td>
                        <td>OpenCV (<code>cv2</code>) 4.8+, NumPy, Segmentación HSV + CIELAB + Filtros Morfológicos y Umbralización Adaptativa</td>
                    </tr>
                    <tr>
                        <td style="font-weight: 700; color: var(--text-muted);">Base de Datos:</td>
                        <td>SQLite3 con SQLAlchemy ORM y capa de repositorios transaccionales</td>
                    </tr>
                    <tr>
                        <td style="font-weight: 700; color: var(--text-muted);">Visualización & UI:</td>
                        <td>Chart.js, Vanilla CSS Moderno, Heatmaps multivariables de Pearson y CSS Paged Media para Dossiers PDF</td>
                    </tr>
                </tbody>
            </table>
        </div>

    </div>

    <!-- TARJETA 5: REGISTRO DE VERSIONES (CHANGELOG) -->
    <div class="card" style="border-radius: 14px; box-shadow: var(--shadow-sm);">
        <div class="card-header">
            <div class="card-header-title">
                <span class="material-icons-round text-primary" style="font-size: 20px;">history</span>
                <span>Historial de Versiones y Novedades</span>
            </div>
        </div>

        <div style="display: flex; flex-direction: column; gap: 16px;">

            <!-- VERSIÓN 1.2 -->
            <div style="border-left: 3.5px solid var(--accent-emerald); padding-left: 16px;">
                <div style="display: flex; align-items: center; gap: 10px; margin-bottom: 4px; flex-wrap: wrap;">
                    <strong style="font-size: 13.5px; color: var(--text-heading);">Versión 1.2 (Versión Actual)</strong>
                    <span class="badge badge-success">Optimización & UX Integral</span>
                    <span style="font-size: 11.5px; color: var(--text-muted);">Octubre 2026</span>
                </div>
                <ul style="margin: 6px 0 0 18px; font-size: 12px; color: var(--text-muted); line-height: 1.6;">
                    <li><strong>Rediseño Dashboard Split Lateral</strong>: Nueva distribución en dos columnas con 5 KPIs verticales extendidos e integración directa de 4 gráficas analíticas sin scroll redundante.</li>
                    <li><strong>Calendario Integrado de Página Única</strong>: Vista mensual adaptativa con panel lateral de métricas agregadas y navegación sincronizada por jornadas.</li>
```

```
                    <li><strong>Ventana Modal de Onboarding y Guía Rápida</strong>: Módulo de bienvenida con backdrop blur y presentación de pasos operativos con el emblema institucional SIFMA.</li>
                    <li><strong>Navegación Interactiva por Logotipo</strong>: Barra lateral colapsable mediante interacción directa sobre el logotipo institucional SIFMA sin botones adicionales.</li>
                    <li><strong>Depuración y Limpieza Visual</strong>: Transición global a bordes sutiles modernos (4px-6px), paleta blanca limpia y eliminación de módulos obsoletos de benchmark.</li>
                </ul>
            </div>

            <!-- VERSIÓN 1.1 -->
            <div style="border-left: 3.5px solid var(--border-color); padding-left: 16px;">
                <div style="display: flex; align-items: center; gap: 10px; margin-bottom: 4px; flex-wrap: wrap;">
                    <strong style="font-size: 13.5px; color: var(--text-heading);">Versión 1.1</strong>
                    <span class="badge" style="background: #f1f5f9; color: #475569;">Producción Científica</span>
                    <span style="font-size: 11.5px; color: var(--text-muted);">Septiembre 2026</span>
                </div>
                <ul style="margin: 6px 0 0 18px; font-size: 12px; color: var(--text-muted); line-height: 1.6;">
                    <li><strong>Generador de Reportes Científicos en PDF (<code>/report/scientific</code>)</strong>: Emisión de dossiers técnicos con matrices estadísticas completas (&amp;mu;, &amp;sigma;, s&amp;sup2;, IQR, CV%), correlaciones de Pearson e imágenes ajustadas proporcionalmente.</li>
                    <li><strong>Suite de Firma Digital Integrada</strong>: Trazador táctil/mouse en canvas y subida de imágenes con remoción automática de fondo blanco y persistencia en navegador.</li>
                    <li><strong>Time-Lapse Interactivo (<code>/timelapse</code>)</strong>: Reproductor de evolución biológica con vista dual, gauges en vivo y cinta de miniaturas.</li>
                    <li><strong>Matriz de Correlación Multivariable de Pearson (<code>/cross_analysis</code>)</strong>: Cruce cruzado entre variables de microclima y biometría vegetal con selector de franjas horarias.</li>
                </ul>
            </div>

            <!-- VERSIÓN 1.0 -->
            <div style="border-left: 3.5px solid var(--border-color); padding-left: 16px;">
                <div style="display: flex; align-items: center; gap: 10px; margin-bottom: 4px; flex-wrap: wrap;">
                    <strong style="font-size: 13.5px; color: var(--text-heading);">Versión 1.0</strong>
                    <span class="badge" style="background: #f1f5f9; color: #475569;">Lanzamiento Base</span>
                    <span style="font-size: 11.5px; color: var(--text-muted);">Agosto 2026</span>
                </div>
                <ul style="margin: 6px 0 0 18px; font-size: 12px; color: var(--text-muted); line-height: 1.6;">
                    <li>Pipeline básico de segmentación cromática HSV y cálculo de área foliar preliminar.</li>
                    <li>Adquisición de telemetría física de sensores mediante enlace serial USB.</li>
                    <li>Base de datos relacional inicial SQLite y panel de monitoreo agronómico básico.</li>
                </ul>
            </div>

        </div>
    </div>

</div>
{% endblock %}
```

ARCHIVO 60/85: local_server/templates/audit_logs.html (317 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Registro de Auditoría y Logs del Sistema{% endblock %}
{% block heading %}Registro de Actividad y Logs de Auditoría{% endblock %}
{% block subheading %}Trazabilidad cronológica de movimientos, modificaciones de cuentas, configuración y operaciones críticas{% endblock %}

{% block header_actions %}
<div style="display: flex; gap: 8px;">
    <a href="{{ url_for('dashboard.export_audit_logs', search=search_query, module=module_filter, level=level_filter, date=date_filter) }}" class="btn btn-outline btn-sm" title="Descargar registro de actividad en CSV">
        <span class="material-icons-round">file_download</span>
        <span>Exportar CSV</span>
    </a>
    <button class="btn btn-danger btn-sm" onclick="handleClearAllLogs()" title="Vaciar todo el historial de registros de logs">
        <span class="material-icons-round">delete_sweep</span>
        <span>Vaciar Logs</span>
    </button>
    <button class="btn btn-primary btn-sm" onclick="window.location.reload()" title="Refrescar registros">
        <span class="material-icons-round">refresh</span>
        <span>Actualizar</span>
    </button>
</div>
{% endblock %}

{% block content %}
<div style="display: flex; flex-direction: column; gap: 20px;">

    <!-- 1. TARJETAS KPI DE ACTIVIDAD -->
    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 14px;">

        <div style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: 12px; padding: 16px; display: flex; align-items: center; gap: 14px; box-shadow: 0 2px 8px rgba(0,0,0,0.02);">
            <div style="width: 44px; height: 44px; border-radius: 10px; background: rgba(5, 150, 105, 0.1); color: var(--accent-success); display: flex; align-items: center; justify-content: center;">
                <span class="material-icons-round" style="font-size: 24px;">receipt_long</span>
            </div>
            <div>
                <div style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; letter-spacing: 0.5px;">Total Movimientos</div>
                <div style="font-size: 22px; font-weight: 800; color: var(--text-heading); line-height: 1.2;" id="kpi-total-logs">{{ stats.total_logs }}</div>
            </div>
        </div>

        <div style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: 12px; padding: 16px; display: flex; align-items: center; gap: 14px; box-shadow: 0 2px 8px rgba(0,0,0,0.02);">
            <div style="width: 44px; height: 44px; border-radius: 10px; background: rgba(2, 132, 199, 0.1); color: #0284c7; display: flex; align-items: center; justify-content: center;">
                <span class="material-icons-round" style="font-size: 24px;">today</span>
            </div>
            <div>
```

```
                <div style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; letter-spacing: 0.5px;">Actividad Hoy</div>
                <div style="font-size: 22px; font-weight: 800; color: var(--text-heading); line-height: 1.2;">{{ stats.today_logs }}</div>
            </div>
        </div>

        <div style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: 12px; padding: 16px; display: flex; align-items: center; gap: 14px; box-shadow: 0 2px 8px rgba(0,0,0,0.02);">
            <div style="width: 44px; height: 44px; border-radius: 10px; background: rgba(225, 29, 72, 0.1); color: var(--accent-danger); display: flex; align-items: center; justify-content: center;">
                <span class="material-icons-round" style="font-size: 24px;">warning_amber</span>
            </div>
            <div>
                <div style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; letter-spacing: 0.5px;">Acciones Críticas</div>
                <div style="font-size: 22px; font-weight: 800; color: var(--text-heading); line-height: 1.2;">{{ stats.critical_logs }}</div>
            </div>
        </div>

        <div style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: 12px; padding: 16px; display: flex; align-items: center; gap: 14px; box-shadow: 0 2px 8px rgba(0,0,0,0.02);">
            <div style="width: 44px; height: 44px; border-radius: 10px; background: rgba(124, 58, 237, 0.1); color: #7c3aed; display: flex; align-items: center; justify-content: center;">
                <span class="material-icons-round" style="font-size: 24px;">people</span>
            </div>
            <div>
                <div style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; letter-spacing: 0.5px;">Usuarios Activos Hoy</div>
                <div style="font-size: 22px; font-weight: 800; color: var(--text-heading); line-height: 1.2;">{{ stats.active_users_today }}</div>
            </div>
        </div>

    </div>

    <!-- 2. BARRA DE BÚSQUEDA Y FILTROS -->
    <div style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: 12px; padding: 16px 20px; box-shadow: 0 2px 8px rgba(0,0,0,0.02);">
        <form method="GET" action="{{ url_for('dashboard.audit_logs') }}" style="display: flex; flex-wrap: wrap; gap: 12px; align-items: flex-end;">

            <!-- Búsqueda de texto -->
            <div style="flex: 1; min-width: 220px;">
                <label style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); margin-bottom: 5px; display: block;">Buscar en Registros</label>
                <div style="position: relative;">
                    <span class="material-icons-round" style="position: absolute; left: 10px; top: 50%; transform: translateY(-50%); font-size: 18px; color: var(--text-muted);">search</span>
                    <input type="text" name="search" value="{{ search_query }}" placeholder="Buscar por usuario, acción, detalle o IP..." class="form-input-control" style="padding-left: 34px; font-size: 12.5px;">
                </div>
            </div>

            <!-- Filtro por Módulo -->
            <div style="min-width: 160px;">
                <label style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); margin-bottom: 5px; display: block;">Módulo</label>
                <select name="module" class="form-input-control" style="font-size: 12.5px;">
                    <option value="">Todos los Módulos</option>
                    {% for mod in available_modules %}
                    <option value="{{ mod }}" {% if module_filter == mod %}selected{% endif %}>{{ mod }}</option>
                    {% endfor %}
                </select>
            </div>

            <!-- Filtro por Nivel -->
```

```
            <div style="min-width: 160px;">
                <label style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); margin-bottom: 5px; display: block;">Severidad / Tipo</label>
                <select name="level" class="form-input-control" style="font-size: 12.5px;">
                    <option value="">Todos los Tipos</option>
                    {% for lvl_val, lvl_lbl in available_levels %}
                    <option value="{{ lvl_val }}" {% if level_filter == lvl_val %}selected{% endif %}>{{ lvl_lbl }}</option>
                    {% endfor %}
                </select>
            </div>

            <!-- Filtro por Fecha -->
            <div style="min-width: 150px;">
                <label style="font-size: 11.5px; font-weight: 700; color: var(--text-muted); margin-bottom: 5px; display: block;">Fecha Específica</label>
                <input type="date" name="date" value="{{ date_filter }}" class="form-input-control" style="font-size: 12.5px;">
            </div>

            <!-- Botones Filtrar y Limpiar -->
            <div style="display: flex; gap: 8px;">
                <button type="submit" class="btn btn-primary btn-sm" style="height: 38px;">
                    <span class="material-icons-round" style="font-size: 18px;">filter_alt</span>
                    <span>Filtrar</span>
                </button>
                {% if search_query or module_filter or level_filter or date_filter %}
                <a href="{{ url_for('dashboard.audit_logs') }}" class="btn btn-outline btn-sm" style="height: 38px;" title="Limpiar todos los filtros">
                    <span class="material-icons-round" style="font-size: 18px;">clear_all</span>
                    <span>Limpiar</span>
                </a>
                {% endif %}
            </div>

        </form>
    </div>

    <!-- 3. TABLA DE REGISTRO DE AUDITORÍA -->
    <div class="users-table-container" style="background: var(--bg-card); border: 1px solid var(--border-color); border-radius: 12px; overflow: hidden; box-shadow: 0 2px 8px rgba(0,0,0,0.02);">
        <table class="users-table" style="width: 100%; border-collapse: collapse;">
            <thead>
                <tr style="background: var(--bg-card-subtle); border-bottom: 1px solid var(--border-color);">
                    <th style="padding: 12px 16px; font-size: 11.5px; font-weight: 800; text-transform: uppercase; color: var(--text-muted); width: 140px;">Fecha y Hora</th>
                    <th style="padding: 12px 16px; font-size: 11.5px; font-weight: 800; text-transform: uppercase; color: var(--text-muted); width: 210px;">Usuario Responsable</th>
                    <th style="padding: 12px 16px; font-size: 11.5px; font-weight: 800; text-transform: uppercase; color: var(--text-muted); width: 120px;">Módulo</th>
                    <th style="padding: 12px 16px; font-size: 11.5px; font-weight: 800; text-transform: uppercase; color: var(--text-muted); width: 170px;">Acción Realizada</th>
                    <th style="padding: 12px 16px; font-size: 11.5px; font-weight: 800; text-transform: uppercase; color: var(--text-muted);">Descripción / Movimiento</th>
                    <th style="padding: 12px 16px; font-size: 11.5px; font-weight: 800; text-transform: uppercase; color: var(--text-muted); text-align: right; width: 100px;">Origen (IP)</th>
                    <th style="padding: 12px 16px; font-size: 11.5px; font-weight: 800; text-transform: uppercase; color: var(--text-muted); text-align: right; width: 60px;">Acción</th>
                </tr>
            </thead>
            <tbody id="audit-logs-tbody">
                {% if logs and logs|length > 0 %}
                    {% for l in logs %}
                    <tr id="log-row-{{ l.id }}" style="border-bottom: 1px solid var(--border-color); transition: all 0.2s ease;" onmouseover="this.style.background='var(--bg-card-subtle)'" onmouseout="this.style.background='transparent'">
```

```
                        <!-- Fecha y Hora -->
                        <td style="padding: 12px 16px; vertical-align: middle;">
                            <div style="display: flex; align-items: center; gap: 6px;">
                                <span class="material-icons-round" style="font-size: 16px; color: var(--text-muted);">schedule</span>
                                <div>
                                    <div style="font-weight: 700; color: var(--text-heading); font-size: 12.5px;">{{ l.timestamp.strftime('%d/%m/%Y') }}</div>
                                    <div style="font-size: 11px; color: var(--text-muted); font-family: monospace;">{{ l.timestamp.strftime('%H:%M:%S') }}</div>
                                </div>
                            </div>
                        </td>

                        <!-- Usuario -->
                        <td style="padding: 12px 16px; vertical-align: middle;">
                            <div style="display: flex; align-items: center; gap: 8px;">
                                {% if l.user and l.user.avatar_url %}
                                <img src="{{ l.user.avatar_url }}" alt="{{ l.user_fullname or l.username }}" class="user-avatar-circle" style="width: 32px; height: 32px;
object-fit: cover; border: 1px solid var(--border-color); flex-shrink: 0;">
                                {% else %}
                                <div class="user-avatar-circle" style="width: 32px; height: 32px; font-size: 11px; background: {% if l.user_role == 'admin' or l.username
== 'admin' %}linear-gradient(135deg, #059669, #10b981){% else %}var(--border-color){% endif %}; color: #ffffff; flex-shrink: 0;">
                                    {{ l.user_fullname[:2].upper() if l.user_fullname else l.username[:2].upper() }}
                                </div>
                                {% endif %}
                                <div style="min-width: 0;">
                                    <div style="font-weight: 700; color: var(--text-heading); font-size: 12.5px; white-space: nowrap; overflow: hidden; text-overflow:
ellipsis;">
                                        {{ l.user_fullname or l.username }}
                                    </div>
                                    <div style="font-size: 11px; color: var(--text-muted); display: flex; align-items: center; gap: 4px;">
                                        <span>@{{ l.username }}</span>
                                        {% if l.user_role %}
                                        <span class="badge {% if l.user_role == 'admin' %}badge-primary{% elif l.user_role == 'investigador' %}badge-success{% else
%}badge-info{% endif %}" style="font-size: 9px; padding: 1px 4px;">
                                            {{ l.user_role }}
                                        </span>
                                        {% endif %}
                                    </div>
                                </div>
                            </div>
                        </td>

                        <!-- Módulo -->
                        <td style="padding: 12px 16px; vertical-align: middle;">
                            <span class="badge badge-info" style="font-size: 10.5px; font-weight: 700; padding: 3px 7px; text-transform: uppercase; letter-spacing:
0.4px;">
                                {{ l.module }}
                            </span>
                        </td>

                        <!-- Acción -->
                        <td style="padding: 12px 16px; vertical-align: middle;">
                            <span class="badge {{ l.level_badge_class }}" style="font-size: 11px; padding: 3px 8px; display: inline-flex; align-items: center; gap: 4px;
font-weight: 700;">
                                <span class="material-icons-round" style="font-size: 14px;">{{ l.action_icon }}</span>
                                <span>{{ l.action_display or l.action }}</span>
                            </span>
                        </td>
```

```
                        <!-- Descripción -->
                        <td style="padding: 12px 16px; vertical-align: middle;">
                            <div style="font-size: 12.5px; color: var(--text-heading); line-height: 1.4;">
                                {{ l.description }}
                            </div>
                        </td>

                        <!-- Origen / IP -->
                        <td style="padding: 12px 16px; vertical-align: middle; text-align: right;">
                            <span style="font-size: 11.5px; font-family: monospace; color: var(--text-muted); background: var(--bg-card-subtle); padding: 2px 6px; border-radius: 4px; border: 1px solid var(--border-color);">
                                {{ l.ip_address or '127.0.0.1' }}
                            </span>
                        </td>

                        <!-- Acción Individual: Eliminar Log -->
                        <td style="padding: 12px 16px; vertical-align: middle; text-align: right;">
                            <button class="btn btn-outline btn-sm" style="padding: 4px 6px; font-size: 11px; color: var(--accent-danger); border-color: transparent;" onclick="handleDeleteSingleLog({{ l.id }})" title="Eliminar este registro individual">
                                <span class="material-icons-round" style="font-size: 16px;">delete</span>
                            </button>
                        </td>

                    </tr>
                    {% endfor %}
                {% else %}
                    <tr id="empty-logs-row">
                        <td colspan="7" style="text-align: center; padding: 48px 20px;">
                            <div style="max-width: 320px; margin: 0 auto; display: flex; flex-direction: column; align-items: center; gap: 10px;">
                                <span class="material-icons-round" style="font-size: 48px; color: var(--text-muted); opacity: 0.5;">find_in_page</span>
                                <div style="font-size: 15px; font-weight: 700; color: var(--text-heading);">No se encontraron registros de logs</div>
                                <div style="font-size: 12px; color: var(--text-muted);">No hay acciones registradas o el historial ha sido vaciado.</div>
                                {% if search_query or module_filter or level_filter or date_filter %}
                                <a href="{{ url_for('dashboard.audit_logs') }}" class="btn btn-outline btn-sm" style="margin-top: 6px;">
                                    <span class="material-icons-round">clear_all</span>
                                    <span>Ver Todos los Registros</span>
                                </a>
                                {% endif %}
                            </div>
                        </td>
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>

</div>
{% endblock %}

{% block scripts %}
<script>
async function handleDeleteSingleLog(logId) {
    if (!confirm(`¿Estás seguro de que deseas eliminar este registro de auditoría (#${logId})?`)) return;

    try {
```

```
        const res = await fetch(`/api/audit-logs/${logId}`, { method: 'DELETE' });
        const data = await res.json();
        if (data.status === 'success') {
            const row = document.getElementById(`log-row-${logId}`);
            if (row) {
                row.style.opacity = '0';
                row.style.transform = 'scale(0.95)';
                setTimeout(() => {
                    row.remove();
                    const tbody = document.getElementById('audit-logs-tbody');
                    if (tbody && tbody.querySelectorAll('tr[id^="log-row-"]').length === 0) {
                        window.location.reload();
                    }
                }, 200);
            }
            if (window.showToast) {
                window.showToast({
                    type: 'info',
                    title: 'Registro Eliminado',
                    message: data.message
                });
            }
        } else {
            alert('Error: ' + data.message);
        }
    } catch (err) {
        alert('Error al eliminar registro de log.');
    }
}

async function handleClearAllLogs() {
    if (!confirm('¿Está seguro de que desea VACIAR TODO EL HISTORIAL de auditoría?\n\nEsta acción eliminará de forma irreversible todos los registros de movimientos almacenados.')) {
        return;
    }

    try {
        const res = await fetch('/api/audit-logs/clear', { method: 'POST' });
        const data = await res.json();
        if (data.status === 'success') {
            if (window.showToast) {
                window.showToast({
                    type: 'warning',
                    title: 'Historial Vaciado',
                    message: data.message
                });
            } else {
                alert(data.message);
            }
            setTimeout(() => window.location.reload(), 700);
        } else {
            alert('Error: ' + data.message);
        }
    } catch (err) {
        alert('Error al vaciar los logs de auditoría.');
    }
```

```
315 }
316 </script>
317 {% endblock %}
```

ARCHIVO 61/85: local_server/templates/base.html (602 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=5.0">
    <meta name="description" content="SIFMA: Sistema Integrado de Fenotipado y Monitoreo Agronómico.">
    <title>SIFMA - {% block title %}Dashboard Analítico{% endblock %}</title>

    <!-- Favicon Oficial SIFMA -->
    <link rel="icon" type="image/svg+xml" href="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}">

    <!-- Google Fonts: Philosopher (marca y títulos) + Literata (cuerpo y datos) -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Literata:ital,opsz,wght@0,7..72,400;0,7..72,500;0,7..72,600;0,7..72,700;1,7..72,400;1,7..72,600&family=Philosopher:ital,wght@0,400;0,700;1,400;1,700&display=swap" rel="stylesheet">

    <!-- Google Material Icons Round -->
    <link href="https://fonts.googleapis.com/icon?family=Material+Icons+Round" rel="stylesheet">

    <!-- Chart.js para gráficas fluidas -->
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

    <!-- Estilos Ejecutivos Personalizados -->
    <link rel="stylesheet" href="{{ url_for('static', filename='css/index.css') }}">

    {% block head %}{% endblock %}
</head>
<body>
    <div class="app-container">
        <!-- OVERLAY PARA DISPOSITIVOS MÓVILES -->
        <div class="sidebar-overlay" id="sidebar-overlay" onclick="toggleSidebar()"></div>

        <!-- BARRA LATERAL (SIDEBAR VERDE BOSQUE PROFUNDO) -->
        <aside class="sidebar" id="sidebar">
            <div class="sidebar-header">
                <!-- LOGO Y TÍTULO SIFMA (ACCIONA COLAPSAR / EXPANDIR) -->
                <div class="sidebar-brand-group" id="sidebar-toggle" onclick="toggleSidebar()" title="Plegar / Desplegar Barra Lateral">
                    <img src="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}" alt="SIFMA Emblem" class="sidebar-brand-logo">
                    <div class="sidebar-brand-text">
                        <h2>SIFMA</h2>
                        <span>Fenotipado & Telemetría</span>
                    </div>
                </div>
            </div>

            <div class="sidebar-scrollable">
```

```
                <!-- SECCIÓN: GENERAL -->
                {% if current_user.is_super_admin or current_user.permissions.dashboard or current_user.permissions.calendar %}
                <div>
                    <div class="nav-section-title">General</div>
                    <div class="nav-group">
                        {% if current_user.is_super_admin or current_user.permissions.dashboard %}
                        <a href="{{ url_for('dashboard.index') }}" class="nav-link {% if request.path == '/' %}active{% endif %}" id="nav-dashboard" title="Resumen Diario">
                            <span class="material-icons-round">dashboard</span>
                            <span class="nav-link-text">Resumen Diario</span>
                        </a>
                        {% endif %}
                        {% if current_user.is_super_admin or current_user.permissions.calendar %}
                        <a href="{{ url_for('dashboard.calendar_view') }}" class="nav-link {% if '/calendar' in request.path %}active{% endif %}" id="nav-calendar" title="Calendario de Muestreos y Telemetría">
                            <span class="material-icons-round">calendar_month</span>
                            <span class="nav-link-text">Calendario</span>
                        </a>
                        {% endif %}
                    </div>
                </div>
                {% endif %}

                <!-- SECCIÓN: FENOTIPADO Y VISIÓN -->
                {% if current_user.is_super_admin or current_user.permissions.processing or current_user.permissions.gallery or current_user.permissions.timelapse %}
                <div>
                    <div class="nav-section-title">Fenotipado y Visión</div>
                    <div class="nav-group">
                        {% if current_user.is_super_admin or current_user.permissions.processing %}
                        <a href="{{ url_for('dashboard.processing_panel') }}" class="nav-link {% if '/processing' in request.path %}active{% endif %}" id="nav-processing" title="Procesamiento">
                            <span class="material-icons-round">camera_enhance</span>
                            <span class="nav-link-text">Procesamiento</span>
                        </a>
                        {% endif %}
                        {% if current_user.is_super_admin or current_user.permissions.gallery %}
                        <a href="{{ url_for('dashboard.gallery') }}" class="nav-link {% if '/gallery' in request.path %}active{% endif %}" id="nav-gallery" title="Galería de Lotes">
                            <span class="material-icons-round">photo_library</span>
                            <span class="nav-link-text">Galería de Lotes</span>
                        </a>
                        {% endif %}
                        {% if current_user.is_super_admin or current_user.permissions.timelapse %}
                        <a href="{{ url_for('dashboard.timelapse_viewer') }}" class="nav-link {% if '/timelapse' in request.path %}active{% endif %}" id="nav-timelapse" title="Time-Lapse Evolutivo">
                            <span class="material-icons-round">slow_motion_video</span>
                            <span class="nav-link-text">Time-Lapse Evolutivo</span>
                        </a>
                        {% endif %}
                    </div>
                </div>
                {% endif %}

                <!-- SECCIÓN: TELEMETRÍA Y AMBIENTE -->
                {% if current_user.is_super_admin or current_user.permissions.sensors or current_user.permissions.cross_analysis or current_user.permissions.reports %}
                <div>
                    <div class="nav-section-title">Telemetría y Reportes</div>
```

```
                    <div class="nav-group">
                        {% if current_user.is_super_admin or current_user.permissions.sensors %}
                        <a href="{{ url_for('dashboard.detailed_sensors') }}" class="nav-link {% if '/sensors' in request.path %}active{% endif %}" id="nav-sensors" title="Sensores Ambientales">
                            <span class="material-icons-round">sensors</span>
                            <span class="nav-link-text">Sensores Ambientales</span>
                        </a>
                        {% endif %}
                        {% if current_user.is_super_admin or current_user.permissions.cross_analysis %}
                        <a href="{{ url_for('dashboard.cross_analysis') }}" class="nav-link {% if '/cross_analysis' in request.path %}active{% endif %}" id="nav-cross-analysis" title="Cruce y Correlaciones">
                            <span class="material-icons-round">sync_alt</span>
                            <span class="nav-link-text">Cruce y Correlaciones</span>
                        </a>
                        {% endif %}
                        {% if current_user.is_super_admin or current_user.permissions.reports %}
                        <a href="{{ url_for('dashboard.scientific_report_view') }}" class="nav-link {% if '/report' in request.path or '/dossier' in request.path or '/scientific_report' in request.path %}active{% endif %}" id="nav-reports" title="Dossier y Generador de Reportes">
                            <span class="material-icons-round">assessment</span>
                            <span class="nav-link-text">Dossier y Reportes</span>
                        </a>
                        {% endif %}
                    </div>
                </div>
                {% endif %}

                <!-- SECCIÓN: ADMINISTRACIÓN Y CONFIGURACIÓN -->
                <div>
                    <div class="nav-section-title">Configuración</div>
                    <div class="nav-group">
                        {% if current_user.is_super_admin or current_user.permissions.config %}
                        <a href="{{ url_for('dashboard.configuration_panel') }}" class="nav-link {% if '/config' in request.path %}active{% endif %}" id="nav-config" title="Parámetros de Cultivo">
                            <span class="material-icons-round">tune</span>
                            <span class="nav-link-text">Parámetros de Cultivo</span>
                        </a>
                        {% endif %}
                        {% if current_user.is_super_admin %}
                        <a href="{{ url_for('dashboard.users_management') }}" class="nav-link {% if '/users' in request.path %}active{% endif %}" id="nav-users" title="Usuarios y Roles">
                            <span class="material-icons-round">group</span>
                            <span class="nav-link-text">Usuarios y Roles</span>
                        </a>
                        <a href="{{ url_for('dashboard.audit_logs') }}" class="nav-link {% if '/audit' in request.path or '/logs' in request.path %}active{% endif %}" id="nav-audit-logs" title="Registro de Actividad y Logs">
                            <span class="material-icons-round">receipt_long</span>
                            <span class="nav-link-text">Registro de Logs</span>
                        </a>
                        {% endif %}
                        <a href="{{ url_for('dashboard.system_info') }}" class="nav-link {% if '/about' in request.path or '/system_info' in request.path %}active{% endif %}" id="nav-about" title="Información & Créditos">
                            <span class="material-icons-round">info</span>
                            <span class="nav-link-text">Información & Créditos</span>
                        </a>
                    </div>
                </div>
            </div>

            <!-- PIE DE PERFIL DE USUARIO / LOGO SIFMA INFERIOR -->
```

```
            <div class="sidebar-footer">
                <!-- WIDGET EXPANDIDO DE USUARIO -->
                <div class="user-profile-widget">
                    {% if current_user.avatar_url %}
                    <img src="{{ current_user.avatar_url }}" alt="{{ current_user.name }}" class="user-avatar-circle" style="object-fit: cover; border: 2px solid var(--accent-success);" title="{{ current_user.role_display }}">
                    {% else %}
                    <div class="user-avatar-circle" title="{{ current_user.role_display }}">
                        {{ current_user.avatar }}
                    </div>
                    {% endif %}
                    <div class="user-info-text">
                        <div class="user-name" title="{{ current_user.name }}">{{ current_user.name }}</div>
                        <div class="user-role">{{ current_user.role_display }}</div>
                    </div>
                    <a href="{{ url_for('dashboard.logout') }}" class="user-logout-btn" title="Cerrar Sesión">
                        <span class="material-icons-round">logout</span>
                    </a>
                </div>

                <!-- ELEMENTOS INFERIORES CUANDO ESTÁ COLAPSADO: AVATAR + LOGOUT -->
                <div class="sidebar-collapsed-bottom">
                    {% if current_user.avatar_url %}
                    <img src="{{ current_user.avatar_url }}" alt="{{ current_user.name }}" class="user-avatar-circle" style="object-fit: cover; border: 2px solid var(--accent-success);" title="{{ current_user.name }} ({{ current_user.role_display }})">
                    {% else %}
                    <div class="user-avatar-circle" title="{{ current_user.name }} ({{ current_user.role_display }})">
                        {{ current_user.avatar }}
                    </div>
                    {% endif %}
                    <a href="{{ url_for('dashboard.logout') }}" class="user-logout-btn" title="Cerrar Sesión ({{ current_user.name }})">
                        <span class="material-icons-round">logout</span>
                    </a>
                </div>
            </div>
        </aside>

        <!-- CONTENIDO PRINCIPAL -->
        <main class="main-content" id="main-content">
            <header class="top-header">
                <div class="header-left-col">
                    <!-- BOTÓN MÓVIL SOLO EN PANTALLAS PEQUEÑAS -->
                    <button class="mobile-sidebar-trigger" onclick="toggleSidebar()" title="Abrir Menú">
                        <svg xmlns="http://www.w3.org/2000/svg" width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round" class="lucide lucide-menu-icon lucide-menu"><path d="M4 5h16"/><path d="M4 12h16"/><path d="M4 19h16"/></svg>
                    </button>

                    <div class="header-title-block">
                        <h1 id="page-heading">{% block heading %}Dashboard Analítico{% endblock %}</h1>
                        <p id="page-subheading">{% block subheading %}Resumen general de la jornada y fenotipado{% endblock %}</p>
                    </div>
                </div>

                <div class="header-controls-row">
                    {% block header_controls %}
                    <!-- Selector de Canastilla / Torre -->
```

```
                    <div style="display: flex; align-items: center; gap: 6px;">
                        <span class="material-icons-round text-muted" style="font-size: 18px;">view_in_ar</span>
                        <select id="header-plant-selector" class="form-input-control" style="padding: 6px 10px; font-weight: 600; font-size: 13px; cursor: pointer; width: auto;" onchange="switchActivePlant(this.value)">
                            <option value="1" {% if active_plant_id == 1 %}selected{% endif %}>Canastilla #1</option>
                            <option value="2" {% if active_plant_id == 2 %}selected{% endif %}>Canastilla #2</option>
                            <option value="3" {% if active_plant_id == 3 %}selected{% endif %}>Canastilla #3</option>
                            <option value="4" {% if active_plant_id == 4 %}selected{% endif %}>Canastilla #4</option>
                        </select>
                    </div>
                    {% endblock %}

                    {% block header_actions %}{% endblock %}
                </div>
            </header>

            <div class="content-viewport">
                {% block content %}{% endblock %}
            </div>
        </main>
    </div>

    <!-- MODAL DE BIENVENIDA E INSTRUCCIONES INICIALES (ONBOARDING) -->
    <div id="sifma-onboarding-modal" class="onboarding-modal-backdrop" style="display: none;" onclick="closeOnboardingOnBackdrop(event)">
        <div class="onboarding-dialog-card">
            <!-- Header Centrado -->
            <div class="onboarding-header">
                <button type="button" class="btn-close-onboarding" onclick="closeOnboardingModal()" title="Cerrar">
                    <span class="material-icons-round">close</span>
                </button>
                <img src="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}" alt="SIFMA" class="onboarding-logo-img">
                <h2 style="font-family: var(--font-brand); font-size: 26px; font-weight: 700; color: var(--text-heading); margin: 0; line-height: 1.15; letter-spacing: 0.8px;">
                    Bienvenido a SIFMA
                </h2>
                <p style="margin: 5px 0 0 0; font-size: 13.5px; color: var(--text-muted); font-weight: 600; letter-spacing: 0.3px;">
                    Sistema Integral de Fenotipado y Monitoreo Agronómico
                </p>
            </div>

            <!-- Body: Instrucciones y Guía Rápida -->
            <div class="onboarding-body">
                <p style="font-size: 13px; color: var(--text-body); margin-bottom: 16px; line-height: 1.5;">
                    Plataforma científica para la adquisición, procesamiento y análisis biométrico y ambiental en torres aeropónicas verticales. Guía rápida de funcionamiento:
                </p>

                <div class="onboarding-steps-grid">
                    <!-- Paso 1 -->
                    <div class="onboarding-step-item">
                        <div class="step-num-badge">1</div>
                        <div class="step-content">
                            <h4>Procesamiento y Visión</h4>
                            <p>Importa lotes de fotografías cenitales desde USB o arrastra imágenes para segmentar el canopia vegetal y cuantificar área foliar (<span style="font-family: var(--font-mono); font-size: 11px;">cm²</span>) y verdor.</p>
                        </div>
                    </div>
```

```
263
264                     <!-- Paso 2 -->
265                     <div class="onboarding-step-item">
266                         <div class="step-num-badge">2</div>
267                         <div class="step-content">
268                             <h4>Telemetría Ambiental</h4>
269                             <p>Monitorea variables de microclima (temperatura, humedad, radiación solar UV y corriente de bombeo) sincronizadas en cada jornada.</p>
270                         </div>
271                     </div>
272
273                     <!-- Paso 3 -->
274                     <div class="onboarding-step-item">
275                         <div class="step-num-badge">3</div>
276                         <div class="step-content">
277                             <h4>Cruce y Correlaciones</h4>
278                             <p>Descubre patrones de causa-efecto entre clima y crecimiento mediante matrices de Pearson y modelos de regresión lineal.</p>
279                         </div>
280                     </div>
281
282                     <!-- Paso 4 -->
283                     <div class="onboarding-step-item">
284                         <div class="step-num-badge">4</div>
285                         <div class="step-content">
286                             <h4>Reportes Científicos & PDF</h4>
287                             <p>Genera dossiers técnicos completos con matrices estadísticas (&amp;mu;, &amp;sigma;, IQR, CV%) y firma digital para certificación de ensayos.</p>
288                         </div>
289                     </div>
290                 </div>
291
292                 <!-- Soporte Técnico & Contacto del Desarrollador (WhatsApp) -->
293                 <div style="margin-top: 14px; background: rgba(5, 150, 105, 0.06); border: 1px solid rgba(5, 150, 105, 0.25); border-radius: var(--radius-sm); padding:
    10px 14px; display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 10px;">
294                     <div style="display: flex; align-items: center; gap: 8px;">
295                         <span class="material-icons-round" style="color: var(--accent-emerald); font-size: 20px;">support_agent</span>
296                         <div style="font-size: 11.5px; color: var(--text-body);">
297                             <strong style="color: var(--text-heading);">¿Inconvenientes técnicos o dudas del sistema?</strong>
298                             <div style="color: var(--text-muted); font-size: 11px;">Soporte y atención directa vía WhatsApp:</div>
299                         </div>
300                     </div>
301                     <a href="https://wa.me/573128919009?text=Hola%20Andr%C3%A9s,%20te%20contacto%20desde%20el%20sistema%20SIFMA%20por%20una%20consulta%20t%C3%A9cnica."
    target="_blank" rel="noopener noreferrer" style="display: inline-flex; align-items: center; gap: 6px; font-size: 11.5px; font-weight: 700; color: #ffffff; background:
    #059669; padding: 5px 12px; border-radius: 4px; text-decoration: none; border: 1px solid #047857; transition: background 0.15s ease;"
    onmouseover="this.style.background='#047857';" onmouseout="this.style.background='#059669';">
302                         <svg height="14" width="14" viewBox="0 0 24 24" fill="currentColor">
303                             <path d="M.057 2411.687-6.163c-1.041-1.804-1.588-3.849-1.587-5.946.003-6.556 5.338-11.891 11.893-11.891 3.181.001 6.167 1.24 8.413 3.488
    2.245 2.248 3.481 5.236 3.48 8.414-.003 6.557-5.338 11.892-11.893 11.892-1.99-.001-3.951-.5-5.688-1.448l-6.305 1.654zm6.597-3.807c1.676.995 3.276 1.591 5.392 1.592 5.448
    0 9.886-4.434 9.889-9.885.002-5.462-4.415-9.89-9.881-9.892-5.452 0-9.887 4.434-9.889 9.884-.001 2.225.651 3.891 1.746 5.634l-.999 3.648 3.742-.981zm11.387-
    5.464c-.074-.124-.272-.198-.57-.347-.297-.149-1.758-.868-2.031-.967-.272-.099-.47-.149-.669.149-.198.297-.768.967-.941 1.165-.173.198-.347.223-.644.074-.297-.149-
    1.255-.462-2.39-1.475-.883-.788-1.48-1.761-1.653-
    2.059-.173-.297-.018-.458.13-.606.134-.133.297-.347.446-.521.151-.172.2-.296.3-.495.099-.198.05-.372-.025-.521-.075-.148-.669-1.611-.916-
    2.206-.242-.579-.487-.501-.669-.511-.57-.01c-.198 0-.52.074-.792.372s-1.04 1.016-1.04 2.479 1.065 2.876 1.213 3.074c.149.198 2.095 3.2 5.076 4.487.709.306 1.263.489
    1.694.626.712.226 1.36.194 1.872.118.571-.085 1.758-.719 2.006-1.413.248-.695.248-1.29.173-1.414z"/>
304                         </svg>
305                         <span>WhatsApp: 312 891 9009</span>
306                     </a>
307                 </div>
308             </div>
309
310             <!-- Footer -->
311             <div class="onboarding-footer">
312                 <label style="display: flex; align-items: center; gap: 8px; font-size: 12px; color: var(--text-muted); cursor: pointer; user-select: none;">
```

```
                    <input type="checkbox" id="onboarding-dont-show-again" style="cursor: pointer;">
                    <span>No volver a mostrar al iniciar</span>
                </label>
                <button type="button" class="btn btn-primary" onclick="confirmOnboarding()" style="padding: 7px 18px; font-size: 13px;">
                    <span>Entendido, Comenzar</span>
                    <span class="material-icons-round" style="font-size: 16px;">arrow_forward</span>
                </button>
            </div>
        </div>
    </div>

    <script>
        function toggleSidebar() {
            const sidebar = document.getElementById('sidebar');
            const main = document.getElementById('main-content');
            const overlay = document.getElementById('sidebar-overlay');

            if (window.innerWidth <= 992) {
                const isOpen = sidebar.classList.toggle('open');
                if (overlay) {
                    if (isOpen) overlay.classList.add('show');
                    else overlay.classList.remove('show');
                }
            } else {
                const isCollapsed = sidebar.classList.toggle('collapsed');
                if (main) main.classList.toggle('expanded');
                localStorage.setItem('sifma_sidebar_collapsed', isCollapsed);
            }
        }

        function switchActivePlant(newPlantId) {
            const url = new URL(window.location.href);
            url.searchParams.set('plant_id', newPlantId);
            window.location.href = url.toString();
        }

        function openOnboardingModal() {
            const modal = document.getElementById('sifma-onboarding-modal');
            if (modal) modal.style.display = 'flex';
        }

        function closeOnboardingModal() {
            const modal = document.getElementById('sifma-onboarding-modal');
            if (modal) modal.style.display = 'none';
        }

        function closeOnboardingOnBackdrop(e) {
            if (e.target.id === 'sifma-onboarding-modal') {
                closeOnboardingModal();
            }
        }

        function confirmOnboarding() {
            const dontShow = document.getElementById('onboarding-dont-show-again');
            if (dontShow && dontShow.checked) {
```

```
                localStorage.setItem('sifma_onboarding_shown', 'true');
            } else {
                localStorage.setItem('sifma_onboarding_shown', 'true');
            }
            closeOnboardingModal();
        }

        // Configuración Global de Chart.js
        if (window.Chart) {
            Chart.defaults.color = '#64748b';
            Chart.defaults.borderColor = 'rgba(0, 0, 0, 0.06)';
            if (Chart.defaults.plugins && Chart.defaults.plugins.tooltip) {
                Chart.defaults.plugins.tooltip.backgroundColor = '#0f172a';
                Chart.defaults.plugins.tooltip.titleColor = '#ffffff';
                Chart.defaults.plugins.tooltip.bodyColor = '#f8fafc';
                Chart.defaults.plugins.tooltip.borderColor = '#334155';
                Chart.defaults.plugins.tooltip.borderWidth = 1;
            }
        }

        // Toast Notification System Global
        window.showToast = function({ type = 'success', title = '', message = '', duration = 4000 }) {
            let container = document.getElementById('sifma-toast-container');
            if (!container) {
                container = document.createElement('div');
                container.id = 'sifma-toast-container';
                container.style.cssText = 'position: fixed; top: 20px; right: 24px; z-index: 999999; display: flex; flex-direction: column; gap: 10px; pointer-events: none;';
                document.body.appendChild(container);
            }

            const toast = document.createElement('div');
            toast.className = 'sifma-toast';

            let iconName = 'check_circle';
            if (type === 'error') iconName = 'error';
            else if (type === 'warning') iconName = 'warning';
            else if (type === 'info') iconName = 'info';

            toast.innerHTML = `
                <div class="sifma-toast-body">
                    <div class="sifma-toast-icon ${type}">
                        <span class="material-icons-round" style="font-size: 20px;">${iconName}</span>
                    </div>
                    <div style="flex: 1; padding-right: 6px;">
                        <div class="sifma-toast-title">${title}</div>
                        <div class="sifma-toast-msg">${message}</div>
                    </div>
                    <button type="button" class="sifma-toast-close" title="Cerrar">
                        <span class="material-icons-round" style="font-size: 16px;">close</span>
                    </button>
                </div>
                <div class="sifma-toast-progress-bar">
                    <div class="sifma-toast-progress-fill ${type}" style="transition: width ${duration}ms linear; width: 100%;"></div>
                </div>
```

```
            `;

            container.appendChild(toast);

            const fill = toast.querySelector('.sifma-toast-progress-fill');
            requestAnimationFrame(() => {
                setTimeout(() => {
                    if (fill) fill.style.width = '0%';
                }, 50);
            });

            const closeToast = () => {
                if (toast.classList.contains('closing')) return;
                toast.classList.add('closing');
                setTimeout(() => {
                    if (toast.parentElement) toast.parentElement.removeChild(toast);
                }, 300);
            };

            toast.querySelector('.sifma-toast-close').addEventListener('click', closeToast);
            setTimeout(closeToast, duration);
        };

        // Monitor Global en segundo plano para procesamientos activos
        let globalPollInterval = null;
        window.startGlobalProcessingMonitor = function() {
            if (globalPollInterval) return;
            globalPollInterval = setInterval(async () => {
                const isLocallyActive = localStorage.getItem('sifma_processing_active') === 'true';
                if (!isLocallyActive) {
                    clearInterval(globalPollInterval);
                    globalPollInterval = null;
                    return;
                }

                try {
                    const res = await fetch('/api/processing/progress');
                    if (res.ok) {
                        const prog = await res.json();
                        if (prog && prog.percent === 100 && !prog.is_processing) {
                            localStorage.removeItem('sifma_processing_active');
                            clearInterval(globalPollInterval);
                            globalPollInterval = null;
                            window.showToast({
                                type: 'success',
                                title: '¡Lote Fenotípico Finalizado!',
                                message: prog.message || 'El procesamiento de fotos ha concluido exitosamente.',
                                duration: 5000
                            });
                        } else if (prog && !prog.is_processing && prog.percent === 0 && prog.message && prog.message.startsWith('Error')) {
                            localStorage.removeItem('sifma_processing_active');
                            clearInterval(globalPollInterval);
                            globalPollInterval = null;
                            window.showToast({
                                type: 'error',
```

```
                                title: 'Error en Procesamiento',
                                message: prog.message,
                                duration: 6000
                            });
                        }
                    }
                } catch (e) {}
            }, 2000);
        };

        document.addEventListener('DOMContentLoaded', () => {
            if (window.innerWidth > 992) {
                const saved = localStorage.getItem('sifma_sidebar_collapsed');
                if (saved === 'true') {
                    document.getElementById('sidebar')?.classList.add('collapsed');
                    document.getElementById('main-content')?.classList.add('expanded');
                }
            }

            // Onboarding inicial automático para nuevos accesos
            const hasSeenOnboarding = localStorage.getItem('sifma_onboarding_shown');
            if (!hasSeenOnboarding) {
                setTimeout(() => {
                    openOnboardingModal();
                }, 400);
            }

            // Iniciar monitoreo si hay un proceso en segundo plano lanzado desde otra pestaña
            if (localStorage.getItem('sifma_processing_active') === 'true') {
                window.startGlobalProcessingMonitor();
            }
        });
    </script>

    <style>
    @keyframes toastSlideIn {
        from { opacity: 0; transform: translateY(-20px) scale(0.95); }
        to { opacity: 1; transform: translateY(0) scale(1); }
    }
    @keyframes toastSlideOut {
        from { opacity: 1; transform: translateY(0) scale(1); }
        to { opacity: 0; transform: translateY(-20px) scale(0.95); }
    }
    .sifma-toast {
        display: flex;
        flex-direction: column;
        background: #0f172a;
        color: #ffffff;
        border-radius: 12px;
        box-shadow: 0 10px 30px -5px rgba(0, 0, 0, 0.35), 0 0 0 1px rgba(255, 255, 255, 0.1);
        min-width: 320px;
        max-width: 420px;
        pointer-events: auto;
        position: relative;
        overflow: hidden;
```

```
        animation: toastSlideIn 0.35s cubic-bezier(0.16, 1, 0.3, 1) forwards;
        transition: all 0.3s ease;
    }
    .sifma-toast.closing {
        animation: toastSlideOut 0.3s ease forwards;
    }
    .sifma-toast-body {
        display: flex;
        align-items: flex-start;
        gap: 12px;
        padding: 14px 16px;
    }
    .sifma-toast-icon {
        width: 34px;
        height: 34px;
        border-radius: 8px;
        display: flex;
        align-items: center;
        justify-content: center;
        flex-shrink: 0;
    }
    .sifma-toast-icon.success { background: rgba(16, 185, 129, 0.2); color: #10b981; }
    .sifma-toast-icon.info { background: rgba(37, 99, 235, 0.2); color: #3b82f6; }
    .sifma-toast-icon.warning { background: rgba(245, 158, 11, 0.2); color: #f59e0b; }
    .sifma-toast-icon.error { background: rgba(239, 68, 68, 0.2); color: #ef4444; }

    .sifma-toast-title {
        font-size: 13.5px;
        font-weight: 700;
        color: #ffffff;
        margin-bottom: 2px;
    }
    .sifma-toast-msg {
        font-size: 12px;
        color: #94a3b8;
        line-height: 1.4;
    }
    .sifma-toast-close {
        background: transparent;
        border: none;
        color: #64748b;
        cursor: pointer;
        padding: 2px;
        display: flex;
        align-items: center;
        justify-content: center;
        border-radius: 4px;
        margin-left: auto;
        transition: color 0.2s;
    }
    .sifma-toast-close:hover { color: #ffffff; }
    .sifma-toast-progress-bar {
        width: 100%;
        height: 3.5px;
        background: rgba(255, 255, 255, 0.1);
```

```
        position: relative;
    }
    .sifma-toast-progress-fill {
        height: 100%;
        width: 100%;
        transform-origin: left;
    }
    .sifma-toast-progress-fill.success { background: #10b981; }
    .sifma-toast-progress-fill.info { background: #3b82f6; }
    .sifma-toast-progress-fill.warning { background: #f59e0b; }
    .sifma-toast-progress-fill.error { background: #ef4444; }
    </style>

    {% block scripts %}{% endblock %}
</body>
</html>
```

ARCHIVO 62/85: local_server/templates/benchmark.html (367 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Comparador Inter-Canastillas & Benchmark{% endblock %}
{% block heading %}Comparador Inter-Canastillas & Benchmark{% endblock %}
{% block subheading %}Evaluación simultánea de crecimiento, gradiente vertical en la torre y Tasa de Crecimiento Relativo (RGR){% endblock %}

{% block header_controls %}
<!-- Selector de Canastilla / Torre para redirecciones rápidas -->
<div style="display: flex; align-items: center; gap: 6px;">
    <select id="header-plant-selector" class="form-input-control" style="padding: 6px 10px; font-weight: 600; font-size: 13px; cursor: pointer; width: auto;" onchange="switchActivePlant(this.value)">
        <option value="1" {% if active_plant_id == 1 %}selected{% endif %}>Canastilla #1</option>
        <option value="2" {% if active_plant_id == 2 %}selected{% endif %}>Canastilla #2</option>
        <option value="3" {% if active_plant_id == 3 %}selected{% endif %}>Canastilla #3</option>
        <option value="4" {% if active_plant_id == 4 %}selected{% endif %}>Canastilla #4</option>
    </select>
</div>
{% endblock %}

{% block content %}

<!-- SECCIÓN 1: TARJETAS DE BENCHMARK POR NODO/CANASTILLA -->
<div class="kpi-row-grid" style="margin-bottom: 24px;">
    {% for pid in [1, 2, 3, 4] %}
        {% set c = benchmark_data.canastillas[pid] %}
        <div class="kpi-card" style="border-top: 3px solid {{ c.color.border }};">
            <div class="kpi-card-header">
                <div>
                    <span class="kpi-label" style="font-size: 11px; font-weight: 700; color: {{ c.color.border }};">
                        CANASTILLA #{{ pid }} ({{ c.level_name.split(' - ')[0] }})
                    </span>
                    <div style="font-size: 13.5px; font-weight: 700; color: var(--text-heading); margin-top: 2px;">
                        {{ c.display_name }}
                    </div>
                </div>
                <div class="kpi-icon-badge" style="background: {{ c.color.bg }}; color: {{ c.color.border }};">
                    <span class="material-icons-round">spa</span>
                </div>
            </div>

            {% if c.has_data %}
                <div class="kpi-value-row" style="margin-top: 8px;">
                    <span class="kpi-big-number">{{ c.latest_area }}</span>
                    <span class="kpi-unit">cm² foliares</span>
                </div>
                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 6px; margin-top: 10px; font-size: 11.5px;">
                    <div style="background: var(--bg-card-subtle); padding: 5px 7px; border-radius: 4px;">
```

```
                        <span style="color: var(--text-muted); display: block; font-size: 10px;">Ganancia ΔA:</span>
                        <strong style="color: var(--accent-success);">+{{ c.total_area_gain }} cm²</strong>
                    </div>
                    <div style="background: var(--bg-card-subtle); padding: 5px 7px; border-radius: 4px;">
                        <span style="color: var(--text-muted); display: block; font-size: 10px;">Tasa RGR:</span>
                        <strong style="color: #0284c7;">{{ c.rgr_pct_day }} %/día</strong>
                    </div>
                    <div style="background: var(--bg-card-subtle); padding: 5px 7px; border-radius: 4px;">
                        <span style="color: var(--text-muted); display: block; font-size: 10px;">Altura:</span>
                        <strong>{{ c.latest_height }} cm</strong>
                    </div>
                    <div style="background: var(--bg-card-subtle); padding: 5px 7px; border-radius: 4px;">
                        <span style="color: var(--text-muted); display: block; font-size: 10px;">Salud Media:</span>
                        <strong>{{ c.avg_health_overall }}%</strong>
                    </div>
                </div>
            {% else %}
                <div style="padding: 24px 0; text-align: center; color: var(--text-muted); font-size: 12px;">
                    <span class="material-icons-round" style="font-size: 24px; opacity: 0.5; display: block; margin-bottom: 4px;">hourglass_empty</span>
                    Sin muestras fenotípicas
                </div>
            {% endif %}
        </div>
    {% endfor %}
</div>

<!-- SECCIÓN 2: GRÁFICA MULTIVARIABLE COMPARATIVA SUPERPUESTA -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 14px; margin-bottom: 18px; padding-bottom: 14px; border-bottom: 1px solid var(--border-color);">
        <div>
            <h3 style="font-size: 15px; font-weight: 700; color: var(--text-heading); margin: 0;">
                Curvas Comparativas Superpuestas Inter-Canastillas
            </h3>
            <p style="font-size: 12px; color: var(--text-muted); margin: 2px 0 0 0;">
                Superposición temporal para identificar variaciones de vigor híbrido y velocidad de expansión foliar
            </p>
        </div>

        <!-- Selector de Métrica a Comparar -->
        <div class="filter-group">
            <button type="button" class="filter-btn active" id="btn-metric-area" onclick="switchBenchmarkMetric('area')">
                <span class="material-icons-round" style="font-size: 15px;">yard</span>
                <span>Área Foliar (cm²)</span>
            </button>
            <button type="button" class="filter-btn" id="btn-metric-height" onclick="switchBenchmarkMetric('height')">
                <span class="material-icons-round" style="font-size: 15px;">height</span>
                <span>Altura (cm)</span>
            </button>
            <button type="button" class="filter-btn" id="btn-metric-health" onclick="switchBenchmarkMetric('health')">
                <span class="material-icons-round" style="font-size: 15px;">favorite</span>
                <span>Salud Foliar (%)</span>
            </button>
        </div>
    </div>

```

```
    <!-- Filtro de Canastillas Visibles (Checkboxes) -->
    <div style="display: flex; align-items: center; gap: 18px; flex-wrap: wrap; margin-bottom: 16px; padding: 10px 14px; background: var(--bg-card-subtle); border-radius: 8px; font-size: 12.5px; font-weight: 600;">
        <span style="color: var(--text-muted);">Canastillas visibles:</span>
        {% for pid in [1, 2, 3, 4] %}
            {% set c = benchmark_data.canastillas[pid] %}
            <label style="display: flex; align-items: center; gap: 6px; cursor: pointer;">
                <input type="checkbox" id="toggle-plant-{{ pid }}" checked onchange="toggleDatasetVisibility({{ pid - 1 }})" style="accent-color: {{ c.color.border }}; cursor: pointer;">
                <span style="display: inline-block; width: 10px; height: 10px; border-radius: 50%; background: {{ c.color.border }};"></span>
                <span>Canastilla #{{ pid }} ({{ c.display_name }})</span>
            </label>
        {% endfor %}
    </div>

    <!-- Contenedor del Canvas -->
    <div style="position: relative; height: 380px; width: 100%;">
        <canvas id="benchmarkComparisonChart"></canvas>
    </div>
</div>

<!-- SECCIÓN 3: ANÁLISIS DE GRADIENTE VERTICAL EN LA TORRE -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 16px;">
        <div>
            <h3 style="font-size: 15px; font-weight: 700; color: var(--text-heading); margin: 0;">
                Evaluación del Gradiente Vertical en la Torre
            </h3>
            <p style="font-size: 12px; color: var(--text-muted); margin: 2px 0 0 0;">
                Influencia del estrato de altura (Nivel 4 Cúspide vs Nivel 1 Base) en la tasa de fotosíntesis y transpiración
            </p>
        </div>
        <span class="badge badge-primary">Estructura Hidropónica</span>
    </div>

    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 14px;">
        {% for pid in [4, 3, 2, 1] %}
            {% set c = benchmark_data.canastillas[pid] %}
            <div style="background: #ffffff; border: 1px solid var(--border-color); border-left: 4px solid {{ c.color.border }}; border-radius: 10px; padding: 14px; box-shadow: var(--shadow-sm);">
                <div style="display: flex; justify-content: space-between; align-items: flex-start; margin-bottom: 8px;">
                    <div>
                        <span style="font-size: 10.5px; font-weight: 700; color: var(--text-muted); text-transform: uppercase;">
                            {{ c.level_name }}
                        </span>
                        <h4 style="font-size: 14px; font-weight: 700; color: var(--text-heading); margin: 2px 0 0 0;">
                            Canastilla #{{ pid }}: {{ c.display_name }}
                        </h4>
                    </div>
                    <span class="badge" style="background: {{ c.color.bg }}; color: {{ c.color.border }}; font-weight: 700; font-size: 11px;">
                        {% if pid == 4 %}Máxima Luz{% elif pid == 1 %}Mayor Humedad{% else %}Intermedio{% endif %}
                    </span>
                </div>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 8px; font-size: 12px; margin-top: 10px;">
                    <div>
```

```
                        <span style="color: var(--text-muted); font-size: 10.5px;">Tasa RGR:</span>
                        <strong style="display: block; color: #0284c7; font-size: 13px;">{{ c.rgr_pct_day }} %/día</strong>
                    </div>
                    <div>
                        <span style="color: var(--text-muted); font-size: 10.5px;">Tasa AGR:</span>
                        <strong style="display: block; color: var(--accent-success); font-size: 13px;">{{ c.agr_cm2_day }} cm²/día</strong>
                    </div>
                    <div>
                        <span style="color: var(--text-muted); font-size: 10.5px;">Expansión Total:</span>
                        <strong style="display: block;">{{ c.total_area_gain }} cm²</strong>
                    </div>
                    <div>
                        <span style="color: var(--text-muted); font-size: 10.5px;">Salud Foliar:</span>
                        <strong style="display: block;">{{ c.avg_health_overall }}%</strong>
                    </div>
                </div>
            </div>
        {% endfor %}
    </div>
</div>

<!-- SECCIÓN 4: TABLA CONSOLIDADA DE BENCHMARK CIENTÍFICO -->
<div class="widget-card">
    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 16px; flex-wrap: wrap; gap: 10px;">
        <div>
            <h3 style="font-size: 15px; font-weight: 700; color: var(--text-heading); margin: 0;">
                Matriz Comparativa de Desempeño Biométrico
            </h3>
            <p style="font-size: 12px; color: var(--text-muted); margin: 2px 0 0 0;">
                Métricas biométricas consolidadas para análisis de publicaciones y reportes
            </p>
        </div>
        <a href="/api/cross_analysis/export_csv?plant_id={{ active_plant_id }}" class="btn btn-outline btn-sm">
            <span class="material-icons-round">download</span>
            <span>Exportar CSV Completo</span>
        </a>
    </div>

    <div class="table-responsive">
        <table class="table" style="font-size: 12.5px;">
            <thead>
                <tr>
                    <th>Canastilla</th>
                    <th>Especie / Variedad</th>
                    <th>Nivel en Torre</th>
                    <th>Lotes Procesados</th>
                    <th>Área Inicial</th>
                    <th>Área Actual</th>
                    <th>Ganancia ΔA</th>
                    <th>Tasa RGR (%/día)</th>
                    <th>Tasa AGR (cm²/día)</th>
                    <th>Altura Actual</th>
                    <th>Salud Foliar</th>
                    <th>Acción</th>
                </tr>
```

```
            </thead>
            <tbody>
                {% for pid in [1, 2, 3, 4] %}
                    {% set c = benchmark_data.canastillas[pid] %}
                    <tr>
                        <td style="font-weight: 700;">
                            <span style="display: inline-flex; align-items: center; gap: 6px;">
                                <span style="display: inline-block; width: 8px; height: 8px; border-radius: 50%; background: {{ c.color.border }};"></span>
                                <span>#{{ pid }}</span>
                            </span>
                        </td>
                        <td>
                            <strong>{{ c.display_name }}</strong>
                            {% if c.scientific_name %}
                                <span style="display: block; font-size: 10.5px; color: var(--text-muted); font-style: italic;">
                                    {{ c.scientific_name }}
                                </span>
                            {% endif %}
                        </td>
                        <td><span class="badge badge-neutral" style="font-size: 11px;">{{ c.level_name.split(' - ')[0] }}</span></td>
                        <td>{{ c.sessions_count }} sesiones</td>
                        <td>{{ c.initial_area }} cm²</td>
                        <td><strong style="color: var(--text-heading);">{{ c.latest_area }} cm²</strong></td>
                        <td style="color: var(--accent-success); font-weight: 700;">+{{ c.total_area_gain }} cm²</td>
                        <td style="color: #0284c7; font-weight: 700;">{{ c.rgr_pct_day }} %</td>
                        <td style="font-weight: 600;">{{ c.agr_cm2_day }} cm²/d</td>
                        <td>{{ c.latest_height }} cm</td>
                        <td>
                            <span class="badge {% if c.avg_health_overall >= 80 %}badge-success{% else %}badge-warning{% endif %}">
                                {{ c.avg_health_overall }}%
                            </span>
                        </td>
                        <td>
                            <a href="/?plant_id={{ pid }}" class="btn btn-outline btn-sm" style="padding: 3px 7px; font-size: 11px;" title="Ver Dashboard de Canastilla #{{ pid }}">
                                <span class="material-icons-round" style="font-size: 14px;">visibility</span>
                            </a>
                        </td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
</div>

{% endblock %}

{% block scripts %}
<script>
    const chartLabels = {{ benchmark_data.chart_labels | tojson | safe }};
    const chartsData = {{ benchmark_data.charts | tojson | safe }};
    let activeMetric = 'area';
    let comparisonChartInstance = null;

    Chart.defaults.font.family = "'Inter', sans-serif";
    Chart.defaults.color = '#64748b';
```

```
271
272     function initBenchmarkChart() {
273         const ctx = document.getElementById('benchmarkComparisonChart').getContext('2d');
274
275         const datasets = getDatasetsForMetric(activeMetric);
276
277         comparisonChartInstance = new Chart(ctx, {
278             type: 'line',
279             data: {
280                 labels: chartLabels.length ? chartLabels : ['Sin registros'],
281                 datasets: datasets
282             },
283             options: {
284                 responsive: true,
285                 maintainAspectRatio: false,
286                 interaction: {
287                     mode: 'index',
288                     intersect: false
289                 },
290                 plugins: {
291                     legend: {
292                         position: 'top',
293                         labels: {
294                             usePointStyle: true,
295                             boxWidth: 8,
296                             padding: 14,
297                             font: { size: 12, weight: '600' }
298                         }
299                     },
300                     tooltip: {
301                         backgroundColor: '#0f172a',
302                         titleFont: { size: 12, weight: '700' },
303                         bodyFont: { size: 11.5 },
304                         padding: 10,
305                         cornerRadius: 8
306                     }
307                 },
308                 scales: {
309                     x: {
310                         grid: { color: 'rgba(226, 232, 240, 0.6)' },
311                         title: { display: true, text: 'Fecha de Muestreo', font: { size: 11, weight: '600' } }
312                     },
313                     y: {
314                         grid: { color: 'rgba(226, 232, 240, 0.6)' },
315                         title: { display: true, text: getYAxisTitle(activeMetric), font: { size: 11, weight: '600' } },
316                         beginAtZero: true
317                     }
318                 }
319             }
320         });
321     }
322
323     function getYAxisTitle(metric) {
324         if (metric === 'area') return 'Área Foliar Promedio (cm²)';
325         if (metric === 'height') return 'Altura de Planta (cm)';
```

```
        if (metric === 'health') return 'Índice de Salud Foliar (%)';
        return '';
    }

    function getDatasetsForMetric(metric) {
        return chartsData[metric] || [];
    }

    function switchBenchmarkMetric(metric) {
        activeMetric = metric;
        document.querySelectorAll('.filter-group .filter-btn').forEach(b => b.classList.remove('active'));
        const activeBtn = document.getElementById(`btn-metric-${metric}`);
        if (activeBtn) activeBtn.classList.add('active');

        if (comparisonChartInstance) {
            comparisonChartInstance.data.datasets = getDatasetsForMetric(metric);
            comparisonChartInstance.options.scales.y.title.text = getYAxisTitle(metric);

            // Sincronizar visibilidad de checkboxes
            for (let i = 0; i < 4; i++) {
                const cb = document.getElementById(`toggle-plant-${i + 1}`);
                if (cb && comparisonChartInstance.data.datasets[i]) {
                    comparisonChartInstance.data.datasets[i].hidden = !cb.checked;
                }
            }
            comparisonChartInstance.update();
        }
    }

    function toggleDatasetVisibility(datasetIndex) {
        if (comparisonChartInstance && comparisonChartInstance.data.datasets[datasetIndex]) {
            const cb = document.getElementById(`toggle-plant-${datasetIndex + 1}`);
            comparisonChartInstance.data.datasets[datasetIndex].hidden = !cb.checked;
            comparisonChartInstance.update();
        }
    }

    document.addEventListener('DOMContentLoaded', () => {
        initBenchmarkChart();
    });
</script>
{% endblock %}
```

ARCHIVO 63/85: local_server/templates/calendar.html (653 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Calendario de Muestreos y Telemetría{% endblock %}
{% block heading %}Calendario de Muestreos y Telemetría{% endblock %}
{% block subheading %}Bitácora interactiva de fenotipado fotográfico y telemetría ambiental para Canastilla #{{ active_plant_id }}{% endblock %}

{% block header_controls %}
<!-- Navegador de Mes / Año -->
<div class="date-navigator-box">
    <button class="date-nav-btn" onclick="navigateMonth({{ cal_data.prev_year }}, {{ cal_data.prev_month }})" title="Mes Anterior">
        <span class="material-icons-round">chevron_left</span>
    </button>
    <div style="display: flex; align-items: center; gap: 6px; font-weight: 700; font-size: 13.5px; color: var(--text-heading); padding: 0 4px;">
        <span class="material-icons-round text-muted" style="font-size: 17px;">calendar_month</span>
        <span>{{ cal_data.month_name }} {{ cal_data.year }}</span>
    </div>
    <button class="date-nav-btn" onclick="navigateMonth({{ cal_data.next_year }}, {{ cal_data.next_month }})" title="Mes Siguiente">
        <span class="material-icons-round">chevron_right</span>
    </button>
</div>

<!-- Selector de Canastilla / Torre -->
<div style="display: flex; align-items: center; gap: 6px;">
    <select id="header-plant-selector" class="form-input-control" style="padding: 6px 10px; font-weight: 600; font-size: 13px; cursor: pointer; width: auto;"
onchange="switchActivePlant(this.value)">
        <option value="1" {% if active_plant_id == 1 %}selected{% endif %}>Canastilla #1</option>
        <option value="2" {% if active_plant_id == 2 %}selected{% endif %}>Canastilla #2</option>
        <option value="3" {% if active_plant_id == 3 %}selected{% endif %}>Canastilla #3</option>
        <option value="4" {% if active_plant_id == 4 %}selected{% endif %}>Canastilla #4</option>
    </select>
</div>
{% endblock %}

{% block content %}

<!-- CONTENEDOR PRINCIPAL: DISTRIBUCIÓN LATERAL (TARJETAS KPI AL LADO DEL CALENDARIO) -->
<div class="dashboard-split-layout calendar-split-container">
    <!-- COLUMNA LATERAL: 4 TARJETAS DE RESUMEN MENSUAL -->
    <div class="dashboard-kpi-column">
        <!-- Card 1: Días con Fotos -->
        <div class="kpi-card">
            <div class="kpi-card-header">
                <span class="kpi-label">Fenotipado (Fotos)</span>
                <div class="kpi-icon-badge emerald">
                    <span class="material-icons-round">camera</span>
                </div>
            </div>
```

```
            <div class="kpi-value-row">
                <span class="kpi-big-number">{{ cal_data.summary.total_photo_days }}</span>
                <span class="kpi-unit">/ {{ cal_data.summary.days_in_month }} días</span>
            </div>
            <div class="kpi-footer-badge positive">
                <span class="material-icons-round" style="font-size: 14px;">spa</span>
                <span>{{ cal_data.summary.total_sessions }} Muestreos en Canastilla #{{ active_plant_id }}</span>
            </div>
        </div>

        <!-- Card 2: Días con Telemetría -->
        <div class="kpi-card">
            <div class="kpi-card-header">
                <span class="kpi-label">Telemetría (Sensores)</span>
                <div class="kpi-icon-badge blue">
                    <span class="material-icons-round">sensors</span>
                </div>
            </div>
            <div class="kpi-value-row">
                <span class="kpi-big-number">{{ cal_data.summary.total_telemetry_days }}</span>
                <span class="kpi-unit">/ {{ cal_data.summary.days_in_month }} días</span>
            </div>
            <div class="kpi-footer-badge {% if cal_data.summary.total_telemetry_days > 0 %}positive{% else %}neutral{% endif %}">
                <span class="material-icons-round" style="font-size: 14px;">thermostat</span>
                <span>{{ cal_data.summary.total_readings }} Lecturas registradas</span>
            </div>
        </div>

        <!-- Card 3: Jornadas Completas Sincronizadas -->
        <div class="kpi-card">
            <div class="kpi-card-header">
                <span class="kpi-label">Jornadas Sincronizadas</span>
                <div class="kpi-icon-badge cyan">
                    <span class="material-icons-round">sync_alt</span>
                </div>
            </div>
            <div class="kpi-value-row">
                <span class="kpi-big-number">{{ cal_data.summary.total_combined_days }}</span>
                <span class="kpi-unit">días</span>
            </div>
            <div class="kpi-footer-badge positive">
                <span class="material-icons-round" style="font-size: 14px;">check_circle</span>
                <span>Aptos para Cruce Multivariable</span>
            </div>
        </div>

        <!-- Card 4: Cobertura Mensual -->
        <div class="kpi-card">
            <div class="kpi-card-header">
                <span class="kpi-label">Cobertura del Mes</span>
                <div class="kpi-icon-badge purple">
                    <span class="material-icons-round">pie_chart</span>
                </div>
            </div>
            <div class="kpi-value-row">
```

```
                <span class="kpi-big-number">{{ cal_data.summary.coverage_pct }}</span>
                <span class="kpi-unit">%</span>
            </div>
            <div class="kpi-footer-badge neutral">
                <span class="material-icons-round" style="font-size: 14px;">date_range</span>
                <span>{{ cal_data.month_name }} {{ cal_data.year }}</span>
            </div>
        </div>
    </div>

    <!-- COLUMNA PRINCIPAL: CALENDARIO MENSUAL EXPANDIDO -->
    <div class="dashboard-charts-main">
        <div class="widget-card calendar-main-card">
            <!-- Barra Superior del Calendario y Leyenda -->
            <div class="calendar-header-bar">
                <div style="display: flex; align-items: center; gap: 8px;">
                    <span style="font-size: 15px; font-weight: 700; color: var(--text-heading);">
                        {{ cal_data.month_name }} {{ cal_data.year }}
                    </span>
                </div>

                <!-- Convención y Leyenda de Colores -->
                <div style="display: flex; align-items: center; gap: 12px; flex-wrap: wrap; font-size: 11px; font-weight: 600;">
                    <div style="display: flex; align-items: center; gap: 5px;">
                        <span style="display: inline-block; width: 9px; height: 9px; border-radius: 2px; background: var(--accent-success);"></span>
                        <span style="color: var(--text-heading);">Fenotipado</span>
                    </div>
                    <div style="display: flex; align-items: center; gap: 5px;">
                        <span style="display: inline-block; width: 9px; height: 9px; border-radius: 2px; background: #0284c7;"></span>
                        <span style="color: var(--text-heading);">Telemetría</span>
                    </div>
                    <div style="display: flex; align-items: center; gap: 5px;">
                        <span style="display: inline-block; width: 9px; height: 9px; border-radius: 2px; background: linear-gradient(135deg, var(--accent-success) 50%,
#0284c7 50%);"></span>
                        <span style="color: var(--text-heading);">Sincronizado</span>
                    </div>
                </div>
            </div>

            <!-- Encabezado de Días de la Semana -->
            <div class="calendar-weekdays-grid">
                <div class="calendar-weekday">LUN</div>
                <div class="calendar-weekday">MAR</div>
                <div class="calendar-weekday">MIÉ</div>
                <div class="calendar-weekday">JUE</div>
                <div class="calendar-weekday">VIE</div>
                <div class="calendar-weekday">SÁB</div>
                <div class="calendar-weekday">DOM</div>
            </div>

            <!-- Cuadrícula de Días del Mes -->
            <div class="calendar-month-grid" style="grid-template-rows: repeat({{ cal_data.weeks | length }}, minmax(0, 1fr));">
                {% for week in cal_data.weeks %}
                    {% for day in week %}
                        <div class="calendar-day-cell {% if not day.is_current_month %}out-of-month{% endif %} {% if day.is_today %}is-today{% endif %} {% if
day.has_photos or day.has_telemetry %}has-records{% endif %}"
```

```
                         onclick="openDayDetail('{{ day.date_str }}')"
                         title="Clic para ver desglose de la jornada {{ day.date_str }}">

                    <!-- Cabecera de la celda: Número de día y tag Hoy -->
                    <div class="calendar-day-header">
                        <span class="calendar-day-number {% if day.is_today %}today-number{% endif %}">
                            {{ day.day_number }}
                        </span>
                        {% if day.is_today %}
                            <span class="badge badge-primary" style="font-size: 8.5px; padding: 1px 3px;">HOY</span>
                        {% endif %}
                    </div>

                    <!-- Indicadores de Datos del Día -->
                    <div class="calendar-day-badges">
                        <!-- Indicador Verde: Fenotipado / Fotos -->
                        {% if day.has_photos %}
                            <div class="cal-badge green" title="{{ day.photo_count }} Sesiones de Fenotipado">
                                <div style="display: flex; align-items: center; justify-content: space-between;">
                                    <span style="display: flex; align-items: center; gap: 2px; font-weight: 700;">
                                        <span class="material-icons-round" style="font-size: 11px;">spa</span>
                                        <span>{{ day.photo_count }} fotos</span>
                                    </span>
                                    {% if day.avg_foliar_area > 0 %}
                                        <span style="font-size: 9.5px; opacity: 0.9;">{{ day.avg_foliar_area }} cm²</span>
                                    {% endif %}
                                </div>
                            </div>
                        {% endif %}

                        <!-- Indicador Azul: Telemetría / Sensores -->
                        {% if day.has_telemetry %}
                            <div class="cal-badge blue" title="{{ day.telemetry_count }} Lecturas de Sensores">
                                <div style="display: flex; align-items: center; justify-content: space-between;">
                                    <span style="display: flex; align-items: center; gap: 2px; font-weight: 700;">
                                        <span class="material-icons-round" style="font-size: 11px;">sensors</span>
                                        <span>{{ day.telemetry_count }} sens.</span>
                                    </span>
                                    {% if day.avg_temp > 0 %}
                                        <span style="font-size: 9.5px; opacity: 0.9;">{{ day.avg_temp }}°C</span>
                                    {% endif %}
                                </div>
                            </div>
                        {% endif %}

                        <!-- Si no tiene datos en este mes -->
                        {% if day.is_current_month and not day.has_photos and not day.has_telemetry %}
                            <div class="cal-empty-label">
                                Sin datos
                            </div>
                        {% endif %}
                    </div>
                </div>
            {% endfor %}
        {% endfor %}
```

```
            </div>
        </div>
    </div>
</div>

<!-- MODAL / DRAWER INTERACTIVO: DESGLOSE COMPLETO DE LA JORNADA -->
<div id="day-detail-modal" class="modal-backdrop" style="display: none;" onclick="closeModalOnBackdrop(event)">
    <div class="modal-card" style="max-width: 780px; width: 95%;">

        <!-- Cabecera del Modal -->
        <div class="modal-header">
            <div style="display: flex; align-items: center; gap: 10px;">
                <div class="kpi-icon-badge emerald" style="width: 36px; height: 36px;">
                    <span class="material-icons-round" style="font-size: 20px;">event_available</span>
                </div>
                <div>
                    <h3 id="modal-day-title" style="margin: 0; font-size: 16px; color: var(--text-heading);">
                        Detalle de la Jornada
                    </h3>
                    <p id="modal-day-subtitle" style="margin: 0; font-size: 12px; color: var(--text-muted);">
                        Canastilla #{{ active_plant_id }} ({{ selected_profile.display_name }})
                    </p>
                </div>
            </div>
            <button type="button" class="btn btn-outline btn-sm" onclick="closeDayModal()" style="padding: 4px 8px;">
                <span class="material-icons-round">close</span>
            </button>
        </div>

        <div class="modal-body" style="padding: 20px; max-height: 70vh; overflow-y: auto;">

            <!-- CONTENEDOR DE ESTADO DE CARGA -->
            <div id="modal-loading-state" style="display: flex; flex-direction: column; align-items: center; justify-content: center; padding: 40px 0;">
                <span class="material-icons-round spinner" style="font-size: 32px; color: var(--accent-success);">sync</span>
                <p style="margin-top: 10px; color: var(--text-muted); font-size: 13px;">Cargando datos de la jornada...</p>
            </div>

            <div id="modal-content-state" style="display: none;">

                <!-- SECTOR A: RESUMEN DE TELEMETRÍA AMBIENTAL -->
                <div style="margin-bottom: 22px;">
                    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 12px;">
                        <h4 style="font-size: 13.5px; font-weight: 700; color: #0284c7; margin: 0; display: flex; align-items: center; gap: 6px;">
                            <span class="material-icons-round" style="font-size: 18px;">sensors</span>
                            <span>Microclima Ambiental de la Jornada</span>
                        </h4>
                        <span class="badge badge-primary" style="font-size: 10.5px;">Promedios Diarios (24h)</span>
                    </div>

                    <div id="modal-env-grid" style="display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 12px;">
                        <!-- Inyectado dinámicamente con JS -->
                    </div>
                </div>

                <!-- SECTOR B: LOTES FOTOGRÁFICOS Y FENOTIPADO -->
```

```
                <div style="margin-bottom: 22px;">
                    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 12px;">
                        <h4 style="font-size: 13.5px; font-weight: 700; color: var(--accent-success); margin: 0; display: flex; align-items: center; gap: 6px;">
                            <span class="material-icons-round" style="font-size: 18px;">spa</span>
                            <span>Muestreos de Fenotipado Digital</span>
                        </h4>
                        <span class="badge badge-success" style="font-size: 10.5px;">Promedios de 5 Fotos / Período</span>
                    </div>

                    <div id="modal-sessions-container">
                        <!-- Inyectado dinámicamente con JS -->
                    </div>
                </div>

                <!-- SECTOR C: ACCIONES DIRECTAS A OTRAS SECCIONES -->
                <div style="border-top: 1px solid var(--border-color); padding-top: 16px; display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px;">
                    <span style="font-size: 12px; font-weight: 600; color: var(--text-muted);">
                        Explorar esta jornada en:
                    </span>
                    <div style="display: flex; gap: 8px; flex-wrap: wrap;">
                        <a href="#" id="modal-btn-dashboard" class="btn btn-primary btn-sm">
                            <span class="material-icons-round">dashboard</span>
                            <span>Ver en Dashboard</span>
                        </a>
                        <a href="#" id="modal-btn-sensors" class="btn btn-outline btn-sm">
                            <span class="material-icons-round">sensors</span>
                            <span>Ver en Sensores</span>
                        </a>
                        <a href="#" id="modal-btn-gallery" class="btn btn-outline btn-sm">
                            <span class="material-icons-round">photo_library</span>
                            <span>Galería de Fotos</span>
                        </a>
                        <a href="#" id="modal-btn-export-day" class="btn btn-outline btn-sm" style="border-color: #059669; color: #059669; font-weight: 700;" title="Descargar Resumen Agronómico del Día en CSV">
                            <span class="material-icons-round">download</span>
                            <span>Exportar Día (CSV)</span>
                        </a>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>

{% endblock %}

{% block scripts %}
<style>
/* ESTILOS DEL CALENDARIO SIFMA EN SPLIT LAYOUT SIN SCROLL */
.calendar-split-container {
    height: calc(100vh - 125px);
    margin-bottom: 0;
}

.calendar-main-card {
```

```
    height: 100%;
    margin-bottom: 0;
    padding: 14px 18px;
    display: flex;
    flex-direction: column;
}

.calendar-header-bar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: 10px;
    padding-bottom: 8px;
    border-bottom: 1px solid var(--border-color);
    margin-bottom: 8px;
}

.calendar-weekdays-grid {
    display: grid;
    grid-template-columns: repeat(7, 1fr);
    gap: 6px;
    margin-bottom: 6px;
    text-align: center;
}

.calendar-weekday {
    font-size: 10.5px;
    font-weight: 700;
    color: var(--text-muted);
    letter-spacing: 0.5px;
    padding: 4px 2px;
    background: var(--bg-card-subtle);
    border-radius: var(--radius-xs);
    border: 1px solid var(--border-color);
}

.calendar-month-grid {
    flex: 1;
    display: grid;
    grid-template-columns: repeat(7, 1fr);
    gap: 6px;
    min-height: 0;
}

.calendar-day-cell {
    min-height: 0;
    height: 100%;
    background: #ffffff;
    border: 1px solid var(--border-color);
    border-radius: var(--radius-sm);
    padding: 4px 6px;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
```

```
    gap: 3px;
    cursor: pointer;
    transition: all 0.15s cubic-bezier(0.4, 0, 0.2, 1);
    position: relative;
    overflow: hidden;
}

.calendar-day-cell:hover {
    border-color: var(--accent-success);
    transform: translateY(-1.5px);
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
    z-index: 2;
}

.calendar-day-cell.out-of-month {
    background: #fafafa;
    opacity: 0.4;
}

.calendar-day-cell.is-today {
    border: 2px solid var(--accent-success);
    background: rgba(16, 185, 129, 0.03);
}

.calendar-day-cell.has-records {
    border-left: 3px solid var(--accent-success);
}

.calendar-day-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 2px;
}

.calendar-day-number {
    font-size: 11.5px;
    font-weight: 700;
    color: var(--text-heading);
}

.calendar-day-number.today-number {
    color: var(--accent-success);
    font-size: 12px;
}

.calendar-day-badges {
    display: flex;
    flex-direction: column;
    gap: 3px;
    overflow: hidden;
}

.cal-badge {
    padding: 2px 4px;
```

```
    border-radius: var(--radius-xs);
    font-size: 9.5px;
    line-height: 1.15;
    transition: transform 0.15s ease;
}

.cal-badge:hover {
    transform: scale(1.02);
}

.cal-badge.green {
    background: rgba(16, 185, 129, 0.12);
    border: 1px solid rgba(16, 185, 129, 0.3);
    color: #065f46;
}

.cal-badge.blue {
    background: rgba(2, 132, 199, 0.12);
    border: 1px solid rgba(2, 132, 199, 0.3);
    color: #0369a1;
}

.cal-empty-label {
    font-size: 9px;
    color: #cbd5e1;
    text-align: center;
    padding-top: 4px;
}

/* MODAL / DRAWER */
.modal-backdrop {
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    background: rgba(15, 23, 42, 0.6);
    backdrop-filter: blur(4px);
    z-index: 9999;
    display: flex;
    align-items: center;
    justify-content: center;
    animation: fadeIn 0.2s ease;
}

.modal-card {
    background: #ffffff;
    border-radius: 16px;
    border: 1px solid var(--border-color);
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.2);
    overflow: hidden;
    animation: slideUp 0.25s cubic-bezier(0.4, 0, 0.2, 1);
}

.modal-header {
```

```
    padding: 16px 20px;
    background: var(--bg-card-subtle);
    border-bottom: 1px solid var(--border-color);
    display: flex;
    justify-content: space-between;
    align-items: center;
}

@keyframes fadeIn {
    from { opacity: 0; }
    to { opacity: 1; }
}

@keyframes slideUp {
    from { transform: translateY(20px); opacity: 0; }
    to { transform: translateY(0); opacity: 1; }
}

@media (max-width: 900px) {
    .calendar-weekdays-grid, .calendar-month-grid {
        grid-template-columns: repeat(7, minmax(80px, 1fr));
        overflow-x: auto;
    }
}
</style>

<script>
    const activePlantId = {{ active_plant_id }};

    function navigateMonth(year, month) {
        window.location.href = `/calendar?year=${year}&month=${month}&plant_id=${activePlantId}`;
    }

    function closeModalOnBackdrop(e) {
        if (e.target.id === 'day-detail-modal') {
            closeDayModal();
        }
    }

    function closeDayModal() {
        document.getElementById('day-detail-modal').style.display = 'none';
    }

    async function openDayDetail(dateStr) {
        const modal = document.getElementById('day-detail-modal');
        const loading = document.getElementById('modal-loading-state');
        const content = document.getElementById('modal-content-state');

        modal.style.display = 'flex';
        loading.style.display = 'flex';
        content.style.display = 'none';

        document.getElementById('modal-day-title').innerText = `Jornada del ${dateStr}`;
        document.getElementById('modal-btn-dashboard').href = `/?date=${dateStr}&plant_id=${activePlantId}`;
        document.getElementById('modal-btn-sensors').href = `/sensors?date=${dateStr}&plant_id=${activePlantId}`;
```

```
        document.getElementById('modal-btn-gallery').href = `/gallery?plant_id=${activePlantId}`;
        const btnExport = document.getElementById('modal-btn-export-day');
        if (btnExport) btnExport.href = `/api/daily_summary/export_csv?date=${dateStr}&plant_id=${activePlantId}`;

        try {
            const resp = await fetch(`/api/calendar/day_detail?date=${dateStr}&plant_id=${activePlantId}`);
            const data = await resp.json();

            loading.style.display = 'none';
            content.style.display = 'block';

            // 1. Renderizar Bloque Ambiental
            const envGrid = document.getElementById('modal-env-grid');
            const env = data.environment;
            if (env && env.has_readings) {
                envGrid.innerHTML = `
                    <div style="background: rgba(37, 99, 235, 0.06); border: 1px solid rgba(37, 99, 235, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">Temp. Ambiente</span>
                        <strong style="font-size: 15px; color: #2563eb;">${env.avg_temp} °C</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Mín: ${env.min_temp}°C | Máx: ${env.max_temp}°C</div>
                    </div>
                    <div style="background: rgba(2, 132, 199, 0.06); border: 1px solid rgba(2, 132, 199, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">Humedad Relativa</span>
                        <strong style="font-size: 15px; color: #0284c7;">${env.avg_hum} %</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Mín: ${env.min_hum}% | Máx: ${env.max_hum}%</div>
                    </div>
                    <div style="background: rgba(236, 72, 153, 0.06); border: 1px solid rgba(236, 72, 153, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">pH Solución</span>
                        <strong style="font-size: 15px; color: #ec4899;">${env.avg_ph}</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Hidroponía óptima</div>
                    </div>
                    <div style="background: rgba(139, 92, 246, 0.06); border: 1px solid rgba(139, 92, 246, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">Conductividad (EC)</span>
                        <strong style="font-size: 15px; color: #8b5cf6;">${env.avg_ec} µS/cm</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Nutrición basal</div>
                    </div>
                    <div style="background: rgba(249, 115, 22, 0.06); border: 1px solid rgba(249, 115, 22, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">Turbidez Agua</span>
                        <strong style="font-size: 15px; color: #f97316;">${env.avg_turbidity} NTU</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Claridad de solución</div>
                    </div>
                    <div style="background: rgba(6, 182, 212, 0.06); border: 1px solid rgba(6, 182, 212, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">Oxígeno Disuelto (DO)</span>
                        <strong style="font-size: 15px; color: #06b6d4;">${env.avg_do} mg/L</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Aeración radicular</div>
                    </div>
                    <div style="background: rgba(14, 165, 233, 0.06); border: 1px solid rgba(14, 165, 233, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">Temp. del Agua</span>
                        <strong style="font-size: 15px; color: #0ea5e9;">${env.avg_water_temp} °C</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Estabilidad térmica</div>
                    </div>
                    <div style="background: rgba(5, 150, 105, 0.06); border: 1px solid rgba(5, 150, 105, 0.2); padding: 10px; border-radius: 8px;">
                        <span style="font-size: 11px; font-weight: 600; color: var(--text-muted); display: block;">Bomba / Recirculación</span>
                        <strong style="font-size: 15px; color: #059669;">${env.motor_active_pct}% ON</strong>
                        <div style="font-size: 10px; color: var(--text-muted); margin-top: 2px;">Corriente: ${env.avg_current} A</div>
```

```
                    </div>
                `;
            } else {
                envGrid.innerHTML = `
                    <div style="grid-column: 1 / -1; background: var(--bg-card-subtle); padding: 12px; border-radius: 8px; border: 1px dashed var(--border-color); color: var(--text-muted); font-size: 12.5px;">
                        No se registraron lecturas ambientales de sensores para esta fecha.
                    </div>
                `;
            }

            // 2. Renderizar Bloque de Sesiones Fotográficas
            const sessContainer = document.getElementById('modal-sessions-container');
            if (data.sessions && data.sessions.length > 0) {
                let html = '<div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 12px;">';
                data.sessions.forEach(s => {
                    html += `
                        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 10px; padding: 12px; box-shadow: var(--shadow-sm);">
                            <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px;">
                                <span class="badge badge-success" style="font-size: 11px; font-weight: 700;">${s.period}</span>
                                <span style="font-size: 11px; color: var(--text-muted);">${s.timestamp}</span>
                            </div>
                            ${s.thumbnail_url ? `
                                <div style="height: 100px; border-radius: 6px; overflow: hidden; background: #000; margin-bottom: 8px;">
                                    <img src="${s.thumbnail_url}" style="width: 100%; height: 100%; object-fit: cover;">
                                </div>
                            ` : ''}
                            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 4px; font-size: 11.5px;">
                                <div><span style="color: var(--text-muted);">Área Foliar:</span> <strong>${s.foliar_area_cm2} cm²</strong></div>
                                <div><span style="color: var(--text-muted);">Altura:</span> <strong>${s.plant_height_cm} cm</strong></div>
                                <div><span style="color: var(--text-muted);">Tallo:</span> <strong>${s.stem_diameter_mm} mm</strong></div>
                                <div><span style="color: var(--text-muted);">Salud:</span> <strong>${s.health_index}%</strong></div>
                            </div>
                        </div>
                    `;
                });
                html += '</div>';
                sessContainer.innerHTML = html;
            } else {
                sessContainer.innerHTML = `
                    <div style="background: var(--bg-card-subtle); padding: 12px; border-radius: 8px; border: 1px dashed var(--border-color); color: var(--text-muted); font-size: 12.5px;">
                        No se registraron capturas de fenotipado para esta fecha en la Canastilla #${activePlantId}.
                    </div>
                `;
            }

        } catch (err) {
            loading.style.display = 'none';
            content.style.display = 'block';
            document.getElementById('modal-sessions-container').innerHTML = `<p style="color: #dc2626;">Error al cargar datos de la jornada: ${err.message}</p>`;
        }
    }
</script>
{% endblock %}
```

ARCHIVO 64/85: local_server/templates/config.html (1305 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Parámetros de Cultivo{% endblock %}
{% block heading %}Parámetros de Cultivo y Calibración{% endblock %}
{% block subheading %}Asignación de especies por Canastilla y ajuste de algoritmos de visión artificial{% endblock %}

{% block content %}

<!-- CARD 1: ASIGNACIÓN DE ESPECIE BOTÁNICA POR CADA CANASTILLA DE LA TORRE -->
<div class="widget-card" style="max-width: 960px; margin: 0 auto 24px auto;">
    <div class="widget-header">
        <div class="widget-header-title" style="display: flex; align-items: center; gap: 12px;">
            <div class="kpi-icon-badge emerald" style="width: 38px; height: 38px;">
                <span class="material-icons-round" style="font-size: 22px;">view_in_ar</span>
            </div>
            <div>
                <h3>Asignación de Especies Vegetales por Canastilla / Torre</h3>
                <p>Configura qué cultivo está sembrado en cada una de las 4 canastillas del sistema</p>
            </div>
        </div>
        <span class="badge badge-primary">
            Canastilla Activa: #{{ active_plant_id }}
        </span>
    </div>

    <form action="{{ url_for('dashboard.configuration_panel') }}" method="POST" style="background: var(--bg-card-subtle); padding: 20px; border-radius: 12px; border: 1px solid var(--border-color);">
        <input type="hidden" name="action" value="assign_plant_crops">

        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 16px; margin-bottom: 20px;">
            {% for pid in range(1, 5) %}
                {% set pinfo = plant_profiles_map[pid] %}
                <div style="background: #ffffff; padding: 14px 16px; border-radius: 10px; border: 2px solid {% if active_plant_id == pid %}var(--accent-success){% else %}var(--border-color){% endif %};">
                    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px;">
                        <span style="font-weight: 700; font-size: 13.5px; color: var(--text-heading);">
                            Canastilla #{{ pid }}
                        </span>
                        {% if active_plant_id == pid %}
                            <span class="badge badge-success" style="font-size: 10.5px;">En Sesión</span>
                        {% endif %}
                    </div>

                    <label style="font-size: 11.5px; color: var(--text-muted); display: block; margin-bottom: 4px; font-weight: 600;">Especie Asignada:</label>
                    <select name="plant_{{ pid }}_crop" class="form-control" style="font-size: 13px; font-weight: 600;">
                        {% for profile in profiles %}
                            <option value="{{ profile.crop_type }}" {% if pinfo.crop_type == profile.crop_type %}selected{% endif %}>
                                {{ profile.display_name }}
```

```
                            </option>
                        {% endfor %}
                    </select>
                </div>
            {% endfor %}
        </div>

        <div style="display: flex; justify-content: flex-end; gap: 10px; align-items: center;">
            <button type="submit" class="btn btn-primary">
                <span class="material-icons-round">save</span>
                <span>Guardar Asignación de Canastillas</span>
            </button>
        </div>
    </form>
</div>

<!-- CARD 2: MODO DE VINCULACIÓN DE TELEMETRÍA Y MIGRACIÓN/SINCRONIZACIÓN -->
<div class="widget-card" style="max-width: 960px; margin: 0 auto 30px auto;">
    <div class="widget-header">
        <div class="widget-header-title" style="display: flex; align-items: center; gap: 12px;">
            <div class="kpi-icon-badge cyan" style="width: 38px; height: 38px;">
                <span class="material-icons-round" style="font-size: 22px;">hub</span>
            </div>
            <div>
                <h3>Modo de Telemetría y Sensores por Canastilla</h3>
                <p>Configura si el microclima ambiental se comparte en toda la torre o se aísla por cada nodo</p>
            </div>
        </div>
        <span class="badge {% if config.shared_telemetry %}badge-success{% else %}badge-primary{% endif %}" style="font-size: 12px;">
            Modo: {{ 'Telemetría Compartida de Torre' if config.shared_telemetry else 'Telemetría Aislada por Canastilla' }}
        </span>
    </div>

    <!-- FORMULARIO DE MODO DE TELEMETRÍA -->
    <form action="{{ url_for('dashboard.configuration_panel') }}" method="POST" style="margin-bottom: 20px;">
        <input type="hidden" name="action" value="update_telemetry_mode">

        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 16px; margin-bottom: 20px;">
            <!-- Opción 1: Telemetría Compartida -->
            <label style="background: var(--bg-card-subtle); border: 2px solid {% if config.shared_telemetry %}var(--accent-success){% else %}var(--border-color){% endif %}; padding: 16px; border-radius: 12px; cursor: pointer; display: flex; flex-direction: column; gap: 8px;">
                <div style="display: flex; align-items: center; justify-content: space-between;">
                    <div style="display: flex; align-items: center; gap: 8px;">
                        <input type="radio" name="telemetry_mode" value="shared_tower" {% if config.shared_telemetry %}checked{% endif %} style="accent-color: var(--accent-success); width: 18px; height: 18px;">
                        <strong style="font-size: 13.5px; color: var(--text-heading);">Telemetría Compartida de Torre</strong>
                    </div>
                    <span class="badge badge-success" style="font-size: 10px;">Recomendado</span>
                </div>
                <p style="font-size: 12px; color: var(--text-muted); margin: 0; line-height: 1.5;">
                    Todas las canastillas (#1 a #4) de la misma torre comparten los datos de los sensores ambientales (temperatura, humedad, luz UV y bombeo) captados por la antena en la torre.
                </p>
            </label>

            <!-- Opción 2: Telemetría Aislada -->
```

```
            <label style="background: var(--bg-card-subtle); border: 2px solid {% if not config.shared_telemetry %}var(--accent-primary){% else %}var(--border-color){% endif %}; padding: 16px; border-radius: 12px; cursor: pointer; display: flex; flex-direction: column; gap: 8px;">
                <div style="display: flex; align-items: center; justify-content: space-between;">
                    <div style="display: flex; align-items: center; gap: 8px;">
                        <input type="radio" name="telemetry_mode" value="isolated_node" {% if not config.shared_telemetry %}checked{% endif %} style="accent-color: var(--accent-primary); width: 18px; height: 18px;">
                        <strong style="font-size: 13.5px; color: var(--text-heading);">Telemetría Aislada por Canastilla</strong>
                    </div>
                    <span class="badge badge-outline" style="font-size: 10px;">Independiente</span>
                </div>
                <p style="font-size: 12px; color: var(--text-muted); margin: 0; line-height: 1.5;">
                    Cada canastilla maneja su propio flujo de telemetría y solo correlaciona lecturas de sensores generadas durante sus horarios de muestreo específicos.
                </p>
            </label>
        </div>

        <div style="display: flex; justify-content: flex-end;">
            <button type="submit" class="btn btn-primary btn-sm">
                <span class="material-icons-round">save</span>
                <span>Guardar Modo de Telemetría</span>
            </button>
        </div>
    </form>

    <!-- HERRAMIENTA DE SINCRONIZACIÓN / REPLICACIÓN DIRECTA -->
    <div style="border-top: 1px dashed var(--border-color); padding-top: 16px; display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 14px;">
        <div>
            <h4 style="font-size: 13px; font-weight: 700; color: var(--text-heading); margin-bottom: 2px;">
                Sincronización y Replicación de Telemetría
            </h4>
            <p style="font-size: 11.5px; color: var(--text-muted); margin: 0;">
                Vincula automáticamente las sesiones fotográficas de todas las canastillas con la lectura ambiental más cercana registrada en la torre.
            </p>
        </div>
        <button type="button" class="btn btn-outline btn-sm" id="btn-sync-telemetry" onclick="syncTowerTelemetry()">
            <span class="material-icons-round">sync</span>
            <span>Sincronizar Telemetría con Canastillas</span>
        </button>
    </div>
    <div id="sync-result-box" style="display: none; margin-top: 12px; padding: 10px 14px; border-radius: 8px; font-size: 12.5px; font-weight: 600;"></div>
</div>

<!-- CARD 3: EDICIÓN Y CALIBRACIÓN DE UMBRALES CROMÁTICOS DE SEGMENTACIÓN -->
<div class="widget-card" style="max-width: 960px; margin: 0 auto 30px auto;">
    <div class="widget-header">
        <div class="widget-header-title" style="display: flex; align-items: center; gap: 12px;">
            <div class="kpi-icon-badge blue" style="width: 38px; height: 38px;">
                <span class="material-icons-round" style="font-size: 22px;">tune</span>
            </div>
            <div>
                <h3>Calibración de Umbrales Cromáticos por Especie</h3>
                <p>Segmentación en espacios de color HSV y LAB para OpenCV / PlantCV</p>
            </div>
        </div>
        <span class="badge badge-success" style="font-size: 12px;">
            Especie en Calibración: {{ selected_profile.display_name }}
```

```
        </span>
    </div>

    <!-- SELECTOR DE ESPECIE A CALIBRAR -->
    <div style="background: var(--bg-card-subtle); padding: 18px 20px; border-radius: 12px; border: 1px solid var(--border-color); margin-bottom: 24px;">
        <label for="profile-selector" style="font-weight: 700; font-size: 13.5px; color: var(--text-heading); display: block; margin-bottom: 6px;">
            Seleccionar Especie Vegetal para Calibrar Umbrales:
        </label>
        <select id="profile-selector" class="form-control" style="font-size: 14px; font-weight: 600;" onchange="onProfileSelectChange(this.value)">
            {% for profile in profiles %}
                <option value="{{ profile.crop_type }}" {% if selected_profile.crop_type == profile.crop_type %}selected{% endif %}>
                    {{ profile.display_name }}
                </option>
            {% endfor %}
        </select>
        <small style="color: var(--text-muted); font-size: 11.5px; margin-top: 6px; display: block;">
            Al guardar, el sistema aplicará automáticamente estos umbrales al procesar fotos de esta especie.
        </small>
    </div>

    <!-- PESTAÑAS DE SELECCIÓN DE CÁMARA (CENITAL vs LATERAL) -->
    <div style="display: flex; gap: 8px; margin-bottom: 20px; border-bottom: 2px solid var(--border-color); padding-bottom: 4px; overflow-x: auto;">
        <button type="button" id="btn-tab-cenital" onclick="switchCameraTab('cenital')" style="background: none; border: none; padding: 10px 18px; border-radius: 8px 8px 0 0; font-size: 13.5px; font-weight: 700; cursor: pointer; display: flex; align-items: center; gap: 8px; color: var(--accent-success); border-bottom: 3px solid var(--accent-success); margin-bottom: -6px; transition: all 0.2s;">
            <span class="material-icons-round" style="font-size: 20px;">photo_camera</span>
            <span>Cámara Cenital (Vista Superior - Área Foliar)</span>
        </button>
        <button type="button" id="btn-tab-lateral" onclick="switchCameraTab('lateral')" style="background: none; border: none; padding: 10px 18px; border-radius: 8px 8px 0 0; font-size: 13.5px; font-weight: 700; cursor: pointer; display: flex; align-items: center; gap: 8px; color: var(--text-muted); border-bottom: 3px solid transparent; margin-bottom: -6px; transition: all 0.2s;">
            <span class="material-icons-round" style="font-size: 20px;">crop_portrait</span>
            <span>Cámara Lateral (Perfil Vertical - Altura & Tallo)</span>
        </button>
    </div>

    <form action="{{ url_for('dashboard.configuration_panel') }}" method="POST">
        <input type="hidden" name="action" value="update_profile">
        <input type="hidden" name="crop_type" id="prof-crop_type" value="{{ selected_profile.crop_type }}">

        <!-- PESTAÑA 1: CÁMARA CENITAL -->
        <div id="tab-content-cenital" class="camera-tab-content">
            <div style="background: rgba(16, 185, 129, 0.08); border: 1px solid rgba(16, 185, 129, 0.25); border-radius: 10px; padding: 12px 16px; margin-bottom: 20px; font-size: 12px; color: var(--text-heading); display: flex; align-items: center; gap: 10px;">
                <span class="material-icons-round text-success" style="font-size: 22px;">verified</span>
                <span><strong>Pipeline Cenital Activo:</strong> Optimizado para cuantificar cobertura foliar en dosel horizontal ($cm^2$), envoltura convexa y diagnóstico cromático de clorofila.</span>
            </div>

            <!-- SECCIÓN 1: ESPACIO DE COLOR HSV CENITAL -->
            <div style="margin-bottom: 24px;">
                <h4 style="font-size: 14px; font-weight: 700; color: var(--accent-success); margin-bottom: 14px; border-bottom: 1px solid var(--border-light); padding-bottom: 8px; display: flex; align-items: center; gap: 8px;">
                    <span class="material-icons-round" style="font-size: 18px;">palette</span>
                    <span>1. Umbrales HSV Cenital (Aislamiento de Follaje Horizontal)</span>
                </h4>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px;">
```

```
                    <div class="form-group">
                        <label for="h_min">Hue Mínimo (H): <span id="label-h_min" style="font-weight: 700; color: var(--accent-success);">{{ selected_profile.h_min }}
</span></label>
                        <input type="range" name="h_min" id="h_min" min="0" max="180" value="{{ selected_profile.h_min }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="h_max">Hue Máximo (H): <span id="label-h_max" style="font-weight: 700; color: var(--accent-success);">{{ selected_profile.h_max }}
</span></label>
                        <input type="range" name="h_max" id="h_max" min="0" max="180" value="{{ selected_profile.h_max }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="s_min">Saturación Mínima (S): <span id="label-s_min" style="font-weight: 700; color: var(--accent-success);">{{
selected_profile.s_min }}</span></label>
                        <input type="range" name="s_min" id="s_min" min="0" max="255" value="{{ selected_profile.s_min }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="s_max">Saturación Máxima (S): <span id="label-s_max" style="font-weight: 700; color: var(--accent-success);">{{
selected_profile.s_max }}</span></label>
                        <input type="range" name="s_max" id="s_max" min="0" max="255" value="{{ selected_profile.s_max }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="v_min">Brillo Mínimo (V): <span id="label-v_min" style="font-weight: 700; color: var(--accent-success);">{{ selected_profile.v_min }}
</span></label>
                        <input type="range" name="v_min" id="v_min" min="0" max="255" value="{{ selected_profile.v_min }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="v_max">Brillo Máximo (V): <span id="label-v_max" style="font-weight: 700; color: var(--accent-success);">{{ selected_profile.v_max }}
</span></label>
                        <input type="range" name="v_max" id="v_max" min="0" max="255" value="{{ selected_profile.v_max }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                </div>
            </div>

            <!-- SECCIÓN 2: ESPACIO DE COLOR LAB CENITAL -->
            <div style="margin-bottom: 24px;">
                <h4 style="font-size: 14px; font-weight: 700; color: var(--accent-primary); margin-bottom: 14px; border-bottom: 1px solid var(--border-light); padding-
bottom: 8px; display: flex; align-items: center; gap: 8px;">
                    <span class="material-icons-round" style="font-size: 18px;">auto_awesome</span>
                    <span>2. Umbrales LAB Cenital (Canal A y B de Clorofila)</span>
                </h4>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px;">
                    <div class="form-group">
                        <label for="a_min">Canal 'a' Mínimo: <span id="label-a_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.a_min }}
</span></label>
                        <input type="range" name="a_min" id="a_min" min="0" max="255" value="{{ selected_profile.a_min }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="a_max">Canal 'a' Máximo: <span id="label-a_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.a_max }}
</span></label>
                        <input type="range" name="a_max" id="a_max" min="0" max="255" value="{{ selected_profile.a_max }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="b_min">Canal 'b' Mínimo: <span id="label-b_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.b_min }}
</span></label>
                        <input type="range" name="b_min" id="b_min" min="0" max="255" value="{{ selected_profile.b_min }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">
```

```
                    </div>

                    <div class="form-group">

                        <label for="b_max">Canal 'b' Máximo: <span id="label-b_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.b_max }}
</span></label>

                        <input type="range" name="b_max" id="b_max" min="0" max="255" value="{{ selected_profile.b_max }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">

                    </div>

                    <div class="form-group">

                        <label for="l_min">Luminosidad 'L' Mínima: <span id="label-l_min" style="font-weight: 700; color: var(--accent-primary);">{{
selected_profile.l_min }}</span></label>

                        <input type="range" name="l_min" id="l_min" min="0" max="255" value="{{ selected_profile.l_min }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">

                    </div>

                    <div class="form-group">

                        <label for="l_max">Luminosidad 'L' Máxima: <span id="label-l_max" style="font-weight: 700; color: var(--accent-primary);">{{
selected_profile.l_max }}</span></label>

                        <input type="range" name="l_max" id="l_max" min="0" max="255" value="{{ selected_profile.l_max }}" oninput="updateRangeLabel(this)" style="width:
100%; cursor: pointer;">

                    </div>

                </div>

            </div>



            <!-- SECCIÓN 3: CALIBRADOR INTERACTIVO DE ROI CENITAL (ÁREA DE CANASTILLA Y PLANTA) -->

            <div style="margin-bottom: 24px; background: rgba(16, 185, 129, 0.04); border: 1.5px solid rgba(16, 185, 129, 0.25); border-radius: 12px; padding: 20px;">

                <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 14px; border-bottom: 1px solid var(--border-light);
padding-bottom: 10px;">

                    <h4 style="font-size: 14.5px; font-weight: 700; color: var(--accent-success); margin: 0; display: flex; align-items: center; gap: 8px;">

                        <span class="material-icons-round" style="font-size: 20px;">adjust</span>

                        <span>3. Calibrador Visual Dinámico de ROI (Región de Interés)</span>

                    </h4>

                    <span class="badge badge-success" style="font-size: 11px;">Ajuste con Mouse y Tiempo Real</span>

                </div>



                <p style="font-size: 12.5px; color: var(--text-muted); margin-bottom: 16px; line-height: 1.5;">

                    Haz <strong>clic o arrastra</strong> sobre la imagen para mover el centro del círculo hacia el tallo de la canastilla. Ajusta el
<strong>radio</strong> para cubrir el follaje sin cortar hojas grandes. El filtro inteligente descartará automáticamente hojas de canastillas vecinas.

                </p>



                <div style="display: grid; grid-template-columns: minmax(320px, 1fr) 300px; gap: 20px; align-items: start;">

                    <!-- Canvas Interactivo con la Foto Cenital -->

                    <div style="position: relative; background: #0f172a; border-radius: 10px; overflow: hidden; border: 2px solid #334155; text-align: center; box-
shadow: 0 4px 12px rgba(0,0,0,0.15);">

                        <canvas id="roi-canvas" style="display: block; max-width: 100%; height: auto; margin: 0 auto; cursor: crosshair;"></canvas>

                        <div id="roi-canvas-overlay-hint" style="position: absolute; bottom: 8px; left: 8px; right: 8px; background: rgba(0,0,0,0.7); backdrop-filter:
blur(4px); color: #f8fafc; font-size: 11px; padding: 4px 10px; border-radius: 6px; pointer-events: none; display: flex; justify-content: space-between;">

                            <span>Clic/Arrastrar: Mover Centro (X, Y)</span>

                            <span id="roi-coords-display">X: 50.0% | Y: 44.1% | R: 14.5%</span>

                        </div>

                    </div>



                    <!-- Controles y Formulario de Guardado de ROI -->

                    <div style="background: #ffffff; padding: 16px; border-radius: 10px; border: 1px solid var(--border-color); display: flex; flex-direction: column;
gap: 14px;">

                        <div class="form-group" style="margin: 0;">

                            <label for="roi_radius_slider" style="font-size: 12px; font-weight: 700; color: var(--text-heading); display: flex; justify-content: space-
between;">

                                <span>Radio del Círculo (Cobertura):</span>

                                <span id="label-roi_radius" style="color: var(--accent-success); font-weight: 800;">{{ '%.1f'|format((config.cenital_roi_radius or 0.145)
* 100) }}%</span>

                            </label>

                            <input type="range" id="roi_radius_slider" min="5" max="35" step="0.5" value="{{ ((config.cenital_roi_radius or 0.145) * 100)|round(1) }}"
oninput="onRoiRadiusChange(this.value)" style="width: 100%; cursor: pointer; accent-color: var(--accent-success);">
```

```
                            <small style="color: var(--text-muted); font-size: 10.5px; display: block; margin-top: 2px;">Agranda si la planta sobrepasa la canastilla.</small>
                        </div>

                        <div class="form-group" style="margin: 0;">
                            <label style="font-size: 12px; font-weight: 700; color: var(--text-heading); display: block; margin-bottom: 6px;">
                                Modo de Filtrado de Vecinos:
                            </label>
                            <div style="display: flex; flex-direction: column; gap: 6px;">
                                <label style="display: flex; align-items: center; gap: 8px; font-size: 11.5px; cursor: pointer;">
                                    <input type="radio" name="roi_mode_radio" value="dynamic_conn" {% if (config.cenital_roi_mode or 'dynamic_conn') == 'dynamic_conn' %}checked{% endif %} onchange="onRoiModeChange(this.value)" style="accent-color: var(--accent-success);">
                                    <span><strong>Conectividad Inteligente</strong> (Ignora hojas ajenas)</span>
                                </label>
                                <label style="display: flex; align-items: center; gap: 8px; font-size: 11.5px; cursor: pointer;">
                                    <input type="radio" name="roi_mode_radio" value="strict_circle" {% if config.cenital_roi_mode == 'strict_circle' %}checked{% endif %} onchange="onRoiModeChange(this.value)" style="accent-color: var(--accent-success);">
                                    <span><strong>Círculo Estricto</strong> (Todo lo que esté dentro)</span>
                                </label>
                            </div>
                        </div>

                        <div style="display: flex; gap: 8px; margin-top: 4px;">
                            <button type="button" class="btn btn-outline btn-sm" onclick="resetRoiToDefault()" style="flex: 1; font-size: 11.5px;">
                                <span class="material-icons-round" style="font-size: 16px;">restart_alt</span>
                                <span>Restablecer</span>
                            </button>
                            <button type="button" class="btn btn-success btn-sm" onclick="saveRoiCalibration()" id="btn-save-roi" style="flex: 1.2; font-size: 11.5px; font-weight: 700;">
                                <span class="material-icons-round" style="font-size: 16px;">save</span>
                                <span>Guardar ROI</span>
                            </button>
                        </div>

                        <div id="roi-save-status" style="display: none; font-size: 11.5px; padding: 8px; border-radius: 6px; text-align: center;"></div>
                    </div>
                </div>
            </div>

            <!-- SECCIÓN 4: MORFOMETRÍA CENITAL -->
            <div style="margin-bottom: 24px;">
                <h4 style="font-size: 14px; font-weight: 700; color: var(--accent-warning); margin-bottom: 14px; border-bottom: 1px solid var(--border-light); padding-bottom: 8px; display: flex; align-items: center; gap: 8px;">
                    <span class="material-icons-round" style="font-size: 18px;">straighten</span>
                    <span>4. Factor de Conversión Morfométrica Cenital</span>
                </h4>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px; align-items: center;">
                    <div class="form-group">
                        <label for="pixel_to_cm_ratio">Factor de Escala Cenital (cm por píxel):</label>
                        <input type="number" step="any" name="pixel_to_cm_ratio" id="pixel_to_cm_ratio" class="form-control" value="{{ selected_profile.pixel_to_cm_ratio }}" required>
                        <small style="color: var(--text-muted); font-size: 11px;">Escala horizontal típica para cálculo de área foliar: ~0.038 cm/px</small>
                    </div>
                </div>
            </div>
        </div>

```

```
        <!-- PESTAÑA 2: CÁMARA LATERAL -->

        <div id="tab-content-lateral" class="camera-tab-content" style="display: none;">

            <div style="background: rgba(37, 99, 235, 0.08); border: 1px solid rgba(37, 99, 235, 0.25); border-radius: 10px; padding: 12px 16px; margin-bottom: 20px; font-size: 12px; color: var(--text-heading); display: flex; align-items: center; gap: 10px;">

                <span class="material-icons-round text-primary" style="font-size: 22px;">tune</span>

                <span><strong>Pipeline Lateral Independiente:</strong> Configurado con filtros morfológicos y descarte de ruido en los bordes de la maceta para medir altura real ($H$) y diámetro basal ($D$).</span>

            </div>



            <!-- SECCIÓN 1: ESPACIO DE COLOR HSV LATERAL -->

            <div style="margin-bottom: 24px;">

                <h4 style="font-size: 14px; font-weight: 700; color: var(--accent-primary); margin-bottom: 14px; border-bottom: 1px solid var(--border-light); padding-bottom: 8px; display: flex; align-items: center; gap: 8px;">

                    <span class="material-icons-round" style="font-size: 18px;">palette</span>

                    <span>1. Umbrales HSV Lateral (Perfil Vertical)</span>

                </h4>



                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px;">

                    <div class="form-group">

                        <label for="lat_h_min">Hue Mínimo (H): <span id="label-lat_h_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_h_min or selected_profile.h_min }}</span></label>

                        <input type="range" name="lat_h_min" id="lat_h_min" min="0" max="180" value="{{ selected_profile.lat_h_min or selected_profile.h_min }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">

                    </div>

                    <div class="form-group">

                        <label for="lat_h_max">Hue Máximo (H): <span id="label-lat_h_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_h_max or selected_profile.h_max }}</span></label>

                        <input type="range" name="lat_h_max" id="lat_h_max" min="0" max="180" value="{{ selected_profile.lat_h_max or selected_profile.h_max }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">

                    </div>

                    <div class="form-group">

                        <label for="lat_s_min">Saturación Mínima (S): <span id="label-lat_s_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_s_min or selected_profile.s_min }}</span></label>

                        <input type="range" name="lat_s_min" id="lat_s_min" min="0" max="255" value="{{ selected_profile.lat_s_min or selected_profile.s_min }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">

                    </div>

                    <div class="form-group">

                        <label for="lat_s_max">Saturación Máxima (S): <span id="label-lat_s_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_s_max or selected_profile.s_max }}</span></label>

                        <input type="range" name="lat_s_max" id="lat_s_max" min="0" max="255" value="{{ selected_profile.lat_s_max or selected_profile.s_max }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">

                    </div>

                    <div class="form-group">

                        <label for="lat_v_min">Brillo Mínimo (V): <span id="label-lat_v_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_v_min or selected_profile.v_min }}</span></label>

                        <input type="range" name="lat_v_min" id="lat_v_min" min="0" max="255" value="{{ selected_profile.lat_v_min or selected_profile.v_min }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">

                    </div>

                    <div class="form-group">

                        <label for="lat_v_max">Brillo Máximo (V): <span id="label-lat_v_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_v_max or selected_profile.v_max }}</span></label>

                        <input type="range" name="lat_v_max" id="lat_v_max" min="0" max="255" value="{{ selected_profile.lat_v_max or selected_profile.v_max }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">

                    </div>

                </div>

            </div>



            <!-- SECCIÓN 2: ESPACIO DE COLOR LAB LATERAL -->

            <div style="margin-bottom: 24px;">

                <h4 style="font-size: 14px; font-weight: 700; color: var(--accent-primary); margin-bottom: 14px; border-bottom: 1px solid var(--border-light); padding-bottom: 8px; display: flex; align-items: center; gap: 8px;">

                    <span class="material-icons-round" style="font-size: 18px;">auto_awesome</span>

                    <span>2. Umbrales LAB Lateral</span>
```

```
                </h4>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px;">
                    <div class="form-group">
                        <label for="lat_a_min">Canal 'a' Mínimo: <span id="label-lat_a_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_a_min or selected_profile.a_min }}</span></label>
                        <input type="range" name="lat_a_min" id="lat_a_min" min="0" max="255" value="{{ selected_profile.lat_a_min or selected_profile.a_min }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="lat_a_max">Canal 'a' Máximo: <span id="label-lat_a_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_a_max or selected_profile.a_max }}</span></label>
                        <input type="range" name="lat_a_max" id="lat_a_max" min="0" max="255" value="{{ selected_profile.lat_a_max or selected_profile.a_max }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="lat_b_min">Canal 'b' Mínimo: <span id="label-lat_b_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_b_min or selected_profile.b_min }}</span></label>
                        <input type="range" name="lat_b_min" id="lat_b_min" min="0" max="255" value="{{ selected_profile.lat_b_min or selected_profile.b_min }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="lat_b_max">Canal 'b' Máximo: <span id="label-lat_b_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_b_max or selected_profile.b_max }}</span></label>
                        <input type="range" name="lat_b_max" id="lat_b_max" min="0" max="255" value="{{ selected_profile.lat_b_max or selected_profile.b_max }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="lat_l_min">Luminosidad 'L' Mínima: <span id="label-lat_l_min" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_l_min or selected_profile.l_min }}</span></label>
                        <input type="range" name="lat_l_min" id="lat_l_min" min="0" max="255" value="{{ selected_profile.lat_l_min or selected_profile.l_min }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">
                    </div>
                    <div class="form-group">
                        <label for="lat_l_max">Luminosidad 'L' Máxima: <span id="label-lat_l_max" style="font-weight: 700; color: var(--accent-primary);">{{ selected_profile.lat_l_max or selected_profile.l_max }}</span></label>
                        <input type="range" name="lat_l_max" id="lat_l_max" min="0" max="255" value="{{ selected_profile.lat_l_max or selected_profile.l_max }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">
                    </div>
                </div>
            </div>

            <!-- SECCIÓN 3: CALIBRADOR INTERACTIVO DE LÍNEA BASE LATERAL Y ZONA DE TALLO -->
            <div style="margin-bottom: 24px; background: rgba(37, 99, 235, 0.04); border: 1.5px solid rgba(37, 99, 235, 0.25); border-radius: 12px; padding: 20px;">
                <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 14px; border-bottom: 1px solid var(--border-light); padding-bottom: 10px;">
                    <h4 style="font-size: 14.5px; font-weight: 700; color: var(--accent-primary); margin: 0; display: flex; align-items: center; gap: 8px;">
                        <span class="material-icons-round" style="font-size: 20px;">straighten</span>
                        <span>3. Calibrador Visual de Línea Base y Zona de Medición de Tallo (Bordes & Escaneo)</span>
                    </h4>
                    <span class="badge badge-primary" style="font-size: 11px;">Línea Base + Cuadro de Tallo Móvil</span>
                </div>

                <p style="font-size: 12.5px; color: var(--text-muted); margin-bottom: 16px; line-height: 1.5;">
                    Calibra visualmente dos elementos clave:
                    <br>1. <strong style="color: #ef4444;">Línea Roja (Base de Maceta):</strong> Nivel exacto de inserción del tallo en el sustrato para medir la altura ($H$).
                    <br>2. <strong style="color: #f59e0b;">Recuadro Ámbar/Cian (Zona de Tallo):</strong> Franja ajustable donde OpenCV analiza los bordes cilíndricos y transiciones de gradiente horizontal (sin importar si el tallo es verde, café o leñoso) para calcular el diámetro ($D$) en milímetros.
                </p>

                <div style="display: grid; grid-template-columns: minmax(320px, 1fr) 340px; gap: 20px; align-items: start;">
```

```
451                     <!-- Canvas Interactivo con la Foto Lateral -->
452                     <div style="position: relative; background: #0f172a; border-radius: 10px; overflow: hidden; border: 2px solid #334155; text-align: center; box-
    shadow: 0 4px 12px rgba(0,0,0,0.15);">
453                         <canvas id="lateral-canvas" style="display: block; max-width: 100%; height: auto; margin: 0 auto; cursor: crosshair;"></canvas>
454                         <div style="position: absolute; bottom: 8px; left: 8px; right: 8px; background: rgba(0,0,0,0.75); backdrop-filter: blur(4px); color: #f8fafc;
    font-size: 10.5px; padding: 5px 10px; border-radius: 6px; pointer-events: none; display: flex; justify-content: space-between; gap: 6px;">
455                             <span>↕️ Arrastrar Línea o Cuadro de Tallo</span>
456                             <span id="lateral-coords-display">Base: {{ '%.1f'|format((config.lateral_baseline_y or 0.72) * 100) }}% | Tallo: {{
    '%.1f'|format((config.lateral_stem_roi_y or 0.66) * 100) }}%</span>
457                         </div>
458                     </div>
459
460                     <!-- Controles y Formulario de Guardado de Línea Base y Tallo Lateral -->
461                     <div style="background: #ffffff; padding: 16px; border-radius: 10px; border: 1px solid var(--border-color); display: flex; flex-direction: column;
    gap: 14px;">
462
463                         <!-- Control de Línea Base -->
464                         <div class="form-group" style="margin: 0; background: rgba(239, 68, 68, 0.05); padding: 10px; border-radius: 8px; border: 1px solid rgba(239, 68,
    68, 0.2);">
465                             <label for="lateral_baseline_slider" style="font-size: 11.5px; font-weight: 700; color: #b91c1c; display: flex; justify-content: space-
    between;">
466                                 <span>1. Posición Línea Base (Maceta Y):</span>
467                                 <span id="label-lateral_baseline" style="font-weight: 800;">{{ '%.1f'|format((config.lateral_baseline_y or 0.72) * 100) }}%</span>
468                             </label>
469                             <input type="range" id="lateral_baseline_slider" min="20" max="95" step="0.5" value="{{ ((config.lateral_baseline_y or 0.72) * 100)|round(1)
    }}" oninput="onLateralBaselineSliderChange(this.value)" style="width: 100%; cursor: pointer; accent-color: #ef4444;">
470                         </div>
471
472                         <!-- Controles de Cuadro / Franja de Tallo -->
473                         <div style="background: rgba(245, 158, 11, 0.05); padding: 12px; border-radius: 8px; border: 1px solid rgba(245, 158, 11, 0.25); display: flex;
    flex-direction: column; gap: 10px;">
474                             <strong style="font-size: 12px; color: #b45309; display: flex; align-items: center; gap: 6px;">
475                                 <span class="material-icons-round" style="font-size: 16px;">crop</span>
476                                 <span>2. Zona de Escaneo de Tallo (Bordes)</span>
477                             </strong>
478
479                             <div class="form-group" style="margin: 0;">
480                                 <label for="lateral_stem_y_slider" style="font-size: 11px; font-weight: 700; color: var(--text-heading); display: flex; justify-content:
    space-between;">
481                                     <span>Altura del Escaneo (Y):</span>
482                                     <span id="label-stem_roi_y" style="color: #d97706; font-weight: 800;">{{ '%.1f'|format((config.lateral_stem_roi_y or 0.66) * 100) }}%
    </span>
483                                 </label>
484                                 <input type="range" id="lateral_stem_y_slider" min="15" max="90" step="0.5" value="{{ ((config.lateral_stem_roi_y or 0.66) *
    100)|round(1) }}" oninput="onStemSliderChange('y', this.value)" style="width: 100%; cursor: pointer; accent-color: #f59e0b;">
485                             </div>
486
487                             <div class="form-group" style="margin: 0;">
488                                 <label for="lateral_stem_x_slider" style="font-size: 11px; font-weight: 700; color: var(--text-heading); display: flex; justify-content:
    space-between;">
489                                     <span>Centro Horizontal del Tallo (X):</span>
490                                     <span id="label-stem_roi_x" style="color: #d97706; font-weight: 800;">{{ '%.1f'|format((config.lateral_stem_roi_x or 0.50) * 100) }}%
    </span>
491                                 </label>
492                                 <input type="range" id="lateral_stem_x_slider" min="10" max="90" step="0.5" value="{{ ((config.lateral_stem_roi_x or 0.50) *
    100)|round(1) }}" oninput="onStemSliderChange('x', this.value)" style="width: 100%; cursor: pointer; accent-color: #f59e0b;">
493                             </div>
494
495                             <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 8px;">
496                                 <div class="form-group" style="margin: 0;">
497                                     <label for="lateral_stem_w_slider" style="font-size: 10.5px; font-weight: 700; color: var(--text-heading); display: flex; justify-
    content: space-between;">
498                                         <span>Ancho (W):</span>
```

```
                                    <span id="label-stem_roi_w" style="color: #d97706;">{{ '%.0f'|format((config.lateral_stem_roi_width or 0.20) * 100) }}%</span>
                                </label>
                                <input type="range" id="lateral_stem_w_slider" min="6" max="45" step="1" value="{{ ((config.lateral_stem_roi_width or 0.20) * 100)|round(0) }}" oninput="onStemSliderChange('w', this.value)" style="width: 100%; cursor: pointer; accent-color: #f59e0b;">
                            </div>
                            <div class="form-group" style="margin: 0;">
                                <label for="lateral_stem_h_slider" style="font-size: 10.5px; font-weight: 700; color: var(--text-heading); display: flex; justify-content: space-between;">
                                    <span>Alto Franja (H):</span>
                                    <span id="label-stem_roi_h" style="color: #d97706;">{{ '%.0f'|format((config.lateral_stem_roi_height or 0.05) * 100) }}%</span>
                                </label>
                                <input type="range" id="lateral_stem_h_slider" min="2" max="15" step="0.5" value="{{ ((config.lateral_stem_roi_height or 0.05) * 100)|round(1) }}" oninput="onStemSliderChange('h', this.value)" style="width: 100%; cursor: pointer; accent-color: #f59e0b;">
                            </div>
                        </div>
                    </div>

                    <div style="display: flex; gap: 8px; margin-top: 4px;">
                        <button type="button" class="btn btn-outline btn-sm" onclick="resetLateralCalibrationToDefault()" style="flex: 1; font-size: 11.5px;">
                            <span class="material-icons-round" style="font-size: 16px;">restart_alt</span>
                            <span>Restablecer</span>
                        </button>
                        <button type="button" class="btn btn-primary btn-sm" onclick="saveLateralBaselineCalibration()" id="btn-save-lateral-baseline" style="flex: 1.3; font-size: 11.5px; font-weight: 700;">
                            <span class="material-icons-round" style="font-size: 16px;">save</span>
                            <span>Guardar Calibración Lateral</span>
                        </button>
                    </div>

                    <div id="lateral-save-status" style="display: none; font-size: 11.5px; padding: 8px; border-radius: 6px; text-align: center;"></div>
                </div>
            </div>

            <!-- SECCIÓN 4: FILTROS DE RUIDO Y BORDES DE MACETA -->
            <div style="margin-bottom: 24px;">
                <h4 style="font-size: 14px; font-weight: 700; color: var(--accent-warning); margin-bottom: 14px; border-bottom: 1px solid var(--border-light); padding-bottom: 8px; display: flex; align-items: center; gap: 8px;">
                    <span class="material-icons-round" style="font-size: 18px;">filter_alt</span>
                    <span>4. Filtros Morfológicos y Morfometría Lateral</span>
                </h4>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px; margin-bottom: 16px;">
                    <div class="form-group">
                        <label for="lat_min_area">Área Mínima de Contorno (Píxeles): <span id="label-lat_min_area" style="font-weight: 700; color: var(--accent-warning);">{{ selected_profile.lat_min_area or 60 }}</span> px</label>
                        <input type="range" name="lat_min_area" id="lat_min_area" min="10" max="300" step="5" value="{{ selected_profile.lat_min_area or 60 }}" oninput="updateRangeLabel(this)" style="width: 100%; cursor: pointer;">
                        <small style="color: var(--text-muted); font-size: 11px;">Descarta manchas pequeñas y motas de polvo en el borde de la maceta (Recomendado: 60-100 px).</small>
                    </div>

                    <div class="form-group" style="background: var(--bg-card-subtle); padding: 14px; border-radius: 10px; border: 1px solid var(--border-color); display: flex; align-items: center; gap: 12px;">
                        <input type="checkbox" name="lat_filter_isolated" id="lat_filter_isolated" value="true" {% if selected_profile.lat_filter_isolated is none or selected_profile.lat_filter_isolated %}checked{% endif %} style="width: 20px; height: 20px; cursor: pointer; accent-color: var(--accent-primary);">
                        <div>
                            <label for="lat_filter_isolated" style="font-weight: 700; font-size: 13px; cursor: pointer; margin: 0; color: var(--text-heading); display: block;">Filtro de Cluster Central de Planta</label>
                            <span style="font-size: 11px; color: var(--text-muted); display: block; margin-top: 2px;">Ignora contornos lejanos en los bordes de la maceta y asegura que la línea de altura ($H$) mida la planta real.</span>
```

```
                        </div>
                    </div>
                </div>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px; margin-bottom: 16px;">
                    <div class="form-group" style="background: rgba(239, 68, 68, 0.04); padding: 14px; border-radius: 10px; border: 1px solid rgba(239, 68, 68, 0.2); display: flex; align-items: center; gap: 12px;">
                        <input type="checkbox" name="lat_draw_contours" id="lat_draw_contours" value="true" {% if selected_profile.lat_draw_contours %}checked{% endif %} style="width: 20px; height: 20px; cursor: pointer; accent-color: #ef4444;">
                        <div>
                            <label for="lat_draw_contours" style="font-weight: 700; font-size: 13px; cursor: pointer; margin: 0; color: #b91c1c; display: block;">Dibujar Contornos Rojos sobre las Hojas</label>
                            <span style="font-size: 11px; color: var(--text-muted); display: block; margin-top: 2px;">Desmarcado (Recomendado): La foto procesada se muestra limpia con solo la Altura ($H$), Línea Base y Tallo ($D$), sin tapar el tejido con líneas rojas.</span>
                        </div>
                    </div>

                    <div class="form-group" style="background: var(--bg-card-subtle); padding: 14px; border-radius: 10px; border: 1px solid var(--border-color); display: flex; align-items: center; gap: 12px;">
                        <input type="checkbox" name="has_stem" id="has_stem" value="true" {% if selected_profile.has_stem %}checked{% endif %} style="width: 20px; height: 20px; cursor: pointer; accent-color: var(--accent-success);">
                        <div>
                            <label for="has_stem" style="font-weight: 700; font-size: 13px; cursor: pointer; margin: 0; color: var(--text-heading); display: block;">Medir Grosor / Diámetro del Tallo ($D$)</label>
                            <span style="font-size: 11px; color: var(--text-muted); display: block; margin-top: 2px;">Activa el escaneo por gradiente en el cuello basal para calcular el diámetro en milímetros ($mm$).</span>
                        </div>
                    </div>
                </div>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 18px; align-items: center;">
                    <div class="form-group">
                        <label for="lat_pixel_to_cm_ratio">Factor de Escala Lateral (cm por píxel vertical):</label>
                        <input type="number" step="any" name="lat_pixel_to_cm_ratio" id="lat_pixel_to_cm_ratio" class="form-control" value="{{ selected_profile.lat_pixel_to_cm_ratio or selected_profile.pixel_to_cm_ratio }}" required>
                        <small style="color: var(--text-muted); font-size: 11px;">Escala vertical para altura y grosor de tallo: ~0.038 cm/px</small>
                    </div>
                </div>
            </div>
        </div>

        <div style="display: flex; justify-content: flex-end; gap: 10px; margin-top: 20px; border-top: 1px solid var(--border-color); padding-top: 16px;">
            <button type="submit" class="btn btn-primary">
                <span class="material-icons-round">save</span>
                <span>Guardar Umbrales y Calibración de {{ selected_profile.display_name }}</span>
            </button>
        </div>
    </form>
</div>

{% endblock %}

{% block scripts %}
<script>
    function switchCameraTab(tabName) {
        const btnCenital = document.getElementById('btn-tab-cenital');
        const btnLateral = document.getElementById('btn-tab-lateral');
        const contentCenital = document.getElementById('tab-content-cenital');
        const contentLateral = document.getElementById('tab-content-lateral');
```

```
598
599         if (tabName === 'cenital') {
600             btnCenital.style.color = 'var(--accent-success)';
601             btnCenital.style.borderBottomColor = 'var(--accent-success)';
602             btnLateral.style.color = 'var(--text-muted)';
603             btnLateral.style.borderBottomColor = 'transparent';
604             contentCenital.style.display = 'block';
605             contentLateral.style.display = 'none';
606         } else {
607             btnLateral.style.color = 'var(--accent-primary)';
608             btnLateral.style.borderBottomColor = 'var(--accent-primary)';
609             btnCenital.style.color = 'var(--text-muted)';
610             btnCenital.style.borderBottomColor = 'transparent';
611             contentCenital.style.display = 'none';
612             contentLateral.style.display = 'block';
613             setTimeout(() => { drawLateral(); }, 50);
614         }
615     }
616
617     function updateRangeLabel(input) {
618         const label = document.getElementById('label-' + input.id);
619         if (label) {
620             label.innerText = input.value;
621         }
622     }
623
624     function onProfileSelectChange(cropType) {
625         window.location.href = "{{ url_for('dashboard.configuration_panel') }}?crop_type=" + cropType;
626     }
627
628     async function syncTowerTelemetry() {
629         const btn = document.getElementById('btn-sync-telemetry');
630         const box = document.getElementById('sync-result-box');
631         if (!btn || !box) return;
632
633         btn.disabled = true;
634         btn.innerHTML = '<span class="material-icons-round spinner">sync</span> <span>Sincronizando...</span>';
635
636         try {
637             const resp = await fetch('/api/telemetry/sync_tower', {
638                 method: 'POST',
639                 headers: { 'Content-Type': 'application/json' }
640             });
641             const data = await resp.json();
642
643             box.style.display = 'block';
644             if (data.status === 'success') {
645                 box.style.background = 'rgba(16, 185, 129, 0.12)';
646                 box.style.border = '1px solid rgba(16, 185, 129, 0.3)';
647                 box.style.color = '#065f46';
648                 box.innerHTML = `<span class="material-icons-round" style="vertical-align: middle; margin-right: 6px;">check_circle</span> ${data.message}`;
649             } else {
650                 box.style.background = 'rgba(239, 68, 68, 0.12)';
651                 box.style.border = '1px solid rgba(239, 68, 68, 0.3)';
652                 box.style.color = '#991b1b';
```

```
                box.innerHTML = `<span class="material-icons-round" style="vertical-align: middle; margin-right: 6px;">error</span> ${data.message || 'Error al sincronizar telemetría'}`;
            }
        } catch (err) {
            box.style.display = 'block';
            box.style.background = 'rgba(239, 68, 68, 0.12)';
            box.style.border = '1px solid rgba(239, 68, 68, 0.3)';
            box.style.color = '#991b1b';
            box.innerHTML = `<span class="material-icons-round" style="vertical-align: middle; margin-right: 6px;">error</span> Error de conexión al servidor: ${err.message}`;
        } finally {
            btn.disabled = false;
            btn.innerHTML = '<span class="material-icons-round">sync</span> <span>Sincronizar Telemetría con Canastillas</span>';
        }
    }

    // ========================================
    // CALIBRADOR INTERACTIVO DE ROI CENITAL
    // ========================================
    let roiState = {
        cx: parseFloat("{{ config.cenital_roi_cx or 0.4955 }}"),
        cy: parseFloat("{{ config.cenital_roi_cy or 0.4408 }}"),
        radius: parseFloat("{{ config.cenital_roi_radius or 0.1450 }}"),
        mode: "{{ config.cenital_roi_mode or 'dynamic_conn' }}",
        isDragging: false
    };

    const canvas = document.getElementById('roi-canvas');
    const ctx = canvas ? canvas.getContext('2d') : null;
    const bgImage = new Image();
    const latestCenitalUrl = "{{ url_for('static', filename=latest_cenital_img) if latest_cenital_img else '' }}";

    function initRoiCanvas() {
        if (!canvas || !ctx) return;

        if (latestCenitalUrl) {
            bgImage.src = latestCenitalUrl;
            bgImage.onload = () => {
                canvas.width = bgImage.naturalWidth || 640;
                canvas.height = bgImage.naturalHeight || 480;
                drawRoi();
            };
            bgImage.onerror = () => {
                drawFallbackCanvas();
            };
        } else {
            drawFallbackCanvas();
        }

        // Eventos de Mouse sobre el Canvas
        canvas.addEventListener('mousedown', (e) => {
            roiState.isDragging = true;
            updateRoiCenterFromEvent(e);
        });

        window.addEventListener('mousemove', (e) => {
```

```
            if (roiState.isDragging) {
                updateRoiCenterFromEvent(e);
            }
        });

        window.addEventListener('mouseup', () => {
            roiState.isDragging = false;
        });

        // Touch para pantallas táctiles
        canvas.addEventListener('touchstart', (e) => {
            roiState.isDragging = true;
            if (e.touches.length > 0) updateRoiCenterFromEvent(e.touches[0]);
            e.preventDefault();
        }, { passive: false });

        canvas.addEventListener('touchmove', (e) => {
            if (roiState.isDragging && e.touches.length > 0) {
                updateRoiCenterFromEvent(e.touches[0]);
            }
            e.preventDefault();
        }, { passive: false });

        canvas.addEventListener('touchend', () => {
            roiState.isDragging = false;
        });
    }

    function drawFallbackCanvas() {
        canvas.width = 640;
        canvas.height = 480;
        drawRoi();
    }

    function updateRoiCenterFromEvent(e) {
        if (!canvas) return;
        const rect = canvas.getBoundingClientRect();
        const scaleX = canvas.width / rect.width;
        const scaleY = canvas.height / rect.height;

        const x = (e.clientX - rect.left) * scaleX;
        const y = (e.clientY - rect.top) * scaleY;

        roiState.cx = Math.max(0.05, Math.min(0.95, x / canvas.width));
        roiState.cy = Math.max(0.05, Math.min(0.95, y / canvas.height));

        drawRoi();
        updateRoiLabels();
    }

    function drawRoi() {
        if (!ctx || !canvas) return;
        const w = canvas.width;
        const h = canvas.height;
```

```
        ctx.clearRect(0, 0, w, h);

        if (bgImage.complete && bgImage.naturalWidth > 0) {
            ctx.drawImage(bgImage, 0, 0, w, h);
        } else {
            // Fondo simulado de la torre
            ctx.fillStyle = '#1e293b';
            ctx.fillRect(0, 0, w, h);
            ctx.fillStyle = '#f8fafc';
            ctx.font = '14px Inter, sans-serif';
            ctx.textAlign = 'center';
            ctx.fillText('No hay fotos cenitales capturadas aún. Usa el patrón de referencia:', w / 2, h / 2 - 40);
        }

        const px = roiState.cx * w;
        const py = roiState.cy * h;
        const minDim = Math.min(w, h);
        const r = roiState.radius * minDim;

        // 1. Zona oscurecida fuera del ROI (Máscara translúcida)
        ctx.save();
        ctx.fillStyle = 'rgba(0, 0, 0, 0.45)';
        ctx.fillRect(0, 0, w, h);

        // Recortar círculo transparente
        ctx.globalCompositeOperation = 'destination-out';
        ctx.beginPath();
        ctx.arc(px, py, r, 0, Math.PI * 2, false);
        ctx.fill();
        ctx.restore();

        // 2. Anillo del ROI (Verde / Cian Neón)
        ctx.save();
        ctx.beginPath();
        ctx.arc(px, py, r, 0, Math.PI * 2, false);
        ctx.lineWidth = 3;
        ctx.strokeStyle = '#10b981';
        ctx.shadowColor = '#10b981';
        ctx.shadowBlur = 10;
        ctx.stroke();

        // Núcleo central mínimo (~45% del radio)
        ctx.beginPath();
        ctx.arc(px, py, r * 0.45, 0, Math.PI * 2, false);
        ctx.lineWidth = 1.5;
        ctx.setLineDash([4, 4]);
        ctx.strokeStyle = '#38bdf8';
        ctx.stroke();

        // Cruz Central
        ctx.setLineDash([]);
        ctx.lineWidth = 2;
        ctx.strokeStyle = '#22c55e';
        const cl = 16;
        ctx.beginPath();
```

```
        ctx.moveTo(px - cl, py); ctx.lineTo(px + cl, py);
        ctx.moveTo(px, py - cl); ctx.lineTo(px, py + cl);
        ctx.stroke();

        // Etiqueta
        ctx.fillStyle = '#10b981';
        ctx.font = 'bold 12px Inter, sans-serif';
        ctx.textAlign = 'center';
        ctx.shadowColor = '#000000';
        ctx.shadowBlur = 4;
        ctx.fillText('ROI (CANASTILLA)', px, Math.max(20, py - r - 8));
        ctx.restore();
    }

    function onRoiRadiusChange(val) {
        roiState.radius = parseFloat(val) / 100.0;
        const label = document.getElementById('label-roi_radius');
        if (label) label.innerText = parseFloat(val).toFixed(1) + '%';
        drawRoi();
        updateRoiLabels();
    }

    function onRoiModeChange(mode) {
        roiState.mode = mode;
    }

    function updateRoiLabels() {
        const coords = document.getElementById('roi-coords-display');
        if (coords) {
            coords.innerText = `X: ${(roiState.cx * 100).toFixed(1)}% | Y: ${(roiState.cy * 100).toFixed(1)}% | R: ${(roiState.radius * 100).toFixed(1)}%`;
        }
    }

    function resetRoiToDefault() {
        roiState.cx = 0.4955;
        roiState.cy = 0.4408;
        roiState.radius = 0.1450;
        roiState.mode = 'dynamic_conn';

        const slider = document.getElementById('roi_radius_slider');
        if (slider) slider.value = 14.5;
        const label = document.getElementById('label-roi_radius');
        if (label) label.innerText = '14.5%';

        drawRoi();
        updateRoiLabels();
    }

    async function saveRoiCalibration() {
        const btn = document.getElementById('btn-save-roi');
        const status = document.getElementById('roi-save-status');
        if (!btn) return;

        btn.disabled = true;
        btn.innerHTML = '<span class="material-icons-round spinner">sync</span> <span>Guardando...</span>';
```

```
872
873         const formData = new FormData();
874         formData.append('action', 'update_cenital_roi');
875         formData.append('cenital_roi_cx', roiState.cx.toFixed(4));
876         formData.append('cenital_roi_cy', roiState.cy.toFixed(4));
877         formData.append('cenital_roi_radius', roiState.radius.toFixed(4));
878         formData.append('cenital_roi_mode', roiState.mode);
879
880         try {
881             const resp = await fetch("{{ url_for('dashboard.configuration_panel') }}", {
882                 method: 'POST',
883                 body: formData
884             });
885
886             if (resp.ok) {
887                 if (status) {
888                     status.style.display = 'block';
889                     status.style.background = 'rgba(16, 185, 129, 0.12)';
890                     status.style.color = '#065f46';
891                     status.innerHTML = '<span class="material-icons-round" style="vertical-align: middle; font-size: 16px;">check_circle</span> ¡ROI Cenital guardado y
    calibrado con éxito!';
892                     setTimeout(() => { status.style.display = 'none'; }, 4000);
893                 }
894             } else {
895                 throw new Error('Error al guardar en el servidor');
896             }
897         } catch (err) {
898             if (status) {
899                 status.style.display = 'block';
900                 status.style.background = 'rgba(239, 68, 68, 0.12)';
901                 status.style.color = '#991b1b';
902                 status.innerHTML = `<span class="material-icons-round" style="vertical-align: middle; font-size: 16px;">error</span> Error: ${err.message}`;
903             }
904         } finally {
905             btn.disabled = false;
906             btn.innerHTML = '<span class="material-icons-round" style="font-size: 16px;">save</span> <span>Guardar ROI</span>';
907         }
908     }
909
910     // ========================================
911     // CALIBRADOR INTERACTIVO DE LÍNEA BASE Y TALLO LATERAL
912     // ========================================
913     let lateralState = {
914         baselineY: parseFloat("{{ config.lateral_baseline_y or 0.7200 }}"),
915         mode: "{{ config.lateral_baseline_mode or 'manual_baseline' }}",
916         stemRoiX: parseFloat("{{ config.lateral_stem_roi_x or 0.5000 }}"),
917         stemRoiY: parseFloat("{{ config.lateral_stem_roi_y or 0.6600 }}"),
918         stemRoiW: parseFloat("{{ config.lateral_stem_roi_width or 0.2000 }}"),
919         stemRoiH: parseFloat("{{ config.lateral_stem_roi_height or 0.0500 }}"),
920         stemMode: "{{ config.lateral_stem_mode or 'auto_edge' }}",
921         dragTarget: null, // 'baseline', 'stem_box', 'stem_resize'
922         isDragging: false,
923         dragOffsetX: 0,
924         dragOffsetY: 0
925     };
926
```

```
    const lateralCanvas = document.getElementById('lateral-canvas');
    const lateralCtx = lateralCanvas ? lateralCanvas.getContext('2d') : null;
    const bgLateralImage = new Image();
    const latestLateralUrl = "{{ url_for('static', filename=latest_lateral_img) if latest_lateral_img else '' }}";

    function initLateralCanvas() {
        if (!lateralCanvas || !lateralCtx) return;

        if (latestLateralUrl) {
            bgLateralImage.src = latestLateralUrl;
            bgLateralImage.onload = () => {
                lateralCanvas.width = bgLateralImage.naturalWidth || 480;
                lateralCanvas.height = bgLateralImage.naturalHeight || 640;
                drawLateral();
            };
            bgLateralImage.onerror = () => {
                drawFallbackLateralCanvas();
            };
        } else {
            drawFallbackLateralCanvas();
        }

        // Eventos Mouse
        lateralCanvas.addEventListener('mousedown', (e) => {
            handleLateralMouseDown(e);
        });

        window.addEventListener('mousemove', (e) => {
            if (lateralState.isDragging) {
                handleLateralMouseMove(e);
            }
        });

        window.addEventListener('mouseup', () => {
            lateralState.isDragging = false;
            lateralState.dragTarget = null;
        });

        // Eventos Touch
        lateralCanvas.addEventListener('touchstart', (e) => {
            if (e.touches.length > 0) handleLateralMouseDown(e.touches[0]);
            e.preventDefault();
        }, { passive: false });

        lateralCanvas.addEventListener('touchmove', (e) => {
            if (lateralState.isDragging && e.touches.length > 0) {
                handleLateralMouseMove(e.touches[0]);
            }
            e.preventDefault();
        }, { passive: false });

        lateralCanvas.addEventListener('touchend', () => {
            lateralState.isDragging = false;
            lateralState.dragTarget = null;
        });
```

```
}

function drawFallbackLateralCanvas() {
    if (!lateralCanvas) return;
    lateralCanvas.width = 480;
    lateralCanvas.height = 640;
    drawLateral();
}

function getLateralCanvasCoords(e) {
    const rect = lateralCanvas.getBoundingClientRect();
    const scaleX = lateralCanvas.width / rect.width;
    const scaleY = lateralCanvas.height / rect.height;
    return {
        x: (e.clientX - rect.left) * scaleX,
        y: (e.clientY - rect.top) * scaleY
    };
}

function handleLateralMouseDown(e) {
    if (!lateralCanvas) return;
    const pos = getLateralCanvasCoords(e);
    const w = lateralCanvas.width;
    const h = lateralCanvas.height;

    const basePy = lateralState.baselineY * h;
    const stemBx = (lateralState.stemRoiX - lateralState.stemRoiW / 2) * w;
    const stemBy = (lateralState.stemRoiY - lateralState.stemRoiH / 2) * h;
    const stemBw = lateralState.stemRoiW * w;
    const stemBh = lateralState.stemRoiH * h;

    // Comprobar si se hizo clic dentro del cuadro de tallo
    if (pos.x >= stemBx && pos.x <= stemBx + stemBw && pos.y >= stemBy && pos.y <= stemBy + stemBh) {
        lateralState.isDragging = true;
        lateralState.dragTarget = 'stem_box';
        lateralState.dragOffsetX = pos.x / w - lateralState.stemRoiX;
        lateralState.dragOffsetY = pos.y / h - lateralState.stemRoiY;
    } else if (Math.abs(pos.y - basePy) < 25) {
        // Clic cerca de la línea base
        lateralState.isDragging = true;
        lateralState.dragTarget = 'baseline';
    } else {
        // Clic en cualquier otra parte mueve la línea base o el tallo según proximidad
        if (pos.y > basePy - 40) {
            lateralState.isDragging = true;
            lateralState.dragTarget = 'baseline';
            lateralState.baselineY = Math.max(0.15, Math.min(0.98, pos.y / h));
            updateLateralSliders();
            drawLateral();
        } else {
            lateralState.isDragging = true;
            lateralState.dragTarget = 'stem_box';
            lateralState.stemRoiX = Math.max(0.10, Math.min(0.90, pos.x / w));
            lateralState.stemRoiY = Math.max(0.10, Math.min(0.90, pos.y / h));
            lateralState.dragOffsetX = 0;
```

```
                lateralState.dragOffsetY = 0;
                updateLateralSliders();
                drawLateral();
            }
        }
    }

    function handleLateralMouseMove(e) {
        if (!lateralCanvas || !lateralState.isDragging) return;
        const pos = getLateralCanvasCoords(e);
        const w = lateralCanvas.width;
        const h = lateralCanvas.height;

        if (lateralState.dragTarget === 'baseline') {
            lateralState.baselineY = Math.max(0.15, Math.min(0.98, pos.y / h));
            updateLateralSliders();
            drawLateral();
        } else if (lateralState.dragTarget === 'stem_box') {
            const newX = pos.x / w - lateralState.dragOffsetX;
            const newY = pos.y / h - lateralState.dragOffsetY;
            lateralState.stemRoiX = Math.max(0.10, Math.min(0.90, newX));
            lateralState.stemRoiY = Math.max(0.10, Math.min(0.95, newY));
            updateLateralSliders();
            drawLateral();
        }
    }

    function onLateralBaselineSliderChange(val) {
        lateralState.baselineY = parseFloat(val) / 100.0;
        const label = document.getElementById('label-lateral_baseline');
        if (label) label.innerText = parseFloat(val).toFixed(1) + '%';
        drawLateral();
        updateLateralLabels();
    }

    function onStemSliderChange(axis, val) {
        if (axis === 'y') {
            lateralState.stemRoiY = parseFloat(val) / 100.0;
            const lbl = document.getElementById('label-stem_roi_y');
            if (lbl) lbl.innerText = parseFloat(val).toFixed(1) + '%';
        } else if (axis === 'x') {
            lateralState.stemRoiX = parseFloat(val) / 100.0;
            const lbl = document.getElementById('label-stem_roi_x');
            if (lbl) lbl.innerText = parseFloat(val).toFixed(1) + '%';
        } else if (axis === 'w') {
            lateralState.stemRoiW = parseFloat(val) / 100.0;
            const lbl = document.getElementById('label-stem_roi_w');
            if (lbl) lbl.innerText = parseFloat(val).toFixed(0) + '%';
        } else if (axis === 'h') {
            lateralState.stemRoiH = parseFloat(val) / 100.0;
            const lbl = document.getElementById('label-stem_roi_h');
            if (lbl) lbl.innerText = parseFloat(val).toFixed(1) + '%';
        }
        drawLateral();
        updateLateralLabels();
```

```
    }

    function updateLateralSliders() {
        const baseSlider = document.getElementById('lateral_baseline_slider');
        if (baseSlider) baseSlider.value = (lateralState.baselineY * 100).toFixed(1);
        const baseLabel = document.getElementById('label-lateral_baseline');
        if (baseLabel) baseLabel.innerText = (lateralState.baselineY * 100).toFixed(1) + '%';

        const stemYSlider = document.getElementById('lateral_stem_y_slider');
        if (stemYSlider) stemYSlider.value = (lateralState.stemRoiY * 100).toFixed(1);
        const stemYLabel = document.getElementById('label-stem_roi_y');
        if (stemYLabel) stemYLabel.innerText = (lateralState.stemRoiY * 100).toFixed(1) + '%';

        const stemXSlider = document.getElementById('lateral_stem_x_slider');
        if (stemXSlider) stemXSlider.value = (lateralState.stemRoiX * 100).toFixed(1);
        const stemXLabel = document.getElementById('label-stem_roi_x');
        if (stemXLabel) stemXLabel.innerText = (lateralState.stemRoiX * 100).toFixed(1) + '%';

        updateLateralLabels();
    }

    function updateLateralLabels() {
        const coords = document.getElementById('lateral-coords-display');
        if (coords) {
            coords.innerText = `Base: ${(lateralState.baselineY * 100).toFixed(1)}% | Tallo: ${(lateralState.stemRoiY * 100).toFixed(1)}%`;
        }
    }

    function resetLateralCalibrationToDefault() {
        lateralState.baselineY = 0.7200;
        lateralState.stemRoiX = 0.5000;
        lateralState.stemRoiY = 0.6600;
        lateralState.stemRoiW = 0.2000;
        lateralState.stemRoiH = 0.0500;
        lateralState.mode = 'manual_baseline';
        lateralState.stemMode = 'auto_edge';

        const baseSlider = document.getElementById('lateral_baseline_slider');
        if (baseSlider) baseSlider.value = 72.0;
        const stemYSlider = document.getElementById('lateral_stem_y_slider');
        if (stemYSlider) stemYSlider.value = 66.0;
        const stemXSlider = document.getElementById('lateral_stem_x_slider');
        if (stemXSlider) stemXSlider.value = 50.0;
        const stemWSlider = document.getElementById('lateral_stem_w_slider');
        if (stemWSlider) stemWSlider.value = 20.0;
        const stemHSlider = document.getElementById('lateral_stem_h_slider');
        if (stemHSlider) stemHSlider.value = 5.0;

        updateLateralSliders();
        drawLateral();
    }

    function drawLateral() {
        if (!lateralCtx || !lateralCanvas) return;
        const w = lateralCanvas.width;
```

```
        const h = lateralCanvas.height;

        lateralCtx.clearRect(0, 0, w, h);

        if (bgLateralImage.complete && bgLateralImage.naturalWidth > 0) {
            lateralCtx.drawImage(bgLateralImage, 0, 0, w, h);
        } else {
            lateralCtx.fillStyle = '#1e293b';
            lateralCtx.fillRect(0, 0, w, h);
            lateralCtx.fillStyle = '#f8fafc';
            lateralCtx.font = '14px Inter, sans-serif';
            lateralCtx.textAlign = 'center';
            lateralCtx.fillText('No hay fotos laterales capturadas aún.', w / 2, h / 2);
        }

        const py = lateralState.baselineY * h;

        // 1. Sombra bajo la línea de la maceta (Sustrato / Tubo de soporte)
        lateralCtx.save();
        lateralCtx.fillStyle = 'rgba(0, 0, 0, 0.45)';
        lateralCtx.fillRect(0, py, w, h - py);
        lateralCtx.restore();

        // 2. Línea Base Horizontal Roja (Estilo guía agrónoma)
        lateralCtx.save();
        lateralCtx.beginPath();
        lateralCtx.moveTo(0, py);
        lateralCtx.lineTo(w, py);
        lateralCtx.lineWidth = 3.5;
        lateralCtx.strokeStyle = '#ef4444';
        lateralCtx.shadowColor = '#ef4444';
        lateralCtx.shadowBlur = 12;
        lateralCtx.stroke();

        // Pequeñas flechas indicadoras en los extremos
        lateralCtx.fillStyle = '#ef4444';
        lateralCtx.beginPath();
        lateralCtx.moveTo(10, py - 6); lateralCtx.lineTo(22, py); lateralCtx.lineTo(10, py + 6); lateralCtx.fill();
        lateralCtx.beginPath();
        lateralCtx.moveTo(w - 10, py - 6); lateralCtx.lineTo(w - 22, py); lateralCtx.lineTo(w - 10, py + 6); lateralCtx.fill();

        // Cartela indicadora
        lateralCtx.fillStyle = '#ef4444';
        lateralCtx.font = 'bold 12px Inter, sans-serif';
        lateralCtx.textAlign = 'left';
        lateralCtx.shadowColor = '#000000';
        lateralCtx.shadowBlur = 6;
        lateralCtx.fillText('LÍNEA BASE (MACETA / INICIO ALTURA)', 30, Math.max(20, py - 8));
        lateralCtx.restore();

        // 3. Cuadro / Franja de Medición de Tallo (Scanline de Bordes)
        const bx = (lateralState.stemRoiX - lateralState.stemRoiW / 2) * w;
        const by = (lateralState.stemRoiY - lateralState.stemRoiH / 2) * h;
        const bw = lateralState.stemRoiW * w;
        const bh = lateralState.stemRoiH * h;
```

```
        lateralCtx.save();
        // Relleno suave translúcido
        lateralCtx.fillStyle = 'rgba(245, 158, 11, 0.15)';
        lateralCtx.fillRect(bx, by, bw, bh);

        // Borde exterior
        lateralCtx.setLineDash([6, 4]);
        lateralCtx.lineWidth = 2;
        lateralCtx.strokeStyle = '#f59e0b';
        lateralCtx.strokeRect(bx, by, bw, bh);
        lateralCtx.setLineDash([]);

        // Línea central de escaneo horizontal
        const midY = by + bh / 2;
        lateralCtx.beginPath();
        lateralCtx.moveTo(bx, midY);
        lateralCtx.lineTo(bx + bw, midY);
        lateralCtx.lineWidth = 2;
        lateralCtx.strokeStyle = '#38bdf8';
        lateralCtx.stroke();

        // Marcas de límite de escaneo
        lateralCtx.fillStyle = '#38bdf8';
        lateralCtx.fillRect(bx, midY - 6, 3, 12);
        lateralCtx.fillRect(bx + bw - 3, midY - 6, 3, 12);

        // Esquinas de anclaje
        lateralCtx.fillStyle = '#f59e0b';
        const handleSize = 6;
        lateralCtx.fillRect(bx - handleSize/2, by - handleSize/2, handleSize, handleSize);
        lateralCtx.fillRect(bx + bw - handleSize/2, by - handleSize/2, handleSize, handleSize);
        lateralCtx.fillRect(bx - handleSize/2, by + bh - handleSize/2, handleSize, handleSize);
        lateralCtx.fillRect(bx + bw - handleSize/2, by + bh - handleSize/2, handleSize, handleSize);

        // Etiqueta de la zona de tallo
        lateralCtx.fillStyle = '#f59e0b';
        lateralCtx.font = 'bold 11px Inter, sans-serif';
        lateralCtx.textAlign = 'left';
        lateralCtx.shadowColor = '#000000';
        lateralCtx.shadowBlur = 5;
        lateralCtx.fillText('ZONA DE MEDICIÓN DE TALLO (SCANLINE)', bx, Math.max(16, by - 6));

        lateralCtx.restore();
    }

    async function saveLateralBaselineCalibration() {
        const btn = document.getElementById('btn-save-lateral-baseline');
        const status = document.getElementById('lateral-save-status');
        if (!btn) return;

        btn.disabled = true;
        btn.innerHTML = '<span class="material-icons-round spinner">sync</span> <span>Guardando...</span>';

        const formData = new FormData();
```

```
        formData.append('action', 'update_lateral_baseline');
        formData.append('lateral_baseline_y', lateralState.baselineY.toFixed(4));
        formData.append('lateral_baseline_mode', lateralState.mode);
        formData.append('lateral_stem_roi_x', lateralState.stemRoiX.toFixed(4));
        formData.append('lateral_stem_roi_y', lateralState.stemRoiY.toFixed(4));
        formData.append('lateral_stem_roi_width', lateralState.stemRoiW.toFixed(4));
        formData.append('lateral_stem_roi_height', lateralState.stemRoiH.toFixed(4));
        formData.append('lateral_stem_mode', lateralState.stemMode);

        try {
            const resp = await fetch("{{ url_for('dashboard.configuration_panel') }}", {
                method: 'POST',
                body: formData
            });

            if (resp.ok) {
                if (status) {
                    status.style.display = 'block';
                    status.style.background = 'rgba(37, 99, 235, 0.12)';
                    status.style.color = '#1e40af';
                    status.innerHTML = '<span class="material-icons-round" style="vertical-align: middle; font-size: 16px;">check_circle</span> ¡Línea base y zona de tallo guardadas y calibradas con éxito!';
                    setTimeout(() => { status.style.display = 'none'; }, 4000);
                }
            } else {
                throw new Error('Error al guardar en el servidor');
            }
        } catch (err) {
            if (status) {
                status.style.display = 'block';
                status.style.background = 'rgba(239, 68, 68, 0.12)';
                status.style.color = '#991b1b';
                status.innerHTML = `<span class="material-icons-round" style="vertical-align: middle; font-size: 16px;">error</span> Error: ${err.message}`;
            }
        } finally {
            btn.disabled = false;
            btn.innerHTML = '<span class="material-icons-round" style="font-size: 16px;">save</span> <span>Guardar Calibración Lateral</span>';
        }
    }

    // Iniciar ambos canvas cuando cargue la página
    document.addEventListener('DOMContentLoaded', () => {
        initRoiCanvas();
        updateRoiLabels();

        initLateralCanvas();
        updateLateralLabels();
    });
</script>
{% endblock %}
```

ARCHIVO 65/85: local_server/templates/cross_analysis.html (1626 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Cruce de Datos y Análisis Multivariable{% endblock %}
{% block heading %}Cruce de Datos y Análisis Multivariable{% endblock %}
{% block subheading %}Sincronización cronológica, matriz de correlación de Pearson y laboratorio gráfico para Canastilla #{{ active_plant_id }}{% endblock %}

{% block header_actions %}
<div style="display: flex; gap: 10px; flex-wrap: wrap;">
    <a href="{{ url_for('dashboard.detailed_sensors') }}" class="btn btn-outline btn-sm">
        <span class="material-icons-round">settings_input_antenna</span>
        <span>Gestionar Antena USB / Sensores</span>
    </a>
    <a href="{{ url_for('api.export_cross_analysis_csv', plant_id=active_plant_id) }}" class="btn btn-primary btn-sm">
        <span class="material-icons-round">download</span>
        <span>Descargar Dataset en CSV (Excel)</span>
    </a>
</div>
{% endblock %}

{% block content %}

<!-- SECCIÓN 1: FILTROS GLOBALES Y TARJETAS DE RESUMEN -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div class="widget-header">
        <div class="widget-header-title">
            <h3>Filtros por Período de Muestreo y Jornada</h3>
            <p>Segmenta el cruce de datos por jornada horaria o fecha de evaluación</p>
        </div>
        <div id="cross-table-count-badge" class="badge badge-primary">
            {{ cross_data | length }} sesiones sincronizadas
        </div>
    </div>

    <!-- Filtro por Período del Día -->
    <div style="margin-bottom: 16px;">
        <label style="font-size: 12px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; display: block; margin-bottom: 8px;">
            Franja Horaria / Período:
        </label>
        <div class="filter-group">
            <button class="filter-pill cross-period-btn active" data-period="todos" onclick="filterCrossPeriod('todos', this)">
                <span class="material-icons-round" style="font-size: 15px;">apps</span>
                <span>Todos los Períodos</span>
            </button>
            <button class="filter-pill cross-period-btn" data-period="mañana" onclick="filterCrossPeriod('mañana', this)">
                <span class="material-icons-round" style="font-size: 15px;">light_mode</span>
                <span>Mañana (07:00)</span>
            </button>
```

```
            <button class="filter-pill cross-period-btn" data-period="mediodía" onclick="filterCrossPeriod('mediodía', this)">
                <span class="material-icons-round" style="font-size: 15px;">wb_sunny</span>
                <span>Mediodía (12:00)</span>
            </button>
            <button class="filter-pill cross-period-btn" data-period="tarde" onclick="filterCrossPeriod('tarde', this)">
                <span class="material-icons-round" style="font-size: 15px;">nights_stay</span>
                <span>Tarde (17:00)</span>
            </button>
        </div>
    </div>

    <!-- Filtro por Fecha de Muestreo -->
    <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 12px;">
        <div class="filter-group">
            <button class="filter-pill cross-date-btn active" data-date="todos" onclick="filterCrossDate('todos', this)">
                <span class="material-icons-round" style="font-size: 15px;">view_timeline</span>
                <span>Todas las Fechas</span>
            </button>

            {% for d in available_dates %}
                <button class="filter-pill cross-date-btn" data-date="{{ d }}" onclick="filterCrossDate('{{ d }}', this)">
                    <span class="material-icons-round" style="font-size: 15px;">event</span>
                    <span>{{ d }}</span>
                </button>
            {% endfor %}
        </div>

        <div style="display: flex; align-items: center; gap: 8px;">
            <label style="font-size: 12px; color: var(--text-muted); font-weight: 600;">Fecha específica:</label>
            <input type="date" id="cross-custom-date" class="form-control" style="width: auto; padding: 5px 8px; font-size: 12.5px;"
onchange="filterCrossDate(this.value, null)">
        </div>
    </div>
</div>

<!-- SECCIÓN 2: MATRIZ DE CORRELACIÓN DE PEARSON (HEATMAP) -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div class="widget-header">
        <div class="widget-header-title">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="color: var(--accent-success); font-size: 24px;">grid_on</span>
                <div>
                    <h3>Matriz de Correlación Multivariable (Heatmap de Pearson)</h3>
                    <p>Coeficiente de correlación lineal (r) entre variables de sensores y métricas de fenotipado. Haz clic en cualquier celda para graficar ese par.</p>
                </div>
            </div>
        </div>
        <div class="badge badge-success" id="matrix-strongest-badge">
            Par más significativo: {{ correlation_matrix.strongest_pair }}
        </div>
    </div>

    <div class="heatmap-table-container">
        <table class="heatmap-table">
            <thead>
```

```
                <tr>
                    <th style="text-align: left; width: 220px;">Variable Ambiental (Sensor) \ Fenotipo</th>
                    {% for p in correlation_matrix.phenotypes %}
                        <th>
                            <div>{{ p.label }}</div>
                            <div style="font-size: 10px; color: var(--text-muted); font-weight: normal;">({{ p.unit }})</div>
                        </th>
                    {% endfor %}
                </tr>
            </thead>
            <tbody>
                {% for row in correlation_matrix.matrix_rows %}
                    <tr>
                        <td style="text-align: left; font-weight: 700; color: var(--text-heading); background: #f8fafc;">
                            <div style="display: flex; align-items: center; gap: 8px;">
                                <span class="status-dot active" style="background-color: var(--accent-primary);"></span>
                                <div>
                                    <div>{{ row.sensor_label }}</div>
                                    <div style="font-size: 11px; color: var(--text-muted); font-weight: normal;">({{ row.sensor_unit }})</div>
                                </div>
                            </div>
                        </td>
                        {% for cell in row.cells %}
                            <td style="background-color: {{ cell.bg_color }}; color: {{ cell.text_color }};"
                                onclick="selectCustomPair('{{ cell.sensor_key }}', '{{ cell.pheno_key }}')"
                                title="{{ row.sensor_label }} vs {{ cell.pheno_key }}: {{ cell.interpretation }} (r = {{ cell.r }}, R² = {{ cell.r2 }})">
                                <div class="heatmap-cell-content">
                                    <span class="heatmap-r-val">{% if cell.r > 0 %}+{% endif %}{{ cell.r }}</span>
                                    <span class="heatmap-r2-val">R² = {{ cell.r2 }}</span>
                                    <span class="heatmap-interp-badge" style="background: rgba(0,0,0,0.06);">
                                        {{ cell.interpretation }}
                                    </span>
                                </div>
                            </td>
                        {% endfor %}
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>

    <!-- Leyenda explicativa de la matriz -->
    <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 12px; margin-top: 14px; font-size: 11.5px; color: var(--text-muted);">
        <div style="display: flex; align-items: center; gap: 16px; flex-wrap: wrap;">
            <span style="display: flex; align-items: center; gap: 6px;">
                <span style="width: 14px; height: 14px; border-radius: 3px; background: rgba(16, 185, 129, 0.4); border: 1px solid #10b981;"></span>
                <span>Directa Fuerte (r &ge; 0.65)</span>
            </span>
            <span style="display: flex; align-items: center; gap: 6px;">
                <span style="width: 14px; height: 14px; border-radius: 3px; background: rgba(52, 211, 153, 0.25); border: 1px solid #34d399;"></span>
                <span>Directa Moderada (0.25 &le; r &lt; 0.65)</span>
            </span>
            <span style="display: flex; align-items: center; gap: 6px;">
                <span style="width: 14px; height: 14px; border-radius: 3px; background: rgba(241, 245, 249, 0.9); border: 1px solid #cbd5e1;"></span>
                <span>Sin Correlación Lineal (-0.25 &lt; r &lt; 0.25)</span>
```

```
            </span>
            <span style="display: flex; align-items: center; gap: 6px;">
                <span style="width: 14px; height: 14px; border-radius: 3px; background: rgba(248, 113, 113, 0.25); border: 1px solid #f87171;"></span>
                <span>Inversa Moderada (-0.65 &lt; r &le; -0.25)</span>
            </span>
            <span style="display: flex; align-items: center; gap: 6px;">
                <span style="width: 14px; height: 14px; border-radius: 3px; background: rgba(239, 68, 68, 0.35); border: 1px solid #ef4444;"></span>
                <span>Inversa Fuerte (r &le; -0.65)</span>
            </span>
        </div>
        <span style="font-weight: 600; color: var(--text-heading);">* Haz clic en cualquier celda para graficarla automáticamente abajo.</span>
    </div>
</div>

<!-- SECCIÓN 3: LABORATORIO DINÁMICO DE CRUCE A LA CARTA -->
<div class="comparator-lab-card" style="margin-bottom: 24px;">
    <div class="widget-header" style="border-bottom: 1px solid var(--border-color); padding-bottom: 16px; margin-bottom: 18px;">
        <div class="widget-header-title">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="color: var(--accent-primary); font-size: 24px;">tune</span>
                <div>
                    <h3 style="font-size: 16px;">Laboratorio Dinámico de Cruce y Selector de Métricas</h3>
                    <p>Selecciona cualquier par de variables y explora su relación en múltiples tipos de gráficos interactivos</p>
                </div>
            </div>
        </div>

        <!-- Selector de Tipo de Gráfica -->
        <div class="filter-group">
            <button type="button" class="filter-btn active" id="btn-chart-scatter" onclick="setChartType('scatter', this)">
                <span class="material-icons-round" style="font-size: 15px;">scatter_plot</span>
                <span>Dispersión (Scatter + Tendencia)</span>
            </button>
            <button type="button" class="filter-btn" id="btn-chart-dual" onclick="setChartType('dual', this)">
                <span class="material-icons-round" style="font-size: 15px;">stacked_line_chart</span>
                <span>Serie Temporal Doble Eje</span>
            </button>
            <button type="button" class="filter-btn" id="btn-chart-radar" onclick="setChartType('radar', this)">
                <span class="material-icons-round" style="font-size: 15px;">radar</span>
                <span>Radar de Inmersión</span>
            </button>
            <button type="button" class="filter-btn" id="btn-chart-polar" onclick="setChartType('polar', this)">
                <span class="material-icons-round" style="font-size: 15px;">pie_chart</span>
                <span>Distribución Polar / Dona</span>
            </button>
            <button type="button" class="filter-btn" id="btn-chart-bar" onclick="setChartType('bar', this)">
                <span class="material-icons-round" style="font-size: 15px;">bar_chart</span>
                <span>Barras Agrupadas</span>
            </button>
        </div>
    </div>

    <!-- Controles de Selección de Variables -->
    <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 16px; margin-bottom: 18px; background: var(--bg-card-subtle); padding: 14px 18px; border-radius: 10px; border: 1px solid var(--border-color);">
```

```
        <div>
            <label style="font-size: 12px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; display: block; margin-bottom: 6px;">
                Variable 1: Fenotipado / Planta (Eje Y)
            </label>
            <select id="select-pheno-var" class="form-control" onchange="renderCustomComparatorChart()">
                <option value="foliar_area_cm2" selected>Área Foliar (cm²)</option>
                <option value="plant_height_cm">Altura de la Planta (cm)</option>
                <option value="stem_diameter_mm">Diámetro del Tallo (mm)</option>
                <option value="health_index">Índice de Salud Foliar (%)</option>
                <option value="compacity_index">Índice de Compacidad</option>
            </select>
        </div>

        <div>
            <label style="font-size: 12px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; display: block; margin-bottom: 6px;">
                Variable 2: Telemetría / Sensor (Eje X o Eje 2)
            </label>
            <select id="select-sensor-var" class="form-control" onchange="renderCustomComparatorChart()">
                <option value="temperature_c" selected>Temperatura Ambiente (°C)</option>
                <option value="humidity_rh">Humedad Relativa (% HR)</option>
                <option value="uv_solar_lux">Radiación Solar / UV (lux)</option>
                <option value="motor_current_a">Corriente de Bomba (A)</option>
                <option value="ec_conductivity">Conductividad Eléctrica (µS/cm)</option>
                <option value="water_ph">pH de la Solución (pH)</option>
                <option value="turbidity_ntu">Turbidez del Agua (NTU)</option>
                <option value="dissolved_oxygen_mg_l">Oxígeno Disuelto (mg/L)</option>
                <option value="water_temp_c">Temperatura del Agua (°C)</option>
                <option value="motor_state">Estado del Motor / Bomba (1/0)</option>
            </select>
        </div>

        <div style="display: flex; flex-direction: column; justify-content: center;">
            <div style="font-size: 12px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; margin-bottom: 4px;">
                Diagnóstico Estadístico en Vivo:
            </div>
            <div style="font-size: 13px; font-weight: 700; color: var(--text-heading);" id="custom-math-summary">
                Calculando regresión lineal...
            </div>
        </div>
    </div>

    <!-- Lienzo del Gráfico Dinámico -->
    <div class="chart-stage-container" style="height: 380px; margin-top: 10px;">
        <canvas id="customComparatorChart"></canvas>
    </div>
</div>

<!-- SECCIÓN 4: GRÁFICAS INDIVIDUALES DEDICADAS DE ALTA RESOLUCIÓN -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div class="widget-header">
        <div class="widget-header-title">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="color: var(--accent-success); font-size: 24px;">insights</span>
                <div>
                    <h3>Paneles Gráficos Individuales de Investigación</h3>
```

```
                    <p>Monitoreo continuo desglosado de morfología, vigor y curvas climáticas</p>
                </div>
            </div>
        </div>

    <div class="analytics-grid-2x2">
        <!-- Gráfica 1: Área Foliar -->
        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 18px;">
            <div class="widget-header-title" style="margin-bottom: 12px; display: flex; justify-content: space-between; align-items: center;">
                <div>
                    <h4 style="font-size: 14px; font-weight: 700; margin: 0;">Evolución de Área Foliar (cm²)</h4>
                    <span style="font-size: 11px; color: var(--text-muted); font-weight: 600;">Promedio de 5 fotos por período</span>
                </div>
                <span class="badge badge-success">Promedio 5 Fotos / Período</span>
            </div>
            <div class="chart-stage-container" style="height: 260px;">
                <canvas id="dedicFoliarChart"></canvas>
            </div>
        </div>

        <!-- Gráfica 2: Altura y Tallo -->
        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 18px;">
            <div class="widget-header-title" style="margin-bottom: 12px; display: flex; justify-content: space-between; align-items: center;">
                <div>
                    <h4 style="font-size: 14px; font-weight: 700; margin: 0;">Altura de Planta (cm) y Tallo (mm)</h4>
                    <span style="font-size: 11px; color: var(--text-muted); font-weight: 600;">Promedio de 5 fotos por período</span>
                </div>
                <span class="badge badge-primary">Promedio 5 Fotos / Período</span>
            </div>
            <div class="chart-stage-container" style="height: 260px;">
                <canvas id="dedicMorphChart"></canvas>
            </div>
        </div>

        <!-- Gráfica 3: Clima Temperatura y Humedad -->
        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 18px;">
            <div class="widget-header-title" style="margin-bottom: 12px; display: flex; justify-content: space-between; align-items: center;">
                <div>
                    <h4 style="font-size: 14px; font-weight: 700; margin: 0;">Temperatura (°C) y Humedad (% HR)</h4>
                    <span style="font-size: 11px; color: var(--text-muted); font-weight: 600;">Telemetría captada en la franja horaria</span>
                </div>
                <span class="badge badge-primary">Telemetría Sincronizada</span>
            </div>
            <div class="chart-stage-container" style="height: 260px;">
                <canvas id="dedicClimateChart"></canvas>
            </div>
        </div>

        <!-- Gráfica 4: Radiación y Bombeo -->
        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 18px;">
            <div class="widget-header-title" style="margin-bottom: 12px; display: flex; justify-content: space-between; align-items: center;">
                <div>
                    <h4 style="font-size: 14px; font-weight: 700; margin: 0;">Radiación Solar (lux) y Bombeo (A)</h4>
                    <span style="font-size: 11px; color: var(--text-muted); font-weight: 600;">Telemetría captada en la franja horaria</span>
```

```
                </div>
                <span class="badge badge-warning">Telemetría Sincronizada</span>
            </div>
            <div class="chart-stage-container" style="height: 260px;">
                <canvas id="dedicSolarChart"></canvas>
            </div>
        </div>

        <!-- Gráfica 5: pH y Oxígeno Disuelto -->
        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 18px;">
            <div class="widget-header-title" style="margin-bottom: 12px; display: flex; justify-content: space-between; align-items: center;">
                <div>
                    <h4 style="font-size: 14px; font-weight: 700; margin: 0;">pH y Oxígeno Disuelto (mg/L)</h4>
                    <span style="font-size: 11px; color: var(--text-muted); font-weight: 600;">Telemetría captada en la franja horaria</span>
                </div>
                <span class="badge badge-info" style="background: rgba(6, 182, 212, 0.15); color: #0891b2;">Telemetría Sincronizada</span>
            </div>
            <div class="chart-stage-container" style="height: 260px;">
                <canvas id="dedicWaterPhChart"></canvas>
            </div>
        </div>

        <!-- Gráfica 6: Conductividad, Turbidez y Temp. Agua -->
        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 18px;">
            <div class="widget-header-title" style="margin-bottom: 12px; display: flex; justify-content: space-between; align-items: center;">
                <div>
                    <h4 style="font-size: 14px; font-weight: 700; margin: 0;">Conductividad (µS/cm), Turbidez (NTU) y Temp. Agua (°C)</h4>
                    <span style="font-size: 11px; color: var(--text-muted); font-weight: 600;">Telemetría captada en la franja horaria</span>
                </div>
                <span class="badge badge-primary" style="background: rgba(139, 92, 246, 0.15); color: #7c3aed;">Telemetría Sincronizada</span>
            </div>
            <div class="chart-stage-container" style="height: 260px;">
                <canvas id="dedicWaterQualityChart"></canvas>
            </div>
        </div>
    </div>
</div>

<!-- SECCIÓN 5: TABLA DE CRUCE SINCRONIZADO POR PERÍODO -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div class="widget-header">
        <div class="widget-header-title">
            <h3>Matriz de Cruce Sincronizado por Período de Muestreo</h3>
            <p>Valores promediados de las 5 fotos por sesión vs lecturas de telemetría ambiental</p>
        </div>
        <div class="badge badge-primary">
            {{ cross_data | length }} Períodos Registrados
        </div>
    </div>

    <div class="table-responsive">
        <table class="table" id="cross-matrix-table">
            <thead>
                <tr>
                    <th>Fecha y Período</th>
```

```
                    <th>Hora</th>
                    <th>Área Foliar (Prom. 5 Fotos)</th>
                    <th>Altura (Prom. 5 Fotos)</th>
                    <th>Tallo (Prom. 5 Fotos)</th>
                    <th>Salud (Prom.)</th>
                    <th>Temp. Amb.</th>
                    <th>Humedad</th>
                    <th>pH</th>
                    <th>C.E.</th>
                    <th>Turbidez</th>
                    <th>O₂ Disuelto</th>
                    <th>Temp. Agua</th>
                    <th>Bomba</th>
                    <th>Fotos Ráfaga</th>
                    <th style="text-align: right;">Acciones</th>
                </tr>
            </thead>
            <tbody id="cross-table-body">
                {% if cross_data %}
                    {% for row in cross_data %}
                        <tr class="cross-row" data-period="{{ row.period_name }}" data-date="{{ row.date_str }}">
                            <td>
                                <div style="font-weight: 700; color: var(--text-heading);">{{ row.full_period_label }}</div>
                                <div style="font-size: 11px; color: var(--text-muted);">{{ row.crop_type | capitalize }} (Canastilla #{{ row.plant_id }})</div>
                            </td>
                            <td style="font-size: 12px; font-weight: 600; color: var(--text-muted);">
                                {{ row.exact_time }}
                            </td>
                            <td style="font-weight: 700; color: var(--accent-success);">
                                {{ row.foliar_area_cm2 }} cm²
                            </td>
                            <td>{{ row.plant_height_cm }} cm</td>
                            <td>{{ row.stem_diameter_mm }} mm</td>
                            <td>
                                <span class="badge {% if row.health_index >= 80 %}badge-success{% elif row.health_index >= 60 %}badge-warning{% else %}badge-danger{% endif %}">
                                    {{ row.health_index }}%
                                </span>
                            </td>
                            <td>
                                <span style="font-weight: 600; color: #2563eb;">
                                    {% if row.temperature_c is not none %}{{ row.temperature_c }} °C{% else %}<span style="color: var(--text-muted);">--</span>{% endif %}
                                </span>
                            </td>
                            <td>
                                <span style="font-weight: 600; color: #0284c7;">
                                    {% if row.humidity_rh is not none %}{{ row.humidity_rh }}%{% else %}<span style="color: var(--text-muted);">--</span>{% endif %}
                                </span>
                            </td>
                            <td>
                                <span style="font-weight: 600; color: #ec4899;">
                                    {% if row.water_ph is not none %}{{ row.water_ph }}{% else %}<span style="color: var(--text-muted);">--</span>{% endif %}
                                </span>
                            </td>
                            <td>
```

```
                                <span style="font-weight: 600; color: #8b5cf6;">
                                    {% if row.ec_conductivity is not none %}{{ row.ec_conductivity }} µS/cm{% else %}<span style="color: var(--text-muted);">--</span>{% endif %}
                                </span>
                            </td>
                            <td>
                                <span style="font-weight: 600; color: #f97316;">
                                    {% if row.turbidity_ntu is not none %}{{ row.turbidity_ntu }} NTU{% else %}<span style="color: var(--text-muted);">--</span>{% endif %}
                                </span>
                            </td>
                            <td>
                                <span style="font-weight: 600; color: #06b6d4;">
                                    {% if row.dissolved_oxygen_mg_l is not none %}{{ row.dissolved_oxygen_mg_l }} mg/L{% else %}<span style="color: var(--text-muted);">--</span>{% endif %}
                                </span>
                            </td>
                            <td>
                                <span style="font-weight: 600; color: #0ea5e9;">
                                    {% if row.water_temp_c is not none %}{{ row.water_temp_c }} °C{% else %}<span style="color: var(--text-muted);">--</span>{% endif %}
                                </span>
                            </td>
                            <td>
                                {% if row.motor_state is not none %}
                                    <span class="badge {% if row.motor_state == 1 %}badge-success{% else %}badge-secondary{% endif %}" style="font-size: 11px;">
                                        {{ "ON" if row.motor_state == 1 else "OFF" }} ({{ row.motor_current_a or 0.0 }} A)
                                    </span>
                                {% else %}
                                    <span style="color: var(--text-muted); font-size: 12px;">--</span>
                                {% endif %}
                            </td>
                            <td>
                                <span class="badge badge-success">{{ row.photos_count }} fotos</span>
                            </td>
                            <td style="text-align: right;">
                                <button type="button" class="btn btn-outline btn-sm" onclick="showIndividualPhotosModal({{ row.session_id }})" title="Ver fotos cenital y lateral procesadas">
                                    <span class="material-icons-round" style="font-size: 14px;">visibility</span>
                                    <span>Ver Fotos</span>
                                </button>
                                <button type="button" class="btn btn-danger btn-sm" onclick="deleteSessionFromCross({{ row.session_id }})" title="Eliminar este período">
                                    <span class="material-icons-round" style="font-size: 14px;">delete</span>
                                </button>
                            </td>
                        </tr>
                    {% endfor %}
                {% else %}
                    <tr>
                        <td colspan="16" style="text-align: center; padding: 35px; color: var(--text-muted);">
                            No hay sesiones cruzadas registradas para la Canastilla #{{ active_plant_id }}.
                        </td>
                    </tr>
                {% endif %}
            </tbody>
        </table>
    </div>
</div>
```

```
489
490 <!-- SECCIÓN 6: ESCRUTINIO DETALLADO FOTO A FOTO CON PLEGADO Y DESPLEGADO -->
491 <div class="widget-card" style="margin-bottom: 24px;">
492     <div class="widget-header">
493         <div class="widget-header-title">
494             <div style="display: flex; align-items: center; gap: 8px;">
495                 <span class="material-icons-round" style="color: var(--accent-primary); font-size: 24px;">photo_library</span>
496                 <div>
497                     <h3>Escrutinio y Desglose Individual Foto a Foto (5 Fotos por Período)</h3>
498                     <p>Detalle individual de cada captura fotográfica comparado directamente con la telemetría ambiental exacta</p>
499                 </div>
500             </div>
501         </div>
502
503         <!-- Botón maestro para Plegar/Desplegar todas las sesiones -->
504         <div style="display: flex; gap: 8px;">
505             <button type="button" class="btn btn-outline btn-sm" id="btn-toggle-all-photos" onclick="toggleAllPhotoSessions()">
506                 <span class="material-icons-round" id="icon-toggle-all">unfold_more</span>
507                 <span id="text-toggle-all">Desplegar Todas las Fotos</span>
508             </button>
509         </div>
510     </div>
511
512     {% if cross_data %}
513         <div style="display: flex; flex-direction: column; gap: 14px;" id="photo-sessions-accordion">
514             {% for s in cross_data %}
515                 <div class="session-accordion-card" id="session-card-{{ s.session_id }}" data-period="{{ s.period_name }}" data-date="{{ s.date_str }}"
style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; overflow: hidden;">
516                     <!-- Cabecera de la sesión (Clickeable para plegar/desplegar) -->
517                     <div style="padding: 14px 18px; background: #f8fafc; border-bottom: 1px solid var(--border-color); display: flex; justify-content: space-between;
align-items: center; cursor: pointer; user-select: none;"
518                          onclick="toggleSinglePhotoSession({{ s.session_id }})">
519                         <div style="display: flex; align-items: center; gap: 10px;">
520                             <span class="material-icons-round accordion-arrow-icon" id="arrow-session-{{ s.session_id }}" style="font-size: 20px; transition: transform
0.2s; transform: rotate(0deg);">
521                                 chevron_right
522                             </span>
523                             <div>
524                                 <span style="font-weight: 700; font-size: 14px; color: var(--text-heading);">{{ s.full_period_label }}</span>
525                                 <span style="font-size: 12px; color: var(--text-muted); margin-left: 8px;">| Hora: <strong>{{ s.exact_time }}</strong> | Área Media:
<strong style="color: var(--accent-success);">{{ s.foliar_area_cm2 }} cm²</strong></span>
526                             </div>
527                         </div>
528
529                         <div style="display: flex; align-items: center; gap: 10px; flex-wrap: wrap;">
530                             <span style="font-size: 12px; color: #2563eb; font-weight: 600;" title="Temperatura Ambiente">{{ s.temperature_c }} °C</span>
531                             <span style="font-size: 12px; color: #0284c7; font-weight: 600;" title="Humedad Relativa">{{ s.humidity_rh }}%</span>
532                             <span style="font-size: 12px; color: #ec4899; font-weight: 600;" title="pH">{{ s.water_ph }} pH</span>
533                             <span style="font-size: 12px; color: #8b5cf6; font-weight: 600;" title="Conductividad">{{ s.ec_conductivity }} µS/cm</span>
534                             <span style="font-size: 12px; color: #06b6d4; font-weight: 600;" title="Oxígeno Disuelto">{{ s.dissolved_oxygen_mg_l }} mg/L</span>
535                             <span style="font-size: 12px; color: #0ea5e9; font-weight: 600;" title="Temperatura del Agua">{{ s.water_temp_c }} °C</span>
536                             <span class="badge badge-success">{{ s.individual_photos | length }} Fotos</span>
537                         </div>
538                     </div>
539
540                     <!-- Contenedor plegable de fotos individuales -->
541                     <div class="session-photos-content" id="photos-content-{{ s.session_id }}" style="display: none; padding: 14px 18px;">
```

```
                        <div class="table-responsive">
                            <table class="table" style="font-size: 12.5px; margin: 0;">
                                <thead>
                                    <tr>
                                        <th>Toma</th>
                                        <th>Vista Cenital</th>
                                        <th>Vista Lateral</th>
                                        <th>Área Foliar</th>
                                        <th>Altura</th>
                                        <th>Diámetro</th>
                                        <th>Salud</th>
                                        <th>Temp. Amb.</th>
                                        <th>Humedad</th>
                                        <th>pH</th>
                                        <th>C.E.</th>
                                        <th>Turbidez</th>
                                        <th>O₂ Disuelto</th>
                                        <th>Temp. Agua</th>
                                        <th>Bomba</th>
                                    </tr>
                                </thead>
                                <tbody>
                                    {% for p in s.individual_photos %}
                                        <tr>
                                            <td>
                                                <span class="badge badge-primary">Toma #{{ p.photo_num }}</span>
                                            </td>
                                            <td>
                                                {% if p.cenital_url %}
                                                    <img src="{{ p.cenital_url }}" class="photo-thumb-mini" alt="Cenital {{ p.photo_num }}" onclick="openLightbox('{{ p.cenital_url }}', 'Toma #{{ p.photo_num }} Cenital - {{ s.full_period_label }}')" style="cursor: pointer;" title="Clic para ampliar">
                                                {% else %}
                                                    <span style="color: var(--text-muted); font-size: 11px;">Sin imagen</span>
                                                {% endif %}
                                            </td>
                                            <td>
                                                {% if p.lateral_url %}
                                                    <img src="{{ p.lateral_url }}" class="photo-thumb-mini" alt="Lateral {{ p.photo_num }}" onclick="openLightbox('{{ p.lateral_url }}', 'Toma #{{ p.photo_num }} Lateral - {{ s.full_period_label }}')" style="cursor: pointer;" title="Clic para ampliar">
                                                {% else %}
                                                    <span style="color: var(--text-muted); font-size: 11px;">Sin imagen</span>
                                                {% endif %}
                                            </td>
                                            <td style="font-weight: 700; color: var(--accent-success);">{{ p.foliar_area_cm2 }} cm²</td>
                                            <td>{{ p.plant_height_cm }} cm</td>
                                            <td>{{ p.stem_diameter_mm }} mm</td>
                                            <td>
                                                <span class="badge {% if p.health_index >= 80 %}badge-success{% elif p.health_index >= 60 %}badge-warning{% else %}badge-danger{% endif %}">
                                                    {{ p.health_index }}%
                                                </span>
                                            </td>
                                            <td><span style="font-weight: 600; color: #2563eb;">{{ p.temperature_c }} °C</span></td>
                                            <td><span style="font-weight: 600; color: #0284c7;">{{ p.humidity_rh }}%</span></td>
                                            <td><span style="font-weight: 600; color: #ec4899;">{{ p.water_ph }}</span></td>
                                            <td><span style="font-weight: 600; color: #8b5cf6;">{{ p.ec_conductivity }}</span></td>
                                            <td><span style="font-weight: 600; color: #f97316;">{{ p.turbidity_ntu }}</span></td>
```

```
                                        <td><span style="font-weight: 600; color: #06b6d4;">{{ p.dissolved_oxygen_mg_l }}</span></td>
                                        <td><span style="font-weight: 600; color: #0ea5e9;">{{ p.water_temp_c }} °C</span></td>
                                        <td>
                                            <span class="badge {% if p.motor_state == 1 %}badge-success{% else %}badge-secondary{% endif %}" style="font-size: 10px;">
                                                {{ "ON" if p.motor_state == 1 else "OFF" }}
                                            </span>
                                        </td>
                                    </tr>
                                {% endfor %}
                            </tbody>
                        </table>
                    </div>
                </div>
            </div>
        {% endfor %}
    </div>
{% else %}
    <p style="text-align: center; padding: 30px; color: var(--text-muted);">
        No hay tomas fotográficas registradas para desglosar.
    </p>
{% endif %}
</div>

<!-- SECCIÓN 7: BITÁCORA AGRONÓMICA DEL INVESTIGADOR Y GENERADOR DE REPORTES CIENTÍFICOS PDF -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div class="widget-header" style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 12px; margin-bottom: 18px; padding-bottom: 14px; border-bottom: lpx solid var(--border-color);">
        <div class="widget-header-title">
            <h3 style="font-size: 16px; font-weight: 800; color: var(--text-heading); margin: 0;">
                Bitácora de Investigación y Conclusiones Agronómicas
            </h3>
            <p style="font-size: 12px; color: var(--text-muted); margin: 2px 0 0 0;">
                Documentación científica, generación de fichas técnicas certificadas en PDF y registro de inferencias de la torre
            </p>
        </div>

        <!-- Botón de Generación de Ficha Técnica PDF -->
        <div style="display: flex; gap: 8px; flex-wrap: wrap;">
            <a href="/report/scientific?date={{ active_date if active_date != 'todos' else (available_dates[0] if available_dates else '') }}&plant_id={{ active_plant_id }}"
               target="_blank"
               class="btn btn-primary"
               style="background: #059669; border-color: #059669; font-weight: 700; display: inline-flex; align-items: center; gap: 6px; box-shadow: 0 4px 12px rgba(5, 150, 105, 0.25);"
               title="Generar e imprimir Ficha Técnica de Investigación en PDF">
                <span class="material-icons-round">picture_as_pdf</span>
                <span>Generar Ficha Técnica / Reporte PDF</span>
            </a>
            <a href="/api/cross_analysis/export_csv?plant_id={{ active_plant_id }}" class="btn btn-outline" style="font-weight: 600; display: inline-flex; align-items: center; gap: 4px;">
                <span class="material-icons-round">table_view</span>
                <span>Exportar Datos CSV</span>
            </a>
        </div>
    </div>
```

```
    <!-- TARJETA INFORMATIVA DEL REPORTE CIENTÍFICO -->
    <div style="background: linear-gradient(135deg, rgba(5, 150, 105, 0.08), rgba(2, 132, 199, 0.08)); border: 1px solid rgba(5, 150, 105, 0.25); border-radius: 10px; padding: 14px 18px; margin-bottom: 20px; display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 12px;">
        <div style="display: flex; align-items: center; gap: 12px;">
            <div style="width: 38px; height: 38px; border-radius: 8px; background: #059669; color: #ffffff; display: flex; align-items: center; justify-content: center; font-size: 20px;">
                <span class="material-icons-round">science</span>
            </div>
            <div>
                <strong style="font-size: 13.5px; color: var(--text-heading); display: block;">
                    Dossier Científico & Ficha Técnica Lista para Tesis y Publicaciones
                </strong>
                <span style="font-size: 11.5px; color: var(--text-muted);">
                    El informe PDF integra matrices estadísticas paramétricas (&amp;mu;, &amp;sigma;, IQR, CV%), correlaciones de Pearson, comparativa fotográfica (RAW vs Segmentada) y firmas de validación.
                </span>
            </div>
        </div>
        <a href="/report/scientific?date={{ active_date if active_date != 'todos' else (available_dates[0] if available_dates else '') }}&plant_id={{ active_plant_id }}"
           target="_blank"
           class="btn btn-outline btn-sm"
           style="background: #ffffff; color: #059669; border-color: #059669; font-weight: 700;">
            <span class="material-icons-round" style="font-size: 15px;">open_in_new</span>
            <span>Abrir Vista de Impresión PDF</span>
        </a>
    </div>

    <form id="agronomic-conclusion-form" onsubmit="saveAgronomicConclusion(event)" style="background: var(--bg-card-subtle); padding: 20px; border-radius: 12px; border: 1px solid var(--border-color); margin-bottom: 24px;">
        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 15px; margin-bottom: 18px;">
            <div>
                <label style="font-size: 12.5px; font-weight: 600; color: var(--text-muted); display: block; margin-bottom: 6px;">
                    Período de Evaluación:
                </label>
                <select id="conclusion-period-type" class="form-control">
                    <option value="diario">Conclusión Diaria</option>
                    <option value="semanal">Conclusión Semanal</option>
                    <option value="mensual">Conclusión Mensual</option>
                </select>
            </div>

            <div>
                <label style="font-size: 12.5px; font-weight: 600; color: var(--text-muted); display: block; margin-bottom: 6px;">
                    Fecha de la Observación:
                </label>
                <input type="date" id="conclusion-date" class="form-control" required value="{{ available_dates[0] if available_dates else '' }}">
            </div>

            <div>
                <label style="font-size: 12.5px; font-weight: 600; color: var(--text-muted); display: block; margin-bottom: 6px;">
                    Nombre del Investigador:
                </label>
                <input type="text" id="conclusion-author" class="form-control" value="{{ current_user.name }}" placeholder="Nombre del evaluador">
            </div>
        </div>

        <div style="display: flex; flex-direction: column; gap: 14px; margin-bottom: 18px;">
```

```
            <div>
                <label style="font-size: 13px; font-weight: 600; color: var(--text-heading); display: block; margin-bottom: 6px;">
                    1. Crecimiento Foliar, Altura y Vigor:
                </label>
                <textarea id="conclusion-growth" class="form-control" rows="2" placeholder="Evolución del área foliar, desarrollo foliar y vigor vegetal..."></textarea>
            </div>

            <div>
                <label style="font-size: 13px; font-weight: 600; color: var(--text-heading); display: block; margin-bottom: 6px;">
                    2. Respuesta Ambiental (Temperatura, Humedad, Luz UV):
                </label>
                <textarea id="conclusion-climate" class="form-control" rows="2" placeholder="Respuesta del cultivo a los rangos climáticos registrados..."></textarea>
            </div>

            <div>
                <label style="font-size: 13px; font-weight: 600; color: var(--text-heading); display: block; margin-bottom: 6px;">
                    3. Conclusión General y Decisiones Agronómicas:
                </label>
                <textarea id="conclusion-general" class="form-control" rows="3" required placeholder="Conclusión final del evaluador para el reporte de investigación..."></textarea>
            </div>
        </div>

        <div style="display: flex; justify-content: flex-end;">
            <button type="submit" id="btn-save-conclusion" class="btn btn-primary">
                <span class="material-icons-round">save</span>
                <span>Guardar Conclusión de Investigación</span>
            </button>
        </div>

        <div id="conclusion-feedback" style="display: none; margin-top: 12px; font-size: 13px; padding: 10px; border-radius: 8px;"></div>
    </form>

    <!-- Historial de Conclusiones Guardadas -->
    <div>
        <h4 style="font-size: 14px; font-weight: 700; color: var(--text-heading); margin-bottom: 14px;">
            Historial de Conclusiones Registradas:
        </h4>

        {% if conclusions %}
            <div style="display: flex; flex-direction: column; gap: 14px;">
                {% for note in conclusions %}
                    <div id="note-card-{{ note.id }}" style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 18px;">
                        <div style="display: flex; justify-content: space-between; align-items: flex-start; margin-bottom: 10px;">
                            <div>
                                <span class="badge badge-success" style="font-size: 11.5px; text-transform: uppercase;">
                                    {{ note.period_type }}
                                </span>
                                <span style="font-size: 13.5px; font-weight: 700; color: var(--text-heading); margin-left: 8px;">
                                    Fecha: {{ note.date_str }}
                                </span>
                                <span style="font-size: 12px; color: var(--text-muted); margin-left: 8px;">
                                    | Autor: <strong>{{ note.author }}</strong> ({{ note.created_at }})
                                </span>
                            </div>
                            <button type="button" class="btn btn-danger btn-sm" onclick="deleteConclusion({{ note.id }})" title="Eliminar conclusión">
```

```
                                <span class="material-icons-round">delete</span>
                            </button>
                        </div>

                        {% if note.growth_obs %}
                            <p style="font-size: 13px; margin-bottom: 6px;">
                                <strong>Crecimiento y Vigor:</strong> {{ note.growth_obs }}
                            </p>
                        {% endif %}

                        {% if note.climate_obs %}
                            <p style="font-size: 13px; margin-bottom: 6px;">
                                <strong>Respuesta Ambiental:</strong> {{ note.climate_obs }}
                            </p>
                        {% endif %}

                        <div style="background: var(--bg-card-subtle); padding: 12px 14px; border-radius: 8px; margin-top: 8px; border-left: 4px solid var(--accent-success);">
                            <p style="font-size: 13px; font-weight: 600; color: var(--text-heading); margin: 0;">
                                {{ note.general_conclusion }}
                            </p>
                        </div>
                    </div>
                {% endfor %}
            </div>
        {% else %}
            <p style="font-size: 13px; color: var(--text-muted); text-align: center; padding: 20px;">
                No se han registrado conclusiones agronómicas para esta canastilla aún.
            </p>
        {% endif %}
    </div>
</div>

<!-- MODAL PARA VER FOTOS INDIVIDUALES DE UN PERÍODO ESPECÍFICO -->
<div id="photosModal" style="display: none; position: fixed; inset: 0; background: rgba(15, 23, 42, 0.65); backdrop-filter: blur(4px); z-index: 9999; align-items: center; justify-content: center; padding: 20px;">
    <div style="background: #ffffff; border-radius: 16px; width: 100%; max-width: 960px; max-height: 90vh; overflow-y: auto; padding: 24px; box-shadow: 0 20px 25px -5px rgba(0,0,0,0.2);">
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 16px; border-bottom: 1px solid var(--border-color); padding-bottom: 12px;">
            <div>
                <h3 id="modalPeriodTitle" style="font-size: 16px; font-weight: 700; color: var(--text-heading); margin: 0;">Desglose de Fotos del Período</h3>
                <p id="modalPeriodSub" style="font-size: 12px; color: var(--text-muted); margin: 2px 0 0 0;">5 capturas individuales analizadas</p>
            </div>
            <button type="button" class="btn btn-outline btn-sm" onclick="closePhotosModal()">
                <span class="material-icons-round">close</span>
            </button>
        </div>

        <div id="modalPhotosGrid" style="display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 16px;">
            <!-- Contenido dinámico inyectado por JavaScript -->
        </div>
    </div>
</div>

<!-- LIGHTBOX PARA AMPLIAR FOTOS EN ALTA RESOLUCIÓN -->
```

```
<div id="imageLightbox" style="display: none; position: fixed; inset: 0; background: rgba(0, 0, 0, 0.85); backdrop-filter: blur(6px); z-index: 10000; align-items: center; justify-content: center; flex-direction: column; padding: 20px;" onclick="closeLightbox()">
    <div style="position: relative; max-width: 90vw; max-height: 85vh;" onclick="event.stopPropagation()">
        <img id="lightboxImg" src="" style="max-width: 100%; max-height: 80vh; border-radius: 12px; box-shadow: 0 25px 50px -12px rgba(0,0,0,0.5); object-fit: contain;">
        <div id="lightboxTitle" style="text-align: center; color: #ffffff; font-size: 14px; font-weight: 600; margin-top: 10px;"></div>
        <button type="button" class="btn btn-sm" style="position: absolute; top: -12px; right: -12px; background: #ffffff; border-radius: 50%; width: 32px; height: 32px; padding: 0; display: flex; align-items: center; justify-content: center; box-shadow: 0 4px 6px rgba(0,0,0,0.3);" onclick="closeLightbox()">
            <span class="material-icons-round" style="font-size: 18px; color: #0f172a;">close</span>
        </button>
    </div>
</div>

{% endblock %}

{% block scripts %}
<script>
    const rawCrossData = {{ cross_data | tojson | safe }};
    const activePlantId = {{ active_plant_id }};
    let currentPeriodFilter = 'todos';
    let currentDateFilter = 'todos';
    let currentChartType = 'scatter';
    let customComparatorChart = null;
    let dedicCharts = {};
    let allPhotosExpanded = false;

    Chart.defaults.font.family = "'Inter', sans-serif";
    Chart.defaults.color = '#64748b';

    const varMeta = {
        foliar_area_cm2: { label: 'Área Foliar', unit: 'cm²', color: '#059669', type: 'pheno' },
        plant_height_cm: { label: 'Altura de Planta', unit: 'cm', color: '#16a34a', type: 'pheno' },
        stem_diameter_mm: { label: 'Diámetro de Tallo', unit: 'mm', color: '#84cc16', type: 'pheno' },
        health_index: { label: 'Índice de Salud', unit: '%', color: '#10b981', type: 'pheno' },
        compacity_index: { label: 'Compacidad', unit: 'pts', color: '#047857', type: 'pheno' },
        temperature_c: { label: 'Temperatura Ambiente', unit: '°C', color: '#2563eb', type: 'sensor' },
        humidity_rh: { label: 'Humedad Relativa', unit: '% HR', color: '#0284c7', type: 'sensor' },
        uv_solar_lux: { label: 'Radiación Solar', unit: 'lux', color: '#d97706', type: 'sensor' },
        motor_current_a: { label: 'Corriente Bomba', unit: 'A', color: '#059669', type: 'sensor' },
        ec_conductivity: { label: 'Conductividad Eléctrica', unit: 'µS/cm', color: '#8b5cf6', type: 'sensor' },
        water_ph: { label: 'pH de la Solución', unit: 'pH', color: '#ec4899', type: 'sensor' },
        turbidity_ntu: { label: 'Turbidez del Agua', unit: 'NTU', color: '#f97316', type: 'sensor' },
        dissolved_oxygen_mg_l: { label: 'Oxígeno Disuelto', unit: 'mg/L', color: '#06b6d4', type: 'sensor' },
        water_temp_c: { label: 'Temperatura del Agua', unit: '°C', color: '#0ea5e9', type: 'sensor' },
        motor_state: { label: 'Estado Bomba (1/0)', unit: 'pts', color: '#059669', type: 'sensor' }
    };

    function getFilteredCrossData() {
        return rawCrossData.filter(d => {
            const matchPeriod = (currentPeriodFilter === 'todos') || (d.period_name && d.period_name.includes(currentPeriodFilter));
            const matchDate = (currentDateFilter === 'todos') || (d.date_str === currentDateFilter);
            return matchPeriod && matchDate;
        });
    }

    function selectCustomPair(sensorKey, phenoKey) {
        document.getElementById('select-sensor-var').value = sensorKey;
```

```
        document.getElementById('select-pheno-var').value = phenoKey;

        document.querySelector('.comparator-lab-card').scrollIntoView({ behavior: 'smooth', block: 'start' });
        renderCustomComparatorChart();
    }

    function setChartType(type, btnElement) {
        currentChartType = type;
        document.querySelectorAll('.comparator-lab-card .filter-btn').forEach(b => b.classList.remove('active'));
        if (btnElement) btnElement.classList.add('active');
        renderCustomComparatorChart();
    }

    function renderCustomComparatorChart() {
        const data = getFilteredCrossData();
        const phenoKey = document.getElementById('select-pheno-var').value;
        const sensorKey = document.getElementById('select-sensor-var').value;

        const pMeta = varMeta[phenoKey];
        const sMeta = varMeta[sensorKey];

        const ctx = document.getElementById('customComparatorChart');
        if (!ctx) return;
        if (customComparatorChart) customComparatorChart.destroy();

        if (!data || data.length === 0) {
            document.getElementById('custom-math-summary').innerText = 'Sin datos disponibles para el filtro actual.';
            return;
        }

        const labels = data.map(d => d.full_period_label || d.period);
        const phenoVals = data.map(d => d[phenoKey]);
        const sensorVals = data.map(d => d[sensorKey]);

        // Cálculo de Regresión y Pearson
        let r = 0, r2 = 0, slope = 0, intercept = 0;
        let hasVariance = false;

        if (data.length >= 2) {
            const n = data.length;
            const sumX = sensorVals.reduce((a, b) => a + b, 0);
            const sumY = phenoVals.reduce((a, b) => a + b, 0);
            const sumXY = sensorVals.reduce((acc, x, i) => acc + x * phenoVals[i], 0);
            const sumX2 = sensorVals.reduce((a, b) => a + b * b, 0);
            const sumY2 = phenoVals.reduce((a, b) => a + b * b, 0);

            const denom = (n * sumX2 - sumX * sumX) * (n * sumY2 - sumY * sumY);
            if (denom > 1e-6) {
                r = (n * sumXY - sumX * sumY) / Math.sqrt(denom);
                hasVariance = true;
            }
            r2 = r * r;

            const slopeDenom = (n * sumX2 - sumX * sumX);
            if (Math.abs(slopeDenom) > 1e-6) {
```

```
                slope = (n * sumXY - sumX * sumY) / slopeDenom;
                intercept = (sumY - slope * sumX) / n;
            } else {
                slope = 0;
                intercept = sumY / n;
            }
        }

        const signStr = r >= 0 ? '+' : '';
        if (hasVariance) {
            document.getElementById('custom-math-summary').innerHTML = `
                Pearson: <strong style="color: ${r >= 0 ? '#059669' : '#dc2626'};">${signStr}${r.toFixed(2)}</strong> |
                R²: <strong>${r2.toFixed(3)}</strong> |
                Tendencia: <em>y = ${slope.toFixed(2)}x ${intercept >= 0 ? '+' : ''}${intercept.toFixed(2)}</em>
            `;
        } else {
            document.getElementById('custom-math-summary').innerHTML = `
                <span>Valores constantes en la muestra (${sensorVals[0]} ${sMeta.unit}, ${phenoVals[0]} ${pMeta.unit})</span>
            `;
        }

        if (currentChartType === 'scatter') {
            const scatterPoints = data.map(d => ({
                x: d[sensorKey],
                y: d[phenoKey],
                label: d.full_period_label
            }));

            let minX = Math.min(...sensorVals);
            let maxX = Math.max(...sensorVals);
            if (minX === maxX) {
                minX = minX * 0.9 - 1;
                maxX = maxX * 1.1 + 1;
            }

            const trendLine = [
                { x: minX, y: slope * minX + intercept },
                { x: maxX, y: slope * maxX + intercept }
            ];

            customComparatorChart = new Chart(ctx, {
                type: 'scatter',
                data: {
                    datasets: [
                        {
                            label: `${pMeta.label} vs ${sMeta.label}`,
                            data: scatterPoints,
                            backgroundColor: pMeta.color,
                            borderColor: pMeta.color,
                            pointRadius: 6,
                            pointHoverRadius: 9
                        },
                        {
                            label: 'Línea de Tendencia',
                            data: trendLine,
```

```
                        type: 'line',
                        borderColor: '#ef4444',
                        borderWidth: 2,
                        borderDash: [5, 5],
                        fill: false,
                        pointRadius: 0
                    }
                ]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                plugins: {
                    tooltip: {
                        callbacks: {
                            label: (c) => ` ${c.raw.label || ''} -> ${sMeta.label}: ${c.raw.x} ${sMeta.unit}, ${pMeta.label}: ${c.raw.y} ${pMeta.unit}`
                        }
                    }
                },
                scales: {
                    x: {
                        title: { display: true, text: `${sMeta.label} (${sMeta.unit})` },
                        grid: { color: '#f1f5f9' }
                    },
                    y: {
                        title: { display: true, text: `${pMeta.label} (${pMeta.unit})` },
                        grid: { color: '#f1f5f9' }
                    }
                }
            }
        });

    } else if (currentChartType === 'dual') {
        customComparatorChart = new Chart(ctx, {
            type: 'line',
            data: {
                labels: labels,
                datasets: [
                    {
                        label: `${sMeta.label} (${sMeta.unit})`,
                        data: sensorVals,
                        borderColor: sMeta.color,
                        backgroundColor: sMeta.color + '15',
                        yAxisID: 'ySensor',
                        tension: 0.3,
                        borderWidth: 2.2,
                        fill: true,
                        pointRadius: 4
                    },
                    {
                        label: `${pMeta.label} (${pMeta.unit})`,
                        data: phenoVals,
                        borderColor: pMeta.color,
                        backgroundColor: pMeta.color + '20',
                        yAxisID: 'yPheno',
```

```
                            tension: 0.3,
                            borderWidth: 2.5,
                            fill: false,
                            pointRadius: 5
                        }
                    ]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    scales: {
                        x: { grid: { display: false } },
                        ySensor: {
                            type: 'linear',
                            position: 'left',
                            title: { display: true, text: `${sMeta.label} (${sMeta.unit})`, color: sMeta.color },
                            grid: { color: '#f1f5f9' }
                        },
                        yPheno: {
                            type: 'linear',
                            position: 'right',
                            title: { display: true, text: `${pMeta.label} (${pMeta.unit})`, color: pMeta.color },
                            grid: { display: false }
                        }
                    }
                }
            });

        } else if (currentChartType === 'radar') {
            const periods = ['mañana', 'mediodía', 'tarde'];
            const avgByPeriod = periods.map(p => {
                const subset = rawCrossData.filter(d => d.period_name && d.period_name.includes(p));
                if (!subset.length) return { p, pheno: 0, sensor: 0 };
                const avgP = subset.reduce((a, b) => a + b[phenoKey], 0) / subset.length;
                const avgS = subset.reduce((a, b) => a + b[sensorKey], 0) / subset.length;
                return { p: p.toUpperCase(), pheno: avgP, sensor: avgS };
            });

            customComparatorChart = new Chart(ctx, {
                type: 'radar',
                data: {
                    labels: ['Mañana (07:00)', 'Mediodía (12:00)', 'Tarde (17:00)'],
                    datasets: [
                        {
                            label: `${pMeta.label} (${pMeta.unit})`,
                            data: avgByPeriod.map(a => a.pheno),
                            borderColor: pMeta.color,
                            backgroundColor: pMeta.color + '33',
                            borderWidth: 2
                        },
                        {
                            label: `${sMeta.label} (${sMeta.unit})`,
                            data: avgByPeriod.map(a => a.sensor),
                            borderColor: sMeta.color,
                            backgroundColor: sMeta.color + '33',
```

```
                        borderWidth: 2
                    }
                ]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false
            }
        });

    } else if (currentChartType === 'polar') {
        customComparatorChart = new Chart(ctx, {
            type: 'polarArea',
            data: {
                labels: labels,
                datasets: [{
                    label: `${pMeta.label}`,
                    data: phenoVals,
                    backgroundColor: [
                        'rgba(37, 99, 235, 0.65)',
                        'rgba(5, 150, 105, 0.65)',
                        'rgba(217, 119, 6, 0.65)',
                        'rgba(147, 51, 234, 0.65)',
                        'rgba(16, 185, 129, 0.65)',
                        'rgba(239, 68, 68, 0.65)'
                    ]
                }]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false
            }
        });

    } else if (currentChartType === 'bar') {
        customComparatorChart = new Chart(ctx, {
            type: 'bar',
            data: {
                labels: labels,
                datasets: [
                    {
                        label: `${pMeta.label} (${pMeta.unit})`,
                        data: phenoVals,
                        backgroundColor: pMeta.color,
                        borderRadius: 6
                    },
                    {
                        label: `${sMeta.label} (${sMeta.unit})`,
                        data: sensorVals,
                        backgroundColor: sMeta.color,
                        borderRadius: 6
                    }
                ]
            },
            options: {
```

```
                    responsive: true,
                    maintainAspectRatio: false,
                    scales: {
                        x: { grid: { display: false } },
                        y: { grid: { color: '#f1f5f9' } }
                    }
                }
            });
        }
    }

    // Inicializar gráficas individuales dedicadas
    function initDedicatedCharts() {
        const data = getFilteredCrossData();
        const labels = data.map(d => d.full_period_label || d.period);

        // 1. Área Foliar
        const ctxFoliar = document.getElementById('dedicFoliarChart');
        if (ctxFoliar) {
            if (dedicCharts.foliar) dedicCharts.foliar.destroy();
            dedicCharts.foliar = new Chart(ctxFoliar, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [{
                        label: 'Área Foliar (cm²)',
                        data: data.map(d => d.foliar_area_cm2),
                        borderColor: '#059669',
                        backgroundColor: 'rgba(5, 150, 105, 0.15)',
                        borderWidth: 2.5,
                        fill: true,
                        tension: 0.3,
                        pointRadius: 4
                    }]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    plugins: { legend: { display: false } },
                    scales: { x: { grid: { display: false } }, y: { grid: { color: '#f1f5f9' }, ticks: { callback: v => `${v} cm²` } } }
                }
            });
        }

        // 2. Altura y Diámetro
        const ctxMorph = document.getElementById('dedicMorphChart');
        if (ctxMorph) {
            if (dedicCharts.morph) dedicCharts.morph.destroy();
            dedicCharts.morph = new Chart(ctxMorph, {
                type: 'bar',
                data: {
                    labels: labels,
                    datasets: [
                        {
                            label: 'Altura (cm)',
```

```
                            data: data.map(d => d.plant_height_cm),
                            backgroundColor: '#16a34a',
                            borderRadius: 6
                        },
                        {
                            label: 'Diámetro Tallo (mm)',
                            data: data.map(d => d.stem_diameter_mm),
                            backgroundColor: '#84cc16',
                            borderRadius: 6
                        }
                    ]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    scales: { x: { grid: { display: false } }, y: { grid: { color: '#f1f5f9' } } }
                }
            });
        }

        // 3. Clima Temp y Humedad
        const ctxClimate = document.getElementById('dedicClimateChart');
        if (ctxClimate) {
            if (dedicCharts.climate) dedicCharts.climate.destroy();
            dedicCharts.climate = new Chart(ctxClimate, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [
                        {
                            label: 'Temperatura (°C)',
                            data: data.map(d => d.temperature_c),
                            borderColor: '#2563eb',
                            backgroundColor: 'rgba(37, 99, 235, 0.12)',
                            tension: 0.3,
                            borderWidth: 2,
                            fill: true,
                            pointRadius: 4
                        },
                        {
                            label: 'Humedad (% HR)',
                            data: data.map(d => d.humidity_rh),
                            borderColor: '#0284c7',
                            backgroundColor: 'transparent',
                            tension: 0.3,
                            borderWidth: 2,
                            pointRadius: 4
                        }
                    ]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    scales: { x: { grid: { display: false } }, y: { grid: { color: '#f1f5f9' } } }
                }
```

```
            });
        }

        // 4. Radiación y Bombeo
        const ctxSolar = document.getElementById('dedicSolarChart');
        if (ctxSolar) {
            if (dedicCharts.solar) dedicCharts.solar.destroy();
            dedicCharts.solar = new Chart(ctxSolar, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [
                        {
                            label: 'Radiación Solar (lux)',
                            data: data.map(d => d.uv_solar_lux),
                            borderColor: '#d97706',
                            backgroundColor: 'rgba(217, 119, 6, 0.15)',
                            tension: 0.3,
                            borderWidth: 2,
                            fill: true,
                            yAxisID: 'yLux',
                            pointRadius: 4
                        },
                        {
                            label: 'Corriente Bomba (A)',
                            data: data.map(d => d.motor_current_a),
                            borderColor: '#059669',
                            backgroundColor: 'transparent',
                            tension: 0.3,
                            borderWidth: 2,
                            yAxisID: 'yAmp',
                            pointRadius: 4
                        }
                    ]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    scales: {
                        x: { grid: { display: false } },
                        yLux: { type: 'linear', position: 'left', grid: { color: '#f1f5f9' }, title: { display: true, text: 'lux' } },
                        yAmp: { type: 'linear', position: 'right', grid: { display: false }, title: { display: true, text: 'Amperios' } }
                    }
                }
            });
        }

        // 5. pH y Oxígeno Disuelto
        const ctxPh = document.getElementById('dedicWaterPhChart');
        if (ctxPh) {
            if (dedicCharts.waterPh) dedicCharts.waterPh.destroy();
            dedicCharts.waterPh = new Chart(ctxPh, {
                type: 'line',
                data: {
                    labels: labels,
```

```
                datasets: [
                    {
                        label: 'pH de la Solución',
                        data: data.map(d => d.water_ph),
                        borderColor: '#ec4899',
                        backgroundColor: 'transparent',
                        tension: 0.3,
                        borderWidth: 2.2,
                        yAxisID: 'yPh',
                        pointRadius: 4
                    },
                    {
                        label: 'Oxigeno Disuelto (mg/L)',
                        data: data.map(d => d.dissolved_oxygen_mg_l),
                        borderColor: '#06b6d4',
                        backgroundColor: 'rgba(6, 182, 212, 0.12)',
                        tension: 0.3,
                        borderWidth: 2.2,
                        fill: true,
                        yAxisID: 'yDo',
                        pointRadius: 4
                    }
                ]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                scales: {
                    x: { grid: { display: false } },
                    yPh: { type: 'linear', position: 'left', grid: { color: '#f1f5f9' }, title: { display: true, text: 'pH' }, min: 4, max: 9 },
                    yDo: { type: 'linear', position: 'right', grid: { display: false }, title: { display: true, text: 'mg/L' } }
                }
            }
        });
    }

    // 6. Conductividad, Turbidez y Temperatura del Agua
    const ctxQuality = document.getElementById('dedicWaterQualityChart');
    if (ctxQuality) {
        if (dedicCharts.waterQuality) dedicCharts.waterQuality.destroy();
        dedicCharts.waterQuality = new Chart(ctxQuality, {
            type: 'line',
            data: {
                labels: labels,
                datasets: [
                    {
                        label: 'Conductividad (µS/cm)',
                        data: data.map(d => d.ec_conductivity),
                        borderColor: '#8b5cf6',
                        backgroundColor: 'transparent',
                        tension: 0.3,
                        borderWidth: 2.2,
                        yAxisID: 'yEc',
                        pointRadius: 4
                    },
```

```
                    {
                        label: 'Turbidez (NTU)',
                        data: data.map(d => d.turbidity_ntu),
                        borderColor: '#f97316',
                        backgroundColor: 'transparent',
                        tension: 0.3,
                        borderWidth: 2,
                        yAxisID: 'yTurb',
                        pointRadius: 4
                    },
                    {
                        label: 'Temp. Agua (°C)',
                        data: data.map(d => d.water_temp_c),
                        borderColor: '#0ea5e9',
                        backgroundColor: 'rgba(14, 165, 233, 0.10)',
                        tension: 0.3,
                        borderWidth: 2,
                        fill: true,
                        yAxisID: 'yTemp',
                        pointRadius: 4
                    }
                ]
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
                scales: {
                    x: { grid: { display: false } },
                    yEc: { type: 'linear', position: 'left', grid: { color: '#f1f5f9' }, title: { display: true, text: 'µS/cm' } },
                    yTurb: { type: 'linear', position: 'right', grid: { display: false }, title: { display: true, text: 'NTU' } },
                    yTemp: { type: 'linear', position: 'right', grid: { display: false }, title: { display: true, text: '°C' } }
                }
            }
        });
    }
}

function toggleSinglePhotoSession(sessionId) {
    const content = document.getElementById(`photos-content-${sessionId}`);
    const arrow = document.getElementById(`arrow-session-${sessionId}`);
    if (!content) return;

    if (content.style.display === 'none') {
        content.style.display = 'block';
        if (arrow) arrow.style.transform = 'rotate(90deg)';
    } else {
        content.style.display = 'none';
        if (arrow) arrow.style.transform = 'rotate(0deg)';
    }
}

function toggleAllPhotoSessions() {
    allPhotosExpanded = !allPhotosExpanded;
    const contents = document.querySelectorAll('.session-photos-content');
    const arrows = document.querySelectorAll('.accordion-arrow-icon');
```

```
        const btnText = document.getElementById('text-toggle-all');
        const btnIcon = document.getElementById('icon-toggle-all');

        contents.forEach(c => {
            c.style.display = allPhotosExpanded ? 'block' : 'none';
        });
        arrows.forEach(a => {
            a.style.transform = allPhotosExpanded ? 'rotate(90deg)' : 'rotate(0deg)';
        });

        if (btnText) btnText.innerText = allPhotosExpanded ? 'Plegar Todas las Fotos' : 'Desplegar Todas las Fotos';
        if (btnIcon) btnIcon.innerText = allPhotosExpanded ? 'unfold_less' : 'unfold_more';
    }

    function filterCrossPeriod(period, btnElement) {
        currentPeriodFilter = period.toLowerCase();
        document.querySelectorAll('.cross-period-btn').forEach(b => b.classList.remove('active'));
        if (btnElement) btnElement.classList.add('active');
        applyCrossFilters();
    }

    function filterCrossDate(dateStr, btnElement) {
        currentDateFilter = dateStr;
        document.querySelectorAll('.cross-date-btn').forEach(b => b.classList.remove('active'));
        if (btnElement) btnElement.classList.add('active');
        applyCrossFilters();
    }

    function applyCrossFilters() {
        const rows = document.querySelectorAll('#cross-table-body .cross-row');
        let visibleCount = 0;

        rows.forEach(r => {
            const rPeriod = r.getAttribute('data-period') || '';
            const rDate = r.getAttribute('data-date') || '';

            const matchPeriod = (currentPeriodFilter === 'todos') || rPeriod.includes(currentPeriodFilter);
            const matchDate = (currentDateFilter === 'todos') || (rDate === currentDateFilter);

            if (matchPeriod && matchDate) {
                r.style.display = '';
                visibleCount++;
            } else {
                r.style.display = 'none';
            }
        });

        // Filtrar tarjetas de acordeón de fotos
        document.querySelectorAll('.session-accordion-card').forEach(card => {
            const rPeriod = card.getAttribute('data-period') || '';
            const rDate = card.getAttribute('data-date') || '';
            const matchPeriod = (currentPeriodFilter === 'todos') || rPeriod.includes(currentPeriodFilter);
            const matchDate = (currentDateFilter === 'todos') || (rDate === currentDateFilter);
            card.style.display = (matchPeriod && matchDate) ? '' : 'none';
        });
```

```
1467
1468         const badge = document.getElementById('cross-table-count-badge');
1469         if (badge) badge.innerText = `${visibleCount} sesiones mostradas`;
1470
1471         renderCustomComparatorChart();
1472         initDedicatedCharts();
1473     }
1474
1475     function showIndividualPhotosModal(sessionId) {
1476         const session = rawCrossData.find(s => s.session_id === sessionId);
1477         if (!session) return;
1478
1479         document.getElementById('modalPeriodTitle').innerText = `Desglose Fotográfico: ${session.full_period_label}`;
1480         document.getElementById('modalPeriodSub').innerText = `${session.photos_count} capturas | T. Amb: ${session.temperature_c}°C, Hum: ${session.humidity_rh}%, pH:
    ${session.water_ph}, CE: ${session.ec_conductivity} µS/cm, OD: ${session.dissolved_oxygen_mg_l} mg/L, T. Agua: ${session.water_temp_c}°C`;
1481
1482         const grid = document.getElementById('modalPhotosGrid');
1483         grid.innerHTML = '';
1484
1485         if (session.individual_photos && session.individual_photos.length) {
1486             session.individual_photos.forEach((p, idx) => {
1487                 const card = document.createElement('div');
1488                 card.style.cssText = 'background: #f8fafc; border: 1px solid #e2e8f0; border-radius: 12px; padding: 14px; display: flex; flex-direction: column; gap:
    8px;';
1489
1490                 const cenitalImg = p.cenital_url ? `<img src="${p.cenital_url}" style="width: 100%; height: 130px; object-fit: cover; border-radius: 8px; border: 1px
    solid #cbd5e1; cursor: pointer;" onclick="openLightbox('${p.cenital_url}', 'Toma #${idx+1} Cenital')" title="Clic para ampliar">` : '';
1491                 const lateralImg = p.lateral_url ? `<img src="${p.lateral_url}" style="width: 100%; height: 130px; object-fit: cover; border-radius: 8px; border: 1px
    solid #cbd5e1; cursor: pointer;" onclick="openLightbox('${p.lateral_url}', 'Toma #${idx+1} Lateral')" title="Clic para ampliar">` : '';
1492
1493                 card.innerHTML = `
1494                     <div style="display: flex; justify-content: space-between; align-items: center;">
1495                         <span class="badge badge-primary">Toma #${idx + 1}</span>
1496                         <span style="font-size: 12px; font-weight: 700; color: #059669;">${p.foliar_area_cm2} cm²</span>
1497                     </div>
1498                     <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 8px;">
1499                         <div>
1500                             <div style="font-size: 10.5px; font-weight: 600; color: #64748b; margin-bottom: 2px;">Cenital</div>
1501                             ${cenitalImg || '<div style="height:130px; background:#e2e8f0; border-radius:8px; display:flex; align-items:center; justify-content:center;
    font-size:11px; color:#94a3b8;">N/A</div>'}
1502                         </div>
1503                         <div>
1504                             <div style="font-size: 10.5px; font-weight: 600; color: #64748b; margin-bottom: 2px;">Lateral</div>
1505                             ${lateralImg || '<div style="height:130px; background:#e2e8f0; border-radius:8px; display:flex; align-items:center; justify-content:center;
    font-size:11px; color:#94a3b8;">N/A</div>'}
1506                         </div>
1507                     </div>
1508                     <div style="font-size: 11.5px; color: #64748b; display: grid; grid-template-columns: 1fr 1fr; gap: 4px; margin-top: 4px;">
1509                         <div>Altura: <strong>${p.plant_height_cm} cm</strong></div>
1510                         <div>Tallo: <strong>${p.stem_diameter_mm} mm</strong></div>
1511                         <div>Salud: <strong>${p.health_index}%</strong></div>
1512                         <div>Compacidad: <strong>${p.compacity_index}</strong></div>
1513                     </div>
1514                     <div style="font-size: 11px; color: #475569; background: #e2e8f0; padding: 6px 8px; border-radius: 6px; display: grid; grid-template-columns: 1fr
    1fr; gap: 4px; margin-top: 4px;">
1515                         <div>pH: <strong>${p.water_ph}</strong></div>
1516                         <div>C.E.: <strong>${p.ec_conductivity}</strong></div>
1517                         <div>OD: <strong>${p.dissolved_oxygen_mg_l} mg/L</strong></div>
1518                         <div>T. Agua: <strong>${p.water_temp_c}°C</strong></div>
```

```
                    </div>
                `;
                grid.appendChild(card);
            });
        } else {
            grid.innerHTML = '<p style="color: #94a3b8; text-align: center; grid-column: 1/-1;">No hay fotos procesadas disponibles para esta sesión.</p>';
        }

        document.getElementById('photosModal').style.display = 'flex';
    }

    function closePhotosModal() {
        document.getElementById('photosModal').style.display = 'none';
    }

    function openLightbox(url, title) {
        if (!url) return;
        document.getElementById('lightboxImg').src = url;
        document.getElementById('lightboxTitle').innerText = title || '';
        document.getElementById('imageLightbox').style.display = 'flex';
    }

    function closeLightbox() {
        document.getElementById('imageLightbox').style.display = 'none';
    }

    async function saveAgronomicConclusion(e) {
        e.preventDefault();
        const pType = document.getElementById('conclusion-period-type').value;
        const dStr = document.getElementById('conclusion-date').value;
        const auth = document.getElementById('conclusion-author').value;
        const gObs = document.getElementById('conclusion-growth').value;
        const cObs = document.getElementById('conclusion-climate').value;
        const gen = document.getElementById('conclusion-general').value;

        const fb = document.getElementById('conclusion-feedback');
        fb.style.display = 'block';
        fb.style.background = 'rgba(5, 150, 105, 0.1)';
        fb.style.color = '#047857';
        fb.innerText = 'Guardando conclusión...';

        try {
            const res = await fetch('/api/conclusions', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    plant_id: activePlantId,
                    period_type: pType,
                    date_str: dStr,
                    author: auth,
                    growth_obs: gObs,
                    climate_obs: cObs,
                    general_conclusion: gen
                })
            });
```

```
            const data = await res.json();
            if (data.status === 'success') {
                fb.innerText = 'Conclusión registrada exitosamente.';
                setTimeout(() => window.location.reload(), 1000);
            } else {
                fb.style.background = 'rgba(220, 38, 38, 0.1)';
                fb.style.color = '#b91c1c';
                fb.innerText = 'Error al registrar: ' + data.message;
            }
        } catch (err) {
            fb.style.background = 'rgba(220, 38, 38, 0.1)';
            fb.style.color = '#b91c1c';
            fb.innerText = 'Error de conexión al servidor.';
        }
    }

    async function deleteConclusion(noteId) {
        if (!confirm('¿Deseas eliminar esta conclusión de investigación?')) return;
        try {
            const res = await fetch(`/api/conclusions/${noteId}`, { method: 'DELETE' });
            const data = await res.json();
            if (data.status === 'success') {
                const el = document.getElementById(`note-card-${noteId}`);
                if (el) el.remove();
            } else {
                alert('Error al eliminar: ' + data.message);
            }
        } catch (e) {
            alert('Error de comunicación.');
        }
    }

    async function deleteSessionFromCross(sessionId) {
        if (!confirm('¿Deseas eliminar este período y sus capturas asociadas?')) return;
        try {
            const res = await fetch(`/api/delete_session/${sessionId}`, { method: 'POST' });
            const data = await res.json();
            if (data.status === 'success') {
                window.location.reload();
            } else {
                alert('Error: ' + data.message);
            }
        } catch (e) {
            alert('Error al eliminar sesión.');
        }
    }

    document.addEventListener('DOMContentLoaded', () => {
        renderCustomComparatorChart();
        initDedicatedCharts();
    });
</script>
{% endblock %}
```

ARCHIVO 66/85: local_server/templates/dashboard.html (508 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Dashboard Analitico{% endblock %}
{% block heading %}Dashboard analítico{% endblock %}
{% block subheading %}Resumen general de la jornada: {{ target_date }}{% endblock %}

{% block header_controls %}
<!-- Navegador por Día / Fecha -->
<div class="date-navigator-box">
    <button class="date-nav-btn" id="btn-prev-day" title="Día Anterior">
        <span class="material-icons-round">chevron_left</span>
    </button>
    <div style="display: flex; align-items: center; gap: 4px;">
        <span class="material-icons-round text-muted" style="font-size: 16px;">calendar_today</span>
        <input type="date" id="dashboard-date-picker" class="date-selector-input" value="{{ target_date }}" onchange="navigateDate(this.value)">
    </div>
    <button class="date-nav-btn" id="btn-next-day" title="Día Siguiente">
        <span class="material-icons-round">chevron_right</span>
    </button>
</div>

<!-- Selector de Canastilla / Torre -->
<div style="display: flex; align-items: center; gap: 6px;">
    <select id="header-plant-selector" class="form-input-control" style="padding: 6px 10px; font-weight: 600; font-size: 13px; cursor: pointer; width: auto;" onchange="switchActivePlant(this.value)">
        <option value="1" {% if active_plant_id == 1 %}selected{% endif %}>Canastilla #1</option>
        <option value="2" {% if active_plant_id == 2 %}selected{% endif %}>Canastilla #2</option>
        <option value="3" {% if active_plant_id == 3 %}selected{% endif %}>Canastilla #3</option>
        <option value="4" {% if active_plant_id == 4 %}selected{% endif %}>Canastilla #4</option>
    </select>
</div>

<!-- Botones Exportar del Día -->
<div style="display: flex; gap: 8px;">
    <a href="/api/daily_summary/export_csv?date={{ target_date }}&plant_id={{ active_plant_id }}" class="btn-header-action" title="Exportar Resumen Consolidado del Día (CSV / Excel)">
        <span class="material-icons-round">today</span>
        <span>Exportar Día</span>
    </a>
    <a href="{{ url_for('api.export_cross_analysis_csv', plant_id=active_plant_id) }}" class="btn-header-action" style="background: rgba(255,255,255,0.08);" title="Exportar Dataset Científico Multivariable Completo en CSV">
        <span class="material-icons-round">download</span>
        <span>Histórico</span>
    </a>
</div>
{% endblock %}

{% block content %}
```

```
52
53 <!-- AVISO DE JORNADA SIN DATOS / ESTADO BASE CERO -->
54 {% if not daily.has_data %}
55 <div class="zero-data-banner">
56     <div class="zero-data-icon">
57         <span class="material-icons-round">event_busy</span>
58     </div>
59     <div class="zero-data-text">
60         <h4>Canastilla #{{ active_plant_id }} ({{ active_crop_profile.display_name }}) - Sin Muestreo en Fecha: {{ target_date }}</h4>
61         <p>No se encontraron capturas de fenotipado para esta fecha en la <strong>Canastilla #{{ active_plant_id }}</strong>. Todas las métricas de la planta se muestran en estado base 0. Para registrar datos, sube un lote en la sección de <em>Procesamiento</em> o selecciona una fecha con muestras registradas.</p>
62     </div>
63 </div>
64 {% endif %}
65
66 <!-- CONTENEDOR PRINCIPAL: DISTRIBUCIÓN LATERAL (TARJETAS KPI AL LADO DE LAS GRÁFICAS) -->
67 <div class="dashboard-split-layout">
68     <!-- COLUMNA LATERAL: 5 TARJETAS KPI RESUMEN -->
69     <div class="dashboard-kpi-column">
70         <!-- KPI 1: Área Foliar -->
71         <div class="kpi-card">
72             <div class="kpi-card-header">
73                 <div>
74                     <span class="kpi-label">Área Foliar</span>
75                     <div style="font-size: 10px; font-weight: 700; color: var(--accent-success); text-transform: uppercase; letter-spacing: 0.5px; margin-top: 1px;">Promedio del Día (3 Períodos)</div>
76                 </div>
77                 <div class="kpi-icon-badge emerald">
78                     <span class="material-icons-round">spa</span>
79                 </div>
80             </div>
81             <div class="kpi-value-row">
82                 <span class="kpi-big-number">{{ daily.kpis.foliar_area_cm2 }}</span>
83                 <span class="kpi-unit">cm²</span>
84             </div>
85             <div class="kpi-footer-badge {% if daily.kpis.foliar_area_diff >= 0 %}positive{% else %}neutral{% endif %}">
86                 <span class="material-icons-round" style="font-size: 14px;">{% if daily.kpis.foliar_area_diff >= 0 %}trending_up{% else %}trending_flat{% endif %}</span>
87                 <span>{% if daily.kpis.foliar_area_diff != 0.0 %}{{ '+ ' if daily.kpis.foliar_area_diff > 0 else '' }}{{ daily.kpis.foliar_area_diff }} cm² vs día ant.{% else %}Promedio consolidado{% endif %}</span>
88             </div>
89         </div>
90
91         <!-- KPI 2: Temperatura -->
92         <div class="kpi-card">
93             <div class="kpi-card-header">
94                 <div>
95                     <span class="kpi-label">Temperatura Media</span>
96                     <div style="font-size: 10px; font-weight: 700; color: #2563eb; text-transform: uppercase; letter-spacing: 0.5px; margin-top: 1px;">Promedio Diario (24h)</div>
97                 </div>
98                 <div class="kpi-icon-badge blue">
99                     <span class="material-icons-round">thermostat</span>
100                 </div>
101             </div>
102             <div class="kpi-value-row">
103                 <span class="kpi-big-number">{{ daily.kpis.temperature_c }}</span>
104                 <span class="kpi-unit">°C</span>
```

```
            </div>
            <div class="kpi-footer-badge {% if 'Óptimo' in daily.kpis.temperature_status %}positive{% else %}neutral{% endif %}">
                <span class="material-icons-round" style="font-size: 14px;">check_circle</span>
                <span>{{ daily.kpis.temperature_status }}</span>
            </div>
        </div>

        <!-- KPI 3: Humedad Relativa -->
        <div class="kpi-card">
            <div class="kpi-card-header">
                <div>
                    <span class="kpi-label">Humedad Relativa</span>
                    <div style="font-size: 10px; font-weight: 700; color: #0284c7; text-transform: uppercase; letter-spacing: 0.5px; margin-top: 1px;">Promedio Diario (24h)</div>
                </div>
                <div class="kpi-icon-badge cyan">
                    <span class="material-icons-round">water_drop</span>
                </div>
            </div>
            <div class="kpi-value-row">
                <span class="kpi-big-number">{{ daily.kpis.humidity_rh }}</span>
                <span class="kpi-unit">% HR</span>
            </div>
            <div class="kpi-footer-badge {% if 'Adecuado' in daily.kpis.humidity_status %}positive{% else %}neutral{% endif %}">
                <span class="material-icons-round" style="font-size: 14px;">info</span>
                <span>{{ daily.kpis.humidity_status }}</span>
            </div>
        </div>

        <!-- KPI 4: Radiación Solar UV -->
        <div class="kpi-card">
            <div class="kpi-card-header">
                <div>
                    <span class="kpi-label">Radiación Solar</span>
                    <div style="font-size: 10px; font-weight: 700; color: #d97706; text-transform: uppercase; letter-spacing: 0.5px; margin-top: 1px;">Promedio Diario</div>
                </div>
                <div class="kpi-icon-badge amber">
                    <span class="material-icons-round">wb_sunny</span>
                </div>
            </div>
            <div class="kpi-value-row">
                <span class="kpi-big-number">{{ daily.kpis.uv_solar_lux }}</span>
                <span class="kpi-unit">lux</span>
            </div>
            <div class="kpi-footer-badge neutral">
                <span class="material-icons-round" style="font-size: 14px;">light_mode</span>
                <span>{{ daily.kpis.uv_solar_status }}</span>
            </div>
        </div>

        <!-- KPI 5: Salud Vegetal -->
        <div class="kpi-card">
            <div class="kpi-card-header">
                <div>
                    <span class="kpi-label">Salud y Vigor Foliar</span>
```

```
                    <div style="font-size: 10px; font-weight: 700; color: var(--accent-success); text-transform: uppercase; letter-spacing: 0.5px; margin-top: 1px;">Promedio del Día</div>
                </div>
                <div class="kpi-icon-badge emerald">
                    <span class="material-icons-round">health_and_safety</span>
                </div>
            </div>
            <div class="kpi-value-row">
                <span class="kpi-big-number">{{ daily.kpis.health_index }}</span>
                <span class="kpi-unit">%</span>
            </div>
            <div class="kpi-footer-badge positive">
                <span class="material-icons-round" style="font-size: 14px;">eco</span>
                <span>{{ daily.kpis.health_status }}</span>
            </div>
        </div>

        <!-- KPI 6: Clima Exterior de la Zona (Open-Meteo API / Sincronización GPS) -->
        <div class="kpi-card" style="border-left: 3px solid var(--primary); position: relative;">
            <div class="kpi-card-header">
                <div style="display: flex; flex-direction: column;">
                    <span class="kpi-label">Clima Exterior</span>
                    <span id="weather-city-label" style="font-size: 10.5px; font-weight: 700; color: var(--primary); text-transform: uppercase; letter-spacing: 0.3px;">
                        {% if weather %}{{ weather.location_name }}{% else %}Santa Marta, Magdalena{% endif %}
                    </span>
                </div>
                <div style="display: flex; align-items: center; gap: 4px;">
                    <button type="button" onclick="syncBrowserLocation()" title="Sincronizar Ubicación GPS de la Torre" style="background: none; border: none; cursor: pointer; color: var(--text-muted); padding: 2px; display: flex; align-items: center; border-radius: 4px; transition: color 0.2s;" onmouseover="this.style.color='var(--primary)'" onmouseout="this.style.color='var(--text-muted)'">
                        <span class="material-icons-round" style="font-size: 16px;">my_location</span>
                    </button>
                    <div class="kpi-icon-badge blue" style="background: rgba(37, 99, 235, 0.12); color: #2563eb;">
                        <span class="material-icons-round" id="weather-main-icon">{% if weather %}{{ weather.icon }}{% else %}cloud{% endif %}</span>
                    </div>
                </div>
            </div>
            <div class="kpi-value-row">
                <span class="kpi-big-number" id="weather-temp-val">{% if weather %}{{ weather.temperature_c }}{% else %}--{% endif %}</span>
                <span class="kpi-unit">°C</span>
            </div>
            <div style="font-size: 11px; font-weight: 600; color: var(--text-secondary); margin: 2px 0 6px 0; display: flex; align-items: center; justify-content: space-between;">
                <span id="weather-desc-val">{% if weather %}{{ weather.weather_description }}{% else %}Cargando clima...{% endif %}</span>
                <span id="weather-wind-val" style="color: var(--text-muted); font-size: 10.5px;">{% if weather %}Viento: {{ weather.wind_speed_kmh }} km/h{% endif %}</span>
            </div>
            <div class="kpi-footer-badge {% if weather and weather.online %}positive{% else %}neutral{% endif %}" id="weather-footer-badge">
                <span class="material-icons-round" style="font-size: 13px;">{% if weather and weather.online %}wifi{% else %}wifi_off{% endif %}</span>
                <span id="weather-status-text">{% if weather %}{{ weather.status_text }}{% else %}Sincronizando{% endif %} &bull; HR: {{ weather.humidity_rh if weather else '--' }}%</span>
            </div>
        </div>
    </div>

    <!-- COLUMNA PRINCIPAL: 4 WIDGETS ANALÍTICOS Y GRÁFICAS -->
    <div class="dashboard-charts-main">
        <div class="analytics-grid-2x2">
```

```
            <!-- WIDGET 1: Temperatura y Curva Térmica de la Jornada -->
            <div class="widget-card">
                <div class="widget-header">
                    <div class="widget-header-title">
                        <h3>Temperatura y Clima de la Jornada</h3>
                        <p>Comportamiento térmico con banda de amplitud diaria</p>
                    </div>
                    <div class="widget-actions-box">
                        <span class="widget-badge-tag active">Jornada Diaria</span>
                    </div>
                </div>

                <div class="chart-stage-container">
                    <canvas id="thermalCurveChart"></canvas>
                </div>

                <div class="widget-stats-row">
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Máx. Diaria</span>
                        <span class="stat-item-value">{{ daily.thermal_curve.max_temp }} °C</span>
                        <span class="stat-item-desc">Pico térmico</span>
                    </div>
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Mín. Diaria</span>
                        <span class="stat-item-value">{{ daily.thermal_curve.min_temp }} °C</span>
                        <span class="stat-item-desc">Mínimo nocturno</span>
                    </div>
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Amplitud</span>
                        <span class="stat-item-value">{{ daily.thermal_curve.amplitude }} °C</span>
                        <span class="stat-item-desc">Delta térmico</span>
                    </div>
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Régimen</span>
                        <span class="stat-item-value">{{ daily.thermal_curve.regime }}</span>
                        <span class="stat-item-desc">Evaluación</span>
                    </div>
                </div>
            </div>

            <!-- WIDGET 2: Última Toma de Fenotipado (Cenital y Lateral) -->
            <div class="widget-card">
                <div class="widget-header">
                    <div class="widget-header-title">
                        <h3>Última Captura de Fenotipado</h3>
                        <p>{% if daily.latest_capture.has_images %}Período: <strong>{{ daily.latest_capture.period }}</strong> &bull; {{ daily.latest_capture.timestamp_str }}{% else %}Tomas registradas de la canastilla{% endif %}</p>
                    </div>
                    <div class="widget-actions-box">
                        {% if daily.latest_capture.has_images %}
                        <span class="widget-badge-tag active">{{ daily.latest_capture.period }}</span>
                        {% else %}
                        <span class="widget-badge-tag">Sin tomas</span>
                        {% endif %}
                    </div>
                </div>
```

```
266
267                 {% if daily.latest_capture.has_images %}
268                 <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 12px; padding: 12px 16px; background: rgba(0,0,0,0.02); min-height: 220px; align-items:
    center;">
269                     <!-- Toma Cenital -->
270                     <div style="display: flex; flex-direction: column; align-items: center; background: #ffffff; border: 1px solid var(--border-color); border-radius:
    10px; padding: 8px; box-shadow: 0 1px 3px rgba(0,0,0,0.04);">
271                         <div style="font-size: 11px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; letter-spacing: 0.5px; margin-bottom: 6px;
    display: flex; align-items: center; gap: 4px;">
272                             <span class="material-icons-round" style="font-size: 14px; color: var(--primary);">camera</span>
273                             <span>Cámara Cenital (ROI Canastilla)</span>
274                         </div>
275                         <div style="width: 100%; height: 165px; border-radius: 6px; overflow: hidden; background: #0f172a; display: flex; align-items: center; justify-
    content: center; position: relative;">
276                             <img src="{{ url_for('static', filename=daily.latest_capture.cenital_proc) }}" alt="Última Toma Cenital" style="width: 100%; height: 100%;
    object-fit: contain; display: block;" onerror="this.onerror=null; this.src='{{ url_for('static', filename=daily.latest_capture.cenital_orig) }}';">
277                             <span style="position: absolute; bottom: 6px; right: 6px; background: rgba(0,0,0,0.7); color: #fff; font-size: 10px; font-weight: 600;
    padding: 2px 6px; border-radius: 4px;">{{ daily.latest_capture.foliar_area_cm2 }} cm²</span>
278                         </div>
279                     </div>
280
281                     <!-- Toma Lateral -->
282                     <div style="display: flex; flex-direction: column; align-items: center; background: #ffffff; border: 1px solid var(--border-color); border-radius:
    10px; padding: 8px; box-shadow: 0 1px 3px rgba(0,0,0,0.04);">
283                         <div style="font-size: 11px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; letter-spacing: 0.5px; margin-bottom: 6px;
    display: flex; align-items: center; gap: 4px;">
284                             <span class="material-icons-round" style="font-size: 14px; color: var(--primary);">filter_center_focus</span>
285                             <span>Cámara Lateral (Perfil Vertical)</span>
286                         </div>
287                         <div style="width: 100%; height: 165px; border-radius: 6px; overflow: hidden; background: #0f172a; display: flex; align-items: center; justify-
    content: center; position: relative;">
288                             <img src="{{ url_for('static', filename=daily.latest_capture.lateral_proc) }}" alt="Última Toma Lateral" style="width: 100%; height: 100%;
    object-fit: contain; display: block;" onerror="this.onerror=null; this.src='{{ url_for('static', filename=daily.latest_capture.lateral_orig) }}';">
289                             <span style="position: absolute; bottom: 6px; right: 6px; background: rgba(0,0,0,0.7); color: #fff; font-size: 10px; font-weight: 600;
    padding: 2px 6px; border-radius: 4px;">{{ daily.latest_capture.plant_height_cm }} cm</span>
290                         </div>
291                     </div>
292                 </div>
293                 {% else %}
294                 <div style="display: flex; flex-direction: column; align-items: center; justify-content: center; height: 220px; color: var(--text-muted); text-align:
    center; padding: 20px;">
295                     <span class="material-icons-round" style="font-size: 40px; color: #cbd5e1; margin-bottom: 8px;">hide_image</span>
296                     <p style="font-weight: 600; margin: 0 0 4px 0; color: var(--text-secondary);">Sin capturas para esta fecha</p>
297                     <p style="font-size: 12px; margin: 0;">Selecciona otra fecha o realiza un lote de fenotipado para ver las tomas.</p>
298                 </div>
299                 {% endif %}
300
301                 <div class="widget-stats-row">
302                     <div class="widget-stats-item">
303                         <span class="stat-item-label">Área Foliar</span>
304                         <span class="stat-item-value">{{ daily.latest_capture.foliar_area_cm2 }} cm²</span>
305                         <span class="stat-item-desc">Última toma</span>
306                     </div>
307                     <div class="widget-stats-item">
308                         <span class="stat-item-label">Altura Planta</span>
309                         <span class="stat-item-value">{{ daily.latest_capture.plant_height_cm }} cm</span>
310                         <span class="stat-item-desc">Morfometría vertical</span>
311                     </div>
312                     <div class="widget-stats-item">
313                         <span class="stat-item-label">Diámetro Tallo</span>
314                         <span class="stat-item-value">{{ daily.latest_capture.stem_diameter_mm }} mm</span>
```

```
                    <span class="stat-item-desc">Espesor basal</span>
                </div>
                <div class="widget-stats-item">
                    <span class="stat-item-label">Índice Salud</span>
                    <span class="stat-item-value">{{ daily.latest_capture.health_index }}%</span>
                    <span class="stat-item-desc">Vigor botánico</span>
                </div>
            </div>
        </div>

        <!-- WIDGET 3: Estado de Bomba / Motor de Recirculación (1=ON, 0=OFF) -->
        <div class="widget-card">
            <div class="widget-header">
                <div class="widget-header-title">
                    <h3>Estado de Bomba (Recirculación)</h3>
                    <p>Ciclos de activación del motor (1 = Encendido, 0 = Apagado)</p>
                </div>
                <div class="widget-actions-box">
                    <span class="widget-badge-tag {% if daily.motor_chart.current_state == 1 %}active{% endif %}">
                        {{ daily.motor_chart.current_state_label }}
                    </span>
                </div>
            </div>

            <div class="chart-stage-container">
                <canvas id="motorStateChart"></canvas>
            </div>

            <div class="widget-stats-row">
                <div class="widget-stats-item">
                    <span class="stat-item-label">Estado Actual</span>
                    <span class="stat-item-value" style="color: {% if daily.motor_chart.current_state == 1 %}#059669{% else %}#64748b{% endif %};">
                        {{ daily.motor_chart.current_state_label }}
                    </span>
                    <span class="stat-item-desc">Último registro</span>
                </div>
                <div class="widget-stats-item">
                    <span class="stat-item-label">Tiempo Activo</span>
                    <span class="stat-item-value">{{ daily.motor_chart.active_pct }}%</span>
                    <span class="stat-item-desc">Del período diario</span>
                </div>
                <div class="widget-stats-item">
                    <span class="stat-item-label">Ciclos Riego</span>
                    <span class="stat-item-value">{{ daily.motor_chart.cycles_count }}</span>
                    <span class="stat-item-desc">Encendidos registrados</span>
                </div>
                <div class="widget-stats-item">
                    <span class="stat-item-label">Solución Nutritiva</span>
                    <span class="stat-item-value">{{ daily.motor_chart.status }}</span>
                    <span class="stat-item-desc">Operación</span>
                </div>
            </div>
        </div>

        <!-- WIDGET 4: Radiación Solar UV y Fotoperíodo -->
```

```
            <div class="widget-card">
                <div class="widget-header">
                    <div class="widget-header-title">
                        <h3>Radiación Solar UV (Irradiancia)</h3>
                        <p>Distribución de luz útil a lo largo de las horas</p>
                    </div>
                    <div class="widget-actions-box">
                        <span class="widget-badge-tag active">Luximetría</span>
                    </div>
                </div>

                <div class="chart-stage-container">
                    <canvas id="solarIrradianceChart"></canvas>
                </div>

                <div class="widget-stats-row">
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Radiación Media</span>
                        <span class="stat-item-value">{{ daily.solar_chart.avg_uv }} lux</span>
                        <span class="stat-item-desc">Media diurna</span>
                    </div>
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Máx. Radiación</span>
                        <span class="stat-item-value">{{ daily.solar_chart.max_uv }} lux</span>
                        <span class="stat-item-desc">Pico solar</span>
                    </div>
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Especie Activa</span>
                        <span class="stat-item-value">{{ selected_profile.display_name }}</span>
                        <span class="stat-item-desc">Perfil botánico</span>
                    </div>
                    <div class="widget-stats-item">
                        <span class="stat-item-label">Fotoperíodo</span>
                        <span class="stat-item-value">Adecuado</span>
                        <span class="stat-item-desc">Luz acumulada</span>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
{% endblock %}

{% block scripts %}
<script>
    const dailyData = {{ daily | tojson | safe }};
    const availableDates = {{ available_dates | tojson | safe }};
    const activePlantId = {{ active_plant_id }};

    function navigateDate(selectedDate) {
        if (!selectedDate) return;
        window.location.href = `/?date=${selectedDate}&plant_id=${activePlantId}`;
    }

    document.getElementById('btn-prev-day').addEventListener('click', () => {
```

```
        const currentDateStr = document.getElementById('dashboard-date-picker').value;
        const d = new Date(currentDateStr + 'T00:00:00');
        d.setDate(d.getDate() - 1);
        const prevStr = d.toISOString().split('T')[0];
        navigateDate(prevStr);
    });

    document.getElementById('btn-next-day').addEventListener('click', () => {
        const currentDateStr = document.getElementById('dashboard-date-picker').value;
        const d = new Date(currentDateStr + 'T00:00:00');
        d.setDate(d.getDate() + 1);
        const nextStr = d.toISOString().split('T')[0];
        navigateDate(nextStr);
    });
    // Sincronización inteligente de ubicación GPS para el Clima Exterior
    function syncBrowserLocation() {
        if (!navigator.geolocation) {
            console.log("Geolocalización no soportada por el navegador");
            return;
        }

        const cityLabel = document.getElementById('weather-city-label');
        if (cityLabel) cityLabel.innerText = "Sincronizando GPS...";

        navigator.geolocation.getCurrentPosition(
            async (position) => {
                const lat = position.coords.latitude;
                const lon = position.coords.longitude;
                localStorage.setItem('sifma_location_synced', 'true');

                try {
                    let cityName = "Santa Marta, Magdalena";
                    try {
                        // Geocodificación inversa ligera
                        const geoResp = await fetch(`https://nominatim.openstreetmap.org/reverse?format=json&lat=${lat}&lon=${lon}&zoom=10`);
                        if (geoResp.ok) {
                            const geoData = await geoResp.json();
                            cityName = geoData.address.city || geoData.address.town || geoData.address.state || cityName;
                        }
                    } catch (e) {}

                    const resp = await fetch('/api/config/location', {
                        method: 'POST',
                        headers: { 'Content-Type': 'application/json' },
                        body: JSON.stringify({ latitude: lat, longitude: lon, city_name: cityName })
                    });

                    const resData = await resp.json();
                    if (resData.status === 'success' && resData.weather) {
                        const w = resData.weather;
                        if (cityLabel) cityLabel.innerText = w.location_name;
                        const tempVal = document.getElementById('weather-temp-val');
                        if (tempVal) tempVal.innerText = w.temperature_c;
                        const descVal = document.getElementById('weather-desc-val');
                        if (descVal) descVal.innerText = w.weather_description;
```

```
                        const windVal = document.getElementById('weather-wind-val');
                        if (windVal) windVal.innerText = `Viento: ${w.wind_speed_kmh} km/h`;
                        const statusText = document.getElementById('weather-status-text');
                        if (statusText) statusText.innerHTML = `${w.status_text} &bull; HR: ${w.humidity_rh}%`;
                        const iconEl = document.getElementById('weather-main-icon');
                        if (iconEl) iconEl.innerText = w.icon;
                    }
                } catch (err) {
                    console.warn("Error actualizando coordenadas de la torre:", err);
                    if (cityLabel) cityLabel.innerText = "Santa Marta, Magdalena";
                }
            },
            (error) => {
                console.log("Ubicación rechazada o no disponible, usando Santa Marta por defecto:", error.message);
                if (cityLabel) cityLabel.innerText = "Santa Marta, Magdalena";
            },
            { timeout: 8000, maximumAge: 86400000 }
        );
    }

    // Solicitar permiso de ubicación automáticamente la primera vez
    document.addEventListener('DOMContentLoaded', () => {
        if (!localStorage.getItem('sifma_location_synced') && navigator.geolocation) {
            syncBrowserLocation();
        }
    });
</script>
<script src="{{ url_for('static', filename='js/dashboard.js') }}"></script>
{% endblock %}
```

ARCHIVO 67/85: local_server/templates/error.html (25 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Acceso Restringido{% endblock %}
{% block heading %}Aviso de Seguridad{% endblock %}
{% block subheading %}Permisos del sistema{% endblock %}

{% block content %}
<div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 14px; padding: 40px 24px; text-align: center; max-width: 550px; margin: 40px auto; box-shadow: var(--shadow-card);">
    <div style="width: 56px; height: 56px; border-radius: 50%; background: var(--accent-danger-subtle); color: var(--accent-danger); display: flex; align-items: center; justify-content: center; margin: 0 auto 16px auto;">
        <span class="material-icons-round" style="font-size: 32px;">shield</span>
    </div>
    <h2 style="font-family: var(--font-heading); font-size: 20px; font-weight: 700; color: var(--text-heading); margin-bottom: 8px;">{{ error_title or 'Acceso Restringido' }}</h2>
    <p style="font-size: 14px; color: var(--text-muted); margin-bottom: 24px; line-height: 1.5;">{{ error_message or 'No posees los permisos necesarios para realizar esta acción.' }}</p>
    <a href="{{ url_for('dashboard.index') }}" class="btn-header-action btn-header-primary" style="display: inline-flex; align-items: center; gap: 8px; text-decoration: none;">
        <span class="material-icons-round">arrow_back</span>
        <span>Volver al Dashboard Principal</span>
    </a>
</div>
{% endblock %}
```

ARCHIVO 68/85: local_server/templates/gallery.html (668 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Galería de Capturas{% endblock %}
{% block heading %}Comparador de Procesamiento Digital{% endblock %}
{% block subheading %}Inspección visual antes/después y segmentación para Canastilla #{{ active_plant_id }}{% endblock %}

{% block content %}

<!-- Filtros de Períodos y Selección de Capturas -->
<div class="widget-card" style="margin-bottom: 24px;">
    <div class="widget-header">
        <div class="widget-header-title">
            <h3>Filtros por Período de Muestreo (Canastilla #{{ active_plant_id }} - {{ active_crop_profile.display_name }})</h3>
            <p>Selecciona el turno de captura para explorar los lotes fotográficos</p>
        </div>
    </div>

    <!-- Botones de Período -->
    <div class="filter-group" style="margin-bottom: 20px;">
        <button class="filter-pill active" data-target-period="todos" onclick="filterPeriod('todos', this)">
            <span class="material-icons-round">apps</span>
            <span>Todos los Períodos</span>
        </button>
        <button class="filter-pill" data-target-period="mañana" onclick="filterPeriod('mañana', this)">
            <span class="material-icons-round">light_mode</span>
            <span>Mañana (07:00)</span>
        </button>
        <button class="filter-pill" data-target-period="mediodía" onclick="filterPeriod('mediodía', this)">
            <span class="material-icons-round">wb_sunny</span>
            <span>Mediodía (12:00)</span>
        </button>
        <button class="filter-pill" data-target-period="tarde" onclick="filterPeriod('tarde', this)">
            <span class="material-icons-round">nights_stay</span>
            <span>Tarde (17:00)</span>
        </button>
    </div>

    <!-- Lista Horizontal de Capturas Disponibles para el Período Activo -->
    <div style="border-top: 1px solid var(--border-light); padding-top: 18px;">
        <h4 style="font-size: 13.5px; font-weight: 700; margin-bottom: 14px; color: var(--text-heading);">
            Muestreos Registrados para esta Canastilla:
        </h4>

        <div id="captures-grid" style="display: flex; gap: 14px; overflow-x: auto; padding-bottom: 10px;">
            {% if gallery_sessions %}
                {% for g in gallery_sessions %}
                    {% set s = g.session %}
```

```
                    {% set m = g.avg_metric %}
                    <div class="capture-thumb-card" data-period="{{ s.period }}" data-session-id="{{ s.id }}" onclick="selectSession({{ s.id }})" style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 10px; cursor: pointer; min-width: 170px; box-shadow: var(--shadow-sm); transition: var(--transition-fast);">
                        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 4px; border-radius: 8px; overflow: hidden; height: 75px; background: #090d16; margin-bottom: 8px; border: 1px solid #1e293b;">
                            <img src="{{ url_for('static', filename=m.image_path_cenital_orig) }}" style="width: 100%; height: 100%; object-fit: contain;">
                            <img src="{{ url_for('static', filename=m.image_path_lateral_orig) }}" style="width: 100%; height: 100%; object-fit: contain;">
                        </div>
                        <div style="display: flex; flex-direction: column; gap: 2px;">
                            <span style="font-size: 11.5px; font-weight: 700; color: var(--text-heading);">{{ s.timestamp.strftime('%d/%m %H:%M') }}</span>
                            <span class="badge badge-success" style="font-size: 10px; padding: 2px 6px;">{{ s.period.upper() }}</span>
                        </div>
                    </div>
                {% endfor %}
            {% else %}
                <p style="color: var(--text-muted); font-size: 13px;">No hay capturas registradas para la Canastilla #{{ active_plant_id }}.</p>
            {% endif %}
        </div>

        <div id="no-captures-message" style="display: none; color: var(--text-muted); font-size: 13px; padding: 12px 0;">
            No se encontraron capturas registradas en este período para la canastilla activa.
        </div>
    </div>
</div>

<!-- SECCIÓN DETALLADA DEL COMPARADOR COMPARATIVO DE LA SESIÓN SELECCIONADA -->
<div id="comparators-container">
    {% if gallery_sessions %}
        {% for g in gallery_sessions %}
            {% set s = g.session %}
            {% set avg = g.avg_metric %}
            <div class="session-comparator-card" id="comparator-{{ s.id }}" data-session-json="{{ g.all_metrics_json | tojson | safe }}" style="display: none;">
                <div class="widget-card" style="margin-bottom: 24px;">

                    <!-- Cabecera de la sesión -->
                    <div class="widget-header">
                        <div style="display: flex; align-items: center; gap: 12px; flex-wrap: wrap;">
                            <span class="badge badge-success" style="font-size: 12px; padding: 6px 12px;">
                                {{ s.period.upper() }}
                            </span>
                            <div class="widget-header-title">
                                <h3>Comparador de Fenotipado - Captura del {{ s.timestamp.strftime('%d/%m/%Y a las %H:%M') }}</h3>
                            </div>
                        </div>
                        <div style="display: flex; gap: 8px; align-items: center;">
                            <a href="{{ url_for('dashboard.cross_analysis') }}" class="btn btn-primary btn-sm">
                                <span class="material-icons-round">sync_alt</span>
                                <span>Cruzar con Telemetría</span>
                            </a>
                            <button type="button" class="btn btn-danger btn-sm" onclick="deleteSession({{ s.id }})" title="Eliminar este período de fotos">
                                <span class="material-icons-round">delete</span>
                                <span>Eliminar Período</span>
                            </button>
                        </div>
                    </div>
```

```
107
108                     <!-- Selector de Tomas Individuales (Promedio vs Foto 1 a 5) -->
109                     <div style="background: var(--bg-card-subtle); padding: 14px 18px; border-radius: 10px; border: 1px solid var(--border-color); margin-bottom: 20px;">
110                         <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 10px;">
111                             <span style="font-size: 13px; font-weight: 700; color: var(--text-heading);">
112                                 Seleccionar Toma Específica del Lote:
113                             </span>
114                             <div class="photo-selector-group" id="photo-pills-{{ s.id }}" style="display: flex; gap: 6px; flex-wrap: wrap;">
115                                 <button type="button" class="btn btn-primary btn-sm" onclick="switchSessionPhoto({{ s.id }}, 0, this)">
116                                     Promedio Consolidado (5 tomas)
117                                 </button>
118                                 {% for p in g.individual_photos %}
119                                     <button type="button" class="btn btn-outline btn-sm" onclick="switchSessionPhoto({{ s.id }}, {{ p.photo_index }}, this)">
120                                         Foto #{{ p.photo_index }}
121                                     </button>
122                                 {% endfor %}
123                             </div>
124                         </div>
125                     </div>
126
127                     <!-- Datos Consolidados / Individuales de la sesión -->
128                     <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); gap: 14px; background: var(--bg-card-subtle); padding: 16px;
    border-radius: 12px; border: 1px solid var(--border-color); margin-bottom: 24px; text-align: center;">
129                         <div>
130                             <p style="font-size: 11.5px; color: var(--text-muted); font-weight: 600;">Área Foliar</p>
131                             <p id="metric-area-{{ s.id }}" style="font-size: 18px; font-family: var(--font-heading); font-weight: 700; color: var(--accent-success);
    margin-top: 4px;">
132                                 {{ avg.foliar_area_cm2 }} cm²
133                             </p>
134                         </div>
135                         <div>
136                             <p style="font-size: 11.5px; color: var(--text-muted); font-weight: 600;">Altura de Planta</p>
137                             <p id="metric-height-{{ s.id }}" style="font-size: 18px; font-family: var(--font-heading); font-weight: 700; color: var(--accent-primary);
    margin-top: 4px;">
138                                 {{ avg.plant_height_cm }} cm
139                             </p>
140                         </div>
141                         <div>
142                             <p style="font-size: 11.5px; color: var(--text-muted); font-weight: 600;">Diámetro del Tallo</p>
143                             <p id="metric-diameter-{{ s.id }}" style="font-size: 18px; font-family: var(--font-heading); font-weight: 700; color: var(--accent-warning);
    margin-top: 4px;">
144                                 {{ avg.stem_diameter_mm }} mm
145                             </p>
146                         </div>
147                         <div>
148                             <p style="font-size: 11.5px; color: var(--text-muted); font-weight: 600;">Índice de Salud</p>
149                             <p id="metric-health-{{ s.id }}" style="font-size: 18px; font-family: var(--font-heading); font-weight: 700; color: {% if avg.health_index >=
    90 %}var(--accent-success){% else %}var(--accent-danger){% endif %}; margin-top: 4px;">
150                                 {{ avg.health_index }}%
151                             </p>
152                         </div>
153                     </div>
154
155                     <!-- TARJETAS DE COMPARACIÓN DE IMÁGENES -->
156                     <div class="analytics-grid-2x2" style="display: grid; grid-template-columns: repeat(auto-fit, minmax(420px, 1fr)); gap: 18px;">
157
158                         <!-- COMPARADOR 1: CÁMARA CENITAL (VISTA SUPERIOR) -->
159                         <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 16px; box-shadow: var(--shadow-sm);">
```

```
                            <div style="display: flex; align-items: center; justify-content: space-between; margin-bottom: 12px;">
                                <div style="display: flex; align-items: center; gap: 8px;">
                                    <span class="material-icons-round" style="color: var(--accent-success);">camera</span>
                                    <strong style="font-size: 14px; color: var(--text-heading);">Cámara Cenital (Vista Superior)</strong>
                                </div>
                                <span class="badge badge-success">Área Foliar</span>
                            </div>

                            <div style="position: relative; width: 100%; height: 460px; border-radius: 10px; overflow: hidden; border: 1px solid var(--border-color); background: #090d16;">
                                <div style="display: grid; grid-template-columns: 1fr 1fr; width: 100%; height: 100%;">
                                    <div class="photo-inspect-trigger" style="position: relative; border-right: 1px solid #1e293b; display: flex; align-items: center; justify-content: center; background: #090d16; cursor: zoom-in;" onclick="openImageLightbox(document.getElementById('img-cenital-orig-{{ s.id }}').src, 'Cámara Cenital - Foto Original')">
                                        <img id="img-cenital-orig-{{ s.id }}" src="{{ url_for('static', filename=avg.image_path_cenital_orig) }}" style="width: 100%; height: 100%; object-fit: contain; transition: transform 0.2s;">
                                        <div style="position: absolute; bottom: 8px; left: 8px; background: rgba(0,0,0,0.75); padding: 3px 8px; border-radius: 4px; font-size: 10.5px; color: #fff; font-weight: 700; letter-spacing: 0.5px;">ORIGINAL</div>
                                        <div style="position: absolute; top: 8px; right: 8px; background: rgba(0,0,0,0.6); padding: 5px 8px; border-radius: 6px; color: #38bdf8; display: flex; align-items: center; gap: 4px; font-size: 11px; font-weight: 600;">
                                            <span class="material-icons-round" style="font-size: 16px;">search</span>
                                            <span>Lupa / Zoom</span>
                                        </div>
                                    </div>
                                    <div class="photo-inspect-trigger" style="position: relative; display: flex; align-items: center; justify-content: center; background: #090d16; cursor: zoom-in;" onclick="openImageLightbox(document.getElementById('img-cenital-proc-{{ s.id }}').src, 'Cámara Cenital - Segmentación ExG y ROI')">
                                        <img id="img-cenital-proc-{{ s.id }}" src="{{ url_for('static', filename=avg.image_path_cenital_proc) }}" style="width: 100%; height: 100%; object-fit: contain; transition: transform 0.2s;">
                                        <div style="position: absolute; bottom: 8px; right: 8px; background: rgba(5,150,105,0.9); padding: 3px 8px; border-radius: 4px; font-size: 10.5px; color: #fff; font-weight: 700; letter-spacing: 0.5px;">MÁSCARA ExG</div>
                                        <div style="position: absolute; top: 8px; right: 8px; background: rgba(0,0,0,0.6); padding: 5px 8px; border-radius: 6px; color: #38bdf8; display: flex; align-items: center; gap: 4px; font-size: 11px; font-weight: 600;">
                                            <span class="material-icons-round" style="font-size: 16px;">search</span>
                                            <span>Lupa / Zoom</span>
                                        </div>
                                    </div>
                                </div>
                            </div>
                        </div>

                        <!-- COMPARADOR 2: CÁMARA LATERAL (ALTURA Y TALLO) -->
                        <div style="background: #ffffff; border: 1px solid var(--border-color); border-radius: 12px; padding: 16px; box-shadow: var(--shadow-sm);">
                            <div style="display: flex; align-items: center; justify-content: space-between; margin-bottom: 12px;">
                                <div style="display: flex; align-items: center; gap: 8px;">
                                    <span class="material-icons-round" style="color: var(--accent-primary);">photo_camera_front</span>
                                    <strong style="font-size: 14px; color: var(--text-heading);">Cámara Lateral (Perfil Vertical)</strong>
                                </div>
                                <span class="badge badge-primary">Altura & Tallo</span>
                            </div>

                            <div style="position: relative; width: 100%; height: 460px; border-radius: 10px; overflow: hidden; border: 1px solid var(--border-color); background: #090d16;">
                                <div style="display: grid; grid-template-columns: 1fr 1fr; width: 100%; height: 100%;">
                                    <div class="photo-inspect-trigger" style="position: relative; border-right: 1px solid #1e293b; display: flex; align-items: center; justify-content: center; background: #090d16; cursor: zoom-in;" onclick="openImageLightbox(document.getElementById('img-lateral-orig-{{ s.id }}').src, 'Cámara Lateral - Foto Original')">
                                        <img id="img-lateral-orig-{{ s.id }}" src="{{ url_for('static', filename=avg.image_path_lateral_orig) }}" style="width: 100%; height: 100%; object-fit: contain; transition: transform 0.2s;">
                                        <div style="position: absolute; bottom: 8px; left: 8px; background: rgba(0,0,0,0.75); padding: 3px 8px; border-radius: 4px; font-size: 10.5px; color: #fff; font-weight: 700; letter-spacing: 0.5px;">ORIGINAL</div>
                                        <div style="position: absolute; top: 8px; right: 8px; background: rgba(0,0,0,0.6); padding: 5px 8px; border-radius: 6px; color: #38bdf8; display: flex; align-items: center; gap: 4px; font-size: 11px; font-weight: 600;">
```

```
                                        <span class="material-icons-round" style="font-size: 16px;">search</span>
                                        <span>Lupa / Zoom</span>
                                    </div>
                                </div>
                                <div class="photo-inspect-trigger" style="position: relative; display: flex; align-items: center; justify-content: center; background: #090d16; cursor: zoom-in;" onclick="openImageLightbox(document.getElementById('img-lateral-proc-{{ s.id }}').src, 'Cámara Lateral - Medición de Altura y Tallo')">
                                    <img id="img-lateral-proc-{{ s.id }}" src="{{ url_for('static', filename=avg.image_path_lateral_proc) }}" style="width: 100%; height: 100%; object-fit: contain; transition: transform 0.2s;">
                                    <div style="position: absolute; bottom: 8px; right: 8px; background: rgba(37,99,235,0.9); padding: 3px 8px; border-radius: 4px; font-size: 10.5px; color: #fff; font-weight: 700; letter-spacing: 0.5px;">CALIBRACIÓN</div>
                                    <div style="position: absolute; top: 8px; right: 8px; background: rgba(0,0,0,0.6); padding: 5px 8px; border-radius: 6px; color: #38bdf8; display: flex; align-items: center; gap: 4px; font-size: 11px; font-weight: 600;">
                                        <span class="material-icons-round" style="font-size: 16px;">search</span>
                                        <span>Lupa / Zoom</span>
                                    </div>
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        {% endfor %}
    {% endif %}
</div>


<!-- MODAL LIGHTBOX PARA VER FOTOS EN TAMAÑO COMPLETO CON LUPA BOTÁNICA Y ZOOM ULTRA-ALTO -->
<div id="image-lightbox-modal" style="display: none; position: fixed; inset: 0; background: rgba(3, 7, 18, 0.95); backdrop-filter: blur(10px); z-index: 9999; justify-content: center; align-items: center; flex-direction: column; user-select: none;" onclick="closeImageLightbox(event)">

    <!-- Barra superior de controles del lightbox -->
    <div style="position: absolute; top: 12px; left: 20px; right: 20px; display: flex; align-items: center; justify-content: space-between; z-index: 20; flex-wrap: wrap; gap: 12px;" onclick="event.stopPropagation()">
        <div style="color: #f8fafc; font-size: 14.5px; font-weight: 700; display: flex; align-items: center; gap: 10px;">
            <div style="background: rgba(56, 189, 248, 0.15); border: 1px solid rgba(56, 189, 248, 0.4); padding: 6px 10px; border-radius: 8px; display: flex; align-items: center; gap: 6px;">
                <span class="material-icons-round" style="color: #38bdf8; font-size: 18px;">nature</span>
                <span id="lightbox-title">Inspección de Tejido Vegetal</span>
            </div>
        </div>

        <!-- Barra de herramientas de Zoom y Lupa -->
        <div style="display: flex; align-items: center; gap: 8px; flex-wrap: wrap;">

            <!-- Controles de Zoom General -->
            <div style="background: rgba(15, 23, 42, 0.9); border: 1px solid rgba(255,255,255,0.15); border-radius: 8px; padding: 4px 6px; display: flex; align-items: center; gap: 4px; box-shadow: 0 4px 12px rgba(0,0,0,0.5);">
                <button type="button" class="btn btn-outline btn-sm" onclick="adjustLightboxZoom(-0.5)" style="padding: 4px 8px; font-size: 11px; color: #fff; border-color: rgba(255,255,255,0.2);" title="Alejar (-)">
                    <span class="material-icons-round" style="font-size: 15px;">remove</span>
                </button>
                <span id="lightbox-zoom-level" style="color: #38bdf8; font-size: 12px; font-weight: 700; min-width: 50px; text-align: center;">100%</span>
                <button type="button" class="btn btn-outline btn-sm" onclick="adjustLightboxZoom(0.5)" style="padding: 4px 8px; font-size: 11px; color: #fff; border-color: rgba(255,255,255,0.2);" title="Acercar (+)">
                    <span class="material-icons-round" style="font-size: 15px;">add</span>
                </button>
                <button type="button" class="btn btn-outline btn-sm" onclick="resetLightboxZoom()" style="padding: 4px 8px; font-size: 11px; color: #cbd5e1; border-color: rgba(255,255,255,0.2);" title="Restablecer (1:1 / 0)">
                    <span class="material-icons-round" style="font-size: 15px;">restart_alt</span>
                </button>
```

```
            </div>

            <!-- Controles del Modo Lupa -->
            <div style="background: rgba(15, 23, 42, 0.9); border: 1px solid rgba(56, 189, 248, 0.35); border-radius: 8px; padding: 4px 6px; display: flex; align-items: center; gap: 5px; box-shadow: 0 4px 12px rgba(0,0,0,0.5);">
                <button type="button" id="btn-toggle-loupe" class="btn btn-outline btn-sm" onclick="toggleLoupeMode()" style="padding: 4px 10px; font-size: 11.5px; color: #fff; border-color: rgba(255,255,255,0.25); display: flex; align-items: center; gap: 5px;" title="Activar Lupa Circular de Detalle (Tecla L)">
                    <span class="material-icons-round" style="font-size: 16px;">search</span>
                    <span>Modo Lupa</span>
                </button>

                <!-- Selector de potencia de aumento de lupa -->
                <div id="loupe-power-selector" style="display: flex; align-items: center; gap: 3px; border-left: 1px solid rgba(255,255,255,0.15); padding-left: 6px;">
                    <button type="button" class="btn-loupe-power" onclick="setLoupePower(2.5, this)" style="background: transparent; border: 1px solid rgba(255,255,255,0.2); color: #cbd5e1; border-radius: 4px; padding: 3px 6px; font-size: 10.5px; font-weight: 700; cursor: pointer;">2.5x</button>
                    <button type="button" class="btn-loupe-power active" onclick="setLoupePower(4.0, this)" style="background: #38bdf8; border: 1px solid #38bdf8; color: #090d16; border-radius: 4px; padding: 3px 6px; font-size: 10.5px; font-weight: 700; cursor: pointer;">4x</button>
                    <button type="button" class="btn-loupe-power" onclick="setLoupePower(8.0, this)" style="background: transparent; border: 1px solid rgba(255,255,255,0.2); color: #cbd5e1; border-radius: 4px; padding: 3px 6px; font-size: 10.5px; font-weight: 700; cursor: pointer;">8x</button>
                </div>
            </div>

            <!-- Botón de Descargar -->
            <a id="lightbox-download-link" href="#" download="captura_sifma_hd.png" class="btn btn-outline btn-sm" style="padding: 6px 10px; font-size: 11.5px; color: #cbd5e1; border-color: rgba(255,255,255,0.2); display: flex; align-items: center; gap: 4px; background: rgba(15, 23, 42, 0.9);" title="Descargar Imagen Original">
                <span class="material-icons-round" style="font-size: 16px;">download</span>
            </a>

            <!-- Cerrar Modal -->
            <button type="button" onclick="closeImageLightbox()" style="background: rgba(239, 68, 68, 0.25); border: 1px solid rgba(239, 68, 68, 0.6); color: #ffffff; border-radius: 50%; width: 34px; height: 34px; display: flex; align-items: center; justify-content: center; cursor: pointer; transition: all 0.2s;" title="Cerrar (Esc)">
                <span class="material-icons-round" style="font-size: 20px;">close</span>
            </button>
        </div>
    </div>

    <!-- Contenedor interactivo con Pan y Zoom -->
    <div id="lightbox-viewport" style="position: relative; width: 94vw; height: 84vh; display: flex; align-items: center; justify-content: center; overflow: hidden; cursor: grab;" onclick="event.stopPropagation()">
        <img id="lightbox-img" src="" draggable="false" style="max-width: 100%; max-height: 84vh; border-radius: 8px; box-shadow: 0 16px 48px rgba(0,0,0,0.85); object-fit: contain; border: 1px solid #1e293b; transform-origin: 0 0; transition: transform 0.05s linear; pointer-events: auto; user-select: none;">

        <!-- Lupa Circular Dinámica con Retícula y Aumento Óptico -->
        <div id="lightbox-loupe" style="display: none; position: absolute; width: 240px; height: 240px; border-radius: 50%; border: 3px solid #38bdf8; box-shadow: 0 0 25px rgba(56, 189, 248, 0.65), inset 0 0 15px rgba(0,0,0,0.7); pointer-events: none; overflow: hidden; background-repeat: no-repeat; z-index: 60; background-color: #030712;">
            <!-- Retícula de precisión botánica -->
            <div style="position: absolute; inset: 0; display: flex; align-items: center; justify-content: center; pointer-events: none;">
                <div style="width: 14px; height: 1.5px; background: rgba(56, 189, 248, 0.9); box-shadow: 0 0 4px #38bdf8;"></div>
                <div style="width: 1.5px; height: 14px; background: rgba(56, 189, 248, 0.9); position: absolute; box-shadow: 0 0 4px #38bdf8;"></div>
                <div style="width: 60px; height: 60px; border: 1px dashed rgba(56, 189, 248, 0.4); border-radius: 50%; position: absolute;"></div>
            </div>
            <!-- Etiqueta de potencia de aumento en la lupa -->
            <div id="loupe-power-badge" style="position: absolute; bottom: 8px; left: 50%; transform: translateX(-50%); background: rgba(15, 23, 42, 0.85); border: 1px solid #38bdf8; color: #38bdf8; font-size: 10px; font-weight: 800; padding: 1px 8px; border-radius: 10px; letter-spacing: 0.5px;">
                4.0x LOUPE
            </div>
        </div>
    </div>

    <!-- Barra inferior informativa -->
```

```
    <div style="position: absolute; bottom: 12px; color: #94a3b8; font-size: 11.5px; display: flex; align-items: center; gap: 18px; background: rgba(15, 23, 42, 0.85); border: 1px solid rgba(255,255,255,0.1); padding: 5px 20px; border-radius: 20px; z-index: 20;">
        <span><strong>Rueda</strong>: Zoom continuo (o dentro de la lupa)</span>
        <span><strong>Arrastrar</strong>: Pan panorámico</span>
        <span><strong>Doble Clic</strong>: Zoom rápido 3x</span>
        <span><strong>Tecla L</strong>: Activar/Desactivar Modo Lupa</span>
        <span><strong>Tecla Esc</strong>: Cerrar</span>
    </div>
</div>

{% endblock %}

{% block scripts %}
<script>
    function filterPeriod(period, btnElement) {
        document.querySelectorAll('.filter-group .filter-pill').forEach(b => b.classList.remove('active'));
        if (btnElement) btnElement.classList.add('active');

        const cards = document.querySelectorAll('.capture-thumb-card');
        let visibleCount = 0;

        cards.forEach(card => {
            const cardPeriod = card.getAttribute('data-period').toLowerCase();
            if (period === 'todos' || cardPeriod.includes(period)) {
                card.style.display = 'block';
                visibleCount++;
            } else {
                card.style.display = 'none';
            }
        });

        const noMsg = document.getElementById('no-captures-message');
        if (noMsg) noMsg.style.display = visibleCount === 0 ? 'block' : 'none';
    }

    function selectSession(sessionId) {
        document.querySelectorAll('.session-comparator-card').forEach(c => c.style.display = 'none');
        document.querySelectorAll('.capture-thumb-card').forEach(c => c.style.borderColor = 'var(--border-color)');

        const target = document.getElementById('comparator-' + sessionId);
        if (target) {
            target.style.display = 'block';
            target.scrollIntoView({ behavior: 'smooth', block: 'start' });
        }

        const thumb = document.querySelector(`.capture-thumb-card[data-session-id="${sessionId}"]`);
        if (thumb) thumb.style.borderColor = 'var(--accent-success)';
    }

    function switchSessionPhoto(sessionId, photoIndex, btnElement) {
        const card = document.getElementById('comparator-' + sessionId);
        if (!card) return;

        const group = document.getElementById('photo-pills-' + sessionId);
        if (group) {
            group.querySelectorAll('button').forEach(b => {
```

```
                b.className = 'btn btn-outline btn-sm';
            });
            if (btnElement) btnElement.className = 'btn btn-primary btn-sm';
        }

        const allMetrics = JSON.parse(card.getAttribute('data-session-json') || '[]');
        let selected = null;
        if (photoIndex === 0) {
            selected = allMetrics.find(m => m.is_average) || allMetrics[0];
        } else {
            selected = allMetrics.find(m => m.photo_index === photoIndex);
        }

        if (selected) {
            document.getElementById(`img-cenital-orig-${sessionId}`).src = '/static/' + selected.image_path_cenital_orig;
            document.getElementById(`img-cenital-proc-${sessionId}`).src = '/static/' + selected.image_path_cenital_proc;
            document.getElementById(`img-lateral-orig-${sessionId}`).src = '/static/' + selected.image_path_lateral_orig;
            document.getElementById(`img-lateral-proc-${sessionId}`).src = '/static/' + selected.image_path_lateral_proc;

            document.getElementById(`metric-area-${sessionId}`).innerText = `${selected.foliar_area_cm2} cm²`;
            document.getElementById(`metric-height-${sessionId}`).innerText = `${selected.plant_height_cm} cm`;
            document.getElementById(`metric-diameter-${sessionId}`).innerText = `${selected.stem_diameter_mm} mm`;
            document.getElementById(`metric-health-${sessionId}`).innerText = `${selected.health_index}%`;
        }
    }

    async function deleteSession(sessionId) {
        if (!confirm('¿Estás seguro de que deseas eliminar permanentemente este período de fotos?')) return;
        try {
            const res = await fetch(`/api/sessions/${sessionId}`, { method: 'DELETE' });
            const data = await res.json();
            if (data.status === 'success') {
                window.location.reload();
            } else {
                alert('Error: ' + data.message);
            }
        } catch (e) {
            alert('Error al eliminar período.');
        }
    }

    // ========================================
    // VISOR LIGHTBOX CON ZOOM ULTRA-DETALLE Y LUPA BOTÁNICA
    // ========================================
    let lightboxState = {
        zoom: 1.0,
        panX: 0,
        panY: 0,
        isPanning: false,
        startX: 0,
        startY: 0,
        isLoupeActive: false,
        loupePower: 4.0
    };

```

```
    function openImageLightbox(src, title) {
        if (!src) return;
        const modal = document.getElementById('image-lightbox-modal');
        const img = document.getElementById('lightbox-img');
        const titleEl = document.getElementById('lightbox-title');
        const downloadLink = document.getElementById('lightbox-download-link');

        if (modal && img) {
            img.src = src;
            if (downloadLink) {
                downloadLink.href = src;
                downloadLink.download = (title ? title.replace(/[^a-zA-Z0-9_-]/g, '_') : 'captura_sifma') + '.jpg';
            }
            if (titleEl && title) titleEl.innerText = title;
            modal.style.display = 'flex';
            document.body.style.overflow = 'hidden';
            resetLightboxZoom();
        }
    }

    function closeImageLightbox(e) {
        const modal = document.getElementById('image-lightbox-modal');
        if (modal) {
            modal.style.display = 'none';
            document.body.style.overflow = 'auto';
            if (lightboxState.isLoupeActive) toggleLoupeMode();
        }
    }

    function adjustLightboxZoom(delta, originClientX, originClientY) {
        const oldZoom = lightboxState.zoom;
        const newZoom = Math.max(0.5, Math.min(12.0, oldZoom + delta));
        if (newZoom === oldZoom) return;

        const img = document.getElementById('lightbox-img');
        const viewport = document.getElementById('lightbox-viewport');

        if (originClientX !== undefined && originClientY !== undefined && img && viewport) {
            const vpRect = viewport.getBoundingClientRect();
            const mouseX = originClientX - vpRect.left;
            const mouseY = originClientY - vpRect.top;

            // Zoom centrado hacia el cursor
            const zoomRatio = newZoom / oldZoom;
            lightboxState.panX = mouseX - (mouseX - lightboxState.panX) * zoomRatio;
            lightboxState.panY = mouseY - (mouseY - lightboxState.panY) * zoomRatio;
        } else if (viewport) {
            // Zoom centrado en el medio del viewport
            const vpW = viewport.clientWidth / 2;
            const vpH = viewport.clientHeight / 2;
            const zoomRatio = newZoom / oldZoom;
            lightboxState.panX = vpW - (vpW - lightboxState.panX) * zoomRatio;
            lightboxState.panY = vpH - (vpH - lightboxState.panY) * zoomRatio;
        }

```

```
        lightboxState.zoom = newZoom;
        applyLightboxTransform();
    }

    function resetLightboxZoom() {
        lightboxState.zoom = 1.0;
        const img = document.getElementById('lightbox-img');
        const viewport = document.getElementById('lightbox-viewport');
        if (img && viewport) {
            lightboxState.panX = (viewport.clientWidth - img.offsetWidth) / 2;
            lightboxState.panY = (viewport.clientHeight - img.offsetHeight) / 2;
        } else {
            lightboxState.panX = 0;
            lightboxState.panY = 0;
        }
        applyLightboxTransform();
    }

    function applyLightboxTransform() {
        const img = document.getElementById('lightbox-img');
        const level = document.getElementById('lightbox-zoom-level');
        if (img) {
            img.style.transform = `translate(${lightboxState.panX}px, ${lightboxState.panY}px) scale(${lightboxState.zoom})`;
        }
        if (level) {
            level.innerText = `${Math.round(lightboxState.zoom * 100)}%`;
        }
    }

    function setLoupePower(power, btnElement) {
        lightboxState.loupePower = power;
        document.querySelectorAll('.btn-loupe-power').forEach(b => {
            b.style.background = 'transparent';
            b.style.borderColor = 'rgba(255,255,255,0.2)';
            b.style.color = '#cbd5e1';
        });
        if (btnElement) {
            btnElement.style.background = '#38bdf8';
            btnElement.style.borderColor = '#38bdf8';
            btnElement.style.color = '#090d16';
        }
        const badge = document.getElementById('loupe-power-badge');
        if (badge) badge.innerText = `${power.toFixed(1)}x LOUPE`;

        // Si la lupa no está activa, activarla automáticamente
        if (!lightboxState.isLoupeActive) {
            toggleLoupeMode();
        }
    }

    function toggleLoupeMode() {
        lightboxState.isLoupeActive = !lightboxState.isLoupeActive;
        const loupe = document.getElementById('lightbox-loupe');
        const btn = document.getElementById('btn-toggle-loupe');
        const viewport = document.getElementById('lightbox-viewport');
```

```
        const img = document.getElementById('lightbox-img');

        if (lightboxState.isLoupeActive) {
            if (loupe && img) {
                loupe.style.display = 'block';
                loupe.style.backgroundImage = `url('${img.src}')`;
            }
            if (btn) {
                btn.className = 'btn btn-primary btn-sm';
                btn.style.background = '#38bdf8';
                btn.style.color = '#090d16';
            }
            if (viewport) viewport.style.cursor = 'crosshair';
        } else {
            if (loupe) loupe.style.display = 'none';
            if (btn) {
                btn.className = 'btn btn-outline btn-sm';
                btn.style.background = 'transparent';
                btn.style.color = '#fff';
            }
            if (viewport) viewport.style.cursor = 'grab';
        }
    }

    // Inicialización de eventos interactivos del Lightbox
    const lbViewport = document.getElementById('lightbox-viewport');
    if (lbViewport) {
        // Zoom con rueda del ratón
        lbViewport.addEventListener('wheel', (e) => {
            e.preventDefault();
            if (lightboxState.isLoupeActive) {
                // Si la lupa está activa, ajustar aumento de la lupa con la rueda
                const deltaPower = e.deltaY < 0 ? 0.5 : -0.5;
                lightboxState.loupePower = Math.max(1.5, Math.min(12.0, lightboxState.loupePower + deltaPower));
                const badge = document.getElementById('loupe-power-badge');
                if (badge) badge.innerText = `${lightboxState.loupePower.toFixed(1)}x LOUPE`;
            } else {
                // Si no, hacer zoom al viewport hacia la posición del mouse
                const delta = e.deltaY < 0 ? 0.35 : -0.35;
                adjustLightboxZoom(delta, e.clientX, e.clientY);
            }
        }, { passive: false });

        // Doble Clic para Zoom Rápido hacia el punto
        lbViewport.addEventListener('dblclick', (e) => {
            if (lightboxState.isLoupeActive) return;
            if (lightboxState.zoom > 1.2) {
                resetLightboxZoom();
            } else {
                lightboxState.zoom = 3.0;
                const vpRect = lbViewport.getBoundingClientRect();
                const mouseX = e.clientX - vpRect.left;
                const mouseY = e.clientY - vpRect.top;
                lightboxState.panX = mouseX - mouseX * 3.0;
                lightboxState.panY = mouseY - mouseY * 3.0;
```

```
                applyLightboxTransform();
            }
        });

        // Arrastrar y Mover Imagen (Pan)
        lbViewport.addEventListener('mousedown', (e) => {
            if (lightboxState.isLoupeActive) return;
            lightboxState.isPanning = true;
            lightboxState.startX = e.clientX - lightboxState.panX;
            lightboxState.startY = e.clientY - lightboxState.panY;
            lbViewport.style.cursor = 'grabbing';
        });

        window.addEventListener('mousemove', (e) => {
            const loupe = document.getElementById('lightbox-loupe');
            const img = document.getElementById('lightbox-img');

            // 1. Manejo del Modo Lupa Circular
            if (lightboxState.isLoupeActive && loupe && img && img.complete) {
                const rect = lbViewport.getBoundingClientRect();
                const imgRect = img.getBoundingClientRect();
                const mouseX = e.clientX;
                const mouseY = e.clientY;

                if (mouseX >= rect.left && mouseX <= rect.right && mouseY >= rect.top && mouseY <= rect.bottom) {
                    loupe.style.display = 'block';
                    const loupeSize = 240;
                    loupe.style.left = `${mouseX - rect.left - loupeSize / 2}px`;
                    loupe.style.top = `${mouseY - rect.top - loupeSize / 2}px`;

                    // Calcular coordenadas relativas dentro de la imagen renderizada
                    const relX = (mouseX - imgRect.left) / imgRect.width;
                    const relY = (mouseY - imgRect.top) / imgRect.height;

                    const zoomFactor = lightboxState.loupePower;
                    const bgW = imgRect.width * zoomFactor;
                    const bgH = imgRect.height * zoomFactor;

                    loupe.style.backgroundSize = `${bgW}px ${bgH}px`;
                    loupe.style.backgroundPosition = `-${relX * bgW - loupeSize / 2}px -${relY * bgH - loupeSize / 2}px`;
                } else {
                    loupe.style.display = 'none';
                }
            }

            // 2. Manejo de Pan / Arrastre
            if (lightboxState.isPanning) {
                lightboxState.panX = e.clientX - lightboxState.startX;
                lightboxState.panY = e.clientY - lightboxState.startY;
                applyLightboxTransform();
            }
        });

        window.addEventListener('mouseup', () => {
            if (lightboxState.isPanning) {
```

```
                lightboxState.isPanning = false;
                if (!lightboxState.isLoupeActive) lbViewport.style.cursor = 'grab';
            }
        });
    }

    document.addEventListener('keydown', (e) => {
        if (e.key === 'Escape') {
            closeImageLightbox();
        } else if (e.key === '+' || e.key === '=') {
            adjustLightboxZoom(0.5);
        } else if (e.key === '-' || e.key === '_') {
            adjustLightboxZoom(-0.5);
        } else if (e.key === '0') {
            resetLightboxZoom();
        } else if (e.key.toLowerCase() === 'l') {
            toggleLoupeMode();
        } else if (e.key === '1') {
            setLoupePower(2.5, document.querySelectorAll('.btn-loupe-power')[0]);
        } else if (e.key === '2') {
            setLoupePower(4.0, document.querySelectorAll('.btn-loupe-power')[1]);
        } else if (e.key === '3') {
            setLoupePower(8.0, document.querySelectorAll('.btn-loupe-power')[2]);
        }
    });

    document.addEventListener('DOMContentLoaded', () => {
        const firstThumb = document.querySelector('.capture-thumb-card');
        if (firstThumb) {
            const firstId = firstThumb.getAttribute('data-session-id');
            selectSession(firstId);
        }
    });
</script>
{% endblock %}
```

ARCHIVO 69/85: local_server/templates/login.html (77 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Inicio de sesión seguro para el Panel de Control SIFMA.">
    <title>SIFMA - Iniciar Sesión</title>

    <!-- Favicon Oficial SIFMA -->
    <link rel="icon" type="image/svg+xml" href="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}">

    <!-- Google Fonts: Philosopher (marca oficial) + Literata (cuerpo y datos) -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Literata:ital,opsz,wght@0,7..72,400;0,7..72,500;0,7..72,600;0,7..72,700;1,7..72,400;1,7..72,600&family=Philosopher:ital,wght@0,400;0,700;1,400;1,700&display=swap" rel="stylesheet">

    <!-- Google Material Icons Round -->
    <link href="https://fonts.googleapis.com/icon?family=Material+Icons+Round" rel="stylesheet">

    <!-- Estilos Globales Unificados -->
    <link rel="stylesheet" href="{{ url_for('static', filename='css/index.css') }}">
</head>
<body class="login-page-wrapper">
    <div class="login-glass-panel">
        <!-- Logo y Marca Oficial -->
        <div class="login-brand-header">
            <div style="width: 80px; height: 80px; margin: 0 auto 12px; display: flex; align-items: center; justify-content: center;">
                <img src="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}" alt="SIFMA Emblem" style="width: 100%; height: 100%; display: block;">
            </div>
            <h1>SIFMA</h1>
            <p>Sistema Integrado de Fenotipado y Monitoreo Agronómico</p>
        </div>

        <!-- Alerta de Error -->
        {% if error %}
        <div class="error-alert" style="width: 100%; background: rgba(220, 38, 38, 0.08); border: 1px solid rgba(220, 38, 38, 0.25); color: #b91c1c; border-radius: 10px; padding: 12px 16px; font-size: 13px; font-weight: 600; display: flex; align-items: center; gap: 10px; margin-bottom: 20px;">
            <span class="material-icons-round" style="font-size: 20px;">error_outline</span>
            <span>{{ error }}</span>
        </div>
        {% endif %}

        <!-- Formulario de Acceso -->
        <form style="width: 100%; display: flex; flex-direction: column; gap: 16px;" method="POST" action="/login">
            <div class="form-group" style="margin-bottom: 0;">
                <label style="font-size: 12.5px; font-weight: 600; color: var(--text-heading); margin-bottom: 6px;">Usuario</label>
```

```
                <div style="position: relative; display: flex; align-items: center;">
                    <span class="material-icons-round" style="position: absolute; left: 12px; color: var(--text-muted); font-size: 20px; pointer-events: none;">person</span>
                    <input class="form-control" type="text" name="username" placeholder="Ingresa tu usuario" required autocomplete="username" style="padding-left: 40px;" autofocus>
                </div>
            </div>

            <div class="form-group" style="margin-bottom: 0;">
                <label style="font-size: 12.5px; font-weight: 600; color: var(--text-heading); margin-bottom: 6px;">Contraseña</label>
                <div style="position: relative; display: flex; align-items: center;">
                    <span class="material-icons-round" style="position: absolute; left: 12px; color: var(--text-muted); font-size: 20px; pointer-events: none;">lock</span>
                    <input class="form-control" type="password" name="password" placeholder="Ingresa tu contraseña" required autocomplete="current-password" style="padding-left: 40px;">
                </div>
            </div>

            <button class="login-btn-submit" type="submit">
                <span class="material-icons-round">login</span>
                <span>Iniciar Sesión</span>
            </button>
        </form>

        <div style="margin-top: 24px; text-align: center; font-size: 11.5px; color: var(--text-muted);">
            Plataforma Segura SIFMA 2026
        </div>
    </div>
</body>
</html>
```

ARCHIVO 70/85: local_server/templates/processing.html (816 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Procesamiento de Imágenes{% endblock %}
{% block heading %}Pipeline de Visión Artificial y Segmentación{% endblock %}
{% block subheading %}Procesamiento de capturas USB / archivos ZIP por fecha y canastilla hidropónica{% endblock %}

{% block content %}

<!-- CARD PRINCIPAL: IMPORTACIÓN DE LOTE FOTOGRÁFICO Y CALENDARIO (RETRAÍBLE POR DEFECTO) -->
<div class="widget-card" style="margin-bottom: 14px; padding: 12px 18px;">
    <div class="widget-header" style="margin-bottom: 0; padding-bottom: 0; cursor: pointer;" onclick="toggleCollapsibleSection('section-lots-body', 'icon-lots-toggle', 'btn-lots-text')">
        <div class="widget-header-title" style="display: flex; align-items: center; gap: 10px;">
            <div class="kpi-icon-badge emerald" style="width: 32px; height: 32px;">
                <span class="material-icons-round" style="font-size: 18px;">camera</span>
            </div>
            <div>
                <h3 style="font-size: 13.5px; margin-bottom: 2px;">Gestión de Lotes y Fechas de Muestreo</h3>
                <p style="font-size: 11.5px; margin: 0;">Canastilla Activa: <strong>#{{ active_plant_id }}</strong> | Especie Asignada: <strong>{{ active_crop_profile.display_name }}</strong></p>
            </div>
        </div>
        <div style="display: flex; align-items: center; gap: 10px;">
            <span class="badge badge-success" style="font-size: 11.5px;">
                <span class="material-icons-round" style="font-size: 13px; vertical-align: middle;">schedule</span>
                {{ sessions | length }} Muestreos en Canastilla #{{ active_plant_id }}
            </span>
            <button type="button" class="btn btn-outline btn-sm" style="padding: 4px 10px; font-size: 11.5px; display: flex; align-items: center; gap: 4px;">
                <span id="btn-lots-text">Ver Lotes y Fechas / Importar</span>
                <span class="material-icons-round" id="icon-lots-toggle" style="font-size: 18px; transition: transform 0.2s;">expand_more</span>
            </button>
        </div>
    </div>

    <!-- CUERPO RETRAÍBLE (OCULTO POR DEFECTO) -->
    <div id="section-lots-body" style="display: none; margin-top: 14px; border-top: 1px dashed var(--border-color); padding-top: 14px;">

        <!-- CALENDARIO SELECTOR DE FECHA -->
        <div style="background: var(--bg-card-subtle); padding: 14px 18px; border-radius: 10px; border: 1px solid var(--border-color);">
            <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 12px;">
                <div>
                    <label for="calendar-date-picker" style="font-weight: 700; font-size: 13px; color: var(--text-heading); display: block; margin-bottom: 2px;">
                        Fecha de la Jornada a Procesar:
                    </label>
                    <span style="font-size: 11.5px; color: var(--text-muted);">
                        Selecciona la fecha para asociar los lotes fotográficos a su respectiva jornada
                    </span>
                </div>
```

```
                <div style="display: flex; align-items: center; gap: 8px; background: #ffffff; padding: 6px 12px; border-radius: 8px; border: 1px solid var(--border-color);">
                    <span class="material-icons-round" style="font-size: 18px; color: var(--accent-success);">edit_calendar</span>
                    <input type="date" id="calendar-date-picker" class="form-control" style="background: transparent; border: none; font-weight: 700; font-size: 13px; color: var(--text-heading); cursor: pointer; padding: 0;" onchange="onCalendarDateChange(this.value)">
                </div>
            </div>

            <!-- LISTADO DE FECHAS YA REGISTRADAS Y PROCESADAS -->
            <div style="margin-top: 12px; padding-top: 10px; border-top: 1px dashed var(--border-color); display: flex; align-items: center; gap: 8px; flex-wrap: wrap;">
                <span style="font-size: 11px; font-weight: 700; color: var(--accent-success); text-transform: uppercase; display: flex; align-items: center; gap: 4px;">
                    <span class="material-icons-round" style="font-size: 13px;">check_circle</span> Fechas ya Registradas:
                </span>
                {% if processed_periods %}
                    {% for p in processed_periods %}
                        <span class="badge badge-success" style="font-size: 11px; cursor: pointer; padding: 3px 8px;" title="Haz clic para seleccionar esta fecha" onclick="selectProcessedDate('{{ p }}')">
                            {{ p }}
                        </span>
                    {% endfor %}
                {% else %}
                    <span style="font-size: 11.5px; color: var(--text-muted);">Ninguna fecha registrada aún para esta canastilla</span>
                {% endif %}
            </div>

            <!-- ADVERTENCIA DE SOBREESCRITURA SI LA FECHA YA FUE PROCESADA -->
            <div id="overwrite-warning-box" style="display: none; margin-top: 10px; background: rgba(217, 119, 6, 0.08); border: 1px solid rgba(217, 119, 6, 0.3); padding: 8px 12px; border-radius: 8px; color: var(--accent-warning); font-size: 12px; font-weight: 600; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="font-size: 18px;">warning</span>
                <span id="overwrite-warning-text">Advertencia: Esta fecha ya tiene un muestreo registrado. Si procesas de nuevo se actualizarán los datos.</span>
            </div>
        </div>

        <!-- SELECTOR DE MODO DENTRO DE LA TARJETA -->
        <div style="display: flex; gap: 8px; margin-top: 12px;">
            <button type="button" class="btn btn-primary btn-sm" id="tab-mode-usb" onclick="switchImportMode('usb')">
                <span class="material-icons-round" style="font-size: 16px;">usb</span>
                <span>Importación Automática USB</span>
            </button>
            <button type="button" class="btn btn-outline btn-sm" id="tab-mode-drag" onclick="switchImportMode('drag')">
                <span class="material-icons-round" style="font-size: 16px;">cloud_upload</span>
                <span>Subir Archivo .ZIP o Imágenes</span>
            </button>
        </div>

        <!-- MODO 1: DETECCIÓN AUTOMÁTICA DE MEMORIAS USB CONECTADAS -->
        <div id="import-mode-usb-container" style="margin-top: 10px; background: #ffffff; padding: 14px 16px; border-radius: 10px; border: 1px dashed var(--border-color);">
            <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px;">
                <span style="font-size: 13px; font-weight: 700; color: var(--text-heading);">Carpetas y Lotes USB Detectados:</span>
                <button class="btn btn-primary btn-sm" onclick="scanUSBDrives()">
                    <span class="material-icons-round">search</span>
                    <span>Escanear Memorias USB</span>
                </button>
            </div>
```

```
            <div id="usb-scan-results" style="display: flex; flex-direction: column; gap: 8px; min-height: 40px;">
                <p style="font-size: 12.5px; color: var(--text-muted); text-align: center; padding: 10px; margin: 0;">
                    Haz clic en <strong>"Escanear Memorias USB"</strong> para buscar las carpetas guardadas en la memoria.
                </p>
            </div>
        </div>

        <!-- MODO 2: ARRASTRAR Y SOLTAR ARCHIVOS / ZIP (DRAG & DROP) -->
        <div id="import-mode-drag-container" style="margin-top: 14px; display: none;">
            <form id="upload-form" enctype="multipart/form-data">
                <div id="drop-zone" style="border: 2px dashed var(--accent-success); border-radius: 10px; padding: 28px 20px; text-align: center; background: rgba(5, 150, 105, 0.03); cursor: pointer; transition: var(--transition-fast);">
                    <span class="material-icons-round" style="font-size: 40px; color: var(--accent-success); margin-bottom: 6px;">cloud_upload</span>
                    <h4 style="margin-bottom: 4px; font-size: 14px; font-weight: 700; color: var(--text-heading);">Arrastra aquí el archivo .ZIP o las imágenes de la fecha seleccionada</h4>
                    <p style="font-size: 12px; color: var(--text-muted); margin-bottom: 12px;">Soporta archivos .ZIP con subcarpetas de turnos (mañana, mediodía, tarde) o fotos directas</p>
                    <input type="file" id="file-input" name="images" multiple accept="image/*,.zip" style="display: none;">
                    <button type="button" class="btn btn-outline btn-sm" onclick="document.getElementById('file-input').click();">
                        <span class="material-icons-round">folder_open</span>
                        <span>Seleccionar Archivo .ZIP o Imágenes</span>
                    </button>
                </div>

                <div id="selected-files-summary" style="margin-top: 12px; font-size: 12.5px; color: var(--accent-success); display: none; align-items: center; justify-content: space-between;">
                    <div style="display: flex; align-items: center; gap: 6px;">
                        <span class="material-icons-round" style="font-size: 18px;">check_circle</span>
                        <span id="files-count-text" style="font-weight: 600;">0 archivos seleccionados</span>
                    </div>
                    <button type="button" class="btn btn-primary btn-sm" id="btn-submit-manual" onclick="submitManualBatch()">
                        <span>Subir y Procesar Lote para Canastilla #{{ active_plant_id }}</span>
                    </button>
                </div>
            </form>
        </div>

        <!-- BARRA DE PROGRESO DE PROCESAMIENTO (0% A 100%) -->
        <div id="processing-status-bar" style="margin-top: 14px; display: none; background: #09211a; padding: 14px 18px; border-radius: 10px; border: 1px solid #134e3d; color: #ffffff;">
            <div style="display: flex; align-items: center; justify-content: space-between; font-size: 13px; margin-bottom: 6px;">
                <span id="status-message" style="font-weight: 600; color: #10b981;">Iniciando algoritmos de visión artificial OpenCV/PlantCV...</span>
                <span id="status-percent" style="font-weight: 700; color: #10b981; font-size: 14px;">0%</span>
            </div>
            <div style="width: 100%; height: 7px; background: rgba(255,255,255,0.15); border-radius: 4px; overflow: hidden;">
                <div id="progress-bar-fill" style="width: 0%; height: 100%; background: linear-gradient(90deg, #10b981, #34d399); transition: width 0.3s ease;"></div>
            </div>
        </div>
    </div>
</div>

<!-- GRID DE GRÁFICAS BIOMÉTRICAS DETALLADAS (CUADRÍCULA 2X2 AJUSTADA A UNA SOLA PANTALLA) -->
<div class="analytics-grid-2x2" style="display: grid; grid-template-columns: repeat(2, 1fr); gap: 14px; margin-bottom: 14px;">

    <!-- Gráfica 1: Área Foliar -->
    <div class="widget-card" style="padding: 12px 14px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 6px; padding-bottom: 4px;">
            <div class="widget-header-title">
```

```
                <h3 style="font-size: 13px;">Área Foliar Estimada (cm²)</h3>
                <p style="font-size: 11px;">Evolución morfológica en Canastilla #{{ active_plant_id }}</p>
            </div>
            <span class="badge badge-success" style="font-size: 10px;">ExG + Otsu</span>
        </div>
        <div class="chart-stage-container" style="height: 185px; margin-bottom: 0;">
            <canvas id="foliarAreaChart"></canvas>
        </div>
    </div>

    <!-- Gráfica 2: Altura de Planta -->
    <div class="widget-card" style="padding: 12px 14px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 6px; padding-bottom: 4px;">
            <div class="widget-header-title">
                <h3 style="font-size: 13px;">Altura Estimada de la Planta (cm)</h3>
                <p style="font-size: 11px;">Calibración lateral vertical en Canastilla #{{ active_plant_id }}</p>
            </div>
            <span class="badge badge-primary" style="font-size: 10px;">Cámara Lateral</span>
        </div>
        <div class="chart-stage-container" style="height: 185px; margin-bottom: 0;">
            <canvas id="plantHeightChart"></canvas>
        </div>
    </div>

    <!-- Gráfica 3: Diámetro del Tallo -->
    <div class="widget-card" style="padding: 12px 14px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 6px; padding-bottom: 4px;">
            <div class="widget-header-title">
                <h3 style="font-size: 13px;">Diámetro del Tallo Basal (mm)</h3>
                <p style="font-size: 11px;">Espesor promedio en Canastilla #{{ active_plant_id }}</p>
            </div>
            <span class="badge badge-warning" style="font-size: 10px;">Calibración Basal</span>
        </div>
        <div class="chart-stage-container" style="height: 185px; margin-bottom: 0;">
            <canvas id="stemDiameterChart"></canvas>
        </div>
    </div>

    <!-- Gráfica 4: Salud Foliar -->
    <div class="widget-card" style="padding: 12px 14px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 6px; padding-bottom: 4px;">
            <div class="widget-header-title">
                <h3 style="font-size: 13px;">Índice de Salud Foliar (%)</h3>
                <p style="font-size: 11px;">Proporción de clorofila activa en Canastilla #{{ active_plant_id }}</p>
            </div>
            <span class="badge badge-danger" style="font-size: 10px;">Vigor Vegetal</span>
        </div>
        <div class="chart-stage-container" style="height: 185px; margin-bottom: 0;">
            <canvas id="healthIndexChart"></canvas>
        </div>
    </div>
</div>

<!-- COMPARADOR DE MÁSCARA Y DETALLE DE LA ÚLTIMA SESIÓN -->
<div class="dashboard-grid-2col" style="margin-top: 14px; display: grid; grid-template-columns: repeat(2, 1fr); gap: 14px;">
```

```
    <!-- Vista Previa de Segmentación -->
    <div class="widget-card" style="padding: 12px 14px;">
        <div class="widget-header" style="margin-bottom: 8px; padding-bottom: 4px;">
            <div class="widget-header-title">
                <h3 style="font-size: 13px;">Última Segmentación Biométrica</h3>
                <p style="font-size: 11px;">Máscaras generadas por OpenCV</p>
            </div>
            <span class="badge badge-success" style="font-size: 10px;">Procesada</span>
        </div>

        {% if latest_session and latest_metric %}
        <div style="display: flex; flex-direction: column; gap: 10px;">
            <div style="display: flex; justify-content: space-between; font-size: 12px; color: var(--text-muted);">
                <span>Muestreo: <strong style="color: var(--accent-success);">{{ latest_session.period.upper() }}</strong></span>
                <span>Especie: <strong>{{ selected_profile.display_name if selected_profile else latest_session.crop_type }}</strong></span>
                <span>Hora: <strong>{{ latest_session.timestamp.strftime('%d/%m/%Y %H:%M') }}</strong></span>
            </div>

            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 10px;">
                <div style="border-radius: 8px; overflow: hidden; border: 1px solid var(--border-color); position: relative; height: 180px; background: #090d16; display:
flex; align-items: center; justify-content: center;">
                    <img src="{{ url_for('static', filename=latest_metric.image_path_cenital_proc) }}" style="width: 100%; height: 100%; object-fit: contain;">
                    <div style="position: absolute; bottom: 6px; left: 6px; background: rgba(0,0,0,0.75); padding: 2px 6px; border-radius: 4px; font-size: 10px; font-
weight: 700; color: #10b981;">
                        CENITAL (Máscara)
                    </div>
                </div>
                <div style="border-radius: 8px; overflow: hidden; border: 1px solid var(--border-color); position: relative; height: 180px; background: #090d16; display:
flex; align-items: center; justify-content: center;">
                    <img src="{{ url_for('static', filename=latest_metric.image_path_lateral_proc) }}" style="width: 100%; height: 100%; object-fit: contain;">
                    <div style="position: absolute; bottom: 6px; left: 6px; background: rgba(0,0,0,0.75); padding: 2px 6px; border-radius: 4px; font-size: 10px; font-
weight: 700; color: #3b82f6;">
                        LATERAL (Calibración)
                    </div>
                </div>
            </div>

            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 10px; font-size: 12.5px; text-align: center;">
                <div style="background: var(--bg-card-subtle); padding: 8px; border-radius: 8px; border: 1px solid var(--border-color);">
                    <p style="font-size: 11px; color: var(--text-muted); font-weight: 600; margin-bottom: 2px;">Índice de Compacidad</p>
                    <p style="font-weight: 700; font-size: 16px; margin: 0; color: var(--accent-success);">{{ latest_metric.compacity_index }}</p>
                </div>
                <div style="background: var(--bg-card-subtle); padding: 8px; border-radius: 8px; border: 1px solid var(--border-color);">
                    <p style="font-size: 11px; color: var(--text-muted); font-weight: 600; margin-bottom: 2px;">Índice de Salud Foliar</p>
                    <p style="font-weight: 700; font-size: 16px; margin: 0; color: var(--accent-primary);">{{ latest_metric.health_index }}%</p>
                </div>
            </div>
        </div>
        {% else %}
        <div style="display: flex; flex-direction: column; align-items: center; justify-content: center; height: 200px; color: var(--text-muted); text-align: center;">
            <span class="material-icons-round" style="font-size: 40px; margin-bottom: 8px; color: var(--text-dim);">photo_camera</span>
            <p style="font-weight: 600; font-size: 13px;">No hay muestreos registrados para la <strong>Canastilla #{{ active_plant_id }}</strong>.</p>
            <p style="font-size: 11.5px; margin-top: 2px;">Selecciona la fecha en el calendario arriba y procesa fotos desde la USB o sube el ZIP.</p>
        </div>
        {% endif %}
    </div>
```

```
    <!-- Historial de Muestreos -->
    <div class="widget-card" style="padding: 12px 14px;">
        <div class="widget-header" style="margin-bottom: 8px; padding-bottom: 4px;">
            <div class="widget-header-title">
                <h3 style="font-size: 13px;">Historial de Muestreo</h3>
                <p style="font-size: 11px;">Canastilla #{{ active_plant_id }}</p>
            </div>
        </div>

        <div class="table-responsive" style="max-height: 235px; margin-bottom: 0;">
            <table class="table" style="font-size: 12px;">
                <thead>
                    <tr>
                        <th>Fecha / Hora</th>
                        <th>Periodo</th>
                        <th>Área Foliar</th>
                        <th>Altura</th>
                        <th>Salud</th>
                    </tr>
                </thead>
                <tbody>
                    {% if sessions %}
                        {% for session in sessions %}
                            {% if session.metrics %}
                            {% set metric = session.metrics[0] %}
                            <tr>
                                <td style="font-size: 11.5px;">{{ session.timestamp.strftime('%d/%m/%Y %H:%M') }}</td>
                                <td>
                                    <span class="badge badge-success" style="font-size: 10px; padding: 2px 6px;">
                                        {{ session.period.upper() }}
                                    </span>
                                </td>
                                <td style="font-weight: 600;">{{ metric.foliar_area_cm2 }} cm²</td>
                                <td>{{ metric.plant_height_cm }} cm</td>
                                <td>{{ metric.health_index }}%</td>
                            </tr>
                            {% endif %}
                        {% endfor %}
                    {% else %}
                        <tr>
                            <td colspan="5" style="text-align: center; color: var(--text-muted); padding: 20px;">
                                Sin datos registrados aún.
                            </td>
                        </tr>
                    {% endif %}
                </tbody>
            </table>
        </div>
    </div>
</div>

{% endblock %}

{% block scripts %}
<script>
```

```
    const growthData = {{ growth_data | tojson | safe }};
    const activePlantId = {{ active_plant_id }};
    const processedDates = {{ processed_periods | tojson | safe }};

    function toggleCollapsibleSection(bodyId, iconId, textId) {
        const body = document.getElementById(bodyId);
        const icon = document.getElementById(iconId);
        const txt = document.getElementById(textId);
        if (!body) return;

        if (body.style.display === 'none' || body.style.display === '') {
            body.style.display = 'block';
            if (icon) icon.style.transform = 'rotate(180deg)';
            if (txt) {
                if (textId === 'btn-lots-text') txt.innerText = 'Ocultar Panel de Lotes';
            }
        } else {
            body.style.display = 'none';
            if (icon) icon.style.transform = 'rotate(0deg)';
            if (txt) {
                if (textId === 'btn-lots-text') txt.innerText = 'Ver Lotes y Fechas / Importar';
            }
        }
    }

    function switchImportMode(mode) {
        const lotsBody = document.getElementById('section-lots-body');
        if (lotsBody && (lotsBody.style.display === 'none' || lotsBody.style.display === '')) {
            toggleCollapsibleSection('section-lots-body', 'icon-lots-toggle', 'btn-lots-text');
        }

        const usbContainer = document.getElementById('import-mode-usb-container');
        const dragContainer = document.getElementById('import-mode-drag-container');
        const tabUsb = document.getElementById('tab-mode-usb');
        const tabDrag = document.getElementById('tab-mode-drag');

        if (mode === 'usb') {
            if (usbContainer) usbContainer.style.display = 'block';
            if (dragContainer) dragContainer.style.display = 'none';
            if (tabUsb) tabUsb.className = 'btn btn-primary btn-sm';
            if (tabDrag) tabDrag.className = 'btn btn-outline btn-sm';
            scanUSBDrives();
        } else {
            if (usbContainer) usbContainer.style.display = 'none';
            if (dragContainer) dragContainer.style.display = 'block';
            if (tabUsb) tabUsb.className = 'btn btn-outline btn-sm';
            if (tabDrag) tabDrag.className = 'btn btn-primary btn-sm';
        }
    }

    function onCalendarDateChange(selectedDate) {
        checkDateOverwrite(selectedDate);
    }

    function selectProcessedDate(dateStr) {
```

```
        const dPicker = document.getElementById('calendar-date-picker');
        if (dPicker) {
            dPicker.value = dateStr;
            checkDateOverwrite(dateStr);
        }
    }

    function checkDateOverwrite(dateStr) {
        const warnBox = document.getElementById('overwrite-warning-box');
        if (processedDates.includes(dateStr)) {
            warnBox.style.display = 'flex';
        } else {
            warnBox.style.display = 'none';
        }
    }

    document.addEventListener('DOMContentLoaded', () => {
        const todayStr = new Date().toISOString().split('T')[0];
        const dPicker = document.getElementById('calendar-date-picker');
        if (dPicker && !dPicker.value) {
            dPicker.value = todayStr;
            checkDateOverwrite(todayStr);
        }

        // Gráficas de procesamiento biométrico
        Chart.defaults.font.family = "'Inter', sans-serif";
        Chart.defaults.color = '#64748b';

        const labels = (growthData && growthData.labels && growthData.labels.length) ? growthData.labels : ['Sin registros'];
        const areas = (growthData && growthData.foliar_areas && growthData.foliar_areas.length) ? growthData.foliar_areas : [0];
        const heights = (growthData && growthData.plant_heights && growthData.plant_heights.length) ? growthData.plant_heights : [0];
        const diameters = (growthData && growthData.stem_diameters && growthData.stem_diameters.length) ? growthData.stem_diameters : [0];
        const healths = (growthData && growthData.health_indices && growthData.health_indices.length) ? growthData.health_indices : [100];

        function createSimpleLine(id, label, data, color) {
            const ctx = document.getElementById(id);
            if (!ctx) return;
            new Chart(ctx, {
                type: 'line',
                data: {
                    labels: labels,
                    datasets: [{
                        label: label,
                        data: data,
                        borderColor: color,
                        backgroundColor: color + '18',
                        borderWidth: 2.5,
                        fill: true,
                        tension: 0.35,
                        pointBackgroundColor: color,
                        pointRadius: 4,
                        pointHoverRadius: 6
                    }]
                },
                options: {
```

```
                responsive: true,
                maintainAspectRatio: false,
                plugins: {
                    legend: { display: false },
                    tooltip: {
                        backgroundColor: '#0f172a',
                        titleFont: { size: 12, weight: '700' },
                        bodyFont: { size: 11.5 },
                        padding: 10,
                        cornerRadius: 8
                    }
                },
                scales: {
                    x: {
                        grid: { display: false },
                        ticks: { font: { size: 11 } }
                    },
                    y: {
                        grid: { color: 'rgba(226, 232, 240, 0.6)' },
                        beginAtZero: true,
                        ticks: { font: { size: 11 } }
                    }
                }
            });
        }

        createSimpleLine('foliarAreaChart', 'Área (cm²)', areas, '#059669');
        createSimpleLine('plantHeightChart', 'Altura (cm)', heights, '#2563eb');
        createSimpleLine('stemDiameterChart', 'Diámetro (mm)', diameters, '#d97706');
        createSimpleLine('healthIndexChart', 'Salud (%)', healths, '#dc2626');

        // Escanear USB automáticamente al cargar la vista
        scanUSBDrives();

        // Configuración de Drag & Drop
        const dropZone = document.getElementById('drop-zone');
        const fileInput = document.getElementById('file-input');

        if (dropZone && fileInput) {
            dropZone.addEventListener('dragover', (e) => {
                e.preventDefault();
                dropZone.style.borderColor = '#10b981';
                dropZone.style.background = 'rgba(16, 185, 129, 0.08)';
            });

            dropZone.addEventListener('dragleave', (e) => {
                e.preventDefault();
                dropZone.style.borderColor = 'var(--accent-success)';
                dropZone.style.background = 'rgba(5, 150, 105, 0.03)';
            });

            dropZone.addEventListener('drop', (e) => {
                e.preventDefault();
                dropZone.style.borderColor = 'var(--accent-success)';
```

```
                dropZone.style.background = 'rgba(5, 150, 105, 0.03)';
                if (e.dataTransfer.files.length > 0) {
                    fileInput.files = e.dataTransfer.files;
                    handleFilesSelected();
                }
            });

            fileInput.addEventListener('change', () => {
                handleFilesSelected();
            });
        }
    });

    function handleFilesSelected() {
        const fileInput = document.getElementById('file-input');
        const summary = document.getElementById('selected-files-summary');
        const countText = document.getElementById('files-count-text');

        if (fileInput.files.length > 0) {
            summary.style.display = 'flex';
            countText.innerText = `${fileInput.files.length} archivo(s) preparado(s) para procesar`;
        } else {
            summary.style.display = 'none';
        }
    }

    let currentUsbBatches = [];

    async function scanUSBDrives() {
        const resBox = document.getElementById('usb-scan-results');
        resBox.innerHTML = '<p style="text-align: center; color: var(--text-muted); font-size: 13px;">Buscando memorias USB conectadas...</p>';
        try {
            const res = await fetch('/api/scan_usb');
            const data = await res.json();
            const batches = data.batches || data.drives || [];
            currentUsbBatches = batches;

            if (data.status === 'success' && batches.length > 0) {
                let html = '';

                // Barra superior de acciones masivas
                html += `
                    <div style="background: #09211a; color: #ffffff; border-radius: 10px; padding: 14px 18px; display: flex; align-items: center; justify-content: space-
between; flex-wrap: wrap; gap: 10px; margin-bottom: 12px; box-shadow: 0 4px 12px rgba(0,0,0,0.15);">
                        <div>
                            <div style="font-weight: 700; font-size: 14px; color: #10b981; display: flex; align-items: center; gap: 6px;">
                                <span class="material-icons-round" style="font-size: 18px;">usb</span>
                                <span>Memoria USB Detectada (${batches.length} lote(s) encontrados)</span>
                            </div>
                            <div style="font-size: 12px; color: #94a3b8; margin-top: 2px;">Carpeta: SIFMA_CAPTURES &bull; Ubicación: ${batches[0].drive_root || 'USB'}
</div>
                        </div>
                        <div style="display: flex; align-items: center; gap: 8px;">
                            <label style="display: flex; align-items: center; gap: 6px; font-size: 12.5px; font-weight: 600; color: #cbd5e1; cursor: pointer; margin: 0
4px 0 0;">
                                <input type="checkbox" id="check-all-usb" onchange="toggleSelectAllUSB(this.checked)" checked style="cursor: pointer; width: 16px;
height: 16px; accent-color: #10b981;">
```

```
                                <span>Seleccionar Todo</span>
                            </label>
                            <button class="btn btn-primary btn-sm" id="btn-process-selected-usb" style="background: #10b981; color: #09211a; font-weight: 700;" onclick="processSelectedUSBBatches()">
                                <span class="material-icons-round">play_arrow</span>
                                <span id="btn-process-selected-text">Procesar Seleccionados (${batches.length})</span>
                            </button>
                            <button class="btn btn-outline btn-sm" style="color: #ffffff; border-color: rgba(255,255,255,0.25);" onclick="processAllUSBBatches()" title="Procesar todas las carpetas y subcarpetas recursivamente">
                                <span class="material-icons-round">playlist_play</span>
                                <span>Todo el Historial USB</span>
                            </button>
                        </div>
                    </div>
                `;

                batches.forEach((b, idx) => {
                    const cleanPath = b.path.replace(/\\/g, '\\\\');
                    html += `
                        <div style="background: #ffffff; border: 1.5px solid var(--border-color); border-left: 4px solid var(--accent-success); border-radius: 9px; padding: 12px 16px; display: flex; align-items: center; justify-content: space-between; margin-bottom: 8px;">
                            <div style="display: flex; align-items: center; gap: 12px;">
                                <input type="checkbox" class="usb-batch-check" value="${cleanPath}" data-date="${b.date_str || ''}" data-index="${idx}" onchange="updateSelectedUSBCount()" checked style="cursor: pointer; width: 17px; height: 17px; accent-color: var(--accent-success);">
                                <div>
                                    <div style="font-weight: 800; font-size: 14px; color: var(--text-heading); display: flex; align-items: center; gap: 8px;">
                                        <span class="material-icons-round" style="font-size: 19px; color: var(--accent-success);">calendar_today</span>
                                        <span>Jornada: ${b.date_str || 'Día de Muestreo'}</span>
                                        <span class="badge badge-success" style="font-size: 11px;">${b.turns_count || 1} Turnos (${b.period_str || 'Muestreo'})</span>
                                    </div>
                                    <div style="font-size: 11.5px; color: var(--text-muted); margin-top: 3px;">
                                        <strong>${b.images_count} imágenes</strong> captadas &bull; Carpeta: <code style="font-size: 11px; background: #f1f5f9; padding: 1px 5px; border-radius: 4px;">SIFMA_CAPTURES/${b.date_str}</code>
                                    </div>
                                </div>
                            </div>
                            <button class="btn btn-outline btn-sm" style="font-weight: 700;" onclick="processSingleBatch('${cleanPath}', '${b.date_str}')">
                                <span class="material-icons-round">play_arrow</span>
                                <span>Procesar Este Lote</span>
                            </button>
                        </div>
                    `;
                });
                resBox.innerHTML = html;
                updateSelectedUSBCount();
            } else {
                resBox.innerHTML = '<p style="text-align: center; color: var(--accent-warning); font-size: 13px; font-weight: 600; padding: 12px 0;">No se encontró la carpeta <code>SIFMA_CAPTURES</code> en la memoria USB conectada. Asegúrate de que la Raspberry Pi haya guardado fotos en la memoria o usa el modo "Subir ZIP o Fotos".</p>';
            }
        } catch (e) {
            resBox.innerHTML = '<p style="text-align: center; color: var(--accent-danger); font-size: 13px;">Error al consultar memorias USB.</p>';
        }
    }

    function toggleSelectAllUSB(isChecked) {
        document.querySelectorAll('.usb-batch-check').forEach(chk => {
            chk.checked = isChecked;
```

```
        });
        updateSelectedUSBCount();
    }

    function updateSelectedUSBCount() {
        const checked = document.querySelectorAll('.usb-batch-check:checked');
        const btnText = document.getElementById('btn-process-selected-text');
        const btnProcess = document.getElementById('btn-process-selected-usb');
        const count = checked.length;
        if (btnText) btnText.innerText = `Procesar Seleccionados (${count})`;
        if (btnProcess) {
            btnProcess.disabled = count === 0;
            btnProcess.style.opacity = count === 0 ? '0.5' : '1';
        }
    }

    function createProgressTracker(initialText) {
        const pBar = document.getElementById('processing-status-bar');
        const pFill = document.getElementById('progress-bar-fill');
        const pText = document.getElementById('status-message');
        const pPct = document.getElementById('status-percent');

        pBar.style.display = 'block';
        pFill.style.background = 'linear-gradient(90deg, #10b981, #34d399)';
        pFill.style.width = '5%';
        pPct.innerText = '5%';
        pText.innerText = initialText || 'Iniciando procesamiento...';

        localStorage.setItem('sifma_processing_active', 'true');
        if (window.startGlobalProcessingMonitor) {
            window.startGlobalProcessingMonitor();
        }

        let isPolling = true;

        const pollInterval = setInterval(async () => {
            if (!isPolling) return;
            try {
                const res = await fetch('/api/processing/progress');
                if (res.ok) {
                    const prog = await res.json();
                    if (prog && typeof prog.percent === 'number') {
                        const targetPct = Math.max(5, Math.min(99, prog.percent));
                        pFill.style.width = `${targetPct}%`;
                        pPct.innerText = `${targetPct}%`;
                        if (prog.message) {
                            pText.innerText = prog.message;
                        }
                    }
                }
            } catch (e) {}
        }, 350);

        return {
            finish: (successMsg) => {
```

```
                isPolling = false;
                clearInterval(pollInterval);
                localStorage.removeItem('sifma_processing_active');
                pFill.style.width = '100%';
                pPct.innerText = '100%';
                pText.innerText = successMsg || '¡Procesamiento fenotípico completado con éxito!';

                if (window.showToast) {
                    window.showToast({
                        type: 'success',
                        title: '¡Procesamiento Fenotípico Finalizado!',
                        message: successMsg || 'Lote procesado exitosamente para Canastilla #' + activePlantId,
                        duration: 4500
                    });
                }
            },
            fail: (errorMsg) => {
                isPolling = false;
                clearInterval(pollInterval);
                localStorage.removeItem('sifma_processing_active');
                pFill.style.background = '#ef4444';
                pText.innerText = 'Error: ' + errorMsg;

                if (window.showToast) {
                    window.showToast({
                        type: 'error',
                        title: 'Error de Procesamiento',
                        message: errorMsg,
                        duration: 5000
                    });
                }
            }
        };
    }

    async function processSingleBatch(folderPath, periodStr) {
        const lotsBody = document.getElementById('section-lots-body');
        if (lotsBody && (lotsBody.style.display === 'none' || lotsBody.style.display === '')) {
            toggleCollapsibleSection('section-lots-body', 'icon-lots-toggle', 'btn-lots-text');
        }

        const calDate = document.getElementById('calendar-date-picker').value;
        const tracker = createProgressTracker(`Iniciando procesamiento de ${periodStr}...`);

        try {
            const finalPeriod = calDate ? `${calDate} - ${periodStr}` : periodStr;
            const res = await fetch('/api/process_usb_path', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ folder_path: folderPath, period: finalPeriod, plant_id: activePlantId })
            });
            const data = await res.json();

            if (res.ok && data.status === 'success') {
                tracker.finish(data.message || '¡Procesamiento fenotípico finalizado con éxito!');
```

```
                setTimeout(() => window.location.reload(), 1400);
            } else {
                tracker.fail(data.message || 'Fallo en procesamiento');
            }
        } catch (e) {
            tracker.fail('Error de comunicación con el servidor.');
        }
    }

    async function processSelectedUSBBatches() {
        const checked = Array.from(document.querySelectorAll('.usb-batch-check:checked'));
        if (checked.length === 0) {
            alert('Por favor selecciona al menos una carpeta para procesar.');
            return;
        }

        const selectedPaths = checked.map(chk => chk.value);
        const lotsBody = document.getElementById('section-lots-body');
        if (lotsBody && (lotsBody.style.display === 'none' || lotsBody.style.display === '')) {
            toggleCollapsibleSection('section-lots-body', 'icon-lots-toggle', 'btn-lots-text');
        }

        const calDate = document.getElementById('calendar-date-picker').value;
        const tracker = createProgressTracker(`Iniciando procesamiento de ${selectedPaths.length} lote(s) seleccionados...`);

        try {
            const res = await fetch('/api/process_usb_path', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    folder_paths: selectedPaths,
                    period: calDate || 'Día 1',
                    plant_id: activePlantId
                })
            });
            const data = await res.json();

            if (res.ok && data.status === 'success') {
                tracker.finish(data.message || 'Lotes seleccionados procesados exitosamente.');
                setTimeout(() => window.location.reload(), 1400);
            } else {
                tracker.fail(data.message || 'Fallo en procesamiento de lotes.');
            }
        } catch (e) {
            tracker.fail('Error de comunicación con el servidor.');
        }
    }

    async function processAllUSBBatches() {
        const lotsBody = document.getElementById('section-lots-body');
        if (lotsBody && (lotsBody.style.display === 'none' || lotsBody.style.display === '')) {
            toggleCollapsibleSection('section-lots-body', 'icon-lots-toggle', 'btn-lots-text');
        }

        const calDate = document.getElementById('calendar-date-picker').value;
```

```
        const tracker = createProgressTracker('Iniciando procesamiento completo de memoria USB...');

        try {
            const res = await fetch('/api/process_usb_mount', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ mountpoint: 'D:\\', target_date: calDate, plant_id: activePlantId })
            });
            const data = await res.json();

            if (res.ok && data.status === 'success') {
                tracker.finish(data.message || 'Lotes USB procesados exitosamente.');
                setTimeout(() => window.location.reload(), 1400);
            } else {
                tracker.fail(data.message || 'Fallo en procesamiento');
            }
        } catch (e) {
            tracker.fail('Error de comunicación.');
        }
    }

    async function submitManualBatch() {
        const lotsBody = document.getElementById('section-lots-body');
        if (lotsBody && (lotsBody.style.display === 'none' || lotsBody.style.display === '')) {
            toggleCollapsibleSection('section-lots-body', 'icon-lots-toggle', 'btn-lots-text');
        }

        const fileInput = document.getElementById('file-input');
        const calDate = document.getElementById('calendar-date-picker').value;
        if (!fileInput.files || fileInput.files.length === 0) {
            alert('Por favor selecciona un archivo .ZIP o imágenes para procesar.');
            return;
        }

        const tracker = createProgressTracker('Subiendo y procesando archivos...');

        const formData = new FormData();
        for (let i = 0; i < fileInput.files.length; i++) {
            formData.append('images', fileInput.files[i]);
        }
        formData.append('period', calDate || 'Día 1');
        formData.append('plant_id', activePlantId);

        try {
            const res = await fetch('/api/upload_manual_batch', {
                method: 'POST',
                body: formData
            });
            const data = await res.json();

            if (res.ok && data.status === 'success') {
                tracker.finish(data.message || 'Archivos procesados con éxito.');
                setTimeout(() => window.location.reload(), 1400);
            } else {
                tracker.fail(data.message || 'Error al procesar lote.');
```

```
            }
        } catch (e) {
            tracker.fail('Error de conexión durante la subida.');
        }
    }
</script>
{% endblock %}
```

ARCHIVO 71/85: local_server/templates/scientific_report.html (1973 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dossier y Reporte Científico - {{ dossier.report_id }} - SIFMA</title>

    <!-- Favicon Oficial SIFMA -->
    <link rel="icon" type="image/svg+xml" href="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}">

    <!-- Google Fonts: Philosopher + Inter / Literata & JetBrains Mono -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Literata:ital,opsz,wght@0,7..72,400;0,7..72,600;0,7..72,700;1,7..72,400&family=Philosopher:ital,wght@0,400;0,700;1,400;1,700&rel="stylesheet">

    <!-- Material Icons -->
    <link href="https://fonts.googleapis.com/icon?family=Material+Icons+Round" rel="stylesheet">

    <!-- Chart.js para Renderizado de Gráficas Científicas -->
    <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>

    <style>
        :root {
            --text-main: #0f172a;
            --text-muted: #475569;
            --border-color: #cbd5e1;
            --border-dark: #334155;
            --bg-subtle: #f8fafc;
            --accent-primary: #0f172a;
            --accent-emerald: #059669;
            --accent-blue: #0284c7;
        }

        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
        }

        body {
            font-family: 'Literata', Georgia, -apple-system, BlinkMacSystemFont, serif;
            color: var(--text-main);
            background: #e2e8f0;
            font-size: 11.5px;
            line-height: 1.45;
            padding: 20px 24px;
```

```
        }

        /* CONTENEDOR TIPO HOJA A4 */
        .report-page-sheet {
            max-width: 980px;
            margin: 0 auto 30px auto;
            background: #ffffff;
            padding: 36px 42px;
            border-radius: 4px;
            box-shadow: 0 10px 30px rgba(0, 0, 0, 0.12);
            border: 1px solid #cbd5e1;
            position: relative;
        }

        /* BARRA SUPERIOR DE HERRAMIENTAS (FLOTANTE EN PANTALLA) */
        .toolbar-screen-only {
            max-width: 980px;
            margin: 0 auto 16px auto;
            background: #0f172a;
            color: #ffffff;
            padding: 12px 20px;
            border-radius: 6px;
            display: flex;
            justify-content: space-between;
            align-items: center;
            box-shadow: 0 4px 16px rgba(0,0,0,0.2);
            flex-wrap: wrap;
            gap: 12px;
        }

        .btn-action {
            display: inline-flex;
            align-items: center;
            gap: 6px;
            padding: 7px 14px;
            border-radius: 4px;
            font-size: 12px;
            font-weight: 600;
            cursor: pointer;
            text-decoration: none;
            transition: all 0.15s ease;
            border: none;
            font-family: 'Inter', sans-serif;
        }

        .btn-primary-action {
            background: #059669;
            color: #ffffff;
        }
        .btn-primary-action:hover { background: #047857; }

        .btn-sign-action {
            background: #2563eb;
            color: #ffffff;
        }
```

```
        .btn-sign-action:hover { background: #1d4ed8; }

        .btn-config-action {
            background: #334155;
            color: #ffffff;
        }
        .btn-config-action:hover { background: #1e293b; }

        .btn-outline-action {
            background: transparent;
            color: #ffffff;
            border: 1px solid rgba(255, 255, 255, 0.3);
        }
        .btn-outline-action:hover { background: rgba(255, 255, 255, 0.1); }

        /* PANEL DE CONFIGURACIÓN Y FILTROS MODULARES */
        .config-drawer {
            max-width: 980px;
            margin: 0 auto 16px auto;
            background: #ffffff;
            border: 1px solid #cbd5e1;
            border-radius: 6px;
            padding: 18px 22px;
            box-shadow: 0 4px 14px rgba(0,0,0,0.06);
            font-family: 'Inter', sans-serif;
            display: block;
            transition: all 0.3s ease;
        }

        .config-drawer.collapsed {
            display: none;
        }

        .config-section-title {
            font-size: 11.5px;
            font-weight: 800;
            text-transform: uppercase;
            color: #0f172a;
            letter-spacing: 0.5px;
            margin-bottom: 8px;
            display: flex;
            align-items: center;
            gap: 6px;
        }

        .checkbox-grid {
            display: grid;
            grid-template-columns: repeat(auto-fill, minmax(210px, 1fr));
            gap: 6px 12px;
            margin-bottom: 12px;
        }

        .checkbox-item {
            display: flex;
            align-items: center;
```

```
            gap: 8px;
            font-size: 11.5px;
            color: #334155;
            cursor: pointer;
            user-select: none;
            padding: 4px 6px;
            border-radius: 4px;
            transition: background 0.15s ease;
        }
        .checkbox-item:hover {
            background: #f1f5f9;
        }
        .checkbox-item input[type="checkbox"] {
            width: 14px;
            height: 14px;
            accent-color: #0f172a;
            cursor: pointer;
        }

        /* ENCABEZADO INSTITUCIONAL */
        .report-header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            padding-bottom: 18px;
            border-bottom: 2px solid var(--border-dark);
            margin-bottom: 20px;
        }

        .report-logo-box {
            display: flex;
            align-items: center;
            gap: 14px;
        }

        .report-logo-svg-wrapper {
            width: 52px;
            height: 52px;
            flex-shrink: 0;
            display: flex;
            align-items: center;
            justify-content: center;
        }

        .report-header-title h1 {
            font-family: 'Philosopher', serif;
            font-size: 21px;
            font-weight: 700;
            color: var(--text-main);
            letter-spacing: 0.8px;
            line-height: 1.15;
        }

        .report-header-title p {
            font-size: 10px;
```

```
            color: var(--text-muted);
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 0.5px;
            margin-top: 3px;
        }

        .report-meta-box {
            text-align: right;
            font-size: 11px;
            color: var(--text-muted);
        }

        .report-meta-box strong {
            color: var(--text-main);
            font-family: 'JetBrains Mono', monospace;
            font-size: 11.5px;
        }

        /* METADATOS DE LA MUESTRA */
        .meta-grid-table {
            width: 100%;
            border-collapse: collapse;
            margin-bottom: 22px;
            background: var(--bg-subtle);
            border: 1px solid var(--border-color);
        }

        .meta-grid-table td {
            padding: 7px 12px;
            border: 1px solid var(--border-color);
            font-size: 11px;
        }

        .meta-grid-table .meta-label {
            font-weight: 700;
            color: var(--text-muted);
            width: 22%;
            background: #f1f5f9;
        }

        .meta-grid-table .meta-val {
            font-weight: 600;
            color: var(--text-main);
        }

        /* SECCIONES Y TÍTULOS */
        .section-block {
            margin-bottom: 24px;
            page-break-inside: avoid;
            transition: all 0.2s ease;
        }

        .section-title {
            font-size: 12px;
```

```
            font-weight: 800;
            color: var(--text-main);
            text-transform: uppercase;
            letter-spacing: 0.5px;
            padding-bottom: 5px;
            border-bottom: 1.5px solid var(--border-color);
            margin-bottom: 12px;
            display: flex;
            align-items: center;
            gap: 8px;
        }

        .section-title .section-num {
            display: inline-flex;
            align-items: center;
            justify-content: center;
            width: 18px;
            height: 18px;
            background: var(--text-main);
            color: #ffffff;
            border-radius: 3px;
            font-size: 10px;
        }

        /* TABLAS CIENTÍFICAS */
        .scientific-table {
            width: 100%;
            border-collapse: collapse;
            font-size: 10.5px;
            margin-bottom: 12px;
        }

        .scientific-table th {
            background: #f1f5f9;
            color: var(--text-main);
            font-weight: 700;
            text-align: left;
            padding: 6px 8px;
            border-top: 1.5px solid var(--border-dark);
            border-bottom: 1.5px solid var(--border-dark);
            font-size: 10px;
            text-transform: uppercase;
            letter-spacing: 0.3px;
        }

        .scientific-table td {
            padding: 5px 8px;
            border-bottom: 1px solid #e2e8f0;
            color: var(--text-main);
        }

        .scientific-table tr:last-child td {
            border-bottom: 1.5px solid var(--border-dark);
        }

```

```
        .text-center { text-align: center !important; }
        .text-right { text-align: right !important; }
        .font-mono { font-family: 'JetBrains Mono', monospace !important; font-size: 10px; }

        /* DIAGNÓSTICO CIENTÍFICO (SIN RAYITAS LATERALES DE COLOR) */
        .insight-box {
            padding: 10px 14px;
            border-radius: 4px;
            margin-bottom: 10px;
            font-size: 11px;
            line-height: 1.45;
            border: 1px solid var(--border-color);
            background: var(--bg-subtle);
            color: var(--text-main);
        }

        /* CONTENEDOR DE GRÁFICAS */
        .charts-report-grid {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
            gap: 12px;
            margin-top: 10px;
            margin-bottom: 12px;
        }

        .chart-report-card {
            background: #ffffff;
            border: 1px solid var(--border-color);
            border-radius: 4px;
            padding: 10px;
            page-break-inside: avoid;
        }

        .chart-report-header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            font-size: 10.5px;
            font-weight: 700;
            color: var(--text-main);
            margin-bottom: 6px;
            padding-bottom: 4px;
            border-bottom: 1px solid #f1f5f9;
            text-transform: uppercase;
            letter-spacing: 0.3px;
        }

        .chart-report-canvas-wrapper {
            position: relative;
            height: 130px;
            width: 100%;
        }

        /* REGISTRO FOTOGRÁFICO */
        .photo-comp-card {
```

```
            background: #ffffff;
            border: 1px solid var(--border-color);
            border-radius: 4px;
            padding: 12px;
            margin-bottom: 14px;
            page-break-inside: avoid;
        }

        .photo-comp-header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            font-size: 10.5px;
            margin-bottom: 10px;
            padding-bottom: 6px;
            border-bottom: 1px solid var(--border-color);
        }

        .photo-duo-container {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 12px;
        }

        .photo-box {
            text-align: center;
        }

        .photo-frame {
            border: 1px solid #cbd5e1;
            border-radius: 3px;
            overflow: hidden;
            background: #0f172a;
            display: flex;
            align-items: center;
            justify-content: center;
            height: 140px;
        }

        .photo-frame img {
            max-width: 100%;
            max-height: 100%;
            object-fit: contain;
            display: block;
        }

        .photo-box-caption {
            font-size: 9.5px;
            color: var(--text-muted);
            margin-top: 4px;
            font-weight: 600;
        }
        .photo-box-caption.proc {
            color: var(--text-main);
            font-weight: 700;
```

```
        }

        /* PREGUNTAS Y CONCLUSIONES (SIN RAYITAS DE COLOR) */
        .question-card {
            background: var(--bg-subtle);
            border: 1px solid var(--border-color);
            border-radius: 4px;
            padding: 12px;
            margin-bottom: 10px;
            page-break-inside: avoid;
            position: relative;
        }

        .question-header {
            display: flex;
            justify-content: space-between;
            align-items: flex-start;
            gap: 10px;
            margin-bottom: 6px;
        }

        .question-title {
            font-weight: 700;
            color: var(--text-main);
            font-size: 11.5px;
            flex: 1;
        }

        .question-tools-screen {
            display: flex;
            gap: 4px;
        }

        .question-btn-mini {
            background: transparent;
            border: none;
            color: var(--text-muted);
            cursor: pointer;
            padding: 2px 4px;
            border-radius: 3px;
            font-size: 11px;
            display: inline-flex;
            align-items: center;
        }
        .question-btn-mini:hover {
            color: #dc2626;
            background: #fee2e2;
        }

        .question-response-area {
            width: 100%;
            font-family: inherit;
            font-size: 11px;
            color: var(--text-main);
            background: #ffffff;
```

```
            border: 1px solid #cbd5e1;
            border-radius: 3px;
            padding: 8px;
            min-height: 48px;
            resize: vertical;
            line-height: 1.45;
        }

        /* BLOQUE DE FIRMAS */
        .signatures-row {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 40px;
            margin-top: 30px;
            padding-top: 20px;
            page-break-inside: avoid;
        }

        .signature-line {
            text-align: center;
        }

        .signature-box-area {
            height: 70px;
            display: flex;
            align-items: flex-end;
            justify-content: center;
            margin-bottom: 6px;
            position: relative;
        }

        .signature-stamp-img {
            max-height: 65px;
            max-width: 180px;
            object-fit: contain;
            filter: drop-shadow(0 1px 2px rgba(0,0,0,0.15));
        }

        .signature-placeholder-btn {
            border: 1px dashed #94a3b8;
            background: #f8fafc;
            color: #64748b;
            font-size: 11px;
            font-weight: 600;
            padding: 6px 12px;
            border-radius: 3px;
            cursor: pointer;
            display: inline-flex;
            align-items: center;
            gap: 6px;
        }
        .signature-placeholder-btn:hover {
            background: #f1f5f9;
            color: #0f172a;
            border-color: #475569;
```

```
        }

        .signature-rule {
            border-top: 1.5px solid var(--border-dark);
            padding-top: 6px;
        }
        .signature-rule strong {
            display: block;
            font-size: 11.5px;
            color: var(--text-main);
        }
        .signature-rule span {
            font-size: 10px;
            color: var(--text-muted);
        }

        .editable-field {
            outline: none;
            border-bottom: 1px dashed transparent;
            transition: border 0.15s ease;
        }
        .editable-field:hover, .editable-field:focus {
            border-bottom-color: #0f172a;
            background: rgba(15, 23, 42, 0.04);
        }

        /* MODAL DE FIRMA DIGITAL */
        .sig-modal-backdrop {
            position: fixed;
            top: 0;
            left: 0;
            right: 0;
            bottom: 0;
            background: rgba(15, 23, 42, 0.75);
            backdrop-filter: blur(4px);
            display: none;
            align-items: center;
            justify-content: center;
            z-index: 9999;
            padding: 20px;
        }

        .sig-modal-card {
            background: #ffffff;
            border-radius: 8px;
            max-width: 520px;
            width: 100%;
            box-shadow: 0 20px 40px rgba(0,0,0,0.3);
            overflow: hidden;
            font-family: 'Inter', sans-serif;
        }

        .sig-modal-header {
            background: #0f172a;
            color: #ffffff;
```

```
            padding: 14px 20px;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }

        .sig-modal-body {
            padding: 20px;
        }

        .sig-tabs-nav {
            display: flex;
            border-bottom: 2px solid #e2e8f0;
            margin-bottom: 16px;
        }

        .sig-tab-btn {
            flex: 1;
            padding: 10px;
            background: transparent;
            border: none;
            font-weight: 700;
            font-size: 12px;
            color: #64748b;
            cursor: pointer;
            display: flex;
            align-items: center;
            justify-content: center;
            gap: 6px;
            border-bottom: 2px solid transparent;
            margin-bottom: -2px;
        }
        .sig-tab-btn.active {
            color: #0f172a;
            border-bottom-color: #0f172a;
        }

        .sig-tab-content {
            display: none;
        }
        .sig-tab-content.active {
            display: block;
        }

        .sig-dropzone {
            border: 2px dashed #cbd5e1;
            background: #f8fafc;
            border-radius: 6px;
            padding: 24px 16px;
            text-align: center;
            cursor: pointer;
        }

        .sig-canvas-wrapper {
            border: 1px solid #cbd5e1;
```

```
            border-radius: 6px;
            background: #ffffff;
            overflow: hidden;
            height: 160px;
            position: relative;
        }

        #signature-pad {
            width: 100%;
            height: 100%;
            display: block;
            touch-action: none;
        }

        /* ESTILOS DE IMPRESIÓN Y EXPORTACIÓN A PDF */
        @media print {
            body {
                background: #ffffff;
                padding: 0;
                color: #000000;
            }

            .toolbar-screen-only,
            .config-drawer,
            .question-tools-screen,
            .signature-placeholder-btn,
            .sig-modal-backdrop,
            .no-print {
                display: none !important;
            }

            .report-page-sheet {
                max-width: 100%;
                margin: 0;
                padding: 0;
                border: none;
                box-shadow: none;
            }

            .section-block, .photo-comp-card, .signatures-row, .chart-report-card, .question-card {
                page-break-inside: avoid;
            }

            @page {
                size: A4 portrait;
                margin: 12mm 10mm 12mm 10mm;
            }
        }
    </style>
</head>
<body>

    <!-- 1. BARRA SUPERIOR DE ACCIONES (FLOTANTE EN PANTALLA) -->
    <div class="toolbar-screen-only">
        <div style="display: flex; align-items: center; gap: 12px;">
```

```
            <a href="{{ url_for('dashboard.index') }}" class="btn-action btn-outline-action" title="Volver al Panel Principal">
                <span class="material-icons-round" style="font-size: 16px;">arrow_back</span>
                <span>Volver a SIFMA</span>
            </a>
            <span style="font-weight: 700; font-size: 13px; font-family: 'Inter', sans-serif;">
                Dossier & Reportes: Canastilla #{{ dossier.plant_id }} ({{ dossier.target_date }})
            </span>
        </div>

        <div style="display: flex; align-items: center; gap: 8px; flex-wrap: wrap;">

            <!-- Selector de Canastilla / Torre -->
            <select onchange="window.location.href='/reports?plant_id=' + this.value + '&date={{ dossier.target_date }}'" style="background: rgba(255,255,255,0.15); border: 1px solid rgba(255,255,255,0.3); color: #ffffff; padding: 6px 10px; border-radius: 4px; font-size: 12px; font-weight: 600; cursor: pointer;">
                {% for pid in range(1, 5) %}
                    <option value="{{ pid }}" {% if dossier.plant_id == pid %}selected{% endif %} style="color: #000;">
                        Canastilla #{{ pid }} ({{ plant_profiles_map[pid].display_name if plant_profiles_map and pid in plant_profiles_map else 'Torre ' ~ pid }})
                    </option>
                {% endfor %}
            </select>

            <!-- Selector de Fecha -->
            <select onchange="window.location.href='/reports?plant_id={{ dossier.plant_id }}&date=' + this.value" style="background: rgba(255,255,255,0.15); border: 1px solid rgba(255,255,255,0.3); color: #ffffff; padding: 6px 10px; border-radius: 4px; font-size: 12px; font-weight: 600; cursor: pointer;">
                {% for d in dossier.available_dates %}
                    <option value="{{ d }}" {% if dossier.target_date == d %}selected{% endif %} style="color: #000;">Fecha: {{ d }}</option>
                {% endfor %}
            </select>

            <!-- Botón de Personalización -->
            <button type="button" class="btn-action btn-config-action" onclick="toggleConfigDrawer()" title="Personalizar secciones, gráficas y fotos">
                <span class="material-icons-round" style="font-size: 16px;">tune</span>
                <span>Personalizar Reporte</span>
            </button>

            <!-- Botón de Firmador Digital -->
            <button type="button" class="btn-action btn-sign-action" onclick="openSignatureModal(1)" title="Estampar firma digital del investigador o validador">
                <span class="material-icons-round" style="font-size: 16px;">draw</span>
                <span>Firmar</span>
            </button>

            <!-- Botón de Impresión / PDF -->
            <button type="button" class="btn-action btn-primary-action" onclick="window.print()" title="Imprimir o Guardar como PDF">
                <span class="material-icons-round" style="font-size: 16px;">picture_as_pdf</span>
                <span>Imprimir / PDF</span>
            </button>

        </div>
    </div>

    <!-- 2. PANEL DESPLEGABLE DE PERSONALIZACIÓN Y SELECCIÓN MODULAR -->
    <div id="config-drawer" class="config-drawer">
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 14px; padding-bottom: 8px; border-bottom: 1px solid #e2e8f0;">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="color: #0f172a; font-size: 18px;">checklist</span>
                <strong style="font-size: 12.5px; color: var(--text-main);">Configuración Modular del Documento</strong>
```

```
            </div>
            <div style="display: flex; gap: 8px;">
                <button type="button" onclick="setAllCheckboxes(true)" style="background: transparent; border: 1px solid #cbd5e1; padding: 4px 8px; border-radius: 3px; font-size: 11px; cursor: pointer; color: #475569;">
                    Seleccionar Todo
                </button>
                <button type="button" onclick="toggleConfigDrawer()" style="background: transparent; border: none; color: #64748b; cursor: pointer;">
                    <span class="material-icons-round" style="font-size: 18px;">close</span>
                </button>
            </div>
        </div>

        <!-- FILA 1: SECCIONES PRINCIPALES DEL INFORME -->
        <div class="config-section-title">
            <span class="material-icons-round" style="font-size: 15px; color: #0f172a;">view_agenda</span>
            <span>1. Secciones del Documento</span>
        </div>
        <div class="checkbox-grid">
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-meta" checked onchange="toggleSection('sec-meta', this.checked)"> Metadatos y Datos del Lote</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-diagnostico" checked onchange="toggleSection('sec-diagnostico', this.checked)"> Diagnóstico Agronómico</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-sensors-table" checked onchange="toggleSection('sec-sensors-table', this.checked)"> Matriz 8 Sensores Ambientales</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-biometric-table" checked onchange="toggleSection('sec-biometric-table', this.checked)"> Matriz 4 Métricas de Cámara</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-pearson" checked onchange="toggleSection('sec-pearson', this.checked)"> Matriz de Correlación Pearson</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-charts" checked onchange="toggleSection('sec-charts', this.checked)"> Curvas y Gráficas de Tendencia</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-sessions" checked onchange="toggleSection('sec-sessions', this.checked)"> Desglose Lote a Lote</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-photos" checked onchange="toggleSection('sec-photos', this.checked)"> Registro Fotográfico Comparativo</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-conclusions" checked onchange="toggleSection('sec-conclusions', this.checked)"> Preguntas, Conclusiones y Bitácora</label>
            <label class="checkbox-item"><input type="checkbox" id="chk-sec-signatures" checked onchange="toggleSection('sec-signatures', this.checked)"> Bloque de Firmas de Validación</label>
        </div>

        <!-- FILA 2: DESGLOSE INDIVIDUAL DE GRÁFICAS (SIN EMOJIS) -->
        <div class="config-section-title" style="margin-top: 6px;">
            <span class="material-icons-round" style="font-size: 15px; color: #0f172a;">show_chart</span>
            <span>2. Desglose Individual de Gráficas</span>
        </div>
        <div class="checkbox-grid">
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-temp" checked onchange="toggleChartCard('chart-card-temp', this.checked)"> Temperatura (°C)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-hum" checked onchange="toggleChartCard('chart-card-hum', this.checked)"> Humedad Relativa (%)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-uv" checked onchange="toggleChartCard('chart-card-uv', this.checked)"> Radiación Solar / UV (lux)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-curr" checked onchange="toggleChartCard('chart-card-curr', this.checked)"> Corriente de Bomba (A)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-ph" checked onchange="toggleChartCard('chart-card-ph', this.checked)"> pH de Solución Nutritiva</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-ec" checked onchange="toggleChartCard('chart-card-ec', this.checked)"> Conductividad Eléctrica (EC)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-turb" checked onchange="toggleChartCard('chart-card-turb', this.checked)"> Turbidez del Agua (NTU)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-do" checked onchange="toggleChartCard('chart-card-do', this.checked)"> Oxígeno Disuelto (DO)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-area" checked onchange="toggleChartCard('chart-card-area', this.checked)"> Área Foliar Cenital (cm²)</label>
```

```
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-height" checked onchange="toggleChartCard('chart-card-height', this.checked)"> Altura de Planta (cm)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-stem" checked onchange="toggleChartCard('chart-card-stem', this.checked)"> Diámetro de Tallo (mm)</label>
            <label class="checkbox-item"><input type="checkbox" class="chk-chart" id="chk-chart-health" checked onchange="toggleChartCard('chart-card-health', this.checked)"> Índice de Salud Foliar (%)</label>
        </div>

        <!-- FILA 3: FILTRO DE FOTOGRAFÍAS -->
        <div style="display: flex; gap: 20px; flex-wrap: wrap; margin-top: 10px; padding-top: 12px; border-top: 1px solid #f1f5f9; align-items: center;">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="font-size: 16px; color: #0f172a;">photo_camera</span>
                <label style="font-size: 11.5px; font-weight: 700; color: var(--text-main);">Filtro de Fotografías:</label>
                <select id="select-photo-filter" onchange="applyPhotoDisplayFilter(this.value)" style="padding: 5px 8px; border-radius: 4px; border: 1px solid #cbd5e1; font-size: 11.5px; font-weight: 600;">
                    <option value="all">Todas las Tomas del Día</option>
                    <option value="last" selected>Solo la Última Toma (Reporte Compacto)</option>
                    <option value="morning">Tomas de la Mañana (06:00 - 12:00)</option>
                    <option value="afternoon">Tomas de la Tarde (12:00 - 18:00)</option>
                    <option value="none">Ocultar Fotografías (Solo Datos)</option>
                </select>
            </div>

            <div style="display: flex; align-items: center; gap: 8px;">
                <label style="font-size: 11.5px; font-weight: 700; color: var(--text-main);">Vistas:</label>
                <select id="select-photo-views" onchange="applyPhotoViewFilter(this.value)" style="padding: 5px 8px; border-radius: 4px; border: 1px solid #cbd5e1; font-size: 11.5px; font-weight: 600;">
                    <option value="both" selected>Cenital + Lateral (Completo)</option>
                    <option value="cenital">Solo Cenital</option>
                    <option value="lateral">Solo Lateral</option>
                </select>
            </div>
        </div>

    </div>

    <!-- 3. CUERPO DEL DOCUMENTO CIENTÍFICO (HOJA A4) -->
    <div class="report-page-sheet" id="report-sheet">

        <!-- ENCABEZADO CIENTÍFICO INSTITUCIONAL -->
        <header class="report-header">
            <div class="report-logo-box">
                <div class="report-logo-svg-wrapper">
                    <img src="{{ url_for('static', filename='img/sifma_sin_texto.svg') }}" alt="SIFMA Emblem" style="width: 100%; height: 100%; display: block;">
                </div>
                <div class="report-header-title">
                    <h1>SIFMA - DOSSIER CIENTÍFICO</h1>
                    <p>Sistema de Fenotipado & Telemetría Agronómica en Torres Hidropónicas</p>
                </div>
            </div>
            <div class="report-meta-box">
                <div>INFORME ID: <strong>{{ dossier.report_id }}</strong></div>
                <div>FECHA DE EMISIÓN: <strong>{{ dossier.generated_at }}</strong></div>
                <div>PIPELINE: <strong>SIFMA-CV v1.1 (Otsu + LAB + HSV)</strong></div>
            </div>
        </header>

        <!-- SECCIÓN: METADATOS TÉCNICOS DE LA JORNADA -->
```

```
        <div id="sec-meta" class="section-block">
            <table class="meta-grid-table">
                <tr>
                    <td class="meta-label">CANASTILLA / NODO:</td>
                    <td class="meta-val">Canastilla #{{ dossier.plant_id }} ({{ dossier.level_name }})</td>
                    <td class="meta-label">ESPECIE EVALUADA:</td>
                    <td class="meta-val">{{ dossier.display_name }} ({{ dossier.crop_key | capitalize }})</td>
                </tr>
                <tr>
                    <td class="meta-label">FECHA DE MUESTREO:</td>
                    <td class="meta-val"><strong>{{ dossier.target_date }}</strong></td>
                    <td class="meta-label">SESIONES REGISTRADAS:</td>
                    <td class="meta-val">{{ dossier.detailed_sessions | length }} Lotes fotográficos</td>
                </tr>
                <tr>
                    <td class="meta-label">INVESTIGADOR PRINCIPAL:</td>
                    <td class="meta-val"><span id="meta-evaluator-name" class="editable-field" contenteditable="true">{{ current_user.name if current_user else 'Ing. Andrés Luna' }}</span></td>
                    <td class="meta-label">ESTRUCTURA DE CULTIVO:</td>
                    <td class="meta-val">Torre Hidropónica Vertical - Módulo Central</td>
                </tr>
            </table>
        </div>

        <!-- SECCIÓN 1: DIAGNÓSTICO AGRONÓMICO & ALERTAS (FORMAL, SIN RAYITAS) -->
        <div id="sec-diagnostico" class="section-block">
            <div class="section-title">
                <span class="section-num">1</span>
                <span>Diagnóstico Agronómico Automatizado de la Jornada</span>
            </div>

            {% if dossier.automated_insights %}
                {% for insight in dossier.automated_insights %}
                    <div class="insight-box">
                        <strong>{{ insight.title }}:</strong> {{ insight.detail }}
                    </div>
                {% endfor %}
            {% else %}
                <div class="insight-box">
                    <strong>Condiciones Normales:</strong> Los parámetros ambientales y biométricos se mantuvieron estables dentro de los rangos fisiológicos previstos para la jornada.
                </div>
            {% endif %}
        </div>

        <!-- SECCIÓN 2: MATRIZ ESTADÍSTICA DE TELEMETRÍA AMBIENTAL (8 SENSORES) -->
        <div id="sec-sensors-table" class="section-block">
            <div class="section-title">
                <span class="section-num">2</span>
                <span>Matriz Estadística Descriptiva de Microclima (8 Sensores Ambientales)</span>
            </div>

            <table class="scientific-table">
                <thead>
                    <tr>
                        <th>Parámetro Físico / Calidad</th>
```

```
                        <th class="text-center">Símbolo</th>
                        <th class="text-center">Unidad</th>
                        <th class="text-center">Rango Óptimo</th>
                        <th class="text-right">Media (&mu;)</th>
                        <th class="text-right">Desv. Est (&sigma;)</th>
                        <th class="text-right">Mínimo</th>
                        <th class="text-right">Máximo</th>
                        <th class="text-right">Mediana</th>
                        <th class="text-right">IQR</th>
                        <th class="text-right">CV (%)</th>
                        <th class="text-right">Muestras (N)</th>
                    </tr>
                </thead>
                <tbody>
                    {% for row in dossier.sensor_stats_table %}
                        <tr>
                            <td><strong>{{ row.param }}</strong></td>
                            <td class="text-center font-mono">{{ row.symbol }}</td>
                            <td class="text-center">{{ row.unit }}</td>
                            <td class="text-center" style="font-size: 9.5px; color: var(--text-muted);">{{ row.optimal }}</td>
                            <td class="text-right font-mono"><strong>{{ row.stats.mean }}</strong></td>
                            <td class="text-right font-mono">{{ row.stats.std }}</td>
                            <td class="text-right font-mono">{{ row.stats.min }}</td>
                            <td class="text-right font-mono">{{ row.stats.max }}</td>
                            <td class="text-right font-mono">{{ row.stats.median }}</td>
                            <td class="text-right font-mono">{{ row.stats.iqr }}</td>
                            <td class="text-right font-mono">{{ row.stats.cv_pct }}%</td>
                            <td class="text-right font-mono">{{ row.stats.count }}</td>
                        </tr>
                    {% endfor %}
                </tbody>
            </table>
            <div style="font-size: 9px; color: var(--text-muted); text-align: right; margin-top: -6px;">
                &mu; = Media Aritmética | &sigma; = Desviación Estándar Muestral | IQR = Rango Intercuartílico | CV = Coeficiente de Variación de Pearson
            </div>
        </div>

        <!-- SECCIÓN 3: MATRIZ ESTADÍSTICA BIOMÉTRICA FENOTÍPICA (4 MÉTRICAS DE CÁMARA) -->
        <div id="sec-biometric-table" class="section-block">
            <div class="section-title">
                <span class="section-num">3</span>
                <span>Matriz Estadística Descriptiva de Biometría Foliar (4 Mediciones de Visión)</span>
            </div>

            <table class="scientific-table">
                <thead>
                    <tr>
                        <th>Biomarcador Morfométrico</th>
                        <th class="text-center">Símbolo</th>
                        <th class="text-center">Unidad</th>
                        <th class="text-right">Media Diaria (&mu;)</th>
                        <th class="text-right">Desv. Est (&sigma;)</th>
                        <th class="text-right">Mínimo</th>
                        <th class="text-right">Máximo</th>
                        <th class="text-right">Mediana</th>
```

```
                    <th class="text-right">CV (%)</th>
                    <th class="text-right">Lotes (N)</th>
                </tr>
            </thead>
            <tbody>
                {% for row in dossier.biometric_stats_table %}
                    <tr>
                        <td><strong>{{ row.param }}</strong></td>
                        <td class="text-center font-mono">{{ row.symbol }}</td>
                        <td class="text-center">{{ row.unit }}</td>
                        <td class="text-right font-mono"><strong>{{ row.stats.mean }}</strong></td>
                        <td class="text-right font-mono">{{ row.stats.std }}</td>
                        <td class="text-right font-mono">{{ row.stats.min }}</td>
                        <td class="text-right font-mono">{{ row.stats.max }}</td>
                        <td class="text-right font-mono">{{ row.stats.median }}</td>
                        <td class="text-right font-mono">{{ row.stats.cv_pct }}%</td>
                        <td class="text-right font-mono">{{ row.stats.count }}</td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>

    <!-- SECCIÓN 4: MATRIZ DE CORRELACIÓN DE PEARSON -->
    <div id="sec-pearson" class="section-block">
        <div class="section-title">
            <span class="section-num">4</span>
            <span>Matriz de Correlación Lineal de Pearson (Cruce Sensor vs Planta)</span>
        </div>

        <table class="scientific-table">
            <thead>
                <tr>
                    <th>Par de Variables Cruzadas (Microclima vs Biometría)</th>
                    <th class="text-center">Coeficiente de Pearson (r)</th>
                    <th>Interpretación Fisiológica / Agronómica</th>
                </tr>
            </thead>
            <tbody>
                {% for c in dossier.correlation_matrix %}
                    <tr>
                        <td><strong>{{ c.pair }}</strong></td>
                        <td class="text-center font-mono">
                            <strong>{{ c.r }}</strong>
                        </td>
                        <td>{{ c.desc }}</td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>

    <!-- SECCIÓN 5: CURVAS Y GRÁFICAS DE TENDENCIA (SIN EMOJIS) -->
    <div id="sec-charts" class="section-block">
        <div class="section-title">
```

```
                <span class="section-num">5</span>
                <span>Curvas y Gráficas Estadísticas de la Jornada</span>
            </div>

            <div class="charts-report-grid">

                <!-- Gráfica 1: Temperatura -->
                <div class="chart-report-card" id="chart-card-temp">
                    <div class="chart-report-header">
                        <span>Temperatura Ambiental (°C)</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[0].stats.mean }} °C</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-temp"></canvas>
                    </div>
                </div>

                <!-- Gráfica 2: Humedad -->
                <div class="chart-report-card" id="chart-card-hum">
                    <div class="chart-report-header">
                        <span>Humedad Relativa (%)</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[1].stats.mean }} %</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-hum"></canvas>
                    </div>
                </div>

                <!-- Gráfica 3: Radiación UV -->
                <div class="chart-report-card" id="chart-card-uv">
                    <div class="chart-report-header">
                        <span>Radiación Solar / UV (lux)</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[2].stats.mean }} lux</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-uv"></canvas>
                    </div>
                </div>

                <!-- Gráfica 4: Corriente Bomba -->
                <div class="chart-report-card" id="chart-card-curr">
                    <div class="chart-report-header">
                        <span>Corriente de Bomba (A)</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[3].stats.mean }} A</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-curr"></canvas>
                    </div>
                </div>

                <!-- Gráfica 5: pH Solución -->
                <div class="chart-report-card" id="chart-card-ph">
                    <div class="chart-report-header">
                        <span>pH de Solución Nutritiva</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[5].stats.mean }} pH</span>
```

```
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-ph"></canvas>
                    </div>
                </div>

                <!-- Gráfica 6: EC Conductividad -->
                <div class="chart-report-card" id="chart-card-ec">
                    <div class="chart-report-header">
                        <span>Conductividad Eléctrica (EC)</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[4].stats.mean }} µS/cm</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-ec"></canvas>
                    </div>
                </div>

                <!-- Gráfica 7: Turbidez -->
                <div class="chart-report-card" id="chart-card-turb">
                    <div class="chart-report-header">
                        <span>Turbidez del Agua (NTU)</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[6].stats.mean }} NTU</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-turb"></canvas>
                    </div>
                </div>

                <!-- Gráfica 8: Oxígeno Disuelto -->
                <div class="chart-report-card" id="chart-card-do">
                    <div class="chart-report-header">
                        <span>Oxígeno Disuelto (DO)</span>
                        <span class="font-mono">{{ dossier.sensor_stats_table[7].stats.mean }} mg/L</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-do"></canvas>
                    </div>
                </div>

                <!-- Gráfica 9: Área Foliar -->
                <div class="chart-report-card" id="chart-card-area">
                    <div class="chart-report-header">
                        <span>Área Foliar Cenital (cm²)</span>
                        <span class="font-mono">{{ dossier.biometric_stats_table[0].stats.mean }} cm²</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-area"></canvas>
                    </div>
                </div>

                <!-- Gráfica 10: Altura de Planta -->
                <div class="chart-report-card" id="chart-card-height">
                    <div class="chart-report-header">
                        <span>Altura de Planta (cm)</span>
                        <span class="font-mono">{{ dossier.biometric_stats_table[1].stats.mean }} cm</span>
```

```
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-height"></canvas>
                    </div>
                </div>

                <!-- Gráfica 11: Diámetro de Tallo -->
                <div class="chart-report-card" id="chart-card-stem">
                    <div class="chart-report-header">
                        <span>Diámetro de Tallo (mm)</span>
                        <span class="font-mono">{{ dossier.biometric_stats_table[2].stats.mean }} mm</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-stem"></canvas>
                    </div>
                </div>

                <!-- Gráfica 12: Salud Foliar -->
                <div class="chart-report-card" id="chart-card-health">
                    <div class="chart-report-header">
                        <span>Índice de Salud Foliar (%)</span>
                        <span class="font-mono">{{ dossier.biometric_stats_table[3].stats.mean }} %</span>
                    </div>
                    <div class="chart-report-canvas-wrapper">
                        <canvas id="canvas-report-health"></canvas>
                    </div>
                </div>

            </div>
        </div>

        <!-- SECCIÓN 6: DESGLOSE MUESTRA A MUESTRA DE LA JORNADA -->
        <div id="sec-sessions" class="section-block">
            <div class="section-title">
                <span class="section-num">6</span>
                <span>Desglose Sincronizado Lote a Lote (Período por Período)</span>
            </div>

            <table class="scientific-table">
                <thead>
                    <tr>
                        <th>Período</th>
                        <th class="text-center">Hora</th>
                        <th class="text-right">Temp (°C)</th>
                        <th class="text-right">HR (%)</th>
                        <th class="text-right">UV (lux)</th>
                        <th class="text-right">Bomba (A)</th>
                        <th class="text-right">Área Foliar (cm²)</th>
                        <th class="text-right">Altura (cm)</th>
                        <th class="text-right">Tallo (mm)</th>
                        <th class="text-right">Salud (%)</th>
                    </tr>
                </thead>
                <tbody>
                    {% for s in dossier.detailed_sessions %}
```

```
                        <tr>
                            <td><strong>{{ s.period }}</strong></td>
                            <td class="text-center font-mono">{{ s.timestamp }}</td>
                            <td class="text-right font-mono">{{ s.temperature }}</td>
                            <td class="text-right font-mono">{{ s.humidity }}%</td>
                            <td class="text-right font-mono">{{ s.uv_solar }}</td>
                            <td class="text-right font-mono">{{ s.motor_current }}</td>
                            <td class="text-right font-mono"><strong>{{ s.foliar_area_cm2 }}</strong></td>
                            <td class="text-right font-mono">{{ s.plant_height_cm }}</td>
                            <td class="text-right font-mono">{{ s.stem_diameter_mm }}</td>
                            <td class="text-right font-mono">{{ s.health_index }}%</td>
                        </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>

        <!-- SECCIÓN 7: REGISTRO FOTOGRÁFICO COMPARATIVO (CÁMARAS) -->
        <div id="sec-photos" class="section-block">
            <div class="section-title">
                <span class="section-num">7</span>
                <span>Registro Fotográfico Comparativo (Original Sin Procesar vs Segmentada)</span>
            </div>

            <div id="photos-cards-container">
                {% if dossier.photo_comparisons %}
                    {% for photo in dossier.photo_comparisons %}
                        <div class="photo-comp-card" data-time="{{ photo.time }}" data-period="{{ photo.period }}" data-index="{{ loop.index0 }}" data-is-last="{{ 'true' if loop.last else 'false' }}">
                            <div class="photo-comp-header">
                                <div>
                                    <span style="font-weight: 800; text-transform: uppercase;">Muestreo: {{ photo.period }}</span>
                                    <span style="color: var(--text-muted); margin-left: 6px; font-family: 'JetBrains Mono', monospace;">({{ photo.time }})</span>
                                </div>
                                <div>
                                    <span>Área: <strong>{{ photo.area_cm2 }} cm²</strong></span>
                                    <span style="margin: 0 6px; color: var(--border-color);">|</span>
                                    <span>Altura: <strong>{{ photo.height_cm }} cm</strong></span>
                                    <span style="margin: 0 6px; color: var(--border-color);">|</span>
                                    <span>Salud: <strong>{{ photo.health_index }}%</strong></span>
                                </div>
                            </div>

                            <!-- Par Cenital -->
                            <div class="photo-pair-cenital" style="margin-bottom: 14px;">
                                <div style="font-size: 10px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; margin-bottom: 5px; letter-spacing: 0.5px;">
                                    1. Vista Cenital (Segmentación de Área Foliar & Clorofila)
                                </div>
                                <div class="photo-duo-container">
                                    <div class="photo-box">
                                        <div class="photo-frame">
                                            <img src="{{ photo.cenital_orig }}" alt="Cenital Original">
                                        </div>
                                        <div class="photo-box-caption">Original RAW (Sin Procesar)</div>
                                    </div>
```

```
                                    <div class="photo-box">
                                        <div class="photo-frame">
                                            <img src="{{ photo.cenital_proc }}" alt="Cenital Procesada">
                                        </div>
                                        <div class="photo-box-caption proc">Segmentada (Contorno OpenCV + Máscara)</div>
                                    </div>
                                </div>
                            </div>

                            <!-- Par Lateral -->
                            <div class="photo-pair-lateral">
                                <div style="font-size: 10px; font-weight: 700; color: var(--text-muted); text-transform: uppercase; margin-bottom: 5px; letter-spacing: 0.5px;">
                                    2. Vista Lateral (Estimación de Altura de Planta & Tallo)
                                </div>
                                <div class="photo-duo-container">
                                    <div class="photo-box">
                                        <div class="photo-frame">
                                            <img src="{{ photo.lateral_orig }}" alt="Lateral Original">
                                        </div>
                                        <div class="photo-box-caption">Original RAW (Sin Procesar)</div>
                                    </div>
                                    <div class="photo-box">
                                        <div class="photo-frame">
                                            <img src="{{ photo.lateral_proc }}" alt="Lateral Procesada">
                                        </div>
                                        <div class="photo-box-caption proc">Calibrada (Perfil OpenCV & Eje de Altura)</div>
                                    </div>
                                </div>
                            </div>
                        </div>
                    {% endfor %}
                {% else %}
                    <div style="background: var(--bg-subtle); padding: 18px; border-radius: 4px; border: 1px dashed var(--border-color); text-align: center; color: var(--text-muted);">
                        No se registraron capturas fotográficas procesadas en la fecha seleccionada.
                    </div>
                {% endif %}
            </div>
        </div>

        <!-- SECCIÓN 8: PREGUNTAS TÉCNICAS, CONCLUSIONES Y BITÁCORA LIBRE (FORMAL) -->
        <div id="sec-conclusions" class="section-block">
            <div style="display: flex; justify-content: space-between; align-items: center; border-bottom: 1.5px solid var(--border-color); padding-bottom: 5px; margin-bottom: 12px;">
                <div class="section-title" style="border: none; margin: 0; padding: 0;">
                    <span class="section-num">8</span>
                    <span>Preguntas Agronómicas, Conclusiones y Bitácora del Investigador</span>
                </div>
                <button type="button" class="btn-action btn-outline-action no-print" onclick="openAddQuestionModal()" style="color: var(--text-main); border-color: var(--border-color); padding: 4px 8px; font-size: 11px;">
                    <span class="material-icons-round" style="font-size: 14px; color: #0f172a;">add_circle</span>
                    <span>+ Añadir Pregunta</span>
                </button>
            </div>

            <!-- Contenedor dinámico de preguntas -->
```

```
            <div id="questions-container">
                <!-- Pregunta 1 Predeterminada -->
                <div class="question-card" id="qcard-1">
                    <div class="question-header">
                        <div class="question-title" id="qtitle-1" contenteditable="true">
                            1. ¿El desarrollo vegetativo (área foliar y elongación) se ajustó a la curva teórica del cultivo?
                        </div>
                        <div class="question-tools-screen no-print">
                            <button type="button" class="question-btn-mini" onclick="removeQuestionCard('qcard-1')" title="Eliminar pregunta">
                                <span class="material-icons-round" style="font-size: 14px;">delete</span>
                            </button>
                        </div>
                    </div>
                    <textarea class="question-response-area" id="qresp-1" placeholder="Escribe aquí las observaciones sobre el crecimiento foliar, vigor y morfometría..." oninput="autoSaveQuestions()">El cultivo mantuvo una tasa de expansión foliar estable con un índice de salud superior al promedio de referencia, sin presentar clorosis ni anomalías visuales en el dosel.</textarea>
                </div>

                <!-- Pregunta 2 Predeterminada -->
                <div class="question-card" id="qcard-2">
                    <div class="question-header">
                        <div class="question-title" id="qtitle-2" contenteditable="true">
                            2. ¿Cómo influyeron las condiciones ambientales y de iluminación en la fisiología del lote?
                        </div>
                        <div class="question-tools-screen no-print">
                            <button type="button" class="question-btn-mini" onclick="removeQuestionCard('qcard-2')" title="Eliminar pregunta">
                                <span class="material-icons-round" style="font-size: 14px;">delete</span>
                            </button>
                        </div>
                    </div>
                    <textarea class="question-response-area" id="qresp-2" placeholder="Escribe aquí la respuesta agronómica sobre temperatura, humedad y radiación..." oninput="autoSaveQuestions()">El microclima se mantuvo dentro de la zona de confort térmico. Los niveles de radiación y fotoperiodo favorecieron una transpiración adecuada sin estrés por desecación.</textarea>
                </div>

                <!-- Pregunta 3 Predeterminada -->
                <div class="question-card" id="qcard-3">
                    <div class="question-header">
                        <div class="question-title" id="qtitle-3" contenteditable="true">
                            3. ¿La solución nutritiva (pH, EC, oxígeno) y el bombeo presentaron niveles óptimos?
                        </div>
                        <div class="question-tools-screen no-print">
                            <button type="button" class="question-btn-mini" onclick="removeQuestionCard('qcard-3')" title="Eliminar pregunta">
                                <span class="material-icons-round" style="font-size: 14px;">delete</span>
                            </button>
                        </div>
                    </div>
                    <textarea class="question-response-area" id="qresp-3" placeholder="Escribe aquí la respuesta sobre electroconductividad, pH y recirculación..." oninput="autoSaveQuestions()">La conductividad eléctrica y el pH se mantuvieron en rangos balanceados para la absorción de micronutrientes, asegurando una correcta oxigenación radical.</textarea>
                </div>
            </div>

            <!-- CAMPO LIBRE: BITÁCORA Y EVENTOS DE MANEJO DE CAMPO (SIN RAYITAS) -->
            <div class="question-card" id="card-field-notes">
                <div class="question-header">
                    <div class="question-title">
                        Bitácora de Campo y Eventos de Manejo (Notas del Investigador)
```

```
                    </div>
                </div>
                <textarea class="question-response-area" id="free-field-notes" placeholder="Registra eventos especiales del día (ej. '10:30 AM: Rotación de canastilla al nivel superior', '14:00 PM: Recarga de solución N-P-K', 'Ajuste de fotoperiodo a 16h')..." oninput="autoSaveQuestions()" style="min-height: 55px;">Sin incidencias operativas extraordinarias durante la jornada. Se verificó el funcionamiento regular de las boquillas de aspersión y el flujo de recirculación.</textarea>
            </div>

        </div>

        <!-- SECCIÓN 9: BLOQUE DE FIRMAS DE VALIDACIÓN CIENTÍFICA -->
        <div id="sec-signatures" class="signatures-row">
            <!-- FIRMA 1: INVESTIGADOR -->
            <div class="signature-line" id="sig-col-1">
                <div class="signature-box-area" id="sig-area-1">
                    <div id="sig-slot-empty-1" class="signature-placeholder-btn" onclick="openSignatureModal(1)">
                        <span class="material-icons-round" style="font-size: 16px;">draw</span>
                        <span>Clic para Estampar Firma del Investigador</span>
                    </div>
                    <div id="sig-slot-filled-1" style="display: none; width: 100%;">
                        <img id="sig-img-preview-1" class="signature-stamp-img" src="" alt="Firma Investigador">
                        <div class="signature-actions-hover no-print" style="margin-top: 4px;">
                            <button type="button" class="question-btn-mini" onclick="openSignatureModal(1)">
                                <span class="material-icons-round" style="font-size: 12px;">edit</span>
                                <span>Cambiar</span>
                            </button>
                            <button type="button" class="question-btn-mini" onclick="removeSignature(1)">
                                <span class="material-icons-round" style="font-size: 12px;">delete</span>
                                <span>Quitar</span>
                            </button>
                        </div>
                    </div>
                </div>
                <div class="signature-rule">
                    <strong id="sig-name-1" class="editable-field" contenteditable="true">{{ current_user.name if current_user else 'Ing. Andrés Luna' }}</strong>
                    <span id="sig-role-1" class="editable-field" contenteditable="true">Investigador Responsable / Adquisición Fenotípica</span>
                </div>
            </div>

            <!-- FIRMA 2: PROFESOR / VALIDADOR -->
            <div class="signature-line" id="sig-col-2">
                <div class="signature-box-area" id="sig-area-2">
                    <div id="sig-slot-empty-2" class="signature-placeholder-btn" onclick="openSignatureModal(2)">
                        <span class="material-icons-round" style="font-size: 16px;">draw</span>
                        <span>Clic para Estampar Firma del Evaluador</span>
                    </div>
                    <div id="sig-slot-filled-2" style="display: none; width: 100%;">
                        <img id="sig-img-preview-2" class="signature-stamp-img" src="" alt="Firma Validador">
                        <div class="signature-actions-hover no-print" style="margin-top: 4px;">
                            <button type="button" class="question-btn-mini" onclick="openSignatureModal(2)">
                                <span class="material-icons-round" style="font-size: 12px;">edit</span>
                                <span>Cambiar</span>
                            </button>
                            <button type="button" class="question-btn-mini" onclick="removeSignature(2)">
                                <span class="material-icons-round" style="font-size: 12px;">delete</span>
                                <span>Quitar</span>
                            </button>
```

```
                        </div>
                    </div>
                </div>
                <div class="signature-rule">
                    <strong id="sig-name-2" class="editable-field" contenteditable="true">Prof. Críspulo Enrique Deluque</strong>
                    <span id="sig-role-2" class="editable-field" contenteditable="true">Profesor Asesor & Director de Investigación</span>
                </div>
            </div>
        </div>

    </div>

    <!-- MODAL PARA AGREGAR NUEVA PREGUNTA PERSONALIZADA -->
    <div id="add-question-modal" class="sig-modal-backdrop" onclick="handleQuestionModalBackdrop(event)">
        <div class="sig-modal-card">
            <div class="sig-modal-header">
                <h3 style="font-size: 13px; display: flex; align-items: center; gap: 6px;">
                    <span class="material-icons-round">help_outline</span>
                    <span>Añadir Pregunta / Observación al Reporte</span>
                </h3>
                <button type="button" style="background: transparent; border: none; color: #ffffff; cursor: pointer;" onclick="closeAddQuestionModal()">
                    <span class="material-icons-round">close</span>
                </button>
            </div>
            <div class="sig-modal-body">
                <div style="margin-bottom: 12px;">
                    <label style="font-size: 11.5px; font-weight: 700; color: var(--text-main); display: block; margin-bottom: 4px;">
                        Título o Enunciado de la Pregunta:
                    </label>
                    <input type="text" id="new-q-title" placeholder="Ej. ¿Qué efecto tuvo el cambio de nutrientes en el tallo?" style="width: 100%; padding: 8px 10px;
border-radius: 4px; border: 1px solid #cbd5e1; font-size: 12px;">
                </div>
                <div style="margin-bottom: 16px;">
                    <label style="font-size: 11.5px; font-weight: 700; color: var(--text-main); display: block; margin-bottom: 4px;">
                        Respuesta / Observación Inicial:
                    </label>
                    <textarea id="new-q-response" placeholder="Redacta la conclusión o respuesta técnica aquí..." style="width: 100%; padding: 8px 10px; border-radius:
4px; border: 1px solid #cbd5e1; font-size: 12px; min-height: 70px; resize: vertical;"></textarea>
                </div>
                <div style="display: flex; justify-content: flex-end; gap: 8px;">
                    <button type="button" class="btn-action btn-outline-action" style="color: #0f172a; border-color: #cbd5e1;"
onclick="closeAddQuestionModal()">Cancelar</button>
                    <button type="button" class="btn-action btn-primary-action" onclick="saveNewCustomQuestion()">
                        <span class="material-icons-round" style="font-size: 15px;">add</span>
                        <span>Insertar en Reporte</span>
                    </button>
                </div>
            </div>
        </div>
    </div>

    <!-- MODAL DEL FIRMADOR DIGITAL DE DOCUMENTOS -->
    <div id="signature-modal" class="sig-modal-backdrop" onclick="handleBackdropClick(event)">
        <div class="sig-modal-card">
            <div class="sig-modal-header">
                <h3 style="font-size: 13px; display: flex; align-items: center; gap: 6px;">
```

```
                    <span class="material-icons-round">edit_document</span>
                    <span>Firmador Digital de Reporte Cientifico</span>
                </h3>
                <button type="button" style="background: transparent; border: none; color: #ffffff; cursor: pointer;" onclick="closeSignatureModal()">
                    <span class="material-icons-round">close</span>
                </button>
            </div>

            <div class="sig-modal-body">
                <div style="margin-bottom: 16px;">
                    <label style="font-size: 11.5px; font-weight: 700; color: var(--text-main); display: block; margin-bottom: 4px;">
                        Estampar firma en:
                    </label>
                    <select id="modal-target-signee" style="width: 100%; padding: 7px 10px; border-radius: 4px; border: 1px solid var(--border-color); font-size: 12px;
font-weight: 600;">
                        <option value="1">Firmante 1: Investigador Responsable</option>
                        <option value="2">Firmante 2: Asesor Científico / Evaluador</option>
                    </select>
                </div>

                <div class="sig-tabs-nav">
                    <button type="button" class="sig-tab-btn active" id="tab-btn-upload" onclick="switchSigTab('upload')">
                        <span class="material-icons-round" style="font-size: 16px;">upload_file</span>
                        <span>Subir Imagen de Firma</span>
                    </button>
                    <button type="button" class="sig-tab-btn" id="tab-btn-draw" onclick="switchSigTab('draw')">
                        <span class="material-icons-round" style="font-size: 16px;">gesture</span>
                        <span>Dibujar Firma Digital</span>
                    </button>
                </div>

                <!-- Pestaña 1: Subir Firma -->
                <div id="sig-tab-upload" class="sig-tab-content active">
                    <input type="file" id="sig-file-input" accept="image/png, image/jpeg, image/webp" style="display: none;" onchange="handleSignatureFile(event)">
                    <div class="sig-dropzone" onclick="document.getElementById('sig-file-input').click()">
                        <span class="material-icons-round" style="font-size: 36px; color: #0f172a; margin-bottom: 6px;">cloud_upload</span>
                        <div style="font-weight: 700; font-size: 13px; color: var(--text-main);">
                            Haz clic o arrastra aquí la imagen de tu firma
                        </div>
                        <div style="font-size: 11px; color: var(--text-muted); margin-top: 4px;">
                            Formatos: PNG transparente, JPG o WEBP
                        </div>
                    </div>

                    <div style="margin-top: 12px; display: flex; align-items: center; gap: 8px;">
                        <input type="checkbox" id="chk-remove-white-bg" checked style="accent-color: #0f172a; width: 14px; height: 14px; cursor: pointer;">
                        <label for="chk-remove-white-bg" style="font-size: 11.5px; color: var(--text-main); cursor: pointer;">
                            <strong>Remover fondo blanco automáticamente</strong> (ideal para firmas en papel)
                        </label>
                    </div>
                </div>

                <!-- Pestaña 2: Dibujar Firma -->
                <div id="sig-tab-draw" class="sig-tab-content">
                    <div class="sig-canvas-wrapper">
                        <canvas id="signature-pad"></canvas>
```

```
                    </div>
                    <div style="display: flex; justify-content: space-between; align-items: center; margin-top: 8px;">
                        <span style="font-size: 11px; color: var(--text-muted);">Usa el cursor o tu dedo en pantalla táctil</span>
                        <button type="button" onclick="clearSignatureCanvas()" style="background: transparent; border: 1px solid #cbd5e1; padding: 3px 8px; border-radius: 4px; font-size: 11px; cursor: pointer; color: #64748b;">
                            <span class="material-icons-round" style="font-size: 12px; vertical-align: middle;">refresh</span>
                            <span>Limpiar Lienzo</span>
                        </button>
                    </div>
                </div>

                <!-- Preview de firma -->
                <div id="sig-crop-preview-box" style="display: none; margin-top: 14px; background: #f8fafc; border: 1px solid var(--border-color); border-radius: 4px; padding: 10px; text-align: center;">
                    <div style="font-size: 10.5px; font-weight: 700; color: var(--text-muted); margin-bottom: 6px;">
                        VISTA PREVIA DEL SELLO DIGITAL
                    </div>
                    <img id="sig-modal-preview-img" src="" style="max-height: 70px; max-width: 100%; object-fit: contain; filter: drop-shadow(0 1px 2px rgba(0,0,0,0.2));">
                </div>

                <div style="margin-top: 14px; display: flex; align-items: center; gap: 8px;">
                    <input type="checkbox" id="chk-remember-sig" checked style="accent-color: #0f172a; width: 14px; height: 14px; cursor: pointer;">
                    <label for="chk-remember-sig" style="font-size: 11px; color: var(--text-muted); cursor: pointer;">
                        Recordar esta firma digital en este navegador para futuros reportes
                    </label>
                </div>

                <div style="display: flex; justify-content: flex-end; gap: 8px; margin-top: 18px; padding-top: 12px; border-top: 1px solid var(--border-color);">
                    <button type="button" class="btn-action btn-outline-action" style="color: var(--text-main); border-color: var(--border-color);" onclick="closeSignatureModal()">
                        <span>Cancelar</span>
                    </button>
                    <button type="button" class="btn-action btn-primary-action" onclick="applySignatureFromModal()">
                        <span class="material-icons-round" style="font-size: 15px;">check_circle</span>
                        <span>Estampar Firma</span>
                    </button>
                </div>
            </div>
        </div>
    </div>

    <!-- SCRIPTS DINÁMICOS DEL DOSSIER -->
    <script>
        const DOSSIER_DATA = {{ dossier | tojson | safe }};
        let currentTargetSignee = 1;
        let activeTab = 'upload';
        let isDrawing = false;
        let penColor = '#0f172a';
        let canvas, ctx;
        let stagedSignatureDataUrl = null;
        let customQuestionCounter = 10;

        document.addEventListener('DOMContentLoaded', () => {
            initSignatureCanvas();
            loadSavedSignatures();
            initReportCharts();
```

```
            applyPhotoDisplayFilter('last');
            loadSavedQuestions();

            // Sincronizar nombre editable con firma 1
            const metaName = document.getElementById('meta-evaluator-name');
            const sigName1 = document.getElementById('sig-name-1');
            if (metaName && sigName1) {
                metaName.addEventListener('input', () => {
                    sigName1.innerText = metaName.innerText;
                });
                sigName1.addEventListener('input', () => {
                    metaName.innerText = sigName1.innerText;
                });
            }
        });

        function toggleConfigDrawer() {
            const drawer = document.getElementById('config-drawer');
            if (drawer) {
                drawer.classList.toggle('collapsed');
            }
        }

        function setAllCheckboxes(val) {
            document.querySelectorAll('.checkbox-grid input[type="checkbox"]').forEach(chk => {
                chk.checked = val;
                chk.dispatchEvent(new Event('change'));
            });
        }

        function toggleSection(sectionId, isVisible) {
            const sec = document.getElementById(sectionId);
            if (sec) {
                sec.style.display = isVisible ? 'block' : 'none';
            }
        }

        function toggleChartCard(cardId, isVisible) {
            const card = document.getElementById(cardId);
            if (card) {
                card.style.display = isVisible ? 'block' : 'none';
            }
        }

        function applyPhotoDisplayFilter(filterType) {
            const container = document.getElementById('sec-photos');
            const cards = document.querySelectorAll('.photo-comp-card');

            if (filterType === 'none') {
                if (container) container.style.display = 'none';
                return;
            } else {
                if (container) container.style.display = 'block';
            }
```

```
            cards.forEach(card => {
                const isLast = card.dataset.isLast === 'true';
                const timeStr = card.dataset.time || '';
                const hour = parseInt(timeStr.split(':')[0] || '0', 10);

                if (filterType === 'all') {
                    card.style.display = 'block';
                } else if (filterType === 'last') {
                    card.style.display = isLast ? 'block' : 'none';
                } else if (filterType === 'morning') {
                    card.style.display = (hour >= 6 && hour < 12) ? 'block' : 'none';
                } else if (filterType === 'afternoon') {
                    card.style.display = (hour >= 12 && hour <= 18) ? 'block' : 'none';
                }
            });
        }

        function applyPhotoViewFilter(viewType) {
            const cenitals = document.querySelectorAll('.photo-pair-cenital');
            const laterals = document.querySelectorAll('.photo-pair-lateral');

            cenitals.forEach(c => {
                c.style.display = (viewType === 'both' || viewType === 'cenital') ? 'block' : 'none';
            });
            laterals.forEach(l => {
                l.style.display = (viewType === 'both' || viewType === 'lateral') ? 'block' : 'none';
            });
        }

        function initReportCharts() {
            if (!DOSSIER_DATA || !DOSSIER_DATA.chart_series) return;
            const cs = DOSSIER_DATA.chart_series;
            const sensorLabels = cs.sensors.timestamps || [];
            const visionLabels = cs.vision.periods || cs.vision.timestamps || [];

            function makeSparkChart(canvasId, labels, data, color, fill = true) {
                const el = document.getElementById(canvasId);
                if (!el) return;
                const ctx2d = el.getContext('2d');
                new Chart(ctx2d, {
                    type: 'line',
                    data: {
                        labels: labels.length ? labels : ['00:00', '06:00', '12:00', '18:00'],
                        datasets: [{
                            data: data.length ? data : [0, 0, 0, 0],
                            borderColor: color,
                            backgroundColor: fill ? color.replace(')', ', 0.08)').replace('rgb', 'rgba') : 'transparent',
                            borderWidth: 1.8,
                            pointRadius: data.length <= 10 ? 2.5 : 1,
                            pointBackgroundColor: color,
                            fill: fill,
                            tension: 0.25
                        }]
                    },
                    options: {
```

```
                    responsive: true,
                    maintainAspectRatio: false,
                    plugins: { legend: { display: false } },
                    scales: {
                        x: { grid: { display: false }, ticks: { font: { size: 8 } } },
                        y: { grid: { color: '#f1f5f9' }, ticks: { font: { size: 8 } } }
                    }
                }
            });
        }

        // Sensores (8) - Colores sobrios y formales
        makeSparkChart('canvas-report-temp', sensorLabels, cs.sensors.temp, 'rgb(185, 28, 28)');
        makeSparkChart('canvas-report-hum', sensorLabels, cs.sensors.hum, 'rgb(3, 105, 161)');
        makeSparkChart('canvas-report-uv', sensorLabels, cs.sensors.uv, 'rgb(194, 65, 12)');
        makeSparkChart('canvas-report-curr', sensorLabels, cs.sensors.current, 'rgb(71, 85, 105)');
        makeSparkChart('canvas-report-ph', sensorLabels, cs.sensors.ph, 'rgb(109, 40, 217)');
        makeSparkChart('canvas-report-ec', sensorLabels, cs.sensors.ec, 'rgb(15, 118, 110)');
        makeSparkChart('canvas-report-turb', sensorLabels, cs.sensors.turb, 'rgb(161, 98, 7)');
        makeSparkChart('canvas-report-do', sensorLabels, cs.sensors.do, 'rgb(4, 120, 87)');

        // Visión / Cámara (4) - Colores sobrios
        makeSparkChart('canvas-report-area', visionLabels, cs.vision.foliar_area, 'rgb(4, 120, 87)');
        makeSparkChart('canvas-report-height', visionLabels, cs.vision.height, 'rgb(3, 105, 161)');
        makeSparkChart('canvas-report-stem', visionLabels, cs.vision.stem, 'rgb(120, 53, 15)');
        makeSparkChart('canvas-report-health', visionLabels, cs.vision.health, 'rgb(21, 128, 61)');
    }

    function removeQuestionCard(cardId) {
        const card = document.getElementById(cardId);
        if (card) {
            card.remove();
            autoSaveQuestions();
        }
    }

    function openAddQuestionModal() {
        document.getElementById('add-question-modal').style.display = 'flex';
        document.getElementById('new-q-title').value = '';
        document.getElementById('new-q-response').value = '';
    }

    function closeAddQuestionModal() {
        document.getElementById('add-question-modal').style.display = 'none';
    }

    function handleQuestionModalBackdrop(e) {
        if (e.target.id === 'add-question-modal') closeAddQuestionModal();
    }

    function saveNewCustomQuestion() {
        const title = document.getElementById('new-q-title').value.trim();
        const resp = document.getElementById('new-q-response').value.trim();
        if (!title) {
            alert('Ingresa el título de la pregunta.');
```

```
                return;
            }

            customQuestionCounter++;
            const qId = `qcard-${customQuestionCounter}`;
            const container = document.getElementById('questions-container');

            const cardHtml = `
                <div class="question-card" id="${qId}">
                    <div class="question-header">
                        <div class="question-title" id="qtitle-${customQuestionCounter}" contenteditable="true">
                            ${title}
                        </div>
                        <div class="question-tools-screen no-print">
                            <button type="button" class="question-btn-mini" onclick="removeQuestionCard('${qId}')" title="Eliminar pregunta">
                                <span class="material-icons-round" style="font-size: 14px;">delete</span>
                            </button>
                        </div>
                    </div>
                    <textarea class="question-response-area" id="qresp-${customQuestionCounter}" placeholder="Escribe tu respuesta aquí..."
oninput="autoSaveQuestions()">${resp}</textarea>
                </div>
            `;
            container.insertAdjacentHTML('beforeend', cardHtml);
            closeAddQuestionModal();
            autoSaveQuestions();
        }

        function autoSaveQuestions() {
            const key = `sifma_dossier_notes_${DOSSIER_DATA.plant_id}_${DOSSIER_DATA.target_date}`;
            const data = {
                free_notes: document.getElementById('free-field-notes')?.value || '',
                q1: document.getElementById('qresp-1')?.value || '',
                q2: document.getElementById('qresp-2')?.value || '',
                q3: document.getElementById('qresp-3')?.value || ''
            };
            localStorage.setItem(key, JSON.stringify(data));
        }

        function loadSavedQuestions() {
            const key = `sifma_dossier_notes_${DOSSIER_DATA.plant_id}_${DOSSIER_DATA.target_date}`;
            const raw = localStorage.getItem(key);
            if (!raw) return;
            try {
                const data = JSON.parse(raw);
                if (data.free_notes && document.getElementById('free-field-notes')) {
                    document.getElementById('free-field-notes').value = data.free_notes;
                }
                if (data.q1 && document.getElementById('qresp-1')) document.getElementById('qresp-1').value = data.q1;
                if (data.q2 && document.getElementById('qresp-2')) document.getElementById('qresp-2').value = data.q2;
                if (data.q3 && document.getElementById('qresp-3')) document.getElementById('qresp-3').value = data.q3;
            } catch (e) {}
        }

        function initSignatureCanvas() {
```

```
            canvas = document.getElementById('signature-pad');
            if (!canvas) return;
            ctx = canvas.getContext('2d');

            function resizeCanvas() {
                const rect = canvas.getBoundingClientRect();
                canvas.width = rect.width * 2;
                canvas.height = rect.height * 2;
                ctx.scale(2, 2);
                ctx.lineCap = 'round';
                ctx.lineJoin = 'round';
                ctx.lineWidth = 2.5;
                ctx.strokeStyle = penColor;
            }
            resizeCanvas();

            function getPos(e) {
                const rect = canvas.getBoundingClientRect();
                const clientX = e.touches ? e.touches[0].clientX : e.clientX;
                const clientY = e.touches ? e.touches[0].clientY : e.clientY;
                return { x: clientX - rect.left, y: clientY - rect.top };
            }

            function startDraw(e) {
                isDrawing = true;
                const pos = getPos(e);
                ctx.beginPath();
                ctx.moveTo(pos.x, pos.y);
                e.preventDefault();
            }

            function draw(e) {
                if (!isDrawing) return;
                const pos = getPos(e);
                ctx.lineTo(pos.x, pos.y);
                ctx.stroke();
                e.preventDefault();
            }

            function stopDraw() {
                if (isDrawing) {
                    isDrawing = false;
                    stagedSignatureDataUrl = canvas.toDataURL('image/png');
                }
            }

            canvas.addEventListener('mousedown', startDraw);
            canvas.addEventListener('mousemove', draw);
            window.addEventListener('mouseup', stopDraw);
            canvas.addEventListener('touchstart', startDraw, { passive: false });
            canvas.addEventListener('touchmove', draw, { passive: false });
            canvas.addEventListener('touchend', stopDraw);
        }

        function switchSigTab(tab) {
```

```
            activeTab = tab;
            document.getElementById('tab-btn-upload').classList.toggle('active', tab === 'upload');
            document.getElementById('tab-btn-draw').classList.toggle('active', tab === 'draw');
            document.getElementById('sig-tab-upload').classList.toggle('active', tab === 'upload');
            document.getElementById('sig-tab-draw').classList.toggle('active', tab === 'draw');

            if (tab === 'draw') {
                setTimeout(() => { initSignatureCanvas(); }, 50);
            }
        }

        function openSignatureModal(targetIndex = 1) {
            currentTargetSignee = targetIndex;
            document.getElementById('modal-target-signee').value = String(targetIndex);
            document.getElementById('signature-modal').style.display = 'flex';
            document.getElementById('sig-crop-preview-box').style.display = 'none';
            stagedSignatureDataUrl = null;
        }

        function closeSignatureModal() {
            document.getElementById('signature-modal').style.display = 'none';
            clearSignatureCanvas();
        }

        function clearSignatureCanvas() {
            if (!canvas || !ctx) return;
            ctx.clearRect(0, 0, canvas.width, canvas.height);
            stagedSignatureDataUrl = null;
            document.getElementById('sig-crop-preview-box').style.display = 'none';
        }

        function handleBackdropClick(e) {
            if (e.target.id === 'signature-modal') closeSignatureModal();
        }

        function handleSignatureFile(e) {
            const file = e.target.files[0];
            if (!file) return;

            const reader = new FileReader();
            reader.onload = (event) => {
                const img = new Image();
                img.onload = () => {
                    const removeWhite = document.getElementById('chk-remove-white-bg').checked;
                    if (removeWhite) {
                        const tempCanvas = document.createElement('canvas');
                        tempCanvas.width = img.width;
                        tempCanvas.height = img.height;
                        const tempCtx = tempCanvas.getContext('2d');
                        tempCtx.drawImage(img, 0, 0);
                        const imgData = tempCtx.getImageData(0, 0, tempCanvas.width, tempCanvas.height);
                        const data = imgData.data;

                        for (let i = 0; i < data.length; i += 4) {
                            const r = data[i];
```

```
                        const g = data[i+1];
                        const b = data[i+2];
                        if (r > 215 && g > 215 && b > 215) {
                            data[i+3] = 0;
                        }
                    }
                    tempCtx.putImageData(imgData, 0, 0);
                    stagedSignatureDataUrl = tempCanvas.toDataURL('image/png');
                } else {
                    stagedSignatureDataUrl = event.target.result;
                }

                const pImg = document.getElementById('sig-modal-preview-img');
                pImg.src = stagedSignatureDataUrl;
                document.getElementById('sig-crop-preview-box').style.display = 'block';
            };
            img.src = event.target.result;
        };
        reader.readAsDataURL(file);
    }

    function applySignatureFromModal() {
        const targetId = document.getElementById('modal-target-signee').value;
        let finalDataUrl = stagedSignatureDataUrl;

        if (activeTab === 'draw' && (!finalDataUrl || finalDataUrl.length < 100)) {
            finalDataUrl = canvas.toDataURL('image/png');
        }

        if (!finalDataUrl) {
            alert('Por favor dibuja o sube una imagen de tu firma antes de estampar.');
            return;
        }

        renderSignatureSlot(targetId, finalDataUrl);

        if (document.getElementById('chk-remember-sig').checked) {
            localStorage.setItem(`sifma_saved_signature_${targetId}`, finalDataUrl);
        }

        closeSignatureModal();
    }

    function renderSignatureSlot(targetId, dataUrl) {
        const emptySlot = document.getElementById(`sig-slot-empty-${targetId}`);
        const filledSlot = document.getElementById(`sig-slot-filled-${targetId}`);
        const previewImg = document.getElementById(`sig-img-preview-${targetId}`);

        if (emptySlot && filledSlot && previewImg) {
            previewImg.src = dataUrl;
            emptySlot.style.display = 'none';
            filledSlot.style.display = 'block';
        }
    }

```

```
        function removeSignature(targetId) {
            const emptySlot = document.getElementById(`sig-slot-empty-${targetId}`);
            const filledSlot = document.getElementById(`sig-slot-filled-${targetId}`);
            const previewImg = document.getElementById(`sig-img-preview-${targetId}`);

            if (emptySlot && filledSlot && previewImg) {
                previewImg.src = '';
                filledSlot.style.display = 'none';
                emptySlot.style.display = 'flex';
                localStorage.removeItem(`sifma_saved_signature_${targetId}`);
            }
        }

        function loadSavedSignatures() {
            const sig1 = localStorage.getItem('sifma_saved_signature_1');
            const sig2 = localStorage.getItem('sifma_saved_signature_2');
            if (sig1) renderSignatureSlot(1, sig1);
            if (sig2) renderSignatureSlot(2, sig2);
        }
    </script>

</body>
</html>
```

ARCHIVO 72/85: local_server/templates/sensors.html (991 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Detalle de Sensores{% endblock %}
{% block heading %}Análisis y Telemetría Detallada de Sensores{% endblock %}
{% block subheading %}Monitoreo ambiental en vivo, recepción inalámbrica XBee y registro continuo{% endblock %}

{% block header_actions %}
<a href="/api/telemetry/download_csv?date={{ sensor_data.target_date }}" class="btn btn-outline btn-sm" title="Descargar archivo CSV de telemetría pura de esta jornada">
    <span class="material-icons-round">download</span>
    <span>Descargar Telemetría CSV ({{ sensor_data.target_date }})</span>
</a>
<a href="/api/daily_summary/export_csv?date={{ sensor_data.target_date }}&plant_id={{ active_plant_id }}" class="btn btn-primary btn-sm" title="Descargar Resumen Consolidado del Día (CSV / Excel)">
    <span class="material-icons-round">table_chart</span>
    <span>Exportar Resumen Diario</span>
</a>
{% endblock %}

{% block content %}

<!-- SECCIÓN 1: PANEL DE CONTROL DE LA ANTENA USB Y SUBIDA DE CSV (RETRAÍBLE POR DEFECTO) -->
<div class="widget-card" style="margin-bottom: 14px; padding: 12px 18px;">
    <div class="widget-header" style="margin-bottom: 0; padding-bottom: 0; cursor: pointer;" onclick="toggleCollapsibleSection('section-antenna-body', 'icon-antenna-toggle', 'btn-antenna-text')">
        <div class="widget-header-title" style="display: flex; align-items: center; gap: 10px;">
            <div class="kpi-icon-badge blue" style="width: 32px; height: 32px;">
                <span class="material-icons-round" style="font-size: 18px;">settings_input_antenna</span>
            </div>
            <div>
                <h3 style="font-size: 13.5px; margin-bottom: 2px;">Control de la Antena Receptora de Telemetría (XBee PRO S2B / USB)</h3>
                <p style="font-size: 11.5px; margin: 0;">Recepción inalámbrica RF e importación manual de registros CSV de la torre</p>
            </div>
        </div>
        <div style="display: flex; align-items: center; gap: 10px;">
            <span class="status-dot {% if telemetry_status.is_recording %}active{% else %}inactive{% endif %}"></span>
            <span id="antenna-live-status-text" style="font-size: 11.5px; font-weight: 700; color: {% if telemetry_status.is_recording %}var(--accent-success){% else %}var(--text-muted){% endif %};">
                {% if telemetry_status.is_recording %}Transmitiendo en Vivo{% else %}Antena Inactiva{% endif %}
            </span>
            <button type="button" class="btn btn-outline btn-sm" style="padding: 4px 10px; font-size: 11.5px; display: flex; align-items: center; gap: 4px;">
                <span id="btn-antenna-text">Ver Antena / Importar CSV</span>
                <span class="material-icons-round" id="icon-antenna-toggle" style="font-size: 18px; transition: transform 0.2s;">expand_more</span>
            </button>
        </div>
    </div>

    <!-- CUERPO RETRAÍBLE (OCULTO POR DEFECTO) -->
    <div id="section-antenna-body" style="display: none; margin-top: 16px; border-top: 1px dashed var(--border-color); padding-top: 16px;">
```

```
        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(320px, 1fr)); gap: 16px;">

            <!-- Bloque 1: Captación Directa desde Antena USB -->
            <div style="background: var(--bg-card-subtle); padding: 14px; border-radius: 10px; border: 1px solid var(--border-color);">
                <h4 style="font-size: 13px; font-weight: 700; margin-bottom: 10px; color: var(--text-heading);">
                    Recepción Inalámbrica por Antena USB
                </h4>

                <div style="margin-bottom: 12px;">
                    <label style="font-size: 11.5px; font-weight: 600; color: var(--text-muted); display: block; margin-bottom: 4px;">
                        Puerto o Interfaz de la Antena:
                    </label>
                    <div style="display: flex; gap: 8px;">
                        <select id="antenna-port-select" class="form-control" style="flex-grow: 1; font-size: 12.5px;">
                            {% if available_ports %}
                                {% for p in available_ports %}
                                    <option value="{{ p }}" {% if p == telemetry_status.port %}selected{% endif %}>{{ p }}</option>
                                {% endfor %}
                            {% else %}
                                <option value="" disabled selected>No se detecta antena USB conectada</option>
                            {% endif %}
                        </select>
                        <button type="button" class="btn btn-outline btn-sm" onclick="refreshAntennaPorts()" title="Escanear puertos USB">
                            <span class="material-icons-round" style="font-size: 16px;">refresh</span>
                        </button>
                    </div>
                </div>

                <div style="margin-bottom: 12px;">
                    <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 6px;">
                        <div style="font-size: 11px; color: var(--text-muted);">
                            Archivo CSV activo: <code id="antenna-csv-file" style="color: var(--text-heading); font-weight: 600;">{{ telemetry_status.file }}</code>
                        </div>
                        <a href="/api/telemetry/download_active_csv" class="btn btn-outline btn-sm" style="padding: 2px 6px; font-size: 11px;" title="Descargar archivo CSV de telemetría">
                            <span class="material-icons-round" style="font-size: 13px; vertical-align: middle;">download</span>
                            <span>Descargar CSV Activo</span>
                        </a>
                    </div>
                </div>

                <div style="display: flex; gap: 8px;">
                    <button type="button" id="btn-start-telemetry" class="btn btn-primary btn-sm" onclick="toggleTelemetry(true)" {% if telemetry_status.is_recording %}style="display:none;"{% endif %}>
                        <span class="material-icons-round" style="font-size: 16px;">play_arrow</span>
                        <span>Iniciar Captación</span>
                    </button>
                    <button type="button" id="btn-stop-telemetry" class="btn btn-danger btn-sm" {% if not telemetry_status.is_recording %}style="display:none;"{% endif %} onclick="toggleTelemetry(false)">
                        <span class="material-icons-round" style="font-size: 16px;">stop</span>
                        <span>Detener Captación</span>
                    </button>
                </div>
            </div>

            <!-- Bloque 2: Importación Manual de Archivo CSV de la Torre -->
            <div style="background: var(--bg-card-subtle); padding: 14px; border-radius: 10px; border: 1px solid var(--border-color);">
```

```
                <h4 style="font-size: 13px; font-weight: 700; margin-bottom: 6px; color: var(--text-heading);">
                    Importar Archivo CSV de la Torre Hidropónica
                </h4>
                <p style="font-size: 11.5px; color: var(--text-muted); margin-bottom: 10px;">
                    Sube el archivo CSV exportado por la torre (Fecha;Temperatura;Humedad;Radiación;Amperaje) para registrarlo.
                </p>

                <form id="csv-upload-form" onsubmit="handleCsvUpload(event)">
                    <div style="margin-bottom: 10px;">
                        <input type="file" id="telemetry-csv-input" accept=".csv,text/csv" class="form-control" style="font-size: 12px;" required>
                    </div>
                    <button type="submit" id="btn-upload-csv" class="btn btn-primary btn-sm" style="width: 100%;">
                        <span class="material-icons-round" style="font-size: 16px;">upload_file</span>
                        <span>Cargar y Procesar CSV de Sensores</span>
                    </button>
                </form>

                <div id="csv-upload-feedback" style="display: none; margin-top: 8px; font-size: 11.5px; padding: 8px; border-radius: 6px;"></div>
            </div>
        </div>
    </div>
</div>

<!-- SECCIÓN 2: FILTRO DE TELEMETRÍA POR FECHA (RETRAÍBLE / COMPACTO POR DEFECTO) -->
<div class="widget-card" style="margin-bottom: 14px; padding: 12px 18px;">
    <div class="widget-header" style="margin-bottom: 0; padding-bottom: 0; cursor: pointer;" onclick="toggleCollapsibleSection('section-filter-body', 'icon-filter-toggle', 'btn-filter-text')">
        <div class="widget-header-title" style="display: flex; align-items: center; gap: 10px;">
            <div class="kpi-icon-badge cyan" style="width: 32px; height: 32px;">
                <span class="material-icons-round" style="font-size: 18px;">filter_alt</span>
            </div>
            <div>
                <h3 style="font-size: 13.5px; margin-bottom: 2px;">Filtro de Telemetría por Jornada de Muestreo</h3>
                <p style="font-size: 11.5px; margin: 0;">Jornada activa: <strong>{{ 'Todos los Días' if sensor_data.target_date == 'todos' else sensor_data.target_date }}</strong></p>
            </div>
        </div>
        <div style="display: flex; align-items: center; gap: 10px;">
            <span class="badge badge-primary" id="timeline-count-badge" style="font-size: 11px;">
                {{ sensor_data.get('records_count', 0) }} LECTURAS ({{ sensor_data.target_date }})
            </span>
            <button type="button" class="btn btn-outline btn-sm" style="padding: 4px 10px; font-size: 11.5px; display: flex; align-items: center; gap: 4px;">
                <span id="btn-filter-text">Cambiar Fecha / Filtrar</span>
                <span class="material-icons-round" id="icon-filter-toggle" style="font-size: 18px; transition: transform 0.2s;">expand_more</span>
            </button>
        </div>
    </div>

    <!-- CUERPO DE FILTRO RETRAÍBLE (OCULTO POR DEFECTO) -->
    <div id="section-filter-body" style="display: none; margin-top: 14px; border-top: 1px dashed var(--border-color); padding-top: 14px;">
        <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 12px;">
            <div class="filter-group">
                <button type="button" class="filter-btn {% if sensor_data.target_date == 'todos' %}active{% endif %}" onclick="filterByDate('todos', this)">
                    <span class="material-icons-round" style="font-size: 14px;">calendar_month</span>
                    <span>Todos los Días</span>
                </button>
```

```
                {% for d in available_dates %}
                    <button type="button" class="filter-btn {% if sensor_data.target_date == d %}active{% endif %}" onclick="filterByDate('{{ d }}', this)">
                        <span class="material-icons-round" style="font-size: 14px;">event</span>
                        <span>{{ d }}</span>
                    </button>
                {% endfor %}
            </div>

            <div style="display: flex; align-items: center; gap: 8px;">
                <label style="font-size: 11.5px; color: var(--text-muted); font-weight: 600;">Fecha específica:</label>
                <input type="date" id="custom-date-filter" class="form-control" style="width: auto; padding: 4px 8px; font-size: 12px;" value="{{ sensor_data.target_date if sensor_data.target_date != 'todos' else '' }}" onchange="filterByDate(this.value, null)">
                {% if sensor_data.get('records_count', 0) > 0 %}
                    <button type="button" class="btn btn-outline btn-sm" style="color: #dc2626; border-color: rgba(220,38,38,0.3); font-size: 11px; padding: 4px 8px;" onclick="deleteTelemetryForActiveDate()" title="Eliminar telemetría de esta fecha">
                        <span class="material-icons-round" style="font-size: 14px;">delete</span>
                        <span>Eliminar Datos</span>
                    </button>
                {% endif %}
            </div>
        </div>
    </div>
</div>

<!-- AVISO DE CANASTILLA SIN TELEMETRÍA EN MODO AISLADO -->
{% if sensor_data.get('records_count', 0) == 0 %}
<div class="zero-data-banner" style="margin-bottom: 14px; padding: 12px 16px;">
    <div class="zero-data-icon" style="width: 36px; height: 36px;">
        <span class="material-icons-round" style="font-size: 20px;">sensors_off</span>
    </div>
    <div class="zero-data-text">
        <h4 style="font-size: 13px;">Canastilla #{{ active_plant_id }} - Sin Telemetría Registrada</h4>
        <p style="font-size: 11.5px;">No se encontraron lecturas de sensores ambientales asociadas a la <strong>Canastilla #{{ active_plant_id }}</strong> para la fecha {{ sensor_data.target_date }}. En modo aislado, cada canastilla requiere sus propias lecturas o turnos de muestreo.</p>
    </div>
</div>
{% endif %}

<!-- SECCIÓN 3: KPI CARDS DE TELEMETRÍA (8 VARIABLES OFICIALES) -->
<div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 8px; margin-bottom: 10px;">
    <div style="font-size: 12px; font-weight: 700; color: var(--text-muted); text-transform: uppercase;">
        Panel de Indicadores en Tiempo Real (8 Variables de Telemetría)
    </div>
    <div class="filter-group">
        <button type="button" class="filter-btn active" id="btn-kpi-all" onclick="toggleKpiCategory('all', this)" style="padding: 3px 8px; font-size: 11px;">
            <span>Todas (8)</span>
        </button>
        <button type="button" class="filter-btn" id="btn-kpi-water" onclick="toggleKpiCategory('water', this)" style="padding: 3px 8px; font-size: 11px;">
            <span>Solución / Agua (5)</span>
        </button>
        <button type="button" class="filter-btn" id="btn-kpi-ambient" onclick="toggleKpiCategory('ambient', this)" style="padding: 3px 8px; font-size: 11px;">
            <span>Ambiente y Motor (3)</span>
        </button>
    </div>
</div>

```

```
<div class="kpi-row-grid" id="kpi-container-grid" style="display: grid; grid-template-columns: repeat(auto-fit, minmax(230px, 1fr)); gap: 12px; margin-bottom: 14px;">

    <!-- 1. pH Solución -->
    {% set ph_stat = sensor_data.summary.get('water_ph', {'last': 0.0, 'min': 0.0, 'max': 0.0, 'avg': 0.0, 'status': 'Sin datos'}) %}
    <div class="kpi-card kpi-card-water" style="padding: 12px 14px;">
        <div class="kpi-card-header" style="margin-bottom: 4px;">
            <span class="kpi-label" style="font-size: 11.5px;">1. pH Solución</span>
            <div class="kpi-icon-badge pink" style="width: 28px; height: 28px; background: rgba(236, 72, 153, 0.15); color: #ec4899;">
                <span class="material-icons-round" style="font-size: 16px;">science</span>
            </div>
        </div>
        <div class="kpi-value-row" style="margin-bottom: 3px;">
            <span class="kpi-big-number" id="kpi-ph-val" style="font-size: 22px;">{{ ph_stat.last }}</span>
            <span class="kpi-unit">pH</span>
        </div>
        <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-ph-sub">
            Mín: <strong id="kpi-ph-min">{{ ph_stat.min }}</strong> | Máx: <strong id="kpi-ph-max">{{ ph_stat.max }}</strong> | Prom: <strong id="kpi-ph-avg">{{ ph_stat.avg }}</strong>
        </div>
        <div class="kpi-footer-badge {% if 'Óptimo' in ph_stat.status %}positive{% elif 'Sin datos' in ph_stat.status %}neutral{% else %}warning{% endif %}" id="kpi-ph-badge" style="font-size: 10px; padding: 2px 6px;">
            <span class="material-icons-round" style="font-size: 12px;">{% if 'Óptimo' in ph_stat.status %}check_circle{% elif 'Sin datos' in ph_stat.status %}warning_amber{% else %}warning{% endif %}</span>
            <span id="kpi-ph-status">{{ ph_stat.status }}</span>
        </div>
    </div>

    <!-- 2. Conductividad Eléctrica (EC) -->
    {% set ec_stat = sensor_data.summary.get('ec_conductivity', {'last': 0.0, 'min': 0.0, 'max': 0.0, 'avg': 0.0, 'status': 'Sin datos'}) %}
    <div class="kpi-card kpi-card-water" style="padding: 12px 14px;">
        <div class="kpi-card-header" style="margin-bottom: 4px;">
            <span class="kpi-label" style="font-size: 11.5px;">2. Conductividad Eléctrica</span>
            <div class="kpi-icon-badge purple" style="width: 28px; height: 28px; background: rgba(139, 92, 246, 0.15); color: #8b5cf6;">
                <span class="material-icons-round" style="font-size: 16px;">bolt</span>
            </div>
        </div>
        <div class="kpi-value-row" style="margin-bottom: 3px;">
            <span class="kpi-big-number" id="kpi-ec-val" style="font-size: 22px;">{{ ec_stat.last }}</span>
            <span class="kpi-unit">µS/cm</span>
        </div>
        <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-ec-sub">
            Mín: <strong id="kpi-ec-min">{{ ec_stat.min }}</strong> | Máx: <strong id="kpi-ec-max">{{ ec_stat.max }}</strong> | Prom: <strong id="kpi-ec-avg">{{ ec_stat.avg }}</strong>
        </div>
        <div class="kpi-footer-badge {% if 'Óptimo' in ec_stat.status or 'Óptimos' in ec_stat.status %}positive{% elif 'Sin datos' in ec_stat.status %}neutral{% else %}warning{% endif %}" id="kpi-ec-badge" style="font-size: 10px; padding: 2px 6px;">
            <span class="material-icons-round" style="font-size: 12px;">{% if 'Óptimo' in ec_stat.status or 'Óptimos' in ec_stat.status %}check_circle{% elif 'Sin datos' in ec_stat.status %}warning_amber{% else %}warning{% endif %}</span>
            <span id="kpi-ec-status">{{ ec_stat.status }}</span>
        </div>
    </div>

    <!-- 3. Turbidez (NTU) -->
    {% set turb_stat = sensor_data.summary.get('turbidity_ntu', {'last': 0.0, 'min': 0.0, 'max': 0.0, 'avg': 0.0, 'status': 'Sin datos'}) %}
    <div class="kpi-card kpi-card-water" style="padding: 12px 14px;">
        <div class="kpi-card-header" style="margin-bottom: 4px;">
            <span class="kpi-label" style="font-size: 11.5px;">3. Turbidez</span>
            <div class="kpi-icon-badge amber" style="width: 28px; height: 28px; background: rgba(249, 115, 22, 0.15); color: #f97316;">
```

```
                <span class="material-icons-round" style="font-size: 16px;">opacity</span>
            </div>
        </div>
        <div class="kpi-value-row" style="margin-bottom: 3px;">
            <span class="kpi-big-number" id="kpi-turb-val" style="font-size: 22px;">{{ turb_stat.last }}</span>
            <span class="kpi-unit">NTU</span>
        </div>
        <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-turb-sub">
            Mín: <strong id="kpi-turb-min">{{ turb_stat.min }}</strong> | Máx: <strong id="kpi-turb-max">{{ turb_stat.max }}</strong> | Prom: <strong id="kpi-turb-avg">
{{ turb_stat.avg }}</strong>
        </div>
        <div class="kpi-footer-badge {% if 'Cristalina' in turb_stat.status %}positive{% elif 'Sin datos' in turb_stat.status %}neutral{% else %}warning{% endif %}"
id="kpi-turb-badge" style="font-size: 10px; padding: 2px 6px;">
            <span class="material-icons-round" style="font-size: 12px;">{% if 'Cristalina' in turb_stat.status %}check_circle{% elif 'Sin datos' in turb_stat.status
%}warning_amber{% else %}warning{% endif %}</span>
            <span id="kpi-turb-status">{{ turb_stat.status }}</span>
        </div>
    </div>

    <!-- 4. Oxígeno Disuelto (mg/L) -->
    {% set do_stat = sensor_data.summary.get('dissolved_oxygen_mg_l', {'last': 0.0, 'min': 0.0, 'max': 0.0, 'avg': 0.0, 'status': 'Sin datos'}) %}
    <div class="kpi-card kpi-card-water" style="padding: 12px 14px;">
        <div class="kpi-card-header" style="margin-bottom: 4px;">
            <span class="kpi-label" style="font-size: 11.5px;">4. Oxígeno Disuelto</span>
            <div class="kpi-icon-badge cyan" style="width: 28px; height: 28px; background: rgba(6, 182, 212, 0.15); color: #06b6d4;">
                <span class="material-icons-round" style="font-size: 16px;">air</span>
            </div>
        </div>
        <div class="kpi-value-row" style="margin-bottom: 3px;">
            <span class="kpi-big-number" id="kpi-do-val" style="font-size: 22px;">{{ do_stat.last }}</span>
            <span class="kpi-unit">mg/L</span>
        </div>
        <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-do-sub">
            Mín: <strong id="kpi-do-min">{{ do_stat.min }}</strong> | Máx: <strong id="kpi-do-max">{{ do_stat.max }}</strong> | Prom: <strong id="kpi-do-avg">{{
do_stat.avg }}</strong>
        </div>
        <div class="kpi-footer-badge {% if 'Óptima' in do_stat.status %}positive{% elif 'Sin datos' in do_stat.status %}neutral{% else %}warning{% endif %}" id="kpi-do-
badge" style="font-size: 10px; padding: 2px 6px;">
            <span class="material-icons-round" style="font-size: 12px;">{% if 'Óptima' in do_stat.status %}check_circle{% elif 'Sin datos' in do_stat.status
%}warning_amber{% else %}warning{% endif %}</span>
            <span id="kpi-do-status">{{ do_stat.status }}</span>
        </div>
    </div>

    <!-- 5. Temperatura de Agua (°C) -->
    {% set wt_stat = sensor_data.summary.get('water_temp', {'last': 0.0, 'min': 0.0, 'max': 0.0, 'avg': 0.0, 'status': 'Sin datos'}) %}
    <div class="kpi-card kpi-card-water" style="padding: 12px 14px;">
        <div class="kpi-card-header" style="margin-bottom: 4px;">
            <span class="kpi-label" style="font-size: 11.5px;">5. Temperatura Agua</span>
            <div class="kpi-icon-badge blue" style="width: 28px; height: 28px; background: rgba(37, 99, 235, 0.15); color: #2563eb;">
                <span class="material-icons-round" style="font-size: 16px;">water</span>
            </div>
        </div>
        <div class="kpi-value-row" style="margin-bottom: 3px;">
            <span class="kpi-big-number" id="kpi-watertemp-val" style="font-size: 22px;">{{ wt_stat.last }}</span>
            <span class="kpi-unit">°C</span>
        </div>
        <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-watertemp-sub">
```

```
317             Mín: <strong id="kpi-watertemp-min">{{ wt_stat.min }}°C</strong> | Máx: <strong id="kpi-watertemp-max">{{ wt_stat.max }}°C</strong> | Prom: <strong id="kpi-watertemp-avg">{{ wt_stat.avg }}°C</strong>
318         </div>
319         <div class="kpi-footer-badge {% if 'Óptima' in wt_stat.status %}positive{% elif 'Sin datos' in wt_stat.status %}neutral{% else %}warning{% endif %}" id="kpi-watertemp-badge" style="font-size: 10px; padding: 2px 6px;">
320             <span class="material-icons-round" style="font-size: 12px;">{% if 'Óptima' in wt_stat.status %}check_circle{% elif 'Sin datos' in wt_stat.status %}warning_amber{% else %}warning{% endif %}</span>
321             <span id="kpi-watertemp-status">{{ wt_stat.status }}</span>
322         </div>
323     </div>
324
325     <!-- 6. Estado Motor (1: Encendido / 0: Apagado) -->
326     {% set mot_stat = sensor_data.summary.get('motor_state', {'last': 0, 'min': 0, 'max': 0, 'avg': 0.0, 'status': 'Sin datos'}) %}
327     <div class="kpi-card kpi-card-ambient" style="padding: 12px 14px;">
328         <div class="kpi-card-header" style="margin-bottom: 4px;">
329             <span class="kpi-label" style="font-size: 11.5px;">6. Estado Motor</span>
330             <div class="kpi-icon-badge emerald" style="width: 28px; height: 28px;">
331                 <span class="material-icons-round" style="font-size: 16px;">power</span>
332             </div>
333         </div>
334         <div class="kpi-value-row" style="margin-bottom: 3px;">
335             <span class="kpi-big-number" id="kpi-motor-val" style="font-size: 22px;">{{ mot_stat.last }}</span>
336             <span class="kpi-unit" style="font-size: 11px;">[1/0]</span>
337         </div>
338         <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-motor-sub">
339             Actividad: <strong id="kpi-motor-avg">{{ (mot_stat.avg * 100)|round(0) }}% del tiempo</strong>
340         </div>
341         <div class="kpi-footer-badge {% if mot_stat.last == 1 %}positive{% elif 'Sin datos' in mot_stat.status %}neutral{% else %}neutral{% endif %}" id="kpi-motor-badge" style="font-size: 10px; padding: 2px 6px;">
342             <span class="material-icons-round" style="font-size: 12px;">{% if mot_stat.last == 1 %}check_circle{% elif 'Sin datos' in mot_stat.status %}warning_amber{% else %}radio_button_unchecked{% endif %}</span>
343             <span id="kpi-motor-status">{{ mot_stat.status }}</span>
344         </div>
345     </div>
346
347     <!-- 7. Temperatura Exterior (°C) -->
348     {% set t_stat = sensor_data.summary.get('temperature', {'last': 0.0, 'min': 0.0, 'max': 0.0, 'avg': 0.0, 'status': 'Sin datos'}) %}
349     <div class="kpi-card kpi-card-ambient" style="padding: 12px 14px;">
350         <div class="kpi-card-header" style="margin-bottom: 4px;">
351             <span class="kpi-label" style="font-size: 11.5px;">7. Temp Exterior</span>
352             <div class="kpi-icon-badge red" style="width: 28px; height: 28px; background: rgba(239, 68, 68, 0.15); color: #ef4444;">
353                 <span class="material-icons-round" style="font-size: 16px;">thermostat</span>
354             </div>
355         </div>
356         <div class="kpi-value-row" style="margin-bottom: 3px;">
357             <span class="kpi-big-number" id="kpi-temp-val" style="font-size: 22px;">{{ t_stat.last }}</span>
358             <span class="kpi-unit">°C</span>
359         </div>
360         <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-temp-sub">
361             Mín: <strong id="kpi-temp-min">{{ t_stat.min }}°C</strong> | Máx: <strong id="kpi-temp-max">{{ t_stat.max }}°C</strong> | Prom: <strong id="kpi-temp-avg">{{ t_stat.avg }}°C</strong>
362         </div>
363         <div class="kpi-footer-badge {% if 'Óptimo' in t_stat.status %}positive{% elif 'Sin datos' in t_stat.status %}neutral{% else %}warning{% endif %}" id="kpi-temp-badge" style="font-size: 10px; padding: 2px 6px;">
364             <span class="material-icons-round" style="font-size: 12px;">{% if 'Óptimo' in t_stat.status %}check_circle{% elif 'Sin datos' in t_stat.status %}warning_amber{% else %}warning{% endif %}</span>
365             <span id="kpi-temp-status">{{ t_stat.status }}</span>
366         </div>
367     </div>
```

```
368
369     <!-- 8. Humedad Relativa Exterior (% HR) -->
370     {% set h_stat = sensor_data.summary.get('humidity', {'last': 0.0, 'min': 0.0, 'max': 0.0, 'avg': 0.0, 'status': 'Sin datos'}) %}
371     <div class="kpi-card kpi-card-ambient" style="padding: 12px 14px;">
372         <div class="kpi-card-header" style="margin-bottom: 4px;">
373             <span class="kpi-label" style="font-size: 11.5px;">8. Humedad Rel. Exterior</span>
374             <div class="kpi-icon-badge cyan" style="width: 28px; height: 28px;">
375                 <span class="material-icons-round" style="font-size: 16px;">water_drop</span>
376             </div>
377         </div>
378         <div class="kpi-value-row" style="margin-bottom: 3px;">
379             <span class="kpi-big-number" id="kpi-hum-val" style="font-size: 22px;">{{ h_stat.last }}</span>
380             <span class="kpi-unit">% HR</span>
381         </div>
382         <div style="font-size: 10.5px; color: var(--text-muted); margin-bottom: 4px;" id="kpi-hum-sub">
383             Mín: <strong id="kpi-hum-min">{{ h_stat.min }}%</strong> | Máx: <strong id="kpi-hum-max">{{ h_stat.max }}%</strong> | Prom: <strong id="kpi-hum-avg">{{
    h_stat.avg }}%</strong>
384         </div>
385         <div class="kpi-footer-badge {% if 'Óptimo' in h_stat.status %}positive{% elif 'Sin datos' in h_stat.status %}neutral{% else %}warning{% endif %}" id="kpi-hum-
    badge" style="font-size: 10px; padding: 2px 6px;">
386             <span class="material-icons-round" style="font-size: 12px;">{% if 'Óptimo' in h_stat.status %}check_circle{% elif 'Sin datos' in h_stat.status
    %}warning_amber{% else %}warning{% endif %}</span>
387             <span id="kpi-hum-status">{{ h_stat.status }}</span>
388         </div>
389     </div>
390 </div>
391
392 <!-- SECCIÓN 4: BARRA DE CONTROL DE VENTANA DESLIZANTE Y GRUPO DE GRÁFICAS -->
393 <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap; gap: 10px; margin-bottom: 12px; background: #ffffff; padding: 8px 14px;
    border-radius: 10px; border: 1px solid var(--border-color);">
394     <div style="display: flex; align-items: center; gap: 8px;">
395         <span class="material-icons-round text-muted" style="font-size: 18px;">tune</span>
396         <span style="font-weight: 700; font-size: 12px; color: var(--text-heading);">Vista de Gráficas:</span>
397         <div class="filter-group">
398             <button type="button" class="filter-btn active" id="btn-group-all" onclick="setChartGroup('all', this)" style="padding: 3px 10px; font-size: 11.5px;">
399                 <span class="material-icons-round" style="font-size: 14px;">grid_view</span>
400                 <span>Ver los 8 Parámetros</span>
401             </button>
402             <button type="button" class="filter-btn" id="btn-group-water" onclick="setChartGroup('water', this)" style="padding: 3px 10px; font-size: 11.5px;">
403                 <span class="material-icons-round" style="font-size: 14px;">water</span>
404                 <span>Calidad de Agua (5)</span>
405             </button>
406             <button type="button" class="filter-btn" id="btn-group-ambient" onclick="setChartGroup('ambient', this)" style="padding: 3px 10px; font-size: 11.5px;">
407                 <span class="material-icons-round" style="font-size: 14px;">thermostat</span>
408                 <span>Exterior y Motor (3)</span>
409             </button>
410         </div>
411     </div>
412
413     <div style="display: flex; align-items: center; gap: 6px;">
414         <span style="font-weight: 600; font-size: 11.5px; color: var(--text-muted);">Muestras:</span>
415         <div class="filter-group">
416             <button type="button" class="filter-btn active" id="btn-win-30" onclick="setWindowSize(30, this)" style="padding: 3px 8px; font-size: 11px;">
417                 <span>30 (En vivo)</span>
418             </button>
419             <button type="button" class="filter-btn" id="btn-win-60" onclick="setWindowSize(60, this)" style="padding: 3px 8px; font-size: 11px;">
420                 <span>60</span>
```

```
            </button>
            <button type="button" class="filter-btn" id="btn-win-120" onclick="setWindowSize(120, this)" style="padding: 3px 8px; font-size: 11px;">
                <span>120</span>
            </button>
            <button type="button" class="filter-btn" id="btn-win-all" onclick="setWindowSize('all', this)" style="padding: 3px 8px; font-size: 11px;">
                <span>Todo</span>
            </button>
        </div>
    </div>
</div>

<!-- SECCIÓN 5: GRÁFICAS TEMPORALES (CUADRÍCULA CON LOS 8 PARÁMETROS EN EXACTO ORDEN) -->
<div id="all-charts-group" class="analytics-grid-2x2" style="display: grid; grid-template-columns: repeat(2, 1fr); gap: 12px; margin-bottom: 12px;">

    <!-- 1. pH Solución -->
    <div class="widget-card chart-card-water" style="padding: 10px 12px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
            <div class="widget-header-title">
                <h3 style="font-size: 12.5px;">1. pH Solución</h3>
                <p style="font-size: 10.5px;">Balance ácido-base de absorción radicular</p>
            </div>
            <span class="badge" style="background: rgba(236, 72, 153, 0.15); color: #ec4899; font-size: 9.5px; font-weight: 700;">pH</span>
        </div>
        <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
            <canvas id="phTimelineChart"></canvas>
        </div>
    </div>

    <!-- 2. Conductividad Eléctrica (EC) -->
    <div class="widget-card chart-card-water" style="padding: 10px 12px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
            <div class="widget-header-title">
                <h3 style="font-size: 12.5px;">2. Conductividad Eléctrica (µS/cm)</h3>
                <p style="font-size: 10.5px;">Concentración de sales disueltas y fertilización</p>
            </div>
            <span class="badge" style="background: rgba(139, 92, 246, 0.15); color: #8b5cf6; font-size: 9.5px; font-weight: 700;">EC µS/cm</span>
        </div>
        <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
            <canvas id="ecTimelineChart"></canvas>
        </div>
    </div>

    <!-- 3. Turbidez (NTU) -->
    <div class="widget-card chart-card-water" style="padding: 10px 12px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
            <div class="widget-header-title">
                <h3 style="font-size: 12.5px;">3. Turbidez (NTU)</h3>
                <p style="font-size: 10.5px;">Claridad y partículas en suspensión</p>
            </div>
            <span class="badge" style="background: rgba(249, 115, 22, 0.15); color: #f97316; font-size: 9.5px; font-weight: 700;">Turbidez</span>
        </div>
        <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
            <canvas id="turbTimelineChart"></canvas>
        </div>
    </div>
```

```
476
477     <!-- 4. Oxígeno Disuelto (mg/L) -->
478     <div class="widget-card chart-card-water" style="padding: 10px 12px; margin-bottom: 0;">
479         <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
480             <div class="widget-header-title">
481                 <h3 style="font-size: 12.5px;">4. Oxígeno Disuelto (mg/L)</h3>
482                 <p style="font-size: 10.5px;">Oxigenación de la cascada hidropónica</p>
483             </div>
484             <span class="badge" style="background: rgba(6, 182, 212, 0.15); color: #06b6d4; font-size: 9.5px; font-weight: 700;">OD mg/L</span>
485         </div>
486         <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
487             <canvas id="doTimelineChart"></canvas>
488         </div>
489     </div>
490
491     <!-- 5. Temperatura de Agua (°C) -->
492     <div class="widget-card chart-card-water" style="padding: 10px 12px; margin-bottom: 0;">
493         <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
494             <div class="widget-header-title">
495                 <h3 style="font-size: 12.5px;">5. Temperatura de Agua (°C)</h3>
496                 <p style="font-size: 10.5px;">Térmica en el tanque de solución</p>
497             </div>
498             <span class="badge badge-primary" style="font-size: 9.5px;">Temp Agua</span>
499         </div>
500         <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
501             <canvas id="watertempTimelineChart"></canvas>
502         </div>
503     </div>
504
505     <!-- 6. Estado Motor (1 / 0) -->
506     <div class="widget-card chart-card-ambient" style="padding: 10px 12px; margin-bottom: 0;">
507         <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
508             <div class="widget-header-title">
509                 <h3 style="font-size: 12.5px;">6. Estado del Motor (1 / 0)</h3>
510                 <p style="font-size: 10.5px;">Pulsos de encendido y apagado de bomba</p>
511             </div>
512             <span class="badge badge-success" style="font-size: 9.5px;">Estado Motor</span>
513         </div>
514         <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
515             <canvas id="motorTimelineChart"></canvas>
516         </div>
517     </div>
518
519     <!-- 7. Temperatura Exterior (°C) -->
520     <div class="widget-card chart-card-ambient" style="padding: 10px 12px; margin-bottom: 0;">
521         <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
522             <div class="widget-header-title">
523                 <h3 style="font-size: 12.5px;">7. Temperatura Exterior (°C)</h3>
524                 <p style="font-size: 10.5px;">Termometría del entorno del cultivo</p>
525             </div>
526             <span class="badge badge-danger" style="background: rgba(239, 68, 68, 0.15); color: #ef4444; font-size: 9.5px;">Temp Exterior</span>
527         </div>
528         <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
529             <canvas id="tempTimelineChart"></canvas>
530         </div>
```

```
    </div>

    <!-- 8. Humedad Relativa Exterior (% HR) -->
    <div class="widget-card chart-card-ambient" style="padding: 10px 12px; margin-bottom: 0;">
        <div class="widget-header" style="margin-bottom: 4px; padding-bottom: 2px;">
            <div class="widget-header-title">
                <h3 style="font-size: 12.5px;">8. Humedad Relativa Exterior (%)</h3>
                <p style="font-size: 10.5px;">Higrometría ambiental</p>
            </div>
            <span class="badge badge-primary" style="font-size: 9.5px;">% HR Exterior</span>
        </div>
        <div class="chart-stage-container" style="height: 175px; margin-bottom: 0;">
            <canvas id="humTimelineChart"></canvas>
        </div>
    </div>
</div>

{% endblock %}

{% block scripts %}
<script>
    let rawTimelineData = {{ sensor_data.timeline | tojson | safe }};
    const activeDate = "{{ sensor_data.target_date }}";
    let activeWindowSize = 30; // Por defecto: ventana deslizante de 30 puntos
    let currentChartGroup = 'ambient';
    let charts = {};

    Chart.defaults.font.family = "'Inter', sans-serif";
    Chart.defaults.color = '#64748b';

    function getSlicedData(dataArray) {
        if (!dataArray || !dataArray.length) return [];
        if (activeWindowSize === 'all') return dataArray;
        return dataArray.slice(-activeWindowSize);
    }

    function toggleKpiCategory(cat, btn) {
        document.querySelectorAll('#btn-kpi-all, #btn-kpi-ambient, #btn-kpi-water').forEach(b => b.classList.remove('active'));
        if (btn) btn.classList.add('active');

        const ambientCards = document.querySelectorAll('.kpi-card-ambient');
        const waterCards = document.querySelectorAll('.kpi-card-water');

        if (cat === 'all') {
            ambientCards.forEach(c => c.style.display = 'block');
            waterCards.forEach(c => c.style.display = 'block');
        } else if (cat === 'ambient') {
            ambientCards.forEach(c => c.style.display = 'block');
            waterCards.forEach(c => c.style.display = 'none');
        } else if (cat === 'water') {
            ambientCards.forEach(c => c.style.display = 'none');
            waterCards.forEach(c => c.style.display = 'block');
        }
    }

```

```
function setChartGroup(group, btn) {
    currentChartGroup = group;
    document.querySelectorAll('#btn-group-ambient, #btn-group-water, #btn-group-all').forEach(b => b.classList.remove('active'));
    if (btn) btn.classList.add('active');

    const waterCharts = document.querySelectorAll('.chart-card-water');
    const ambientCharts = document.querySelectorAll('.chart-card-ambient');

    if (group === 'all') {
        waterCharts.forEach(c => c.style.display = 'block');
        ambientCharts.forEach(c => c.style.display = 'block');
    } else if (group === 'water') {
        waterCharts.forEach(c => c.style.display = 'block');
        ambientCharts.forEach(c => c.style.display = 'none');
    } else if (group === 'ambient') {
        waterCharts.forEach(c => c.style.display = 'none');
        ambientCharts.forEach(c => c.style.display = 'block');
    }

    // Forzar redibujado de charts para ajustar dimensiones al cambiar display
    Object.values(charts).forEach(c => { if (c) c.resize(); });
}

function initSensorCharts() {
    const sliced = getSlicedData(rawTimelineData);
    const labels = sliced.map(d => d.time);

    // 1. pH Solución
    const phs = sliced.map(d => (d.water_ph !== undefined && d.water_ph !== null ? d.water_ph : 0.0));
    // 2. Conductividad Eléctrica (EC)
    const ecs = sliced.map(d => (d.ec_conductivity !== undefined && d.ec_conductivity !== null ? d.ec_conductivity : 0.0));
    // 3. Turbidez (NTU)
    const turbs = sliced.map(d => (d.turbidity_ntu !== undefined && d.turbidity_ntu !== null ? d.turbidity_ntu : 0.0));
    // 4. Oxígeno Disuelto (mg/L)
    const dos = sliced.map(d => (d.dissolved_oxygen_mg_l !== undefined && d.dissolved_oxygen_mg_l !== null ? d.dissolved_oxygen_mg_l : 0.0));
    // 5. Temperatura Agua (°C)
    const waterTemps = sliced.map(d => (d.water_temp !== undefined && d.water_temp !== null ? d.water_temp : 0.0));
    // 6. Estado Motor (1/0)
    const motorStates = sliced.map(d => (d.motor_state !== undefined && d.motor_state !== null ? d.motor_state : 0));
    // 7. Temperatura Exterior (°C)
    const temps = sliced.map(d => (d.temperature !== undefined && d.temperature !== null ? d.temperature : 0.0));
    // 8. Humedad Relativa Exterior (% HR)
    const hums = sliced.map(d => (d.humidity !== undefined && d.humidity !== null ? d.humidity : 0.0));

    function createChart(ctxId, label, data, color, yUnit, isStep = false) {
        const ctx = document.getElementById(ctxId);
        if (!ctx) return;
        if (charts[ctxId]) charts[ctxId].destroy();

        charts[ctxId] = new Chart(ctx, {
            type: 'line',
            data: {
                labels: labels.length ? labels : ['Sin datos'],
                datasets: [{
                    label: label,
```

```
                        data: data.length ? data : [0],
                        borderColor: color,
                        backgroundColor: color + '15',
                        borderWidth: 2.2,
                        fill: true,
                        tension: isStep ? 0 : 0.28,
                        stepped: isStep ? 'before' : false,
                        pointRadius: 0,
                        pointHoverRadius: 5,
                        pointHitRadius: 8
                    }]
                },
                options: {
                    responsive: true,
                    maintainAspectRatio: false,
                    animation: { duration: 250 },
                    plugins: {
                        legend: { display: false },
                        tooltip: {
                            mode: 'index',
                            intersect: false,
                            callbacks: {
                                label: (context) => ` ${label}: ${context.parsed.y} ${yUnit}`
                            }
                        }
                    },
                    scales: {
                        x: {
                            grid: { display: false },
                            ticks: { maxTicksLimit: 8, font: { size: 10 } }
                        },
                        y: {
                            grid: { color: '#f1f5f9' },
                            ticks: { callback: (v) => `${v} ${yUnit}`, font: { size: 10 } }
                        }
                    }
                }
            });
        }

        // Inicialización de los 8 canales en orden estricto
        createChart('phTimelineChart', 'pH Solución', phs, '#ec4899', 'pH');
        createChart('ecTimelineChart', 'Conductividad (EC)', ecs, '#8b5cf6', 'µS/cm');
        createChart('turbTimelineChart', 'Turbidez', turbs, '#f97316', 'NTU');
        createChart('doTimelineChart', 'Oxígeno Disuelto (DO)', dos, '#06b6d4', 'mg/L');
        createChart('watertempTimelineChart', 'Temp. Agua', waterTemps, '#2563eb', '°C');
        createChart('motorTimelineChart', 'Estado Motor', motorStates, '#059669', '', true);
        createChart('tempTimelineChart', 'Temp. Exterior', temps, '#ef4444', '°C');
        createChart('humTimelineChart', 'Humedad Exterior', hums, '#0284c7', '% HR');
    }

    function setWindowSize(size, btnElement) {
        activeWindowSize = size;
        document.querySelectorAll('#btn-win-30, #btn-win-60, #btn-win-120, #btn-win-all').forEach(b => b.classList.remove('active'));
        if (btnElement) btnElement.classList.add('active');
```

```
        updateChartsWithNewData(rawTimelineData);
    }

    function toggleCollapsibleSection(bodyId, iconId, textId) {
        const body = document.getElementById(bodyId);
        const icon = document.getElementById(iconId);
        const text = document.getElementById(textId);
        if (!body) return;

        if (body.style.display === 'none' || body.style.display === '') {
            body.style.display = 'block';
            if (icon) icon.innerText = 'expand_less';
            if (text && textId === 'btn-antenna-text') text.innerText = 'Ocultar Antena';
            if (text && textId === 'btn-filter-text') text.innerText = 'Ocultar Filtros';
        } else {
            body.style.display = 'none';
            if (icon) icon.innerText = 'expand_more';
            if (text && textId === 'btn-antenna-text') text.innerText = 'Ver Antena / Importar CSV';
            if (text && textId === 'btn-filter-text') text.innerText = 'Cambiar Fecha / Filtrar';
        }
    }

    function updateChartsWithNewData(newTimeline) {
        if (!newTimeline || !newTimeline.length) return;
        rawTimelineData = newTimeline;

        const sliced = getSlicedData(rawTimelineData);
        const labels = sliced.map(d => d.time);

        const phs = sliced.map(d => (d.water_ph !== undefined && d.water_ph !== null ? d.water_ph : 0.0));
        const ecs = sliced.map(d => (d.ec_conductivity !== undefined && d.ec_conductivity !== null ? d.ec_conductivity : 0.0));
        const turbs = sliced.map(d => (d.turbidity_ntu !== undefined && d.turbidity_ntu !== null ? d.turbidity_ntu : 0.0));
        const dos = sliced.map(d => (d.dissolved_oxygen_mg_l !== undefined && d.dissolved_oxygen_mg_l !== null ? d.dissolved_oxygen_mg_l : 0.0));
        const waterTemps = sliced.map(d => (d.water_temp !== undefined && d.water_temp !== null ? d.water_temp : 0.0));
        const motorStates = sliced.map(d => (d.motor_state !== undefined && d.motor_state !== null ? d.motor_state : 0));
        const temps = sliced.map(d => (d.temperature !== undefined && d.temperature !== null ? d.temperature : 0.0));
        const hums = sliced.map(d => (d.humidity !== undefined && d.humidity !== null ? d.humidity : 0.0));

        const chartMap = [
            ['phTimelineChart', phs],
            ['ecTimelineChart', ecs],
            ['turbTimelineChart', turbs],
            ['doTimelineChart', dos],
            ['watertempTimelineChart', waterTemps],
            ['motorTimelineChart', motorStates],
            ['tempTimelineChart', temps],
            ['humTimelineChart', hums]
        ];

        chartMap.forEach(([cid, dataVals]) => {
            if (charts[cid]) {
                charts[cid].data.labels = labels;
                charts[cid].data.datasets[0].data = dataVals;
                charts[cid].update('none');
            }
```

```
        });

        // Actualizar tarjetas KPI con el último punto o resumen
        const lastP = rawTimelineData[rawTimelineData.length - 1];
        if (lastP) {
            const setVal = (id, val) => { const el = document.getElementById(id); if (el) el.innerText = val; };

            setVal('kpi-ph-val', (lastP.water_ph !== undefined && lastP.water_ph !== null ? lastP.water_ph : 0.0).toFixed(2));
            setVal('kpi-ec-val', (lastP.ec_conductivity !== undefined && lastP.ec_conductivity !== null ? lastP.ec_conductivity : 0.0).toFixed(1));
            setVal('kpi-turb-val', (lastP.turbidity_ntu !== undefined && lastP.turbidity_ntu !== null ? lastP.turbidity_ntu : 0.0).toFixed(2));
            setVal('kpi-do-val', (lastP.dissolved_oxygen_mg_l !== undefined && lastP.dissolved_oxygen_mg_l !== null ? lastP.dissolved_oxygen_mg_l : 0.0).toFixed(2));
            setVal('kpi-watertemp-val', (lastP.water_temp !== undefined && lastP.water_temp !== null ? lastP.water_temp : 0.0).toFixed(1));
            setVal('kpi-motor-val', (lastP.motor_state !== undefined && lastP.motor_state !== null ? lastP.motor_state : 0));
            setVal('kpi-temp-val', lastP.temperature || 0.0);
            setVal('kpi-hum-val', lastP.humidity || 0.0);

            const badge = document.getElementById('timeline-count-badge');
            if (badge) badge.innerText = `${rawTimelineData.length} LECTURAS (${activeDate})`;
        }
    }

    const activePlantId = {{ active_plant_id }};

    document.addEventListener('DOMContentLoaded', () => {
        initSensorCharts();
    });

    function filterByDate(dateStr, btnElement) {
        if (!dateStr) return;
        window.location.href = `/sensors?date=${dateStr}&plant_id=${activePlantId}`;
    }

    async function handleCsvUpload(e) {
        e.preventDefault();
        const fileInput = document.getElementById('telemetry-csv-input');
        if (!fileInput.files.length) return;

        const formData = new FormData();
        formData.append('csv_file', fileInput.files[0]);

        const feedback = document.getElementById('csv-upload-feedback');
        feedback.style.display = 'block';
        feedback.style.background = 'rgba(5, 150, 105, 0.1)';
        feedback.style.color = '#047857';
        feedback.innerText = 'Procesando archivo CSV de 8 sensores...';

        try {
            const res = await fetch('/api/import_tower_csv', {
                method: 'POST',
                body: formData
            });
            const data = await res.json();
            if (data.status === 'success') {
                feedback.innerText = `CSV importado exitosamente (${data.imported_records} registros agregados).`;
                setTimeout(() => window.location.reload(), 1200);
```

```
            } else {
                feedback.style.background = 'rgba(220, 38, 38, 0.1)';
                feedback.style.color = '#b91c1c';
                feedback.innerText = 'Error: ' + data.message;
            }
        } catch (err) {
            feedback.style.background = 'rgba(220, 38, 38, 0.1)';
            feedback.style.color = '#b91c1c';
            feedback.innerText = 'Error al subir archivo CSV.';
        }
    }

    async function refreshAntennaPorts() {
        try {
            const res = await fetch('/api/telemetry/ports');
            const data = await res.json();
            const select = document.getElementById('antenna-port-select');
            if (data.status === 'success' && select) {
                const ports = data.ports || [];
                select.innerHTML = '';
                if (ports.length === 0) {
                    const opt = document.createElement('option');
                    opt.value = '';
                    opt.textContent = 'No se detecta antena USB conectada';
                    opt.disabled = true;
                    opt.selected = true;
                    select.appendChild(opt);
                } else {
                    ports.forEach(p => {
                        const opt = document.createElement('option');
                        opt.value = p;
                        opt.textContent = p;
                        select.appendChild(opt);
                    });
                }
            }
        } catch (e) {
            console.error('Error al actualizar puertos:', e);
        }
    }

    let livePollingTimer = null;
    let isCurrentlyRecording = {{ 'true' if telemetry_status.is_recording else 'false' }};

    function startLivePolling() {
        if (livePollingTimer) clearInterval(livePollingTimer);
        livePollingTimer = setInterval(checkLiveTelemetry, 1500);
    }

    function stopLivePolling() {
        if (livePollingTimer) {
            clearInterval(livePollingTimer);
            livePollingTimer = null;
        }
    }
```

```
861
862     async function checkLiveTelemetry() {
863         try {
864             const sRes = await fetch('/api/telemetry/status');
865             const sData = await sRes.json();
866
867             if (sData.status === 'success' && sData.telemetry) {
868                 const tel = sData.telemetry;
869                 const r = tel.latest_reading;
870
871                 const btnStart = document.getElementById('btn-start-telemetry');
872                 const btnStop = document.getElementById('btn-stop-telemetry');
873                 const statusDot = document.querySelector('.status-dot');
874                 const statusText = document.getElementById('antenna-live-status-text');
875
876                 if (tel.is_recording) {
877                     if (btnStart) btnStart.style.display = 'none';
878                     if (btnStop) btnStop.style.display = 'inline-flex';
879                     if (statusDot) { statusDot.classList.remove('inactive'); statusDot.classList.add('active'); }
880                     if (statusText) { statusText.innerText = `Transmitiendo en Vivo (${tel.readings_count} lecturas)`; statusText.style.color = 'var(--accent-success)';
}
881
882                     if (r && tel.readings_count > 0) {
883                         const nowLabel = (r.timestamp && r.timestamp.includes(' ')) ? r.timestamp.split(' ')[1] : new Date().toLocaleTimeString();
884
885                         const newPoint = {
886                             time: nowLabel,
887                             timestamp: r.timestamp || new Date().toISOString(),
888                             water_ph: r.water_ph !== undefined && r.water_ph !== null ? r.water_ph : 0.0,
889                             ec_conductivity: r.ec_conductivity !== undefined && r.ec_conductivity !== null ? r.ec_conductivity : 0.0,
890                             turbidity_ntu: r.turbidity_ntu !== undefined && r.turbidity_ntu !== null ? r.turbidity_ntu : 0.0,
891                             dissolved_oxygen_mg_l: r.dissolved_oxygen_mg_l !== undefined && r.dissolved_oxygen_mg_l !== null ? r.dissolved_oxygen_mg_l : 0.0,
892                             water_temp: r.water_temp !== undefined && r.water_temp !== null ? r.water_temp : 0.0,
893                             motor_state: r.motor_state !== undefined && r.motor_state !== null ? r.motor_state : 0,
894                             temperature: r.temperature !== undefined && r.temperature !== null ? r.temperature : 0.0,
895                             humidity: r.humidity !== undefined && r.humidity !== null ? r.humidity : 0.0
896                         };
897
898                         if (!rawTimelineData.length || rawTimelineData[rawTimelineData.length - 1].time !== newPoint.time) {
899                             rawTimelineData.push(newPoint);
900                             if (rawTimelineData.length > 200) rawTimelineData.shift();
901                             updateChartsWithNewData(rawTimelineData);
902                         }
903                     }
904                 } else {
905                     // Si ya se detuvo la captación, detener el sondeo en segundo plano
906                     stopLivePolling();
907                     if (btnStart) btnStart.style.display = 'inline-flex';
908                     if (btnStop) btnStop.style.display = 'none';
909                     if (statusDot) { statusDot.classList.remove('active'); statusDot.classList.add('inactive'); }
910                     if (statusText) { statusText.innerText = 'Antena Inactiva'; statusText.style.color = 'var(--text-muted)'; }
911                 }
912             }
913         } catch (e) {
914             console.warn('Error en ciclo de actualización de telemetría:', e);
915         }
```

```
}

async function toggleTelemetry(start) {
    const select = document.getElementById('antenna-port-select');
    const port = select ? select.value : '';

    if (start && (!port || port.trim() === '')) {
        alert('Atención: No se detecta ninguna antena receptora conectada por USB.\nConecte la antena a la computadora e intente nuevamente.');
        return;
    }

    const endpoint = start ? '/api/start_telemetry_recording' : '/api/stop_telemetry_recording';
    try {
        const res = await fetch(endpoint, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ port: port })
        });
        const data = await res.json();
        if (res.ok && data.status === 'success') {
            const btnStart = document.getElementById('btn-start-telemetry');
            const btnStop = document.getElementById('btn-stop-telemetry');
            const statusDot = document.querySelector('.status-dot');
            const statusText = document.getElementById('antenna-live-status-text');

            if (start) {
                if (btnStart) btnStart.style.display = 'none';
                if (btnStop) btnStop.style.display = 'inline-flex';
                if (statusDot) { statusDot.classList.remove('inactive'); statusDot.classList.add('active'); }
                if (statusText) { statusText.innerText = 'Transmitiendo en Vivo'; statusText.style.color = 'var(--accent-success)'; }
                startLivePolling();
            } else {
                stopLivePolling();
                if (btnStart) btnStart.style.display = 'inline-flex';
                if (btnStop) btnStop.style.display = 'none';
                if (statusDot) { statusDot.classList.remove('active'); statusDot.classList.add('inactive'); }
                if (statusText) { statusText.innerText = 'Antena Inactiva'; statusText.style.color = 'var(--text-muted)'; }
                setTimeout(() => window.location.reload(), 600);
            }
        } else {
            stopLivePolling();
            alert('No se pudo iniciar la captación:\n' + (data.message || 'No se detectó antena o hubo un error al conectar.'));
        }
    } catch (e) {
        stopLivePolling();
        alert('Error de comunicación con el servidor.');
    }
}

async function deleteTelemetryForActiveDate() {
    const dateStr = document.getElementById('custom-date-filter').value || '{{ sensor_data.target_date }}';
    if (!confirm(`¿Eliminar los registros de telemetría de la fecha '${dateStr}'?`)) return;

    try {
        const res = await fetch('/api/delete_telemetry_by_date', {
```

```
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ date_str: dateStr })
            });
            const data = await res.json();
            if (data.status === 'success') {
                window.location.href = `/sensors?date=todos&plant_id=${activePlantId}`;
            } else {
                alert('Error: ' + data.message);
            }
        } catch (e) {
            alert('Error al eliminar datos.');
        }
    }

    // Iniciar sondeo únicamente si ya estaba grabando al abrir la vista
    if (isCurrentlyRecording) {
        startLivePolling();
    }
</script>
{% endblock %}
```

ARCHIVO 73/85: local_server/templates/timelapse.html (592 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Time-Lapse & Evolución Biológica{% endblock %}
{% block heading %}Time-Lapse & Evolución Biológica{% endblock %}
{% block subheading %}Visor cronológico y reproducción secuencial de desarrollo vegetativo para Canastilla #{{ active_plant_id }}{% endblock %}

{% block header_controls %}
<!-- Selector de Canastilla / Torre -->
<div style="display: flex; align-items: center; gap: 6px;">
    <select id="header-plant-selector" class="form-input-control" style="padding: 6px 10px; font-weight: 600; font-size: 13px; cursor: pointer; width: auto;" onchange="switchActivePlant(this.value)">
        <option value="1" {% if active_plant_id == 1 %}selected{% endif %}>Canastilla #1 (Cebollín)</option>
        <option value="2" {% if active_plant_id == 2 %}selected{% endif %}>Canastilla #2 (Albahaca)</option>
        <option value="3" {% if active_plant_id == 3 %}selected{% endif %}>Canastilla #3 (Lechuga)</option>
        <option value="4" {% if active_plant_id == 4 %}selected{% endif %}>Canastilla #4 (Fresa)</option>
    </select>
</div>
{% endblock %}

{% block content %}

{% if not timelapse_data.has_frames %}
<!-- ESTADO SIN FOTOGRAMAS -->
<div class="zero-data-banner" style="margin-bottom: 24px;">
    <div class="zero-data-icon">
        <span class="material-icons-round">slow_motion_video</span>
    </div>
    <div class="zero-data-text">
        <h4>Canastilla #{{ active_plant_id }} - Sin Fotogramas Registrados</h4>
        <p>No se encontraron capturas procesadas para la <strong>Canastilla #{{ active_plant_id }}</strong> ({{ selected_profile.display_name }}). Puedes procesar lotes fotográficos en la sección <strong>Procesamiento</strong> para generar la secuencia temporal.</p>
    </div>
</div>
{% else %}

<!-- REPRODUCTOR TIME-LAPSE PRINCIPAL -->
<div style="display: grid; grid-template-columns: 1fr 340px; gap: 20px; margin-bottom: 24px;">

    <!-- COLUMNA IZQUIERDA: VISOR DE IMÁGENES Y CONTROLES -->
    <div class="widget-card" style="display: flex; flex-direction: column; justify-content: space-between;">

        <!-- Barra de Modos de Vista -->
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 14px; flex-wrap: wrap; gap: 10px;">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="badge badge-success" id="player-crop-badge" style="font-size: 11px;">
                    Canastilla #{{ active_plant_id }}: {{ timelapse_data.crop_name }}
                </span>
                <span id="player-frame-counter" style="font-size: 12px; font-weight: 700; color: var(--text-heading);">
```

```
                    Fotograma 1 / {{ timelapse_data.total_frames }}
                </span>
            </div>

            <!-- Selector de Ángulo de Cámara -->
            <div class="filter-group">
                <button type="button" class="filter-btn active" id="btn-view-cenital" onclick="switchCameraView('cenital')">
                    <span class="material-icons-round" style="font-size: 14px;">visibility</span>
                    <span>Cenital</span>
                </button>
                <button type="button" class="filter-btn" id="btn-view-lateral" onclick="switchCameraView('lateral')">
                    <span class="material-icons-round" style="font-size: 14px;">view_sidebar</span>
                    <span>Lateral</span>
                </button>
                <button type="button" class="filter-btn" id="btn-view-dual" onclick="switchCameraView('dual')">
                    <span class="material-icons-round" style="font-size: 14px;">splitscreen</span>
                    <span>Vista Dual</span>
                </button>
            </div>
        </div>

        <!-- STAGE DE VISUALIZACIÓN DE IMÁGENES CON OVERLAY HUD -->
        <div class="timelapse-stage-container" id="timelapse-stage">

            <!-- Vista Cenital -->
            <div class="timelapse-img-wrapper" id="wrapper-cenital">
                <img id="tl-img-cenital" src="" alt="Cenital" class="timelapse-display-img">
                <div class="timelapse-cam-tag">ÁNGULO CENITAL (ÁREA FOLIAR)</div>
            </div>

            <!-- Vista Lateral -->
            <div class="timelapse-img-wrapper" id="wrapper-lateral" style="display: none;">
                <img id="tl-img-lateral" src="" alt="Lateral" class="timelapse-display-img">
                <div class="timelapse-cam-tag">ÁNGULO LATERAL (ALTURA & TALLO)</div>
            </div>

            <!-- HUD de Telemetría Flotante Superior -->
            <div class="timelapse-hud-overlay">
                <div style="display: flex; align-items: center; gap: 8px;">
                    <span class="material-icons-round" style="font-size: 16px; color: var(--accent-success);">calendar_today</span>
                    <strong id="hud-date-text">2026-09-01</strong>
                    <span style="opacity: 0.7;">|</span>
                    <span id="hud-period-text">Mañana</span>
                </div>
                <div id="hud-time-text" style="font-size: 11px; opacity: 0.85;">08:30:00</div>
            </div>
        </div>

        <!-- CONTROLES DE REPRODUCCIÓN Y SCRUBBER SLIDER -->
        <div style="margin-top: 16px;">

            <!-- Barra Deslizante de Tiempo (Scrubber) -->
            <div style="margin-bottom: 12px;">
                <input type="range" id="timeline-slider" min="0" max="{{ timelapse_data.total_frames - 1 }}" value="0" step="1"
                       oninput="onSliderSeek(this.value)" class="timelapse-slider">
```

```
                <div style="display: flex; justify-content: space-between; font-size: 10.5px; color: var(--text-muted); margin-top: 4px;">
                    <span id="slider-start-label">{{ timelapse_data.frames[0].date_str if timelapse_data.frames else '' }}</span>
                    <span>Línea Temporal de Crecimiento</span>
                    <span id="slider-end-label">{{ timelapse_data.frames[-1].date_str if timelapse_data.frames else '' }}</span>
                </div>
            </div>

            <!-- Botones de Acción del Reproductor -->
            <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px;">

                <div style="display: flex; align-items: center; gap: 8px;">
                    <button type="button" class="btn btn-primary" id="btn-play-pause" onclick="togglePlayPause()" style="min-width: 100px;">
                        <span class="material-icons-round" id="play-pause-icon">play_arrow</span>
                        <span id="play-pause-text">Reproducir</span>
                    </button>
                    <button type="button" class="btn btn-outline btn-sm" onclick="stepFrame(-1)" title="Fotograma Anterior">
                        <span class="material-icons-round">skip_previous</span>
                    </button>
                    <button type="button" class="btn btn-outline btn-sm" onclick="stepFrame(1)" title="Fotograma Siguiente">
                        <span class="material-icons-round">skip_next</span>
                    </button>
                </div>

                <!-- Velocidad y Bucle -->
                <div style="display: flex; align-items: center; gap: 6px;">
                    <span style="font-size: 11.5px; color: var(--text-muted); font-weight: 600;">Velocidad:</span>
                    <button type="button" class="speed-btn" onclick="setPlaySpeed(0.5, this)">0.5x</button>
                    <button type="button" class="speed-btn active" onclick="setPlaySpeed(1.0, this)">1x</button>
                    <button type="button" class="speed-btn" onclick="setPlaySpeed(2.0, this)">2x</button>
                    <button type="button" class="speed-btn" onclick="setPlaySpeed(4.0, this)">4x</button>

                    <button type="button" class="btn btn-outline btn-sm" id="btn-loop-toggle" onclick="toggleLoop()" style="margin-left: 6px;" title="Repetir en Bucle">
                        <span class="material-icons-round" id="loop-icon" style="color: var(--accent-success);">repeat</span>
                    </button>
                </div>
            </div>
        </div>
    </div>

    <!-- COLUMNA DERECHA: GAUGES BIOMÉTRICOS Y AMBIENTALES EN VIVO -->
    <div style="display: flex; flex-direction: column; gap: 14px;">

        <!-- Bloque Biométrico Sincronizado -->
        <div class="widget-card">
            <h4 style="font-size: 13.5px; font-weight: 700; color: var(--accent-success); margin-bottom: 14px; display: flex; align-items: center; gap: 6px;">
                <span class="material-icons-round" style="font-size: 18px;">spa</span>
                <span>Biometría en Este Fotograma</span>
            </h4>

            <div style="display: flex; flex-direction: column; gap: 10px;">
                <div class="timelapse-gauge-card">
                    <div style="display: flex; justify-content: space-between; align-items: center;">
                        <span style="font-size: 11.5px; color: var(--text-muted);">Área Foliar Cenital</span>
                        <strong id="gauge-foliar-area" style="font-size: 16px; color: var(--text-heading);">0.0 cm²</strong>
                    </div>
```

```
                    <div class="gauge-bar-bg">
                        <div id="gauge-bar-area-fill" class="gauge-bar-fill emerald" style="width: 0%;"></div>
                    </div>
                </div>

                <div class="timelapse-gauge-card">
                    <div style="display: flex; justify-content: space-between; align-items: center;">
                        <span style="font-size: 11.5px; color: var(--text-muted);">Altura de Planta</span>
                        <strong id="gauge-plant-height" style="font-size: 16px; color: var(--text-heading);">0.0 cm</strong>
                    </div>
                    <div class="gauge-bar-bg">
                        <div id="gauge-bar-height-fill" class="gauge-bar-fill blue" style="width: 0%;"></div>
                    </div>
                </div>

                <div class="timelapse-gauge-card">
                    <div style="display: flex; justify-content: space-between; align-items: center;">
                        <span style="font-size: 11.5px; color: var(--text-muted);">Diámetro de Tallo</span>
                        <strong id="gauge-stem-dia" style="font-size: 15px; color: var(--text-heading);">0.0 mm</strong>
                    </div>
                </div>

                <div class="timelapse-gauge-card">
                    <div style="display: flex; justify-content: space-between; align-items: center;">
                        <span style="font-size: 11.5px; color: var(--text-muted);">Índice de Salud Foliar</span>
                        <strong id="gauge-health-index" style="font-size: 15px; color: var(--accent-success);">100%</strong>
                    </div>
                </div>
            </div>
        </div>

        <!-- Bloque Ambiental Concurrente -->
        <div class="widget-card">
            <h4 style="font-size: 13.5px; font-weight: 700; color: #0284c7; margin-bottom: 14px; display: flex; align-items: center; gap: 6px;">
                <span class="material-icons-round" style="font-size: 18px;">thermostat</span>
                <span>Telemetría Concurrente</span>
            </h4>

            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 8px;">
                <div style="background: rgba(2, 132, 199, 0.06); padding: 8px 10px; border-radius: 8px; border: 1px solid rgba(2, 132, 199, 0.15);">
                    <span style="font-size: 10px; color: var(--text-muted); display: block;">Temperatura</span>
                    <strong id="gauge-env-temp" style="font-size: 14px; color: #0369a1;">0.0 °C</strong>
                </div>
                <div style="background: rgba(2, 132, 199, 0.06); padding: 8px 10px; border-radius: 8px; border: 1px solid rgba(2, 132, 199, 0.15);">
                    <span style="font-size: 10px; color: var(--text-muted); display: block;">Humedad Relativa</span>
                    <strong id="gauge-env-hum" style="font-size: 14px; color: #0369a1;">0.0 %</strong>
                </div>
                <div style="background: rgba(2, 132, 199, 0.06); padding: 8px 10px; border-radius: 8px; border: 1px solid rgba(2, 132, 199, 0.15);">
                    <span style="font-size: 10px; color: var(--text-muted); display: block;">Radiación Solar</span>
                    <strong id="gauge-env-uv" style="font-size: 14px; color: #0369a1;">0.0 lux</strong>
                </div>
                <div style="background: rgba(2, 132, 199, 0.06); padding: 8px 10px; border-radius: 8px; border: 1px solid rgba(2, 132, 199, 0.15);">
                    <span style="font-size: 10px; color: var(--text-muted); display: block;">Bomba</span>
                    <strong id="gauge-env-curr" style="font-size: 14px; color: #0369a1;">0.00 A</strong>
                </div>
```

```
            </div>
        </div>

    </div>
</div>

<!-- SECCIÓN 3: CARROUSEL DE FOTOGRAMAS MINIATURA (FILMSTRIP) -->
<div class="widget-card">
    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 12px;">
        <h4 style="font-size: 13.5px; font-weight: 700; color: var(--text-heading); margin: 0;">
            Secuencia Completa de Fotogramas (Filmstrip)
        </h4>
        <span style="font-size: 11.5px; color: var(--text-muted);">
            Haz clic en cualquier miniatura para saltar a ese momento del cultivo
        </span>
    </div>

    <div class="filmstrip-scrollable-container">
        {% for f in timelapse_data.frames %}
            <div class="filmstrip-card {% if loop.first %}active{% endif %}" id="filmstrip-item-{{ f.frame_index }}"
                 onclick="jumpToFrame({{ f.frame_index }})">
                <div class="filmstrip-thumb-box">
                    <img src="{{ f.cenital_url or f.lateral_url }}" alt="Frame {{ f.frame_index }}">
                </div>
                <div style="padding: 6px 8px; font-size: 11px;">
                    <strong style="color: var(--text-heading); display: block;">{{ f.date_str }}</strong>
                    <span style="color: var(--text-muted); font-size: 10px;">{{ f.period }}</span>
                    <div style="margin-top: 2px; color: var(--accent-success); font-weight: 700; font-size: 10.5px;">
                        {{ f.foliar_area_cm2 }} cm²
                    </div>
                </div>
            </div>
        {% endfor %}
    </div>
</div>

{% endif %}

{% endblock %}

{% block scripts %}
<style>
/* ESTILOS DEL REPRODUCTOR TIME-LAPSE SIFMA */
.timelapse-stage-container {
    background: #0f172a;
    border-radius: 12px;
    height: 420px;
    position: relative;
    overflow: hidden;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 1px solid var(--border-color);
}

```

```
.timelapse-img-wrapper {
    width: 100%;
    height: 100%;
    position: relative;
    display: flex;
    align-items: center;
    justify-content: center;
}

.timelapse-display-img {
    max-width: 100%;
    max-height: 100%;
    object-fit: contain;
    transition: opacity 0.15s ease;
}

.timelapse-cam-tag {
    position: absolute;
    bottom: 12px;
    left: 12px;
    background: rgba(15, 23, 42, 0.75);
    backdrop-filter: blur(4px);
    color: #ffffff;
    font-size: 10px;
    font-weight: 700;
    padding: 3px 8px;
    border-radius: 4px;
    letter-spacing: 0.5px;
}

.timelapse-hud-overlay {
    position: absolute;
    top: 12px;
    right: 12px;
    background: rgba(15, 23, 42, 0.85);
    backdrop-filter: blur(6px);
    border: 1px solid rgba(255, 255, 255, 0.15);
    color: #ffffff;
    padding: 6px 12px;
    border-radius: 8px;
    font-size: 12px;
    display: flex;
    flex-direction: column;
    align-items: flex-end;
    gap: 2px;
}

.timelapse-slider {
    width: 100%;
    height: 6px;
    border-radius: 3px;
    background: #e2e8f0;
    outline: none;
    accent-color: var(--accent-success);
    cursor: pointer;
```

```
}

.speed-btn {
    background: var(--bg-card-subtle);
    border: 1px solid var(--border-color);
    color: var(--text-muted);
    padding: 3px 7px;
    border-radius: 4px;
    font-size: 11px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.15s ease;
}

.speed-btn.active {
    background: var(--accent-success);
    color: #ffffff;
    border-color: var(--accent-success);
}

.timelapse-gauge-card {
    background: var(--bg-card-subtle);
    padding: 9px 12px;
    border-radius: 8px;
    border: 1px solid var(--border-color);
}

.gauge-bar-bg {
    width: 100%;
    height: 5px;
    background: #e2e8f0;
    border-radius: 3px;
    margin-top: 6px;
    overflow: hidden;
}

.gauge-bar-fill {
    height: 100%;
    border-radius: 3px;
    transition: width 0.2s ease;
}

.gauge-bar-fill.emerald { background: var(--accent-success); }
.gauge-bar-fill.blue { background: #0284c7; }

.filmstrip-scrollable-container {
    display: flex;
    gap: 12px;
    overflow-x: auto;
    padding-bottom: 8px;
}

.filmstrip-card {
    flex: 0 0 130px;
    background: #ffffff;
```

```
    border: 1px solid var(--border-color);
    border-radius: 8px;
    overflow: hidden;
    cursor: pointer;
    transition: all 0.2s ease;
}

.filmstrip-card:hover {
    border-color: var(--accent-success);
    transform: translateY(-2px);
}

.filmstrip-card.active {
    border: 2px solid var(--accent-success);
    box-shadow: 0 4px 12px rgba(16, 185, 129, 0.2);
}

.filmstrip-thumb-box {
    width: 100%;
    height: 75px;
    background: #0f172a;
    overflow: hidden;
}

.filmstrip-thumb-box img {
    width: 100%;
    height: 100%;
    object-fit: cover;
}
</style>

<script>
    const framesData = {{ timelapse_data.frames | tojson | safe }};
    const totalFrames = framesData ? framesData.length : 0;
    let currentFrameIndex = 0;
    let isPlaying = false;
    let playInterval = null;
    let playSpeed = 1.0;
    let isLooping = true;
    let activeCameraView = 'cenital';

    function initTimelapsePlayer() {
        if (!totalFrames) return;
        renderFrame(0);
    }

    function renderFrame(index) {
        if (!framesData || index < 0 || index >= totalFrames) return;
        currentFrameIndex = index;
        const frame = framesData[index];

        // 1. Actualizar Imágenes
        const imgCenital = document.getElementById('tl-img-cenital');
        const imgLateral = document.getElementById('tl-img-lateral');
        if (imgCenital) imgCenital.src = frame.cenital_url || frame.lateral_url;
```

```
        if (imgLateral) imgLateral.src = frame.lateral_url || frame.cenital_url;

        // 2. Actualizar HUD
        document.getElementById('hud-date-text').innerText = frame.date_str;
        document.getElementById('hud-period-text').innerText = frame.period;
        document.getElementById('hud-time-text').innerText = frame.time_str;
        document.getElementById('player-frame-counter').innerText = `Fotograma ${index + 1} / ${totalFrames}`;

        // 3. Sincronizar Slider
        const slider = document.getElementById('timeline-slider');
        if (slider) slider.value = index;

        // 4. Actualizar Gauges Biométricos
        document.getElementById('gauge-foliar-area').innerText = `${frame.foliar_area_cm2} cm²`;
        document.getElementById('gauge-plant-height').innerText = `${frame.plant_height_cm} cm`;
        document.getElementById('gauge-stem-dia').innerText = `${frame.stem_diameter_mm} mm`;
        document.getElementById('gauge-health-index').innerText = `${frame.health_index}%`;

        // Barras porcentuales relativas
        const maxAreaExpected = 150.0;
        const maxHeightExpected = 40.0;
        const areaPct = Math.min((frame.foliar_area_cm2 / maxAreaExpected) * 100, 100);
        const heightPct = Math.min((frame.plant_height_cm / maxHeightExpected) * 100, 100);
        document.getElementById('gauge-bar-area-fill').style.width = `${areaPct}%`;
        document.getElementById('gauge-bar-height-fill').style.width = `${heightPct}%`;

        // 5. Actualizar Gauges Ambientales
        document.getElementById('gauge-env-temp').innerText = `${frame.temperature} °C`;
        document.getElementById('gauge-env-hum').innerText = `${frame.humidity} %`;
        document.getElementById('gauge-env-uv').innerText = `${frame.uv_solar} lux`;
        document.getElementById('gauge-env-curr').innerText = `${frame.motor_current} A`;

        // 6. Resaltar Filmstrip Card activa
        document.querySelectorAll('.filmstrip-card').forEach(c => c.classList.remove('active'));
        const activeCard = document.getElementById(`filmstrip-item-${index}`);
        if (activeCard) {
            activeCard.classList.add('active');
            activeCard.scrollIntoView({ behavior: 'smooth', block: 'nearest', inline: 'center' });
        }
    }

    function togglePlayPause() {
        if (isPlaying) {
            pauseTimelapse();
        } else {
            playTimelapse();
        }
    }

    function playTimelapse() {
        if (totalFrames <= 1) return;
        isPlaying = true;
        document.getElementById('play-pause-icon').innerText = 'pause';
        document.getElementById('play-pause-text').innerText = 'Pausar';

```

```
        const baseIntervalMs = 1000;
        const intervalMs = baseIntervalMs / playSpeed;

        clearInterval(playInterval);
        playInterval = setInterval(() => {
            if (currentFrameIndex >= totalFrames - 1) {
                if (isLooping) {
                    renderFrame(0);
                } else {
                    pauseTimelapse();
                }
            } else {
                renderFrame(currentFrameIndex + 1);
            }
        }, intervalMs);
    }

    function pauseTimelapse() {
        isPlaying = false;
        clearInterval(playInterval);
        document.getElementById('play-pause-icon').innerText = 'play_arrow';
        document.getElementById('play-pause-text').innerText = 'Reproducir';
    }

    function stepFrame(delta) {
        pauseTimelapse();
        let nextIdx = currentFrameIndex + delta;
        if (nextIdx < 0) nextIdx = 0;
        if (nextIdx >= totalFrames) nextIdx = totalFrames - 1;
        renderFrame(nextIdx);
    }

    function onSliderSeek(val) {
        pauseTimelapse();
        renderFrame(parseInt(val));
    }

    function jumpToFrame(index) {
        pauseTimelapse();
        renderFrame(index);
    }

    function setPlaySpeed(speed, btnElement) {
        playSpeed = speed;
        document.querySelectorAll('.speed-btn').forEach(b => b.classList.remove('active'));
        if (btnElement) btnElement.classList.add('active');
        if (isPlaying) {
            playTimelapse();
        }
    }

    function toggleLoop() {
        isLooping = !isLooping;
        const icon = document.getElementById('loop-icon');
        if (isLooping) {
```

```
            icon.style.color = 'var(--accent-success)';
        } else {
            icon.style.color = 'var(--text-muted)';
        }
    }

    function switchCameraView(view) {
        activeCameraView = view;
        document.querySelectorAll('.filter-group .filter-btn').forEach(b => b.classList.remove('active'));
        const activeBtn = document.getElementById(`btn-view-${view}`);
        if (activeBtn) activeBtn.classList.add('active');

        const stage = document.getElementById('timelapse-stage');
        const wrapCenital = document.getElementById('wrapper-cenital');
        const wrapLateral = document.getElementById('wrapper-lateral');

        if (view === 'cenital') {
            stage.style.display = 'flex';
            wrapCenital.style.display = 'flex';
            wrapLateral.style.display = 'none';
        } else if (view === 'lateral') {
            stage.style.display = 'flex';
            wrapCenital.style.display = 'none';
            wrapLateral.style.display = 'flex';
        } else if (view === 'dual') {
            stage.style.display = 'grid';
            stage.style.gridTemplateColumns = '1fr 1fr';
            wrapCenital.style.display = 'flex';
            wrapLateral.style.display = 'flex';
        }
    }

    // Atajo de teclado: Barra espaciadora para reproducir/pausar
    document.addEventListener('keydown', (e) => {
        if (e.code === 'Space' && e.target.tagName !== 'INPUT') {
            e.preventDefault();
            togglePlayPause();
        }
    });

    document.addEventListener('DOMContentLoaded', () => {
        initTimelapsePlayer();
    });
</script>
{% endblock %}
```

ARCHIVO 74/85: local_server/templates/users.html (702 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

{% extends 'base.html' %}

{% block title %}Gestión de Usuarios y Permisos Granulares{% endblock %}
{% block heading %}Control de Acceso y Roles del Sistema{% endblock %}
{% block subheading %}Administración de cuentas, perfiles de investigación y asignación granular de funciones{% endblock %}

{% block header_actions %}
<button class="btn btn-primary btn-sm" onclick="openCreateUserModal()">
    <span class="material-icons-round">person_add</span>
    <span>Nuevo Usuario</span>
</button>
{% endblock %}

{% block content %}
<div class="users-table-container">
    <table class="users-table">
        <thead>
            <tr>
                <th>Usuario</th>
                <th>Rol / Perfil</th>
                <th>Funciones Habilitadas (Permisos)</th>
                <th>Correo Electrónico</th>
                <th>Estado</th>
                <th>Último Acceso</th>
                <th style="text-align: right;">Acciones</th>
            </tr>
        </thead>
        <tbody id="users-table-body">
            {% for u in users %}
            <tr id="user-row-{{ u.id }}">
                <td>
                    <div style="display: flex; align-items: center; gap: 10px;">
                        {% if u.avatar_url %}
                        <img src="{{ u.avatar_url }}" alt="{{ u.full_name }}" class="user-avatar-circle" style="width: 38px; height: 38px; object-fit: cover; border: 2px solid {% if u.is_super_admin %}var(--accent-success){% else %}var(--border-color){% endif %}; flex-shrink: 0;" title="{{ u.full_name }}">
                        {% else %}
                        <div class="user-avatar-circle" style="width: 38px; height: 38px; font-size: 12px; background: {% if u.is_super_admin %}linear-gradient(135deg, #059669, #10b981){% else %}var(--border-color){% endif %}; color: #ffffff; flex-shrink: 0;">
                            {{ u.avatar_initials }}
                        </div>
                        {% endif %}
                        <div>
                            <div style="font-weight: 700; color: var(--text-heading); font-size: 13.5px;">{{ u.full_name }}</div>
                            <div style="font-size: 11.5px; color: var(--text-muted); display: flex; align-items: center; gap: 4px;">
                                <span>@{{ u.username }}</span>
                                {% if u.is_super_admin %}
                                <span class="badge badge-primary" style="font-size: 9.5px; padding: 1px 5px;">Super Usuario</span>
                                {% endif %}
```

```
                            </div>
                        </div>
                    </div>
                </td>
                <td>
                    <span class="badge {% if u.role == 'admin' %}badge-primary{% elif u.role == 'investigador' %}badge-success{% elif u.role == 'operador' %}badge-warning{% else %}badge-info{% endif %}" style="font-size: 11.5px;">
                        {{ u.role_display }}
                    </span>
                </td>
                <td>
                    {% if u.is_super_admin %}
                        <span class="badge badge-primary" style="font-size: 11px; padding: 3px 8px; display: inline-flex; align-items: center; gap: 4px;">
                            <span class="material-icons-round" style="font-size: 13px;">verified_user</span>
                            Acceso Total (Todos los Módulos)
                        </span>
                    {% else %}
                        <div style="display: flex; flex-wrap: wrap; gap: 4px; max-width: 320px;">
                            {% set u_perms = u.get_permissions_dict() %}
                            {% set perm_count = namespace(value=0) %}
                            {% for p_key, p_name, p_icon, p_desc in all_permissions_list %}
                                {% if u_perms.get(p_key) %}
                                    {% set perm_count.value = perm_count.value + 1 %}
                                    <span class="badge badge-success" style="font-size: 10.5px; padding: 2px 6px; display: inline-flex; align-items: center; gap: 3px;" title="{{ p_desc }}">
                                        <span class="material-icons-round" style="font-size: 12px;">{{ p_icon }}</span>
                                        {{ p_name }}
                                    </span>
                                {% endif %}
                            {% endfor %}
                            {% if perm_count.value == 0 %}
                                <span style="font-size: 11px; color: var(--accent-danger); font-style: italic;">Sin funciones asignadas</span>
                            {% endif %}
                        </div>
                    {% endif %}
                </td>
                <td style="font-size: 12.5px; color: var(--text-heading);">{{ u.email or 'Sin registrar' }}</td>
                <td>
                    <span class="badge {% if u.is_active %}badge-success{% else %}badge-danger{% endif %}">
                        {% if u.is_active %}Activo{% else %}Pausado / Inactivo{% endif %}
                    </span>
                </td>
                <td style="font-size: 12px; color: var(--text-muted);">
                    {{ u.last_login.strftime('%d/%m/%Y %H:%M') if u.last_login else 'Sin ingresos' }}
                </td>
                <td style="text-align: right;">
                    <div style="display: inline-flex; align-items: center; gap: 6px;">
                        <button class="btn btn-outline btn-sm" style="padding: 4px 8px; font-size: 12px;" onclick='openEditUserModal({{ u.to_dict() | tojson }})' title="Configurar Permisos y Datos">
                            <span class="material-icons-round" style="font-size: 16px;">tune</span>
                            <span style="font-size: 11.5px;">Permisos</span>
                        </button>
                        {% if not u.is_super_admin %}
                        <button class="btn btn-outline btn-sm" style="padding: 4px 8px; font-size: 12px; color: {% if u.is_active %}#d97706{% else %}#059669{% endif %};" onclick="toggleUserStatus({{ u.id }})" title="{% if u.is_active %}Pausar cuenta temporalmente{% else %}Reactivar cuenta{% endif %}">
                            <span class="material-icons-round" style="font-size: 16px;">{% if u.is_active %}pause_circle{% else %}play_circle{% endif %}</span>
                        </button>
```

```
                        <button class="btn btn-danger btn-sm" style="padding: 4px 8px; font-size: 12px;" onclick="deleteUser({{ u.id }}, '{{ u.username }}')" title="Eliminar Usuario">
                            <span class="material-icons-round" style="font-size: 16px;">delete</span>
                        </button>
                        {% endif %}
                    </div>
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>

<!-- MODAL 1: CREAR NUEVO USUARIO CON PERMISOS GRANULARES -->
<div class="modal-backdrop" id="modal-create-user" onclick="handleBackdropClick(event, 'modal-create-user')">
    <div class="modal-dialog-box" style="max-width: 680px;" onclick="event.stopPropagation()">
        <div class="modal-header-box">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="color: var(--accent-success); font-size: 24px;">person_add</span>
                <h3 style="margin: 0; font-size: 16px;">Registrar Nuevo Usuario y Asignar Permisos</h3>
            </div>
            <button class="btn btn-outline btn-sm" style="padding: 2px 6px; border: none;" onclick="closeModal('modal-create-user')">
                <span class="material-icons-round">close</span>
            </button>
        </div>
        <form id="form-create-user" onsubmit="handleCreateUser(event)">
            <div class="modal-body-box" style="max-height: 75vh; overflow-y: auto;">

                <!-- FOTO DE PERFIL / AVATAR -->
                <div style="display: flex; align-items: center; gap: 16px; margin-bottom: 14px; background: var(--bg-card-subtle); padding: 12px 14px; border-radius: 10px; border: 1px solid var(--border-color);">
                    <div id="create-avatar-preview-box" style="width: 52px; height: 52px; border-radius: 50%; background: var(--border-color); display: flex; align-items: center; justify-content: center; overflow: hidden; flex-shrink: 0; border: 2px solid var(--accent-success);">
                        <span class="material-icons-round" id="create-avatar-default-icon" style="font-size: 28px; color: var(--text-muted);">person</span>
                        <img id="create-avatar-img-preview" src="" alt="Avatar" style="width: 100%; height: 100%; object-fit: cover; display: none;">
                    </div>
                    <div style="flex: 1;">
                        <div style="font-size: 12.5px; font-weight: 700; color: var(--text-heading); margin-bottom: 2px;">Foto de Perfil (Opcional)</div>
                        <div style="font-size: 11px; color: var(--text-muted); margin-bottom: 6px;">Formatos soportados: PNG, JPG, WebP.</div>
                        <div style="display: flex; gap: 8px;">
                            <button type="button" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px;" onclick="document.getElementById('create-avatar-input').click()">
                                <span class="material-icons-round" style="font-size: 14px;">photo_camera</span>
                                <span>Seleccionar Foto</span>
                            </button>
                            <button type="button" id="create-avatar-clear-btn" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px; display: none; color: var(--accent-danger);" onclick="clearCreateAvatarPreview()">
                                <span class="material-icons-round" style="font-size: 14px;">delete</span>
                                <span>Quitar</span>
                            </button>
                        </div>
                        <input type="file" id="create-avatar-input" accept="image/*" style="display: none;" onchange="handleCreateAvatarChange(this)">
                    </div>
                </div>

                <!-- DATOS GENERALES -->
                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 12px; margin-bottom: 12px;">
```

```
                    <div class="form-group-field" style="margin-bottom: 0;">
                        <label style="font-weight: 700; font-size: 12px;">Nombre de Usuario (Login)</label>
                        <input type="text" id="create-username" class="form-input-control" required placeholder="ej. cperez">
                    </div>
                    <div class="form-group-field" style="margin-bottom: 0;">
                        <label style="font-weight: 700; font-size: 12px;">Nombre Completo</label>
                        <input type="text" id="create-fullname" class="form-input-control" required placeholder="ej. Cristian Pérez">
                    </div>
                </div>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 12px; margin-bottom: 14px;">
                    <div class="form-group-field" style="margin-bottom: 0;">
                        <label style="font-weight: 700; font-size: 12px;">Correo Electrónico (Opcional)</label>
                        <input type="email" id="create-email" class="form-input-control" placeholder="ej. cperez@sifma.local">
                    </div>
                    <div class="form-group-field" style="margin-bottom: 0;">
                        <label style="font-weight: 700; font-size: 12px;">Contraseña Inicial</label>
                        <input type="password" id="create-password" class="form-input-control" required placeholder="Mínimo 4 caracteres">
                    </div>
                </div>

                <div class="form-group-field" style="margin-bottom: 16px;">
                    <label style="font-weight: 700; font-size: 12px;">Rol Base</label>
                    <select id="create-role" class="form-input-control" onchange="onRoleChange('create', this.value)" required>
                        <option value="investigador" selected>Investigador / Agrónomo (Análisis, Bitácora y Gráficas)</option>
                        <option value="operador">Técnico de Campo (Capturas USB y Sensores)</option>
                        <option value="personalizado">Rol Personalizado (Configuración manual de chulitos)</option>
                    </select>
                </div>

                <!-- MATRIZ DE CHULITOS / PERMISOS GRANULARES -->
                <div style="background: var(--bg-card-subtle); border: 1px solid var(--border-color); border-radius: 10px; padding: 14px 16px; margin-bottom: 6px;">
                    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px; flex-wrap: wrap; gap: 8px;">
                        <div>
                            <span style="font-size: 13px; font-weight: 800; color: var(--text-heading); display: block;">Permisos de Acceso a Funciones</span>
                            <span style="font-size: 11px; color: var(--text-muted);">Marca las casillas (chulitos) para habilitar las pestañas a este usuario</span>
                        </div>
                        <div style="display: flex; gap: 6px;">
                            <button type="button" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px;" onclick="setAllPerms('create', true)">Marcar Todos</button>
                            <button type="button" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px;" onclick="setAllPerms('create', false)">Desmarcar Todos</button>
                        </div>
                    </div>

                    <div style="display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap: 8px;" id="create-perms-grid">
                        {% for p_key, p_name, p_icon, p_desc in all_permissions_list %}
                        <label style="display: flex; align-items: flex-start; gap: 8px; background: #ffffff; padding: 8px 10px; border-radius: 8px; border: 1px solid var(--border-color); cursor: pointer; transition: all 0.2s;" class="perm-checkbox-card">
                            <input type="checkbox" name="create_perm_{{ p_key }}" id="create_perm_{{ p_key }}" value="{{ p_key }}" style="margin-top: 3px; cursor: pointer; accent-color: var(--accent-success);" {% if p_key in ['dashboard', 'calendar', 'gallery', 'timelapse', 'sensors', 'cross_analysis', 'export_data'] %}checked{% endif %}>
                            <div>
                                <div style="display: flex; align-items: center; gap: 4px; font-size: 12px; font-weight: 700; color: var(--text-heading);">
                                    <span class="material-icons-round" style="font-size: 15px; color: var(--accent-success);">{{ p_icon }}</span>
                                    <span>{{ p_name }}</span>
                                </div>
                                <span style="font-size: 10px; color: var(--text-muted); display: block; line-height: 1.2; margin-top: 2px;">{{ p_desc }}</span>
```

```
                            </div>
                        </label>
                        {% endfor %}
                    </div>
                </div>

            </div>
            <div class="modal-footer-box">
                <button type="button" class="btn btn-outline btn-sm" onclick="closeModal('modal-create-user')">Cancelar</button>
                <button type="submit" class="btn btn-primary btn-sm">Crear Usuario con Permisos</button>
            </div>
        </form>
    </div>
</div>

<!-- MODAL 2: EDITAR USUARIO Y CONFIGURAR PERMISOS -->
<div class="modal-backdrop" id="modal-edit-user" onclick="handleBackdropClick(event, 'modal-edit-user')">
    <div class="modal-dialog-box" style="max-width: 680px;" onclick="event.stopPropagation()">
        <div class="modal-header-box">
            <div style="display: flex; align-items: center; gap: 8px;">
                <span class="material-icons-round" style="color: var(--accent-success); font-size: 24px;">tune</span>
                <h3 style="margin: 0; font-size: 16px;">Editar Usuario y Permisos de Acceso</h3>
            </div>
            <button class="btn btn-outline btn-sm" style="padding: 2px 6px; border: none;" onclick="closeModal('modal-edit-user')">
                <span class="material-icons-round">close</span>
            </button>
        </div>
        <form id="form-edit-user" onsubmit="handleEditUser(event)">
            <input type="hidden" id="edit-user-id">
            <div class="modal-body-box" style="max-height: 75vh; overflow-y: auto;">

                <!-- FOTO DE PERFIL / AVATAR -->
                <div style="display: flex; align-items: center; gap: 16px; margin-bottom: 14px; background: var(--bg-card-subtle); padding: 12px 14px; border-radius:
10px; border: 1px solid var(--border-color);">
                    <div id="edit-avatar-preview-box" style="width: 52px; height: 52px; border-radius: 50%; background: linear-gradient(135deg, #059669, #10b981);
display: flex; align-items: center; justify-content: center; overflow: hidden; flex-shrink: 0; border: 2px solid var(--accent-success);">
                        <span id="edit-avatar-initials-preview" style="font-size: 16px; font-weight: 800; color: #ffffff;"></span>
                        <img id="edit-avatar-img-preview" src="" alt="Avatar" style="width: 100%; height: 100%; object-fit: cover; display: none;">
                    </div>
                    <div style="flex: 1;">
                        <div style="font-size: 12.5px; font-weight: 700; color: var(--text-heading); margin-bottom: 2px;">Foto de Perfil</div>
                        <div style="font-size: 11px; color: var(--text-muted); margin-bottom: 6px;">Personaliza la imagen visible en la barra lateral y tablas.</div>
                        <div style="display: flex; gap: 8px;">
                            <button type="button" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px;" onclick="document.getElementById('edit-
avatar-input').click()">
                                <span class="material-icons-round" style="font-size: 14px;">photo_camera</span>
                                <span>Cambiar Foto</span>
                            </button>
                            <button type="button" id="edit-avatar-remove-btn" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px; display: none;
color: var(--accent-danger);" onclick="handleDeleteAvatar()">
                                <span class="material-icons-round" style="font-size: 14px;">delete</span>
                                <span>Quitar Foto</span>
                            </button>
                        </div>
                        <input type="file" id="edit-avatar-input" accept="image/*" style="display: none;" onchange="handleEditAvatarChange(this)">
                    </div>
                </div>
```

```
                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 12px; margin-bottom: 12px;">
                    <div class="form-group-field" style="margin-bottom: 0;">
                        <label style="font-weight: 700; font-size: 12px;">Nombre Completo</label>
                        <input type="text" id="edit-fullname" class="form-input-control" required>
                    </div>
                    <div class="form-group-field" style="margin-bottom: 0;">
                        <label style="font-weight: 700; font-size: 12px;">Correo Electrónico</label>
                        <input type="email" id="edit-email" class="form-input-control">
                    </div>
                </div>

                <div class="form-group-field" style="margin-bottom: 14px;">
                    <label style="font-weight: 700; font-size: 12px;">Rol y Nivel de Acceso</label>
                    <select id="edit-role" class="form-input-control" onchange="onRoleChange('edit', this.value)" required>
                        <option value="investigador">Investigador / Agrónomo</option>
                        <option value="operador">Técnico de Campo</option>
                        <option value="personalizado">Acceso Personalizado</option>
                        <option value="admin" id="edit-role-admin-opt" disabled>Super Usuario / Administrador Principal (Control Total)</option>
                    </select>
                </div>

                <!-- MATRIZ DE CHULITOS / PERMISOS GRANULARES -->
                <div id="edit-perms-container" style="background: var(--bg-card-subtle); border: 1px solid var(--border-color); border-radius: 10px; padding: 14px 16px;
margin-bottom: 14px;">
                    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px; flex-wrap: wrap; gap: 8px;">
                        <div>
                            <span style="font-size: 13px; font-weight: 800; color: var(--text-heading); display: block;">Permisos de Acceso Asignados</span>
                            <span style="font-size: 11px; color: var(--text-muted);">Activa o desactiva las funciones individuales para este usuario</span>
                        </div>
                        <div style="display: flex; gap: 6px;" id="edit-perms-actions">
                            <button type="button" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px;" onclick="setAllPerms('edit', true)">Marcar
Todos</button>
                            <button type="button" class="btn btn-outline btn-sm" style="padding: 3px 8px; font-size: 11px;" onclick="setAllPerms('edit',
false)">Desmarcar Todos</button>
                        </div>
                    </div>

                    <div style="display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap: 8px;" id="edit-perms-grid">
                        {% for p_key, p_name, p_icon, p_desc in all_permissions_list %}
                        <label style="display: flex; align-items: flex-start; gap: 8px; background: #ffffff; padding: 8px 10px; border-radius: 8px; border: 1px solid
var(--border-color); cursor: pointer; transition: all 0.2s;" class="perm-checkbox-card">
                            <input type="checkbox" name="edit_perm_{{ p_key }}" id="edit_perm_{{ p_key }}" value="{{ p_key }}" style="margin-top: 3px; cursor: pointer;
accent-color: var(--accent-success);">
                            <div>
                                <div style="display: flex; align-items: center; gap: 4px; font-size: 12px; font-weight: 700; color: var(--text-heading);">
                                    <span class="material-icons-round" style="font-size: 15px; color: var(--accent-success);">{{ p_icon }}</span>
                                    <span>{{ p_name }}</span>
                                </div>
                                <span style="font-size: 10px; color: var(--text-muted); display: block; line-height: 1.2; margin-top: 2px;">{{ p_desc }}</span>
                            </div>
                        </label>
                        {% endfor %}
                    </div>
                </div>

                <div class="form-group-field" style="border-top: 1px solid var(--border-light); padding-top: 12px; margin-bottom: 0;">
```

```
                    <label style="font-weight: 700; font-size: 12px;">Restablecer Contraseña (Opcional)</label>
                    <input type="password" id="edit-password" class="form-input-control" placeholder="Dejar en blanco para mantener la contraseña actual">
                </div>
            </div>
            <div class="modal-footer-box">
                <button type="button" class="btn btn-outline btn-sm" onclick="closeModal('modal-edit-user')">Cancelar</button>
                <button type="submit" class="btn btn-primary btn-sm">Actualizar Cambios y Permisos</button>
            </div>
        </form>
    </div>
</div>

{% endblock %}

{% block scripts %}
<script>
const MODULE_KEYS = [
    {% for p_key, _, _, _ in all_permissions_list %}
    '{{ p_key }}',
    {% endfor %}
];

const PRESETS = {
    investigador: {
        dashboard: true,
        calendar: true,
        processing: false,
        gallery: true,
        timelapse: true,
        sensors: true,
        cross_analysis: true,
        config: false,
        export_data: true
    },
    operador: {
        dashboard: true,
        calendar: true,
        processing: true,
        gallery: true,
        timelapse: false,
        sensors: true,
        cross_analysis: false,
        config: false,
        export_data: false
    },
    admin: {
        dashboard: true,
        calendar: true,
        processing: true,
        gallery: true,
        timelapse: true,
        sensors: true,
        cross_analysis: true,
        config: true,
        export_data: true
```

```
    }
};

function setAllPerms(prefix, checked) {
    MODULE_KEYS.forEach(k => {
        const el = document.getElementById(`${prefix}_perm_${k}`);
        if (el && !el.disabled) el.checked = checked;
    });
}

function applyPreset(prefix, roleKey) {
    const preset = PRESETS[roleKey];
    if (preset) {
        MODULE_KEYS.forEach(k => {
            const el = document.getElementById(`${prefix}_perm_${k}`);
            if (el) el.checked = !!preset[k];
        });
    }
}

function onRoleChange(prefix, roleValue) {
    if (roleValue === 'investigador' || roleValue === 'operador' || roleValue === 'admin') {
        applyPreset(prefix, roleValue);
    }
}

function handleCreateAvatarChange(input) {
    if (input.files && input.files[0]) {
        const file = input.files[0];
        const reader = new FileReader();
        reader.onload = function(e) {
            const img = document.getElementById('create-avatar-img-preview');
            img.src = e.target.result;
            img.style.display = 'block';
            document.getElementById('create-avatar-default-icon').style.display = 'none';
            document.getElementById('create-avatar-clear-btn').style.display = 'inline-flex';
        };
        reader.readAsDataURL(file);
    }
}

function clearCreateAvatarPreview() {
    const input = document.getElementById('create-avatar-input');
    if (input) input.value = '';
    const img = document.getElementById('create-avatar-img-preview');
    if (img) {
        img.src = '';
        img.style.display = 'none';
    }
    const defIcon = document.getElementById('create-avatar-default-icon');
    if (defIcon) defIcon.style.display = 'block';
    const clearBtn = document.getElementById('create-avatar-clear-btn');
    if (clearBtn) clearBtn.style.display = 'none';
}

```

```
function handleEditAvatarChange(input) {
    if (input.files && input.files[0]) {
        const file = input.files[0];
        const reader = new FileReader();
        reader.onload = function(e) {
            const img = document.getElementById('edit-avatar-img-preview');
            img.src = e.target.result;
            img.style.display = 'block';
            document.getElementById('edit-avatar-initials-preview').style.display = 'none';
            document.getElementById('edit-avatar-remove-btn').style.display = 'inline-flex';
        };
        reader.readAsDataURL(file);
    }
}

async function handleDeleteAvatar() {
    const userId = document.getElementById('edit-user-id').value;
    if (!userId) return;
    if (!confirm('¿Deseas eliminar la foto de perfil y volver a las iniciales?')) return;

    try {
        const res = await fetch(`/api/users/${userId}/avatar`, { method: 'DELETE' });
        const data = await res.json();
        if (data.status === 'success') {
            const img = document.getElementById('edit-avatar-img-preview');
            img.src = '';
            img.style.display = 'none';
            const initSpan = document.getElementById('edit-avatar-initials-preview');
            initSpan.textContent = data.avatar_initials || 'US';
            initSpan.style.display = 'block';
            document.getElementById('edit-avatar-remove-btn').style.display = 'none';
            document.getElementById('edit-avatar-input').value = '';

            if (window.showToast) {
                window.showToast({
                    type: 'info',
                    title: 'Foto Eliminada',
                    message: data.message
                });
            }
            setTimeout(() => window.location.reload(), 800);
        } else {
            alert('Error: ' + data.message);
        }
    } catch (err) {
        alert('Error al eliminar avatar.');
    }
}

function openCreateUserModal() {
    document.getElementById('form-create-user').reset();
    clearCreateAvatarPreview();
    document.getElementById('create-role').value = 'investigador';
    applyPreset('create', 'investigador');
    document.getElementById('modal-create-user').classList.add('show');
```

```
}

function openEditUserModal(userData) {
    document.getElementById('edit-user-id').value = userData.id;
    document.getElementById('edit-fullname').value = userData.full_name;
    document.getElementById('edit-email').value = userData.email || '';
    document.getElementById('edit-password').value = '';
    document.getElementById('edit-avatar-input').value = '';

    // Manejar vista previa de avatar
    const imgPreview = document.getElementById('edit-avatar-img-preview');
    const initPreview = document.getElementById('edit-avatar-initials-preview');
    const removeBtn = document.getElementById('edit-avatar-remove-btn');

    if (userData.avatar_url) {
        imgPreview.src = userData.avatar_url;
        imgPreview.style.display = 'block';
        initPreview.style.display = 'none';
        removeBtn.style.display = 'inline-flex';
    } else {
        imgPreview.src = '';
        imgPreview.style.display = 'none';
        initPreview.textContent = userData.avatar_initials || 'US';
        initPreview.style.display = 'block';
        removeBtn.style.display = 'none';
    }

    const roleSelect = document.getElementById('edit-role');
    const adminOpt = document.getElementById('edit-role-admin-opt');

    if (userData.is_super_admin) {
        adminOpt.disabled = false;
        roleSelect.value = 'admin';
        roleSelect.disabled = true;
        setAllPerms('edit', true);
        MODULE_KEYS.forEach(k => {
            const el = document.getElementById(`edit_perm_${k}`);
            if (el) el.disabled = true;
        });
        document.getElementById('edit-perms-actions').style.display = 'none';
    } else {
        adminOpt.disabled = true;
        roleSelect.disabled = false;
        roleSelect.value = userData.role || 'personalizado';
        document.getElementById('edit-perms-actions').style.display = 'flex';

        MODULE_KEYS.forEach(k => {
            const el = document.getElementById(`edit_perm_${k}`);
            if (el) {
                el.disabled = false;
                el.checked = !!(userData.permissions && userData.permissions[k]);
            }
        });
    }

```

```
    document.getElementById('modal-edit-user').classList.add('show');
}

function closeModal(modalId) {
    document.getElementById(modalId).classList.remove('show');
}

function handleBackdropClick(e, modalId) {
    if (e.target.id === modalId) {
        closeModal(modalId);
    }
}

function getCheckedPermissions(prefix) {
    const res = {};
    MODULE_KEYS.forEach(k => {
        const el = document.getElementById(`${prefix}_perm_${k}`);
        res[k] = el ? el.checked : false;
    });
    return res;
}

async function handleCreateUser(e) {
    e.preventDefault();
    const perms = getCheckedPermissions('create');
    const formData = new FormData();
    formData.append('username', document.getElementById('create-username').value.trim());
    formData.append('full_name', document.getElementById('create-fullname').value.trim());
    formData.append('email', document.getElementById('create-email').value.trim());
    formData.append('role', document.getElementById('create-role').value);
    formData.append('password', document.getElementById('create-password').value);
    formData.append('permissions', JSON.stringify(perms));

    const avatarFile = document.getElementById('create-avatar-input').files[0];
    if (avatarFile) {
        formData.append('avatar', avatarFile);
    }

    try {
        const res = await fetch('/api/users', {
            method: 'POST',
            body: formData
        });
        const data = await res.json();
        if (data.status === 'success') {
            if (window.showToast) {
                window.showToast({
                    type: 'success',
                    title: 'Usuario Creado',
                    message: data.message || `Usuario registrado correctamente con foto de perfil y permisos.`
                });
            } else {
                alert(data.message);
            }
            setTimeout(() => window.location.reload(), 1000);
```

```
        } else {
            alert('Error: ' + data.message);
        }
    } catch (err) {
        alert('Error al crear usuario.');
    }
}

async function handleEditUser(e) {
    e.preventDefault();
    const userId = document.getElementById('edit-user-id').value;
    const perms = getCheckedPermissions('edit');
    const roleSelect = document.getElementById('edit-role');

    const payload = {
        full_name: document.getElementById('edit-fullname').value.trim(),
        email: document.getElementById('edit-email').value.trim(),
        role: roleSelect.disabled ? 'admin' : roleSelect.value,
        permissions: perms
    };
    const newPass = document.getElementById('edit-password').value;
    const avatarFile = document.getElementById('edit-avatar-input').files[0];

    try {
        const res = await fetch(`/api/users/${userId}`, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        const data = await res.json();

        // Si se seleccionó una nueva foto de avatar
        if (avatarFile) {
            const avatarFormData = new FormData();
            avatarFormData.append('avatar', avatarFile);
            await fetch(`/api/users/${userId}/avatar`, {
                method: 'POST',
                body: avatarFormData
            });
        }

        if (newPass && newPass.length >= 4) {
            await fetch(`/api/users/${userId}/reset_password`, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ password: newPass })
            });
        }

        if (data.status === 'success') {
            if (window.showToast) {
                window.showToast({
                    type: 'success',
                    title: 'Usuario Actualizado',
                    message: 'Los datos, foto de perfil y permisos han sido actualizados con éxito.'
```

```
                });
            } else {
                alert('Usuario actualizado con éxito.');
            }
            setTimeout(() => window.location.reload(), 1000);
        } else {
            alert('Error: ' + data.message);
        }
    } catch (err) {
        alert('Error al actualizar usuario.');
    }
}

async function toggleUserStatus(userId) {
    try {
        const res = await fetch(`/api/users/${userId}/toggle_active`, { method: 'POST' });
        const data = await res.json();
        if (data.status === 'success') {
            if (window.showToast) {
                window.showToast({
                    type: 'info',
                    title: 'Estado de Cuenta',
                    message: data.message
                });
            }
            setTimeout(() => window.location.reload(), 800);
        } else {
            alert('Error: ' + data.message);
        }
    } catch (err) {
        alert('Error de conexión.');
    }
}

async function deleteUser(userId, username) {
    if (!confirm(`¿Estás seguro de que deseas eliminar permanentemente la cuenta de '@${username}'?`)) return;
    try {
        const res = await fetch(`/api/users/${userId}`, { method: 'DELETE' });
        const data = await res.json();
        if (data.status === 'success') {
            if (window.showToast) {
                window.showToast({
                    type: 'warning',
                    title: 'Usuario Eliminado',
                    message: data.message
                });
            }
            setTimeout(() => window.location.reload(), 800);
        } else {
            alert('Error: ' + data.message);
        }
    } catch (err) {
        alert('Error al eliminar usuario.');
    }
}
```

```
</script>
{% endblock %}
```

ARCHIVO 75/85: raspi_node/calibrate_cameras.py (519 líneas)

```
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import sys
import time
import threading
import subprocess
import shutil
import tkinter as tk
from tkinter import messagebox

try:
    import cv2
    import numpy as np
    HAS_OPENCV = True
except ImportError:
    HAS_OPENCV = False

try:
    from PIL import Image, ImageTk
    HAS_PIL = True
except ImportError:
    HAS_PIL = False

HAS_PICAMERA2 = False
try:
    from picamera2 import Picamera2
    HAS_PICAMERA2 = True
except (ImportError, OSError):
    pass

class CameraCalibrationApp(tk.Tk):
    """
    Herramienta Independiente de Calibración y Enfoque en Vivo para Cámaras Arducam (MIPI CSI).
    - Monocámara activa: Solo activa UNA cámara a la vez (Cenital 0 o Lateral 1).
    - Rotación física en tiempo real:
      * Cenital (CAM 0): Rotada 180° + Detección de ROI Circular de Canastilla Azul.
      * Lateral (CAM 1): Rotada 90° a la derecha (sentido horario) para orientación vertical del tallo.
    - Corrección de Color BGR/RGB: Elimina tonos azules no deseados en la piel y muestra colores botánicos reales.
    - Medidor de Nitidez / Enfoque en Tiempo Real (Laplacian Variance).
    - Grilla de alineación vertical de dosel y tallo.
    """
    def __init__(self):
        super().__init__()
        self.title("SIFMA - Calibración y Enfoque en Vivo (Arducam CSI)")
        self.geometry("880x680")
        self.minsize(780, 580)
        self.configure(bg="#0a0f18")

        self.current_camera_index = None
        self.is_streaming = False
```

```
        self.stream_thread = None
        self.picam2_instance = None
        self.cap_cv = None
        self.show_grid = True

        # Por defecto invertimos canales BGR a RGB para corregir el tono azul
        self.swap_rb_channels = True

        self.setup_ui()
        self.protocol("WM_DELETE_WINDOW", self.on_close)

        # Escaneo inicial no bloqueante
        self.after(500, self.diagnose_cameras)

    def setup_ui(self):
        header = tk.Frame(self, bg="#111a2e", padx=12, pady=8, highlightbackground="#1e2d4d", highlightthickness=1)
        header.pack(fill=tk.X, padx=8, pady=(8, 4))

        title_box = tk.Frame(header, bg="#111a2e")
        title_box.pack(side=tk.LEFT)
        tk.Label(title_box, text="CALIBRACIÓN Y ENFOQUE DE CÁMARAS CSI", font=("Segoe UI", 12, "bold"), fg="#10b981", bg="#111a2e").pack(anchor="w")
        self.lbl_cam_detected = tk.Label(title_box, text="Detectando puertos CSI flex...", font=("Segoe UI", 8), fg="#94a3b8", bg="#111a2e")
        self.lbl_cam_detected.pack(anchor="w")

        self.lbl_fps = tk.Label(header, text="FPS: 0 | Enfoque: --", font=("Courier", 10, "bold"), fg="#38bdf8", bg="#111a2e")
        self.lbl_fps.pack(side=tk.RIGHT)

        # 2. Barra de Selección de Cámara (Exclusión Mutua: 0 o 1)
        btn_bar = tk.Frame(self, bg="#0a0f18", padx=4, pady=4)
        btn_bar.pack(fill=tk.X, padx=8, pady=4)

        self.btn_cam0 = tk.Button(
            btn_bar,
            text="1. CÁMARA CENITAL (CAM 0: 180° + ROI CANASTILLA)",
            font=("Segoe UI", 9, "bold"),
            bg="#1e293b",
            fg="#f8fafc",
            activebackground="#059669",
            relief="flat",
            pady=10,
            command=lambda: self.switch_camera(0)
        )
        self.btn_cam0.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=3)

        self.btn_cam1 = tk.Button(
            btn_bar,
            text="2. CÁMARA LATERAL (CAM 1: ROTADA 90° DERECHA)",
            font=("Segoe UI", 9, "bold"),
            bg="#1e293b",
            fg="#f8fafc",
            activebackground="#2563eb",
            relief="flat",
            pady=10,
            command=lambda: self.switch_camera(1)
        )
```

```
        self.btn_cam1.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=3)

        self.btn_stop = tk.Button(
            btn_bar,
            text="DETENER VIDEO",
            font=("Segoe UI", 9, "bold"),
            bg="#7f1d1d",
            fg="#ffffff",
            activebackground="#991b1b",
            relief="flat",
            padx=10,
            pady=10,
            command=self.stop_current_stream
        )
        self.btn_stop.pack(side=tk.LEFT, padx=3)

        self.btn_color_fix = tk.Button(
            btn_bar,
            text="COLOR (RGB/BGR)",
            font=("Segoe UI", 8, "bold"),
            bg="#d97706",
            fg="#ffffff",
            activebackground="#b45309",
            relief="flat",
            padx=8,
            pady=10,
            command=self.toggle_color_format
        )
        self.btn_color_fix.pack(side=tk.LEFT, padx=3)

        # 3. Visor Central de Video
        self.video_container = tk.Frame(self, bg="#000000", highlightbackground="#1e2d4d", highlightthickness=2)
        self.video_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=4)

        self.lbl_video = tk.Label(self.video_container, bg="#000000", text="Seleccione una cámara arriba para iniciar transmisión en vivo", font=("Segoe UI", 11),
fg="#64748b")
        self.lbl_video.pack(fill=tk.BOTH, expand=True)

        # 4. Barra de Enfoque y Herramientas
        bottom_bar = tk.Frame(self, bg="#111a2e", padx=12, pady=8, highlightbackground="#1e2d4d", highlightthickness=1)
        bottom_bar.pack(fill=tk.X, padx=8, pady=(4, 8))

        # Indicador de Enfoque Visual
        focus_box = tk.Frame(bottom_bar, bg="#111a2e")
        focus_box.pack(side=tk.LEFT, fill=tk.X, expand=True)

        self.lbl_focus_text = tk.Label(focus_box, text="Nitidez: -- (Gire la lente Arducam hasta obtener el valor máximo)", font=("Segoe UI", 9, "bold"), fg="#f8fafc",
bg="#111a2e")
        self.lbl_focus_text.pack(anchor="w")

        self.focus_meter = tk.Canvas(focus_box, height=12, bg="#0f172a", highlightthickness=0)
        self.focus_meter.pack(fill=tk.X, pady=(2, 0))

        # Checkbox Grilla
        self.chk_grid_btn = tk.Button(bottom_bar, text="Guías: ACTIVADAS", font=("Segoe UI", 8, "bold"), bg="#065f46", fg="#fff", relief="flat", padx=8, pady=4,
command=self.toggle_grid)
```

```
        self.chk_grid_btn.pack(side=tk.RIGHT, padx=4)

        # Botón Diagnóstico
        btn_diag = tk.Button(bottom_bar, text="Escanear CSI", font=("Segoe UI", 8, "bold"), bg="#1e293b", fg="#94a3b8", relief="flat", padx=8, pady=4,
command=self.diagnose_cameras)
        btn_diag.pack(side=tk.RIGHT, padx=4)

    def toggle_color_format(self):
        self.swap_rb_channels = not self.swap_rb_channels
        state_str = "RGB (Natural)" if self.swap_rb_channels else "BGR (Directo)"
        self.btn_color_fix.configure(text=f"COLOR: {state_str}")

    def toggle_grid(self):
        self.show_grid = not self.show_grid
        self.chk_grid_btn.configure(
            text="Guías: ACTIVADAS" if self.show_grid else "Guías: DESACTIVADAS",
            bg="#065f46" if self.show_grid else "#374151"
        )

    def diagnose_cameras(self):
        found = []
        cmd_tool = shutil.which("rpicam-hello") or shutil.which("libcamera-hello")
        if cmd_tool:
            try:
                res = subprocess.run([cmd_tool, "--list-cameras"], capture_output=True, text=True, timeout=4)
                out = res.stdout + "\n" + res.stderr
                for line in out.splitlines():
                    if "available" in line.lower() or ":" in line:
                        found.append(line.strip())
            except Exception:
                pass

        if not found and HAS_PICAMERA2:
            for idx in [0, 1]:
                try:
                    p = Picamera2(idx)
                    found.append(f"CAM {idx} activa")
                    p.close()
                except Exception:
                    pass

        if found:
            self.lbl_cam_detected.configure(text=f"Hardware CSI: {' | '.join(found)}", fg="#34d399")
        else:
            self.lbl_cam_detected.configure(
                text="Aviso: CAM/DISP 1 no detectada por el kernel. Verifique flexor o agregue dtoverlay en config.txt",
                fg="#f59e0b"
            )

    def switch_camera(self, target_cam_index):
        if self.current_camera_index == target_cam_index and self.is_streaming:
            return

        self.stop_current_stream()
        time.sleep(0.3)

```

```
        self.current_camera_index = target_cam_index
        self.btn_cam0.configure(bg="#059669" if target_cam_index == 0 else "#1e293b", fg="#fff")
        self.btn_cam1.configure(bg="#2563eb" if target_cam_index == 1 else "#1e293b", fg="#fff")

        self.is_streaming = True
        self.lbl_video.configure(text=f"Conectando a Cámara {target_cam_index} por flexor CSI...", image="")

        self.stream_thread = threading.Thread(target=self._run_camera_loop, args=(target_cam_index,), daemon=True)
        self.stream_thread.start()

    def stop_current_stream(self):
        self.is_streaming = False
        if self.stream_thread and self.stream_thread.is_alive():
            self.stream_thread.join(timeout=1.0)

        if self.picam2_instance is not None:
            try:
                self.picam2_instance.stop()
                self.picam2_instance.close()
            except Exception:
                pass
            self.picam2_instance = None

        if self.cap_cv is not None:
            try:
                self.cap_cv.release()
            except Exception:
                pass
            self.cap_cv = None

        self.current_camera_index = None
        self.btn_cam0.configure(bg="#1e293b", fg="#f8fafc")
        self.btn_cam1.configure(bg="#1e293b", fg="#f8fafc")
        self.lbl_video.configure(text="Video detenido. Cámaras libres para captura regular.", image="")
        self.lbl_fps.configure(text="FPS: 0 | Enfoque: --")
        self.lbl_focus_text.configure(text="Nitidez: -- (Video detenido)", fg="#94a3b8")
        self.focus_meter.delete("all")

    def _run_camera_loop(self, cam_idx):
        use_picam2 = False

        if HAS_PICAMERA2:
            try:
                self.picam2_instance = Picamera2(cam_idx)
                preview_cfg = self.picam2_instance.create_preview_configuration(main={"size": (640, 480)})
                self.picam2_instance.configure(preview_cfg)
                self.picam2_instance.start()
                use_picam2 = True
                print(f"[CALIBRACIÓN] Picamera2 iniciada en Puerto #{cam_idx}")
            except Exception as e:
                print(f"[CALIBRACIÓN] No se pudo abrir Picamera2 en puerto {cam_idx}: {e}")
                if self.picam2_instance is not None:
                    try: self.picam2_instance.close()
                    except Exception: pass
                self.picam2_instance = None
```

```
272
273         if not use_picam2 and HAS_OPENCV:
274             try:
275                 if sys.platform.startswith("linux"):
276                     self.cap_cv = cv2.VideoCapture(cam_idx, cv2.CAP_V4L2)
277                 else:
278                     self.cap_cv = cv2.VideoCapture(cam_idx)
279 
280                 if self.cap_cv.isOpened():
281                     self.cap_cv.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
282                     self.cap_cv.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
283                     print(f"[CALIBRACIÓN] OpenCV VideoCapture abierto en Puerto #{cam_idx}")
284             except Exception as e:
285                 print(f"[CALIBRACIÓN] Error en OpenCV: {e}")
286 
287         if not use_picam2 and (self.cap_cv is None or not self.cap_cv.isOpened()):
288             cam_name = "CENITAL (CAM 0)" if cam_idx == 0 else "LATERAL (CAM 1)"
289             self.after(0, self._show_camera_error, cam_name, cam_idx)
290             return
291 
292         frame_count = 0
293         last_time = time.time()
294         fps = 0.0
295 
296         while self.is_streaming:
297             raw_frame = None
298 
299             if use_picam2 and self.picam2_instance is not None:
300                 try:
301                     raw_frame = self.picam2_instance.capture_array()
302                 except Exception:
303                     break
304             elif self.cap_cv is not None and self.cap_cv.isOpened():
305                 ret, frame_bgr = self.cap_cv.read()
306                 if ret and frame_bgr is not None:
307                     raw_frame = frame_bgr
308                 else:
309                     time.sleep(0.05)
310                     continue
311 
312             if raw_frame is None:
313                 time.sleep(0.05)
314                 continue
315 
316             if len(raw_frame.shape) == 3 and raw_frame.shape[2] == 4:
317                 raw_frame = raw_frame[:, :, :3]
318 
319             # Inversión de color BGR a RGB para corregir tonos azules
320             if self.swap_rb_channels:
321                 if len(raw_frame.shape) == 3 and raw_frame.shape[2] >= 3:
322                     frame_oriented = raw_frame[:, :, ::-1].copy()
323                 else:
324                     frame_oriented = raw_frame
325             else:
326                 frame_oriented = raw_frame.copy()
```

```
            # APLICAR ROTACIÓN FÍSICA SEGÚN CÁMARA
            if cam_idx == 0:
                # Cámara 0 Cenital: Rotar 180°
                frame_oriented = cv2.rotate(frame_oriented, cv2.ROTATE_180)
            else:
                # Cámara 1 Lateral: Rotar 90° en sentido horario
                frame_oriented = cv2.rotate(frame_oriented, cv2.ROTATE_90_CLOCKWISE)

            h, w, _ = frame_oriented.shape
            display_frame = frame_oriented.copy()

            # Calcular Nitidez sobre el área central
            gray = cv2.cvtColor(display_frame, cv2.COLOR_RGB2GRAY)
            lap_var = cv2.Laplacian(gray, cv2.CV_64F).var()

            # DIBUJAR GUÍAS DE CALIBRACIÓN
            if self.show_grid:
                if cam_idx == 0:
                    # Guía Cenital: ROI Circular Automático de la Canastilla Azul donde está la planta
                    # 1. Segmentación previa de vegetación (planta/dosel)
                    hsv = cv2.cvtColor(frame_oriented, cv2.COLOR_RGB2HSV) if self.swap_rb_channels else cv2.cvtColor(frame_oriented, cv2.COLOR_BGR2HSV)
                    lower_green = np.array([25, 30, 30])
                    upper_green = np.array([88, 255, 255])
                    mask_green = cv2.inRange(hsv, lower_green, upper_green)

                    if self.swap_rb_channels:
                        # frame_oriented is RGB -> split R, G, B
                        r_ch, g_ch, b_ch = cv2.split(frame_oriented.astype(np.float32))
                    else:
                        b_ch, g_ch, r_ch = cv2.split(frame_oriented.astype(np.float32))
                    exg = 2.0 * g_ch - r_ch - b_ch
                    exg_mask = np.uint8((exg > 15) & (g_ch > r_ch) & (g_ch > b_ch)) * 255
                    plant_mask = cv2.bitwise_or(mask_green, exg_mask)

                    # 2. Segmentación de la canastilla azul
                    lower_blue = np.array([90, 45, 35])
                    upper_blue = np.array([140, 255, 255])
                    mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)
                    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
                    mask_blue_clean = cv2.morphologyEx(mask_blue, cv2.MORPH_CLOSE, kernel)
                    mask_blue_clean = cv2.morphologyEx(mask_blue_clean, cv2.MORPH_OPEN, kernel)

                    b_contours, _ = cv2.findContours(mask_blue_clean, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

                    found_basket = False
                    if b_contours:
                        min_r = int(min(w, h) * 0.05)
                        max_r = int(min(w, h) * 0.45)
                        candidates = []

                        for c in b_contours:
                            area = cv2.contourArea(c)
                            if area < 400:
                                continue
```

```
                    (bcx, bcy), brad = cv2.minEnclosingCircle(c)
                    brad = int(brad)
                    if brad < min_r or brad > max_r:
                        continue

                    peri = cv2.arcLength(c, True)
                    circ = (4 * np.pi * area) / (peri * peri) if peri > 0 else 0
                    bx, by, bw, bh = cv2.boundingRect(c)
                    aspect = float(bw) / float(bh) if bh > 0 else 0

                    c_mask = np.zeros((h, w), dtype=np.uint8)
                    cv2.circle(c_mask, (int(bcx), int(bcy)), brad, 255, -1)
                    green_in = cv2.countNonZero(cv2.bitwise_and(plant_mask, c_mask))

                    aspect_diff = abs(1.0 - aspect)
                    score = (green_in * 5.0) + (circ * 300.0) - (aspect_diff * 150.0)
                    if bcy > h * 0.30:
                        score += 200.0
                    if by < h * 0.08:
                        score -= 500.0

                    candidates.append({'cx': int(bcx), 'cy': int(bcy), 'r': brad, 'score': score})

                if candidates:
                    best = max(candidates, key=lambda k: k['score'])
                    bcx, bcy, brad = best['cx'], best['cy'], int(best['r'] * 1.03)
                    cv2.circle(display_frame, (bcx, bcy), brad, (255, 180, 0), 2)
                    cv2.putText(display_frame, "CANASTILLA AZUL (ROI)", (max(10, bcx - 90), max(25, bcy - brad - 8)),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 180, 0), 2)
                    found_basket = True

            if not found_basket:
                # Guía circular por defecto en la zona de la planta
                cx, cy = w // 2, int(h * 0.60)
                r = int(min(w, h) * 0.22)
                cv2.circle(display_frame, (cx, cy), r, (0, 255, 255), 2)
                cv2.putText(display_frame, "ZONA OBJETIVO CANASTILLA", (cx - 100, cy - r - 8),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)

            # Cruz central suave
            cv2.line(display_frame, (w // 2 - 25, h // 2), (w // 2 + 25, h // 2), (0, 255, 0), 1)
            cv2.line(display_frame, (w // 2, h // 2 - 25), (w // 2, h // 2 + 25), (0, 255, 0), 1)

        else:
            # Guía Lateral: Eje vertical del tallo y nivel de sustrato basal
            cx = w // 2
            # Línea de medición vertical
            cv2.line(display_frame, (cx, int(h * 0.15)), (cx, int(h * 0.85)), (0, 255, 255), 2)
            cv2.line(display_frame, (cx - 25, int(h * 0.15)), (cx + 25, int(h * 0.15)), (0, 255, 255), 2)
            cv2.line(display_frame, (cx - 25, int(h * 0.85)), (cx + 25, int(h * 0.85)), (0, 255, 255), 2)

            # Línea de base del sustrato (abajo)
            cv2.line(display_frame, (int(w * 0.2), int(h * 0.85)), (int(w * 0.8), int(h * 0.85)), (0, 0, 255), 2)
            cv2.putText(display_frame, "BASE SUSTRATO / CANASTILLA", (int(w * 0.2), int(h * 0.85) + 20),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
```

```
                    cv2.putText(display_frame, "CRECIMIENTO VERTICAL ->", (cx + 10, int(h * 0.5)),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)

            # Cálculo de FPS
            frame_count += 1
            if time.time() - last_time >= 1.0:
                fps = frame_count / (time.time() - last_time)
                frame_count = 0
                last_time = time.time()

            # Convertir a imagen Tkinter
            if HAS_PIL:
                pil_img = Image.fromarray(display_frame)
                vw = self.video_container.winfo_width()
                vh = self.video_container.winfo_height()
                if vw > 100 and vh > 100:
                    pil_img.thumbnail((vw - 10, vh - 10))
                tk_img = ImageTk.PhotoImage(pil_img)
                self.after(0, self._update_frame_ui, tk_img, lap_var, fps, cam_idx)

            time.sleep(0.03)

        if self.picam2_instance is not None:
            try:
                self.picam2_instance.stop()
                self.picam2_instance.close()
            except Exception:
                pass
            self.picam2_instance = None

        if self.cap_cv is not None:
            try:
                self.cap_cv.release()
            except Exception:
                pass
            self.cap_cv = None

    def _show_camera_error(self, cam_name, cam_idx):
        self.lbl_video.configure(
            text=f"No se pudo conectar a {cam_name} en el puerto flex CAM/DISP {cam_idx}.\n\n"
                 "Causas comunes:\n"
                 "1. Cable flexor mal encajado o invertido en CAM/DISP 1.\n"
                 "2. Falta agregar el overlay de la cámara en /boot/firmware/config.txt.\n"
                 "   Ejemplo: dtoverlay=imx477,cam1\n\n"
                 "Ejecute 'rpicam-hello --list-cameras' en la terminal para diagnosticar.",
            image="",
            fg="#ef4444"
        )
        self.lbl_fps.configure(text=f"Error en Puerto {cam_idx}")

    def _update_frame_ui(self, tk_img, lap_var, fps, cam_idx):
        if not self.is_streaming:
            return

        self.lbl_video.img_ref = tk_img
```

```
        self.lbl_video.configure(image=tk_img, text="")

        if lap_var < 50:
            quality = "DESENFOCADO (Girar lente)"
            color = "#ef4444"
        elif lap_var < 150:
            quality = "ENFOQUE MEDIO"
            color = "#f59e0b"
        else:
            quality = "EXCELENTE ENFOQUE (Optimo)"
            color = "#10b981"

        cam_name = "Cenital 180° (Cam 0)" if cam_idx == 0 else "Lateral 90° (Cam 1)"
        self.lbl_fps.configure(text=f"{cam_name} | FPS: {fps:.1f} | Nitidez: {lap_var:.1f}")
        self.lbl_focus_text.configure(text=f"Índice de Nitidez: {lap_var:.1f} -> {quality}", fg=color)

        self.focus_meter.delete("all")
        bar_w = self.focus_meter.winfo_width()
        meter_fill = min(bar_w, int((lap_var / 350.0) * bar_w))
        self.focus_meter.create_rectangle(0, 0, meter_fill, 12, fill=color, width=0)

    def on_close(self):
        self.stop_current_stream()
        self.destroy()

if __name__ == "__main__":
    app = CameraCalibrationApp()
    app.mainloop()
```

ARCHIVO 76/85: raspi_node/configurar_camaras_dual.sh (67 líneas)

```
#!/bin/bash

# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

# ============================================================
# Script de Habilitacion de Camaras Duales Arducam en Raspberry Pi 5
# Configura los puertos flexores CAM/DISP 0 y CAM/DISP 1
# ============================================================

echo "======================================================"
echo "Configurando Camaras Duales CSI en Raspberry Pi 5"
echo "======================================================"

CONFIG_TXT="/boot/firmware/config.txt"
[ ! -f "$CONFIG_TXT" ] && CONFIG_TXT="/boot/config.txt"

echo "Archivo de configuracion detectado: $CONFIG_TXT"

# Preguntar o detectar tipo de sensor Arducam
echo ""
echo "Selecciona el sensor de tus camaras Arducam:"
echo "1) OV5647 (Camara V1 de 5MP / Vision Nocturna / Estandar)"
echo "2) IMX219 (Camara V2 de 8MP)"
echo "3) IMX708 (Camara V3 de 12MP / Autofocus)"
echo "4) IMX477 (Camara de Alta Calidad HQ de 12.3MP)"
echo "5) Arducam 64MP / 16MP Autofocus"
echo ""
read -p "Opcion [1-5] (Por defecto 1 - OV5647): " opt
opt=${opt:-1}

OVERLAY_NAME="ov5647"
case $opt in
    1) OVERLAY_NAME="ov5647";;
    2) OVERLAY_NAME="imx219";;
    3) OVERLAY_NAME="imx708";;
    4) OVERLAY_NAME="imx477";;
    5) OVERLAY_NAME="arducam-64mp";;
    *) OVERLAY_NAME="ov5647";;
esac

echo ""
echo "Aplicando configuracion para sensor: $OVERLAY_NAME en CAM/DISP 0 y CAM/DISP 1..."

# Respaldar config.txt
sudo cp "$CONFIG_TXT" "$CONFIG_TXT.backup_sifma"

# Limpiar posibles lineas previas para evitar duplicados
sudo sed -i '/camera_auto_detect/d' "$CONFIG_TXT"
sudo sed -i "/dtoverlay=$OVERLAY_NAME/d" "$CONFIG_TXT"

# Agregar configuracion dual
```

```
54 echo "" | sudo tee -a "$CONFIG_TXT" >/dev/null
55 echo "# SIFMA - Configuracion Camaras Duales CSI (Arducam $OVERLAY_NAME)" | sudo tee -a "$CONFIG_TXT" >/dev/null
56 echo "camera_auto_detect=0" | sudo tee -a "$CONFIG_TXT" >/dev/null
57 echo "dtoverlay=$OVERLAY_NAME,cam0" | sudo tee -a "$CONFIG_TXT" >/dev/null
58 echo "dtoverlay=$OVERLAY_NAME,cam1" | sudo tee -a "$CONFIG_TXT" >/dev/null
59
60 echo ""
61 echo "======================================================"
62 echo "Configuracion completada exitosamente en $CONFIG_TXT."
63 echo "Para que el kernel reconozca la Camara 1 en CAM/DISP 1,"
64 echo "es necesario REINICIAR la Raspberry Pi ahora."
65 echo ""
66 echo "Comando para reiniciar: sudo reboot"
67 echo "======================================================"
```

ARCHIVO 77/85: raspi_node/dependencies.py (71 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import sys
import subprocess
import importlib

RASPI_REQUIRED_PACKAGES = [
    ("flask", "flask>=3.0.0"),
    ("cv2", "opencv-python-headless>=4.8.0"),
    ("PIL", "pillow>=10.2.0"),
    ("psutil", "psutil>=5.9.0"),
    ("requests", "requests>=2.31.0")
]

def install_package(package_name: str) -> bool:
    """Instala un paquete pip detectando automáticamente flags de compatibilidad para Raspberry Pi OS / Linux."""
    print(f"[AUTO-INSTALADOR SIFMA] Instalando paquete ausente: '{package_name}'...")
    base_cmd = [sys.executable, "-m", "pip", "install", package_name]

    try:
        res = subprocess.run(base_cmd, capture_output=True, text=True)
        if res.returncode == 0:
            print(f"[AUTO-INSTALADOR SIFMA] '{package_name}' instalado exitosamente.")
            return True

        # En Raspberry Pi OS (Debian Bookworm con PEP 668), se requiere flag especial
        if "externally-managed-environment" in res.stderr.lower() or sys.platform.startswith("linux"):
            print(f"[AUTO-INSTALADOR SIFMA] Reintentando instalacion con --break-system-packages para Raspberry Pi OS...")
            res_break = subprocess.run(base_cmd + ["--break-system-packages"], capture_output=True, text=True)
            if res_break.returncode == 0:
                print(f"[AUTO-INSTALADOR SIFMA] '{package_name}' instalado exitosamente con --break-system-packages.")
                return True
            else:
                print(f"[AUTO-INSTALADOR AVISO] Error al instalar '{package_name}': {res_break.stderr}")
                return False
        else:
            print(f"[AUTO-INSTALADOR AVISO] Error al instalar '{package_name}': {res.stderr}")
            return False
    except Exception as e:
        print(f"[AUTO-INSTALADOR ERROR] Excepcion al ejecutar pip: {e}")
        return False

def ensure_dependencies():
    """Verifica todas las dependencias requeridas y las instala automáticamente si alguna falta."""
    missing = []
    for module_name, pip_name in RASPI_REQUIRED_PACKAGES:
        try:
            importlib.import_module(module_name)
        except ImportError:
            missing.append((module_name, pip_name))

```

```
    if missing:
        print("\n============================================================")
        print(f"[AUTO-INSTALADOR SIFMA] Se detectaron {len(missing)} librerias faltantes.")
        print("Instalando dependencias necesarias de forma automatica...")
        print("============================================================\n")

        for module_name, pip_name in missing:
            ok = install_package(pip_name)
            if ok:
                try:
                    importlib.invalidate_caches()
                    importlib.import_module(module_name)
                except Exception:
                    pass
        print("\n[AUTO-INSTALADOR SIFMA] Verificacion de dependencias completada.\n")

if __name__ == "__main__":
    ensure_dependencies()
```

ARCHIVO 78/85: raspi_node/install_autostart.sh (80 líneas)

```
#!/bin/bash

# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

# ==================================================================
# Script de Instalacion SIFMA LITE en Raspberry Pi (Bajo Consumo)
# ==================================================================

echo "======================================================"
echo "Instalando servicio SIFMA LITE para Raspberry Pi (2GB RAM)"
echo "======================================================"

CURRENT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" >/dev/null 2>&1 && pwd )"
CURRENT_USER=$(logname || echo $USER)

echo "Usuario detectado: $CURRENT_USER"
echo "Directorio de trabajo: $CURRENT_DIR"

# 1. Asegurar dependencias del sistema y Python (incluye Tkinter)
echo "Verificando paquetes del sistema..."
sudo apt-get update -y
sudo apt-get install -y python3 python3-pip python3-dev python3-venv python3-tk python3-pil python3-pil.imagetk python3-picamera2 rpicam-apps libcamera-apps

echo "Instalando dependencias de Python..."
pip3 install -r $CURRENT_DIR/requirements.txt --break-system-packages 2>/dev/null || pip3 install -r $CURRENT_DIR/requirements.txt

# 2. Configurar servicio del Demonio de Captura (sifma-capture.service)
CAPTURE_SERVICE="/etc/systemd/system/sifma-capture.service"
cat <<EOF > /tmp/sifma-capture.service
[Unit]
Description=Servicio de Captura Fotografica Autonoma SIFMA (Lite)
After=local-fs.target time-sync.target
Wants=local-fs.target

[Service]
Type=simple
User=$CURRENT_USER
WorkingDirectory=$CURRENT_DIR
ExecStart=/usr/bin/python3 $CURRENT_DIR/main_capture.py --daemon
Restart=on-failure
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF

sudo cp /tmp/sifma-capture.service $CAPTURE_SERVICE
sudo chmod 644 $CAPTURE_SERVICE

```

```
# 3. Crear accesos directos en el Escritorio para el Panel Táctil y el Calibrador de Cámaras
if [ -d "/home/$CURRENT_USER/Desktop" ]; then
    sed "s|/home/nodo1/raspi_node|$CURRENT_DIR|g" $CURRENT_DIR/SIFMA_Control.desktop > /home/$CURRENT_USER/Desktop/SIFMA_Control.desktop
    sed "s|/home/nodo1/raspi_node|$CURRENT_DIR|g" $CURRENT_DIR/SIFMA_Calibracion.desktop > /home/$CURRENT_USER/Desktop/SIFMA_Calibracion.desktop
    chmod +x /home/$CURRENT_USER/Desktop/SIFMA_Control.desktop
    chmod +x /home/$CURRENT_USER/Desktop/SIFMA_Calibracion.desktop
    echo "Accesos directos 'SIFMA Control' y 'SIFMA Calibracion' creados en el Escritorio con rutas adaptadas a $CURRENT_DIR."
fi

# 4. Habilitar e iniciar servicio de captura
echo "Habilitando servicio de captura en segundo plano..."
sudo systemctl daemon-reload
sudo systemctl enable sifma-capture.service
sudo systemctl restart sifma-capture.service

echo ""
echo "======================================================"
echo "Instalacion completada con exito."
echo "1. Demonio de captura activo en segundo plano (consumo casi 0% CPU)."
echo "2. Panel Tactil Nativo Lite listo en el Escritorio (Consumo < 15MB RAM)."
echo "   (Hacer doble clic en 'SIFMA Control Tactil Lite' para abrir la interfaz)"
echo ""
echo "Comandos utiles:"
echo "  - Abrir panel manual:  python3 $CURRENT_DIR/panel_touch_lite.py"
echo "  - Ver estado Captura:  sudo systemctl status sifma-capture"
echo "  - Ver logs en vivo:    journalctl -u sifma-capture -f"
echo "======================================================"
```

ARCHIVO 79/85: raspi_node/main_capture.py (351 líneas)

```
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import sys
import time
import json
import argparse
import signal
from datetime import datetime, timedelta

NODE_DIR = os.path.dirname(os.path.abspath(__file__))
if NODE_DIR not in sys.path:
    sys.path.insert(0, NODE_DIR)

from dependencies import ensure_dependencies
ensure_dependencies()

from services import OfflineStorageService, CameraCaptureService

CONFIG_FILE = os.path.join(NODE_DIR, "config.json")
STATUS_FILE = os.path.join(NODE_DIR, "sifma_daemon_status.json")

def load_node_config():
    """Carga la configuración actual desde config.json con valores por defecto seguros."""
    defaults = {
        "plant_id": 1,
        "crop_type": "cebollin",
        "capture_mode": "interval",           # "interval" (cada X minutos) o "scheduled" (horas fijas)
        "interval_minutes": 60,               # Frecuencia de captura (60 = 1 hora para exhibición)
        "scheduled_times": ["07:05", "12:05", "17:05"],
        "photos_per_session": 5,              # Número de fotos por sesión (5 tomas dobles)
        "interval_between_photos_sec": 5,     # Intervalo entre fotos de la misma toma
        "boot_delay_sec": 10,                 # Retardo inicial al encender (10s para exhibición rápida)
        "capture_on_boot": True,              # Tomar fotos inmediatamente al encender
        "cenital_camera_index": 0,
        "lateral_camera_index": 1,
        "server_url": "http://192.168.0.3:5000",
        "safe_shutdown_enabled": False
    }
    if os.path.exists(CONFIG_FILE):
        try:
            with open(CONFIG_FILE, "r", encoding="utf-8") as f:
                loaded = json.load(f)
                defaults.update(loaded)
        except Exception as e:
            print(f"[CONFIG AVISO] Error al leer config.json ({e}). Usando valores predeterminados.")
    return defaults

def save_daemon_status(status_text="En espera", last_capture_info=None, next_capture_dt=None, is_running=True):
    """Guarda el estado operativo en un archivo JSON para que el Panel de Control Web lo consulte en vivo."""
    try:
```

```
        now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        status_data = {
            "status": status_text,
            "is_running": is_running,
            "updated_at": now_str,
            "pid": os.getpid(),
            "next_capture": next_capture_dt.strftime("%Y-%m-%d %H:%M:%S") if next_capture_dt else None,
            "next_capture_human": next_capture_dt.strftime("%H:%M:%S") if next_capture_dt else "N/A"
        }
        if last_capture_info:
            status_data["last_capture"] = last_capture_info
        elif os.path.exists(STATUS_FILE):
            try:
                with open(STATUS_FILE, "r", encoding="utf-8") as f_prev:
                    prev = json.load(f_prev)
                    if "last_capture" in prev:
                        status_data["last_capture"] = prev["last_capture"]
            except Exception:
                pass

        with open(STATUS_FILE, "w", encoding="utf-8") as f:
            json.dump(status_data, f, indent=2)
    except Exception as e:
        pass

def determine_period_by_time(dt=None):
    """
    Determina el nombre canónico del período según la hora:
    - 06:00 a 10:59 -> manana
    - 11:00 a 14:59 -> medio_dia
    - 15:00 en adelante -> tarde
    Incluye la hora exacta en el sufijo si son tomas horarias continuas para no sobreescribir lotes.
    """
    if dt is None:
        dt = datetime.now()
    hour = dt.hour
    time_tag = dt.strftime("%H%M")

    if hour < 11:
        base = "manana"
    elif hour < 15:
        base = "medio_dia"
    else:
        base = "tarde"

    return f"{base}_{time_tag}"

def execute_capture_sequence(
    period=None,
    crop_type=None,
    plant_id=None,
    photos_count=None,
    photo_interval_sec=None,
    cenital_cam=0,
    lateral_cam=1
```

```
):
    """
    Secuencia autónoma de tomas fotográficas (cenital y lateral) guardadas en la USB.
    Registra metadatos de alta resolución y actualiza el estado en vivo.
    """
    cfg = load_node_config()
    crop_type = crop_type or cfg.get("crop_type", "cebollin")
    plant_id = plant_id if plant_id is not None else cfg.get("plant_id", 1)
    photos_count = photos_count or cfg.get("photos_per_session", 3)
    photo_interval_sec = photo_interval_sec if photo_interval_sec is not None else cfg.get("interval_between_photos_sec", 5)
    cenital_cam = cenital_cam if cenital_cam is not None else cfg.get("cenital_camera_index", 0)
    lateral_cam = lateral_cam if lateral_cam is not None else cfg.get("lateral_camera_index", 1)

    session_start_dt = datetime.now()

    if not period:
        period = determine_period_by_time(session_start_dt)

    print("\n============================================================")
    print("SIFMA Raspberry Pi 5 - Captura Fotografica Autonoma")
    print(f"Inicio de Sesion: {session_start_dt.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"Periodo: {period.upper()} | Canastilla: #{plant_id} ({crop_type})")
    print(f"Configuracion: {photos_count} fotos | Intervalo: {photo_interval_sec}s")
    print(f"Camaras: Cenital (ID {cenital_cam}) / Lateral (ID {lateral_cam})")
    print("============================================================\n")

    save_daemon_status(status_text=f"Capturando fotos del lote: {period}...")

    print("1. Verificando unidad de almacenamiento USB...")
    batch_dir = OfflineStorageService.create_batch_directory(period=period, wait_for_usb=True)
    print(f"   Carpeta de destino: {batch_dir}")

    cenital_success = 0
    lateral_success = 0
    photos_metadata_list = []

    for photo_idx in range(1, photos_count + 1):
        if photo_idx > 1 and photo_interval_sec > 0:
            print(f"\n[INTERVALO] Esperando {photo_interval_sec}s antes de la toma #{photo_idx}...")
            save_daemon_status(status_text=f"Esperando {photo_interval_sec}s para toma #{photo_idx}/{photos_count}...")
            time.sleep(photo_interval_sec)

        exact_photo_time = datetime.now()
        print(f"\n---> [TOMA #{photo_idx}/{photos_count}] Hora exacta: {exact_photo_time.strftime('%H:%M:%S')}")
        save_daemon_status(status_text=f"Disparando foto #{photo_idx}/{photos_count} ({exact_photo_time.strftime('%H:%M:%S')})...")

        cenital_path = os.path.join(batch_dir, f"cenital_{photo_idx}.png")
        cen_ok = CameraCaptureService.capture_photo(cenital_path, camera_index=cenital_cam)
        if cen_ok:
            cenital_success += 1
            print(f"   - Foto Cenital #{photo_idx} OK.")

        time.sleep(1.0)

        lateral_path = os.path.join(batch_dir, f"lateral_{photo_idx}.png")
```

```
        lat_ok = CameraCaptureService.capture_photo(lateral_path, camera_index=lateral_cam)
        if lat_ok:
            lateral_success += 1
            print(f"   - Foto Lateral #{photo_idx} OK.")

        # Registrar metadatos precisos
        photos_metadata_list.append({
            "photo_index": photo_idx,
            "timestamp": exact_photo_time.strftime("%Y-%m-%d %H:%M:%S"),
            "exact_time": exact_photo_time.strftime("%H:%M:%S"),
            "cenital_file": f"cenital_{photo_idx}.png",
            "lateral_file": f"lateral_{photo_idx}.png",
            "cenital_ok": cen_ok,
            "lateral_ok": lat_ok
        })

    session_end_dt = datetime.now()
    total_captured = cenital_success + lateral_success

    # 3. Guardar archivo metadata.json completo
    print("\n2. Guardando archivo metadata.json con timestamps...")
    meta_file = OfflineStorageService.save_batch_metadata(
        batch_dir=batch_dir,
        period=period,
        crop_type=crop_type,
        image_files_count=total_captured,
        plant_id=plant_id,
        session_start=session_start_dt.strftime("%Y-%m-%d %H:%M:%S"),
        session_end=session_end_dt.strftime("%Y-%m-%d %H:%M:%S"),
        photos_info=photos_metadata_list
    )

    duration_sec = round((session_end_dt - session_start_dt).total_seconds(), 1)
    print("\n======================================================")
    print("Secuencia de captura finalizada con exito.")
    print(f"Duracion: {duration_sec}s | Fotos: {cenital_success} Cenitales + {lateral_success} Laterales = {total_captured} total.")
    print(f"Lote almacenado en: {batch_dir}")
    print("======================================================\n")

    last_info = {
        "timestamp": session_end_dt.strftime("%Y-%m-%d %H:%M:%S"),
        "period": period,
        "plant_id": plant_id,
        "crop_type": crop_type,
        "photos_count": total_captured,
        "batch_dir": batch_dir,
        "cenital_count": cenital_success,
        "lateral_count": lateral_success
    }
    save_daemon_status(status_text="En espera para el próximo ciclo", last_capture_info=last_info)
    return total_captured

def run_continuous_daemon():
    """
    Demonio autónomo de captura continua para Raspberry Pi 5.
```

```
    Lee dinámicamente config.json y admite:
    - Modo Intervalo (por ejemplo: cada 60 minutos / 1 hora para exhibición).
    - Modo Horas Fijas (por ejemplo: 07:05, 12:05, 17:05).
    - Captura inmediata tras el encendido si capture_on_boot es True.
    """
    cfg = load_node_config()
    boot_delay = cfg.get("boot_delay_sec", 10)
    capture_on_boot = cfg.get("capture_on_boot", True)

    print("\n=========================================================")
    print("SIFMA DEMONIO AUTONOMO - Raspberry Pi 5")
    print(f"Modo de operacion: {cfg.get('capture_mode', 'interval').upper()}")
    if cfg.get("capture_mode") == "interval":
        print(f"Frecuencia configurada: Cada {cfg.get('interval_minutes', 60)} minutos (1 hora)")
    else:
        print(f"Horarios configurados: {cfg.get('scheduled_times', [])}")
    print(f"Fotos por sesion: {cfg.get('photos_per_session', 3)} tomas")
    print("=========================================================\n")

    if boot_delay > 0:
        print(f"[ARRANQUE] Esperando retardo inicial de {boot_delay}s para estabilizacion de hardware...")
        save_daemon_status(status_text=f"Iniciando: esperando retardo de {boot_delay}s...")
        time.sleep(boot_delay)

    # Si está configurado para disparar al encender:
    if capture_on_boot:
        print("\n[ARRANQUE] Ejecutando primera sesion automatica tras encendido...")
        try:
            execute_capture_sequence()
        except Exception as e:
            print(f"[ERROR EN CAPTURA INICIAL]: {e}")

    last_trigger_time = datetime.now()
    last_scheduled_hour = None

    while True:
        try:
            # Recargar configuración fresca en cada iteración
            cfg = load_node_config()
            mode = cfg.get("capture_mode", "interval")
            now = datetime.now()

            if mode == "interval":
                interval_min = max(1, int(cfg.get("interval_minutes", 60)))
                interval_sec = interval_min * 60
                elapsed = (now - last_trigger_time).total_seconds()
                next_trigger = last_trigger_time + timedelta(seconds=interval_sec)

                remaining_sec = max(0, int(interval_sec - elapsed))
                rem_min = remaining_sec // 60
                rem_s = remaining_sec % 60

                save_daemon_status(
                    status_text=f"Modo Intervalo ({interval_min} min): Próxima toma en {rem_min}m {rem_s}s",
                    next_capture_dt=next_trigger
```

```
                )

                if elapsed >= interval_sec:
                    print(f"\n[CRON INTERVALO] Tiempo cumplido ({interval_min} min). Iniciando nueva sesion de captura...")
                    execute_capture_sequence()
                    last_trigger_time = datetime.now()

            elif mode == "scheduled":
                sched_times = cfg.get("scheduled_times", ["07:05", "12:05", "17:05"])
                current_hm = now.strftime("%H:%M")

                # Calcular la próxima hora programada
                next_time_str = None
                for t_str in sorted(sched_times):
                    if t_str > current_hm:
                        next_time_str = t_str
                        break
                if not next_time_str and sched_times:
                    next_time_str = sorted(sched_times)[0]

                save_daemon_status(
                    status_text=f"Modo Horas Fijas: Próxima sesión a las {next_time_str or 'N/A'}",
                    next_capture_dt=None
                )

                if current_hm in sched_times and (now.date(), current_hm) != last_scheduled_hour:
                    print(f"\n[CRON HORARIO] Hora programada alcanzada ({current_hm}). Iniciando sesion...")
                    execute_capture_sequence()
                    last_scheduled_hour = (now.date(), current_hm)

            time.sleep(5)

        except KeyboardInterrupt:
            print("\n[DEMONIO] Detenido por el usuario.")
            save_daemon_status(status_text="Detenido", is_running=False)
            break
        except Exception as e:
            print(f"[DEMONIO ERROR]: {e}")
            save_daemon_status(status_text=f"Error en ejecucion: {e}")
            time.sleep(10)

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="SIFMA Raspberry Pi Node - Captura Autonoma")
    parser.add_argument("--daemon", action="store_true", help="Ejecutar en modo demonio continuo con programador")
    parser.add_argument("--period", type=str, default=None, help="Forzar periodo especifico (manana, medio_dia, tarde)")
    parser.add_argument("--boot-delay", type=int, default=None, help="Segundos de espera inicial al encender")
    parser.add_argument("--interval", type=int, default=None, help="Segundos de intervalo entre fotos individuales")
    parser.add_argument("--photos", type=int, default=None, help="Cantidad de fotos por sesion")
    parser.add_argument("--crop", type=str, default=None, help="Especie de cultivo")
    parser.add_argument("--plant", type=int, default=None, help="ID de canastilla (1-4)")
    parser.add_argument("--now", action="store_true", help="Disparar captura inmediata y salir")

    args = parser.parse_args()

    if args.now:
```

```
        execute_capture_sequence(
            period=args.period,
            crop_type=args.crop,
            plant_id=args.plant,
            photos_count=args.photos,
            photo_interval_sec=args.interval
        )
    elif args.daemon:
        run_continuous_daemon()
    else:
        cfg = load_node_config()
        b_delay = args.boot_delay if args.boot_delay is not None else cfg.get("boot_delay_sec", 10)
        if b_delay > 0:
            print(f"[COOLDOWN] Esperando {b_delay}s antes de iniciar la sesion...")
            time.sleep(b_delay)
        execute_capture_sequence(
            period=args.period,
            crop_type=args.crop,
            plant_id=args.plant,
            photos_count=args.photos,
            photo_interval_sec=args.interval
        )

```

ARCHIVO 80/85: raspi_node/panel_app.py (301 líneas)

```
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import sys
import json
import time
import base64
import subprocess
import threading
import platform
from datetime import datetime
from flask import Flask, render_template, request, jsonify, send_file

NODE_DIR = os.path.dirname(os.path.abspath(__file__))
if NODE_DIR not in sys.path:
    sys.path.insert(0, NODE_DIR)

from dependencies import ensure_dependencies
ensure_dependencies()

from services.storage_service import OfflineStorageService
from services.camera_service import CameraCaptureService

CONFIG_FILE = os.path.join(NODE_DIR, "config.json")
STATUS_FILE = os.path.join(NODE_DIR, "sifma_daemon_status.json")
PREVIEW_CACHE_DIR = os.path.join(NODE_DIR, ".preview_cache")
os.makedirs(PREVIEW_CACHE_DIR, exist_ok=True)

app = Flask(__name__, template_folder=os.path.join(NODE_DIR, "templates"))

camera_lock = threading.Lock()
manual_capture_thread = None

def get_node_config():
    defaults = {
        "plant_id": 1,
        "crop_type": "cebollin",
        "capture_mode": "interval",
        "interval_minutes": 60,
        "scheduled_times": ["07:05", "12:05", "17:05"],
        "photos_per_session": 3,
        "interval_between_photos_sec": 5,
        "boot_delay_sec": 10,
        "capture_on_boot": True,
        "cenital_camera_index": 0,
        "lateral_camera_index": 1,
        "server_url": "http://192.168.0.3:5000",
        "safe_shutdown_enabled": False
    }
    if os.path.exists(CONFIG_FILE):
        try:
```

```
            with open(CONFIG_FILE, "r", encoding="utf-8") as f:
                loaded = json.load(f)
                defaults.update(loaded)
        except Exception:
            pass
    return defaults

def save_node_config(new_cfg):
    current = get_node_config()
    current.update(new_cfg)
    temp_path = CONFIG_FILE + ".tmp"
    with open(temp_path, "w", encoding="utf-8") as f:
        json.dump(current, f, indent=2)
    os.replace(temp_path, CONFIG_FILE)
    return current

def check_process_running():
    """Comprueba si el demonio main_capture.py esta en ejecucion."""
    try:
        if platform.system() != "Windows":
            res = subprocess.run(["systemctl", "is-active", "sifma-capture"], capture_output=True, text=True)
            if "active" in res.stdout.strip():
                return True

        if os.path.exists(STATUS_FILE):
            with open(STATUS_FILE, "r", encoding="utf-8") as f:
                st = json.load(f)
                pid = st.get("pid")
                if pid:
                    import psutil
                    if psutil.pid_exists(pid):
                        proc = psutil.Process(pid)
                        if "python" in proc.name().lower():
                            return True
    except Exception:
        pass
    return False

def get_storage_status():
    """Detecta estado de la memoria USB externa y espacio disponible."""
    target_dir = OfflineStorageService._find_target_storage_directory_once()
    is_usb = "SIFMA_OFFLINE_STORAGE" not in target_dir
    free_gb = 0
    total_gb = 0
    try:
        import shutil
        total, used, free = shutil.disk_usage(os.path.dirname(target_dir) if os.path.exists(target_dir) else target_dir)
        free_gb = round(free / (1024 ** 3), 1)
        total_gb = round(total / (1024 ** 3), 1)
    except Exception:
        pass

    return {
        "target_path": target_dir,
        "is_usb_mounted": is_usb,
```

```
        "free_gb": free_gb,
        "total_gb": total_gb,
        "label": "Memoria USB Conectada" if is_usb else "Almacenamiento Local Interno (Sin USB externa)"
    }

@app.route("/")
def index():
    config = get_node_config()
    storage = get_storage_status()
    return render_template("index.html", config=config, storage=storage)

@app.route("/api/status", methods=["GET"])
def api_status():
    config = get_node_config()
    storage = get_storage_status()
    daemon_running = check_process_running()

    daemon_status = {
        "status": "En espera",
        "is_running": daemon_running,
        "updated_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "last_capture": None,
        "next_capture": None
    }

    if os.path.exists(STATUS_FILE):
        try:
            with open(STATUS_FILE, "r", encoding="utf-8") as f:
                loaded_st = json.load(f)
                daemon_status.update(loaded_st)
                daemon_status["is_running"] = daemon_running or loaded_st.get("is_running", False)
        except Exception:
            pass

    return jsonify({
        "status": "ok",
        "config": config,
        "storage": storage,
        "daemon": daemon_status,
        "server_time": datetime.now().strftime("%H:%M:%S"),
        "server_date": datetime.now().strftime("%Y-%m-%d")
    })

@app.route("/api/config", methods=["POST"])
def api_save_config():
    try:
        data = request.get_json() or {}

        updates = {}
        if "plant_id" in data:
            updates["plant_id"] = int(data["plant_id"])
        if "crop_type" in data:
            updates["crop_type"] = str(data["crop_type"]).strip().lower()
        if "capture_mode" in data and data["capture_mode"] in ["interval", "scheduled"]:
            updates["capture_mode"] = str(data["capture_mode"])
```

```
        if "interval_minutes" in data:
            updates["interval_minutes"] = max(1, int(data["interval_minutes"]))
        if "photos_per_session" in data:
            updates["photos_per_session"] = max(1, min(20, int(data["photos_per_session"])))
        if "interval_between_photos_sec" in data:
            updates["interval_between_photos_sec"] = max(1, int(data["interval_between_photos_sec"]))
        if "boot_delay_sec" in data:
            updates["boot_delay_sec"] = max(0, int(data["boot_delay_sec"]))
        if "capture_on_boot" in data:
            updates["capture_on_boot"] = bool(data["capture_on_boot"])
        if "cenital_camera_index" in data:
            updates["cenital_camera_index"] = int(data["cenital_camera_index"])
        if "lateral_camera_index" in data:
            updates["lateral_camera_index"] = int(data["lateral_camera_index"])
        if "scheduled_times" in data and isinstance(data["scheduled_times"], list):
            valid_times = []
            for t in data["scheduled_times"]:
                t_str = str(t).strip()
                if len(t_str) == 5 and ":" in t_str:
                    valid_times.append(t_str)
            updates["scheduled_times"] = sorted(list(set(valid_times)))

        updated = save_node_config(updates)
        return jsonify({"status": "ok", "message": "Configuracion guardada exitosamente", "config": updated})
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 400

@app.route("/api/preset", methods=["POST"])
def api_apply_preset():
    try:
        data = request.get_json() or {}
        preset_name = data.get("preset")

        if preset_name == "exhibicion":
            # Modo Exhibicion: cada 1 hora (60m), 3 fotos rapidas (5s), cooldown 10s, captura al encender
            preset_cfg = {
                "capture_mode": "interval",
                "interval_minutes": 60,
                "photos_per_session": 3,
                "interval_between_photos_sec": 5,
                "boot_delay_sec": 10,
                "capture_on_boot": True
            }
        elif preset_name == "exhibicion_rapida":
            # Demostracion Rapida: cada 15 min, 2 fotos
            preset_cfg = {
                "capture_mode": "interval",
                "interval_minutes": 15,
                "photos_per_session": 2,
                "interval_between_photos_sec": 3,
                "boot_delay_sec": 5,
                "capture_on_boot": True
            }
        elif preset_name == "agronomico":
            # Modo Agronomico Estandar: 3 jornadas diarias (07:05, 12:05, 17:05), 5 fotos
```

```
            preset_cfg = {
                "capture_mode": "scheduled",
                "scheduled_times": ["07:05", "12:05", "17:05"],
                "photos_per_session": 5,
                "interval_between_photos_sec": 300,
                "boot_delay_sec": 300,
                "capture_on_boot": False
            }
        else:
            return jsonify({"status": "error", "message": "Preset no reconocido"}), 400

        updated = save_node_config(preset_cfg)
        return jsonify({"status": "ok", "message": f"Preset '{preset_name}' aplicado correctamente", "config": updated})
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 500

@app.route("/api/trigger_capture", methods=["POST"])
def api_trigger_capture():
    global manual_capture_thread
    if manual_capture_thread and manual_capture_thread.is_alive():
        return jsonify({"status": "error", "message": "Ya hay una secuencia de captura en ejecucion."}), 409

    def _run_bg():
        try:
            cmd = [sys.executable, os.path.join(NODE_DIR, "main_capture.py"), "--now"]
            subprocess.run(cmd, check=True)
        except Exception as e:
            print(f"[ERROR DISPARO MANUAL]: {e}")

    manual_capture_thread = threading.Thread(target=_run_bg, daemon=True)
    manual_capture_thread.start()
    return jsonify({"status": "ok", "message": "Secuencia de captura iniciada en segundo plano."})

@app.route("/api/preview_camera/<int:camera_index>", methods=["GET"])
def api_preview_camera(camera_index):
    """Toma una foto de prueba instantanea y la devuelve en base64 para alinear la camara."""
    with camera_lock:
        preview_file = os.path.join(PREVIEW_CACHE_DIR, f"preview_cam_{camera_index}.jpg")
        ok = CameraCaptureService.capture_photo(preview_file, camera_index=camera_index, max_retries=1)
        if ok and os.path.exists(preview_file):
            try:
                with open(preview_file, "rb") as f:
                    b64 = base64.b64encode(f.read()).decode("utf-8")
                return jsonify({
                    "status": "ok",
                    "camera_index": camera_index,
                    "image_b64": f"{b64}",
                    "timestamp": datetime.now().strftime("%H:%M:%S")
                })
            except Exception as e:
                return jsonify({"status": "error", "message": str(e)}), 500
        else:
            return jsonify({
                "status": "error",
                "message": f"No se pudo capturar vista previa de la Camara ID {camera_index}. Verifique la conexion del cable o modulo."
```

```
            }), 404

@app.route("/api/daemon/<action>", methods=["POST"])
def api_daemon_control(action):
    """Inicia, detiene o reinicia el demonio de captura."""
    if action not in ["start", "stop", "restart"]:
        return jsonify({"status": "error", "message": "Accion no valida"}), 400

    try:
        if platform.system() != "Windows":
            cmd = ["sudo", "systemctl", action, "sifma-capture"]
            res = subprocess.run(cmd, capture_output=True, text=True)
            if res.returncode == 0:
                return jsonify({"status": "ok", "message": f"Servicio sifma-capture ejecutado: {action}"})
            else:
                return jsonify({"status": "error", "message": res.stderr or "Error ejecutando systemctl"}), 500
        else:
            return jsonify({"status": "ok", "message": f"Accion {action} simulada en entorno de desarrollo."})
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 500

if __name__ == "__main__":
    print("\n=========================================================")
    print("SIFMA PANEL DE CONTROL TACTIL - RASPBERRY PI")
    print("Iniciando servidor web en http://0.0.0.0:8080")
    print("Acceda desde la pantalla tactil o desde un telefono/laptop en la red")
    print("=========================================================\n")
    app.run(host="0.0.0.0", port=8080, debug=False)
```

ARCHIVO 81/85: raspi_node/panel_touch_lite.py (421 líneas)

```
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import sys
import json
import time
import subprocess
import threading
import platform
from datetime import datetime

NODE_DIR = os.path.dirname(os.path.abspath(__file__))
if NODE_DIR not in sys.path:
    sys.path.insert(0, NODE_DIR)

from dependencies import ensure_dependencies
ensure_dependencies()

from services.storage_service import OfflineStorageService
from services.camera_service import CameraCaptureService

CONFIG_FILE = os.path.join(NODE_DIR, "config.json")
STATUS_FILE = os.path.join(NODE_DIR, "sifma_daemon_status.json")
PREVIEW_CACHE_DIR = os.path.join(NODE_DIR, ".preview_cache")
os.makedirs(PREVIEW_CACHE_DIR, exist_ok=True)

import tkinter as tk
from tkinter import ttk, messagebox
try:
    from PIL import Image, ImageTk
    HAS_PIL_TK = True
except ImportError:
    HAS_PIL_TK = False

def load_config():
    defaults = {
        "plant_id": 1,
        "crop_type": "cebollin",
        "capture_mode": "interval",
        "interval_minutes": 60,
        "scheduled_times": ["07:05", "12:05", "17:05"],
        "photos_per_session": 5,
        "interval_between_photos_sec": 5,
        "boot_delay_sec": 10,
        "capture_on_boot": True,
        "cenital_camera_index": 0,
        "lateral_camera_index": 1,
        "server_url": "http://192.168.0.3:5000",
        "safe_shutdown_enabled": False
    }
    if os.path.exists(CONFIG_FILE):
```

```
        try:
            with open(CONFIG_FILE, "r", encoding="utf-8") as f:
                defaults.update(json.load(f))
        except Exception:
            pass
    return defaults

def save_config(cfg):
    try:
        temp = CONFIG_FILE + ".tmp"
        with open(temp, "w", encoding="utf-8") as f:
            json.dump(cfg, f, indent=2)
        os.replace(temp, CONFIG_FILE)
    except Exception as e:
        print(f"[ERROR CONFIG]: {e}")

class SifmaLiteTouchApp(tk.Tk):
    """
    Panel de Control Táctil Ultraligero para Raspberry Pi (Consumo < 15MB RAM).
    100% interactivo mediante toques/clics. Sin navegador ni servidores pesados.
    """
    def __init__(self):
        super().__init__()
        self.title("SIFMA - Panel Táctil Lite")
        self.geometry("860x560")
        self.minsize(780, 500)
        self.configure(bg="#0a0f18")

        self.cfg = load_config()
        self.default_hours = [
            "06:00", "07:05", "08:00", "09:00", "10:00",
            "11:00", "12:05", "13:00", "14:00", "15:00",
            "16:00", "17:05", "18:00", "19:00", "20:00"
        ]

        self.setup_ui()
        self.update_ui_from_config()
        self.refresh_timer()

    def setup_ui(self):
        header_frame = tk.Frame(self, bg="#111a2e", padx=12, pady=8, highlightbackground="#1e2d4d", highlightthickness=1)
        header_frame.pack(fill=tk.X, padx=8, pady=(8, 4))

        title_box = tk.Frame(header_frame, bg="#111a2e")
        title_box.pack(side=tk.LEFT)
        tk.Label(title_box, text="SIFMA LITE", font=("Segoe UI", 14, "bold"), fg="#10b981", bg="#111a2e").pack(anchor="w")
        tk.Label(title_box, text="Centro Táctil Raspberry Pi (Bajo Consumo)", font=("Segoe UI", 8), fg="#94a3b8", bg="#111a2e").pack(anchor="w")

        status_box = tk.Frame(header_frame, bg="#111a2e")
        status_box.pack(side=tk.RIGHT)

        self.lbl_usb = tk.Label(status_box, text="USB: Verificando...", font=("Segoe UI", 9, "bold"), fg="#60a5fa", bg="#0f172a", padx=8, pady=4, relief="ridge")
        self.lbl_usb.pack(side=tk.LEFT, padx=4)

        self.lbl_daemon = tk.Label(status_box, text="Demonio: Activo", font=("Segoe UI", 9, "bold"), fg="#34d399", bg="#0f172a", padx=8, pady=4, relief="ridge")
```

```
        self.lbl_daemon.pack(side=tk.LEFT, padx=4)

        self.lbl_clock = tk.Label(status_box, text="--:--:--", font=("Courier", 11, "bold"), fg="#f8fafc", bg="#111a2e", padx=6)
        self.lbl_clock.pack(side=tk.LEFT, padx=4)

        main_box = tk.Frame(self, bg="#0a0f18")
        main_box.pack(fill=tk.BOTH, expand=True, padx=8, pady=4)

        left_col = tk.Frame(main_box, bg="#111a2e", padx=10, pady=10, highlightbackground="#1e2d4d", highlightthickness=1)
        left_col.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 4))

        tk.Label(left_col, text="PRESETS RAPIDOS A 1 CLIC", font=("Segoe UI", 9, "bold"), fg="#34d399", bg="#111a2e").pack(anchor="w", pady=(0, 4))
        preset_box = tk.Frame(left_col, bg="#111a2e")
        preset_box.pack(fill=tk.X, pady=(0, 8))

        btn_pre1 = tk.Button(preset_box, text="Modo Exhibicion\n(Cada 1h / 3 fotos)", font=("Segoe UI", 8, "bold"), bg="#064e3b", fg="#ffffff",
activebackground="#059669", activeforeground="#ffffff", relief="flat", padx=6, pady=6, command=lambda: self.apply_preset("exhibicion"))
        btn_pre1.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=2)

        btn_pre2 = tk.Button(preset_box, text="Demo Rapida\n(Cada 15m / 2 fotos)", font=("Segoe UI", 8, "bold"), bg="#1e3a8a", fg="#ffffff", activebackground="#2563eb",
activeforeground="#ffffff", relief="flat", padx=6, pady=6, command=lambda: self.apply_preset("exhibicion_rapida"))
        btn_pre2.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=2)

        btn_pre3 = tk.Button(preset_box, text="Agronomico\n(3 turnos / 5 fotos)", font=("Segoe UI", 8, "bold"), bg="#374151", fg="#ffffff", activebackground="#4b5563",
activeforeground="#ffffff", relief="flat", padx=6, pady=6, command=lambda: self.apply_preset("agronomico"))
        btn_pre3.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=2)

        tk.Label(left_col, text="TIPO DE PROGRAMADOR", font=("Segoe UI", 9, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w", pady=(4, 2))
        mode_box = tk.Frame(left_col, bg="#111a2e")
        mode_box.pack(fill=tk.X, pady=(0, 6))

        self.btn_mode_interval = tk.Button(mode_box, text="Intervalo Periodico", font=("Segoe UI", 9, "bold"), relief="flat", pady=6, command=lambda:
self.set_mode("interval"))
        self.btn_mode_interval.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=2)

        self.btn_mode_sched = tk.Button(mode_box, text="Horas Programadas", font=("Segoe UI", 9, "bold"), relief="flat", pady=6, command=lambda:
self.set_mode("scheduled"))
        self.btn_mode_sched.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=2)

        self.interval_frame = tk.Frame(left_col, bg="#111a2e")
        self.interval_frame.pack(fill=tk.X, pady=(0, 6))

        tk.Label(self.interval_frame, text="FRECUENCIA DE CAPTURA", font=("Segoe UI", 8, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w")

        freq_chips = tk.Frame(self.interval_frame, bg="#111a2e")
        freq_chips.pack(fill=tk.X, pady=2)
        for m in [5, 15, 30, 60, 120, 240]:
            lbl_text = f"{m}m" if m < 60 else f"{m//60}h"
            btn = tk.Button(freq_chips, text=lbl_text, font=("Segoe UI", 8, "bold"), bg="#0f172a", fg="#f8fafc", relief="flat", width=4, pady=4, command=lambda val=m:
self.set_interval(val))
            btn.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)

        stepper_box = tk.Frame(self.interval_frame, bg="#111a2e")
        stepper_box.pack(fill=tk.X, pady=2)
        tk.Button(stepper_box, text="-15m", font=("Segoe UI", 9, "bold"), bg="#1e293b", fg="#fff", relief="flat", padx=8, pady=4, command=lambda:
self.step_interval(-15)).pack(side=tk.LEFT)
        self.lbl_interval_val = tk.Label(stepper_box, text="60 min (1.0h)", font=("Courier", 11, "bold"), fg="#34d399", bg="#0f172a", relief="ridge", pady=4)
        self.lbl_interval_val.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=4)
```

```
        tk.Button(stepper_box, text="+15m", font=("Segoe UI", 9, "bold"), bg="#1e293b", fg="#fff", relief="flat", padx=8, pady=4, command=lambda: self.step_interval(15)).pack(side=tk.LEFT)

        self.sched_frame = tk.Frame(left_col, bg="#111a2e")
        tk.Label(self.sched_frame, text="HORAS ACTIVAS (Clic para activar/desactivar)", font=("Segoe UI", 8, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w")
        self.hours_buttons = {}
        grid_h = tk.Frame(self.sched_frame, bg="#111a2e")
        grid_h.pack(fill=tk.X, pady=2)
        for idx, h in enumerate(self.default_hours):
            r, c = idx // 5, idx % 5
            btn = tk.Button(grid_h, text=h, font=("Courier", 8, "bold"), bg="#0f172a", fg="#94a3b8", relief="flat", padx=2, pady=4, command=lambda val=h: self.toggle_hour(val))
            btn.grid(row=r, column=c, sticky="nsew", padx=1, pady=1)
            self.hours_buttons[h] = btn
        for i in range(5):
            grid_h.columnconfigure(i, weight=1)

        tk.Label(left_col, text="CANTIDAD DE FOTOS POR SESION", font=("Segoe UI", 8, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w", pady=(6, 2))
        photos_chips = tk.Frame(left_col, bg="#111a2e")
        photos_chips.pack(fill=tk.X)
        for cnt in [1, 2, 3, 5, 8, 10]:
            btn = tk.Button(photos_chips, text=f"{cnt}", font=("Segoe UI", 9, "bold"), bg="#0f172a", fg="#f8fafc", relief="flat", width=3, pady=4, command=lambda val=cnt: self.set_photos_count(val))
            btn.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)

        tk.Label(left_col, text="ESPACIADO ENTRE TOMAS", font=("Segoe UI", 8, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w", pady=(6, 2))
        spacing_chips = tk.Frame(left_col, bg="#111a2e")
        spacing_chips.pack(fill=tk.X)
        for sec, lbl in [(3, "3s"), (5, "5s"), (10, "10s"), (30, "30s"), (60, "1m"), (300, "5m")]:
            btn = tk.Button(spacing_chips, text=lbl, font=("Segoe UI", 8, "bold"), bg="#0f172a", fg="#f8fafc", relief="flat", pady=4, command=lambda val=sec: self.set_spacing(val))
            btn.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)

        right_col = tk.Frame(main_box, bg="#111a2e", padx=10, pady=10, highlightbackground="#1e2d4d", highlightthickness=1)
        right_col.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=(4, 0))

        btn_fire = tk.Button(right_col, text="DISPARAR CAPTURA AHORA", font=("Segoe UI", 11, "bold"), bg="#059669", fg="#ffffff", activebackground="#047857", activeforeground="#ffffff", relief="flat", pady=12, command=self.trigger_now)
        btn_fire.pack(fill=tk.X, pady=(0, 8))

        # Calibración de Cámaras (Prueba de Enfoque y Encuadre)
        tk.Label(right_col, text="CALIBRACION DE CAMARAS EN VIVO", font=("Segoe UI", 8, "bold"), fg="#38bdf8", bg="#111a2e").pack(anchor="w", pady=(0, 2))
        cam_box = tk.Frame(right_col, bg="#111a2e")
        cam_box.pack(fill=tk.X, pady=(0, 8))

        btn_cam0 = tk.Button(cam_box, text="Probar Cenital (ID 0)", font=("Segoe UI", 8, "bold"), bg="#1e293b", fg="#f8fafc", activebackground="#334155", relief="flat", pady=6, command=lambda: self.preview_camera(0))
        btn_cam0.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=2)

        btn_cam1 = tk.Button(cam_box, text="Probar Lateral (ID 1)", font=("Segoe UI", 8, "bold"), bg="#1e293b", fg="#f8fafc", activebackground="#334155", relief="flat", pady=6, command=lambda: self.preview_camera(1))
        btn_cam1.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=2)

        # Canastilla / Planta
        tk.Label(right_col, text="CANASTILLA / PLANTA ASIGNADA", font=("Segoe UI", 8, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w", pady=(0, 2))
        plant_box = tk.Frame(right_col, bg="#111a2e")
        plant_box.pack(fill=tk.X, pady=(0, 8))
        self.plant_buttons = {}
        for pid in [1, 2, 3, 4]:
```

```
            btn = tk.Button(plant_box, text=f"Planta #{pid}", font=("Segoe UI", 8, "bold"), relief="flat", pady=5, command=lambda val=pid: self.set_plant(val))
            btn.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)
            self.plant_buttons[pid] = btn

        # Captura al encender
        tk.Label(right_col, text="CAPTURA AL ENCENDER", font=("Segoe UI", 8, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w", pady=(0, 2))
        boot_box = tk.Frame(right_col, bg="#111a2e")
        boot_box.pack(fill=tk.X, pady=(0, 8))
        self.btn_boot_on = tk.Button(boot_box, text="Activada (Exhibicion)", font=("Segoe UI", 8, "bold"), relief="flat", pady=4, command=lambda: self.set_boot_capture(True))
        self.btn_boot_on.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)
        self.btn_boot_off = tk.Button(boot_box, text="Desactivada", font=("Segoe UI", 8, "bold"), relief="flat", pady=4, command=lambda: self.set_boot_capture(False))
        self.btn_boot_off.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)

        # Panel de Estado en Vivo (Log Feed)
        tk.Label(right_col, text="ESTADO DEL DEMONIO Y ALMACENAMIENTO", font=("Segoe UI", 8, "bold"), fg="#94a3b8", bg="#111a2e").pack(anchor="w", pady=(0, 2))
        self.status_text_box = tk.Label(right_col, text="Iniciando...", font=("Courier", 8), fg="#cbd5e1", bg="#0a0f18", justify="left", anchor="w", padx=8, pady=6, relief="ridge")
        self.status_text_box.pack(fill=tk.BOTH, expand=True, pady=(0, 8))

        daemon_ctrl_box = tk.Frame(right_col, bg="#111a2e")
        daemon_ctrl_box.pack(fill=tk.X)
        tk.Button(daemon_ctrl_box, text="Iniciar Demonio", font=("Segoe UI", 8, "bold"), bg="#064e3b", fg="#fff", relief="flat", pady=5, command=lambda: self.control_daemon("start")).pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)
        tk.Button(daemon_ctrl_box, text="Detener", font=("Segoe UI", 8, "bold"), bg="#7f1d1d", fg="#fff", relief="flat", pady=5, command=lambda: self.control_daemon("stop")).pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)
        tk.Button(daemon_ctrl_box, text="Reiniciar", font=("Segoe UI", 8, "bold"), bg="#1e3a8a", fg="#fff", relief="flat", pady=5, command=lambda: self.control_daemon("restart")).pack(side=tk.LEFT, fill=tk.X, expand=True, padx=1)

    def update_ui_from_config(self):
        is_interval = self.cfg.get("capture_mode", "interval") == "interval"
        self.btn_mode_interval.configure(bg="#10b981" if is_interval else "#0f172a", fg="#000" if is_interval else "#fff")
        self.btn_mode_sched.configure(bg="#10b981" if not is_interval else "#0f172a", fg="#000" if not is_interval else "#fff")

        if is_interval:
            self.sched_frame.pack_forget()
            self.interval_frame.pack(fill=tk.X, pady=(0, 6))
        else:
            self.interval_frame.pack_forget()
            self.sched_frame.pack(fill=tk.X, pady=(0, 6))

        m = self.cfg.get("interval_minutes", 60)
        self.lbl_interval_val.configure(text=f"{m} min ({(m/60):.1f}h)")

        sched = self.cfg.get("scheduled_times", ["07:05", "12:05", "17:05"])
        for h, btn in self.hours_buttons.items():
            active = h in sched
            btn.configure(bg="#059669" if active else "#0f172a", fg="#ffffff" if active else "#94a3b8")

        # Canastilla
        current_pid = self.cfg.get("plant_id", 1)
        for pid, btn in self.plant_buttons.items():
            btn.configure(bg="#10b981" if pid == current_pid else "#0f172a", fg="#000" if pid == current_pid else "#fff")

        boot_on = self.cfg.get("capture_on_boot", True)
        self.btn_boot_on.configure(bg="#10b981" if boot_on else "#0f172a", fg="#000" if boot_on else "#fff")
        self.btn_boot_off.configure(bg="#10b981" if not boot_on else "#0f172a", fg="#000" if not boot_on else "#fff")
```

```
264
265     def set_mode(self, mode):
266         self.cfg["capture_mode"] = mode
267         save_config(self.cfg)
268         self.update_ui_from_config()
269
270     def set_interval(self, minutes):
271         self.cfg["interval_minutes"] = max(1, minutes)
272         save_config(self.cfg)
273         self.update_ui_from_config()
274
275     def step_interval(self, delta):
276         cur = self.cfg.get("interval_minutes", 60)
277         self.set_interval(cur + delta)
278
279     def set_photos_count(self, count):
280         self.cfg["photos_per_session"] = max(1, min(20, count))
281         save_config(self.cfg)
282         self.update_ui_from_config()
283
284     def set_spacing(self, sec):
285         self.cfg["interval_between_photos_sec"] = max(1, sec)
286         save_config(self.cfg)
287         self.update_ui_from_config()
288
289     def set_plant(self, pid):
290         self.cfg["plant_id"] = pid
291         save_config(self.cfg)
292         self.update_ui_from_config()
293
294     def set_boot_capture(self, enable):
295         self.cfg["capture_on_boot"] = enable
296         save_config(self.cfg)
297         self.update_ui_from_config()
298
299     def toggle_hour(self, hour):
300         sched = list(self.cfg.get("scheduled_times", ["07:05", "12:05", "17:05"]))
301         if hour in sched:
302             sched.remove(hour)
303         else:
304             sched.append(hour)
305         self.cfg["scheduled_times"] = sorted(sched)
306         save_config(self.cfg)
307         self.update_ui_from_config()
308
309     def apply_preset(self, name):
310         if name == "exhibicion":
311             self.cfg.update({
312                 "capture_mode": "interval",
313                 "interval_minutes": 60,
314                 "photos_per_session": 3,
315                 "interval_between_photos_sec": 5,
316                 "boot_delay_sec": 10,
317                 "capture_on_boot": True
318             })
```

```
        elif name == "exhibicion_rapida":
            self.cfg.update({
                "capture_mode": "interval",
                "interval_minutes": 15,
                "photos_per_session": 2,
                "interval_between_photos_sec": 3,
                "boot_delay_sec": 5,
                "capture_on_boot": True
            })
        elif name == "agronomico":
            self.cfg.update({
                "capture_mode": "scheduled",
                "scheduled_times": ["07:05", "12:05", "17:05"],
                "photos_per_session": 5,
                "interval_between_photos_sec": 300,
                "boot_delay_sec": 300,
                "capture_on_boot": False
            })
        save_config(self.cfg)
        self.update_ui_from_config()

    def trigger_now(self):
        def _run():
            try:
                cmd = [sys.executable, os.path.join(NODE_DIR, "main_capture.py"), "--now"]
                subprocess.run(cmd, check=True)
            except Exception as e:
                print(f"[ERROR DISPARO]: {e}")
        threading.Thread(target=_run, daemon=True).start()

    def preview_camera(self, cam_idx):
        win = tk.Toplevel(self)
        win.title(f"Calibración Cámara ID {cam_idx}")
        win.geometry("520x420")
        win.configure(bg="#0f172a")

        lbl_loading = tk.Label(win, text="Capturando fotograma de calibración...", font=("Segoe UI", 10), fg="#38bdf8", bg="#0f172a")
        lbl_loading.pack(pady=20)

        img_label = tk.Label(win, bg="#000")
        img_label.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        def _capture_frame():
            preview_file = os.path.join(PREVIEW_CACHE_DIR, f"preview_cam_{cam_idx}.jpg")
            ok = CameraCaptureService.capture_photo(preview_file, camera_index=cam_idx, max_retries=1)
            if ok and os.path.exists(preview_file) and HAS_PIL_TK:
                try:
                    pil_img = Image.open(preview_file)
                    pil_img.thumbnail((480, 320))
                    tk_img = ImageTk.PhotoImage(pil_img)
                    img_label.img_ref = tk_img
                    img_label.configure(image=tk_img)
                    lbl_loading.configure(text=f"Frame capturado ({datetime.now().strftime('%H:%M:%S')})", fg="#34d399")
                except Exception as err:
                    lbl_loading.configure(text=f"Error cargando imagen: {err}", fg="#ef4444")
```

```
            else:
                lbl_loading.configure(text=f"No se detectó cámara activa en ID {cam_idx}", fg="#ef4444")

        threading.Thread(target=_capture_frame, daemon=True).start()

    def control_daemon(self, action):
        if platform.system() != "Windows":
            try:
                subprocess.run(["sudo", "systemctl", action, "sifma-capture"], check=False)
            except Exception:
                pass

    def refresh_timer(self):
        self.lbl_clock.configure(text=datetime.now().strftime("%H:%M:%S"))

        try:
            target_dir = OfflineStorageService._find_target_storage_directory_once()
            is_usb = "SIFMA_OFFLINE_STORAGE" not in target_dir
            if is_usb:
                self.lbl_usb.configure(text="USB: Conectada", fg="#60a5fa")
            else:
                self.lbl_usb.configure(text="Almacenamiento Local", fg="#f59e0b")

            status_text = "En espera..."
            next_cap = "--"
            last_cap_str = "Ninguno"
            if os.path.exists(STATUS_FILE):
                with open(STATUS_FILE, "r", encoding="utf-8") as f:
                    st = json.load(f)
                    status_text = st.get("status", "En espera")
                    next_cap = st.get("next_capture_human", "--")
                    if "last_capture" in st and st["last_capture"]:
                        last_cap_str = f"{st['last_capture'].get('timestamp', '')} ({st['last_capture'].get('period', '')})"

            self.status_text_box.configure(
                text=f"Estado: {status_text}\n"
                     f"Proxima toma: {next_cap}\n"
                     f"Ultimo lote: {last_cap_str}\n"
                     f"Destino: {target_dir}"
            )
        except Exception:
            pass

        self.after(2000, self.refresh_timer)

if __name__ == "__main__":
    app = SifmaLiteTouchApp()
    app.mainloop()
```

ARCHIVO 82/85: raspi_node/services/__init__.py (8 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

from .storage_service import OfflineStorageService
from .camera_service import CameraCaptureService

__all__ = ["OfflineStorageService", "CameraCaptureService"]
```

ARCHIVO 83/85: raspi_node/services/camera_service.py (185 líneas)

```
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import sys
import time
import subprocess
import shutil

HAS_PICAMERA2 = False
try:
    from picamera2 import Picamera2
    HAS_PICAMERA2 = True
except (ImportError, OSError):
    pass

try:
    import cv2
    HAS_OPENCV = True
except ImportError:
    HAS_OPENCV = False

class CameraCaptureService:
    """
    Servicio de captura física para Raspberry Pi 5 con cámaras ARDUCAM / MIPI CSI
    conectadas mediante cables flexores a los puertos CAM/DISP 0 (Cenital) y CAM/DISP 1 (Lateral).

    Aplica rotación de producción:
    - Cámara 0 (Cenital): Rotación 180° para orientación derecha del dosel.
    - Cámara 1 (Lateral): Rotación 90° a la derecha (sentido horario) para orientación vertical del tallo.
    """

    @staticmethod
    def _apply_orientation(output_path, camera_index=0):
        """Aplica la rotación correspondiente según la posición física de la cámara."""
        if not os.path.exists(output_path):
            return

        try:
            if HAS_OPENCV:
                img = cv2.imread(output_path)
                if img is not None:
                    if camera_index == 0:
                        # Cenital: Rotar 180 grados
                        rotated = cv2.rotate(img, cv2.ROTATE_180)
                    else:
                        # Lateral: Rotar 90 grados en sentido horario
                        rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
                    cv2.imwrite(output_path, rotated)
            else:
                from PIL import Image
                img = Image.open(output_path)
```

```
                if camera_index == 0:
                    rotated = img.rotate(180, expand=True)
                else:
                    rotated = img.rotate(270, expand=True) # 270 deg counter-clockwise = 90 deg clockwise
                rotated.save(output_path)
        except Exception as e:
            print(f"[CAMARA AVISO] Error al rotar imagen: {e}")

    @staticmethod
    def list_detected_cameras():
        """Consulta el hardware MIPI CSI para detectar las cámaras Arducam conectadas."""
        detected = []
        cmd_tool = shutil.which("rpicam-hello") or shutil.which("libcamera-hello")
        if cmd_tool:
            try:
                res = subprocess.run([cmd_tool, "--list-cameras"], capture_output=True, text=True, timeout=5)
                output = res.stdout + "\n" + res.stderr
                lines = [l.strip() for l in output.splitlines() if l.strip()]
                for line in lines:
                    if "available" in line.lower() or ":" in line:
                        detected.append(line)
            except Exception:
                pass

        if not detected and HAS_PICAMERA2:
            try:
                from picamera2 import Picamera2
                for idx in [0, 1]:
                    try:
                        p = Picamera2(idx)
                        detected.append(f"CSI Puerto {idx}: Camara Arducam detectada")
                        p.close()
                    except Exception:
                        pass
            except Exception:
                pass

        return detected

    @staticmethod
    def capture_photo(output_path, camera_index=0, max_retries=2):
        """
        Captura una imagen de alta calidad desde la cámara Arducam conectada al puerto flexor
        CAM/DISP correspondiente (0 = Cenital, 1 = Lateral).
        """
        os.makedirs(os.path.dirname(output_path), exist_ok=True)

        for attempt in range(1, max_retries + 1):
            # 1. Estrategia Principal: Picamera2 (Controlador oficial CSI de Raspberry Pi 5)
            if HAS_PICAMERA2:
                picam2 = None
                try:
                    picam2 = Picamera2(camera_index)
                    still_config = picam2.create_still_configuration(main={"format": "RGB888"})
                    picam2.configure(still_config)
```

```
                    picam2.start()
                    time.sleep(0.8)  # Tiempo para calibración automática de fotometría del sensor Arducam
                    picam2.capture_file(output_path)
                    picam2.stop()
                    picam2.close()
                    picam2 = None

                    if os.path.exists(output_path) and os.path.getsize(output_path) > 1000:
                        CameraCaptureService._apply_orientation(output_path, camera_index=camera_index)
                        print(f"[CAMARA CSI] Foto capturada y orientada en Puerto Flex #{camera_index}: {os.path.basename(output_path)}")
                        return True
                except Exception as e:
                    print(f"[CAMARA CSI] Intento {attempt} con Picamera2 (Puerto Flex {camera_index}): {e}")
                finally:
                    if picam2 is not None:
                        try:
                            picam2.close()
                        except Exception:
                            pass

            # 2. Estrategia Secundaria: Binario Nativo rpicam-still / libcamera-still
            rpicam_bin = shutil.which("rpicam-still") or shutil.which("libcamera-still")
            if rpicam_bin:
                try:
                    cmd = [
                        rpicam_bin,
                        "--camera", str(camera_index),
                        "-o", output_path,
                        "-t", "800",       # 800ms para autoexposición
                        "--nopreview",
                        "-n",              # Sin vista previa en ventana de servidor X
                        "--encoding", "png" if output_path.lower().endswith(".png") else "jpg"
                    ]
                    res = subprocess.run(cmd, capture_output=True, text=True, timeout=8)
                    if res.returncode == 0 and os.path.exists(output_path) and os.path.getsize(output_path) > 1000:
                        CameraCaptureService._apply_orientation(output_path, camera_index=camera_index)
                        print(f"[CAMARA CSI] Foto capturada con {os.path.basename(rpicam_bin)} (Puerto Flex {camera_index}): {os.path.basename(output_path)}")
                        return True
                    else:
                        print(f"[CAMARA CSI] {os.path.basename(rpicam_bin)} fallo en puerto {camera_index}: {res.stderr}")
                except Exception as e:
                    print(f"[CAMARA CSI] Intento con {rpicam_bin} en puerto {camera_index}: {e}")

            # 3. Estrategia Terciaria: OpenCV VideoCapture (V4L2 CSI Backend)
            if HAS_OPENCV:
                cap = None
                try:
                    if sys.platform.startswith("linux"):
                        cap = cv2.VideoCapture(camera_index, cv2.CAP_V4L2)
                    else:
                        cap = cv2.VideoCapture(camera_index)

                    if cap.isOpened():
                        for _ in range(4):
                            cap.read()
```

```
                        time.sleep(0.04)
                    ret, frame = cap.read()
                    cap.release()
                    if ret and frame is not None:
                        cv2.imwrite(output_path, frame)
                        if os.path.exists(output_path) and os.path.getsize(output_path) > 1000:
                            CameraCaptureService._apply_orientation(output_path, camera_index=camera_index)
                            print(f"[CAMARA CSI] Foto capturada con OpenCV (Puerto {camera_index}): {os.path.basename(output_path)}")
                            return True
                else:
                    if cap is not None:
                        cap.release()
            except Exception as e:
                print(f"[CAMARA CSI] Intento con OpenCV (Puerto {camera_index}): {e}")
            finally:
                if cap is not None and cap.isOpened():
                    cap.release()

        time.sleep(0.5)

    print(f"[CAMARA AVISO] No se pudo capturar foto en el Puerto Flex CAM/DISP #{camera_index} ({os.path.basename(output_path)})")
    return False
```

ARCHIVO 84/85: raspi_node/services/storage_service.py (146 líneas)

```python
# SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
# Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
# Año: 2026.

import os
import json
import time
import platform
from datetime import datetime

class OfflineStorageService:
    """
    Servicio encargado de detectar automáticamente memorias USB conectadas a la Raspberry Pi
    y estructurar el almacenamiento por fecha y período con metadatos de alta resolución temporal:
    Estructura en USB: SIFMA_CAPTURES/YYYY-MM-DD/periodo/
    """

    @staticmethod
    def wait_and_find_storage_directory(timeout_sec=15):
        """
        Espera a que el sistema operativo monte la memoria USB al encender.
        Si tras el tiempo límite no se detecta USB, utiliza almacenamiento local de respaldo.
        """
        initial_check = OfflineStorageService._find_target_storage_directory_once()
        if initial_check and "SIFMA_OFFLINE_STORAGE" not in initial_check:
            print(f"[ALMACENAMIENTO] Memoria USB detectada en: {initial_check}")
            return initial_check

        start_time = time.time()
        print(f"[ALMACENAMIENTO] Esperando deteccion de memoria USB (maximo {timeout_sec}s)...")

        while time.time() - start_time < timeout_sec:
            target_path = OfflineStorageService._find_target_storage_directory_once()
            if target_path and "SIFMA_OFFLINE_STORAGE" not in target_path:
                print(f"[ALMACENAMIENTO] Memoria USB detectada y montada en: {target_path}")
                return target_path
            time.sleep(1.5)

        print("[ALMACENAMIENTO AVISO] No se detecto memoria USB externa. Usando almacenamiento local de respaldo.")
        return OfflineStorageService._find_target_storage_directory_once()

    @staticmethod
    def _find_target_storage_directory_once():
        system = platform.system()
        possible_usb_roots = []

        if system == "Windows":
            import string
            for letter in string.ascii_uppercase:
                if letter not in ['C']:
                    drive = f"{letter}:\\"
                    if os.path.exists(drive):
                        possible_usb_roots.append(drive)
```

```
        else: # Linux / Raspberry Pi OS
            possible_usb_roots = [
                "/media/nodo1",
                "/media/computo1",
                "/media/pi",
                "/media",
                "/mnt/usb",
                "/mnt"
            ]

        target_usb_path = None
        for root in possible_usb_roots:
            if os.path.exists(root):
                if os.access(root, os.W_OK):
                    target_usb_path = os.path.join(root, "SIFMA_CAPTURES")
                    break
                else:
                    try:
                        subdirs = [os.path.join(root, d) for d in os.listdir(root) if os.path.isdir(os.path.join(root, d))]
                        for sd in subdirs:
                            if os.access(sd, os.W_OK):
                                target_usb_path = os.path.join(sd, "SIFMA_CAPTURES")
                                break
                    except Exception:
                        pass
                if target_usb_path:
                    break

        if not target_usb_path:
            local_fallback = os.path.join(os.path.dirname(os.path.dirname(__file__)), "SIFMA_OFFLINE_STORAGE")
            target_usb_path = local_fallback

        return target_usb_path

    @staticmethod
    def create_batch_directory(period="manana", wait_for_usb=True):
        """
        Crea una carpeta de lote en la USB con la jerarquía:
        SIFMA_CAPTURES/YYYY-MM-DD/periodo/
        """
        if wait_for_usb:
            base_dir = OfflineStorageService.wait_and_find_storage_directory(timeout_sec=15)
        else:
            base_dir = OfflineStorageService._find_target_storage_directory_once()

        date_str = datetime.now().strftime("%Y-%m-%d")
        period_clean = period.lower().replace("ñ", "n").replace("í", "i").replace(" ", "_")

        batch_dir = os.path.join(base_dir, date_str, period_clean)
        os.makedirs(batch_dir, exist_ok=True)
        return batch_dir

    @staticmethod
    def save_batch_metadata(
        batch_dir,
```

```
        period,
        crop_type="cebollin",
        image_files_count=10,
        plant_id=1,
        session_start=None,
        session_end=None,
        photos_info=None
    ):
        """
        Guarda el archivo metadata.json en el lote de la USB con el desglose de cada una de las 5 tomas
        y sus marcas de tiempo individuales para el cruce cronológico preciso.
        """
        now = datetime.now()
        meta_path = os.path.join(batch_dir, "metadata.json")
        payload = {
            "timestamp": session_start or now.strftime("%Y-%m-%d %H:%M:%S"),
            "date": now.strftime("%Y-%m-%d"),
            "exact_time": now.strftime("%H:%M:%S"),
            "period": period,
            "crop_type": crop_type,
            "plant_id": plant_id,
            "images_count": image_files_count,
            "session_start": session_start or now.strftime("%Y-%m-%d %H:%M:%S"),
            "session_end": session_end or now.strftime("%Y-%m-%d %H:%M:%S"),
            "photos": photos_info or []
        }
        with open(meta_path, "w", encoding="utf-8") as f:
            json.dump(payload, f, indent=2)

        # Forzar sincronización física en la memoria USB
        try:
            if hasattr(os, 'sync'):
                os.sync()
        except Exception:
            pass

        print(f"[ALMACENAMIENTO] Metadatos guardados y sincronizados en USB: {meta_path}")
        return meta_path
```

ARCHIVO 85/85: raspi_node/templates/index.html (961 líneas)

```
<!--
  SIFMA — Sistema Integral de Fenotipado y Monitoreo Agrícola
  Autores: Andrés Felipe Luna Camargo, Cristian David Peña Martínez
  Año: 2026.
-->

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
    <title>SIFMA - Centro de Control Raspberry Pi</title>
    <style>
        :root {
            --bg-base: #0a0f18;
            --bg-card: #111a2e;
            --bg-card-hover: #17233d;
            --border-color: #1e2d4d;
            --primary: #10b981;
            --primary-hover: #059669;
            --primary-glow: rgba(16, 185, 129, 0.25);
            --secondary: #3b82f6;
            --secondary-hover: #2563eb;
            --warning: #f59e0b;
            --danger: #ef4444;
            --text-main: #f8fafc;
            --text-muted: #94a3b8;
            --radius-lg: 16px;
            --radius-md: 10px;
            --radius-sm: 6px;
        }

        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
            -webkit-tap-highlight-color: transparent;
            user-select: none;
        }

        body {
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
            background-color: var(--bg-base);
            color: var(--text-main);
            min-height: 100vh;
            display: flex;
            flex-direction: column;
            padding: 12px;
            overflow-x: hidden;
        }

        /* Top Header */
        header {
```

```
            display: flex;
            justify-content: space-between;
            align-items: center;
            background: var(--bg-card);
            border: 1px solid var(--border-color);
            padding: 14px 20px;
            border-radius: var(--radius-lg);
            margin-bottom: 12px;
            box-shadow: 0 4px 20px rgba(0,0,0,0.4);
        }

        .header-brand {
            display: flex;
            align-items: center;
            gap: 12px;
        }

        .brand-icon {
            width: 36px;
            height: 36px;
            background: linear-gradient(135deg, #10b981, #059669);
            border-radius: var(--radius-md);
            display: flex;
            align-items: center;
            justify-content: center;
            font-weight: 900;
            color: #fff;
            font-size: 1.1rem;
            letter-spacing: 1px;
        }

        .brand-title {
            font-size: 1.25rem;
            font-weight: 700;
            color: #fff;
            letter-spacing: 0.5px;
        }

        .brand-subtitle {
            font-size: 0.75rem;
            color: var(--primary);
            font-weight: 600;
            text-transform: uppercase;
            letter-spacing: 1px;
        }

        .header-status-group {
            display: flex;
            gap: 10px;
            align-items: center;
        }

        .badge {
            display: inline-flex;
            align-items: center;
```

```
            gap: 6px;
            padding: 6px 12px;
            border-radius: 20px;
            font-size: 0.8rem;
            font-weight: 600;
            background: #1e293b;
            color: var(--text-muted);
            border: 1px solid var(--border-color);
        }

        .badge-dot {
            width: 8px;
            height: 8px;
            border-radius: 50%;
            background: var(--text-muted);
        }

        .badge.active {
            background: rgba(16, 185, 129, 0.15);
            color: #34d399;
            border-color: rgba(16, 185, 129, 0.3);
        }
        .badge.active .badge-dot {
            background: #10b981;
            box-shadow: 0 0 8px #10b981;
        }

        .badge.usb-active {
            background: rgba(59, 130, 246, 0.15);
            color: #60a5fa;
            border-color: rgba(59, 130, 246, 0.3);
        }
        .badge.usb-active .badge-dot {
            background: #3b82f6;
            box-shadow: 0 0 8px #3b82f6;
        }

        .clock-display {
            font-family: monospace;
            font-size: 1rem;
            color: #cbd5e1;
            padding-left: 8px;
            border-left: 1px solid var(--border-color);
        }

        /* Main Grid */
        .dashboard-grid {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 12px;
            flex: 1;
        }

        @media (max-width: 900px) {
            .dashboard-grid {
```

```
                grid-template-columns: 1fr;
            }
        }

        .card {
            background: var(--bg-card);
            border: 1px solid var(--border-color);
            border-radius: var(--radius-lg);
            padding: 18px;
            display: flex;
            flex-direction: column;
            gap: 14px;
            box-shadow: 0 4px 16px rgba(0,0,0,0.25);
        }

        .card-header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            border-bottom: 1px solid var(--border-color);
            padding-bottom: 10px;
        }

        .card-title {
            font-size: 1rem;
            font-weight: 700;
            color: #f1f5f9;
            text-transform: uppercase;
            letter-spacing: 0.5px;
        }

        .card-badge {
            font-size: 0.75rem;
            color: var(--text-muted);
            background: #0f172a;
            padding: 4px 8px;
            border-radius: 6px;
        }

        /* 1-Click Presets */
        .preset-grid {
            display: grid;
            grid-template-columns: repeat(3, 1fr);
            gap: 10px;
        }

        .preset-btn {
            background: #0f172a;
            border: 1px solid var(--border-color);
            border-radius: var(--radius-md);
            padding: 14px 10px;
            color: var(--text-main);
            display: flex;
            flex-direction: column;
            align-items: center;
```

```
            gap: 6px;
            cursor: pointer;
            transition: all 0.15s ease;
            text-align: center;
        }

        .preset-btn:active, .preset-btn.selected {
            background: linear-gradient(135deg, rgba(16, 185, 129, 0.2), rgba(5, 150, 105, 0.3));
            border-color: var(--primary);
            box-shadow: 0 0 12px var(--primary-glow);
            transform: scale(0.98);
        }

        .preset-name {
            font-weight: 700;
            font-size: 0.9rem;
            color: #fff;
        }

        .preset-desc {
            font-size: 0.72rem;
            color: var(--text-muted);
            line-height: 1.2;
        }

        /* Chips & Button Groups */
        .control-group {
            display: flex;
            flex-direction: column;
            gap: 8px;
        }

        .control-label {
            font-size: 0.85rem;
            font-weight: 600;
            color: #cbd5e1;
            display: flex;
            justify-content: space-between;
        }

        .control-label span {
            color: var(--primary);
            font-weight: 700;
        }

        .chip-group {
            display: flex;
            flex-wrap: wrap;
            gap: 8px;
        }

        .chip {
            background: #0f172a;
            border: 1px solid var(--border-color);
            color: #cbd5e1;
```

```
            padding: 10px 14px;
            border-radius: var(--radius-md);
            font-size: 0.85rem;
            font-weight: 600;
            cursor: pointer;
            transition: all 0.15s ease;
            flex: 1 1 auto;
            text-align: center;
            min-height: 44px;
            display: flex;
            align-items: center;
            justify-content: center;
        }

        .chip:active, .chip.active {
            background: var(--primary);
            color: #000;
            font-weight: 700;
            border-color: var(--primary);
            box-shadow: 0 0 10px var(--primary-glow);
        }

        .chip-blue.active {
            background: var(--secondary);
            color: #fff;
            border-color: var(--secondary);
        }

        /* Steppers */
        .stepper-container {
            display: flex;
            align-items: center;
            gap: 8px;
        }

        .stepper-btn {
            background: #1e293b;
            border: 1px solid var(--border-color);
            color: #fff;
            font-size: 1.2rem;
            font-weight: 700;
            width: 50px;
            height: 46px;
            border-radius: var(--radius-md);
            cursor: pointer;
            display: flex;
            align-items: center;
            justify-content: center;
        }

        .stepper-btn:active {
            background: var(--primary);
            color: #000;
        }

```

```
        .stepper-value {
            flex: 1;
            background: #0f172a;
            border: 1px solid var(--border-color);
            height: 46px;
            border-radius: var(--radius-md);
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: 1.1rem;
            font-weight: 700;
            color: #34d399;
            font-family: monospace;
        }

        /* Scheduled Hours Matrix */
        .hours-grid {
            display: grid;
            grid-template-columns: repeat(5, 1fr);
            gap: 6px;
        }

        .hour-chip {
            background: #0f172a;
            border: 1px solid var(--border-color);
            color: var(--text-muted);
            padding: 8px 4px;
            border-radius: var(--radius-sm);
            font-size: 0.8rem;
            font-weight: 700;
            text-align: center;
            cursor: pointer;
            font-family: monospace;
        }

        .hour-chip.active {
            background: #059669;
            color: #fff;
            border-color: #10b981;
            box-shadow: 0 0 8px rgba(16, 185, 129, 0.4);
        }

        /* Action Buttons */
        .btn-action-primary {
            background: linear-gradient(135deg, #10b981, #059669);
            color: #fff;
            border: none;
            padding: 16px;
            border-radius: var(--radius-md);
            font-size: 1.05rem;
            font-weight: 700;
            cursor: pointer;
            box-shadow: 0 4px 14px var(--primary-glow);
            transition: all 0.15s ease;
            text-align: center;
```

```
            text-transform: uppercase;
            letter-spacing: 0.5px;
        }

        .btn-action-primary:active {
            transform: scale(0.98);
            background: #047857;
        }

        .btn-action-secondary {
            background: #1e293b;
            border: 1px solid var(--border-color);
            color: #f1f5f9;
            padding: 12px;
            border-radius: var(--radius-md);
            font-size: 0.9rem;
            font-weight: 600;
            cursor: pointer;
            text-align: center;
            transition: all 0.15s ease;
        }

        .btn-action-secondary:active {
            background: #334155;
        }

        /* Camera Live Preview Modal */
        .modal-overlay {
            display: none;
            position: fixed;
            top: 0;
            left: 0;
            right: 0;
            bottom: 0;
            background: rgba(0,0,0,0.85);
            backdrop-filter: blur(4px);
            z-index: 1000;
            align-items: center;
            justify-content: center;
            padding: 20px;
        }

        .modal-card {
            background: var(--bg-card);
            border: 1px solid var(--border-color);
            border-radius: var(--radius-lg);
            width: 100%;
            max-width: 650px;
            padding: 20px;
            display: flex;
            flex-direction: column;
            gap: 16px;
            box-shadow: 0 10px 40px rgba(0,0,0,0.8);
        }

```

```
        .preview-img-container {
            width: 100%;
            height: 340px;
            background: #000;
            border-radius: var(--radius-md);
            overflow: hidden;
            display: flex;
            align-items: center;
            justify-content: center;
            border: 1px solid var(--border-color);
        }

        .preview-img-container img {
            max-width: 100%;
            max-height: 100%;
            object-fit: contain;
        }

        /* Status Toast Banner */
        #toastBanner {
            position: fixed;
            bottom: 16px;
            left: 50%;
            transform: translateX(-50%);
            background: #0f172a;
            border: 1px solid var(--primary);
            color: #fff;
            padding: 12px 24px;
            border-radius: 30px;
            font-size: 0.9rem;
            font-weight: 600;
            box-shadow: 0 4px 20px rgba(0,0,0,0.6);
            display: none;
            z-index: 2000;
        }

        .status-feed {
            background: #0a0f18;
            border: 1px solid var(--border-color);
            border-radius: var(--radius-md);
            padding: 12px;
            font-family: monospace;
            font-size: 0.8rem;
            color: #94a3b8;
            display: flex;
            flex-direction: column;
            gap: 4px;
        }

        .status-feed-highlight {
            color: #34d399;
            font-weight: bold;
        }
    </style>
</head>
```

```
<body>

    <!-- Header / Barra Superior -->
    <header>
        <div class="header-brand">
            <div class="brand-icon">S</div>
            <div>
                <div class="brand-title">SIFMA</div>
                <div class="brand-subtitle">Panel Operativo Raspberry Pi 5</div>
            </div>
        </div>

        <div class="header-status-group">
            <div class="badge active" id="daemonBadge">
                <span class="badge-dot"></span>
                <span id="daemonBadgeText">Demonio Activo</span>
            </div>
            <div class="badge usb-active" id="usbBadge">
                <span class="badge-dot"></span>
                <span id="usbBadgeText">USB: Detectada</span>
            </div>
            <div class="clock-display" id="liveClock">--:--:--</div>
        </div>
    </header>

    <!-- Main Dashboard -->
    <main class="dashboard-grid">

        <!-- Columna Izquierda: Presets y Programación de Captura -->
        <div class="card">
            <div class="card-header">
                <div class="card-title">1. Modo y Frecuencia de Captura</div>
                <div class="card-badge">Touch Friendly</div>
            </div>

            <!-- Presets Rápidos -->
            <div class="control-group">
                <div class="control-label">Ajustes Rápidos Predefinidos</div>
                <div class="preset-grid">
                    <button class="preset-btn" onclick="applyPreset('exhibicion')">
                        <div class="preset-name">Exhibición</div>
                        <div class="preset-desc">Cada 1 hora / 3 fotos rápidas (5s)</div>
                    </button>
                    <button class="preset-btn" onclick="applyPreset('exhibicion_rapida')">
                        <div class="preset-name">Demo Rápida</div>
                        <div class="preset-desc">Cada 15 min / 2 fotos instantáneas</div>
                    </button>
                    <button class="preset-btn" onclick="applyPreset('agronomico')">
                        <div class="preset-name">Agronómico</div>
                        <div class="preset-desc">3 jornadas (07, 12, 17h) / 5 fotos</div>
                    </button>
                </div>
            </div>

            <!-- Selector de Modo -->
```

```
            <div class="control-group">
                <div class="control-label">Tipo de Temporizador</div>
                <div class="chip-group">
                    <div class="chip" id="modeInterval" onclick="setCaptureMode('interval')">Intervalo Periódico</div>
                    <div class="chip" id="modeScheduled" onclick="setCaptureMode('scheduled')">Horas Programadas</div>
                </div>
            </div>

            <!-- Opciones Modo Intervalo -->
            <div id="intervalContainer" class="control-group">
                <div class="control-label">Frecuencia de Captura: <span id="intervalDisplay">60 min (1 hora)</span></div>
                <div class="chip-group">
                    <div class="chip" onclick="setIntervalMin(5)">5 min</div>
                    <div class="chip" onclick="setIntervalMin(15)">15 min</div>
                    <div class="chip" onclick="setIntervalMin(30)">30 min</div>
                    <div class="chip" onclick="setIntervalMin(60)">1 hora</div>
                    <div class="chip" onclick="setIntervalMin(120)">2 horas</div>
                    <div class="chip" onclick="setIntervalMin(240)">4 horas</div>
                </div>
                <div class="stepper-container" style="margin-top: 6px;">
                    <button class="stepper-btn" onclick="stepInterval(-15)">-</button>
                    <div class="stepper-value" id="stepperIntervalVal">60 min</div>
                    <button class="stepper-btn" onclick="stepInterval(15)">+</button>
                </div>
            </div>

            <!-- Opciones Modo Horas Programadas -->
            <div id="scheduledContainer" class="control-group" style="display: none;">
                <div class="control-label">Horas Activas de Captura (Clic para Alternar)</div>
                <div class="hours-grid" id="hoursGrid">
                    <!-- Generado dinámicamente con JS -->
                </div>
            </div>

            <!-- Configuración de Fotos por Sesión -->
            <div class="control-group">
                <div class="control-label">Rango de Fotos por Sesión: <span id="photosCountDisplay">3 fotos</span></div>
                <div class="chip-group">
                    <div class="chip" onclick="setPhotosCount(1)">1 foto</div>
                    <div class="chip" onclick="setPhotosCount(2)">2 fotos</div>
                    <div class="chip" onclick="setPhotosCount(3)">3 fotos</div>
                    <div class="chip" onclick="setPhotosCount(5)">5 fotos</div>
                    <div class="chip" onclick="setPhotosCount(8)">8 fotos</div>
                    <div class="chip" onclick="setPhotosCount(10)">10 fotos</div>
                </div>
                <div class="stepper-container" style="margin-top: 6px;">
                    <button class="stepper-btn" onclick="stepPhotosCount(-1)">-</button>
                    <div class="stepper-value" id="stepperPhotosVal">3 fotos</div>
                    <button class="stepper-btn" onclick="stepPhotosCount(1)">+</button>
                </div>
            </div>

            <!-- Espaciado entre Fotos -->
            <div class="control-group">
                <div class="control-label">Espaciado entre Tomas: <span id="spacingDisplay">5 segundos</span></div>
```

```
                <div class="chip-group">
                    <div class="chip" onclick="setPhotoSpacing(3)">3s</div>
                    <div class="chip" onclick="setPhotoSpacing(5)">5s</div>
                    <div class="chip" onclick="setPhotoSpacing(10)">10s</div>
                    <div class="chip" onclick="setPhotoSpacing(30)">30s</div>
                    <div class="chip" onclick="setPhotoSpacing(60)">1 min</div>
                    <div class="chip" onclick="setPhotoSpacing(300)">5 min</div>
                </div>
            </div>
        </div>

        <!-- Columna Derecha: Acciones, Canastilla, Calibración de Cámaras y Estado -->
        <div class="card">
            <div class="card-header">
                <div class="card-title">2. Acciones y Calibración en Vivo</div>
                <div class="card-badge">Control Hardware</div>
            </div>

            <!-- Botón Disparo Inmediato -->
            <button class="btn-action-primary" onclick="triggerImmediateCapture()">
                Disparar Captura Inmediata Ahora
            </button>

            <!-- Previsualización / Calibración de Cámaras -->
            <div class="control-group">
                <div class="control-label">Calibración y Enfoque de Cámaras</div>
                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 10px;">
                    <button class="btn-action-secondary" onclick="openCameraPreview(0)">
                        Probar Cámara Cenital (ID 0)
                    </button>
                    <button class="btn-action-secondary" onclick="openCameraPreview(1)">
                        Probar Cámara Lateral (ID 1)
                    </button>
                </div>
            </div>

            <!-- Canastilla y Cultivo -->
            <div class="control-group">
                <div class="control-label">Canastilla / Planta Asignada</div>
                <div class="chip-group">
                    <div class="chip" onclick="setPlantId(1)" id="plant1">Planta 1</div>
                    <div class="chip" onclick="setPlantId(2)" id="plant2">Planta 2</div>
                    <div class="chip" onclick="setPlantId(3)" id="plant3">Planta 3</div>
                    <div class="chip" onclick="setPlantId(4)" id="plant4">Planta 4</div>
                </div>
            </div>

            <!-- Captura al Encender -->
            <div class="control-group">
                <div class="control-label">Captura Inmediata al Encender Nodo</div>
                <div class="chip-group">
                    <div class="chip" id="bootCaptureOn" onclick="setBootCapture(true)">Activada (Exhibición)</div>
                    <div class="chip" id="bootCaptureOff" onclick="setBootCapture(false)">Desactivada</div>
                </div>
            </div>
```

```
            <!-- Estado Operativo en Vivo -->
            <div class="control-group">
                <div class="control-label">Estado del Demonio del Sistema</div>
                <div class="status-feed">
                    <div>Estado: <span class="status-feed-highlight" id="daemonStatusText">Cargando...</span></div>
                    <div>Próxima Toma: <span id="daemonNextCapture">--</span></div>
                    <div>Último Lote: <span id="daemonLastCapture">Ninguno registrado</span></div>
                    <div>Destino Almacenamiento: <span id="storagePathText">Buscando USB...</span></div>
                </div>
            </div>

            <!-- Controles del Demonio -->
            <div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 8px; margin-top: auto;">
                <button class="btn-action-secondary" style="color: #34d399;" onclick="controlDaemon('start')">Iniciar</button>
                <button class="btn-action-secondary" style="color: #ef4444;" onclick="controlDaemon('stop')">Detener</button>
                <button class="btn-action-secondary" style="color: #60a5fa;" onclick="controlDaemon('restart')">Reiniciar</button>
            </div>
        </div>

    </main>

    <!-- Modal de Previsualización de Cámara -->
    <div class="modal-overlay" id="previewModal">
        <div class="modal-card">
            <div style="display: flex; justify-content: space-between; align-items: center;">
                <div style="font-weight: 700; font-size: 1.1rem; color: #fff;" id="modalCamTitle">Vista Previa de Cámara</div>
                <button class="btn-action-secondary" style="padding: 6px 14px;" onclick="closePreviewModal()">Cerrar</button>
            </div>
            <div class="preview-img-container" id="previewImgContainer">
                <div id="previewLoadingText" style="color: var(--text-muted); font-size: 0.9rem;">Capturando frame de prueba...</div>
                <img id="previewImg" style="display: none;" alt="Preview">
            </div>
            <div style="display: flex; justify-content: space-between; align-items: center;">
                <div style="font-size: 0.8rem; color: var(--text-muted);" id="previewTimestamp">--</div>
                <button class="btn-action-primary" style="padding: 10px 20px; font-size: 0.9rem;" onclick="refreshCurrentPreview()">Actualizar Frame</button>
            </div>
        </div>
    </div>

    <!-- Toast Notification Banner -->
    <div id="toastBanner">Configuración guardada</div>

    <script>
        let currentConfig = {};
        let activePreviewCam = 0;

        const defaultHours = [
            "06:00", "07:05", "08:00", "09:00", "10:00",
            "11:00", "12:05", "13:00", "14:00", "15:00",
            "16:00", "17:05", "18:00", "19:00", "20:00"
        ];

        function showToast(msg) {
            const t = document.getElementById("toastBanner");
```

```
            t.textContent = msg;
            t.style.display = "block";
            setTimeout(() => { t.style.display = "none"; }, 2500);
        }

        async function fetchStatus() {
            try {
                const res = await fetch("/api/status");
                const data = await res.json();
                if (data.status === "ok") {
                    currentConfig = data.config;
                    updateUIFromConfig(data.config);
                    updateDaemonAndStorageUI(data);
                }
            } catch (e) {
                console.error("Error consultando estado:", e);
            }
        }

        function updateUIFromConfig(cfg) {
            // Modo
            const isInterval = cfg.capture_mode === "interval";
            document.getElementById("modeInterval").classList.toggle("active", isInterval);
            document.getElementById("modeScheduled").classList.toggle("active", !isInterval);
            document.getElementById("intervalContainer").style.display = isInterval ? "flex" : "none";
            document.getElementById("scheduledContainer").style.display = !isInterval ? "flex" : "none";

            // Intervalo
            const min = cfg.interval_minutes || 60;
            document.getElementById("intervalDisplay").textContent = `${min} min (${(min/60).toFixed(1)}h)`;
            document.getElementById("stepperIntervalVal").textContent = `${min} min`;

            // Fotos por sesión
            const photos = cfg.photos_per_session || 3;
            document.getElementById("photosCountDisplay").textContent = `${photos} fotos`;
            document.getElementById("stepperPhotosVal").textContent = `${photos} fotos`;

            // Espaciado
            const spacing = cfg.interval_between_photos_sec || 5;
            document.getElementById("spacingDisplay").textContent = spacing >= 60 ? `${spacing/60} min` : `${spacing} segundos`;

            // Canastilla
            const pid = cfg.plant_id || 1;
            [1, 2, 3, 4].forEach(id => {
                const el = document.getElementById(`plant${id}`);
                if (el) el.classList.toggle("active", id === pid);
            });

            // Captura al encender
            const bootCap = cfg.capture_on_boot !== false;
            document.getElementById("bootCaptureOn").classList.toggle("active", bootCap);
            document.getElementById("bootCaptureOff").classList.toggle("active", !bootCap);

            // Horas programadas
            renderHoursGrid(cfg.scheduled_times || ["07:05", "12:05", "17:05"]);
```

```
        }

        function renderHoursGrid(activeTimes) {
            const grid = document.getElementById("hoursGrid");
            grid.innerHTML = "";
            defaultHours.forEach(h => {
                const chip = document.createElement("div");
                chip.className = "hour-chip" + (activeTimes.includes(h) ? " active" : "");
                chip.textContent = h;
                chip.onclick = () => toggleScheduledHour(h);
                grid.appendChild(chip);
            });
        }

        function updateDaemonAndStorageUI(data) {
            // Reloj
            document.getElementById("liveClock").textContent = data.server_time || "--:--:--";

            // Demonio
            const daemon = data.daemon || {};
            const isRunning = daemon.is_running;
            const badge = document.getElementById("daemonBadge");
            const badgeText = document.getElementById("daemonBadgeText");

            badge.className = "badge" + (isRunning ? " active" : "");
            badgeText.textContent = isRunning ? "Demonio Activo" : "Demonio Detenido";

            document.getElementById("daemonStatusText").textContent = daemon.status || "En espera";
            document.getElementById("daemonNextCapture").textContent = daemon.next_capture_human || daemon.next_capture || "--";

            if (daemon.last_capture) {
                document.getElementById("daemonLastCapture").textContent =
                    `${daemon.last_capture.timestamp} (${daemon.last_capture.photos_count} fotos en ${daemon.last_capture.period})`;
            }

            // USB
            const storage = data.storage || {};
            const usbBadge = document.getElementById("usbBadge");
            const usbText = document.getElementById("usbBadgeText");
            if (storage.is_usb_mounted) {
                usbBadge.className = "badge usb-active";
                usbText.textContent = `USB: ${storage.free_gb}GB libres`;
            } else {
                usbBadge.className = "badge";
                usbText.textContent = "Almacenamiento Local";
            }
            document.getElementById("storagePathText").textContent = `${storage.label} (${storage.target_path})`;
        }

        async function saveConfig(updates) {
            try {
                const res = await fetch("/api/config", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify(updates)
```

```
                });
                const data = await res.json();
                if (data.status === "ok") {
                    showToast("Configuración actualizada y sincronizada");
                    fetchStatus();
                } else {
                    alert("Error guardando configuracion: " + data.message);
                }
            } catch (e) {
                console.error(e);
            }
        }

        function setCaptureMode(mode) {
            saveConfig({ capture_mode: mode });
        }

        function setIntervalMin(min) {
            saveConfig({ interval_minutes: min });
        }

        function stepInterval(delta) {
            let cur = currentConfig.interval_minutes || 60;
            cur = Math.max(1, cur + delta);
            saveConfig({ interval_minutes: cur });
        }

        function setPhotosCount(count) {
            saveConfig({ photos_per_session: count });
        }

        function stepPhotosCount(delta) {
            let cur = currentConfig.photos_per_session || 3;
            cur = Math.max(1, Math.min(20, cur + delta));
            saveConfig({ photos_per_session: cur });
        }

        function setPhotoSpacing(sec) {
            saveConfig({ interval_between_photos_sec: sec });
        }

        function setPlantId(id) {
            saveConfig({ plant_id: id });
        }

        function setBootCapture(enable) {
            saveConfig({ capture_on_boot: enable });
        }

        function toggleScheduledHour(hour) {
            let times = currentConfig.scheduled_times || ["07:05", "12:05", "17:05"];
            if (times.includes(hour)) {
                times = times.filter(t => t !== hour);
            } else {
                times.push(hour);
```

```
            }
            saveConfig({ scheduled_times: times });
        }

        async function applyPreset(presetName) {
            try {
                const res = await fetch("/api/preset", {
                    method: "POST",
                    headers: { "Content-Type": "application/json" },
                    body: JSON.stringify({ preset: presetName })
                });
                const data = await res.json();
                if (data.status === "ok") {
                    showToast(`Preset '${presetName}' aplicado`);
                    fetchStatus();
                }
            } catch (e) {
                console.error(e);
            }
        }

        async function triggerImmediateCapture() {
            if (!confirm("¿Desea iniciar una secuencia de captura fotográfica ahora mismo?")) return;
            try {
                const res = await fetch("/api/trigger_capture", { method: "POST" });
                const data = await res.json();
                showToast(data.message);
                fetchStatus();
            } catch (e) {
                alert("Error al disparar captura: " + e);
            }
        }

        async function openCameraPreview(camIdx) {
            activePreviewCam = camIdx;
            document.getElementById("modalCamTitle").textContent = `Vista Previa - Cámara ${camIdx === 0 ? 'Cenital' : 'Lateral'} (ID ${camIdx})`;
            document.getElementById("previewModal").style.display = "flex";
            document.getElementById("previewLoadingText").style.display = "block";
            document.getElementById("previewImg").style.display = "none";
            document.getElementById("previewTimestamp").textContent = "Tomando frame...";

            try {
                const res = await fetch(`/api/preview_camera/${camIdx}`);
                const data = await res.json();
                if (data.status === "ok") {
                    document.getElementById("previewLoadingText").style.display = "none";
                    const img = document.getElementById("previewImg");
                    img.src = data.image_b64;
                    img.style.display = "block";
                    document.getElementById("previewTimestamp").textContent = `Capturado a las: ${data.timestamp}`;
                } else {
                    document.getElementById("previewLoadingText").textContent = data.message || "Error al capturar.";
                }
            } catch (e) {
                document.getElementById("previewLoadingText").textContent = "Error de conexión con la cámara.";
```

```
            }
        }

        function refreshCurrentPreview() {
            openCameraPreview(activePreviewCam);
        }

        function closePreviewModal() {
            document.getElementById("previewModal").style.display = "none";
        }

        async function controlDaemon(action) {
            try {
                const res = await fetch(`/api/daemon/${action}`, { method: "POST" });
                const data = await res.json();
                showToast(data.message);
                setTimeout(fetchStatus, 1500);
            } catch (e) {
                alert("Error controlando demonio: " + e);
            }
        }

        // Carga inicial y bucle de refresco continuo cada 3 segundos
        fetchStatus();
        setInterval(fetchStatus, 3000);
    </script>
</body>
</html>
```